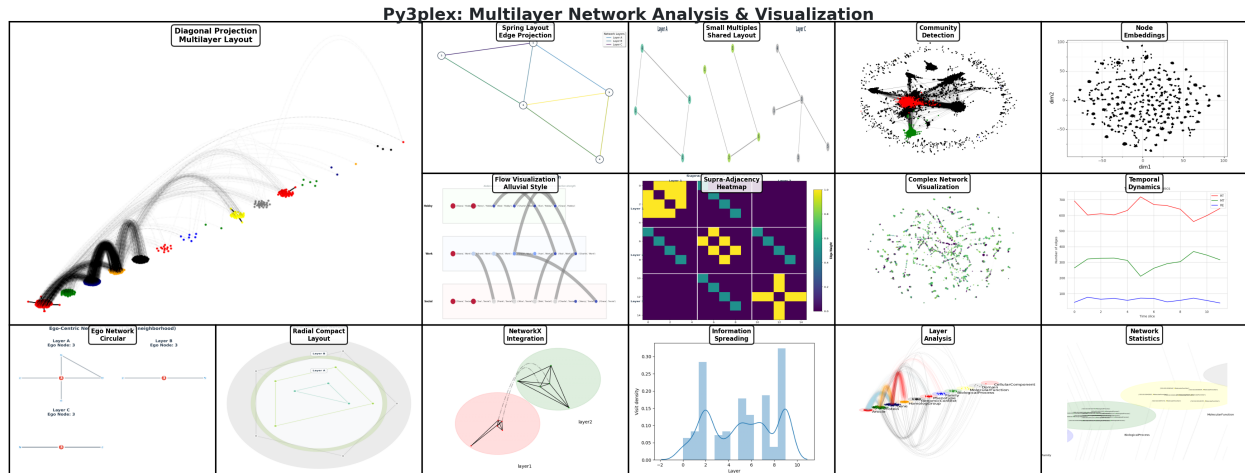

py3plex Documentation

Release 0.95a

Blaž Škrlj

Nov 23, 2025

GETTING STARTED



py3plex enables scalable analysis and visualization of multilayer and multiplex networks in Python, supporting complex network modeling across diverse scientific and applied domains.

OVERVIEW

py3plex is a lightweight Python library designed specifically for analyzing and visualizing heterogeneous and multilayer networks. Unlike traditional network analysis tools that focus on homogeneous networks (single node and edge type), py3plex provides specialized capabilities for networks with:

- Multiple node types
- Multiple edge types
- Multiple layers of interaction
- Temporal dynamics
- Heterogeneous attributes

Target Users: Researchers in network science, computational biology, complex systems, social network analysis, and applied network modeling.

Key Features:

- Native support for multiplex and multilayer network structures
- Diagonal projection visualization for large multilayer networks
- Comprehensive multilayer centrality measures
- Community detection across network layers
- Network decomposition and feature extraction
- Semantic enrichment with external knowledge bases
- Integration with NetworkX, igraph, and other graph libraries

INSTALLATION

2.1 Install from GitHub

```
pip install git+https://github.com/SkBlaz/py3plex.git
```

2.2 Docker Installation (Alternative)

py3plex is also available as a Docker container with all dependencies pre-installed:

```
# Clone and build
git clone https://github.com/SkBlaz/py3plex.git
cd py3plex
docker build -t py3plex:latest .

# Run commands
docker run --rm py3plex:latest --version
docker run --rm py3plex:latest selftest
```

See *Docker Usage Guide* for complete Docker documentation.

2.3 Install from source for development

```
git clone https://github.com/SkBlaz/py3plex.git
cd py3plex
pip install -e .
```

2.4 Optional Dependencies

```
# Advanced community detection with Infomap
pip install git+https://github.com/SkBlaz/py3plex.git#egg=py3plex[infomap]

# Additional algorithms (Louvain, cdlib)
pip install git+https://github.com/SkBlaz/py3plex.git#egg=py3plex[algos]

# Advanced visualization (plotly, igraph)
pip install git+https://github.com/SkBlaz/py3plex.git#egg=py3plex[viz]
```

(continues on next page)

(continued from previous page)

```
# For development (includes testing and linting tools)
pip install -e "[dev]"
```

2.5 Requirements

- Python 3.8 or higher
- NetworkX, NumPy, SciPy (automatically installed)
- Optional: Matplotlib, Plotly for visualization

QUICK START

Create a simple multilayer network:

```
from py3plex.core import multinet

# Create a new multilayer network
network = multinet.multi_layer_network()

# Add edges within layers
network.add_edges([
    ['A', 'layer1', 'B', 'layer1', 1],
    ['B', 'layer1', 'C', 'layer1', 1],
    ['A', 'layer2', 'B', 'layer2', 1],
], input_type="list")

# Display basic statistics
network.basic_stats()

# Visualize
from py3plex.visualization.multilayer import draw_multilayer_default
draw_multilayer_default([network], display=True)
```

Load from file and analyze:

```
# Load from edge list
network = multinet.multi_layer_network().load_network(
    "data.edgelist", input_type="edgelist", directed=False)

# Compute multilayer statistics
from py3plex.algorithms.statistics import multilayer_statistics as mls
density = mls.layer_density(network, 'layer1')
versatility = mls.versatility centrality(network, centrality_type='degree')

# Community detection
from py3plex.algorithms.community_detection import community_wrapper
communities = community_wrapper.best_partition(network.core_network)
```


WHY PY3PLEX?

4.1 vs. NetworkX

NetworkX excels at single-layer homogeneous networks but lacks native multilayer support. py3plex builds on NetworkX while adding:

- Native multilayer data structures
- Layer-aware algorithms
- Specialized multilayer visualizations
- Heterogeneous network decomposition

4.2 vs. Other Multilayer Tools

py3plex provides:

- Lightweight and easy to integrate
- Comprehensive statistical measures (17+ multilayer metrics)
- Publication-ready visualizations
- Active development and research backing

USE CASES

py3plex is particularly well-suited for:

- **Biological networks:** Protein-protein interactions with multiple evidence types, gene regulatory networks
- **Social networks:** Multi-platform analysis (Twitter + Facebook), relationship type analysis
- **Citation networks:** Author-paper-venue multilayer structures, knowledge graphs
- **Transportation:** Multi-modal networks (bus, train, air), urban mobility
- **Temporal networks:** Evolving social networks, dynamic community evolution

DOCUMENTATION CONTENTS

6.1 Installation and Setup

This guide covers all aspects of installing py3plex and setting up your environment.

6.1.1 Basic Installation

Install from GitHub (Recommended)

The latest version is always available on GitHub:

```
pip install git+https://github.com/SkBlaz/py3plex.git
```

This installs the core py3plex library with all required dependencies.

Install from Source

For development or to access the latest features:

```
git clone https://github.com/SkBlaz/py3plex.git
cd py3plex
pip install -e .
```

The `-e` flag installs in editable mode, so changes to the source code are immediately available.

Docker Installation (Alternative)

For a containerized environment with all dependencies pre-installed, use Docker:

```
# Clone the repository
git clone https://github.com/SkBlaz/py3plex.git
cd py3plex

# Build the Docker image
docker build -t py3plex:latest .

# Run py3plex commands
docker run --rm py3plex:latest --version
docker run --rm py3plex:latest selftest
```

Benefits of Docker installation:

- No Python environment setup required

- All dependencies pre-installed and tested
- Consistent environment across different systems
- Isolated from system Python installation

See *Docker Usage Guide* for complete Docker documentation including:

- Building and running the container
- Working with data files via volume mounts
- Docker Compose usage
- Helper scripts (`py3plex-docker.sh`)
- Batch processing and CI/CD integration

6.1.2 Optional Dependencies

py3plex offers several optional feature sets that can be installed separately:

Advanced Community Detection

Install Infomap for advanced overlapping community detection:

```
pip install git+https://github.com/SkBlaz/py3plex.git#egg=py3plex[infomap]
```

Additional Algorithms

Install extra community detection algorithms (Louvain, cdlib):

```
pip install git+https://github.com/SkBlaz/py3plex.git#egg=py3plex[algos]
```

Advanced Visualization

Install Plotly and igraph for interactive and advanced visualizations:

```
pip install git+https://github.com/SkBlaz/py3plex.git#egg=py3plex[viz]
```

Apache Arrow Support

Install pyarrow for high-performance I/O with Arrow/Parquet formats:

```
pip install git+https://github.com/SkBlaz/py3plex.git#egg=py3plex[arrow]
```

Arrow format provides 2-3x faster read/write operations and better compression compared to JSON. See *I/O and Serialization* for details on using Arrow format.

Multiple Extras

Install multiple feature sets at once:

```
pip install git+https://github.com/SkBlaz/py3plex.git#egg=py3plex[infomap,viz,algos,  
↪arrow]
```

Development Installation

For contributors, install with development tools (testing, linting):

```
git clone https://github.com/SkBlaz/py3plex.git
cd py3plex
pip install -e ".[dev]"
```

This installs:

- pytest and coverage tools
- black, ruff, isort for code formatting
- mypy for type checking
- sphinx for documentation building

6.1.3 System Requirements

Python Version

py3plex requires Python 3.8 or higher. We test on:

- Python 3.8
- Python 3.9
- Python 3.10
- Python 3.11
- Python 3.12

Platform Support

py3plex is cross-platform and tested on:

- **Linux** (Ubuntu 20.04, 22.04)
- **macOS** (macOS 11, 12, 13)
- **Windows** (Windows Server 2019, 2022)

Core Dependencies

These are automatically installed with py3plex:

- **networkx**>=2.5 - Graph data structures and algorithms
- **numpy**>=0.8 - Numerical computing
- **scipy**>=1.1.0 - Scientific computing and sparse matrices
- **pandas** - Data manipulation
- **matplotlib** - Static visualization
- **scikit-learn** - Machine learning utilities
- **tqdm** - Progress bars
- **rdflib**>=0.1 - Semantic web support
- **bitarray**>=2.0.0 - Efficient boolean arrays

6.1.4 External Binaries (Optional)

Note: As of October 2025, py3plex no longer bundles external binaries to reduce repository size and improve licensing clarity.

Infomap Community Detection

For advanced community detection using Infomap:

Option 1: Use the bundled Python implementation (recommended):

```
pip install git+https://github.com/SkBlaz/py3plex.git#egg=py3plex[infomap]
```

Option 2: Download the C++ binary from the official source:

- Website: <https://www.mapequation.org/infomap/>
- Follow platform-specific installation instructions

Alternative: Use the built-in Louvain algorithm (no binary needed):

```
from py3plex.algorithms.community_detection import community_louvain
communities = community_louvain.best_partition(network.core_network)
```

Node2Vec Embeddings

For Node2Vec node embeddings, use pure Python alternatives:

Option 1: Use pecanpy (fast parallel implementation):

```
pip install pecanpy
```

Option 2: Use node2vec package:

```
pip install node2vec
```

Option 3: Use py3plex's built-in random walk implementation:

```
from py3plex.algorithms.general.walkers import node2vec_walk, generate_walks
walks = generate_walks(network.core_network, num_walks=10, walk_length=80)
```

6.1.5 Virtual Environment Setup

We strongly recommend using a virtual environment:

Using venv

```
# Create virtual environment
python3 -m venv py3plex-env

# Activate (Linux/macOS)
source py3plex-env/bin/activate

# Activate (Windows)
py3plex-env\Scripts\activate

# Install py3plex
pip install git+https://github.com/SkBlaz/py3plex.git
```

Using conda

```
# Create conda environment
conda create -n py3plex python=3.10

# Activate
conda activate py3plex

# Install py3plex
pip install git+https://github.com/SkBlaz/py3plex.git
```

6.1.6 Common Installation Issues

Issue: “No module named ‘numpy’”

Solution: Install numpy first:

```
pip install numpy
pip install git+https://github.com/SkBlaz/py3plex.git
```

Issue: Compilation errors on Windows

Solution: Install Visual C++ Build Tools:

1. Download from: <https://visualstudio.microsoft.com/visual-cpp-build-tools/>
2. Install “Desktop development with C++”
3. Retry installation

Issue: “Permission denied” on Linux/macOS

Solution: Use pip install with `--user` flag:

```
pip install --user git+https://github.com/SkBlaz/py3plex.git
```

Or use a virtual environment (recommended).

Issue: Old version installed

Solution: Upgrade to latest version:

```
pip install --upgrade git+https://github.com/SkBlaz/py3plex.git
```

6.1.7 Verifying Installation

Test that py3plex is correctly installed:

```
import py3plex
print(py3plex.__version__)

from py3plex.core import multinet
network = multinet.multi_layer_network()
print("py3plex installed successfully!")
```

Run the test suite:

```
# Clone repository (if not already)
git clone https://github.com/SkBlaz/py3plex.git
cd py3plex

# Run tests
python run_tests.py
```

6.1.8 License Considerations

Main Library License

py3plex core is distributed under the MIT License (permissive, commercial-friendly).

Bundled Code Licenses

Important: The repository contains AGPLv3-licensed code in `py3plex/algorithms/community_detection/infomap/`.

If you use Infomap-based community detection functions, your application may be subject to AGPLv3 requirements (copyleft).

License Matrix

Feature Category	License	Commercial Use	Notes
Core multilayer functionality	MIT	[OK] Yes	Safe for proprietary use
Network visualization	MIT	[OK] Yes	Safe for proprietary use
I/O operations	MIT	[OK] Yes	Safe for proprietary use
Louvain community detection	BSD-3-Clause	[OK] Yes	Safe for proprietary use
Label propagation	MIT	[OK] Yes	Safe for proprietary use
Infomap community detection	AGPLv3	WARNING Restricted	Viral license - requires open-sourcing derived works

Recommendations

- For commercial/proprietary projects: Avoid Infomap functions or use the pure Python `infomap` package separately
- For open-source projects: All features are safe to use
- When in doubt: Use alternative algorithms (Louvain, label propagation) which are BSD/MIT licensed

6.1.9 Next Steps

After installation, proceed to:

- *Quickstart Guide* - Quick introduction to basic features
- *10-Minute Tutorial: Getting Started with py3plex* - Comprehensive 10-minute tutorial
- *Quick Start Guide* - Detailed usage guide

6.1.10 Getting Help

If you encounter installation issues:

1. Check [GitHub Issues](#)
2. Search for similar problems
3. Open a new issue with:
 - Your operating system and version
 - Python version (`python --version`)
 - Error message and full traceback
 - Installation command used

For any errors, please open an issue at <https://github.com/SkBlaz/py3plex/issues>

6.2 Quickstart Guide

This guide gets you started with py3plex quickly. For a more comprehensive introduction, see the *10-Minute Tutorial: Getting Started with py3plex*.

6.2.1 Quick Start with Docker

If you prefer using Docker, you can get started immediately without installing Python dependencies:

```
# Clone and build
git clone https://github.com/SkBlaz/py3plex.git
cd py3plex
docker build -t py3plex:latest .

# Create a data directory for files
mkdir -p data

# Create a simple multilayer network
docker run --rm -v $(pwd)/data:/data py3plex:latest \
  create --nodes 50 --layers 3 --output /data/network.edgelist

# Analyze it
docker run --rm -v $(pwd)/data:/data py3plex:latest \
  load /data/network.edgelist --info
```

For complete Docker documentation, see *Docker Usage Guide*.

6.2.2 Creating Your First Multilayer Network

Create a simple multilayer network from scratch:

```
from py3plex.core import multinet

# Create a new multilayer network
network = multinet.multi_layer_network()

# Add edges within layers (nodes are created automatically)
# Format: [source_node, source_layer, target_node, target_layer, weight]
```

(continues on next page)

(continued from previous page)

```
network.add_edges([
    ['A', 'layer1', 'B', 'layer1', 1],
    ['B', 'layer1', 'C', 'layer1', 1],
    ['A', 'layer2', 'B', 'layer2', 1],
    ['B', 'layer2', 'D', 'layer2', 1]
], input_type="list")

# Display basic statistics
network.basic_stats()
```

Expected Output:

```
Number of nodes: 6
Number of edges: 4
Number of unique nodes (as node-layer tuples): 6
Number of unique node IDs (across all layers): 4
Nodes per layer:
  Layer 'layer1': 3 nodes
  Layer 'layer2': 3 nodes
```

6.2.3 Loading Networks from Files

py3plex supports multiple input formats:

From Edge List

Load from a simple edge list:

```
from py3plex.core import multinet

# Load from a simple edge list
network = multinet.multi_layer_network().load_network(
    "data.edgelist",
    input_type="edgelist",
    directed=False
)

network.basic_stats()
```

Expected Output:

```
Number of nodes: 3
Number of edges: 2
Number of unique nodes (as node-layer tuples): 3
Number of unique node IDs (across all layers): 3
Nodes per layer:
  Layer 'default': 3 nodes
```


From Multilayer Edge List

Load from multilayer edge list (source target layer format):

```
# Load from multilayer edge list (source target layer format)
network = multinet.multi_layer_network().load_network(
    "data.multiedgelist",
    input_type="multiedgelist",
    directed=False
)
```

Expected Output:

```
Number of nodes: 4
Number of edges: 4
Number of unique nodes (as node-layer tuples): 7
Number of unique node IDs (across all layers): 4
Nodes per layer:
  Layer 'layer1': 3 nodes
  Layer 'layer2': 3 nodes
```

From GraphML

```
# Load from GraphML format
network = multinet.multi_layer_network().load_network(
    "data.graphml",
    input_type="graphml"
)
```

Expected Output:

```
# Output depends on the GraphML file structure
# Similar to above examples
```

Supported Formats

- edgelist - Simple source-target pairs
- multiedgelist - Source, target, layer format
- gpickle - NetworkX pickle format
- gml - Graph Modeling Language
- graphml - GraphML XML format

6.2.4 Basic Network Analysis

Computing Network Statistics

```
# Get basic network information
num_nodes = len(network.get_nodes())
num_edges = len(network.get_edges())
num_layers = len(network.get_layers())

print(f"Nodes: {num_nodes}, Edges: {num_edges}, Layers: {num_layers}")
```

Expected Output:

```
Nodes: 6, Edges: 4, Layers: 3
```

Multilayer Statistics

```
from py3plex.algorithms.statistics import multilayer_statistics as mls

# Layer density
density = mls.layer_density(network, 'layer1')
print(f"Layer density: {density}")

# Node activity (fraction of layers where node is present)
activity = mls.node_activity(network, 'node_A')
print(f"Node activity: {activity}")

# Versatility centrality (importance across layers)
versatility = mls.versatility_centrality(network, centrality_type='degree')
print(f"Top versatile nodes: {sorted(versatility.items(), key=lambda x: x[1],
↪reverse=True)[:5]}")
```

Expected Output:

```
Layer density: 0.3333333333333333
Node activity: 1.0
Top versatile nodes: [('B', 1.0), ('A', 0.5), ('C', 0.25), ('D', 0.25)]
```

Iterating Over Network Elements

```
# Iterate through nodes
for node in network.get_nodes(data=True):
    print(node)

# Iterate through edges
for edge in network.get_edges(data=True):
    print(edge)

# Get neighbors of a node in a specific layer
neighbors = list(network.get_neighbors('node1', layer_id='layer1'))
print(f"Neighbors: {neighbors}")
```

Expected Output:

```
# Sample nodes (first few):
(('A', 'layer1'), {'pos': array([...]), ...})
(('B', 'layer1'), {'pos': array([...]), ...})
...

# Sample edges:
(('A', 'layer1'), ('B', 'layer1'), {'weight': 1, 'type': 'default'})
(('B', 'layer1'), ('C', 'layer1'), {'weight': 1, 'type': 'default'})
...
```

(continues on next page)

(continued from previous page)

```
Neighbors of node 'A' in 'layer1': [('B', 'layer1')]
```

6.2.5 Community Detection

Using Louvain Algorithm

Detect communities using Louvain modularity optimization:

```
from py3plex.algorithms.community_detection import community_louvain

# Detect communities using Louvain modularity optimization
communities = community_louvain.best_partition(network.core_network)

# Display communities
for node, community_id in communities.items():
    print(f"Node {node} -> Community {community_id}")
```

Expected Output:

```
# Example output (will vary based on network structure):
Node ('A', 'layer1') -> Community 0
Node ('B', 'layer1') -> Community 0
Node ('C', 'layer1') -> Community 1
Node ('A', 'layer2') -> Community 0
Node ('B', 'layer2') -> Community 0
Node ('D', 'layer2') -> Community 1
```

Note: Louvain algorithm requires an undirected graph. For directed multilayer networks, you may need to convert to undirected or use multilayer-specific algorithms.

Using Infomap

Detect communities using Infomap (requires infomap binary):

```
from py3plex.algorithms.community_detection import community_wrapper

# Detect communities using Infomap (requires infomap binary)
communities = community_wrapper.infomap_communities(
    network.core_network,
    binary_path="/path/to/infomap"
)
```

Note: Infomap requires external binary installation. See *Installation and Setup* for details.

Multilayer Modularity

Compute multilayer modularity:

```
from py3plex.algorithms.community_detection import multilayer_modularity as mlm

# Compute multilayer modularity
supra_adj = network.get_supra_adjacency_matrix()
communities = mlm.multilayer_louvain(supra_adj)
```

Expected Output:

```
# Returns dictionary of community assignments
{node_id: community_id, ...}
```

Note: Advanced multilayer community detection that considers layer structure.

6.2.6 Network Visualization

Basic Visualization

```
from py3plex.visualization.multilayer import draw_multilayer_default

# Simple visualization
draw_multilayer_default([network], display=True)
```

Note: This produces a visual plot. No text output to console. See *Visualization Guide* for customization options.

Customized Visualization

```
from py3plex.visualization.multilayer import hairball_plot
from py3plex.visualization import drawing_machinery as dm

# Customize layout and appearance
hairball_plot(
    network.core_network,
    color_by='layer',
    layout_algorithm='force',
    edge_width=0.5,
    node_size=20
)
```

Note: Creates customized network visualization with specified parameters. No text output - produces visual plot.

Diagonal Projection Plot

```
from py3plex.visualization.multilayer import draw_multilayer_default

# Diagonal projection for large multilayer networks
draw_multilayer_default(
    [network],
    display=True,
    layout_style='diagonal'
)
```

Note: Diagonal projection visualization useful for large multilayer networks. No text output - produces visual plot.

6.2.7 Computing Centrality Measures

Degree Centrality

```
from py3plex.algorithms.multilayer_algorithms import centrality

# Multilayer degree centrality
```

(continues on next page)

(continued from previous page)

```
degree_cent = centrality.multilayer_degree_centrality(network)

# Display top nodes
top_nodes = sorted(degree_cent.items(), key=lambda x: x[1], reverse=True)[:10]
print("Top 10 nodes by degree centrality:", top_nodes)
```

Expected Output:

```
Top 10 nodes by degree centrality: [((('B', 'layer1'), 0.4), ((('B', 'layer2'), 0.4), ...]
```

Note: Centrality values depend on network structure. Values are normalized between 0 and 1.

PageRank

```
from py3plex.algorithms.community_detection import node_ranking

# PageRank on multilayer network
pagerank = node_ranking.pagerank(network.core_network)

top_pr = sorted(pagerank.items(), key=lambda x: x[1], reverse=True)[:10]
print("Top 10 nodes by PageRank:", top_pr)
```

Expected Output:

```
Top 10 nodes by PageRank: [((('B', 'layer1'), 0.18), ((('A', 'layer1'), 0.15), ...]
```

Note: PageRank scores sum to 1.0 across all nodes.

Versatility Centrality

```
from py3plex.algorithms.statistics import multilayer_statistics as mls

# Versatility centrality (cross-layer importance)
versatility = mls.versatility_centrality(network, centrality_type='betweenness')
```

Expected Output:

```
# Returns dictionary of versatility scores
{('A',): 0.5, ('B',): 1.0, ('C',): 0.25, ('D',): 0.25}
```

Note: Versatility measures node importance across multiple layers.

6.2.8 Node Embeddings

Node2Vec Embeddings

```
from py3plex.wrappers import node2vec_embedding

# Generate Node2Vec embeddings
embeddings = node2vec_embedding.generate_embeddings(
    network.core_network,
    dimensions=128,
    walk_length=80,
```

(continues on next page)

(continued from previous page)

```
num_walks=10,
p=1.0, # Return parameter
q=1.0 # In-out parameter
)

# embeddings is a matrix of shape (num_nodes, dimensions)
print(f"Embedding shape: {embeddings.shape}")
```

Expected Output:

```
Embedding shape: (6, 128)
```

Note: Node2Vec generates low-dimensional vector representations of nodes. Output is a numpy array of shape (num_nodes, dimensions).

Random Walks

```
from py3plex.algorithms.general.walkers import generate_walks

# Generate random walks
walks = generate_walks(
    network.core_network,
    num_walks=10,
    walk_length=80,
    p=1.0,
    q=1.0,
    seed=42
)

print(f"Generated {len(walks)} walks")
```

Expected Output:

```
Generated 60 walks
```

Note: Random walks are used for network sampling and embedding generation. Each walk is a sequence of nodes traversed following the specified parameters.

6.2.9 Exporting Networks

Save to GraphML

```
# Export to GraphML format
network.save_network("output.graphml", output_type="graphml")
```

Note: Network saved to output.graphml. No console output. GraphML format preserves node and edge attributes.

Save to Pickle

```
# Save as NetworkX pickle
network.save_network("output.gpickle", output_type="gpickle")
```

Note: Network saved to output.gpickle. No console output. Pickle format is fastest for loading/saving in Python.

Save Adjacency Matrix

```
import numpy as np

# Get supra-adjacency matrix
adj_matrix = network.get_supra_adjacency_matrix()

# Save as numpy array
np.save("supra_adjacency.npy", adj_matrix)
```

Expected Output:

```
# Supra-adjacency matrix saved to supra_adjacency.npy
# Matrix shape: (6, 6) for network with 6 node-layer tuples
```

Note: NumPy binary format for efficient matrix storage and loading. The supra-adjacency matrix represents the entire multilayer network structure.

6.2.10 Next Steps

After this quickstart, explore:

- *10-Minute Tutorial: Getting Started with py3plex* - Comprehensive 10-minute tutorial
- *Quick Start Guide* - Detailed usage guide
- *Multilayer Network Centrality Measures* - Multilayer centrality measures
- *Community Detection Tutorial* - Advanced community detection
- *Network Visualization* - Visualization techniques

6.2.11 Examples

See the `examples/` directory for:

- `example_multilayer_visualization.py` - Visualization examples
- `example_community_detection.py` - Community detection
- `example_multilayer_statistics.py` - Statistical analysis
- `example_random_walks.py` - Random walk algorithms
- `example_multilayer_centrality.py` - Centrality measures

6.2.12 Getting Help

- **Documentation:** <https://skblaz.github.io/py3plex/>
- **Examples:** <https://github.com/SkBlaz/py3plex/tree/master/examples>
- **Issues:** <https://github.com/SkBlaz/py3plex/issues>
- **Discussions:** <https://github.com/SkBlaz/py3plex/discussions>

6.3 10-Minute Tutorial: Getting Started with py3plex

This tutorial provides a quick introduction to py3plex, covering the most common tasks for working with multilayer networks.

6.3.1 What You Will Learn

In 10 minutes, you will learn how to:

1. Create and load multilayer networks
2. Perform basic network analysis
3. Compute multilayer statistics
4. Detect communities
5. Perform random walks for embeddings
6. Visualize networks

6.3.2 Prerequisites

Make sure py3plex is installed:

```
pip install git+https://github.com/SkBlaz/py3plex.git
```

6.3.3 1. Creating Your First Multilayer Network (2 minutes)

Start by creating a simple multilayer network from scratch:

```
from py3plex.core import multinet

# Create a new multilayer network
network = multinet.multi_layer_network()

# Add edges within layers (this automatically creates nodes)
# Format: [source_node, source_layer, target_node, target_layer, weight]
network.add_edges([
    ['A', 'layer1', 'B', 'layer1', 1],
    ['B', 'layer1', 'C', 'layer1', 1],
    ['A', 'layer2', 'B', 'layer2', 1],
    ['B', 'layer2', 'D', 'layer2', 1]
], input_type="list")

# Display basic statistics
network.basic_stats()
```

Output:

```
Number of nodes: 5
Number of edges: 4
Number of unique nodes (as node-layer tuples): 5
Number of unique node IDs (across all layers): 4
Nodes per layer:
  Layer 'layer1': 3 nodes
  Layer 'layer2': 3 nodes
```


6.3.4 2. Loading Networks from Files (1 minute)

py3plex supports multiple input formats. Here is how to load from an edge list:

```
from py3plex.core import multinet

# Load from a multiedgelist file
# Format: source target layer
network = multinet.multi_layer_network().load_network(
    "datasets/multiedgelist.txt",
    input_type="multiedgelist",
    directed=False
)

# Check what we loaded
network.basic_stats()
```

Output:

```
Number of nodes: 5
Number of edges: 4
Number of unique nodes (as node-layer tuples): 5
Number of unique node IDs (across all layers): 5
Nodes per layer:
  Layer '1': 3 nodes
  Layer '2': 2 nodes
```

Supported formats:

- multiedgelist - source, target, layer
- edgelist - simple source, target pairs
- gpickle - NetworkX pickle format
- gml, graphml - standard graph formats

6.3.5 3. Exploring Network Structure (2 minutes)

Iterate Through Nodes and Edges

```
# Loop through all nodes
print("Nodes:")
for node in network.get_nodes(data=True):
    print(node)

# Loop through all edges
print("\nEdges:")
for edge in network.get_edges(data=True):
    print(edge)

# Get neighbors of a specific node in a layer
node_of_interest = "1"
layer_id = "1"
neighbors = list(network.get_neighbors(node_of_interest, layer_id=layer_id))
print(f"\nNeighbors of {node_of_interest} in layer {layer_id}:", neighbors)
```

Output (first few items):

```
Nodes (first 5):
(('1', '1'), {'type': '1'})
(('2', '1'), {'type': '1'})
(('6', '2'), {'type': '2'})
(('3', '1'), {'type': '1'})
(('5', '2'), {'type': '2'})

Edges (first 5):
(('1', '1'), ('2', '1'), {'weight': '1'})
(('1', '1'), ('3', '1'), {'weight': '1'})
(('2', '1'), ('6', '2'), {'weight': '1'})
(('3', '1'), ('5', '2'), {'weight': '1'})

Neighbors of 1 in layer 1: [('2', '1'), ('3', '1')]
```

Extract Subnetworks

```
# Extract a single layer
layer_1 = network.subnetwork(['1'], subset_by="layers")
print("Layer 1 nodes:", list(layer_1.get_nodes()))

# Extract specific nodes
node_subset = network.subnetwork(['1', '2'], subset_by="node_names")
print("Node subset:", list(node_subset.get_nodes()))

# Extract specific node-layer pairs
specific_pairs = network.subnetwork(
    [('1', '1'), ('2', '1')],
    subset_by="node_layer_names"
)
print("Specific pairs:", list(specific_pairs.get_nodes()))
```

Output:

```
Layer 1 nodes: 3 nodes
Node subset: 2 node-layer pairs
Specific pairs: [('2', '1'), ('1', '1')]
```

6.3.6 4. Computing Network Metrics (2 minutes)

Basic Centrality Measures

```
# Get a single layer as NetworkX graph
layer_1 = network.subnetwork(['1'], subset_by="layers")

# Compute degree centrality using NetworkX wrapper
degree_centrality = layer_1.monoplex_nx_wrapper("degree_centrality")
print("Degree centrality:", degree_centrality)

# Compute betweenness centrality
```

(continues on next page)

(continued from previous page)

```

betweenness = layer_1.monoplex_nx_wrapper("betweenness centrality")
print("Betweenness centrality:", betweenness)

```

Output (top nodes shown):

Degree centrality (first 5):

```

('1', '1'): 1.0000
('2', '1'): 0.5000
('3', '1'): 0.5000

```

Betweenness centrality (first 5):

```

('1', '1'): 1.0000
('2', '1'): 0.0000
('3', '1'): 0.0000

```

Multilayer Centrality

For multilayer-specific centrality measures:

```

from py3plex.algorithms.multilayer_algorithms.centrality import MultilayerCentrality

# Initialize centrality calculator
calc = MultilayerCentrality(network)

# Compute multilayer degree centrality (considers node participation across layers)
ml_degree = calc.overlapping_degree_centrality(weighted=False)
print("Multilayer degree centrality:", ml_degree)

# Compute multilayer betweenness centrality
ml_betweenness = calc.multilayer_betweenness_centrality()
print("Multilayer betweenness centrality:", ml_betweenness)

```

Output (top nodes shown):

Multilayer degree centrality (first 5):

```

1: 2.0000
2: 1.0000
3: 1.0000
5: 0.0000
6: 0.0000

```

Multilayer betweenness centrality (first 5):

```

('1', '1'): 0.6667
('2', '1'): 0.5000
('3', '1'): 0.5000
('6', '2'): 0.0000
('5', '2'): 0.0000

```

6.3.7 5. Multilayer Network Statistics (2 minutes)

py3plex provides many specialized statistics for analyzing multilayer networks. Here are the most commonly used:

Basic Layer Statistics

```
from py3plex.algorithms.statistics import multilayer_statistics as mls

# Measure how densely connected each layer is
density_layer1 = mls.layer_density(network, 'layer1')
density_layer2 = mls.layer_density(network, 'layer2')
print(f"Layer 1 density: {density_layer1:.3f}")
print(f"Layer 2 density: {density_layer2:.3f}")

# Measure diversity across layers
entropy = mls.entropy_of_multiplexity(network)
print(f"Layer diversity (entropy): {entropy:.3f} bits")
```

Output:

```
Layer 1 density: 0.667
Layer 2 density: 0.000
Layer diversity (entropy): 0.000 bits
```

Node-Level Statistics

```
# Compute node activity (how many layers a node participates in)
activity = mls.node_activity(network, node='1')
print(f"Node activity for node 1: {activity:.3f}")
```

Output:

```
Node activity for node 1: 0.500
```

Network-Level Statistics

```
# Measure how much layers differ in structure (edge overlap)
edge_overlap_value = mls.edge_overlap(network, 'layer1', 'layer2')
print(f"Edge overlap between layers: {edge_overlap_value:.3f}")

# Measure layer similarity using Jaccard index
jaccard = mls.layer_similarity(network, 'layer1', 'layer2', method='jaccard')
print(f"Jaccard similarity between layers: {jaccard:.3f}")
```

Output:

```
Edge overlap between layers: 0.000
Jaccard similarity between layers: 0.000
```

Available statistics:

See the `py3plex.algorithms.statistics.multilayer_statistics` module for all available statistics including:

- `layer_density` - Density of edges within a layer
- `entropy_of_multiplexity` - Diversity across layers

- `node_activity` - Number of layers a node participates in
- `edge_overlap` - Edge overlap between layers
- `layer_similarity` - Similarity between layers (Jaccard, Pearson, etc.)
- `algebraic_connectivity` - Connectivity measure from Laplacian
- `community_participation_coefficient` - How communities span layers
- `interdependence` - Layer interdependence measure
- `multilayer_clustering_coefficient` - Clustering in multilayer context
- `multiplex_betweenness centrality` - Betweenness in multiplex networks
- `multiplex_closeness centrality` - Closeness in multiplex networks
- And more...

6.3.8 6. Community Detection (2 minutes)

py3plex provides Louvain-based community detection that works across multiple layers:

```
from py3plex.algorithms.community_detection.multilayer_modularity import (
    louvain_multilayer
)

# Detect communities using multilayer Louvain
partition = louvain_multilayer(
    network,
    gamma=1.0,      # Resolution parameter
    omega=1.0,      # Inter-layer coupling
    random_state=42 # For reproducibility
)

# Examine results
print("Number of communities:", len(set(partition.values())))
print("\nNode assignments (first 10):")
for node, community in list(partition.items())[:10]:
    print(f"  Node {node}: Community {community}")

# Count community sizes
from collections import Counter
community_sizes = Counter(partition.values())
print("\nCommunity sizes:", dict(community_sizes))
```

Output:

```
Number of communities: 3

Node assignments (first 10):
  Node ('1', '1'): Community 0
  Node ('2', '1'): Community 0
  Node ('6', '2'): Community 1
  Node ('3', '1'): Community 0
  Node ('5', '2'): Community 2

Community sizes (top 5): {0: 3, 1: 1, 2: 1}
```

Note: The `louvain_multilayer` function performs community detection across all layers simultaneously, taking into account both intra-layer and inter-layer connections. Parameters:

- `gamma`: Resolution parameter (higher values = more communities)
- `omega`: Inter-layer coupling strength (higher values = more consistency across layers)
- `random_state`: Seed for reproducibility

6.3.9 7. Random Walks (1 minute)

Random walks are fundamental for graph embeddings (Node2Vec, DeepWalk) and analyzing network structure:

Basic Random Walk

```
from py3plex.algorithms.general.walkers import basic_random_walk

# Load a network
network = multinet.multi_layer_network().load_network(
    "datasets/test.edgelist",
    input_type="edgelist",
    directed=False
)
G = network.core_network

# Perform a random walk
start_node = list(G.nodes())[0]
walk = basic_random_walk(
    G,
    start_node=start_node,
    walk_length=10,
    weighted=True,
    seed=42
)
print(f"Random walk: {walk[:5]}... (length: {len(walk)})")
```

Output:

```
Random walk from ('0', 'null'): [('0', 'null'), ('1', 'null'), ('4872', 'null'), ('786',
↪ 'null'), ('5382', 'null')]... (length: 11)
```

Note: When loading a simple edgelist file (without explicit layer information), py3plex assigns nodes to a default layer named 'null'. Nodes are represented as tuples (node_id, layer_name).

Node2Vec Biased Walks

```
from py3plex.algorithms.general.walkers import node2vec_walk

# Biased walk with Node2Vec parameters
# p: return parameter (higher = less likely to return)
# q: in-out parameter (higher = stay local, lower = explore)

walk_bfs = node2vec_walk(
    G, start_node, walk_length=20,
    p=1.0, q=2.0, # BFS-like (local)
```

(continues on next page)

(continued from previous page)

```

    seed=42
)

walk_dfs = node2vec_walk(
    G, start_node, walk_length=20,
    p=1.0, q=0.5, # DFS-like (explore)
    seed=42
)

```

Output (first 10 nodes of each walk):

```

Node2Vec BFS-like walk: [('0', 'null'), ('1', 'null'), ('5829', 'null'), ('1', 'null'), (
→ '3842', 'null'), ('1', 'null'), ('6364', 'null'), ('1', 'null'), ('3422', 'null'), ('1
→ ', 'null')]
Node2Vec DFS-like walk: [('0', 'null'), ('1', 'null'), ('4872', 'null'), ('786', 'null'),
→ ('5382', 'null'), ('260', 'null'), ('2861', 'null'), ('260', 'null'), ('5128', 'null
→ '), ('260', 'null')]

```

Generating Multiple Walks

```

from py3plex.algorithms.general.walkers import generate_walks

# Generate walks from all nodes for embeddings
walks = generate_walks(
    G,
    num_walks=10,      # Walks per node
    walk_length=10,    # Steps per walk
    p=1.0, q=1.0,     # Node2Vec parameters
    seed=42
)
print(f"Generated {len(walks)} walks")

# Use with Word2Vec for node embeddings
# walks_str = [[str(node) for node in walk] for walk in walks]
# model = Word2Vec(walks_str, vector_size=128, window=10)

```

Output:

```

Generated 63870 walks
Example walk: [('4528', 'null'), ('2611', 'null'), ('4267', 'null'), ('2611', 'null'), (
→ '2894', 'null'), ('2611', 'null'), ('6234', 'null'), ('2611', 'null'), ('3073', 'null
→ '), ('479', 'null'), ('3125', 'null')]

```

Key parameters:

- `walk_length`: Number of steps in the walk
- `p`: Return parameter (controls backtracking)
- `q`: In-out parameter (controls exploration vs. local search)
- `weighted`: Use edge weights in sampling
- `seed`: Random seed for reproducibility

6.3.10 8. Basic Visualization (1 minute)

Visualize your multilayer network:

```
from py3plex.visualization.multilayer import hairball_plot
import matplotlib.pyplot as plt

# Get network for visualization
network_colors, graph = network.get_layers(style="hairball")

# Create a simple hairball plot
plt.figure(figsize=(10, 10))
hairball_plot(
    graph,
    network_colors,
    layout_algorithm="force",
    layout_parameters={"iterations": 50}
)
plt.title("Multilayer Network Visualization")
plt.savefig("my_network.png", dpi=150, bbox_inches='tight')
plt.close()
print("Visualization saved to my_network.png")
```

Output:

```
Visualization saved to my_network.png
```

For more advanced visualizations with community colors:

```
from py3plex.visualization.colors import colors_default

# Get communities
partition = louvain_multilayer(network)

# Select top N communities
top_n = 5
community_counts = Counter(partition.values())
top_communities = [c for c, _ in community_counts.most_common(top_n)]

# Assign colors
color_map = dict(zip(
    top_communities,
    colors_default[:top_n]
))

network_colors = [
    color_map.get(partition.get(node), "gray")
    for node in network.get_nodes()
]

# Plot with community colors
plt.figure(figsize=(10, 10))
hairball_plot(graph, network_colors, layout_algorithm="force")
plt.title("Multilayer Network with Communities")
```

(continues on next page)

(continued from previous page)

```
plt.savefig("my_network_communities.png", dpi=150, bbox_inches='tight')
plt.close()
print("Community visualization saved to my_network_communities.png")
```

Output:

```
Community visualization saved to my_network_communities.png
```

6.3.11 Complete Example: Putting It All Together

Here's a complete workflow:

```
from py3plex.core import multinet
from py3plex.algorithms.community_detection.multilayer_modularity import louvain_
↳ multilayer
from py3plex.visualization.multilayer import hairball_plot
from py3plex.visualization.colors import colors_default
from collections import Counter
import matplotlib.pyplot as plt

# Load network
network = multinet.multi_layer_network().load_network(
    "datasets/multiedgelist2.txt",
    input_type="multiedgelist",
    directed=False
)

# Analyze structure
print("=== Network Statistics ===")
network.basic_stats()

# Compute centrality for one layer
layer_1 = network.subnetwork(['1'], subset_by="layers")
degree_cent = layer_1.monoplex_nx_wrapper("degree_centrality")
print("\n=== Top 5 Nodes by Degree (Layer 1) ===")
for node, score in sorted(degree_cent.items(), key=lambda x: x[1], reverse=True)[:5]:
    print(f"{node}: {score:.3f}")

# Detect communities using multilayer method
partition = louvain_multilayer(network, gamma=1.0, omega=1.0, random_state=42)
print(f"\n=== Communities ===")
print(f"Number of communities: {len(set(partition.values()))}")

# Visualize with communities
network_colors, graph = network.get_layers(style="hairball")
top_n = 3
community_counts = Counter(partition.values())
top_communities = [c for c, _ in community_counts.most_common(top_n)]
color_map = dict(zip(top_communities, colors_default[:top_n]))
network_colors = [
    color_map.get(partition.get(node), "lightgray")
    for node in network.get_nodes()
```

(continues on next page)

(continued from previous page)

```
]

plt.figure(figsize=(12, 12))
hairball_plot(graph, network_colors, layout_algorithm="force")
plt.title("Multilayer Network Analysis")
plt.savefig("complete_analysis.png", dpi=150, bbox_inches='tight')
plt.close()
print("\nComplete analysis saved to complete_analysis.png")
```

Output:

```
=== Network Statistics ===
Number of nodes: 184
Number of edges: 1691
Number of unique nodes (as node-layer tuples): 184
Number of unique node IDs (across all layers): 46
Nodes per layer:
  Layer '1': 46 nodes
  Layer '2': 46 nodes
  Layer '3': 46 nodes
  Layer '4': 46 nodes

=== Top 5 Nodes by Degree (Layer 1) ===
('15', '1'): 0.244
('7', '1'): 0.244
('27', '1'): 0.244
('28', '1'): 0.244
('1', '1'): 0.244

=== Communities ===
Number of communities: 46

Complete analysis saved to complete_analysis.png
```

Note: In this example, the high number of communities (equal to the number of nodes) indicates that the network structure or parameters may need adjustment for meaningful community detection. In practice, adjust `gamma` (resolution) and `omega` (inter-layer coupling) parameters to achieve desired community granularity.

6.3.12 Next Steps

Now that you've completed this tutorial, explore more advanced features:

- **Multilayer Statistics:** Explore all available statistics in `py3plex.algorithms.statistics.multilayer_statistics`
- **Multilayer Centrality:** See examples in `examples/network_analysis/`
- **More Examples:** Check the `examples/` directory for 80+ examples organized by topic
- **Full Documentation:** Visit <https://skblaz.github.io/py3plex/>

6.3.13 Common Issues

File Not Found

Make sure you're running from the repository root or use absolute paths:

```
import os
base_path = "/path/to/py3plex"
network = multinet.multi_layer_network().load_network(
    os.path.join(base_path, "datasets/multiedgelist.txt"),
    input_type="multiedgelist",
    directed=False
)
```

Visualization Not Showing

If using Jupyter, add `%matplotlib inline` at the top of your notebook. For scripts, use `plt.show()` instead of `plt.close()`.

Missing Dependencies

Some features require optional dependencies:

```
# For advanced visualization
pip install plotly

# For community detection with Infomap
pip install infomap

# For all extras
pip install git+https://github.com/SkBlaz/py3plex.git#egg=py3plex[viz,algorithms]
```

6.3.14 Tips for Success

1. **Start Simple:** Begin with small test networks before scaling to large datasets
2. **Check Stats:** Always run `basic_stats()` after loading to verify your network
3. **Use Subnetworks:** Extract layers or subsets for faster prototyping
4. **Seed Your Random:** Use seed parameters in algorithms for reproducible results
5. **Visualize Early:** Quick plots help catch data loading issues early

Happy network analysis!

6.4 Core Principles

py3plex provides simple, off-the-shelf functionality for multilayer and multiplex network analysis.

6.4.1 Key Features

1. Simple API - Modular interface for multilayer networks
2. Advanced Visualization - State-of-the-art multilayer network rendering
3. NetworkX Compatible - Built on top of NetworkX for seamless integration

6.4.2 Learn More

- Paper: <https://link.springer.com/article/10.1007/s41109-019-0203-7>
- Examples: <https://github.com/SkBlaz/Py3Plex/tree/master/examples>
- Repository: <https://github.com/SkBlaz/Py3Plex>

6.5 Quick Start Guide

6.5.1 Installation

Install py3plex:

```
pip install git+https://github.com/SkBlaz/py3plex.git
```

6.5.2 Basic Usage

Import the library:

```
from py3plex.core import multinet
```

6.5.3 Loading Networks

From edge lists:

```
network = multinet.multi_layer_network().load_network(  
    "./datasets/test.edgelist", directed=False, input_type="edgelist")
```

From multilayer edge lists (format: node1 layer1 node2 layer2 weight):

```
network = multinet.multi_layer_network().load_network(  
    "./datasets/multiedgelist.txt", directed=False, input_type="multiedgelist")
```

Multiplex networks (same nodes, different layers):

```
network = multinet.multi_layer_network(network_type="multiplex").load_network(  
    "./datasets/simple_multiplex.edgelist", directed=False, input_type="multiplex_edges")
```

6.5.4 Network Operations

Basic operations:

```
# Get network statistics  
network.basic_stats()  
  
# Visualize  
network.visualize_network()
```

6.5.5 For More Examples

See the `examples/` directory for detailed usage patterns:

- `example_multilayer_visualization.py` - Visualization techniques
- `example_IO.py` - Loading and saving networks

- `example_manipulation.py` - Network manipulation
- `example_multilayer_functionality.py` - Core functionality

Repository: <https://github.com/SkBlaz/Py3Plex/tree/master/examples>

6.6 Network Analysis

6.6.1 Core Operations

The `multi_layer_network` object provides methods for network analysis and manipulation.

Basic Iteration

```
from py3plex.core import multinet

network = multinet.multi_layer_network().load_network(
    "../datasets/multiedgelist.txt", input_type="multiedgelist", directed=False)

# Iterate edges
for edge in network.get_edges(data=True):
    print(edge)

# Iterate nodes
for node in network.get_nodes(data=True):
    print(node)
```

Expected Output (example edges and nodes):

```
('A_layer1', 'B_layer1', {'weight': 1.0, 'source': 'A', 'target': 'B', 'layer': 'layer1'}
↪)
('B_layer1', 'C_layer1', {'weight': 1.0, 'source': 'B', 'target': 'C', 'layer': 'layer1'}
↪)
...
('A_layer1', {'type': 'layer1', 'name': 'A'})
('B_layer1', {'type': 'layer1', 'name': 'B'})
...
```

Subnetworks

```
# Extract by layer
layer_subnet = network.subnetwork(['1'], subset_by="layers")

# Extract by node names
node_subnet = network.subnetwork(['1'], subset_by="node_names")

# Extract by node-layer pairs
specific_subnet = network.subnetwork(
    [('1', '1'), ('2', '1')], subset_by="node_layer_names")
```

NetworkX Integration

py3plex networks are compatible with NetworkX:

```
# Use any NetworkX function
centralities = network.monoplex_nx_wrapper("degree centrality")
print(centralities)
```

Expected Output (node centrality values):

```
{'A_layer1': 0.5, 'B_layer1': 1.0, 'C_layer1': 0.5, ...}
```

6.6.2 For More Examples

See detailed examples:

- `example_multilayer_functionality.py` - Core operations
- `example_networkx_wrapper.py` - NetworkX integration
- `example_spreading.py` - Network traversal
- `example_manipulation.py` - Network manipulation

Repository: <https://github.com/SkBlaz/Py3Plex/tree/master/examples>

6.7 Multiplex Network Analysis

Multiplex networks have the same nodes across different layers, enabling specialized analysis techniques.

6.7.1 Network Aggregation

Combine information across layers into a single network:

```
from py3plex.core import random_generators

# Generate random multiplex network
network = random_generators.random_multiplex_ER(
    num_nodes=500, num_layers=8, probability=0.0005, directed=False)

# Aggregate edges with different metrics
aggregated1 = network.aggregate_edges(metric="count", normalize_by="degree")
aggregated2 = network.aggregate_edges(metric="count", normalize_by="raw")
```

The first network divides the contribution of an individual edge by the average node degree in a given layer, and the second one simply sums them.

```
# Examine edge weights in aggregated networks
for e in aggregated1.edges(data=True):
    print(e)

for e in aggregated2.edges(data=True):
    print(e)
```

6.7.2 Subsetting Multiplex Networks

Subsetting operates in the same manner as for multilayer networks:

```
from py3plex.core import multinet

B = multinet.multi_layer_network(network_type="multiplex")
B.add_edges([[1,1,2,1,1],[1,2,3,2,1],[1,2,3,1,1],[2,1,3,2,1]], input_type="list")

# Subset the network by layers
C = B.subnetwork([2], subset_by="layers")
print(list(C.get_nodes()))

# Subset by node names
C = B.subnetwork([1], subset_by="node_names")
print(list(C.get_nodes()))

# Subset by specific node-layer pairs
C = B.subnetwork([(1,1),(1,2)], subset_by="node_layer_names")
print(list(C.get_nodes()))
```

6.7.3 Examples

See:

- `example_multiplex_aggregate.py` - Network aggregation
- `example_multiplex_dynamics.py` - Temporal dynamics
- `example_multiplex_community_detection.py` - Community detection
- `example_new_multiplex_metrics.py` - New multiplex centrality and robustness metrics

Repository: <https://github.com/SkBlaz/Py3Plex/tree/master/examples>

6.8 I/O and Serialization

py3plex provides a comprehensive I/O system for reading and writing multilayer graphs in various formats. The system is designed to be extensible, efficient, and easy to use.

6.8.1 Supported Formats

The I/O system supports multiple file formats, each with different trade-offs:

- **JSON** - Human-readable, widely compatible, good for small to medium networks
- **JSONL** - Streaming JSON format, efficient for large networks
- **CSV** - Spreadsheet-compatible, easy to edit manually
- **Arrow/Feather** - High-performance columnar format (requires pyarrow)
- **Parquet** - Compressed columnar format, best for storage (requires pyarrow)

6.8.2 Basic Usage

The I/O system provides two main functions: `read()` and `write()`.

Reading Graphs

```
from py3plex.io import read

# Auto-detect format from extension
graph = read('network.json')
graph = read('network.csv')
graph = read('network.arrow')

# Or specify format explicitly
graph = read('myfile.dat', format='json')
```

Writing Graphs

```
from py3plex.io import write

# Auto-detect format from extension
write(graph, 'network.json')
write(graph, 'network.arrow')
write(graph, 'network.parquet')

# Or specify format explicitly
write(graph, 'myfile.dat', format='json')
```

6.8.3 Creating Graphs with the Schema API

The modern I/O system uses a schema-based API for creating graphs:

```
from py3plex.io import MultiLayerGraph, Node, Layer, Edge

# Create graph
graph = MultiLayerGraph(
    directed=True,
    attributes={'name': 'Social Network'}
)

# Add layers
graph.add_layer(Layer(id='facebook', attributes={'type': 'social'}))
graph.add_layer(Layer(id='twitter', attributes={'type': 'social'}))

# Add nodes
graph.add_node(Node(id='alice', attributes={'age': 30}))
graph.add_node(Node(id='bob', attributes={'age': 25}))

# Add edges
graph.add_edge(Edge(
    src='alice',
    dst='bob',
    src_layer='facebook',
```

(continues on next page)

(continued from previous page)

```
dst_layer='facebook',
attributes={'weight': 0.8}
))
```

6.8.4 Apache Arrow Format

Apache Arrow is a high-performance columnar format designed for efficient data interchange. py3plex supports Arrow through two sub-formats:

- **Feather** - Fast, uncompressed format ideal for temporary storage
- **Parquet** - Compressed format ideal for long-term storage

Installing Arrow Support

Arrow support requires the pyarrow package:

```
pip install 'py3plex[arrow]'
# or directly
pip install pyarrow
```

Using Arrow Format

```
from py3plex.io import read, write

# Feather format (fast, uncompressed)
write(graph, 'network.arrow')
graph = read('network.arrow')

# Parquet format (compressed)
write(graph, 'network.parquet', format='parquet')
graph = read('network.parquet', format='parquet')
```

Benefits of Arrow Format

1. **Performance:** Columnar storage enables fast read/write operations
2. **Compression:** Parquet format provides excellent compression ratios
3. **Interoperability:** Arrow is an industry-standard format supported by:
 - pandas, polars (Python data analysis)
 - Apache Spark (big data processing)
 - R, Julia (statistical computing)
 - DuckDB (analytical database)
4. **Type Safety:** Schema preservation with strong typing
5. **Zero-Copy:** Efficient in-memory representation

Performance Comparison

For a typical multilayer network with 1000 nodes and ~5000 edges:

Format	Write Time	Read Time	File Size
Arrow	0.016s	0.008s	0.46 MB
Parquet	0.020s	0.010s	0.35 MB
JSON	0.046s	0.030s	1.09 MB

Arrow format is **2-3x faster** for writes and provides **2-3x better compression** compared to JSON.

When to Use Each Format

Use Arrow/Feather when:

- You need maximum read/write performance
- Working with large networks (>10k nodes)
- Interoperating with data science tools (pandas, polars)
- Building data pipelines

Use Parquet when:

- Long-term storage is important
- Minimizing storage costs
- Sharing data across platforms
- Archiving networks

Use JSON when:

- Human readability is important
- Working with small networks
- Debugging or manual editing
- Maximum compatibility needed

Use CSV when:

- Working with spreadsheet tools (Excel)
- Simple edge lists
- Manual data entry/editing

6.8.5 CSV Format with Sidecars

CSV format supports optional sidecar files for node and layer attributes:

```
from py3plex.io import read, write

# Write with sidecars
write(graph, 'edges.csv', format='csv', write_sidecars=True)
# Creates: edges.csv, nodes.csv, layers.csv

# Read with sidecars
```

(continues on next page)

(continued from previous page)

```
graph = read('edges.csv', format='csv',
            nodes_file='nodes.csv',
            layers_file='layers.csv')
```

6.8.6 Integration with NetworkX

Convert between py3plex I/O format and NetworkX:

```
from py3plex.io import read, to_networkx, from_networkx

# Load graph
graph = read('network.json')

# Convert to NetworkX
G = to_networkx(graph, mode='union') # Merge all layers
# or
G = to_networkx(graph, mode='multiplex') # Preserve layers as (node, layer)

# Convert back from NetworkX
graph = from_networkx(G, mode='multiplex')
```

6.8.7 Example: Complete Workflow

Here's a complete example demonstrating the I/O system:

```
from py3plex.io import (
    MultiLayerGraph, Node, Layer, Edge,
    read, write, to_networkx
)

# Create a multilayer network
graph = MultiLayerGraph(directed=True)

# Add layers
for layer_id in ['social', 'work', 'family']:
    graph.add_layer(Layer(id=layer_id))

# Add nodes
for name in ['alice', 'bob', 'charlie']:
    graph.add_node(Node(id=name))

# Add edges
edges = [
    ('alice', 'bob', 'social', 'social', 0.8),
    ('bob', 'charlie', 'work', 'work', 0.6),
    ('alice', 'charlie', 'family', 'family', 0.9),
]

for src, dst, src_layer, dst_layer, weight in edges:
    graph.add_edge(Edge(
        src=src, dst=dst,
        src_layer=src_layer, dst_layer=dst_layer,
```

(continues on next page)

(continued from previous page)

```

        attributes={'weight': weight}
    ))

# Save in multiple formats
write(graph, 'network.json')
write(graph, 'network.arrow')
write(graph, 'network.parquet')

# Load back
loaded = read('network.arrow')

# Convert to NetworkX for analysis
G = to_networkx(loaded, mode='union')

# Use NetworkX algorithms
import networkx as nx
centrality = nx.degree_centrality(G)
print(f"Most central node: {max(centrality, key=centrality.get)}")

```

6.8.8 Checking Supported Formats

You can query which formats are available at runtime:

```

from py3plex.io import supported_formats

formats = supported_formats()
print(f"Read formats: {formats['read']}")
print(f"Write formats: {formats['write']}")

```

This is useful for checking if optional dependencies (like pyarrow) are installed.

6.8.9 Schema Validation

The I/O system includes automatic validation:

```

from py3plex.io import (
    MultiLayerGraph, Node, Edge,
    ReferentialIntegrityError
)

graph = MultiLayerGraph()
graph.add_node(Node(id='alice'))

try:
    # This will fail - bob doesn't exist
    graph.add_edge(Edge(
        src='alice', dst='bob',
        src_layer='l1', dst_layer='l1'
    ))
except ReferentialIntegrityError as e:
    print(f"Validation error: {e}")

```

Validation ensures:

1. All edge endpoints reference existing nodes
2. All edge layers reference existing layers
3. All attributes are JSON-serializable
4. No duplicate edges (by src, dst, src_layer, dst_layer, key)

6.8.10 Advanced: Custom Formats

The I/O system is extensible. You can register custom format readers/writers:

```
from py3plex.io import register_reader, register_writer

def my_reader(filepath, **kwargs):
    # Custom reading logic
    graph = MultiLayerGraph()
    # ... populate graph ...
    return graph

def my_writer(graph, filepath, **kwargs):
    # Custom writing logic
    with open(filepath, 'w') as f:
        # ... write graph ...
        pass

# Register
register_reader('myformat', my_reader)
register_writer('myformat', my_writer)

# Now you can use it
write(graph, 'network.myformat')
graph = read('network.myformat')
```

6.8.11 Examples

Complete examples are available in `examples/io_and_data/`:

- `example_new_io.py` - Comprehensive I/O demonstration
- `example_save_to_arrow.py` - Apache Arrow format usage
- `example_save_to_gpickle.py` - NetworkX pickle format
- `example_save_to_edgelist.py` - Edge list format
- `example_schema_validation.py` - Schema validation examples

6.8.12 See Also

- *Quickstart Guide* - Getting started guide
- *Quick Start Guide* - Basic network operations
- *NetworkX Interoperability Guide* - NetworkX integration details
- *Performance and Scalability Best Practices* - Performance optimization tips

6.9 Core Concepts and Architecture

This guide explains the fundamental concepts and architectural design of py3plex.

6.9.1 What are Multilayer Networks?

A multilayer network is a complex network structure that goes beyond traditional single-layer graphs by incorporating multiple types of relationships, node types, or interaction contexts.

Key Characteristics

Traditional (Single-Layer) Networks:

- One type of node
- One type of edge
- Homogeneous structure

Multilayer Networks:

- Multiple node types
- Multiple edge types
- Multiple layers of interaction
- Inter-layer and intra-layer connections

Types of Multilayer Networks

Multiplex Networks

Multiple layers with the same set of nodes but different types of edges.

Example: Social network with friendship, colleague, and family layers.

Heterogeneous Networks (HINs)

Different node types with type-specific relationships.

Example: Academic network with authors, papers, and venues.

Temporal Networks

Networks that evolve over time, with time-sliced layers.

Example: Communication network across different time periods.

Interdependent Networks

Multiple networks where nodes in one network depend on nodes in another.

Example: Power grid and communication network interdependency.

6.9.2 Core Data Structure

The `multi_layer_network` Class

The central data structure in py3plex is the `multi_layer_network` class, which provides:

- Layer Management: Add, remove, and query network layers
- Node and Edge Operations: Efficient addition and retrieval
- NetworkX Integration: Full compatibility with NetworkX algorithms
- Matrix Representations: Supra-adjacency and layer-specific matrices

Basic Structure

```
from py3plex.core import multinet

# Create a multilayer network
network = multinet.multi_layer_network()

# The underlying NetworkX graph
nx_graph = network.core_network # MultiDiGraph or MultiGraph

# Layer management
layers = network.get_layers() # List of layer names
layer_map = network.layer_name_map # Layer name to ID mapping
```

Internal Representation

py3plex represents multilayer networks using:

1. **Core NetworkX Graph:** A MultiDiGraph or MultiGraph storing all nodes and edges
2. **Layer Encoding:** Nodes are encoded as node_id---layer_id using a delimiter (default: ---)
3. **Layer Mapping:** Bidirectional mapping between layer names and integer IDs
4. **Attributes:** Node and edge attributes stored in the NetworkX graph

Example:

```
# Node 'A' in 'layer1' is stored as 'A---layer1'
network.add_nodes([('A', 'layer1')])

# Access the encoded node
encoded_nodes = network.core_network.nodes()
# ['A---layer1']
```

6.9.3 Network Construction

Creating Networks from Scratch

```
from py3plex.core import multinet

network = multinet.multi_layer_network()

# Add layers explicitly (optional)
network.add_layer('social')
network.add_layer('professional')

# Add nodes (layer is created if it doesn't exist)
network.add_nodes([
    ('Alice', 'social'),
    ('Bob', 'social'),
    ('Alice', 'professional'),
    ('Bob', 'professional')
])
```

(continues on next page)

(continued from previous page)

```
# Add edges within layers (intra-layer)
network.add_edges([
    [('Alice', 'social'), ('Bob', 'social'), 1.0]
])

# Add edges between layers (inter-layer)
network.add_edges([
    [('Alice', 'social'), ('Alice', 'professional'), 1.0]
])
```

Loading from Files

py3plex supports multiple input formats:

```
# From edge list
network = multinet.multi_layer_network().load_network(
    "data.edgelist",
    input_type="edgelist",
    directed=False
)

# From multilayer edge list (source, target, layer)
network = multinet.multi_layer_network().load_network(
    "data.multiedgelist",
    input_type="multiedgelist"
)

# From GraphML
network = multinet.multi_layer_network().load_network(
    "data.graphml",
    input_type="graphml"
)
```

6.9.4 Network Operations

Querying Network Elements

```
# Get all nodes
nodes = network.get_nodes(data=True)

# Get nodes in a specific layer
layer_nodes = network.get_nodes(layer='layer1')

# Get all edges
edges = network.get_edges(data=True)

# Get neighbors of a node in a layer
neighbors = network.get_neighbors('Alice', layer_id='social')

# Get all layers
layers = network.get_layers()
```


Network Transformations

```
# Aggregate multiple layers into one
aggregated = network.aggregate_layers(['layer1', 'layer2'], 'combined')

# Extract a single layer as a NetworkX graph
layer_graph = network.get_layer_subgraph('layer1')

# Project to a node type (for heterogeneous networks)
projected = network.project_to_node_type('author')
```

Matrix Representations

```
# Get supra-adjacency matrix (all layers stacked)
supra_adj = network.get_supra_adjacency_matrix()

# Get adjacency matrix for a single layer
layer_adj = network.get_adjacency_matrix(layer='layer1')

# Get inter-layer coupling matrix
coupling = network.get_coupling_matrix()
```

6.9.5 Architectural Design

Modular Structure

py3plex follows a modular architecture:

```
py3plex/
├── core/                # Core data structures
│   ├── multinet.py      # multi_layer_network class
│   ├── parsers.py       # I/O for various formats
│   └── converters.py     # Format conversion utilities
├── algorithms/          # Network algorithms
│   ├── community_detection/
│   ├── statistics/
│   ├── multilayer_algorithms/
│   └── general/
├── visualization/       # Plotting and rendering
│   ├── multilayer.py
│   ├── drawing_machinery.py
│   └── layout_algorithms.py
└── wrappers/            # High-level interfaces
```

Design Principles

1. NetworkX Compatibility

All operations are compatible with NetworkX, allowing use of NetworkX algorithms.

2. Lazy Evaluation

Expensive operations (e.g., matrix construction) are computed on demand.

3. Graceful Degradation

Optional features fail gracefully if dependencies are missing.

4. Extensibility

Easy to extend with custom algorithms and visualizations.

Data Flow

```
Input Data (files, NetworkX, edge lists)
↓
Parsers → multi_layer_network object
↓
Network Operations (add, remove, query)
↓
├→ Algorithms → Metrics, Communities, Centrality
├→ Visualization → Plots, Layouts
└→ Export → Files, Matrices, NetworkX
```

6.9.6 Integration with NetworkX

Full NetworkX Compatibility

The `core_network` attribute is a standard NetworkX graph:

```
import networkx as nx
from py3plex.core import multinet

# Create py3plex network
mlnet = multinet.multi_layer_network()
mlnet.add_edges([['A', 'L1', 'B', 'L1', 1]])

# Access underlying NetworkX graph
G = mlnet.core_network

# Use any NetworkX function
betweenness = nx.betweenness_centrality(G)
communities = nx.community.louvain_communities(G)
diameter = nx.diameter(G) if nx.is_connected(G) else float('inf')
```

Converting to/from NetworkX

```
# From NetworkX to py3plex
import networkx as nx
from py3plex.core import multinet

G = nx.karate_club_graph()
mlnet = multinet.multi_layer_network()
mlnet.load_network_from_networkx(G)

# From py3plex to NetworkX
G_export = mlnet.core_network
```

6.9.7 Supra-Adjacency Matrix

Mathematical Definition

The **supra-adjacency matrix** is a block matrix representation of a multilayer network:

$$\mathbf{S} = \begin{bmatrix} A_1 & C_{12} & \cdots & C_{1L} \\ C_{21} & A_2 & \cdots & C_{2L} \\ \vdots & \vdots & \ddots & \vdots \\ C_{L1} & C_{L2} & \cdots & A_L \end{bmatrix}$$

Where:

- A_α is the adjacency matrix of layer α
- $C_{\alpha\beta}$ is the coupling matrix between layers α and β

Construction in py3plex

```
from py3plex.core import multinet

network = multinet.multi_layer_network()
# ... add nodes and edges ...

# Get supra-adjacency matrix (sparse by default)
supra_adj = network.get_supra_adjacency_matrix(sparse=True)

# Shape: (total_nodes, total_nodes)
# where total_nodes = sum of nodes across all layers
print(f"Supra-adjacency shape: {supra_adj.shape}")
```

6.9.8 Layer-Specific Operations

Working with Individual Layers

```
# Get layer as NetworkX subgraph
layer_graph = network.get_layer_subgraph('layer1')

# Compute layer-specific metrics
from py3plex.algorithms.statistics import multilayer_statistics as mls

density = mls.layer_density(network, 'layer1')
clustering = nx.average_clustering(layer_graph)
```

Inter-Layer Analysis

```
# Inter-layer degree correlation
correlation = mls.inter_layer_degree_correlation(network, 'layer1', 'layer2')

# Edge overlap between layers
overlap = mls.edge_overlap(network, 'layer1', 'layer2')

# Layer similarity
similarity = mls.layer_similarity(network, 'layer1', 'layer2')
```

6.9.9 Extensibility

Adding Custom Algorithms

Extend py3plex by adding custom algorithms:

```
from py3plex.core import multinet
import networkx as nx

def my_custom_centrality(ml_network):
    """Compute custom centrality for multilayer network."""
    G = ml_network.core_network

    # Implement your algorithm
    centrality = {}
    for node in G.nodes():
        # Custom computation
        centrality[node] = compute_score(node, G)

    return centrality
```

Custom Visualization

```
from py3plex.visualization import drawing_machinery as dm
import matplotlib.pyplot as plt

def custom_plot(ml_network):
    """Create custom visualization."""
    fig, ax = plt.subplots(figsize=(10, 8))

    # Use drawing machinery primitives
    pos = dm.compute_layout(ml_network.core_network, 'force')
    dm.draw_nodes(ax, ml_network.core_network, pos)
    dm.draw_edges(ax, ml_network.core_network, pos)

    plt.show()
```

6.9.10 Next Steps

- *Quick Start Guide* - Practical usage examples
- *Multilayer Network Centrality Measures* - Centrality measures
- *Multilayer Modularity and Community Detection* - Community detection
- *Algorithm Selection Guide* - Algorithm selection guide
- *Architecture and Design* - Detailed architecture documentation

6.10 SQL-like DSL for Multilayer Networks

Table of Contents

- *Overview*
- *Basic Syntax*
- *Quick Start Example*
- *Query Components*
 - *SELECT Clause*
 - *WHERE Clause*
 - *COMPUTE Clause*
- *Example Queries*
 - *Basic Queries*
 - *Complex Queries*
 - *Analytical Queries*
- *Working with Results*
 - *Formatting Results*
- *Convenience Functions*
- *Use Cases*
 - *Hub Identification*
 - *Layer Comparison*
 - *Node Importance Ranking*
 - *Network Filtering*
- *Error Handling*
- *Complete Working Examples*
 - *Example 1: Basic Network Querying*
 - *Example 2: Multilayer Network Analysis*
 - *Example 3: Hub Identification*
 - *Example 4: Layer Comparison Workflow*
 - *Example Files*
- *API Reference*
 - *Main Functions*
 - *Convenience Functions*
- *Limitations and Future Work*
- *Best Practices*
- *Performance Considerations*
- *Further Reading*
- *See Also*

6.10.1 Overview

Py3plex provides a Domain-Specific Language (DSL) for querying and analyzing multilayer networks using SQL-like syntax. This intuitive interface allows users to filter nodes and edges, compute network measures, and perform complex analyses with simple, readable queries.

The DSL enables you to express complex network queries in a natural, SQL-like language without writing verbose code. For example, instead of manually iterating through nodes and checking conditions, you can write:

```
execute_query(network, 'SELECT nodes WHERE layer="social" AND degree > 5')
```

The DSL is particularly useful for:

- **Interactive network exploration:** Quickly test hypotheses and explore network structure
- **Rapid prototyping:** Build analysis workflows without extensive coding
- **Educational purposes:** Learn network concepts with intuitive queries
- **Production pipelines:** Create maintainable, self-documenting analysis code

6.10.2 Basic Syntax

The DSL follows a SQL-inspired syntax:

```
SELECT target WHERE conditions COMPUTE measures
```

Where:

- **target:** Either nodes or edges
- **conditions:** Filtering criteria (optional)
- **measures:** Network measures to compute (optional)

6.10.3 Quick Start Example

Here's a complete working example to get you started:

```
from py3plex.core import multinet
from py3plex.dsl import execute_query, format_result

# Create a multilayer network
network = multinet.multi_layer_network(directed=False)

# Add nodes to different layers
network.add_nodes([
    {'source': 'Alice', 'type': 'social'},
    {'source': 'Bob', 'type': 'social'},
    {'source': 'Charlie', 'type': 'social'},
    {'source': 'Alice', 'type': 'work'},
    {'source': 'Bob', 'type': 'work'},
])

# Add edges
network.add_edges([
    {'source': 'Alice', 'target': 'Bob', 'source_type': 'social', 'target_type': 'social'
    ↪ },
```

(continues on next page)

(continued from previous page)

```

    {'source': 'Bob', 'target': 'Charlie', 'source_type': 'social', 'target_type':
    ↪ 'social'},
    {'source': 'Alice', 'target': 'Bob', 'source_type': 'work', 'target_type': 'work'},
]

# Query 1: Select all nodes in the social layer
result = execute_query(network, 'SELECT nodes WHERE layer="social"')
print(f"Found {result['count']} nodes in social layer")
print(result['nodes'])

# Query 2: Find high-degree nodes
result = execute_query(network, 'SELECT nodes WHERE degree > 1')
print(format_result(result))

# Query 3: Compute centrality for filtered nodes
result = execute_query(
    network,
    'SELECT nodes WHERE layer="social" COMPUTE betweenness_centrality'
)
for node, centrality in result['computed']['betweenness_centrality'].items():
    print(f"{node}: {centrality:.4f}")

```

6.10.4 Query Components

SELECT Clause

Specifies what to select from the network:

```

SELECT nodes      # Select nodes
SELECT edges      # Select edges (edge queries in development)

```

Note: Current version primarily supports node queries.

WHERE Clause

Filters results based on conditions. Supports:

Layer filtering:

```

WHERE layer="transport"
WHERE layer="social"

```

Degree filtering:

```

WHERE degree > 5
WHERE degree >= 3
WHERE degree <= 10

```

Logical operators:

```

WHERE layer="social" AND degree > 3
WHERE layer="work" OR layer="social"
WHERE NOT layer="transport"

```

Comparison operators:

- `=` : Equal to
- `!=` : Not equal to
- `>` : Greater than
- `<` : Less than
- `>=` : Greater than or equal
- `<=` : Less than or equal

COMPUTE Clause

Calculates network measures for filtered nodes:

```
COMPUTE degree
COMPUTE betweenness centrality
COMPUTE closeness centrality
COMPUTE eigenvector centrality
```

Supported measures:

- `degree` - Node degree
- `degree centrality` - Normalized degree centrality
- `betweenness centrality` - Betweenness centrality
- `closeness centrality` - Closeness centrality
- `eigenvector centrality` - Eigenvector centrality
- `pagerank` - PageRank score
- `clustering` - Clustering coefficient

Multiple measures:

```
COMPUTE degree betweenness centrality closeness centrality
```

6.10.5 Example Queries

Basic Queries

Select all nodes in a layer:

```
result = execute_query(network, 'SELECT nodes WHERE layer="social"')
```

Select high-degree nodes:

```
result = execute_query(network, 'SELECT nodes WHERE degree > 5')
```

Select all nodes (no filter):

```
result = execute_query(network, 'SELECT nodes')
```


Complex Queries

Combine multiple conditions:

```
# Nodes in transport layer with high degree
result = execute_query(
    network,
    'SELECT nodes WHERE layer="transport" AND degree > 5'
)
```

Use OR operator:

```
# Nodes in either social or work layer
result = execute_query(
    network,
    'SELECT nodes WHERE layer="social" OR layer="work"'
)
```

Degree range filtering:

```
# Nodes with moderate degree
result = execute_query(
    network,
    'SELECT nodes WHERE degree >= 2 AND degree <= 5'
)
```

Analytical Queries

Compute centrality for a layer:

```
result = execute_query(
    network,
    'SELECT nodes WHERE layer="transport" COMPUTE betweenness centrality'
)

# Access computed values
for node, centrality in result['computed']['betweenness centrality'].items():
    print(f"{node}: {centrality}")
```

Multiple measures for filtered nodes:

```
result = execute_query(
    network,
    'SELECT nodes WHERE degree > 3 COMPUTE degree centrality closeness centrality'
)
```

6.10.6 Working with Results

The `execute_query` function returns a dictionary containing:

- `query`: Original query string
- `target`: Query target (nodes or edges)
- `nodes or edges`: List of selected items
- `count`: Number of items returned

- `computed`: Dictionary of computed measures (if `COMPUTE` used)

Example:

```
result = execute_query(network, 'SELECT nodes WHERE layer="social"')

# Access results
print(f"Found {result['count']} nodes")
for node in result['nodes']:
    print(node)

# If COMPUTE was used
if 'computed' in result:
    for measure, values in result['computed'].items():
        print(f"{measure}:")
        for node, value in values.items():
            print(f"  {node}: {value}")
```

Formatting Results

Use `format_result` for human-readable output:

```
from py3plex.dsl import format_result

result = execute_query(network, 'SELECT nodes WHERE degree > 3')
print(format_result(result, limit=10))
```

6.10.7 Convenience Functions

The DSL module provides convenience functions for common operations:

Select nodes by layer:

```
from py3plex.dsl import select_nodes_by_layer

nodes = select_nodes_by_layer(network, 'transport')
```

Select high-degree nodes:

```
from py3plex.dsl import select_high_degree_nodes

# All high-degree nodes
nodes = select_high_degree_nodes(network, min_degree=5)

# High-degree nodes in specific layer
nodes = select_high_degree_nodes(network, min_degree=5, layer='social')
```

Compute centrality for a layer:

```
from py3plex.dsl import compute_centrality_for_layer

centrality = compute_centrality_for_layer(
    network,
    layer='transport',
```

(continues on next page)

(continued from previous page)

```

    centrality='betweenness_centrality'
)

```

6.10.8 Use Cases

Hub Identification

Find important nodes in each layer:

```

for layer in ['social', 'work', 'transport']:
    result = execute_query(
        network,
        f'SELECT nodes WHERE layer="{layer}" AND degree > 5'
    )
    print(f"Hubs in {layer}: {result['count']}")

```

Layer Comparison

Compare network properties across layers:

```

layers = ['social', 'work', 'transport']

for layer in layers:
    result = execute_query(
        network,
        f'SELECT nodes WHERE layer="{layer}" COMPUTE degree'
    )
    degrees = result['computed']['degree']
    avg_degree = sum(degrees.values()) / len(degrees)
    print(f"{layer} average degree: {avg_degree:.2f}")

```

Node Importance Ranking

Rank nodes by multiple measures:

```

result = execute_query(
    network,
    'SELECT nodes WHERE layer="social" COMPUTE betweenness_centrality degree_centrality'
)

# Combine measures for ranking
scores = {}
for node in result['nodes']:
    betweenness = result['computed']['betweenness_centrality'].get(node, 0)
    degree_cent = result['computed']['degree_centrality'].get(node, 0)
    scores[node] = betweenness + degree_cent

# Show top nodes
for node, score in sorted(scores.items(), key=lambda x: x[1], reverse=True)[:5]:
    print(f"{node}: {score:.4f}")

```

Network Filtering

Create subnetworks based on queries:

```
# Get high-degree nodes
result = execute_query(network, 'SELECT nodes WHERE degree > 5')
high_degree_nodes = result['nodes']

# Create subnetwork with these nodes
subnetwork = network.subnetwork(
    [node for node in high_degree_nodes],
    subset_by='node_layer_names'
)
```

6.10.9 Error Handling

The DSL raises specific exceptions for different error types:

```
from py3plex.dsl import execute_query, DSLSyntaxError, DSLExecutionError

try:
    result = execute_query(network, 'SELECT nodes WHERE invalid_condition')
except DSLSyntaxError as e:
    print(f"Syntax error: {e}")
except DSLExecutionError as e:
    print(f"Execution error: {e}")
```

Common syntax errors:

- Missing SELECT keyword
- Invalid target (not 'nodes' or 'edges')
- Malformed conditions
- Unknown operators
- Invalid measure names

6.10.10 Complete Working Examples

This section provides complete, runnable examples demonstrating various DSL features.

Example 1: Basic Network Querying

Create a simple social network and query it:

```
from py3plex.core import multinet
from py3plex.dsl import execute_query, format_result

# Create network
network = multinet.multi_layer_network(directed=False)

# Add nodes in social layer
network.add_nodes([
    {'source': 'Alice', 'type': 'social'},
    {'source': 'Bob', 'type': 'social'},
```

(continues on next page)

(continued from previous page)

```

    {'source': 'Charlie', 'type': 'social'},
    {'source': 'David', 'type': 'social'},
])

# Add edges
network.add_edges([
    {'source': 'Alice', 'target': 'Bob', 'source_type': 'social', 'target_type': 'social'
    ↪},
    {'source': 'Bob', 'target': 'Charlie', 'source_type': 'social', 'target_type':
    ↪'social'},
    {'source': 'Charlie', 'target': 'David', 'source_type': 'social', 'target_type':
    ↪'social'},
    {'source': 'Alice', 'target': 'Charlie', 'source_type': 'social', 'target_type':
    ↪'social'},
])

# Query all nodes
result = execute_query(network, 'SELECT nodes WHERE layer="social"')
print(format_result(result))

# Find high-degree nodes
result = execute_query(network, 'SELECT nodes WHERE degree > 1')
print(f"High-degree nodes: {result['count']}")

```

Example 2: Multilayer Network Analysis

Analyze a network with multiple layers:

```

from py3plex.core import multinet
from py3plex.dsl import execute_query

# Create multilayer network
network = multinet.multi_layer_network(directed=False)

# Add nodes to multiple layers
nodes = []
for person in ['Alice', 'Bob', 'Charlie']:
    for layer in ['social', 'work', 'family']:
        nodes.append({'source': person, 'type': layer})
network.add_nodes(nodes)

# Add edges in different layers
edges = [
    # Social connections
    {'source': 'Alice', 'target': 'Bob', 'source_type': 'social', 'target_type': 'social'
    ↪},
    {'source': 'Bob', 'target': 'Charlie', 'source_type': 'social', 'target_type':
    ↪'social'},
    # Work connections
    {'source': 'Alice', 'target': 'Charlie', 'source_type': 'work', 'target_type': 'work'
    ↪},
    # Family connections

```

(continues on next page)

(continued from previous page)

```

    {'source': 'Alice', 'target': 'Charlie', 'source_type': 'family', 'target_type':
    ↪ 'family'},
]
network.add_edges(edges)

# Compare layers
for layer in ['social', 'work', 'family']:
    result = execute_query(network, f'SELECT nodes WHERE layer="{layer}"')
    print(f"{layer} layer: {result['count']} nodes")

    # Compute degree for this layer
    result = execute_query(network, f'SELECT nodes WHERE layer="{layer}" COMPUTE degree')
    degrees = result['computed']['degree']
    avg_degree = sum(degrees.values()) / len(degrees) if degrees else 0
    print(f" Average degree: {avg_degree:.2f}")

```

Example 3: Hub Identification

Find and rank important nodes using multiple centrality measures:

```

from py3plex.core import multinet
from py3plex.dsl import execute_query

# Create network (assuming network is already created)
network = multinet.multi_layer_network(directed=False)
# ... add nodes and edges ...

# Find high-degree nodes in social layer
result = execute_query(
    network,
    'SELECT nodes WHERE layer="social" AND degree >= 2 '
)
print(f"Found {result['count']} hub nodes")

# Compute multiple centrality measures for hubs
result = execute_query(
    network,
    'SELECT nodes WHERE layer="social" AND degree >= 2 '
    'COMPUTE betweenness_centrality closeness_centrality degree_centrality'
)

# Rank nodes by betweenness centrality
if 'computed' in result and 'betweenness_centrality' in result['computed']:
    centralities = result['computed']['betweenness_centrality']
    sorted_nodes = sorted(centralities.items(), key=lambda x: x[1], reverse=True)

    print("\nTop nodes by betweenness centrality:")
    for node, centrality in sorted_nodes[:5]:
        print(f" {node}: {centrality:.4f}")

```

Example 4: Layer Comparison Workflow

Compare network structure across different layers:

```
from py3plex.core import multinet
from py3plex.dsl import execute_query

# Assuming multilayer network is created
network = multinet.multi_layer_network(directed=False)
# ... add nodes and edges ...

layers = ['social', 'work', 'transport']
layer_stats = {}

for layer in layers:
    # Get nodes in this layer
    result = execute_query(network, f'SELECT nodes WHERE layer="{layer}"')
    node_count = result['count']

    # Compute centrality measures
    result = execute_query(
        network,
        f'SELECT nodes WHERE layer="{layer}" COMPUTE betweenness_centrality'
    )

    if 'computed' in result and 'betweenness_centrality' in result['computed']:
        centralities = result['computed']['betweenness_centrality']
        avg_centrality = sum(centralities.values()) / len(centralities) if centralities_
    else 0
        max_centrality = max(centralities.values()) if centralities else 0

    layer_stats[layer] = {
        'nodes': node_count,
        'avg_centrality': avg_centrality,
        'max_centrality': max_centrality
    }

# Print comparison
print("\nLayer Comparison:")
print(f"{'Layer':<12} {'Nodes':<8} {'Avg Centrality':<16} {'Max Centrality':<16}")
print("-" * 55)
for layer, stats in layer_stats.items():
    print(f"{'layer':<12} {stats['nodes']:<8} {stats['avg_centrality']:<16.4f} {stats['max_
    centrality']:<16.4f}")
```

Example Files

Additional complete examples are available in the repository:

- `examples/network_analysis/example_dsl_queries.py` - Basic DSL usage with detailed output
- `examples/network_analysis/example_dsl_advanced.py` - Advanced queries and transportation network analysis

Run these examples:

```
cd examples/network_analysis
python example_dsl_queries.py
python example_dsl_advanced.py
```

6.10.11 API Reference

Main Functions

```
def execute_query(network: Any, query: str) -> Dict[str, Any]:
    """Execute a DSL query on a multilayer network.

    Args:
        network: Multilayer network object
        query: DSL query string

    Returns:
        Dictionary with 'nodes'/'edges', 'count', and optionally 'computed'
    """

def format_result(result: Dict[str, Any], limit: int = 10) -> str:
    """Format query result as human-readable string.

    Args:
        result: Result from execute_query
        limit: Maximum items to display

    Returns:
        Formatted string
    """
```

Convenience Functions

```
def select_nodes_by_layer(network: Any, layer: str) -> List[Any]:
    """Select all nodes in a specific layer."""

def select_high_degree_nodes(network: Any, min_degree: int,
                             layer: Optional[str] = None) -> List[Any]:
    """Select nodes with degree above threshold."""

def compute_centralty_for_layer(network: Any, layer: str,
                                centrality: str = 'betweenness centrality') -> Dict[Any,
↪ float]:
    """Compute centrality for all nodes in a layer."""
```

6.10.12 Limitations and Future Work

Current limitations:

- Edge queries are not yet fully supported
- Complex nested conditions require multiple queries
- Limited to NetworkX-based measures

- No aggregation functions (SUM, AVG, etc.)

Planned enhancements:

- Full edge query support
- Nested subqueries
- Aggregation operators
- Custom measure registration
- Query optimization
- Save/load query results

6.10.13 Best Practices

1. **Start simple:** Begin with basic queries and add complexity incrementally
2. **Validate data:** Ensure network is properly constructed before querying
3. **Check results:** Inspect result counts and samples before processing
4. **Use convenience functions:** For common operations, use provided shortcuts
5. **Handle errors:** Wrap queries in try-except for robust code
6. **Performance:** For large networks, filter by layer first to reduce computation

6.10.14 Performance Considerations

- Computing centrality measures can be expensive on large networks
- Filter by layer first to reduce search space
- Cache computed measures if reusing them
- Consider using convenience functions for better performance
- Pre-compute measures and store in node attributes for repeated use

Example performance optimization:

```
# Less efficient - computes centrality multiple times
for threshold in [3, 5, 7]:
    result = execute_query(
        network,
        f'SELECT nodes WHERE degree > {threshold} COMPUTE betweenness centrality'
    )

# More efficient - compute once, filter in post-processing
result = execute_query(
    network,
    'SELECT nodes COMPUTE betweenness centrality'
)
centralities = result['computed']['betweenness centrality']

for threshold in [3, 5, 7]:
    high_degree = [n for n in result['nodes']
                   if network.core_network.degree(n) > threshold]
```

6.10.15 Further Reading

- *Quick Start Guide* - Network construction basics
- *Core Concepts and Architecture* - Understanding multilayer networks
- *Algorithm Selection Guide* - Network analysis algorithms
- *Analysis Recipes & Workflows* - Common analysis patterns

6.10.16 See Also

- NetworkX documentation for centrality measures
- Examples directory for complete use cases
- API documentation for detailed function signatures

6.11 Analysis Recipes & Workflows

This guide provides **practical recipes** for **common analysis workflows** in py3plex. Each recipe is a **complete, ready-to-use solution** for a real-world task.

Recipe Index

- *Data Import & Setup*
 - *Recipe 1: Load Network from Multiple File Formats*
 - *Recipe 2: Create Synthetic Benchmark Networks*
 - *Recipe 3: Convert Between Network Formats*
- *Network Exploration & Analysis*
 - *Recipe 4: Quick Network Summary*
 - *Recipe 5: Layer-by-Layer Analysis*
 - *Recipe 6: Compute Multilayer Statistics*
- *Centrality & Node Importance*
 - *Recipe 7: Single-Layer Centrality Analysis*
 - *Recipe 8: Multiplex Participation Coefficient*
- *Community Detection*
 - *Recipe 9: Multilayer Community Detection*
 - *Recipe 10: Compare Community Detection Algorithms*
- *Visualization*
 - *Recipe 11: Basic Network Visualization*
 - *Recipe 12: Visualize Communities on Network*
- *Advanced Workflows*
 - *Recipe 13: Complete Analysis Pipeline*
 - *Recipe 14: Network Comparison Study*

- *Recipe 15: Random Walks for Node Embeddings*
- *Performance & Optimization*
 - *Recipe 16: Handle Large Networks Efficiently*
 - *Recipe 17: Benchmark and Profile Your Analysis*
- *Tips & Best Practices*
 - *General Tips*
 - *Performance Tips*
 - *Visualization Tips*
 - *Community Detection Tips*
- *Common Issues & Solutions*
 - *Issue: “Network appears to be empty”*
 - *Issue: “No module named ‘community’”*
 - *Issue: “Visualization doesn’t appear”*
 - *Issue: “Memory error with large network”*
 - *Issue: “Community detection is slow”*
- *Further Reading*

6.11.1 Data Import & Setup

Recipe 1: Load Network from Multiple File Formats

Task: Load **multilayer networks** from various **file formats** (edgelist, GraphML, CSV).

```
from py3plex.core import multinet

# From multiedgelist (node1, layer1, node2, layer2, weight)
network = multinet.multi_layer_network().load_network(
    "data/network.txt",
    input_type="multiedgelist",
    directed=False
)

# From GraphML
network = multinet.multi_layer_network().load_network(
    "data/network.graphml",
    input_type="graphml"
)

# From edges list programmatically
network = multinet.multi_layer_network()
network.add_edges([
    ['A', 'layer1', 'B', 'layer1', 1.0],
    ['B', 'layer1', 'C', 'layer1', 1.0],
    ['A', 'layer2', 'C', 'layer2', 0.5],
```

(continues on next page)

(continued from previous page)

```
], input_type="list")

# Verify the network
network.basic_stats()
```

Expected Output:

```
Number of nodes: 3
Number of edges: 3
Number of unique nodes (as node-layer tuples): 5
Number of unique node IDs (across all layers): 3
Nodes per layer:
  Layer 'layer1': 3 nodes
  Layer 'layer2': 2 nodes
```

When to use: Starting any **analysis project** with external data.

Recipe 2: Create Synthetic Benchmark Networks

Task: Generate **synthetic multilayer networks** for **testing algorithms**.

```
from py3plex.core.random_generators import random_multilayer_ER
import numpy as np

# Set random seed for reproducibility
np.random.seed(42)

# Create a random Erdős-Rényi multilayer network
# Parameters: n_nodes, n_layers, edge_probability
network = random_multilayer_ER(
    n=100,          # 100 nodes
    L=3,           # 3 layers
    p=0.1,         # 10% edge probability
    directed=False
)

network.basic_stats()
```

When to use: Algorithm validation, testing, benchmarking, or when real data is unavailable.

Recipe 3: Convert Between Network Formats

Task: Export networks to **different formats** for use in **other tools**.

```
from py3plex.core import multinet

# Load network
network = multinet.multi_layer_network().load_network(
    "input.txt",
    input_type="multiedgelist"
)

# Save as GraphML (standard format, compatible with Gephi, Cytoscape)
```

(continues on next page)

(continued from previous page)

```

network.save_network("output.graphml", output_type="graphml")

# Save as edge list with node mapping
inverse_map = network.serialize_to_edgelist("output_edges.txt")

# Save as pickle (preserves all Python objects)
import pickle
with open("network.pkl", "wb") as f:
    pickle.dump(network, f)

```

When to use: Sharing data with collaborators, importing to visualization tools, or archiving results.

6.11.2 Network Exploration & Analysis

Recipe 4: Quick Network Summary

Task: Get comprehensive overview of network structure.

```

from py3plex.core import multinet

network = multinet.multi_layer_network().load_network(
    "data/network.txt",
    input_type="multiedgelist"
)

# Basic statistics
print("=== Basic Statistics ===")
network.basic_stats()

# Layer information
layers = network.get_layers()
print(f"\nLayers: {layers}")
print(f"Number of layers: {len(layers)}")

# Node and edge counts
nodes = list(network.get_nodes())
edges = list(network.get_edges())
print(f"Total nodes: {len(nodes)}")
print(f"Total edges: {len(edges)}")

# Get unique nodes (nodes may appear in multiple layers)
unique_nodes = set([n[0] for n in nodes])
print(f"Unique nodes: {len(unique_nodes)}")

```

Expected Output:

```

=== Basic Statistics ===
Number of nodes: 250
Number of edges: 180
Number of unique nodes (as node-layer tuples): 250
Number of unique node IDs (across all layers): 120
Nodes per layer:
  Layer 'collaboration': 85 nodes

```

(continues on next page)

(continued from previous page)

```

Layer 'citation': 90 nodes
Layer 'coauthor': 75 nodes

Layers: ['collaboration', 'citation', 'coauthor']
Number of layers: 3
Total nodes: 250
Total edges: 180
Unique nodes: 120

```

When to use: Initial data exploration, quality checks, reporting.

Recipe 5: Layer-by-Layer Analysis

Task: Analyze each layer independently and compare properties.

```

from py3plex.core import multinet
import networkx as nx

network = multinet.multi_layer_network().load_network(
    "data/network.txt",
    input_type="multiedgelist"
)

# Analyze each layer
layers = network.get_layers()
layer_stats = {}

for layer_name in layers:
    # Extract single layer
    layer = network.subnetwork([layer_name], subset_by="layers")

    # Compute layer statistics
    stats = {}
    stats['nodes'] = len(list(layer.get_nodes()))
    stats['edges'] = len(list(layer.get_edges()))

    # NetworkX metrics on layer
    G = layer.core_network
    if len(G) > 0: # Check if layer has nodes
        stats['density'] = nx.density(G)
        stats['avg_degree'] = sum(dict(G.degree()).values()) / len(G)

    # Check connectivity
    if not layer.directed:
        stats['connected'] = nx.is_connected(G)
        if stats['connected']:
            stats['diameter'] = nx.diameter(G)

    layer_stats[layer_name] = stats

# Display results
print("\n=== Layer-by-Layer Analysis ===")

```

(continues on next page)

(continued from previous page)

```

for layer, stats in layer_stats.items():
    print(f"\nLayer: {layer}")
    for metric, value in stats.items():
        print(f"    {metric}: {value}")

```

When to use: Understanding layer-specific properties, identifying important layers.

Recipe 6: Compute Multilayer Statistics

Task: Calculate multilayer-specific metrics that account for cross-layer structure.

```

from py3plex.core import multinet
from py3plex.algorithms.statistics import multilayer_statistics as mls

network = multinet.multi_layer_network().load_network(
    "data/network.txt",
    input_type="multiedgelist"
)

layers = network.get_layers()

# === Layer-Level Statistics ===
print("=== Layer Statistics ===")
for layer in layers:
    density = mls.layer_density(network, layer)
    print(f"Layer {layer} density: {density:.4f}")

# Layer diversity
entropy = mls.entropy_of_multiplexity(network)
print(f"\nLayer diversity (entropy): {entropy:.4f} bits")

# === Node-Level Statistics ===
print("\n=== Node Activity ===")
sample_nodes = list(network.get_nodes())[:5]
for node_tuple in sample_nodes:
    node = node_tuple[0] # Extract node name
    activity = mls.node_activity(network, node)
    print(f"Node {node} active in {activity*100:.1f}% of layers")

# === Cross-Layer Analysis ===
if len(layers) >= 2:
    print("\n=== Cross-Layer Similarity ===")
    layer1, layer2 = str(layers[0]), str(layers[1])

    # Cosine similarity of adjacency matrices
    similarity = mls.layer_similarity(network, layer1, layer2, method='cosine')
    print(f"Cosine similarity ({layer1} vs {layer2}): {similarity:.4f}")

    # Jaccard similarity of edge sets
    overlap = mls.edge_overlap(network, layer1, layer2)
    print(f"Edge overlap (Jaccard): {overlap:.4f}")

```

(continues on next page)

(continued from previous page)

```
# === Versatility Analysis ===
print("\n=== Node Versatility (Cross-Layer Importance) ===")
versatility = mls.versatility Centrality(network, centrality_type='degree')
top_nodes = sorted(versatility.items(), key=lambda x: x[1], reverse=True)[:5]
for node, score in top_nodes:
    print(f" {node}: {score:.4f}")
```

When to use: Quantifying multilayer structure, comparing layers, identifying versatile nodes.

6.11.3 Centrality & Node Importance

Recipe 7: Single-Layer Centrality Analysis

Task: Identify important nodes within individual layers.

```
from py3plex.core import multinet

network = multinet.multi_layer_network().load_network(
    "data/network.txt",
    input_type="multiedgelist"
)

# Select a layer
layers = network.get_layers()
target_layer = [str(layers[0])]
layer = network.subnetwork(target_layer, subset_by="layers")

# Compute multiple centrality measures
centrality_measures = {
    'degree': layer.monoplex_nx_wrapper("degree_centrality"),
    'betweenness': layer.monoplex_nx_wrapper("betweenness_centrality"),
    'closeness': layer.monoplex_nx_wrapper("closeness_centrality"),
    'eigenvector': layer.monoplex_nx_wrapper("eigenvector_centrality"),
}

# Display top nodes for each measure
print(f"=== Centrality Analysis for Layer {target_layer[0]} ===\n")
for measure_name, scores in centrality_measures.items():
    print(f"{measure_name.capitalize()} Centrality (Top 5):")
    top_nodes = sorted(scores.items(), key=lambda x: x[1], reverse=True)[:5]
    for node, score in top_nodes:
        print(f" {node}: {score:.4f}")
    print()
```

When to use: Identifying influential nodes, hub detection, network simplification.

Recipe 8: Multiplex Participation Coefficient

Task: Measure how evenly nodes participate across layers in a multiplex network.

```
from py3plex.core import multinet
from py3plex.algorithms.multicentrality import multiplex_participation_coefficient
```

(continues on next page)

(continued from previous page)

```

# Load or create multiplex network (same nodes across all layers)
network = multinet.multi_layer_network().load_network(
    "data/multiplex_network.txt",
    input_type="multiedgelist"
)

# Compute MPC (requires true multiplex: identical node set per layer)
try:
    mpc = multiplex_participation_coefficient(
        network,
        normalized=True,
        check_multiplex=True
    )

    # Display results
    print("=== Multiplex Participation Coefficient ===")
    print("(0 = active in one layer only, 1 = equally active across all layers)\n")

    # Sort by MPC score
    sorted_nodes = sorted(mpc.items(), key=lambda x: x[1], reverse=True)

    print("Top 10 most versatile nodes:")
    for node, score in sorted_nodes[:10]:
        print(f" {node}: {score:.4f}")

    print("\nTop 10 most specialized nodes:")
    for node, score in sorted_nodes[-10:]:
        print(f" {node}: {score:.4f}")

except ValueError as e:
    print(f"Error: {e}")
    print("Network is not a true multiplex (layers have different node sets)")

```

When to use: Analyzing multiplex networks, identifying cross-layer hubs, understanding node specialization.

6.11.4 Community Detection

Recipe 9: Multilayer Community Detection

Task: Detect communities that span multiple layers.

```

from py3plex.core import multinet
from py3plex.algorithms.community_detection.multilayer_modularity import louvain_
    multilayer
from collections import Counter

network = multinet.multi_layer_network().load_network(
    "data/network.txt",
    input_type="multiedgelist"
)

# Run multilayer Louvain algorithm
# gamma: resolution parameter (higher = more communities)

```

(continues on next page)

(continued from previous page)

```

# omega: inter-layer coupling strength (higher = communities span layers)
partition = louvain_multilayer(
    network,
    gamma=1.0,      # Standard resolution
    omega=1.0,      # Standard coupling
    random_state=42 # For reproducibility
)

# Analyze results
num_communities = len(set(partition.values()))
print(f"Detected {num_communities} communities")

# Community size distribution
community_sizes = Counter(partition.values())
print("\nTop 10 communities by size:")
for comm_id, size in community_sizes.most_common(10):
    print(f"  Community {comm_id}: {size} nodes")

# Check which nodes are in largest community
largest_comm = community_sizes.most_common(1)[0][0]
largest_comm_nodes = [node for node, comm in partition.items()
                      if comm == largest_comm]
print(f"\nNodes in largest community: {largest_comm_nodes[:10]}...")

```

When to use: Understanding network organization, identifying functional modules, clustering nodes.

Recipe 10: Compare Community Detection Algorithms

Task: Run multiple community detection methods and compare results.

```

from py3plex.core import multinet
from py3plex.algorithms.community_detection import community_wrapper as cw
from py3plex.algorithms.community_detection.multilayer_modularity import louvain_
    ↪ multilayer
from collections import Counter
import numpy as np

network = multinet.multi_layer_network().load_network(
    "data/network.txt",
    input_type="multiedgelist"
)

# Method 1: Single-layer Louvain on aggregated network
partition_louvain = cw.louvain_communities(network)

# Method 2: Multilayer Louvain
partition_multilayer = louvain_multilayer(
    network,
    gamma=1.0,
    omega=1.0,
    random_state=42
)

```

(continues on next page)

(continued from previous page)

```

# Compare results
print("=== Community Detection Comparison ===\n")
print(f"Single-layer Louvain: {len(set(partition_louvain.values()))} communities")
print(f"Multilayer Louvain: {len(set(partition_multilayer.values()))} communities")

# Calculate normalized mutual information (if sklearn available)
try:
    from sklearn.metrics import normalized_mutual_info_score

    # Align node sets
    common_nodes = set(partition_louvain.keys()) & set(partition_multilayer.keys())
    labels1 = [partition_louvain[n] for n in common_nodes]
    labels2 = [partition_multilayer[n] for n in common_nodes]

    nmi = normalized_mutual_info_score(labels1, labels2)
    print(f"\nNormalized Mutual Information: {nmi:.4f}")
    print("(1.0 = identical partitions, 0.0 = independent)")
except ImportError:
    print("\nInstall scikit-learn to compute NMI: pip install scikit-learn")

```

When to use: Method validation, algorithm selection, robustness analysis.

6.11.5 Visualization

Recipe 11: Basic Network Visualization

Task: Create publication-ready network visualizations.

```

from py3plex.core import multinet
from py3plex.visualization.multilayer import hairball_plot
import matplotlib
matplotlib.use('Agg') # For non-interactive environments
import matplotlib.pyplot as plt

network = multinet.multi_layer_network().load_network(
    "data/network.txt",
    input_type="multiedgelist"
)

# Get network in visualization format
network_colors, graph = network.get_layers(style="hairball")

# Create visualization
plt.figure(figsize=(12, 12))
hairball_plot(
    graph,
    network_colors,
    layout_algorithm="force",          # Options: "force", "circular", "spring"
    layout_parameters={"iterations": 100},
    node_size=5,
    edge_width=0.5,
    alpha_channel=0.8,

```

(continues on next page)

(continued from previous page)

```

        scale_by_size=True
    )
    plt.title("Multilayer Network", fontsize=16, fontweight='bold')
    plt.axis('off')
    plt.tight_layout()
    plt.savefig("network_viz.png", dpi=150, bbox_inches='tight',
                facecolor='white', edgecolor='none')
    plt.close()
    print("Visualization saved to network_viz.png")

```

When to use: Presentations, papers, reports, exploratory analysis.

Recipe 12: Visualize Communities on Network

Task: Color nodes by community membership in network plot.

```

from py3plex.core import multinet
from py3plex.visualization.multilayer import hairball_plot
from py3plex.visualization.colors import colors_default
from py3plex.algorithms.community_detection.multilayer_modularity import louvain_
    ↪ multilayer
from collections import Counter
import matplotlib.pyplot as plt
import matplotlib
matplotlib.use('Agg')

network = multinet.multi_layer_network().load_network(
    "data/network.txt",
    input_type="multiedgelist"
)

# Detect communities
partition = louvain_multilayer(network, gamma=1.0, omega=1.0, random_state=42)

# Select top N communities to color
top_n = 10
community_counts = Counter(partition.values())
top_communities = [c for c, _ in community_counts.most_common(top_n)]

# Create color mapping
color_map = dict(zip(top_communities, colors_default[:top_n]))

# Assign colors to nodes
network_colors = [
    color_map.get(partition.get(node), "lightgray")
    for node in network.get_nodes()
]

# Get network for visualization
_, graph = network.get_layers(style="hairball")

# Create plot

```

(continues on next page)

(continued from previous page)

```
plt.figure(figsize=(14, 14))
hairball_plot(
    graph,
    network_colors,
    layout_algorithm="force",
    layout_parameters={"iterations": 150},
    node_size=8,
    edge_width=0.5,
    alpha_channel=0.7,
    scale_by_size=True
)
plt.title(f"Network with {top_n} Largest Communities",
         fontsize=16, fontweight='bold')
plt.axis('off')
plt.tight_layout()
plt.savefig("network_communities.png", dpi=150,
           bbox_inches='tight', facecolor='white')
plt.close()
print("Community visualization saved to network_communities.png")
```

When to use: Visualizing analysis results, understanding community structure.

6.11.6 Advanced Workflows

Recipe 13: Complete Analysis Pipeline

Task: Full workflow from data loading to results export.

```
from py3plex.core import multinet
from py3plex.algorithms.statistics import multilayer_statistics as mls
from py3plex.algorithms.community_detection.multilayer_modularity import louvain_
    ↳ multilayer
from collections import Counter
import json
import numpy as np

# Set random seed
np.random.seed(42)

# === 1. Load Data ===
print("=== Loading Network ===")
network = multinet.multi_layer_network().load_network(
    "data/network.txt",
    input_type="multiedgelist",
    directed=False
)
network.basic_stats()

# === 2. Compute Statistics ===
print("\n=== Computing Multilayer Statistics ===")
layers = network.get_layers()

stats = {
```

(continues on next page)

(continued from previous page)

```

    'num_layers': len(layers),
    'layer_densities': {},
    'layer_diversity': mls.entropy_of_multiplexity(network),
}

for layer in layers:
    stats['layer_densities'][str(layer)] = mls.layer_density(network, layer)

# Versatility
versatility = mls.versatility_centrality(network, centrality_type='degree')
top_versatile = sorted(versatility.items(), key=lambda x: x[1], reverse=True)[:10]
stats['top_versatile_nodes'] = {str(k): float(v) for k, v in top_versatile}

# === 3. Community Detection ===
print("\n=== Detecting Communities ===")
partition = louvain_multilayer(network, gamma=1.0, omega=1.0, random_state=42)
community_sizes = Counter(partition.values())

stats['num_communities'] = len(set(partition.values()))
stats['community_sizes'] = dict(community_sizes.most_common(10))

# === 4. Export Results ===
print("\n=== Exporting Results ===")

# Save statistics as JSON
with open("analysis_results.json", "w") as f:
    json.dump(stats, f, indent=2)
print("Statistics saved to analysis_results.json")

# Save community assignments
with open("communities.txt", "w") as f:
    for node, comm in partition.items():
        f.write(f"{node}\t{comm}\n")
print("Communities saved to communities.txt")

# Save processed network
network.save_network("processed_network.graphml", output_type="graphml")
print("Network saved to processed_network.graphml")

print("\n=== Analysis Complete ===")

```

When to use: Reproducible research workflows, batch processing, automated analysis.

Recipe 14: Network Comparison Study

Task: Compare multiple networks systematically.

```

from py3plex.core import multinet
from py3plex.algorithms.statistics import multilayer_statistics as mls
import pandas as pd
import numpy as np

```

(continues on next page)

(continued from previous page)

```

# List of networks to compare
network_files = [
    ("Network A", "data/network_a.txt"),
    ("Network B", "data/network_b.txt"),
    ("Network C", "data/network_c.txt"),
]

results = []

for name, filepath in network_files:
    print(f"\nAnalyzing {name}...")

    # Load network
    network = multinet.multi_layer_network().load_network(
        filepath,
        input_type="multiedgelist",
        directed=False
    )

    # Compute metrics
    layers = network.get_layers()
    nodes = list(network.get_nodes())
    edges = list(network.get_edges())

    metrics = {
        'Network': name,
        'Layers': len(layers),
        'Total Nodes': len(nodes),
        'Unique Nodes': len(set([n[0] for n in nodes])),
        'Total Edges': len(edges),
        'Layer Diversity': mls.entropy_of_multiplexity(network),
    }

    # Average layer density
    densities = [mls.layer_density(network, layer) for layer in layers]
    metrics['Avg Layer Density'] = np.mean(densities)
    metrics['Std Layer Density'] = np.std(densities)

    results.append(metrics)

# Create comparison table
df = pd.DataFrame(results)
print("\n=== Network Comparison ===")
print(df.to_string(index=False))

# Save to CSV
df.to_csv("network_comparison.csv", index=False)
print("\nComparison saved to network_comparison.csv")

```

When to use: Comparative studies, evaluating datasets, selecting networks for analysis.

Recipe 15: Random Walks for Node Embeddings

Task: Generate node embeddings using random walks (Node2Vec approach).

```
from py3plex.core import multinet
from py3plex.algorithms.general.walkers import generate_walks, node2vec_walk
import numpy as np

# Set random seed
np.random.seed(42)

network = multinet.multi_layer_network().load_network(
    "data/network.txt",
    input_type="multiedgelist"
)

# Get core NetworkX graph
G = network.core_network

# Generate random walks
# p: return parameter (higher = less likely to return to previous node)
# q: in-out parameter (higher = more BFS-like, lower = more DFS-like)
walks = generate_walks(
    G,
    num_walks=10,      # Number of walks per node
    walk_length=80,    # Length of each walk
    p=1.0,             # Return parameter
    q=1.0,             # In-out parameter
    seed=42
)

print(f"Generated {len(walks)} random walks")
print(f"First walk (truncated): {walks[0][:10]}...")

# Use walks for downstream tasks:
# 1. Train Word2Vec model (if gensim available)
try:
    from gensim.models import Word2Vec

    # Convert walks to strings for Word2Vec
    walks_str = [[str(node) for node in walk] for walk in walks]

    # Train embedding model
    model = Word2Vec(
        walks_str,
        vector_size=128,  # Embedding dimension
        window=5,         # Context window
        min_count=1,
        sg=1,             # Skip-gram
        workers=4,
        epochs=5,
        seed=42
    )
```

(continues on next page)

(continued from previous page)

```

print(f"\nTrained embedding model with {len(model.wv)} nodes")

# Get embedding for a node
sample_node = str(list(G.nodes())[0])
if sample_node in model.wv:
    embedding = model.wv[sample_node]
    print(f"Embedding for node {sample_node}: {embedding[:5]}... (dim=
↪{len(embedding)})")

# Find similar nodes
similar = model.wv.most_similar(sample_node, topn=5)
print(f"\nMost similar nodes to {sample_node}:")
for node, similarity in similar:
    print(f" {node}: {similarity:.4f}")

# Save embeddings
model.wv.save_word2vec_format("node_embeddings.txt")
print("\nEmbeddings saved to node_embeddings.txt")

except ImportError:
    print("\nInstall gensim for embedding generation: pip install gensim")

```

When to use: Node classification, link prediction, network clustering, feature extraction.

6.11.7 Performance & Optimization

Recipe 16: Handle Large Networks Efficiently

Task: Process large multilayer networks without running out of memory.

```

from py3plex.core import multinet
import gc

# === 1. Load network efficiently ===
# Use generators instead of loading all at once
network = multinet.multi_layer_network()

# For very large files, load in chunks or stream
# (This is a conceptual example - actual implementation may vary)

# === 2. Process layers independently ===
layers = network.get_layers()

layer_results = {}
for layer_name in layers:
    print(f"Processing layer {layer_name}...")

    # Extract and process one layer at a time
    layer = network.subnetwork([layer_name], subset_by="layers")

    # Compute statistics for this layer
    nodes = len(list(layer.get_nodes()))
    edges = len(list(layer.get_edges()))

```

(continues on next page)

(continued from previous page)

```

layer_results[str(layer_name)] = {'nodes': nodes, 'edges': edges}

# Clean up to free memory
del layer
gc.collect()

print("\n=== Results ===")
for layer, stats in layer_results.items():
    print(f"{layer}: {stats['nodes']} nodes, {stats['edges']} edges")

# === 3. Use sparse representations ===
# When computing supra-adjacency matrix
supra_adj = network.get_supra_adjacency_matrix(sparse=True)
print(f"\nSupra-adjacency matrix: {supra_adj.shape} (sparse)")

# === 4. Sample for exploratory analysis ===
# For visualization or initial exploration, work with a sample
all_nodes = list(network.get_nodes())
import random
random.seed(42)
sample_size = min(1000, len(all_nodes))
sampled_nodes = random.sample(all_nodes, sample_size)

# Create subnetwork with sampled nodes
# (Implementation depends on network structure)
print(f"\nSampled {sample_size} nodes for exploratory analysis")

```

When to use: Networks with >10,000 nodes, limited RAM, large-scale studies.

Tips: - Process layers sequentially rather than all at once - Use sparse matrices for large supra-adjacency representations - Sample networks for visualization and initial exploration - Save intermediate results to disk - Use generators and iterators instead of lists

Recipe 17: Benchmark and Profile Your Analysis

Task: Measure performance and identify bottlenecks.

```

from py3plex.core import multinet
from py3plex.algorithms.community_detection.multilayer_modularity import louvain_
    ↳ multilayer
import time
import psutil
import os

def get_memory_usage():
    """Get current memory usage in MB"""
    process = psutil.Process(os.getpid())
    return process.memory_info().rss / 1024 / 1024

# Load network
print("=== Performance Benchmarking ===\n")
start_mem = get_memory_usage()

```

(continues on next page)

(continued from previous page)

```

start = time.time()
network = multinet.multi_layer_network().load_network(
    "data/network.txt",
    input_type="multiedgelist"
)
load_time = time.time() - start
load_mem = get_memory_usage() - start_mem

print(f"Network loading:")
print(f"  Time: {load_time:.3f} seconds")
print(f"  Memory: {load_mem:.2f} MB")

# Benchmark statistics computation
start = time.time()
network.basic_stats()
stats_time = time.time() - start
print(f"\nBasic statistics:")
print(f"  Time: {stats_time:.3f} seconds")

# Benchmark community detection
start_mem = get_memory_usage()
start = time.time()
partition = louvain_multilayer(network, gamma=1.0, omega=1.0, random_state=42)
comm_time = time.time() - start
comm_mem = get_memory_usage() - start_mem

print(f"\nCommunity detection:")
print(f"  Time: {comm_time:.3f} seconds")
print(f"  Memory: {comm_mem:.2f} MB")
print(f"  Communities found: {len(set(partition.values()))}")

# Total resource usage
print(f"\n=== Total Performance ===")
print(f"Total time: {load_time + stats_time + comm_time:.3f} seconds")
print(f"Peak memory: {get_memory_usage():.2f} MB")

```

When to use: Optimizing workflows, comparing algorithms, capacity planning.

Note: Requires psutil package: `pip install psutil`

6.11.8 Tips & Best Practices

General Tips

1. Always set random seeds for reproducibility:

```

import numpy as np
np.random.seed(42)

```

2. Verify network structure after loading:

```

network.basic_stats() # Quick sanity check

```

3. Handle errors gracefully:

```
try:
    result = network.some_operation()
except Exception as e:
    print(f"Operation failed: {e}")
```

4. Save intermediate results for long analyses:

```
import pickle
with open("checkpoint.pkl", "wb") as f:
    pickle.dump(results, f)
```

5. Use appropriate file formats:

- GraphML: Standard, compatible with other tools
- Pickle: Fast, preserves Python objects, not portable
- Edgelist: Simple, human-readable, large file size

Performance Tips

1. **For large networks**, process layers independently
2. **Use sparse matrices** when working with supra-adjacency
3. **Sample networks** for visualization (>1000 nodes gets crowded)
4. **Profile your code** to find bottlenecks
5. **Close plots** after saving to free memory: `plt.close()`

Visualization Tips

1. **Choose layout carefully**:
 - Force-directed: General purpose, slow for >5000 nodes
 - Circular: Fast, good for small networks
 - Spring: Balance between quality and speed
2. **Adjust node size** based on network size:

```
node_size = max(1, 100 / np.sqrt(num_nodes))
```

3. **Use alpha channel** for edge-dense networks:

```
alpha_channel=0.3 # More transparent
```

4. **Save high-resolution** for publications:

```
plt.savefig("figure.png", dpi=300)
```

Community Detection Tips

1. **Tune resolution parameter** (γ):
 - Higher γ → more, smaller communities
 - Lower γ → fewer, larger communities

2. **Tune coupling parameter** (ω) for multilayer:
 - Higher ω $\rightarrow$ communities span layers
 - Lower ω $\rightarrow$ layer-specific communities
3. **Run multiple times** with different random seeds to check stability
4. **Compare multiple algorithms** to validate findings

6.11.9 Common Issues & Solutions

Issue: “Network appears to be empty”

Cause: Incorrect file format or delimiter.

Solution::

```
# Check file format
with open("data.txt", "r") as f:
    print(f.readlines()[:5]) # Inspect first lines

# Try different input_type
network = multinet.multi_layer_network().load_network(
    "data.txt",
    input_type="multiedgelist" # or "edgelist", "graphml"
)
```

Issue: “No module named ‘community’”

Cause: Optional dependency not installed.

Solution::

```
pip install python-louvain
```

Issue: “Visualization doesn’t appear”

Cause: Using non-interactive backend or missing display.

Solution::

```
import matplotlib
matplotlib.use('Agg') # For saving files
import matplotlib.pyplot as plt

# ... create plot ...
plt.savefig("output.png") # Save instead of show
```

Issue: “Memory error with large network”

Cause: Loading entire network into RAM at once.

Solution: See Recipe 16 for memory-efficient processing.

Issue: “Community detection is slow”

Cause: Large network or complex structure.

Solution::

```
# Use simpler algorithm for quick results
from py3plex.algorithms.community_detection import community_wrapper as cw
partition = cw.louvain_communities(network) # Faster than multilayer
```

6.11.10 Further Reading

Documentation:

- Installation guide - See main documentation
- Quick start tutorial - See main documentation
- Core concepts - See main documentation
- Algorithm selection guide - See main documentation

Examples:

See `examples/` directory for 50+ complete, working examples covering all major functionality.

Research:

- Kivelä et al. (2014) - Multilayer networks theory
- De Domenico et al. (2013) - Mathematical framework
- See Citations section in main documentation for complete reference list

6.12 Network Visualization

py3plex provides specialized visualization for multilayer networks, with support for both static and interactive visualizations.

6.12.1 Basic Visualization

Hairball plot (standard network layout):

```
from py3plex.core import multinet

network = multinet.multi_layer_network().load_network(
    "../datasets/goslim_mirna.gpickle",
    directed=False,
    input_type="gpickle_biomine")

network.visualize_network(style="hairball")
```

Diagonal multilayer layout (py3plex-specific):

```
network.visualize_network(style="diagonal")
```

6.12.2 Interactive Visualization

For interactive exploration in web browsers or Jupyter notebooks:

```
import networkx as nx
from py3plex.visualization.multilayer import interactive_hairball_plot

# Get network as NetworkX graph
network_colors, G = network.get_layers(style="hairball")

# Compute layout
pos = nx.spring_layout(G, seed=42)

# Compute node properties
degrees = dict(G.degree())
node_sizes = [10 + 40 * (degrees[node] / max(degrees.values())) for node in G.nodes()]
color_mapping = {node: node_sizes[i] for i, node in enumerate(G.nodes())}

# Create interactive visualization
fig = interactive_hairball_plot(G, node_sizes, color_mapping, pos, colorscale="Viridis")

# Save to HTML
if fig:
    fig.write_html("network.html")
```

Requirements: Install plotly for interactive features:

```
pip install plotly
# or
pip install git+https://github.com/SkBlaz/py3plex.git#egg=py3plex[viz]
```

6.12.3 Examples

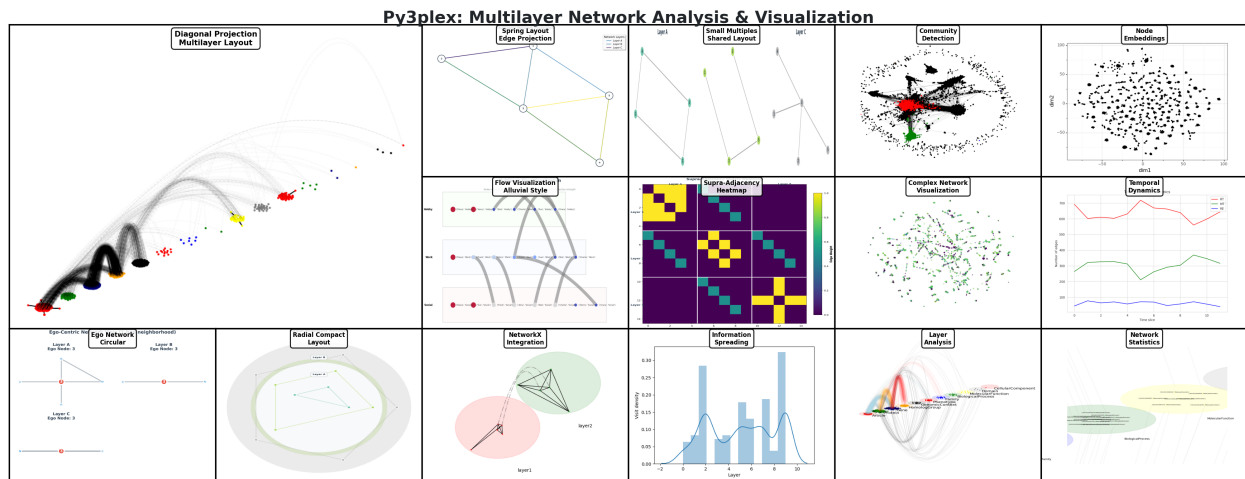
For detailed visualization examples, see:

- `example_multilayer_visualization.py` - Core visualization techniques
- `example_visualization.py` - Various plotting styles
- `example_community_visualization.py` - Community detection visualization
- `example_interactive_hairball.py` - Interactive network visualization
- `example_interactive_multilayer.py` - Advanced interactive multilayer plots
- `example_supra_adjacency.py` - Supra-adjacency matrices

For comprehensive documentation on visualization options, see *Visualization Guide*.

Repository: <https://github.com/SkBlaz/Py3Plex/tree/master/examples>

6.13 Visualization Guide



This guide covers multilayer network visualization in Py3plex, including preset modes, customization options, and best practices for different network scales.

Table of Contents

- *Quick Start*
 - *Basic Multilayer Visualization*
- *Preset Visualization Modes*
 - *Minimal Mode (Large Networks >1000 nodes)*
 - *Balanced Mode (Medium Networks 100-1000 nodes)*
 - *Dense Mode (Small Networks <100 nodes)*
- *Layout Options*
 - *Circular Layout (Default)*
 - *Rectangular Layout*
- *Auto-Scaling Features*
 - *Automatic Node Sizing*
 - *Automatic Color Assignment*
 - *Automatic Layout Adjustment*
- *Customization Options*
 - *Complete Parameter Reference*
 - *Layer-Specific Customization*
 - *Color-Coding by Community*
- *Exporting Visualizations*
 - *Save to File*

- *High-Quality Publications*
- *Interactive Visualizations*
 - *Using Plotly (Optional)*
 - *Basic Interactive Hairball Plot*
 - *Advanced Interactive Multilayer Example*
 - *Jupyter Notebook Integration*
 - *Saving Interactive Visualizations*
 - *Embedding in Web Applications*
 - *Troubleshooting Interactive Visualizations*
 - *Performance Guidelines*
 - *Jupyter Notebook Integration*
- *Performance Tips*
 - *For Large Networks*
- *Troubleshooting*
 - *Visualization Not Showing*
 - *Nodes Overlapping*
 - *Labels Too Crowded*
- *Advanced Visualization Modes*
 - *Overview of Visualization Modes*
 - *Unified API*
 - *Small Multiples View*
 - *Edge-Colored Projection*
 - *Supra-Adjacency Heatmap*
 - *Radial/Concentric Layers*
 - *Ego-Centric Multilayer View*
 - *Flow and Sankey Diagrams*
 - *Example Workflows*
 - *Choosing the Right Visualization*
- *Next Steps*

6.13.1 Quick Start

Basic Multilayer Visualization

```
from py3plex.core import multinet
from py3plex.visualization.multilayer import draw_multilayer_default
import matplotlib.pyplot as plt
```

(continues on next page)

(continued from previous page)

```
# Load network
network = multinet.multi_layer_network()
network.load_network("network.csv", input_type="multiedgelist")

# Visualize with defaults
draw_multilayer_default(
    network.get_layers(),
    display=True,
    labels=True
)
```

6.13.2 Preset Visualization Modes

Py3plex provides three preset modes optimized for different network scales and use cases.

Minimal Mode (Large Networks >1000 nodes)

Optimized for networks with many nodes where detail isn't critical.

```
from py3plex.visualization.multilayer import draw_multilayer_default

# Minimal preset for large networks
draw_multilayer_default(
    network.get_layers(),
    # Node settings
    node_size=5,           # Small nodes
    labels=False,         # No node labels (too cluttered)
    node_labels=False,    # No node IDs

    # Edge settings
    edge_size=0.5,        # Thin edges
    alphalevel=0.3,       # Transparent edges (reduce clutter)

    # Layout
    background_shape="circle", # Circular layout

    # Display
    display=True,
    remove_isolated_nodes=True # Clean up isolated nodes
)
```

Use cases: - Large social networks (>1000 nodes) - Overview visualizations - Pattern detection at macro level - Network topology understanding

Advantages: - Fast rendering - Reduced visual clutter - Shows overall structure - Works with many nodes

Balanced Mode (Medium Networks 100-1000 nodes)

Default settings that work well for most networks. Good balance between detail and clarity.

```
# Balanced preset (this is the default)
draw_multilayer_default(
```

(continues on next page)

(continued from previous page)

```

network.get_layers(),
# Node settings
node_size=10,           # Medium nodes
labels=True,           # Show layer labels
node_labels=False,     # Node IDs hidden by default

# Edge settings
edge_size=1.0,         # Normal edge width
alphalevel=0.13,      # Semi-transparent edges

# Layout
background_shape="circle", # Circular layout
networks_color="rainbow",  # Auto-assign colors

# Display
display=True
)

```

Use cases: - Most research networks - Exploratory data analysis - Publication figures (moderate detail) - Interactive exploration

Advantages: - Good readability - Reasonable performance - Balanced aesthetics - Suitable for most publications

Dense Mode (Small Networks <100 nodes)

Maximum detail for small networks where every node and edge matters.

```

# Dense preset for small, detailed networks
draw_multilayer_default(
    network.get_layers(),
    # Node settings
    node_size=20,           # Large nodes
    labels=True,           # Show all labels
    node_labels=True,      # Show node IDs
    node_font_size=10,     # Readable font
    scale_by_size=True,    # Scale by degree

    # Edge settings
    edge_size=2.0,         # Thick edges
    alphalevel=0.8,        # Mostly opaque
    arrowsize=0.5,         # Visible arrows (if directed)

    # Layout
    background_shape="rectangle", # Rectangular layout
    networks_color="rainbow",

    # Display
    display=True
)

```

Use cases: - Small case studies - Detailed analysis - Node-level examination - High-quality publication figures

Advantages: - Maximum detail - Clear node identification - Edge weights visible - Professional appearance

6.13.3 Layout Options

Py3plex supports multiple layout algorithms for different network structures.

Circular Layout (Default)

Arranges layers in a circle. Good for showing inter-layer connections.

```
draw_multilayer_default(  
    network.get_layers(),  
    background_shape="circle",  
    rectanglex=1.0, # Circle radius x  
    rectangley=1.0  # Circle radius y  
)
```

Best for: - 2-10 layers - Symmetric layer interactions - Publication figures

Rectangular Layout

Arranges layers in a grid. Good for many layers or hierarchical structures.

```
draw_multilayer_default(  
    network.get_layers(),  
    background_shape="rectangle",  
    rectanglex=2.0, # Width of layout  
    rectangley=1.0  # Height of layout  
)
```

Best for: - Many layers (>10) - Hierarchical networks - Time-series data - Wide figures

6.13.4 Auto-Scaling Features

Py3plex automatically adjusts visualization parameters based on network size.

Automatic Node Sizing

Scale node sizes by degree (number of connections):

```
draw_multilayer_default(  
    network.get_layers(),  
    scale_by_size=True, # Enable auto-scaling  
    node_size=10        # Base size (scaled up/down by degree)  
)
```

How it works:

- Nodes with more connections → larger size
- Isolated nodes → smaller size
- Hub nodes are easily identifiable

Automatic Color Assignment

Py3plex uses colorblind-safe palettes by default:

```
# Rainbow colors (automatic assignment)
draw_multilayer_default(
    network.get_layers(),
    networks_color="rainbow" # Auto-assign distinct colors
)

# Custom palette
draw_multilayer_default(
    network.get_layers(),
    networks_color=["#FF6B6B", "#4ECDC4", "#45B7D1", "#FFA07A"]
)
```

Available palettes (from `py3plex.config`):

- `colorblind_safe`: 8 colors safe for colorblind viewers
- `wong`: 7-color palette (scientifically validated)
- `tol_bright`: 7 bright colors
- `rainbow`: Automatic generation

Automatic Layout Adjustment

Layout parameters auto-adjust to network size:

```
# For small networks, layout is compact
small_network = multinet.multi_layer_network()
# ... load 50-node network ...
draw_multilayer_default(small_network.get_layers()) # Compact layout

# For large networks, layout expands
large_network = multinet.multi_layer_network()
# ... load 1000-node network ...
draw_multilayer_default(large_network.get_layers()) # Expanded layout
```

6.13.5 Customization Options

Complete Parameter Reference

```
draw_multilayer_default(
    network_list, # List of layer subgraphs

    # Display control
    display=True, # Show plot immediately
    axis=None, # Matplotlib axis (None = create new)

    # Node appearance
    node_size=10, # Base node size
    scale_by_size=False, # Scale by degree?
    node_labels=False, # Show node IDs?
    node_font_size=5, # Font size for node labels

    # Edge appearance
    edge_size=1.0, # Edge thickness
```

(continues on next page)

(continued from previous page)

```

alphalevel=0.13,          # Edge transparency (0=invisible, 1=opaque)
arrowsize=0.5,           # Arrow size (for directed graphs)

# Layout
background_shape="circle", # "circle" or "rectangle"
rectanglex=1.0,           # Layout width/radius
rectangley=1.0,           # Layout height/radius

# Colors
networks_color="rainbow", # "rainbow" or list of colors
background_color="rainbow", # Background color scheme

# Labels
labels=True,              # Show layer labels?
label_position=1.0,       # Label distance from center

# Cleanup
remove_isolated_nodes=False, # Remove disconnected nodes?

# Debugging
verbose=False             # Print debug info?
)

```

Layer-Specific Customization

Customize individual layers:

```

import matplotlib.pyplot as plt
from py3plex.visualization.multilayer import draw_multilayer_default

# Get layers
layers = network.get_layers()

# Modify specific layer (e.g., highlight layer 0)
for node in layers[0].nodes():
    layers[0].nodes[node]['color'] = 'red'
    layers[0].nodes[node]['size'] = 20

# Visualize with custom layer
draw_multilayer_default(layers, display=True)

```

Color-Coding by Community

Color nodes by community membership:

```

from py3plex.algorithms.community_detection import community_louvain
import matplotlib.pyplot as plt
import matplotlib.cm as cm

# Detect communities
communities = community_louvain.best_partition(network.core_network)

```

(continues on next page)

(continued from previous page)

```

# Assign colors
num_communities = len(set(communities.values()))
colors = cm.rainbow([i/num_communities for i in range(num_communities)])

# Apply to network
for node, comm_id in communities.items():
    network.core_network.nodes[node]['color'] = colors[comm_id]

# Visualize
draw_multilayer_default(network.get_layers(), display=True)

```

6.13.6 Exporting Visualizations

Save to File

```

import matplotlib.pyplot as plt

# Create figure
fig, ax = plt.subplots(1, 1, figsize=(12, 8))

# Draw network
draw_multilayer_default(
    network.get_layers(),
    display=False,
    axis=ax
)

# Customize and save
plt.title("Multilayer Network Visualization")
plt.savefig('network.png', dpi=300, bbox_inches='tight')
plt.savefig('network.pdf', bbox_inches='tight') # Vector format
print("Visualization saved to network.png and network.pdf")

```

High-Quality Publications

```

import matplotlib.pyplot as plt

# Use publication settings
plt.rcParams['font.family'] = 'serif'
plt.rcParams['font.size'] = 10
plt.rcParams['figure.dpi'] = 300

# Large figure for detail
fig, ax = plt.subplots(1, 1, figsize=(10, 8))

# Dense mode for publications
draw_multilayer_default(
    network.get_layers(),
    display=False,
    axis=ax,
    node_size=15,

```

(continues on next page)

(continued from previous page)

```

    labels=True,
    alphalevel=0.5,
    background_shape="circle"
)

plt.title("Figure 1: Multilayer Network Structure", fontsize=12)
plt.savefig('publication_figure.pdf', bbox_inches='tight')
plt.savefig('publication_figure.png', dpi=300, bbox_inches='tight')

```

6.13.7 Interactive Visualizations

Using Plotly (Optional)

Py3plex provides interactive visualization capabilities through Plotly, allowing you to explore networks dynamically in your web browser.

Installation:

```

# Install plotly separately
pip install plotly

# Or install py3plex with visualization extras
pip install git+https://github.com/SkBlaz/py3plex.git#egg=py3plex[viz]

```

Basic Interactive Hairball Plot

The `interactive_hairball_plot` function creates an interactive 2D network visualization:

```

import networkx as nx
from py3plex.core import random_generators
from py3plex.visualization.multilayer import interactive_hairball_plot

# Generate a network
multilayer_net = random_generators.random_multilayer_ER(50, 3, 0.15, directed=False)

# Convert to NetworkX graph
network_colors, G = multilayer_net.get_layers(style="hairball")

# Compute layout
pos = nx.spring_layout(G, iterations=50, seed=42)

# Compute node sizes based on degree
degrees = dict(G.degree())
max_degree = max(degrees.values())
node_sizes = [10 + 40 * (degrees[node] / max_degree) for node in G.nodes()]

# Create color mapping
color_mapping = {node: node_sizes[i] for i, node in enumerate(G.nodes())}

# Generate interactive visualization
fig = interactive_hairball_plot(
    G,
    nsizes=node_sizes,

```

(continues on next page)

(continued from previous page)

```

    final_color_mapping=color_mapping,
    pos=pos,
    colorscale="Viridis" # Options: Viridis, Rainbow, Blues, Reds, etc.
)

# Save to HTML file
if fig:
    fig.write_html("network.html")
    print("Interactive visualization saved to network.html")

```

Interactive Features:

- **Hover:** Move your mouse over nodes to see details
- **Zoom:** Use mouse wheel to zoom in/out
- **Pan:** Click and drag to move the view
- **Select:** Click nodes to highlight connections

Color Scales Available:

- **Viridis:** Perceptually uniform, colorblind-friendly
- **Rainbow:** Full color spectrum
- **Blues, Reds, Greens:** Single-hue gradients
- **RdBu, YlOrRd:** Diverging color schemes
- **Jet, Hot, Electric:** High-contrast schemes

Advanced Interactive Multilayer Example

For more complex multilayer networks with custom styling:

```

from py3plex.core import multinet
from py3plex.visualization.multilayer import interactive_hairball_plot
import networkx as nx

# Create multilayer network
network = multinet.multi_layer_network()

# Add nodes to different layers
layers = ['social', 'professional', 'family']
nodes_per_layer = {
    'social': ['Alice', 'Bob', 'Charlie', 'David', 'Eve'],
    'professional': ['Alice', 'Bob', 'Frank', 'Grace', 'Henry'],
    'family': ['Alice', 'Charlie', 'David', 'Ivan', 'Jane']
}

for layer in layers:
    for node in nodes_per_layer[layer]:
        network.add_nodes([{'source': node, 'type': layer}])

# Add edges (example for one layer)
network.add_edges([

```

(continues on next page)

(continued from previous page)

```

    {'source': 'Alice', 'target': 'Bob', 'source_type': 'social', 'target_type': 'social'
    ↪ },
    {'source': 'Bob', 'target': 'Charlie', 'source_type': 'social', 'target_type':
    ↪ 'social'},
])

# Convert to aggregate graph
G = nx.Graph()
for node in set(n for layer_nodes in nodes_per_layer.values() for n in layer_nodes):
    G.add_node(node)

# Compute layout and visualize
pos = nx.spring_layout(G, seed=42)
degrees = dict(G.degree())
node_sizes = [20 + 80 * (degrees[node] / max(degrees.values())) for node in G.nodes()]

# Assign colors by layer membership
layer_colors = {'social': 0, 'professional': 50, 'family': 100}
color_mapping = {}
for node in G.nodes():
    # Find primary layer for this node
    for layer, nodes in nodes_per_layer.items():
        if node in nodes:
            color_mapping[node] = layer_colors[layer]
            break

# Create interactive visualization
fig = interactive_hairball_plot(G, node_sizes, color_mapping, pos, colorscale="Viridis")

# Customize the figure
if fig:
    fig.update_layout(
        title="Interactive Multilayer Network",
        hovermode='closest',
        plot_bgcolor='rgba(240, 240, 240, 0.9)'
    )
    fig.write_html("multilayer_network.html")

```

Complete Working Examples:

See the following example scripts in the repository:

- `examples/visualization/example_interactive_hairball.py` - Basic interactive visualization
- `examples/visualization/example_interactive_multilayer.py` - Advanced multilayer interactive plot

These examples are standalone and can be run directly:

```

cd examples/visualization
python example_interactive_hairball.py
# Opens browser with interactive visualization

```

Jupyter Notebook Integration

Interactive visualizations work seamlessly in Jupyter notebooks:

```
# In Jupyter notebook
from py3plex.visualization.multilayer import interactive_hairball_plot

# ... prepare your graph, positions, etc. ...

fig = interactive_hairball_plot(G, node_sizes, color_mapping, pos)

# The figure will be displayed inline in the notebook
# No need to call fig.show() - it's called automatically
```

Notebook Tips:

1. The interactive plot appears directly in the notebook
2. All interactive features work in the notebook interface
3. Use `fig.write_html()` to save for sharing
4. Interactive plots work in JupyterLab, Jupyter Notebook, and VS Code notebooks

Saving Interactive Visualizations

Save your interactive visualizations for sharing or publishing:

```
# Create the figure
fig = interactive_hairball_plot(G, node_sizes, color_mapping, pos)

# Save as standalone HTML
fig.write_html("network.html")

# Save as HTML with custom configuration
fig.write_html(
    "network.html",
    config={
        'displayModeBar': True, # Show toolbar
        'displaylogo': False, # Hide Plotly logo
        'modeBarButtonsToRemove': ['pan2d', 'lasso2d'] # Hide specific tools
    }
)

# Save as static image (requires kaleido)
# pip install kaleido
fig.write_image("network.png", width=1200, height=800)
fig.write_image("network.pdf") # Vector format
```

HTML File Features:

- Standalone - works without internet connection
- No dependencies needed in browser
- Full interactivity preserved
- Can be embedded in web pages
- Works on mobile devices

Embedding in Web Applications

Embed interactive visualizations in your web applications:

```
<!DOCTYPE html>
<html>
<head>
  <title>Network Visualization</title>
</head>
<body>
  <h1>My Network Analysis</h1>

  <!-- Embed the interactive plot -->
  <iframe src="network.html" width="100%" height="600px" frameborder="0"></iframe>

  <p>Description of your network...</p>
</body>
</html>
```

Or include the plot div directly:

```
# Generate only the div (not full HTML page)
fig_html = fig.to_html(
    include_plotlyjs='cdn', # Use CDN for Plotly.js
    div_id='my-network-plot'
)

# Use fig_html in your web framework (Flask, Django, etc.)
```

Troubleshooting Interactive Visualizations

Issue: Plotly not available

```
# Solution: Install plotly
pip install plotly
```

Issue: Figure doesn't show in Jupyter

```
# Solution: Call fig.show() explicitly
fig = interactive_hairball_plot(G, node_sizes, color_mapping, pos)
fig.show()
```

Issue: Slow performance with large networks

```
# Solution 1: Sample nodes for interactive view
import random
sample_nodes = random.sample(list(G.nodes()), min(500, G.number_of_nodes()))
G_sample = G.subgraph(sample_nodes)

# Solution 2: Use simpler layout
pos = nx.random_layout(G) # Faster than spring_layout

# Solution 3: Reduce node sizes and simplify colors
node_sizes = [5] * G.number_of_nodes() # Constant size
color_mapping = {node: 0 for node in G.nodes()} # Single color
```

Issue: HTML file is too large

```
# Solution: Reduce data in the plot
# 1. Use fewer nodes (sample or filter)
# 2. Use constant node sizes instead of varying
# 3. Simplify color scheme
# 4. Remove edge traces if not needed
```

Performance Guidelines

Network Size Recommendations:

- **< 100 nodes:** All features work smoothly
- **100-500 nodes:** Good performance, all features
- **500-1000 nodes:** Usable, may have slight lag
- **1000-5000 nodes:** Sample nodes or use simplified styling
- **> 5000 nodes:** Use static visualizations instead

Optimization Tips:

1. Use constant node sizes for large networks
2. Simplify color schemes (single color or discrete categories)
3. Use `nx.random_layout()` instead of `nx.spring_layout()` for speed
4. Sample nodes for interactive exploration
5. Consider static visualization for very large networks

Jupyter Notebook Integration

```
# Enable inline plotting
%matplotlib inline

# Or for interactive plots
%matplotlib notebook

# Visualize
from py3plex.visualization.multilayer import draw_multilayer_default
draw_multilayer_default(network.get_layers(), display=True)
```

6.13.8 Performance Tips

For Large Networks

1. Use **minimal mode** (small nodes, no labels)
2. **Sample nodes** if network is too large:

```
import random

# Sample 1000 random nodes
all_nodes = list(network.get_nodes())
sample_nodes = random.sample(all_nodes, min(1000, len(all_nodes)))
```

(continues on next page)

(continued from previous page)

```
# Create subnetwork
subnetwork = network.get_subnetwork(sample_nodes)

# Visualize sample
draw_multilayer_default(subnetwork.get_layers(), display=True)
```

3. Remove isolated nodes:

```
draw_multilayer_default(
    network.get_layers(),
    remove_isolated_nodes=True # Faster rendering
)
```

4. Use lower resolution for interactive exploration:

```
plt.figure(figsize=(8, 6), dpi=100) # Lower DPI for speed
```

6.13.9 Troubleshooting

Visualization Not Showing

Issue: Plot doesn't appear

Solutions:

```
# Jupyter notebook: Enable inline plots
%matplotlib inline

# Python script: Add plt.show()
draw_multilayer_default(network.get_layers(), display=False)
plt.show()

# Headless server: Save to file
import matplotlib
matplotlib.use('Agg') # Must be before importing pyplot
```

Nodes Overlapping

Issue: Nodes overlap and are hard to distinguish

Solutions:

```
# 1. Use smaller node size
draw_multilayer_default(network.get_layers(), node_size=5)

# 2. Increase layout area
draw_multilayer_default(
    network.get_layers(),
    rectanglex=2.0, # Wider layout
    rectangley=2.0  # Taller layout
)
```

(continues on next page)

(continued from previous page)

```
# 3. Sample nodes
# (See "Performance Tips" above)
```

Labels Too Crowded

Issue: Layer labels overlap or are too dense

Solutions:

```
# 1. Disable labels for large networks
draw_multilayer_default(network.get_layers(), labels=False)

# 2. Adjust label position
draw_multilayer_default(
    network.get_layers(),
    labels=True,
    label_position=1.2 # Move labels outward
)

# 3. Use smaller font
draw_multilayer_default(
    network.get_layers(),
    node_font_size=3 # Smaller font
)
```

6.13.10 Advanced Visualization Modes

Py3plex provides multiple specialized visualization modes for multilayer networks, each optimized for different analysis goals. These modes can be accessed through the unified `visualize_multilayer_network` API or by calling individual plot functions.

Overview of Visualization Modes

The following visualization modes are available:

1. **diagonal** (default): Layer-centric diagonal layout with inter-layer edges
2. **small_multiples**: Side-by-side comparison of layers in a grid
3. **edge_colored_projection**: Aggregated view with color-coded edges by layer
4. **supra_adjacency_heatmap**: Matrix representation of the multilayer structure
5. **radial_layers**: Concentric circles with layers as rings
6. **ego_multilayer**: Focus on a single node's neighborhood across layers
7. **flow/alluvial**: Layered flow visualization with horizontal bands
8. **sankey**: Sankey diagram showing inter-layer flow strength

Unified API

All visualization modes can be accessed through the `visualize_multilayer_network` function:

```

from py3plex.core import multinet
from py3plex.visualization.multilayer import visualize_multilayer_network

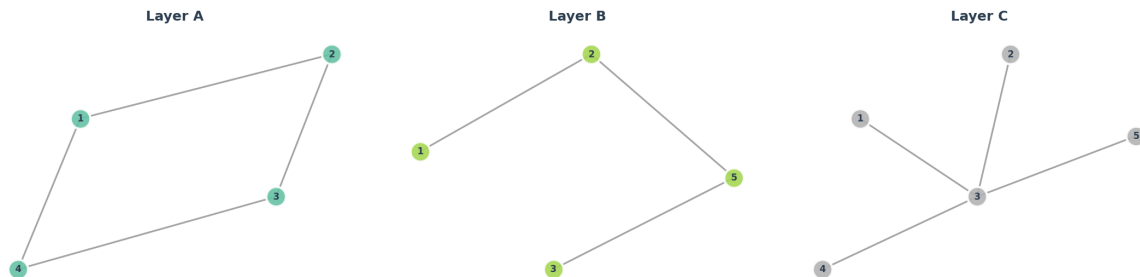
# Load network
network = multinet.multi_layer_network()
network.load_network("network.txt", input_type="multiedgelist")

# Use any visualization mode
fig = visualize_multilayer_network(
    network,
    visualization_type="small_multiples" # or any other mode
)

```

Small Multiples View

Displays each layer as a separate subplot in a grid layout, making it easy to compare layer structures side-by-side.



```

from py3plex.visualization.multilayer import visualize_multilayer_network

# Shared layout (nodes appear at same positions across layers)
fig = visualize_multilayer_network(
    network,
    visualization_type="small_multiples",
    shared_layout=True,          # Same node positions in all layers
    layout="spring",            # Layout algorithm
    node_size=300,
    max_cols=3,                 # Maximum columns in grid
    show_layer_titles=True,
    with_labels=True
)

# Independent layouts (optimized per layer)
fig = visualize_multilayer_network(
    network,
    visualization_type="small_multiples",
    shared_layout=False,        # Different layouts per layer
    layout="circular"
)

```

Best for:

- Comparing layer structures
- Identifying layer-specific patterns

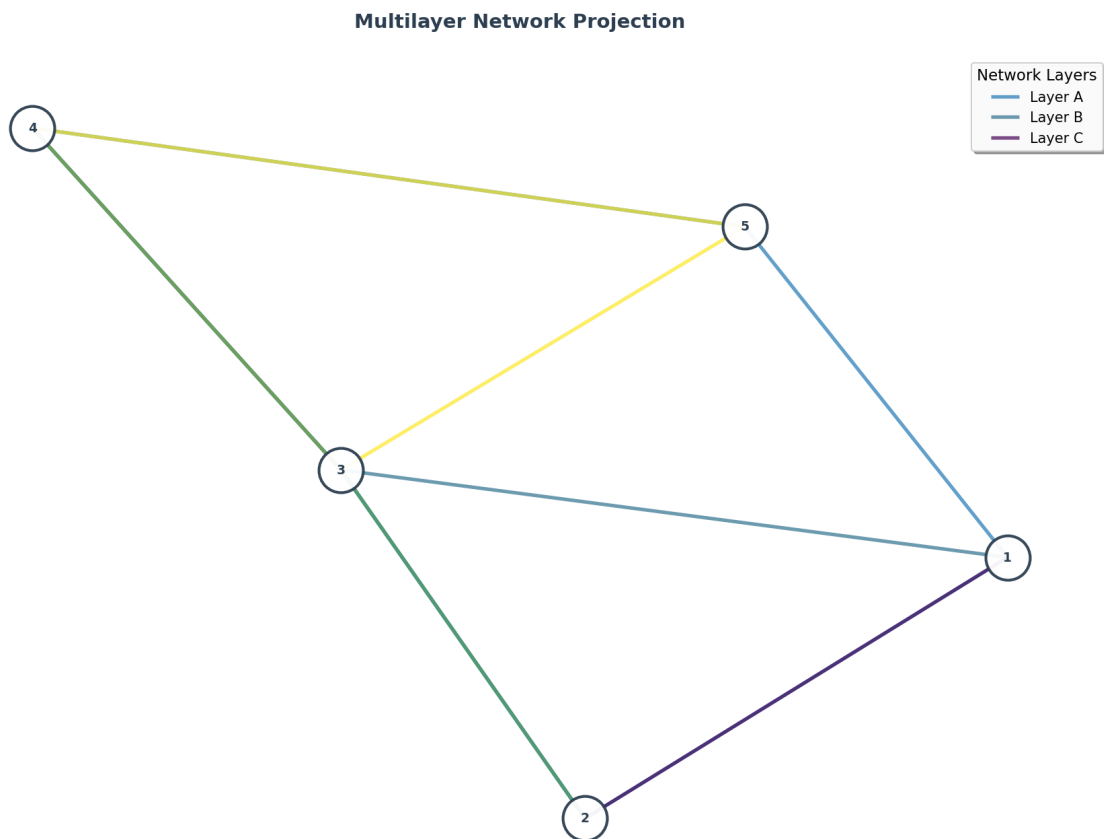
- Understanding node presence across layers
- Networks with 2-10 layers

Parameters:

- `shared_layout`: Use same node positions across layers
- `layout`: “spring”, “circular”, “random”, “kamada_kawai”
- `max_cols`: Maximum number of columns in grid
- `show_layer_titles`: Display layer names

Edge-Colored Projection

Projects all layers onto a single 2D graph, using edge colors to indicate layer membership. Useful for seeing the overall structure while maintaining layer information.



```
from py3plex.visualization.multilayer import visualize_multilayer_network

# Basic projection
fig = visualize_multilayer_network(
    network,
    visualization_type="edge_colored_projection",
    layout="spring",
    node_size=500,
    edge_alpha=0.7,           # Edge transparency
```

(continues on next page)

(continued from previous page)

```
    with_labels=True
)

# Custom layer colors
custom_colors = {
    'layer1': 'red',
    'layer2': 'blue',
    'layer3': 'green'
}

fig = visualize_multilayer_network(
    network,
    visualization_type="edge_colored_projection",
    layer_colors=custom_colors
)
```

Best for:

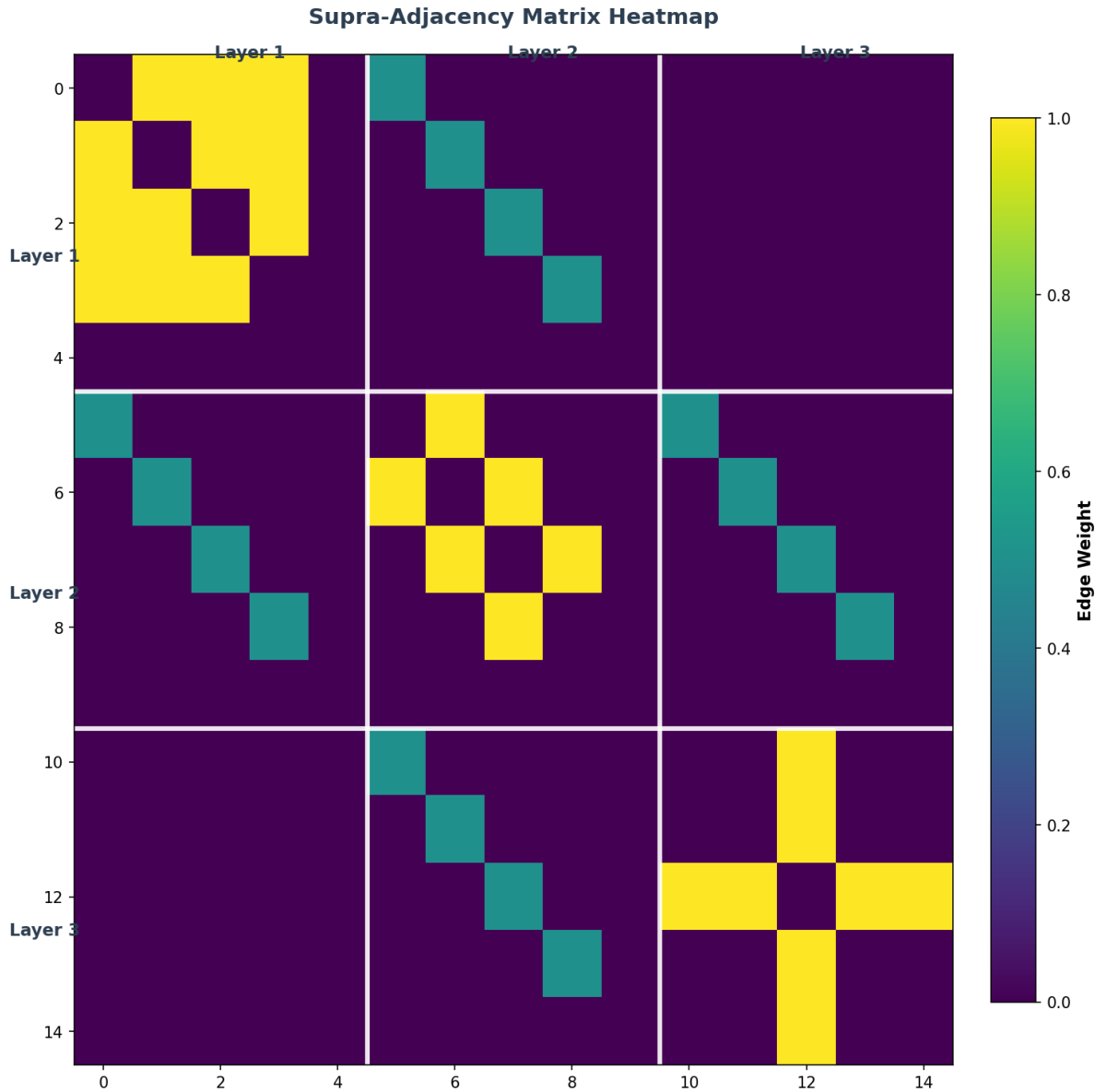
- Aggregate network structure
- Comparing edge distributions across layers
- Identifying layer-dominant connections
- Networks where edges don't heavily overlap

Parameters:

- layout: Layout algorithm for the aggregated graph
- layer_colors: Dict mapping layer names to colors
- edge_alpha: Transparency level for edges (0-1)
- figsize: Figure dimensions

Supra-Adjacency Heatmap

Shows the multilayer network as a block matrix where each block represents the adjacency matrix of one layer. Can optionally include inter-layer connections.



```
from py3plex.visualization.multilayer import visualize_multilayer_network

# Intra-layer only (block-diagonal structure)
fig = visualize_multilayer_network(
    network,
    visualization_type="supra_adjacency_heatmap",
    include_inter_layer=False,
    cmap="Blues"
)

# With inter-layer coupling
fig = visualize_multilayer_network(
    network,
    visualization_type="supra_adjacency_heatmap",
    include_inter_layer=True,
```

(continues on next page)

(continued from previous page)

```
inter_layer_weight=0.5,    # Weight for inter-layer edges
cmap="viridis"
)
```

Best for:

- Mathematical analysis of structure
- Identifying block patterns
- Understanding coupling between layers
- Spectral analysis preparation

Parameters:

- `include_inter_layer`: Show inter-layer connections
- `inter_layer_weight`: Default weight for inter-layer edges
- `cmap`: Matplotlib colormap name
- `figsize`: Figure dimensions

Interpretation:

- Diagonal blocks = intra-layer adjacency matrices
- Off-diagonal blocks = inter-layer connections
- White grid lines = layer boundaries

Radial/Concentric Layers

Arranges layers as concentric circles, with nodes positioned on rings and inter-layer edges shown as radial connections.

Radial Multilayer Network



```

from py3plex.visualization.multilayer import visualize_multilayer_network

# With inter-layer edges
fig = visualize_multilayer_network(
    network,
    visualization_type="radial_layers",
    base_radius=1.0,          # Radius of innermost layer
    radius_step=1.5,         # Distance between layers
    node_size=200,
    draw_inter_layer_edges=True,
    edge_alpha=0.5
)

```

(continues on next page)

(continued from previous page)

```
# Intra-layer only
fig = visualize_multilayer_network(
    network,
    visualization_type="radial_layers",
    draw_inter_layer_edges=False
)
```

Best for:

- Temporal networks (layers as time steps)
- Hierarchical multilayer networks
- Visualizing node evolution across layers
- Networks with clear layer ordering

Parameters:

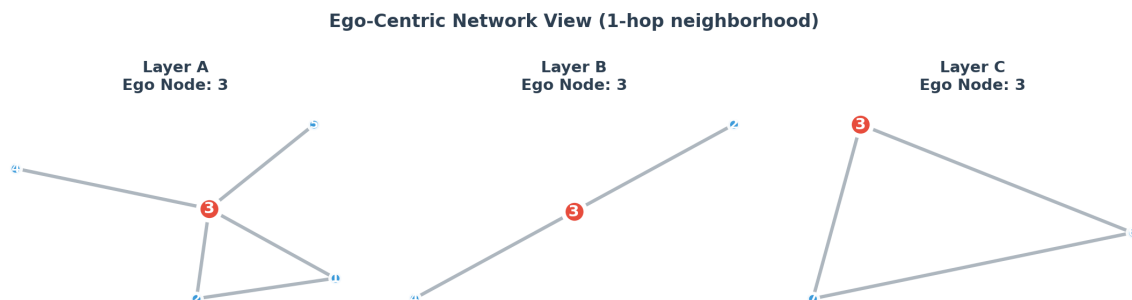
- `base_radius`: Radius of innermost ring
- `radius_step`: Distance between consecutive rings
- `draw_inter_layer_edges`: Show inter-layer connections
- `node_size`: Size of nodes

Interpretation:

- Each ring = one layer
- Same node appears at same angle on all rings
- Dashed lines = inter-layer connections

Ego-Centric Multilayer View

Focuses on a single node (the “ego”) and shows its neighborhood across different layers, highlighting the ego node’s position in each layer context.



```
from py3plex.visualization.multilayer import visualize_multilayer_network

# Basic ego view
fig = visualize_multilayer_network(
    network,
    visualization_type="ego_multilayer",
    ego='node_id',           # Node to focus on
    max_depth=1,             # Neighborhood depth (hops)
)
```

(continues on next page)

(continued from previous page)

```

    layout="spring",
    node_size=100,
    ego_node_size=400          # Highlight ego node
)

# Specific layers only
fig = visualize_multilayer_network(
    network,
    visualization_type="ego_multilayer",
    ego='node_id',
    layers=['layer1', 'layer2'], # Only these layers
    max_depth=2,                # 2-hop neighborhood
    with_labels=True
)

```

Best for:

- Analyzing individual node behavior
- Comparing local structure across layers
- Identifying layer-specific influential neighbors
- Understanding node role variations

Parameters:

- ego: Node ID to focus on
- layers: Specific layers to visualize (or None for all)
- max_depth: Neighborhood depth in hops
- layout: Layout algorithm per ego graph
- ego_node_size: Size of the highlighted ego node

Interpretation:

- Red node = the ego (focal node)
- Blue nodes = neighbors of the ego
- Each subplot = ego's neighborhood in one layer

Flow and Sankey Diagrams

Py3plex provides two complementary visualizations for understanding inter-layer connectivity: flow/alluvial diagrams and Sankey diagrams. Both show how nodes and connections flow between layers, but with different visual approaches.

Flow/Alluvial Visualization

The flow visualization shows each layer as a horizontal band with nodes positioned along the x-axis. Inter-layer connections are drawn as flowing Bezier curves, with node activity encoded by color and size.

```

from py3plex.core import multinet

# Load network
network = multinet.multi_layer_network()

```

(continues on next page)

(continued from previous page)

```
network.load_network("network.txt", input_type="multiedgelist")

# Flow visualization
ax = network.visualize_network(style='flow', show=True)

# Or use 'alluvial' (same as 'flow')
ax = network.visualize_network(style='alluvial', show=True)
```

Best for:

- Visualizing node-level connectivity across layers
- Understanding individual node trajectories
- Showing how specific nodes connect between layers
- Networks with up to 5-6 layers

Sankey Diagram for Inter-Layer Flows

The Sankey-style diagram provides an aggregate view of inter-layer connection strength. The visualization uses text and arrows where width represents the number of connections between layers. This is ideal for understanding overall flow patterns rather than individual node connections.

```
from py3plex.core import multinet

# Load network
network = multinet.multi_layer_network()
network.load_network("network.txt", input_type="multiedgelist")

# Sankey-style diagram showing inter-layer flow strength
ax = network.visualize_network(style='sankey', show=True)
```

You can also use the function directly:

```
from py3plex.visualization.sankey import draw_multilayer_sankey

# Get layers
labels, graphs, multilinks = network.get_layers()

# Create Sankey-style diagram
ax = draw_multilayer_sankey(
    graphs,
    multilinks,
    labels=labels,
    display=True
)
```

Best for:

- Analyzing aggregate inter-layer connection patterns
- Understanding flow strength between layers
- Identifying dominant layer connections
- Networks with 2-5 layers

- Summarizing inter-layer connectivity

Interpretation:

- Text shows layer-to-layer connections with counts
- Arrows indicate connection direction
- Arrow width proportional to connection strength
- Useful for identifying which layer pairs have strongest connections

Note:

This uses a simplified flow diagram approach rather than matplotlib's traditional Sankey class, as it's better suited for multilayer network inter-layer connections.

Example Workflows

Comparing Visualization Modes

Different modes reveal different aspects of the same network:

```
from py3plex.visualization.multilayer import visualize_multilayer_network
import matplotlib.pyplot as plt

# Load network
network = multinet.multi_layer_network()
network.load_network("network.txt", input_type="multiedgelist")

# Compare multiple views
modes = [
    "small_multiples",
    "edge_colored_projection",
    "supra_adjacency_heatmap",
    "radial_layers"
]

for mode in modes:
    fig = visualize_multilayer_network(network, visualization_type=mode)
    fig.savefig(f"network_{mode}.png", dpi=150, bbox_inches='tight')
    plt.close()
```

Analyzing Specific Nodes

Use ego-centric view to understand individual nodes:

```
# Find high-degree nodes
import networkx as nx

# Get aggregated network
agg_graph = nx.Graph()
for node in network.get_nodes():
    agg_graph.add_node(node[0])
for u, v in network.get_edges():
    agg_graph.add_edge(u[0], v[0])
```

(continues on next page)

(continued from previous page)

```
# Get top nodes by degree
degrees = dict(agg_graph.degree())
top_nodes = sorted(degrees, key=degrees.get, reverse=True)[:5]

# Visualize each top node's ego network
for node in top_nodes:
    fig = visualize_multilayer_network(
        network,
        visualization_type="ego_multilayer",
        ego=node,
        max_depth=1
    )
    fig.savefig(f"ego_{node}.png", bbox_inches='tight')
plt.close()
```

Choosing the Right Visualization

Use this guide to select the appropriate visualization mode:

For structural comparison:

- Use `small_multiples` to compare layer structures side-by-side
- Use `edge_colored_projection` to see aggregated structure with layer info

For mathematical analysis:

- Use `supra_adjacency_heatmap` for matrix-based analysis
- Useful for spectral methods and linear algebra operations

For temporal or hierarchical networks:

- Use `radial_layers` when layers have natural ordering
- Shows progression or hierarchy clearly

For local analysis:

- Use `ego_multilayer` to understand individual nodes
- Compare how a node's role varies across layers

For presentations:

- Use `edge_colored_projection` for clean, simple overview
- Use `small_multiples` when detail matters
- Use `radial_layers` for visually striking displays

6.13.11 Next Steps

- *Network Analysis* - Analyze network properties
- *Community Detection* - Detect communities for coloring
- *Performance and Scalability* - Optimize for large networks
- *CSV Data Loading Examples* - Load data from CSV

For more examples, see the [visualization examples](#) in the GitHub repository.

6.14 Dependency Management and Troubleshooting

This guide covers dependency installation, optional features, and common troubleshooting scenarios.

Table of Contents

- *Core Dependencies*
 - *Automatic Installation*
 - *Verifying Installation*
- *Optional Dependencies*
 - *Advanced Visualization (Recommended)*
 - *Additional Algorithms*
 - *Infomap Community Detection*
 - *Installing All Optional Features*
 - *Development Dependencies*
- *Dependency Troubleshooting*
 - *Common Installation Issues*
 - *Runtime Dependency Issues*
 - *Version Conflicts*
- *LLM-Friendly Dependency Guidance*
 - *For AI Assistants Helping Users*
 - *Quick Dependency Checker Script*
- *Best Practices*
 - *Recommendations for Different Use Cases*
 - *Virtual Environment Workflow*
 - *Keeping Dependencies Updated*
- *Next Steps*

6.14.1 Core Dependencies

Automatic Installation

Core dependencies are **automatically installed** when you install Py3plex:

```
pip install git+https://github.com/SkBlaz/py3plex.git
```

Core dependencies include:

- `networkx` `>= 2.5` - Graph data structures and algorithms
- `numpy` `>= 1.19.0` - Numerical computing
- `scipy` `>= 1.5.0` - Scientific computing and sparse matrices

- matplotlib >= 3.3.0 - Static visualization
- pandas - Data manipulation
- scikit-learn >= 0.24.0 - Machine learning utilities
- tqdm >= 4.40.0 - Progress bars
- rdflib >= 6.0.0 - Semantic web support
- bitarray >= 2.0.0 - Efficient boolean arrays
- seaborn >= 0.11.0 - Statistical visualization
- gensim >= 4.0.0 - Topic modeling and embeddings
- cython >= 0.29.0 - C extensions for performance

Verifying Installation

Check that Py3plex and dependencies are installed:

```
import py3plex
print(f"Py3plex version: {py3plex.__version__}")

# Check key dependencies
import networkx as nx
import numpy as np
import scipy
import matplotlib

print(f"NetworkX: {nx.__version__}")
print(f"NumPy: {np.__version__}")
print(f"SciPy: {scipy.__version__}")
print(f"Matplotlib: {matplotlib.__version__}")

print("\n[OK] All core dependencies available")
```

6.14.2 Optional Dependencies

Py3plex offers several optional feature sets. Install only what you need.

Advanced Visualization (Recommended)

Install Plotly and igraph for interactive and advanced visualizations:

```
pip install git+https://github.com/SkBlaz/py3plex.git#egg=py3plex[viz]
```

Provides:

- plotly >= 5.0.0 - Interactive network plots
- python-igraph >= 0.10.0 - Fast C-based graph algorithms

Use cases:

- Interactive exploration in Jupyter notebooks
- Web-based visualizations
- Fast layout computation

- Large network rendering

Example:

```
# Check if available
try:
    import plotly
    import igraph
    print("[OK] Advanced visualization available")
except ImportError:
    print("[X] Install with: pip install py3plex[viz]")
```

Additional Algorithms

Install extra community detection and clustering algorithms:

```
pip install git+https://github.com/SkBlaz/py3plex.git#egg=py3plex[algos]
```

Provides:

- python-louvain >= 0.16 - Louvain modularity optimization
- cdlib >= 0.3.0 - Community discovery library (many algorithms)

Use cases:

- Advanced community detection
- Algorithm comparison studies
- Overlapping community detection
- Hierarchical clustering

Example:

```
# Using Louvain algorithm
from py3plex.algorithms.community_detection import community_louvain

communities = community_louvain.best_partition(network.core_network)
print(f"Detected {len(set(communities.values()))} communities")
```

Infomap Community Detection

Install Infomap for information-theoretic community detection:

```
pip install git+https://github.com/SkBlaz/py3plex.git#egg=py3plex[infomap]
```

Provides:

- infomap >= 2.0.0 - Information flow-based community detection

WARNING Important licensing note:

Infomap is licensed under **AGPLv3** (viral copyleft license). If you use Infomap functions in your project, your project may also need to be AGPLv3 licensed.

For commercial/proprietary projects: Use alternative algorithms:

- Louvain (BSD-3-Clause)
- Label propagation (MIT)

- Built-in NetworkX algorithms (BSD)

Example:

```
# Check if available
try:
    import infomap
    print("[OK] Infomap available")
except ImportError:
    print("[X] Install with: pip install py3plex[infomap]")
    print("  OR use Louvain algorithm instead")
```

Installing All Optional Features

Install everything at once:

```
pip install git+https://github.com/SkBlaz/py3plex.git#egg=py3plex[viz,algorithms,infomap]
```

Development Dependencies

For contributors and developers:

```
git clone https://github.com/SkBlaz/py3plex.git
cd py3plex
pip install -e ".[dev]"
```

Provides:

- pytest - Testing framework
- pytest-cov - Coverage reporting
- pytest-benchmark - Performance testing
- black - Code formatting
- ruff - Linting
- mypy - Type checking
- sphinx - Documentation building
- crosshair-tool - Formal verification
- hypothesis - Property-based testing

6.14.3 Dependency Troubleshooting

Common Installation Issues

Issue: “Could not find a version that satisfies the requirement”

Cause: Python version too old or package unavailable

Solution 1: Upgrade Python to 3.8 or higher

```
python --version # Check current version
# If < 3.8, install Python 3.10 or 3.11
```

Solution 2: Update pip

```
pip install --upgrade pip setuptools wheel
```

Solution 3: Check PyPI availability

```
pip search networkx # Check if package exists
```

Issue: “No matching distribution found”

Cause: Network issues or package not available for your platform

Solution 1: Check internet connection

```
ping pypi.org
```

Solution 2: Use alternative index

```
pip install --index-url https://pypi.org/simple/ \
  git+https://github.com/SkBlaz/py3plex.git
```

Solution 3: Check platform compatibility

Some packages may not be available for all platforms (e.g., Windows ARM, older macOS).

Issue: “Failed building wheel”

Cause: Missing C compiler or development headers

Solution (Ubuntu/Debian):

```
sudo apt-get update
sudo apt-get install python3-dev build-essential
```

Solution (macOS):

```
xcode-select --install
```

Solution (Windows):

1. Download Visual C++ Build Tools
2. Install “Desktop development with C++”
3. Restart and try again

Issue: “Permission denied”

Cause: Trying to install in system Python without sudo

Solution 1: Use virtual environment (recommended)

```
python3 -m venv py3plex-env
source py3plex-env/bin/activate # Linux/macOS
# OR: py3plex-env\Scripts\activate # Windows

pip install git+https://github.com/SkBlaz/py3plex.git
```

Solution 2: User installation

```
pip install --user git+https://github.com/SkBlaz/py3plex.git
```

Runtime Dependency Issues

Issue: “No module named ‘plotly’”

Cause: Optional visualization dependency not installed

Solution:

```
pip install git+https://github.com/SkBlaz/py3plex.git#egg=py3plex[viz]
```

OR install just what you need

```
pip install plotly
```

Workaround: Use matplotlib for visualization (included by default)

Issue: “ImportError: cannot import name ‘infomap’”

Cause: Optional Infomap package not installed

Solution:

```
pip install git+https://github.com/SkBlaz/py3plex.git#egg=py3plex[infomap]
```

Alternative: Use Louvain algorithm instead:

```
from py3plex.algorithms.community_detection import community_louvain
communities = community_louvain.best_partition(network.core_network)
```

Issue: “Qt platform plugin could not be initialized”

Cause: Matplotlib backend issue (usually on headless servers)

Solution: Use non-interactive backend

```
import matplotlib
matplotlib.use('Agg') # Must be before importing pyplot
import matplotlib.pyplot as plt
```

Issue: “MemoryError” with large networks

Cause: Insufficient RAM for dense operations

Solution 1: Use sparse matrices (automatic for most operations)

Solution 2: Sample network

```
import random

# Sample 10% of nodes
all_nodes = list(network.get_nodes())
sample_size = len(all_nodes) // 10
sample_nodes = random.sample(all_nodes, sample_size)

subnetwork = network.get_subnetwork(sample_nodes)
```


Solution 3: Increase swap space or use machine with more RAM

Version Conflicts

Issue: “Incompatible versions of numpy and scipy”

Cause: Version mismatch between numerical packages

Solution: Reinstall in correct order

```
pip uninstall numpy scipy
pip install numpy scipy
pip install git+https://github.com/SkBlaz/py3plex.git
```

Issue: “AttributeError: module ‘networkx’ has no attribute ‘X’”

Cause: NetworkX version too old

Solution: Upgrade NetworkX

```
pip install --upgrade networkx

# Verify version
python -c "import networkx; print(networkx.__version__)"
# Should be >= 2.5
```

6.14.4 LLM-Friendly Dependency Guidance

For AI Assistants Helping Users

When a user encounters a missing dependency error:

Step 1: Identify the missing package from the error message

```
# Example error:
# ModuleNotFoundError: No module named 'plotly'
```

Step 2: Suggest appropriate installation command

```
# For plotly (visualization):
pip install plotly

# OR install full viz bundle:
pip install git+https://github.com/SkBlaz/py3plex.git#egg=py3plex[viz]
```

Step 3: Provide alternative solutions

If installation fails, suggest:

1. Using built-in alternatives (matplotlib instead of plotly)
2. Checking Python version compatibility
3. Using virtual environment
4. Checking firewall/proxy settings

Quick Dependency Checker Script

Provide users with this diagnostic script:

```
"""Check Py3plex dependencies."""

def check_dependencies():
    """Check which dependencies are available."""

    required = {
        'networkx': 'Core',
        'numpy': 'Core',
        'scipy': 'Core',
        'matplotlib': 'Core',
        'pandas': 'Core',
    }

    optional = {
        'plotly': 'Visualization',
        'igraph': 'Visualization',
        'infomap': 'Community detection',
        'python-louvain': 'Community detection',
    }

    print("=== Py3plex Dependency Check ===\n")

    print("Required packages:")
    for package, category in required.items():
        try:
            __import__(package.replace('-', '_'))
            print(f" [OK] {package:20s} ({category})")
        except ImportError:
            print(f" [X] {package:20s} ({category}) - MISSING!")

    print("\nOptional packages:")
    for package, category in optional.items():
        try:
            __import__(package.replace('-', '_'))
            print(f" [OK] {package:20s} ({category})")
        except ImportError:
            print(f" [X] {package:20s} ({category}) - Not installed")

    print("\nInstallation commands:")
    print(" Core: pip install git+https://github.com/SkBlaz/py3plex.git")
    print(" Viz:  pip install py3plex[viz]")
    print(" All:  pip install py3plex[viz,algos]")

if __name__ == '__main__':
    check_dependencies()
```

Save as `check_deps.py` and run:

```
python check_deps.py
```

6.14.5 Best Practices

Recommendations for Different Use Cases

For research (exploratory):

```
# Core only
pip install git+https://github.com/SkBlaz/py3plex.git
```

For visualization-heavy work:

```
# Core + visualization
pip install git+https://github.com/SkBlaz/py3plex.git#egg=py3plex[viz]
```

For comprehensive analysis:

```
# Everything except Infomap (avoid AGPLv3 license)
pip install git+https://github.com/SkBlaz/py3plex.git#egg=py3plex[viz,algos]
```

For commercial projects:

```
# Core + algos (all permissive licenses)
pip install git+https://github.com/SkBlaz/py3plex.git#egg=py3plex[viz,algos]
# Avoid: infomap (AGPLv3)
```

Virtual Environment Workflow

```
# Create project directory
mkdir my_network_project
cd my_network_project

# Create virtual environment
python3 -m venv venv
source venv/bin/activate # Linux/macOS

# Install Py3plex
pip install git+https://github.com/SkBlaz/py3plex.git#egg=py3plex[viz,algos]

# Save dependencies
pip freeze > requirements.txt

# Later, recreate environment
pip install -r requirements.txt
```

Keeping Dependencies Updated

```
# Update Py3plex
pip install --upgrade git+https://github.com/SkBlaz/py3plex.git

# Update all dependencies
pip install --upgrade numpy scipy networkx matplotlib

# Check for outdated packages
pip list --outdated
```

6.14.6 Next Steps

- *Installation and Setup* - Full installation guide
- *Quickstart Guide* - Get started with Py3plex
- *CSV Data Loading Examples* - Load your data
- *Visualization Guide* - Create visualizations

For installation help, open an issue on [GitHub Issues](#).

6.15 NetworkX Interoperability Guide

This guide covers converting between Py3plex and NetworkX formats, preserving attributes, and integrating with external tools.

Table of Contents

- *Why NetworkX Interoperability?*
- *Export to NetworkX*
 - *Basic Export*
 - *What Gets Exported*
- *Attribute Preservation*
 - *Layer Information*
 - *Weight Preservation*
 - *Custom Attributes*
- *Using NetworkX Algorithms*
 - *Shortest Paths*
 - *Centrality Measures*
 - *Community Detection*
 - *Connectivity*
- *Integration with External Tools*
 - *Export to Gephi*
 - *Export to Cytoscape*
 - *Convert to igraph*
- *Py3plex → NetworkX → TensorLy*
 - *Complete Workflow*
 - *Supra-Adjacency Matrix*
- *Import from NetworkX*
 - *Create Py3plex Network from NetworkX*
- *Practical Examples*

- *Example 1: Network Statistics Pipeline*
- *Example 2: Multilayer PageRank*
- *Example 3: Export for Gephi Visualization*
- *Next Steps*

6.15.1 Why NetworkX Interoperability?

Py3plex uses NetworkX as its underlying graph backend, which provides:

- **Compatibility** with hundreds of NetworkX algorithms
- **Easy export** to other tools (Gephi, Cytoscape, graph-tool)
- **Integration** with scientific Python ecosystem (NumPy, SciPy, pandas)
- **Standard formats** (GraphML, GEXF, GML, etc.)

6.15.2 Export to NetworkX

Basic Export

Convert Py3plex multilayer network to NetworkX graph:

```
from py3plex.core import multinet

# Load multilayer network
network = multinet.multi_layer_network()
network.load_network("data.csv", input_type="multiedgelist")

# Export to NetworkX MultiGraph
nx_graph = network.core_network # Direct access

# OR use conversion method (equivalent)
nx_graph = network.to_nx_network()

print(f"NetworkX graph: {nx_graph.number_of_nodes()} nodes, "
      f"{nx_graph.number_of_edges()} edges")
```

What Gets Exported

When exporting to NetworkX, **all attributes are preserved**:

Node attributes:

- **type** - Layer name
- Custom attributes added via `add_nodes()`

Edge attributes:

- **weight** - Edge weight (default 1.0)
- **type** - Edge type (e.g., 'coupling' for inter-layer edges)
- Custom attributes added via `add_edges()`

Example:

```
# Access NetworkX graph
nx_graph = network.core_network

# Inspect node attributes
for node, attrs in list(nx_graph.nodes(data=True))[:3]:
    print(f"Node: {node}")
    print(f"  Attributes: {attrs}")
    print()

# Inspect edge attributes
for u, v, attrs in list(nx_graph.edges(data=True))[:3]:
    print(f"Edge: {u} -> {v}")
    print(f"  Attributes: {attrs}")
    print()
```

6.15.3 Attribute Preservation

Layer Information

Layer information is encoded in node tuples:

```
# Py3plex stores nodes as tuples: (node_id, layer_name)

# Example node: ('A', 'layer1')
# - node_id: 'A'
# - layer_name: 'layer1'

# Iterate over nodes and extract layer info
nx_graph = network.core_network

for node in nx_graph.nodes():
    node_id, layer = node # Unpack tuple
    node_type = nx_graph.nodes[node]['type'] # Should equal layer

    print(f"Node {node_id} in layer {layer} (type={node_type})")
```

Weight Preservation

Edge weights are preserved as edge attributes:

```
nx_graph = network.core_network

# Get edge weights
for u, v, data in nx_graph.edges(data=True):
    weight = data.get('weight', 1.0) # Default to 1.0
    print(f"{u} -> {v}: weight={weight}")
```

Custom Attributes

Any custom attributes you add are preserved:

```
from py3plex.core import multinet
```

(continues on next page)

(continued from previous page)

```

# Create network
network = multinet.multi_layer_network()

# Add edges with custom attributes
network.add_edges([
    ['A', 'layer1', 'B', 'layer1', 1.0, {'color': 'red', 'importance': 0.8}],
    ['B', 'layer1', 'C', 'layer1', 1.0, {'color': 'blue', 'importance': 0.6}],
], input_type="list")

# Export to NetworkX
nx_graph = network.core_network

# Custom attributes are preserved
for u, v, data in nx_graph.edges(data=True):
    print(f"{u} -> {v}:")
    print(f"    color: {data.get('color')}")
    print(f"    importance: {data.get('importance')}")

```

6.15.4 Using NetworkX Algorithms

All NetworkX algorithms work directly on Py3plex networks:

Shortest Paths

```

import networkx as nx
from py3plex.core import multinet

# Load network
network = multinet.multi_layer_network()
network.load_network("network.csv", input_type="multiedgelist")

# Use NetworkX shortest path algorithm
nx_graph = network.core_network

try:
    # Find shortest path between two node-layer tuples
    source = ('A', 'layer1')
    target = ('C', 'layer2')

    path = nx.shortest_path(nx_graph, source=source, target=target)
    print(f"Shortest path: {' -> '.join(str(n) for n in path)}")

    # Path length
    length = nx.shortest_path_length(nx_graph, source=source, target=target)
    print(f"Path length: {length}")

except nx.NetworkXNoPath:
    print("No path exists between these nodes")

```

Centrality Measures

```
import networkx as nx

# Degree centrality
degree_cent = nx.degree_centrality(network.core_network)
top_nodes = sorted(degree_cent.items(), key=lambda x: x[1], reverse=True)[:5]

print("Top 5 nodes by degree centrality:")
for node, centrality in top_nodes:
    print(f"  {node}: {centrality:.3f}")

# Betweenness centrality
between_cent = nx.betweenness_centrality(network.core_network)

# Closeness centrality
close_cent = nx.closeness_centrality(network.core_network)

# PageRank
pagerank = nx.pagerank(network.core_network)
```

Community Detection

```
import networkx as nx
from networkx.algorithms import community

# Greedy modularity communities
communities = community.greedy_modularity_communities(network.core_network)

print(f"Detected {len(communities)} communities")
for i, comm in enumerate(communities):
    print(f"Community {i}: {len(comm)} nodes")
```

Connectivity

```
import networkx as nx

# Check if network is connected
if nx.is_connected(network.core_network.to_undirected()):
    print("Network is connected")
else:
    print("Network has multiple components")

# Find connected components
components = list(nx.connected_components(network.core_network.to_undirected()))
print(f"Number of components: {len(components)}")
for i, comp in enumerate(components):
    print(f"  Component {i}: {len(comp)} nodes")
```


6.15.5 Integration with External Tools

Export to Gephi

Gephi is a popular network visualization tool. Export Py3plex networks to GEXF format:

```
import networkx as nx

# Export to GEXF (Gephi format)
nx_graph = network.core_network
nx.write_gexf(nx_graph, "network_for_gephi.gexf")

print("[OK] Exported to network_for_gephi.gexf")
print("  Open in Gephi: File → Open → network_for_gephi.gexf")
```

Export to Cytoscape

Cytoscape is a bioinformatics network analysis tool. Export to GraphML:

```
import networkx as nx

# Export to GraphML (Cytoscape format)
nx_graph = network.core_network
nx.write_graphml(nx_graph, "network_for_cytoscape.graphml")

print("[OK] Exported to network_for_cytoscape.graphml")
print("  Open in Cytoscape: File → Import → Network from File")
```

Convert to igraph

igraph is a fast C-based network analysis library:

```
# Requires: pip install python-igraph
import igraph as ig
import networkx as nx

# Convert NetworkX to igraph
nx_graph = network.core_network

# Method 1: Via GraphML
nx.write_graphml(nx_graph, "temp.graphml")
ig_graph = ig.Graph.Read_GraphML("temp.graphml")

# Method 2: Via edge list (faster)
edges = list(nx_graph.edges())
nodes = list(nx_graph.nodes())

ig_graph = ig.Graph()
ig_graph.add_vertices(len(nodes))
ig_graph.vs["name"] = [str(n) for n in nodes]

# Map node names to indices
node_to_idx = {n: i for i, n in enumerate(nodes)}
ig_edges = [(node_to_idx[u], node_to_idx[v]) for u, v in edges]
```

(continues on next page)

(continued from previous page)

```

ig_graph.add_edges(ig_edges)

print(f"[OK] Converted to igraph: {ig_graph.vcount()} vertices, {ig_graph.ecount()} edges
→")

# Use igraph algorithms
communities = ig_graph.community_multilevel()
print(f"  Detected {len(communities)} communities")

```

6.15.6 Py3plex → NetworkX → TensorLy

Complete Workflow

Convert multilayer network to tensor representation for tensor decomposition:

```

import numpy as np
import networkx as nx
from py3plex.core import multinet

# Step 1: Load multilayer network
network = multinet.multi_layer_network()
network.load_network("network.csv", input_type="multiedgelist")

# Step 2: Get NetworkX graph
nx_graph = network.core_network

# Step 3: Extract layer information
layers = sorted(set(node[1] for node in nx_graph.nodes()))
nodes = sorted(set(node[0] for node in nx_graph.nodes()))

n_nodes = len(nodes)
n_layers = len(layers)

print(f"Network: {n_nodes} nodes, {n_layers} layers")

# Step 4: Create 3D adjacency tensor (nodes × layers × nodes)
tensor = np.zeros((n_nodes, n_layers, n_nodes))

# Create mappings
node_to_idx = {node: i for i, node in enumerate(nodes)}
layer_to_idx = {layer: i for i, layer in enumerate(layers)}

# Fill tensor
for (u_id, u_layer), (v_id, v_layer), data in nx_graph.edges(data=True):
    if u_layer == v_layer: # Intra-layer edge
        i = node_to_idx[u_id]
        j = node_to_idx[v_id]
        k = layer_to_idx[u_layer]
        weight = data.get('weight', 1.0)

        tensor[i, k, j] = weight
        tensor[j, k, i] = weight # Undirected

```

(continues on next page)

(continued from previous page)

```

print(f"Tensor shape: {tensor.shape}")
print(f"Non-zero entries: {np.count_nonzero(tensor)}")

# Step 5: Use Tensorly for decomposition
try:
    import tensorly as tl
    from tensorly.decomposition import tucker, parafac

    # Tucker decomposition
    core, factors = tucker(tl.tensor(tensor), rank=[5, 2, 5])
    print(f"\n[OK] Tucker decomposition complete")
    print(f"  Core tensor shape: {core.shape}")
    print(f"  Factor matrices: {[f.shape for f in factors]}")

    # PARAFAC/CP decomposition
    factors_cp = parafac(tl.tensor(tensor), rank=5)
    print(f"\n[OK] PARAFAC decomposition complete")
    print(f"  Rank: 5")

except ImportError:
    print("\n[X] TensorLy not installed")
    print("  Install: pip install tensorly")

```

Supra-Adjacency Matrix

Convert to supra-adjacency matrix for tensor analysis:

```

import numpy as np
from scipy.sparse import lil_matrix

# Get supra-adjacency matrix
supra_adj = network.get_supra_adjacency_matrix(sparse=True)

print(f"Supra-adjacency matrix: {supra_adj.shape}")
print(f"Sparsity: {1 - supra_adj.nnz / (supra_adj.shape[0]**2):.2%}")

# Convert to dense (if small enough)
if supra_adj.shape[0] < 1000:
    dense_supra = supra_adj.toarray()

    # Use for analysis
    eigenvalues = np.linalg.eigvals(dense_supra)
    print(f"Largest eigenvalue: {max(eigenvalues):.3f}")

```

6.15.7 Import from NetworkX

Create Py3plex Network from NetworkX

```

import networkx as nx
from py3plex.core import multinet

```

(continues on next page)

(continued from previous page)

```
# Create NetworkX graph
G = nx.Graph()
G.add_edges_from([
    (('A', 'layer1'), ('B', 'layer1'), {'weight': 1.0}),
    (('B', 'layer1'), ('C', 'layer1'), {'weight': 0.8}),
    (('A', 'layer2'), ('C', 'layer2'), {'weight': 0.6}),
])

# Import to Py3plex
network = multinet.multi_layer_network()
network.load_network(G, input_type="nx")

print(f"[OK] Imported {network.core_network.number_of_nodes()} nodes")
```

6.15.8 Practical Examples

Example 1: Network Statistics Pipeline

```
import networkx as nx
from py3plex.core import multinet

# Load network
network = multinet.multi_layer_network()
network.load_network("network.csv", input_type="multiedgelist")

# Get NetworkX graph
G = network.core_network

# Compute various statistics
print("=== Network Statistics ===")
print(f"Nodes: {G.number_of_nodes()}")
print(f"Edges: {G.number_of_edges()}")
print(f"Density: {nx.density(G):.4f}")

# Degree statistics
degrees = dict(G.degree())
print(f"Average degree: {sum(degrees.values()) / len(degrees):.2f}")
print(f"Max degree: {max(degrees.values())}")

# Clustering
clustering = nx.clustering(G.to_undirected())
print(f"Average clustering: {sum(clustering.values()) / len(clustering):.4f}")

# Connected components
if not nx.is_directed(G):
    components = list(nx.connected_components(G))
    print(f"Connected components: {len(components)}")
```

Example 2: Multilayer PageRank

```
import networkx as nx

# Compute PageRank on multilayer network
G = network.core_network
pagerank = nx.pagerank(G, weight='weight')

# Group by layer
layers = {}
for (node_id, layer), score in pagerank.items():
    if layer not in layers:
        layers[layer] = []
    layers[layer].append((node_id, score))

# Print top nodes per layer
for layer, nodes in layers.items():
    top_nodes = sorted(nodes, key=lambda x: x[1], reverse=True)[:5]
    print(f"\nTop 5 nodes in {layer}:")
    for node_id, score in top_nodes:
        print(f"    {node_id}: {score:.4f}")
```

Example 3: Export for Gephi Visualization

```
import networkx as nx
from py3plex.core import multinet

# Load and process network
network = multinet.multi_layer_network()
network.load_network("network.csv", input_type="multiedgelist")

# Add community detection results
from py3plex.algorithms.community_detection import community_louvain
communities = community_louvain.best_partition(network.core_network)

# Add community as node attribute
for node, comm_id in communities.items():
    network.core_network.nodes[node]['community'] = comm_id

# Export to GEXF with communities
nx.write_gexf(network.core_network, "network_with_communities.gexf")

print("[OK] Exported to GEXF with community information")
print("    Open in Gephi and color by 'community' attribute")
```

6.15.9 Next Steps

- *Network Analysis* - Network analysis methods
- *Community Detection* - Community detection algorithms
- *Visualization Guide* - Visualization options
- *CSV Data Loading Examples* - Load data from CSV

For more examples, see the [examples/](#) directory in the GitHub repository.

6.16 R Interoperability Guide

This guide shows how to use py3plex from R via the **reticulate** package, enabling R users (especially those familiar with igraph or MLnet) to leverage py3plex’s multilayer network capabilities.

Table of Contents

- *Why Use py3plex from R?*
- *Prerequisites*
- *Quick Start: Simple Use Case with Centrality*
 - *Complete R Workflow*
 - *Expected Output*
- *Core Functions*
 - *to_igraph_for_r()*
 - *export_edgelist()*
 - *export_nodelist()*
 - *export_adjacency()*
 - *get_network_stats()*
- *Advanced Examples*
 - *Example 1: Layer-by-Layer Analysis*
 - *Example 2: Combining with R Statistical Analysis*
 - *Example 3: Time Series of Multilayer Networks*
- *Working with CSV Files*
- *Integration with MLnet*
- *Troubleshooting*
 - *Python Module Not Found*
 - *igraph Import Error*
 - *Conversion Issues*
- *Performance Tips*
 - *For Large Networks*
 - *Memory Management*
- *Next Steps*

6.16.1 Why Use py3plex from R?

py3plex provides specialized multilayer network analysis that complements R's igraph package:

- **Native multilayer support** - Layer-aware algorithms and statistics
- **Specialized visualizations** - Diagonal projection for large multilayer networks
- **Seamless conversion** - Export to R igraph format for further analysis
- **Rich ecosystem** - Combine py3plex multilayer capabilities with R's statistical tools

6.16.2 Prerequisites

Install required packages:

```
# In R
install.packages("reticulate")
install.packages("igraph")
```

```
# In terminal/shell
pip install git+https://github.com/SkBlaz/py3plex.git
pip install python-igraph
```

6.16.3 Quick Start: Simple Use Case with Centrality

This example demonstrates creating a multilayer social network, converting it to igraph, and computing centrality measures in R.

Complete R Workflow

```
library(reticulate)
library(igraph)

# Import py3plex modules
py3plex <- import("py3plex")
r_interop <- import("py3plex.wrappers.r_interop")

# Create a multilayer social network
# Two layers: Facebook and Twitter
net <- py3plex$multi_layer_network()

# Add nodes
net$add_nodes(list(
  list(source='Alice', type='facebook'),
  list(source='Bob', type='facebook'),
  list(source='Charlie', type='facebook'),
  list(source='Alice', type='twitter'),
  list(source='Bob', type='twitter'),
  list(source='Diana', type='twitter')
))

# Add edges (connections within each platform)
net$add_edges(list(
  list(source='Alice', target='Bob',
```

(continues on next page)

(continued from previous page)

```

        source_type='facebook', target_type='facebook', weight=0.8),
    list(source='Bob', target='Charlie',
        source_type='facebook', target_type='facebook', weight=0.6),
    list(source='Alice', target='Bob',
        source_type='twitter', target_type='twitter', weight=0.9),
    list(source='Bob', target='Diana',
        source_type='twitter', target_type='twitter', weight=0.7)
))

# Convert to igraph for R analysis (union mode merges all layers)
g <- r_interop$to_igraph_for_r(net, mode='union')

# Now use R's igraph functions for centrality analysis
print("=== Network Statistics ===")
print(paste("Vertices:", vcount(g)))
print(paste("Edges:", ecount(g)))

# Compute centrality measures
print("\n=== Centrality Measures ===")

# Degree centrality
deg <- degree(g)
print("\nDegree centrality:")
print(sort(deg, decreasing=TRUE))

# Betweenness centrality
between <- betweenness(g)
print("\nBetweenness centrality:")
print(sort(between, decreasing=TRUE))

# Closeness centrality
close <- closeness(g)
print("\nCloseness centrality:")
print(sort(close, decreasing=TRUE))

# PageRank
pr <- page_rank(g)$vector
print("\nPageRank:")
print(sort(pr, decreasing=TRUE))

# Visualize the network
plot(g,
     vertex.size=deg*5,          # Size by degree
     vertex.label=V(g)$name,
     edge.width=E(g)$weight*2,  # Width by weight
     main="Multilayer Social Network")

```

Expected Output

```

=== Network Statistics ===
[1] "Vertices: 6"

```

(continues on next page)

(continued from previous page)

```
[1] "Edges: 4"

=== Centrality Measures ===

Degree centrality:
('Bob', 'facebook')    2
('Alice', 'facebook')  1
('Charlie', 'facebook') 1
('Bob', 'twitter')     2
('Alice', 'twitter')   1
('Diana', 'twitter')   1

Betweenness centrality:
('Bob', 'facebook')    2.0
('Bob', 'twitter')     2.0
('Alice', 'facebook')  0.0
...
```

This demonstrates how py3plex enables you to:

1. Create complex multilayer networks in Python
2. Export them to R igraph format
3. Use all R igraph centrality functions
4. Visualize and analyze with R's rich statistical tools

6.16.4 Core Functions

to_igraph_for_r()

Convert py3plex network to igraph format optimized for R.

```
# Union mode: Merge all layers (recommended for most use cases)
g <- r_interop$to_igraph_for_r(net, mode='union')

# Multiplex mode: Preserve layer structure
g <- r_interop$to_igraph_for_r(net, mode='multiplex')

# Extract specific layer only
g_facebook <- r_interop$to_igraph_for_r(net, layer='facebook')
```

Parameters:

- mode - How to handle layers: 'union' (merge), 'multiplex' (preserve), 'intersection' (common edges)
- layer - Extract specific layer (overrides mode)

export_edgelist()

Export edges as R data frame compatible structure.

```
# Get edge list
edges <- r_interop$export_edgelist(net, include_attributes=TRUE)
```

(continues on next page)

(continued from previous page)

```

# Convert to R data frame
edges_df <- do.call(rbind, lapply(edges, function(e) {
  data.frame(
    src = e$src,
    dst = e$dst,
    src_layer = e$src_layer,
    dst_layer = e$dst_layer,
    weight = ifelse(is.null(e$weight), 1.0, e$weight),
    stringsAsFactors = FALSE
  )
}))

# Analyze with R
print(head(edges_df))
print(summary(edges_df$weight))

```

export_nodelist()

Export nodes as R data frame compatible structure.

```

# Get node list
nodes <- r_interop$export_nodelist(net, include_attributes=TRUE)

# Convert to R data frame
nodes_df <- do.call(rbind, lapply(nodes, function(n) {
  as.data.frame(n, stringsAsFactors = FALSE)
}))

print(head(nodes_df))

```

export_adjacency()

Export adjacency matrix for matrix operations in R.

```

# Get adjacency matrix
adj_list <- r_interop$export_adjacency(net, mode='union')

# Convert to R matrix
n <- length(adj_list)
adj_matrix <- matrix(unlist(adj_list), nrow=n, byrow=TRUE)

# Matrix operations
eigenvalues <- eigen(adj_matrix)$values
print(paste("Largest eigenvalue:", max(Re(eigenvalues))))

```

get_network_stats()

Get comprehensive network statistics.

```

# Get statistics
stats <- r_interop$get_network_stats(net)

```

(continues on next page)

(continued from previous page)

```
print(paste("Nodes:", stats$num_nodes))
print(paste("Edges:", stats$num_edges))
print(paste("Layers:", stats$num_layers))
print(paste("Directed:", stats$directed))
```

6.16.5 Advanced Examples

Example 1: Layer-by-Layer Analysis

Analyze each layer separately:

```
library(reticulate)
library(igraph)

py3plex <- import("py3plex")
r_interop <- import("py3plex.wrappers.r_interop")

# Create multilayer network (code from Quick Start)
net <- py3plex$multi_layer_network()
# ... add nodes and edges ...

# Get layer names
layers <- r_interop$get_layer_names(net)

# Analyze each layer
for (layer in layers) {
  cat(sprintf("\n=== Layer: %s ===\n", layer))

  # Extract layer-specific graph
  g_layer <- r_interop$to_igraph_for_r(net, layer=layer)

  # Compute centrality for this layer
  deg <- degree(g_layer)
  cat(sprintf("Average degree: %.2f\n", mean(deg)))

  # Community detection
  comm <- cluster_louvain(g_layer)
  cat(sprintf("Communities: %d\n", length(comm)))
}
```

Example 2: Combining with R Statistical Analysis

Use R's statistical tools on network data:

```
library(reticulate)
library(igraph)

py3plex <- import("py3plex")
r_interop <- import("py3plex.wrappers.r_interop")

# Create network
net <- py3plex$multi_layer_network()
```

(continues on next page)

(continued from previous page)

```

# ... build network ...

# Export to igraph
g <- r_interop$to_igraph_for_r(net, mode='union')

# Compute multiple centrality measures
deg <- degree(g)
between <- betweenness(g)
close <- closeness(g)

# Create data frame for analysis
centrality_df <- data.frame(
  node = V(g)$name,
  degree = deg,
  betweenness = between,
  closeness = close
)

# Statistical analysis
print("=== Centrality Correlations ===")
print(cor(centrality_df[, c("degree", "betweenness", "closeness")]))

# Regression analysis
model <- lm(betweenness ~ degree + closeness, data=centrality_df)
print(summary(model))

# Visualization
library(ggplot2)
ggplot(centrality_df, aes(x=degree, y=betweenness)) +
  geom_point() +
  geom_smooth(method="lm") +
  labs(title="Degree vs Betweenness Centrality",
       x="Degree", y="Betweenness")

```

Example 3: Time Series of Multilayer Networks

Analyze temporal evolution:

```

library(reticulate)
library(igraph)

py3plex <- import("py3plex")
r_interop <- import("py3plex.wrappers.r_interop")

# Create networks for different time points
time_points <- c("2020", "2021", "2022")
networks <- list()

for (t in time_points) {
  # Load or create network for this time point
  net <- py3plex$multi_layer_network()
  # ... load data for time t ...

```

(continues on next page)

(continued from previous page)

```

# Convert and store
networks[[t]] <- r_interop$to_igraph_for_r(net, mode='union')
}

# Compare network evolution
metrics <- data.frame(
  time = time_points,
  nodes = sapply(networks, vcount),
  edges = sapply(networks, ecoun),
  density = sapply(networks, graph.density),
  clustering = sapply(networks, transitivity)
)

print(metrics)

# Plot evolution
library(ggplot2)
ggplot(metrics, aes(x=time, y=density, group=1)) +
  geom_line() +
  geom_point() +
  labs(title="Network Density Over Time")

```

6.16.6 Working with CSV Files

Load multilayer network data from CSV:

```

library(reticulate)
library(igraph)

py3plex <- import("py3plex")
r_interop <- import("py3plex.wrappers.r_interop")

# Create network
net <- py3plex$multi_layer_network()

# Load from CSV file
# CSV format: source,source_type,target,target_type,weight
net$load_network("network_data.csv",
  input_type="multiedgelist",
  directed=FALSE)

# Convert and analyze
g <- r_interop$to_igraph_for_r(net, mode='union')

# Get statistics
stats <- r_interop$get_network_stats(net)
print(stats)

```

6.16.7 Integration with MLnet

For users familiar with MLnet (R package for multilayer networks):

```
library(reticulate)
library(igraph)
# library(multinet) # If you have MLnet installed

py3plex <- import("py3plex")
r_interop <- import("py3plex.wrappers.r_interop")

# Create py3plex network
net <- py3plex$multi_layer_network()
# ... build network ...

# Export layer-by-layer for MLnet compatibility
layers <- r_interop$get_layer_names(net)

for (layer in layers) {
  # Get igraph for this layer
  g <- r_interop$to_igraph_for_r(net, layer=layer)

  # Save as GraphML (MLnet compatible)
  write_graph(g, sprintf("layer%s.graphml", layer), format="graphml")
}

# Now you can import these layers into MLnet
# Or continue using igraph functions directly
```

6.16.8 Troubleshooting

Python Module Not Found

If you get `ModuleNotFoundError: No module named 'py3plex'`:

```
library(reticulate)

# Check which Python reticulate is using
py_config()

# Install py3plex in that Python
py_install("git+https://github.com/SkBlaz/py3plex.git")
```

igraph Import Error

If `to_igraph_for_r()` fails with import error:

```
# Install python-igraph
pip install python-igraph
```

Conversion Issues

If conversion produces unexpected results:

```
# Check network statistics first
stats <- r_interop$get_network_stats(net)
print(stats)

# Try different modes
g_union <- r_interop$to_igraph_for_r(net, mode='union')
g_multiplex <- r_interop$to_igraph_for_r(net, mode='multiplex')

print(paste("Union:", vcount(g_union), "nodes"))
print(paste("Multiplex:", vcount(g_multiplex), "nodes"))
```

6.16.9 Performance Tips

For Large Networks

```
# Use union mode for faster processing
g <- r_interop$to_igraph_for_r(net, mode='union')

# For very large networks, export to file instead
# Then use igraph's file reading functions
```

Memory Management

```
# Clear Python objects when done
rm(net)
gc()

# Or restart Python session
py_run_string("import gc; gc.collect()")
```

6.16.10 Next Steps

- *NetworkX Interoperability Guide* - NetworkX integration for Python workflows
- *Network Analysis* - Network analysis methods
- *Community Detection* - Community detection algorithms
- *Visualization Guide* - Visualization options

For Python examples with R code snippets, see [examples/r_interop_example.py](#) in the GitHub repository.

6.17 Algorithm Selection Guide

This guide helps you choose the right algorithms for your multilayer network analysis tasks.

6.17.1 Community Detection

Choose based on your network characteristics and requirements:

Louvain Algorithm

Best for:

- Large networks (10,000+ nodes)
- Fast approximate results
- Non-overlapping communities
- Modularity optimization

Complexity: $O(n \log n)$ where n is the number of nodes

Usage:

```
from py3plex.algorithms.community_detection import community_louvain
communities = community_louvain.best_partition(network.core_network)
```

Pros:

- Very fast on large networks
- Well-established and widely used
- BSD license (commercial-friendly)

Cons:

- Non-deterministic (random initialization)
- Cannot find overlapping communities
- Resolution limit issues

Infomap Algorithm

Best for:

- Flow-based community structure
- Overlapping communities
- Hierarchical community detection
- Information-theoretic optimization

Complexity: $O(m \log n)$ where m is edges, n is nodes

Usage:

```
from py3plex.algorithms.community_detection import community_wrapper
communities = community_wrapper.infomap_communities(
    network.core_network,
    binary_path="/path/to/infomap"
)
```

Pros:

- Can detect overlapping communities
- Flow-based approach (natural for many applications)

- Hierarchical structure

Cons:

- Requires external binary or AGPLv3 code
- Slower than Louvain
- AGPLv3 license (viral copyleft)

Label Propagation

Best for:

- Semi-supervised community detection
- Known seed communities
- Very large sparse networks
- Linear-time approximate results

Complexity: $O(m)$ linear in number of edges

Usage:

```
from py3plex.algorithms.community_detection import label_propagation
communities = label_propagation.propagate(
    network.core_network,
    seed_labels={'node1': 0, 'node2': 1}
)
```

Pros:

- Very fast (linear time)
- Can incorporate prior knowledge
- MIT license

Cons:

- Non-deterministic
- Sensitive to initialization
- May not converge

Multilayer Modularity

Best for:

- True multilayer community detection
- Cross-layer community structure
- Layer coupling analysis
- Research applications

Complexity: $O(n^2)$ for supra-adjacency construction + Louvain complexity

Usage:

```
from py3plex.algorithms.community_detection import multilayer_modularity as mlm
supra_adj = network.get_supra_adjacency_matrix()
communities = mlm.multilayer_louvain(supra_adj)
```

Pros:

- Accounts for multilayer structure
- Implements Mucha et al. (2010) algorithm
- Configurable inter-layer coupling

Cons:

- More computationally expensive
- Requires careful parameter tuning
- May not scale to very large networks

6.17.2 Centrality Measures

Degree Centrality

Best for:

- Quick importance ranking
- Local connectivity measure
- Well-connected node identification

Complexity: $O(n + m)$

Variants:

- Layer-specific degree
- Supra-adjacency degree
- Overlapping degree (sum across layers)

Usage:

```
from py3plex.algorithms.multilayer_algorithms.centrality import MultilayerCentrality
calc = MultilayerCentrality(network)
degrees = calc.overlapping_degree_centrality()
```

Betweenness Centrality

Best for:

- Bridge node identification
- Information flow bottlenecks
- Critical path analysis

Complexity: $O(nm)$ for unweighted, $O(nm + n^2 \log n)$ for weighted

Usage:

```
calc = MultilayerCentrality(network)
betweenness = calc.multilayer_betweenness_centrality()
```

Warning: Computationally expensive for large networks (>1000 nodes)

Closeness Centrality

Best for:

- Broadcasting efficiency
- Average distance to other nodes
- Central position in network

Complexity: $O(nm)$

Usage:

```
closeness = calc.multilayer_closeness centrality()
```

Eigenvector Centrality

Best for:

- Influence measure (connections to important nodes)
- Recursive importance
- Prestige ranking

Complexity: $O(m)$ per iteration, usually converges quickly

Usage:

```
eigenvector = calc.multiplex_eigenvector centrality()
```

PageRank

Best for:

- Directed networks
- Web-like structures
- Random walk centrality

Complexity: $O(m)$ per iteration

Usage:

```
pagerank = calc.pagerank centrality(alpha=0.85)
```

Versatility Centrality

Best for:

- Multilayer-specific importance
- Cross-layer influence
- Multi-context analysis

Complexity: $O(m \times L)$ where L is number of layers

Usage:

```
from py3plex.algorithms.statistics import multilayer_statistics as mls
versatility = mls.versatility centrality(network, centrality_type='degree')
```

6.17.3 New Multiplex Network Metrics

The following metrics extend standard network analysis to multiplex networks, accounting for inter-layer couplings and layer-specific structures.

Multiplex Betweenness Centrality

Best for:

- Identifying bridge nodes across layers
- Finding bottlenecks in multiplex information flow
- Analyzing paths that traverse inter-layer couplings

Complexity: $O(nm)$ where n is node-layer pairs, m is total edges

Usage:

```
from py3plex.algorithms.statistics import multilayer_statistics as mls
betweenness = mls.multiplex_betweenness_centrality(network, normalized=True)
top_nodes = sorted(betweenness.items(), key=lambda x: x[1], reverse=True)[:5]
```

Reference: De Domenico et al. (2015), “Structural reducibility of multilayer networks”

Multiplex Closeness Centrality

Best for:

- Finding nodes that can quickly reach all other node-layers
- Broadcasting efficiency across layers
- Central position analysis in multiplex networks

Complexity: $O(nm)$ where n is node-layer pairs

Usage:

```
closeness = mls.multiplex_closeness_centrality(network, normalized=True)
central_nodes = {k: v for k, v in closeness.items() if v > 0.5}
```

Reference: De Domenico et al. (2015), “Structural reducibility of multilayer networks”

Community Participation Metrics

Best for:

- Measuring node diversity across communities
- Identifying nodes that bridge different communities
- Analyzing cross-community connections

Complexity: $O(k)$ where k is node degree

Usage:

```
# Participation coefficient ( $P = 1 - (k/k)^2$ )
pc = mls.community_participation_coefficient(network, communities, 'Alice')

# Participation entropy ( $H = -(k/k)\log(k/k)$ )
entropy = mls.community_participation_entropy(network, communities, 'Alice')
```

Reference: Guimerà & Amaral (2005), “Functional cartography of complex metabolic networks”

Layer Redundancy Analysis

Best for:

- Measuring edge overlap between layers
- Identifying redundant vs. unique connections
- Understanding layer complementarity

Complexity: $O(m)$ where m is edges

Usage:

```
# Redundancy coefficient ( $R = |E \cap E|/|E|$ )
redundancy = mls.layer_redundancy_coefficient(network, 'social', 'work')

# Count unique and redundant edges
unique, redundant = mls.unique_redundant_edges(network, 'social', 'work')
```

Reference: Nicosia & Latora (2015), “Measuring and modeling correlations in multiplex networks”

Multiplex Rich-Club Coefficient

Best for:

- Analyzing whether high-degree nodes connect preferentially
- Network core structure identification
- Hierarchy analysis in multiplex networks

Complexity: $O(m)$ where m is edges

Usage:

```
rich_club = mls.multiplex_rich_club_coefficient(network, k=10, normalized=True)
```

Reference: Extended from Alstott et al. (2014) to multiplex networks

Robustness and Percolation Analysis

Best for:

- Network resilience assessment
- Critical layer identification
- Cascade failure analysis

Complexity: $O(n \times t)$ where t is number of trials

Usage:

```
# Estimate percolation threshold
threshold = mls.percolation_threshold(
    network,
    removal_strategy='degree', # or 'random', 'betweenness'
    trials=20
)

# Simulate layer removal
resilience = mls.targeted_layer_removal(
    network,
    'social',
    return_resilience=True
)
```

Reference: Buldyrev et al. (2010), “Catastrophic cascade of failures in interdependent networks”

Direct Modularity Computation

Best for:

- Evaluating community partition quality
- Comparing different community structures
- Direct modularity calculation without detection

Complexity: $O(m)$ where m is edges

Usage:

```
# Compute multislice modularity score
Q = mls.compute_modularity_score(
    network,
    communities,
    gamma=1.0, # resolution parameter
    omega=1.0 # inter-layer coupling
)
```

Reference: Mucha et al. (2010), Science 328, 876-878

Complete Example

See `examples/network_analysis/example_new_multiplex_metrics.py` for a comprehensive demonstration of all new metrics.

6.17.4 Network Statistics

Basic Statistics

Fast operations (suitable for large networks):

- Node count: $O(n)$
- Edge count: $O(m)$
- Degree distribution: $O(n + m)$
- Density: $O(1)$ if counts are known

```
network.basic_stats()
layers = network.get_layers()
num_nodes = len(network.get_nodes())
```

Multilayer Statistics

Layer density - $O(m_\alpha)$ per layer:

```
from py3plex.algorithms.statistics import multilayer_statistics as mls
density = mls.layer_density(network, 'layer1')
```

Inter-layer correlation - $O(n)$:

```
correlation = mls.inter_layer_degree_correlation(network, 'layer1', 'layer2')
```

Edge overlap - $O(m)$:

```
overlap = mls.edge_overlap(network, 'layer1', 'layer2')
```

Versatility centrality - $O(m \times L)$:

```
versatility = mls.versatility_centrality(network, centrality_type='betweenness')
```

Path-based Measures

Warning: These are computationally expensive for large networks

- **Diameter** - $O(nm)$ or $O(n^2 \log n)$
- **Average shortest path** - $O(nm)$ or $O(n^2 \log n)$
- **Betweenness centrality** - $O(nm)$ or $O(nm + n^2 \log n)$

Recommendation: Limit to networks with <5,000 nodes for interactive analysis

6.17.5 Node Embeddings

Node2Vec

Best for:

- Feature learning for downstream tasks
- Link prediction
- Node classification
- Similarity computation

Complexity: $O(r \times l \times |V|)$ where r is walks per node, l is walk length

Parameters:

- **dimensions:** Embedding dimensions (default: 128)
- **walk_length:** Length of random walks (default: 80)
- **num_walks:** Number of walks per node (default: 10)
- **p:** Return parameter (default: 1.0)
- **q:** In-out parameter (default: 1.0)

Usage:

```
from py3plex.wrappers import node2vec_embedding
embeddings = node2vec_embedding.generate_embeddings(
    network.core_network,
    dimensions=128,
    walk_length=80,
    num_walks=10,
    p=1.0,
    q=1.0
)
```

Parameter tuning:

- Low p (<1): Encourages backtracking (local exploration)
- High p (>1): Discourages backtracking
- Low q (<1): Encourages outward exploration (BFS-like)
- High q (>1): Encourages staying local (DFS-like)

DeepWalk

Best for:

- Same as Node2Vec but simpler (uniform random walks)
- Faster than Node2Vec

Complexity: $O(r \times l \times |V|)$

Usage:

```
# DeepWalk is Node2Vec with p=1, q=1
embeddings = node2vec_embedding.generate_embeddings(
    network.core_network,
    p=1.0,
    q=1.0
)
```

6.17.6 Visualization

Diagonal Projection Plot

Best for:

- Large multilayer networks (>1000 nodes)
- Clear layer separation
- Publication-ready figures

Scalability: Handles 10,000+ nodes

Usage:

```
from py3plex.visualization.multilayer import draw_multilayer_default
draw_multilayer_default([network], display=True, layout_style='diagonal')
```


Force-Directed Layout

Best for:

- Small to medium networks (<1000 nodes)
- Interactive exploration
- Natural-looking layouts

Scalability: Practical up to ~5,000 nodes

Usage:

```
from py3plex.visualization.multilayer import hairball_plot
hairball_plot(network.core_network, layout_algorithm='force')
```

Matrix Visualization

Best for:

- Pattern recognition
- Very large networks
- Quantitative analysis

Scalability: Handles 100,000+ nodes (limited by memory)

Usage:

```
import matplotlib.pyplot as plt
adj = network.get_supra_adjacency_matrix()
plt.imshow(adj.toarray() if hasattr(adj, 'toarray') else adj, cmap='viridis')
plt.colorbar()
plt.show()
```

6.17.7 Performance Guidelines

Network Size Categories

Small (< 1,000 nodes):

- All algorithms are fast
- Can use expensive path-based measures
- Interactive visualization works well

Medium (1,000 - 10,000 nodes):

- Use Louvain over Infomap
- Avoid full betweenness centrality
- Use diagonal projection for visualization

Large (10,000 - 100,000 nodes):

- Use degree-based measures
- Louvain or label propagation only
- Matrix visualizations or sampling

Very Large (> 100,000 nodes):

- Degree, PageRank only
- Label propagation for communities
- Sparse matrix operations essential

Memory Considerations

Supra-adjacency matrix:

- Sparse representation: $O(m)$ memory
- Dense representation: $O(n^2)$ memory (avoid for large networks)

Always use sparse matrices for networks with >1,000 nodes

```
# Ensure sparse representation
supra_adj = network.get_supra_adjacency_matrix(sparse=True)
```

6.17.8 Algorithm Comparison Table

Algorithm	Complexity	Scalability	License	Best Use Case
Louvain	$O(n \log n)$	Large	BSD	Fast community detection
Infomap	$O(m \log n)$	Medium	AGPLv3	Flow-based communities
Label Prop	$O(m)$	Very Large	MIT	Semi-supervised, very fast
Degree Centrality	$O(n+m)$	Very Large	MIT	Quick importance
Betweenness	$O(nm)$	Small	MIT	Critical paths
PageRank	$O(m)$ per iter	Large	MIT	Directed networks
Node2Vec	$O(r \times l \times n)$	Medium	MIT	Feature learning

6.17.9 Citation Guidelines

When using specific algorithms in publications, cite the original papers:

Louvain:

Blondel, V. D., et al. (2008). Fast unfolding of communities in large networks. *Journal of Statistical Mechanics*.

Infomap:

Rosvall, M., & Bergstrom, C. T. (2008). Maps of random walks on complex networks reveal community structure. *PNAS*, 105(4), 1118-1123.

Node2Vec:

Grover, A., & Leskovec, J. (2016). node2vec: Scalable feature learning for networks. *KDD*, 855-864.

Multilayer Modularity:

Mucha, P. J., et al. (2010). Community structure in time-dependent, multiscale, and multiplex networks. *Science*, 328(5980), 876-878.

For complete references with DOIs, please refer to the algorithm citations provided in the individual algorithm documentation or the published paper.

6.17.10 Next Steps

- *Community Detection Tutorial* - Community detection tutorial
- *Multilayer Network Centrality Measures* - Centrality measures tutorial
- *Performance and Scalability* - Performance optimization guide

6.18 Algorithm Roadmap

This page provides a **comprehensive overview** of all algorithms implemented in py3plex, organized by category and showing relationships between different methods.

Purpose: Help you quickly find the right algorithm for your multilayer network analysis task.

6.18.1 Algorithm Categories

The algorithms in py3plex are organized into these main categories:

1. *Community Detection Algorithms* - Finding community structure
2. *Centrality Measures* - Computing node importance
3. *Network Statistics* - Network-level metrics
4. *Node Ranking & Classification* - Ranking and classification methods
5. *Random Walks & Embeddings* - Random walk-based methods
6. *Visualization Algorithms* - Layout and rendering
7. *Benchmark & Utilities* - Testing and evaluation

6.18.2 Community Detection Algorithms

Module: `py3plex.algorithms.community_detection`

These algorithms identify groups of densely connected nodes (communities) in multilayer networks.

Louvain Algorithm

Path: `community_detection.community_louvain`

Functions:

- `best_partition(graph, partition, weight, resolution, randomize, random_state)` - Main entry point
- `generate_dendrogram(graph, partition, weight, resolution, randomize, random_state)` - Hierarchical communities
- `modularity(partition, graph, weight)` - Calculate modularity score
- `partition_at_level(dendrogram, level)` - Extract partition at specific level
- `induced_graph(partition, graph, weight)` - Create quotient graph

Best for: Fast community detection on large single-layer networks

Complexity: $O(n \log n)$

Related algorithms:

- *Multilayer Louvain* (multilayer extension)
- *Leiden Algorithm* (improved Louvain)

Infomap Algorithm

Path: `community_detection.community_wrapper`

Functions:

- `infomap_communities(network_input, binary_path, arguments)` - Main entry point

- `run_infomap(network, binary_path, arguments)` - Execute Infomap binary
- `parse_infomap(outfile)` - Parse Infomap output

Best for: Flow-based community detection, hierarchical structure

Complexity: $O(m \log n)$ where m is edges, n is nodes

Related algorithms:

- *Louvain Algorithm* (faster alternative)

Multilayer Louvain

Path: `community_detection.multilayer_modularity`

Functions:

- `louvain_multilayer(supra_adj, omega, resolution, seed)` - Multilayer Louvain
- `multilayer_modularity(supra_adj, partition, omega)` - Calculate multilayer modularity
- `build_supra_modularity_matrix(network, omega)` - Build modularity matrix

Best for: Community detection across multiple layers with inter-layer coupling

Complexity: $O(n^2 L)$ where L is number of layers

Algorithm details: Implements Mucha et al. (2010) multilayer modularity optimization

Related algorithms:

- *Louvain Algorithm* (single-layer version)
- *Leiden Algorithm* (alternative optimizer)

Leiden Algorithm

Path: `community_detection.leiden_multilayer`

Functions:

- `leiden_multilayer(supra_adj, omega, resolution, n_iterations, seed)` - Main entry point

Data structures:

- `LeidenResult` - Holds partition results with modularity score

Best for: More stable community detection than Louvain

Complexity: $O(n \log n)$ with better quality guarantees

Related algorithms:

- *Louvain Algorithm* (predecessor)
- *Multilayer Louvain* (multilayer extension)

NoRC Algorithm

Path: `community_detection.NoRC` and `community_detection.community_wrapper`

Functions:

- `NoRC_communities(network, alpha)` - Wrapper function
- `NoRC_communities_main(matrix, alpha)` - Core implementation
- `page_rank_kernel(index_row)` - PageRank computation

- `create_tree(centers)` - Build community hierarchy

Best for: Networks with overlapping communities

Related algorithms:

- *Community Ranking* (related approach)

Community Measures

Path: `community_detection.community_measures`

Functions:

- `modularity(network, partition, weight)` - Calculate modularity
- `size_distribution(network_partition)` - Community size distribution
- `number_of_communities(network_partition)` - Count communities

Purpose: Evaluate quality of community partitions

Related algorithms: All community detection algorithms above

Community Ranking

Path: `community_detection.community_ranking`

Functions:

- `page_rank_kernel(index_row)` - PageRank-based ranking
- `create_tree(centers)` - Build ranking tree
- `return_infomap_communities(network)` - Get Infomap communities

Best for: Ranking nodes within detected communities

Related algorithms:

- *Node Ranking* (general ranking)

Multilayer Benchmarks

Path: `community_detection.multilayer_benchmark`

Functions:

- `generate_multilayer_lfr(n_nodes, n_layers, avg_degree, ...)` - LFR benchmark
- `generate_coupled_er_multilayer(n_nodes, n_layers, p_intra, p_inter)` - ER benchmark
- `generate_sbm_multilayer(sizes, p_matrix, n_layers, ...)` - SBM benchmark

Purpose: Generate synthetic multilayer networks with known community structure for testing

Related algorithms: All community detection algorithms

6.18.3 Centrality Measures

Module: `py3plex.algorithms.multilayer_algorithms.centrality`

These algorithms measure the importance or influence of nodes in multilayer networks.

MultilayerCentrality Class

Path: `multilayer_algorithms.centrality.MultilayerCentrality`

Main class for computing various centrality measures.

Basic Centrality Measures

Methods:

- `overlapping_degree_centrality()` - Sum of degrees across all layers
- `layer_specific_degree_centrality(layer)` - Degree within specific layer
- `multilayer_degree_centrality()` - Supra-adjacency degree
- `average_degree_centrality()` - Average degree across layers

Best for: Quick importance ranking, well-connected node identification

Complexity: $O(n + m)$ where n is nodes, m is edges

Related measures:

- *Versatility Centrality* (cross-layer activity)

Eigenvector-Based Measures

Methods:

- `multiplex_eigenvector_centrality(max_iter, tol)` - Eigenvector centrality
- `pagerank_centrality(alpha, max_iter, tol)` - PageRank
- `katz_centrality(alpha, beta, max_iter, tol)` - Katz centrality
- `communicability_centrality(beta)` - Communicability centrality

Best for: Influence measure, recursive importance, prestige ranking

Complexity: $O(m)$ per iteration, typically converges quickly

Related measures:

- *Hubs and Authorities (HITS)* (directed networks)

Path-Based Measures

Methods:

- `multilayer_betweenness_centrality()` - Betweenness across layers
- `multilayer_closeness_centrality()` - Closeness across layers

Best for: Bridge identification, broadcasting efficiency

Complexity: $O(nm)$ for unweighted, $O(nm + n^2 \log n)$ for weighted

Warning: Expensive for large networks (>1000 nodes)

Related measures:

- *Utility Functions* (batch computation)

Hubs and Authorities (HITS)

Path: `node_ranking.node_ranking.hubs_and_authorities`

Best for: Ranking nodes in directed networks (authority = important destination, hub = important source)

Related measures:

- PageRank in *Centrality Measures*

Utility Functions

Functions:

- `compute_all_centralities(network, include_path_based, include_advanced)` - Compute multiple measures

Purpose: Batch computation of multiple centrality measures

Versatility Centrality

Path: `algorithms.statistics.multilayer_statistics.versatility_centrality`

Function:

- `versatility_centrality(network, centrality_type)` - Cross-layer versatility measure

Best for: Measuring how uniformly a node's importance is distributed across layers

Complexity: $O(m \times L)$ where L is number of layers

Related measures:

- All centrality measures in *Centrality Measures*

Supra Matrix Function Centralities

Path: `multilayer_algorithms.supra_matrix_function_centrality`

Functions:

- `communicability_centrality(supra_matrix, beta)` - Matrix exponential based
- `katz_centrality(supra_matrix, alpha, beta, max_iter, tol)` - Resolvent based

Best for: Advanced centrality using matrix functions on supra-adjacency

Complexity: Depends on matrix operations, typically $O(n^2)$ or $O(n^3)$

Related measures:

- *Centrality Measures* (standard measures)

MultiXRank

Path: `multilayer_algorithms.multixrank`

Functions:

- `multixrank_from_py3plex_networks(networks, config)` - MultiXRank algorithm

Best for: Ranking in multiplex heterogeneous networks with bipartite layers

Algorithm details: Extends PageRank to multiplex/heterogeneous networks

Related algorithms:

- PageRank in *Centrality Measures*

6.18.4 Network Statistics

Module: `py3plex.algorithms.statistics`

These algorithms compute various statistical properties of multilayer networks.

Multilayer Statistics

Path: `statistics.multilayer_statistics`

Layer-level measures:

- `layer_density(network, layer)` - Density of specific layer
- `edge_overlap(network, layer_i, layer_j)` - Edge overlap between layers
- `inter_layer_coupling_strength(network, layer_i, layer_j)` - Coupling strength
- `inter_layer_degree_correlation(network, layer_i, layer_j)` - Degree correlation
- `inter_layer_assortativity(network, layer_i, layer_j)` - Assortativity between layers
- `layer_similarity(network, layer_i, layer_j, method)` - Layer similarity

Node-level measures:

- `node_activity(network, node)` - Activity across layers
- `degree_vector(network, node, weighted)` - Degree vector across layers
- `multilayer_clustering_coefficient(network, node)` - Multilayer clustering

Network-level measures:

- `interdependence(network, sample_size)` - Network interdependence
- `supra_laplacian_spectrum(network, k)` - Spectral properties
- `algebraic_connectivity(network)` - Second smallest Laplacian eigenvalue

Best for: Understanding multilayer network structure and properties

Complexity: Varies by measure, typically $O(n)$ to $O(n^2)$

Related algorithms:

- *Multilayer Statistics (Detailed)* (simpler measures)
- *Topology Analysis* (topological properties)

Multilayer Statistics (Detailed)

See the full section above for complete details on multilayer statistics functions.

Basic Statistics

Path: `statistics.basic_statistics`

Functions:

- `identify_n_hubs(network, n, metric)` - Find top-n hub nodes
- `core_network_statistics(network)` - Comprehensive network statistics

Best for: Quick network overview, hub identification

Complexity: $O(n + m)$ for most measures

Related algorithms:

- *Multilayer Statistics (Detailed)* (advanced measures)

Topology Analysis

Path: `statistics.topology`

Functions for analyzing topological properties of networks.

Purpose: Study structural properties like degree distributions, connected components, etc.

Related algorithms:

- *Multilayer Statistics (Detailed)*

Correlation Networks

Path: `statistics.correlation_networks`

Functions:

- `default_correlation_to_network(correlation_matrix, threshold)` - Convert correlation matrix to network
- `pick_threshold(matrix)` - Automatically select threshold

Best for: Creating networks from correlation/similarity matrices

Related algorithms:

- *Network Statistics* (analysis after construction)

Enrichment Analysis

Path: `statistics.enrichment_modules`

Functions:

- `compute_enrichment(term_dataset, annotation_database, ...)` - General enrichment
- `fet_enrichment_generic(...)` - Fisher's exact test enrichment
- `fet_enrichment_terms(...)` - Term enrichment
- `fet_enrichment_uniprot(...)` - UniProt annotation enrichment
- `calculate_pval(term, alternative)` - p-value calculation
- `multiple_test_correction(input_dataset)` - FDR correction

Best for: Finding over-represented terms/annotations in network communities

Related algorithms:

- *Community Detection Algorithms* (provides communities for enrichment)

Statistical Comparison

Path: `statistics.stats_comparison`

Functions: Various statistical tests for comparing networks or algorithms

Related modules:

- `statistics.bayesian_tests` - Bayesian statistical tests
- `statistics.bayesian_distances` - Bayesian distance measures
- `statistics.critical_distances` - Critical difference diagrams

Best for: Comparing performance of different algorithms or network properties

Power Law Analysis

Path: `statistics.powerlaw`

Functions for testing and fitting power law distributions in networks.

Best for: Analyzing degree distributions, checking for scale-free properties

Related algorithms:

- *Multilayer Statistics (Detailed)* (degree distributions)

6.18.5 Node Ranking & Classification

Module: `py3plex.algorithms.node_ranking` and `py3plex.algorithms.network_classification`

Node Ranking

Path: `node_ranking.node_ranking`

Functions:

- `sparse_page_rank(graph, alpha, personalization, max_iter, tol)` - Sparse PageRank
- `hubs_and_authorities(graph)` - HITS algorithm
- `hub_matrix(graph)` - Hub matrix computation
- `authority_matrix(graph)` - Authority matrix computation
- `stochastic_normalization(matrix)` - Normalize for random walks
- `stochastic_normalization_hin(matrix)` - Normalize for heterogeneous networks

Best for: Ranking nodes by importance in directed networks

Complexity: $O(m)$ per iteration for PageRank, $O(nm)$ for HITS

Related algorithms:

- PageRank in *Centrality Measures*
- *Label Propagation* (classification)

Label Propagation

Path: `network_classification.label_propagation`

Functions:

- `label_propagation(adjacency, labels, alpha, max_iter, tol, normalization)` - Main algorithm
- `validate_label_propagation(...)` - Validation function
- `label_propagation_normalization(matrix)` - Normalization
- `normalize_initial_matrix_freq(mat)` - Frequency normalization
- `normalize_amplify_freq(mat)` - Amplified normalization
- `normalize_exp(mat)` - Exponential normalization

Best for: Semi-supervised node classification

Complexity: $O(m \times i)$ where i is number of iterations

Related algorithms:

- *Personalized PageRank (PPR)* (personalized version)
- Label propagation in *Community Detection Algorithms*

Personalized PageRank (PPR)

Path: `network_classification.PPR`

Functions:

- `construct_PPR_matrix(graph_matrix, parallel)` - Build PPR matrix
- `construct_PPR_matrix_targets(graph_matrix, targets, parallel)` - PPR for specific targets
- `validate_ppr(...)` - Validation function

Best for: Local network exploration, personalized node ranking

Complexity: $O(n^2)$ for dense computation, can be approximated

Related algorithms:

- *Label Propagation* (similar propagation mechanism)
- PageRank in *Node Ranking*

6.18.6 Random Walks & Embeddings

Module: `py3plex.algorithms.general` and `py3plex.wrappers`

Random Walk Primitives

Path: `general.walkers`

Functions:

- `basic_random_walk(graph, start_node, walk_length)` - Simple random walk
- `node2vec_walk(graph, start_node, walk_length, p, q)` - Biased random walk
- `generate_walks(graph, num_walks, walk_length, strategy, ...)` - Generate multiple walks
- `general_random_walk(G, start_node, iterations, teleportation_prob)` - Random walk with restart
- `layer_specific_random_walk(network, start_node, start_layer, walk_length, ...)` - Multi-layer walk

Best for: Foundation for embedding algorithms, network exploration

Complexity: $O(L)$ per walk where L is walk length

Related algorithms:

- *Node2Vec Embedding* (uses these walks)
- *DeepWalk Embedding* (uses these walks)

Node2Vec Embedding

Path: `wrappers.node2vec_embedding`

Functions:

- `generate_embeddings(graph, dimensions, walk_length, num_walks, p, q, ...)` - Generate embeddings
- `train_node2vec(graph, ...)` - Training wrapper

Best for: Feature learning, link prediction, node classification

Complexity: $O(r \times l \times V)$ where r is walks per node, l is walk length, V is number of vertices

Parameters:

- p : Return parameter (controls backtracking)
- q : In-out parameter (controls exploration vs exploitation)

Related algorithms:

- *DeepWalk Embedding* (special case with $p=1, q=1$)
- *Random Walk Primitives* (underlying walks)

DeepWalk Embedding

Implementation: Node2Vec with $p=1, q=1$

Best for: Simpler alternative to Node2Vec with uniform random walks

Related algorithms:

- *Node2Vec Embedding* (generalization)

Entanglement Analysis

Path: `multilayer_algorithms.entanglement`

Functions:

- `compute_entanglement_analysis(network)` - Full entanglement analysis
- `build_occurrence_matrix(network)` - Build node-layer occurrence matrix
- `compute_blocks(c_matrix)` - Compute block structure
- `compute_entanglement(block_matrix)` - Calculate entanglement metrics

Best for: Analyzing node-layer participation patterns

Purpose: Measure how nodes are entangled across layers

Related algorithms:

- *Versatility Centrality* (similar cross-layer analysis)

6.18.7 Visualization Algorithms

Module: `py3plex.visualization`

Layout Algorithms

Path: `visualization.layout_algorithms`

Available layouts:

- Force-directed layouts (Spring, Fruchterman-Reingold)
- Circular layout
- Spectral layout
- Hierarchical layout
- Diagonal projection (multilayer-specific)

Best for: Positioning nodes for visualization

Related algorithms:

- *Multilayer Visualization* (uses these layouts)

Multilayer Visualization

Path: `visualization.multilayer`

Main functions:

- `draw_multilayer_default(networks, ...)` - Default multilayer drawing
- `hairball_plot(network, layout_algorithm)` - Single-layer projection
- Diagonal projection plotting
- Layer-separated visualization

Best for: Visualizing multilayer network structure

Scalability:

- Diagonal projection: 10,000+ nodes
- Force-directed: <5,000 nodes
- Matrix view: 100,000+ nodes

Related algorithms:

- *Layout Algorithms* (positioning)

Drawing Machinery

Path: `visualization.drawing_machinery`

Low-level drawing functions and utilities.

Purpose: Support functions for rendering networks

Color Schemes

Path: `visualization.colors`

Functions for generating color schemes for communities, layers, node types.

Related algorithms:

- *Community Detection Algorithms* (uses colors)
- *Multilayer Visualization* (uses colors)

Embedding Visualization

Path: `visualization.embedding_visualization`

Functions for visualizing node embeddings in 2D/3D space.

Best for: Visualizing results of embedding algorithms

Related algorithms:

- *Node2Vec Embedding*
- *DeepWalk Embedding*

6.18.8 Benchmark & Utilities

Network Classification Benchmark

Path: `general.benchmark_classification`

Functions:

- `evaluate_oracle_F1(probs, Y_real)` - Evaluate classification performance

Best for: Evaluating node classification algorithms

Related algorithms:

- *Label Propagation*
- *Node2Vec Embedding*

Benchmark Visualizations

Path: `visualization.benchmark_visualizations`

Functions for visualizing algorithm comparison results.

Related algorithms:

- All algorithms (for benchmarking)

6.18.9 Algorithm Selection Guide

This section helps you choose the right algorithm category based on your task.

By Task Type

Community Detection:

→ See *Community Detection Algorithms*

Node Importance:

→ See *Centrality Measures*

Node Classification:

→ See *Label Propagation* or *Node2Vec Embedding*

Network Comparison:

→ See *Network Statistics*

Feature Learning:

→ See *Node2Vec Embedding*

Visualization:

→ See *Visualization Algorithms*

By Network Size**Small (<1,000 nodes):**

- All algorithms work well
- Can use expensive path-based measures
- All visualization methods

Medium (1,000-10,000 nodes):

- Use Louvain over Infomap
- Avoid full betweenness centrality
- Use diagonal projection for visualization

Large (10,000-100,000 nodes):

- Degree-based measures only
- Louvain or label propagation
- Matrix visualizations

Very Large (>100,000 nodes):

- Degree, PageRank only
- Label propagation for communities
- Sparse matrix operations essential

By Network Type**Single-layer networks:**

- Standard algorithms: Louvain, PageRank, betweenness
- See *Community Detection Algorithms* and *Centrality Measures*

Multiplex/multilayer networks:

- Multilayer-specific: *Multilayer Louvain*, *Versatility Centrality*
- Layer statistics: *Multilayer Statistics (Detailed)*
- Cross-layer measures

Directed networks:

- PageRank, HITS algorithm
- See *Node Ranking*

Weighted networks:

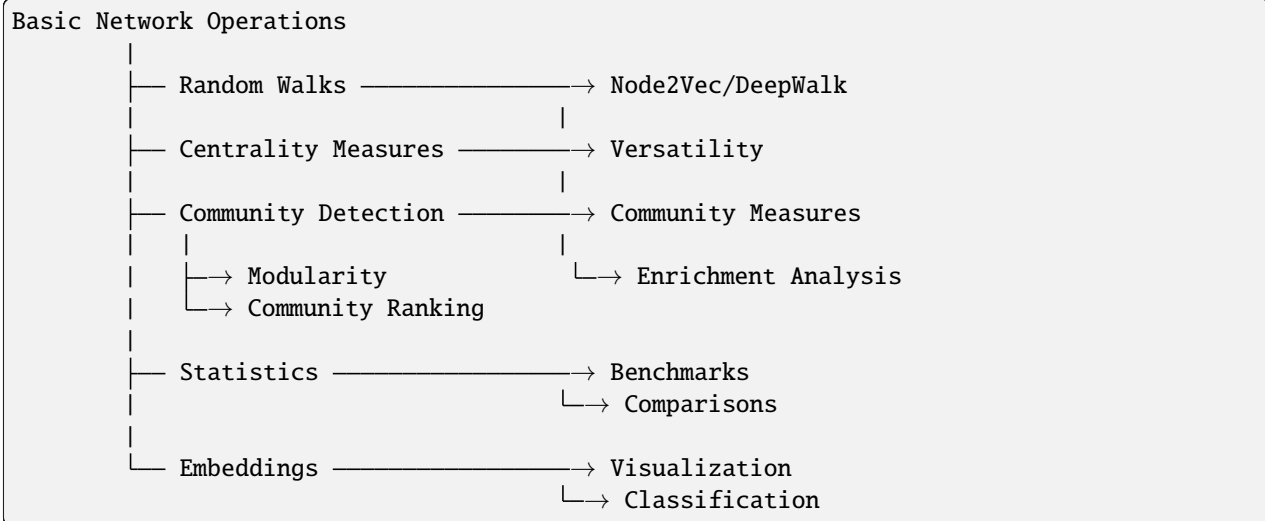
- All algorithms support weights
- Specify weight parameter

Temporal networks:

- Layer-based representation
- Time-respecting random walks: *Random Walk Primitives*

6.18.10 Algorithm Dependencies

This diagram shows how algorithms depend on each other:



6.18.11 Common Algorithm Workflows

Community Detection Workflow

```

# 1. Detect communities
from py3plex.algorithms.community_detection import louvain_multilayer
communities = louvain_multilayer(network.get_supra_adjacency_matrix())

# 2. Evaluate quality
from py3plex.algorithms.community_detection import multilayer_modularity
quality = multilayer_modularity(network.get_supra_adjacency_matrix(), communities)

# 3. Analyze communities
from py3plex.algorithms.statistics import multilayer_statistics as mls
from py3plex.algorithms.statistics.enrichment_modules import compute_enrichment

# 4. Visualize
from py3plex.visualization.multilayer import draw_multilayer_default
draw_multilayer_default([network], communities=communities)

```


Centrality Analysis Workflow

```
# 1. Compute centralities
from py3plex.algorithms.multilayer_algorithms.centrality import MultilayerCentrality
calc = MultilayerCentrality(network)

# 2. Multiple measures
degree = calc.overlapping_degree_centrality()
pagerank = calc.pagerank_centrality()
betweenness = calc.multilayer_betweenness_centrality()

# 3. Cross-layer analysis
from py3plex.algorithms.statistics import multilayer_statistics as mls
versatility = mls.versatility_centrality(network, centrality_type='degree')

# 4. Identify hubs
from py3plex.algorithms.statistics.basic_statistics import identify_n_hubs
hubs = identify_n_hubs(network, n=10, metric='degree')
```

Embedding Workflow

```
# 1. Generate walks
from py3plex.algorithms.general.walkers import generate_walks
walks = generate_walks(network.core_network, num_walks=10,
                       walk_length=80, strategy='node2vec')

# 2. Learn embeddings
from py3plex.wrappers.node2vec_embedding import generate_embeddings
embeddings = generate_embeddings(network.core_network,
                                dimensions=128, p=1.0, q=1.0)

# 3. Use for classification
from py3plex.algorithms.network_classification.label_propagation import label_propagation
# Or use embeddings directly with sklearn
```

Statistical Analysis Workflow

```
# 1. Layer-level statistics
from py3plex.algorithms.statistics import multilayer_statistics as mls
density = mls.layer_density(network, 'layer1')
overlap = mls.edge_overlap(network, 'layer1', 'layer2')
correlation = mls.inter_layer_degree_correlation(network, 'layer1', 'layer2')

# 2. Node-level statistics
for node in important_nodes:
    activity = mls.node_activity(network, node)
    degree_vec = mls.degree_vector(network, node)

# 3. Network-level statistics
interdep = mls.interdependence(network)
connectivity = mls.algebraic_connectivity(network)
```

6.18.12 Further Reading

For detailed usage examples and parameter descriptions:

- *Algorithm Selection Guide* - Algorithm selection guide with complexity analysis
- *Community Detection Tutorial* - Community detection examples
- *Multilayer Network Centrality Measures* - Centrality computation examples
- *Performance and Scalability Best Practices* - Performance optimization tips
- *API Documentation* - Complete API reference

6.18.13 Citation Information

When using specific algorithm families, please cite the original papers:

Louvain & Multilayer Modularity:

Blondel et al. (2008), Mucha et al. (2010)

Node2Vec:

Grover & Leskovec (2016)

Infomap:

Rosvall & Bergstrom (2008)

See *Citation and References* for complete citation information and BibTeX entries.

6.19 Plugin System

The py3plex plugin system allows developers to extend the library with custom algorithms without modifying the core codebase. This makes it easy to contribute new centrality measures, community detection algorithms, layout methods, and network metrics.

6.19.1 Overview

The plugin system provides:

- **Four plugin types** for different algorithm categories
- **Decorator-based registration** for easy plugin creation
- **Automatic discovery** from external directories
- **Safe module loading** with conflict detection
- **Validation** before plugin instantiation

6.19.2 Plugin Types

CentralityPlugin

For custom node centrality measures.

Required Methods:

- `compute(network, **kwargs) -> Dict[str, float]`: Returns node-to-score mapping

Optional Properties:

- `supports_weighted`: bool: Whether algorithm supports weighted networks (default: False)
- `supports_directed`: bool: Whether algorithm supports directed networks (default: False)

- `supports_multilayer`: bool: Whether algorithm supports multilayer networks (default: False)

Example:

```
from py3plex.plugins import CentralityPlugin, PluginRegistry

@PluginRegistry.register('centrality', 'my_centrality')
class MyCustomCentrality(CentralityPlugin):
    @property
    def name(self):
        return 'my_centrality'

    @property
    def description(self):
        return 'My custom centrality measure'

    @property
    def supports_weighted(self):
        return True

    def compute(self, network, normalized=False, **kwargs):
        """Compute centrality scores for all nodes."""
        centrality = {}

        # Your algorithm here
        G = network.core_network
        for node in G.nodes():
            centrality[node] = compute_score(node)

        return centrality
```

CommunityPlugin

For community detection algorithms.

Required Methods:

- `detect(network, **kwargs) -> Dict[str, int]`: Returns node-to-community-id mapping

Optional Properties:

- `supports_weighted`: bool: Whether algorithm supports weighted networks (default: False)
- `supports_overlapping`: bool: Whether algorithm finds overlapping communities (default: False)
- `supports_hierarchical`: bool: Whether algorithm produces hierarchy (default: False)

Example:

```
from py3plex.plugins import CommunityPlugin, PluginRegistry

@PluginRegistry.register('community', 'my_detector')
class MyDetector(CommunityPlugin):
    @property
    def name(self):
        return 'my_detector'
```

(continues on next page)

(continued from previous page)

```

@property
def supports_weighted(self):
    return True

def detect(self, network, resolution=1.0, **kwargs):
    """Detect communities in the network."""
    communities = {}

    # Your algorithm here
    G = network.core_network
    # ... community detection logic ...

    return communities

```

LayoutPlugin

For network layout algorithms.

Required Methods:

- `compute_layout(network, dimensions=2, **kwargs) -> Dict[str, tuple]:` Returns node-to-position mapping

Optional Properties:

- `supports_3d: bool:` Whether layout supports 3D positions (default: False)
- `supports_weighted: bool:` Whether layout considers edge weights (default: False)

Example:

```

from py3plex.plugins import LayoutPlugin, PluginRegistry

@PluginRegistry.register('layout', 'my_layout')
class MyLayout(LayoutPlugin):
    @property
    def name(self):
        return 'my_layout'

    @property
    def supports_3d(self):
        return True

    def compute_layout(self, network, dimensions=2, **kwargs):
        """Compute layout positions."""
        import math

        positions = {}
        G = network.core_network
        nodes = list(G.nodes())

        # Your layout algorithm here
        for i, node in enumerate(nodes):
            if dimensions == 2:
                positions[node] = (x, y)

```

(continues on next page)

(continued from previous page)

```

        elif dimensions == 3:
            positions[node] = (x, y, z)

    return positions

```

MetricPlugin

For custom network metrics.

Required Methods:

- `compute(network, **kwargs) -> Dict[str, Any]`: Returns metric names to values

Optional Properties:

- `metric_type`: str: 'global', 'local', or 'both' (default: 'global')

Example:

```

from py3plex.plugins import MetricPlugin, PluginRegistry

@PluginRegistry.register('metric', 'my_metric')
class MyMetric(MetricPlugin):
    @property
    def name(self):
        return 'my_metric'

    @property
    def metric_type(self):
        return 'global'

    def compute(self, network, **kwargs):
        """Compute network metrics."""
        G = network.core_network

        # Your metric computation
        value = compute_metric(G)

        return {'metric_name': value}

```

6.19.3 Using Plugins

Registering Plugins

Method 1: Decorator (Recommended)

```

from py3plex.plugins import PluginRegistry, CentralityPlugin

@PluginRegistry.register('centrality', 'my_plugin')
class MyPlugin(CentralityPlugin):
    # Implementation
    pass

```

Method 2: Direct Registration

```
from py3plex.plugins import PluginRegistry

registry = PluginRegistry()
registry.register_plugin('centrality', 'my_plugin', MyPluginClass)
```

Getting and Using Plugins

```
from py3plex import PluginRegistry, multi_layer_network

# Get plugin instance
registry = PluginRegistry()
plugin = registry.get('centrality', 'my_plugin')

# Use plugin
network = multi_layer_network()
# ... add nodes and edges ...
results = plugin.compute(network)
```

Listing Available Plugins

```
from py3plex import PluginRegistry

registry = PluginRegistry()

# List all plugins
all_plugins = registry.list_plugins()
print(all_plugins)
# {'centrality': ['my_plugin', ...], 'community': [...], ...}

# List specific type
centralities = registry.list_plugins('centrality')
print(centralities)
# {'centrality': ['my_plugin', 'example_degree', ...]}
```

Getting Plugin Information

```
from py3plex import PluginRegistry

registry = PluginRegistry()
info = registry.get_plugin_info('centrality', 'my_plugin')
print(info)
# {
#     'name': 'my_plugin',
#     'version': '1.0.0',
#     'author': 'Your Name',
#     'description': 'Description here',
#     'type': 'centrality'
# }
```

6.19.4 Plugin Discovery

Automatic Plugin Loading

Py3plex can automatically discover plugins from external directories.

Default Plugin Directory

By default, plugins are loaded from `~/.py3plex/plugins/`

Setting Custom Plugin Directory

Via environment variable:

```
export PY3PLEX_PLUGIN_DIR=/path/to/my/plugins
```

Programmatically:

```
from py3plex.plugins import discover_plugins

# Discover plugins from custom directory
count = discover_plugins('/path/to/my/plugins')
print(f"Loaded {count} plugins")
```

Creating Discoverable Plugins

1. Create a Python file in your plugin directory:

```
mkdir -p ~/.py3plex/plugins
```

2. Write your plugin (e.g., `~/.py3plex/plugins/my_plugin.py`):

```
from py3plex.plugins import CentralityPlugin, PluginRegistry

@PluginRegistry.register('centrality', 'my_auto_plugin')
class MyAutoPlugin(CentralityPlugin):
    @property
    def name(self):
        return 'my_auto_plugin'

    def compute(self, network, **kwargs):
        # Implementation
        return {}
```

3. The plugin will be automatically discovered when you import py3plex:

```
from py3plex.plugins import discover_plugins
discover_plugins() # Loads from default directory
```

6.19.5 Best Practices

Input Validation

Always validate input in your plugins:

```
def compute(self, network, **kwargs):
    if not hasattr(network, 'core_network'):
```

(continues on next page)

(continued from previous page)

```
raise ValueError("Network must be a py3plex multi_layer_network object")

# Your implementation
```

Documentation

Provide clear docstrings:

```
def compute(self, network, threshold=0.5, **kwargs):
    """
    Compute custom centrality.

    Args:
        network: A py3plex multi_layer_network object
        threshold: Minimum score threshold (default: 0.5)
        **kwargs: Additional parameters

    Returns:
        Dictionary mapping node IDs to centrality scores

    Raises:
        ValueError: If network is invalid
    """
```

Dependency Checking

Check for optional dependencies:

```
def validate(self):
    """Check if plugin can run."""
    try:
        import optional_library
        return True
    except ImportError:
        return False
```

Error Handling

Handle errors gracefully:

```
def compute(self, network, **kwargs):
    try:
        # Your algorithm
        return results
    except Exception as e:
        raise ValueError(f"Failed to compute centrality: {e}")
```

6.19.6 Example Plugins

Built-in Examples

Py3plex includes example plugins demonstrating all plugin types. Import them to see available examples:


```
import py3plex.plugins.examples
from py3plex import PluginRegistry

registry = PluginRegistry()
plugins = registry.list_plugins()

# Example plugins available:
# - centrality/example_degree: Simple degree centrality
# - community/example_simple: Simple community detection
# - layout/example_circular: Circular layout
# - metric/example_density: Network density
```

Complete Example

See `examples/workflows/example_plugin_usage.py` for a complete working example demonstrating:

- Creating custom centrality plugins
- Creating custom community detection plugins
- Using built-in example plugins
- Plugin discovery and registration
- Getting plugin information

```
from py3plex import multi_layer_network, PluginRegistry
from py3plex.plugins import CentralityPlugin

# Define custom plugin
@PluginRegistry.register('centrality', 'closeness_simple')
class SimpleCloseness(CentralityPlugin):
    @property
    def name(self):
        return 'closeness_simple'

    def compute(self, network, **kwargs):
        import networkx as nx
        G = network.core_network

        centrality = {}
        for node in G.nodes():
            lengths = nx.single_source_shortest_path_length(G, node)
            lengths.pop(node, None)

            if lengths:
                avg_length = sum(lengths.values()) / len(lengths)
                centrality[node] = 1.0 / avg_length if avg_length > 0 else 0.0
            else:
                centrality[node] = 0.0

        return centrality

# Create network
net = multi_layer_network()
```

(continues on next page)

(continued from previous page)

```

net.add_nodes([
    {'source': 'A', 'type': 'layer1'},
    {'source': 'B', 'type': 'layer1'},
    {'source': 'C', 'type': 'layer1'},
])
net.add_edges([
    {'source': 'A', 'target': 'B', 'source_type': 'layer1', 'target_type': 'layer1'},
    {'source': 'B', 'target': 'C', 'source_type': 'layer1', 'target_type': 'layer1'},
])

# Use plugin
registry = PluginRegistry()
plugin = registry.get('centrality', 'closeness_simple')
scores = plugin.compute(net)

for node, score in sorted(scores.items()):
    print(f"Node {node}: {score:.4f}")

```

6.19.7 Contributing Plugins

To contribute plugins to the py3plex ecosystem:

1. **Create your plugin** following this guide
2. **Test your plugin** thoroughly
3. **Package your plugin** as a Python package
4. **Publish to PyPI** or share on GitHub
5. **Submit a PR** to add your plugin to the official plugin registry

Plugin Package Structure

```

my-py3plex-plugin/
├── setup.py
├── README
├── my_plugin/
│   ├── __init__.py
│   └── plugin.py
├── tests/
│   └── test_plugin.py

```

Example setup.py

```

from setuptools import setup, find_packages

setup(
    name='py3plex-my-plugin',
    version='1.0.0',
    packages=find_packages(),
    install_requires=[
        'py3plex>=0.96',
    ],

```

(continues on next page)

(continued from previous page)

```

entry_points={
    'py3plex.plugins': [
        'my_plugin = my_plugin.plugin',
    ],
},
)

```

6.19.8 API Reference

PluginRegistry

class PluginRegistry

Central registry for managing py3plex plugins.

register_plugin(*plugin_type: str, plugin_name: str, plugin_class: Type[BasePlugin]*) → None

Register a plugin class.

Parameters

- **plugin_type** – Type of plugin ('centrality', 'community', 'layout', 'metric')
- **plugin_name** – Unique name for this plugin
- **plugin_class** – Plugin class (must inherit from appropriate base class)

Raises

- **ValueError** – If plugin_type is invalid or plugin already registered
- **TypeError** – If plugin_class doesn't inherit from BasePlugin

get(*plugin_type: str, plugin_name: str*) → BasePlugin

Get an instance of a registered plugin.

Parameters

- **plugin_type** – Type of plugin
- **plugin_name** – Name of the plugin

Returns

An instance of the requested plugin

Raises

- **KeyError** – If plugin is not found
- **RuntimeError** – If plugin validation fails

list_plugins(*plugin_type: str | None = None*) → Dict[str, List[str]]

List all registered plugins.

Parameters

plugin_type – Optional plugin type to filter by

Returns

Dictionary mapping plugin types to lists of plugin names

get_plugin_info(*plugin_type: str, plugin_name: str*) → Dict[str, Any]

Get metadata about a plugin without instantiating it.

Parameters

- **plugin_type** – Type of plugin
- **plugin_name** – Name of the plugin

Returns

Dictionary with plugin metadata

unregister(*plugin_type: str, plugin_name: str*) → None

Unregister a plugin.

Parameters

- **plugin_type** – Type of plugin
- **plugin_name** – Name of the plugin

Raises

KeyError – If plugin is not found

classmethod **reset**() → None

Reset the registry to initial state. Primarily useful for testing.

discover_plugins

discover_plugins(*plugin_dir: str | None = None*) → int

Discover and load plugins from a directory.

Parameters

plugin_dir – Path to directory containing plugin modules. If None, looks in PY3PLEX_PLUGIN_DIR environment variable or defaults to ~/.py3plex/plugins/

Returns

Number of plugins discovered and loaded

6.19.9 Support

- **Documentation:** <https://py3plex.readthedocs.io>
- **Issues:** <https://github.com/SkBlaz/py3plex/issues>
- **Discussions:** <https://github.com/SkBlaz/py3plex/discussions>

6.19.10 See Also

- *Contributing to py3plex* - General contribution guidelines
- *Algorithm Selection Guide* - Guide to built-in algorithms
- *Development Guide* - Development setup and practices

6.20 Ollivier-Ricci Curvature and Ricci Flow

6.20.1 Overview

py3plex now provides support for **Ollivier-Ricci curvature** and **Ricci flow** on multilayer networks. These geometric measures reveal important structural properties of networks, including community boundaries, bottlenecks, and hierarchical organization.

What is Ollivier-Ricci Curvature?

Ollivier-Ricci curvature is a discrete analog of Ricci curvature from differential geometry, adapted for graphs. For each edge in a network, the Ollivier-Ricci curvature measures the local connectivity between the neighborhoods of the edge's endpoints:

- **Positive curvature** indicates well-connected regions (dense communities)
- **Negative curvature** indicates bottlenecks or community boundaries
- **Near-zero curvature** indicates transitional regions

What is Ricci Flow?

Ricci flow is a geometric process that iteratively adjusts edge weights based on their curvature. The flow amplifies structural features:

- Edges with **negative curvature** (community boundaries) have their weights **reduced**
- Edges with **positive curvature** (within communities) have their weights **increased**

After Ricci flow, community detection becomes more effective, and the network's geometric structure is more pronounced.

Why Use These for Multilayer Networks?

Multilayer networks represent complex systems with multiple types of interactions. Ollivier-Ricci curvature and flow help:

- Identify cross-layer community boundaries
- Reveal hierarchical organization across layers
- Detect structural bottlenecks in coupled systems
- Enhance community detection on aggregated or supra-graph representations

6.20.2 Installation

To use Ollivier-Ricci curvature and Ricci flow, you must install the optional **GraphRicciCurvature** library:

```
pip install GraphRicciCurvature
```

If GraphRicciCurvature is not installed, py3plex will raise a clear error message with installation instructions when you attempt to use these features.

6.20.3 Basic Usage

Computing Curvature on the Aggregated Network

The simplest use case is computing curvature on the **aggregated (core) network**, which combines all layers:

```
from py3plex.core import multinet

# Load or create a multilayer network
net = multinet.multi_layer_network().load_network(
    "path/to/multilayer_network.txt",
    input_type="multiedgelist",
    directed=False,
)
```

(continues on next page)

(continued from previous page)

```

# Compute Ollivier-Ricci curvature on the aggregated core network
result = net.compute_ollivier_ricci(
    mode="core",
    alpha=0.5,
    curvature_attr="ricciCurvature",
    inplace=True,
)

# Access the graph with curvatures
G_core = result["core"]

# Inspect edge curvatures
for u, v, data in G_core.edges(data=True):
    if "ricciCurvature" in data:
        print(f"Edge {u}-{v}: curvature = {data['ricciCurvature']:.4f}")

```

After this, edges in `net.core_network` (if `inplace=True`) will have a `ricciCurvature` attribute containing the computed curvature value.

Applying Ricci Flow

Once curvature is computed, you can apply Ricci flow to adjust edge weights:

```

# Run Ricci flow on the core network
result_flow = net.compute_ollivier_ricci_flow(
    mode="core",
    alpha=0.5,
    iterations=10,
    method="OTD",
    inplace=True,
)

# Now edge weights in net.core_network reflect the Ricci flow metric
G_flow = result_flow["core"]

# Edge weights are updated to reflect Ricci flow
for u, v, data in G_flow.edges(data=True):
    print(f"Edge {u}-{v}: weight after flow = {data.get('weight', 1.0):.4f}")

```

After Ricci flow, edges with negative curvature (community boundaries) will have reduced weights, making them easier to identify or remove during community detection.

Per-Layer Curvature

To compute curvature separately for each layer:

```

# Compute curvature separately for each layer
layer_results = net.compute_ollivier_ricci(
    mode="layers",
    layers=None, # None means all layers
    alpha=0.5,
    inplace=True,

```

(continues on next page)

(continued from previous page)

```
)

# layer_results is a dictionary mapping layer identifiers to NetworkX graphs
for layer_id, G_layer in layer_results.items():
    print(f"\nLayer: {layer_id}")
    for u, v, data in G_layer.edges(data=True):
        if "ricciCurvature" in data:
            print(f"  Edge {u}-{v}: curvature = {data['ricciCurvature']:.4f}")
```

You can also specify a subset of layers:

```
# Compute curvature only for specific layers
layer_results = net.compute_ollivier_ricci(
    mode="layers",
    layers=["layer1", "layer2"], # Only these layers
    alpha=0.5,
    inplace=False,
)
```

Supra-Graph Curvature

The **supra-graph** representation includes both intra-layer edges (within each layer) and inter-layer edges (coupling nodes across layers). This is the most comprehensive view of the multilayer network:

```
# Compute curvature on the supra-graph representation
supra_result = net.compute_ollivier_ricci(
    mode="supra",
    alpha=0.5,
    interlayer_weight=1.0, # Weight for inter-layer coupling edges
    inplace=False,
)

G_supra = supra_result["supra"]

# Identify inter-layer edges with negative curvature (weak coupling)
for u, v, data in G_supra.edges(data=True):
    if isinstance(u, tuple) and isinstance(v, tuple):
        if u[0] == v[0] and u[1] != v[1]: # Same node, different layers
            curvature = data.get("ricciCurvature", 0)
            print(f"Inter-layer edge {u}-{v}: curvature = {curvature:.4f}")
```

Similarly, you can apply Ricci flow to the supra-graph:

```
# Apply Ricci flow on the supra-graph
supra_flow_result = net.compute_ollivier_ricci_flow(
    mode="supra",
    alpha=0.5,
    iterations=5,
    inplace=False,
)

G_supra_flow = supra_flow_result["supra"]
```

6.20.4 API Reference

compute_ollivier_ricci

Compute Ollivier-Ricci curvature on the multilayer network.

Signature:

```
def compute_ollivier_ricci(
    self,
    mode: str = "core",
    layers: Optional[List[Any]] = None,
    alpha: float = 0.5,
    weight_attr: str = "weight",
    curvature_attr: str = "ricciCurvature",
    verbose: str = "ERROR",
    backend_kwargs: Optional[Dict[str, Any]] = None,
    inplace: bool = True,
    interlayer_weight: float = 1.0,
) -> Dict[str, Any]:
```

Parameters:

- **mode**
[str, default "core"] Scope of the computation. Options:
 - "core": Compute curvature on the aggregated (core) network
 - "layers": Compute curvature separately for each layer
 - "supra": Compute curvature on the supra-graph (includes inter-layer edges)
- **layers**
[list or None, default None] List of layer identifiers to process when mode="layers". If None, all layers are processed. Ignored for other modes.
- **alpha**
[float, default 0.5] Ollivier-Ricci parameter in [0, 1] controlling the mass distribution. alpha=0 uses pure neighbors, alpha=0.5 is standard, and alpha=1 uses uniform distribution over all nodes.
- **weight_attr**
[str, default "weight"] Name of the edge attribute containing edge weights.
- **curvature_attr**
[str, default "ricciCurvature"] Name of the edge attribute to store computed curvature values.
- **verbose**
[str, default "ERROR"] Verbosity level for GraphRicciCurvature. Options: "INFO", "DEBUG", "ERROR".
- **backend_kwargs**
[dict or None, default None] Optional dictionary of additional keyword arguments to pass to the OllivierRicci constructor from GraphRicciCurvature.
- **inplace**
[bool, default True] If True, update internal graphs (e.g., self.core_network). If False, return new graphs without modifying the network.
- **interlayer_weight**
[float, default 1.0] Weight for inter-layer coupling edges (only used when mode="supra").

Returns:

- **dict**

Dictionary mapping scope identifiers to NetworkX graphs with computed curvatures:

- mode="core": {"core": graph_with_curvature}
- mode="layers": {layer_id: graph_with_curvature, ...}
- mode="supra": {"supra": supra_graph_with_curvature}

Raises:

- **RicciBackendNotAvailable**

If GraphRicciCurvature is not installed.

- **ValueError**

If mode is invalid or layers contains invalid identifiers.

compute_ollivier_ricci_flow

Compute Ollivier-Ricci flow on the multilayer network.

Signature:

```
def compute_ollivier_ricci_flow(
    self,
    mode: str = "core",
    layers: Optional[List[Any]] = None,
    alpha: float = 0.5,
    iterations: int = 10,
    method: str = "OTD",
    weight_attr: str = "weight",
    curvature_attr: str = "ricciCurvature",
    verbose: str = "ERROR",
    backend_kwargs: Optional[Dict[str, Any]] = None,
    inplace: bool = True,
    interlayer_weight: float = 1.0,
) -> Dict[str, Any]:
```

Parameters:

- **mode**
[str, default "core"] Scope of the computation. Same options as compute_ollivier_ricci.
- **layers**
[list or None, default None] List of layer identifiers to process when mode="layers".
- **alpha**
[float, default 0.5] Ollivier-Ricci parameter in [0, 1].
- **iterations**
[int, default 10] Number of Ricci flow iterations to perform. More iterations lead to stronger effects but take longer to compute.
- **method**
[str, default "OTD"] Method for Ricci flow computation. Options:
 - "OTD": Optimal Transport Distance (recommended)
 - "ATD": Average Transport Distance

- **weight_attr**
[str, default “weight”] Name of the edge attribute containing edge weights. After Ricci flow, these weights are updated to reflect the flow metric.
- **curvature_attr**
[str, default “ricciCurvature”] Name of the edge attribute to store curvature values.
- **verbose**
[str, default “ERROR”] Verbosity level.
- **backend_kwargs**
[dict or None, default None] Additional keyword arguments for the OllivierRicci constructor.
- **inplace**
[bool, default True] If True, update internal graphs. If False, return new graphs.
- **interlayer_weight**
[float, default 1.0] Weight for inter-layer coupling edges (only for mode=“supra”).

Returns:

- **dict**
Dictionary mapping scope identifiers to NetworkX graphs with Ricci flow applied.

Raises:

- **RicciBackendNotAvailable**
If GraphRicciCurvature is not installed.
- **ValueError**
If mode is invalid or layers contains invalid identifiers.

6.20.5 Advanced Examples

Identifying Community Boundaries

Edges with negative curvature often correspond to community boundaries:

```
# Compute curvature
result = net.compute_ollivier_ricci(mode="core", inplace=True)
G = result["core"]

# Find edges with negative curvature (potential community boundaries)
boundary_edges = [
    (u, v, data["ricciCurvature"])
    for u, v, data in G.edges(data=True)
    if data.get("ricciCurvature", 0) < -0.1 # Threshold
]

print(f"Found {len(boundary_edges)} potential community boundary edges")
for u, v, curvature in sorted(boundary_edges, key=lambda x: x[2]):
    print(f" {u} -- {v}: {curvature:.4f}")
```

Enhanced Community Detection with Ricci Flow

Ricci flow can improve community detection by reducing weights on boundary edges:

```
from py3plex.algorithms.community_detection import community_wrapper
```

(continues on next page)

(continued from previous page)

```

# Apply Ricci flow
result = net.compute_ollivier_ricci_flow(
    mode="core",
    iterations=20,
    inplace=True
)
G_flow = result["core"]

# Run community detection on the flow-adjusted network
communities = community_wrapper.best_partition(G_flow)

# Analyze communities
from collections import defaultdict
comm_sizes = defaultdict(int)
for node, comm_id in communities.items():
    comm_sizes[comm_id] += 1

print(f"Detected {len(comm_sizes)} communities")
for comm_id, size in sorted(comm_sizes.items()):
    print(f" Community {comm_id}: {size} nodes")

```

Cross-Layer Analysis with Supra-Graph

Use the supra-graph to analyze relationships between layers:

```

# Compute curvature on supra-graph
supra_result = net.compute_ollivier_ricci(
    mode="supra",
    alpha=0.5,
    interlayer_weight=1.0,
    inplace=False
)
G_supra = supra_result["supra"]

# Analyze intra-layer vs. inter-layer curvatures
intra_curvatures = []
inter_curvatures = []

for u, v, data in G_supra.edges(data=True):
    if isinstance(u, tuple) and isinstance(v, tuple):
        curvature = data.get("ricciCurvature", 0)
        if u[1] == v[1]: # Same layer (intra-layer edge)
            intra_curvatures.append(curvature)
        elif u[0] == v[0]: # Same node, different layers (inter-layer edge)
            inter_curvatures.append(curvature)

import numpy as np
print(f"Intra-layer curvature: mean={np.mean(intra_curvatures):.4f}, "
      f"std={np.std(intra_curvatures):.4f}")
print(f"Inter-layer curvature: mean={np.mean(inter_curvatures):.4f}, "
      f"std={np.std(inter_curvatures):.4f}")

```

6.20.6 Performance Considerations

Computing Ollivier-Ricci curvature can be computationally expensive for large networks, as it involves optimal transport calculations for each edge. Here are some tips:

- **Start small:** Test on small networks or subgraphs first
- **Use mode="layers":** If you only need per-layer analysis, this avoids processing the full core network
- **Adjust alpha:** Lower values of alpha (e.g., 0.3) can be faster to compute
- **Reduce iterations:** For Ricci flow, start with fewer iterations (e.g., 5-10) and increase as needed
- **Parallel computation:** GraphRicciCurvature supports parallel computation via `backend_kwargs={"proc": 4}` (use number of CPU cores)

Example with parallel computation:

```
# Use 4 CPU cores for parallel computation
result = net.compute_ollivier_ricci(
    mode="core",
    alpha=0.5,
    backend_kwargs={"proc": 4}
)
```

6.20.7 References

- Ni, C. C., Lin, Y. Y., Gao, J., Gu, X. D., & Saucan, E. (2015). Ricci curvature of the Internet topology. *2015 IEEE Conference on Computer Communications (INFOCOM)*, 2758-2766.
- Ni, C. C., Lin, Y. Y., Luo, F., & Gao, J. (2019). Community detection on networks with Ricci flow. *Scientific Reports*, 9(1), 9984.
- Ollivier, Y. (2009). Ricci curvature of Markov chains on metric spaces. *Journal of Functional Analysis*, 256(3), 810-864.
- GraphRicciCurvature library: <https://github.com/saibalmars/GraphRicciCurvature>

6.20.8 See Also

- *Ricci-Flow-Based Visualization* - Ricci-flow-based visualization utilities
- *Community Detection* - Community detection algorithms
- *Core Concepts and Architecture* - Understanding multilayer networks
- *Supra-Adjacency Matrices* - Supra-adjacency matrix representation

6.21 Ricci-Flow-Based Visualization

6.21.1 Overview

py3plex provides advanced visualization capabilities leveraging **Ricci flow** to create informative layouts for multilayer networks. Ricci flow transforms edge weights based on the geometric curvature of the network, which helps reveal community structure, bottlenecks, and hierarchical organization.

This page documents the Ricci-flow-based visualization utilities that build upon the *Ollivier-Ricci Curvature* and *Ricci Flow* functionality.

What is Ricci-Flow-Based Visualization?

Traditional network layouts (spring, spectral, etc.) rely on heuristics to position nodes. Ricci-flow-based layouts use a more principled geometric approach:

1. **Ricci flow** adjusts edge weights based on their curvature
2. These adjusted weights reflect the “geometric distance” between nodes
3. Layouts computed from these weights naturally emphasize communities and structural features

The result is visualizations where:

- **Communities** appear as tight, well-separated clusters
- **Bottleneck edges** (negative curvature) are visually emphasized
- **Hierarchical structure** becomes more apparent

Three Visualization Styles

py3plex provides three complementary visualization styles for multilayer networks:

1. **Core (Aggregated) Visualization:** Shows the combined structure across all layers
2. **Per-Layer Visualization:** Shows each layer individually with shared or independent coordinates
3. **Supra-Graph Visualization:** Shows the full multilayer structure including inter-layer connections

6.21.2 Installation Requirements

Ricci-flow-based visualizations require the optional **GraphRicciCurvature** library:

```
pip install GraphRicciCurvature
```

If GraphRicciCurvature is not installed, py3plex will raise a clear error message with installation instructions.

6.21.3 Basic Usage

Core (Aggregated) Visualization

The simplest use case is visualizing the aggregated network after Ricci flow:

```
from py3plex.core import multinet
import matplotlib.pyplot as plt

# Load or create a multilayer network
net = multinet.multi_layer_network().load_network(
    "path/to/multilayer_network.txt",
    input_type="multiedgelist",
    directed=False,
)

# Visualize core network using Ricci flow
# (Ricci flow is computed automatically if not already done)
fig, ax, positions = net.visualize_ricci_core(
    alpha=0.5,          # Ollivier-Ricci parameter
    iterations=10,      # Number of flow iterations
    layout_type="mds",  # Layout algorithm
    dim=2,              # 2D layout
```

(continues on next page)

(continued from previous page)

```
)  
  
plt.show()
```

This creates a 2D visualization where:

- **Node colors** indicate layer participation (how many layers the node appears in)
- **Edge colors** indicate curvature (red = negative/bottleneck, blue = positive/community)
- **Edge widths** reflect post-flow weights (thicker = stronger connection)

Per-Layer Visualization

To compare individual layers side-by-side:

```
# Visualize layers in a grid with shared coordinate system  
fig, layer_positions = net.visualize_ricci_layers(  
    layers=None,          # None means all layers  
    alpha=0.5,  
    iterations=10,  
    layout_type="mds",  
    share_layout=True,    # Use shared coordinates for comparison  
    arrangement="grid",   # Display in grid of subplots  
)  
  
plt.show()
```

With `share_layout=True`, nodes appear at the same positions across all layer plots, making it easy to see how edge structure differs between layers.

To use independent layouts for each layer:

```
# Independent layouts - each layer optimized separately  
fig, layer_positions = net.visualize_ricci_layers(  
    share_layout=False,  
    arrangement="grid",  
)  
  
plt.show()
```

Supra-Graph Visualization

To visualize the full multilayer structure including inter-layer coupling:

```
# 2D supra-graph visualization  
fig, ax, positions = net.visualize_ricci_supra(  
    alpha=0.5,  
    iterations=10,  
    layout_type="mds",  
    dim=2,  
    node_color_by="layer",    # Color nodes by layer  
    edge_color_by="curvature", # Color edges by curvature  
)
```

(continues on next page)

(continued from previous page)

```
plt.show()
```

For 3D visualization with layer separation:

```
# 3D supra-graph with layers separated along z-axis
fig, ax, positions = net.visualize_ricci_supra(
    dim=3,
    layer_separation=1.0, # Separate layers by this distance
    layout_type="spring",
)

plt.show()
```

6.21.4 Advanced Examples

Customizing Node and Edge Appearance

You can customize various visual properties:

```
fig, ax, positions = net.visualize_ricci_core(
    alpha=0.5,
    iterations=15,

    # Node appearance
    node_color_by="curvature", # Options: "layer_overlap", "degree", "curvature"
    node_size=150,            # Fixed size, or "degree" for degree-based sizing
    cmap_nodes="plasma",      # Colormap for nodes

    # Edge appearance
    edge_color_by="weight",   # Options: "curvature", "weight"
    edge_width_scale=3.0,     # Scale factor for edge widths
    cmap_edges="coolwarm",    # Colormap for edges

    # Display options
    show_colorbar=True,
    figsize=(12, 10),
)
```

Using Different Layout Algorithms

Three layout algorithms are available:

```
# MDS (Multidimensional Scaling) - recommended for distance-based layouts
# Good for: Preserving global structure, community visualization
fig1, ax1, pos1 = net.visualize_ricci_core(
    layout_type="mds",
    use_geodesic_distances=True, # Use shortest path distances
)

# Spring (Force-Directed) - good for emphasizing local structure
# Good for: Small to medium networks, emphasizing edge weights
```

(continues on next page)

(continued from previous page)

```

fig2, ax2, pos2 = net.visualize_ricci_core(
    layout_type="spring",
    iterations=15,
    k=0.3, # Spring constant (passed to nx.spring_layout)
)

# Spectral - based on graph Laplacian eigenvectors
# Good for: Fast computation, balanced layouts
fig3, ax3, pos3 = net.visualize_ricci_core(
    layout_type="spectral",
)

```

Pre-Computing Ricci Flow

For more control, you can compute Ricci flow separately:

```

# Step 1: Compute Ricci flow explicitly
result = net.compute_ollivier_ricci_flow(
    mode="core",
    alpha=0.5,
    iterations=20,
    method="OTD", # Optimal Transport Distance
    inplace=True,
)

# Step 2: Visualize with pre-computed flow
fig, ax, positions = net.visualize_ricci_core(
    compute_if_missing=False, # Don't recompute
    layout_type="mds",
)

```

This approach is useful when:

- You want to experiment with different iterations counts
- You need to analyze curvature values before visualizing
- You're visualizing the same network multiple times

Comparing Multiple Layers

Visualize specific layers for comparison:

```

# Compare two specific layers
fig, layer_pos = net.visualize_ricci_layers(
    layers=["social", "biological"], # Only these layers
    share_layout=True,
    arrangement="grid",
    iterations=15,
)

# Access positions for individual layers
social_positions = layer_pos["social"]
biological_positions = layer_pos["biological"]

```


Analyzing Supra-Graph Structure

Visualize and analyze the supra-graph:

```
import numpy as np

# Visualize supra-graph
fig, ax, positions = net.visualize_ricci_supra(
    alpha=0.5,
    iterations=10,
    interlayer_alpha=0.2, # Transparency for inter-layer edges
)

# Extract node positions by layer
layer1_nodes = {node: pos for node, pos in positions.items()
                 if isinstance(node, tuple) and node[1] == "layer1"}

layer2_nodes = {node: pos for node, pos in positions.items()
                 if isinstance(node, tuple) and node[1] == "layer2"}

print(f"Layer 1 has {len(layer1_nodes)} nodes")
print(f"Layer 2 has {len(layer2_nodes)} nodes")
```

6.21.5 API Reference

visualize_ricci_core()

Visualize the aggregated core network using Ricci-flow-based layout.

Signature:

```
def visualize_ricci_core(
    self,
    alpha: float = 0.5,
    iterations: int = 10,
    layout_type: str = "mds",
    dim: int = 2,
    curvature_attr: str = "ricciCurvature",
    weight_attr: str = "weight",
    node_color_by: str = "layer_overlap",
    edge_color_by: str = "curvature",
    node_size: Union[int, str] = 100,
    edge_width_scale: float = 2.0,
    figsize: Tuple[float, float] = (10, 8),
    ax: Optional[plt.Axes] = None,
    cmap_nodes: str = "viridis",
    cmap_edges: str = "RdBu_r",
    show_colorbar: bool = True,
    compute_if_missing: bool = True,
    **kwargs
) -> Tuple[plt.Figure, plt.Axes, Dict[Any, np.ndarray]]:
```

Key Parameters:

- alpha: Ollivier-Ricci parameter (0 to 1). Default: 0.5.

- `iterations`: Number of Ricci flow iterations. Default: 10.
- `layout_type`: Layout algorithm - “mds”, “spring”, or “spectral”. Default: “mds”.
- `dim`: Layout dimensionality (2 or 3). Default: 2.
- `node_color_by`: Node coloring scheme - “layer_overlap”, “degree”, or “curvature”. Default: “layer_overlap”.
- `edge_color_by`: Edge coloring scheme - “curvature” or “weight”. Default: “curvature”.
- `compute_if_missing`: Automatically compute Ricci flow if not done. Default: True.

Returns:

Tuple of (figure, axes, positions_dict) where positions_dict maps nodes to coordinate arrays.

visualize_ricci_layers()

Visualize individual layers using Ricci-flow-based layouts.

Signature:

```
def visualize_ricci_layers(
    self,
    layers: Optional[List[Any]] = None,
    alpha: float = 0.5,
    iterations: int = 10,
    layout_type: str = "mds",
    dim: int = 2,
    arrangement: str = "grid",
    share_layout: bool = True,
    figsize: Tuple[float, float] = (12, 8),
    node_size: int = 100,
    edge_width_scale: float = 2.0,
    compute_if_missing: bool = True,
    **kwargs
) -> Tuple[plt.Figure, Dict[Any, Dict[Any, np.ndarray]]]:
```

Key Parameters:

- `layers`: List of layer IDs to visualize. If None, uses all layers. Default: None.
- `share_layout`: Use shared coordinates across layers. Default: True.
- `arrangement`: How to arrange layers - “grid” or “stacked”. Default: “grid”.

Returns:

Tuple of (figure, layer_positions_dict) where layer_positions_dict maps layer IDs to position dictionaries.

visualize_ricci_supra()

Visualize the full supra-graph using Ricci-flow-based layout.

Signature:

```
def visualize_ricci_supra(
    self,
    alpha: float = 0.5,
    iterations: int = 10,
    layout_type: str = "mds",
```

(continues on next page)

(continued from previous page)

```

dim: int = 2,
layer_separation: Optional[float] = None,
node_color_by: str = "layer",
edge_color_by: str = "curvature",
figsize: Tuple[float, float] = (12, 10),
interlayer_alpha: float = 0.3,
compute_if_missing: bool = True,
**kwargs
) -> Tuple[plt.Figure, plt.Axes, Dict[Any, np.ndarray]]:

```

Key Parameters:

- `layer_separation`: If not `None` and `dim=3`, separates layers along z-axis. Default: `None`.
- `node_color_by`: Node coloring - “layer” or “curvature”. Default: “layer”.
- `interlayer_alpha`: Transparency for inter-layer edges. Default: 0.3.

Returns:

Tuple of (figure, axes, positions_dict) for the supra-graph.

6.21.6 Best Practices

Choosing Parameters

Alpha (Ollivier-Ricci parameter):

- `alpha=0.5`: Standard setting, good starting point
- `alpha < 0.5`: Emphasizes local neighborhood structure
- `alpha > 0.5`: Emphasizes global connectivity
- `alpha=0`: Pure neighbor-based measure (faster but less smooth)

Iterations:

- `iterations=5-10`: Quick preview
- `iterations=10-20`: Standard analysis
- `iterations=30-50`: High-quality results for publication
- More iterations = stronger geometric effects but longer computation

Layout Type:

- `layout_type="mds"`: Best for preserving distances, good for community structure
- `layout_type="spring"`: Best for small networks, emphasizes edge weights
- `layout_type="spectral"`: Fastest, good for balanced layouts

Performance Considerations

Ricci flow and visualization can be computationally expensive for large networks:

- **Start small**: Test on subgraphs or smaller networks first
- **Reduce iterations**: Use 5-10 iterations for initial exploration
- **Use mode=”layers”**: If you only need per-layer analysis, avoid computing the full core

- **Parallel computation:** Pass `backend_kwargs={"proc": 4}` to use multiple CPU cores

Example with parallel computation:

```
# Use 4 CPU cores for Ricci flow computation
fig, ax, pos = net.visualize_ricci_core(
    alpha=0.5,
    iterations=15,
    backend_kwargs={"proc": 4}
)
```

Interpretation Guide

Node Colors (when `node_color_by="layer_overlap"`):

- Brighter colors = node appears in more layers
- Darker colors = node appears in fewer layers
- Helps identify cross-layer hubs vs. layer-specific nodes

Edge Colors (when `edge_color_by="curvature"`):

- **Blue:** Positive curvature → edges within communities
- **Red:** Negative curvature → edges between communities (bottlenecks)
- **White/neutral:** Near-zero curvature → transitional regions

Edge Widths:

- Thicker edges = higher weight after Ricci flow
- Thinner edges = lower weight after Ricci flow
- Width reflects “geometric strength” of the connection

6.21.7 Common Workflows

Workflow 1: Community Detection

Use Ricci flow visualization to identify and validate communities:

```
# Step 1: Visualize with Ricci flow
fig, ax, pos = net.visualize_ricci_core(
    iterations=20,
    edge_color_by="curvature",
)

# Step 2: Identify bottleneck edges (red edges)
# These likely separate communities

# Step 3: Run community detection on flow-enhanced network
net.compute_ollivier_ricci_flow(mode="core", iterations=20, inplace=True)

# Use your preferred community detection algorithm
# Communities should be more separable after flow
```

Workflow 2: Layer Comparison

Compare structural differences across layers:

```
# Visualize layers with shared layout
fig, layer_pos = net.visualize_ricci_layers(
    share_layout=True,
    arrangement="grid",
    iterations=15,
)

# Analyze differences:
# - Which edges appear in which layers?
# - How does curvature differ across layers?
# - Are communities consistent across layers?
```

Workflow 3: Hierarchical Analysis

Use 3D supra-graph to reveal hierarchical structure:

```
# 3D visualization with layer separation
fig, ax, pos = net.visualize_ricci_supra(
    dim=3,
    layer_separation=2.0,
    iterations=15,
    layout_type="spring",
)

# Interpret:
# - Vertical separation shows layer structure
# - Horizontal clustering shows communities
# - Inter-layer edges show coupling strength
```

6.21.8 Troubleshooting

Issue: Visualization is too cluttered

Solutions:

1. Reduce node/edge counts by filtering
2. Increase figsize parameter
3. Reduce node_size or edge_width_scale
4. For supra-graphs, increase interlayer_alpha to make inter-layer edges less visible

Issue: Layout looks random or unstructured

Solutions:

1. Increase iterations (try 20-30)
2. Try different layout_type (e.g., switch from spring to mds)
3. Increase alpha to emphasize global structure
4. Check that your network has meaningful structure to visualize

Issue: Computation is too slow**Solutions:**

1. Reduce iterations to 5-10
2. Use `layout_type="spectral"` (fastest)
3. Reduce `alpha` (lower values are faster)
4. Use parallel computation: `backend_kwargs={"proc": 4}`
5. Visualize a subgraph instead of the full network

Issue: GraphRicciCurvature import error**Solution:**

Install the optional dependency:

```
pip install GraphRicciCurvature
```

If installation fails, you may need to install dependencies first:

```
pip install networkit numpy scipy
```

6.21.9 See Also

- *Ollivier-Ricci Curvature and Ricci Flow* - Ollivier-Ricci curvature and Ricci flow fundamentals
- *Community Detection* - Community detection algorithms
- *Visualization Guide* - General visualization guide
- *Core Concepts and Architecture* - Understanding multilayer networks
- *Supra-Adjacency Matrices* - Supra-adjacency matrix representation

6.21.10 References

- Ni, C. C., Lin, Y. Y., Luo, F., & Gao, J. (2019). Community detection on networks with Ricci flow. *Scientific Reports*, 9(1), 9984.
- Ni, C. C., Lin, Y. Y., Gao, J., Gu, X. D., & Saucan, E. (2015). Ricci curvature of the Internet topology. *2015 IEEE Conference on Computer Communications (INFOCOM)*, 2758-2766.
- GraphRicciCurvature library: <https://github.com/saibalmars/GraphRicciCurvature>

6.22 Performance and Scalability

This guide provides **best practices** for optimizing py3plex performance and scaling to **large networks**.

6.22.1 Memory Management**Sparse vs Dense Matrices**

Always use sparse matrices for large networks:

```
# Good: Sparse representation (default)
supra_adj = network.get_supra_adjacency_matrix(sparse=True)

# Bad: Dense representation for large networks
# supra_adj = network.get_supra_adjacency_matrix(sparse=False)
```

Memory requirements:

- **Sparse:** $O(m)$ where m is **number of edges**
- **Dense:** $O(n^2)$ where n is **total nodes across layers**

Rule of thumb: Use **sparse** for networks with >1,000 nodes or density <10%

Matrix Construction Optimization

Avoid unnecessary matrix constructions:

```
# Bad: Repeated construction
for iteration in range(100):
    adj = network.get_supra_adjacency_matrix()
    # ... use adj ...

# Good: Construct once
adj = network.get_supra_adjacency_matrix()
for iteration in range(100):
    # ... use adj ...
```

6.22.2 Efficient Network Construction

Batch Operations

Add nodes and edges in **batches** rather than one at a time:

```
# Bad: Individual additions
for edge in edge_list:
    network.add_edges([[edge[0], edge[1], edge[2], edge[3], edge[4]]], input_type='list')

# Good: Batch addition
network.add_edges(edge_list, input_type='list')
```

Pre-allocate Data Structures

When possible, provide all data at construction time:

```
# Efficient: Provide all edges at once
network = multinet.multi_layer_network()
network.add_edges(all_edges, input_type='list')

# Less efficient: Incremental addition
network = multinet.multi_layer_network()
for edge_batch in batches:
    network.add_edges(edge_batch, input_type='list')
```

6.22.3 Algorithm Selection

Time Complexity Guidelines

Choose algorithms based on **network size**:

Network Size	Community Detection	Centrality	Embeddings
Small (<1K nodes)	Any	Any (including betweenness)	Node2Vec (full)
Medium (1K-10K)	Louvain	Degree, PageRank	Node2Vec (reduced walks)
Large (10K-100K)	Louvain, Label Prop	Degree only	DeepWalk (sampling)
Very Large (>100K)	Label Prop	Degree	Skip or sample

Avoid Expensive Operations

$O(n^2)$ or worse complexity:

- Full betweenness centrality
- All-pairs shortest paths
- Closeness centrality on large networks
- Dense matrix operations

Alternative approaches:

```
# Instead of betweenness on full network
# betweenness = calc.multilayer_betweenness_centrality() # O(nm)

# Use degree as proxy or sample
degree = calc.overlapping_degree_centrality() # O(n+m)

# Or sample nodes
import random
sample_nodes = random.sample(list(network.get_nodes()), k=100)
# Compute betweenness on induced subgraph
```

6.22.4 Parallel Processing

Using Joblib for Parallelization

py3plex operations can often be parallelized:

```
from joblib import Parallel, delayed
from py3plex.algorithms.statistics import multilayer_statistics as mls

# Parallel layer statistics
layers = network.get_layers()
densities = Parallel(n_jobs=-1)(
    delayed(mls.layer_density)(network, layer)
    for layer in layers
)
```


NumPy Vectorization

Use **vectorized operations** instead of loops:

```
import numpy as np

# Bad: Python loop
degrees = {}
for node in nodes:
    degrees[node] = sum(1 for neighbor in G.neighbors(node))

# Good: Vectorized
degrees = dict(G.degree()) # NetworkX optimized

# Even better for matrices
adj_matrix = network.get_adjacency_matrix(layer='layer1')
degrees = np.asarray(adj_matrix.sum(axis=1)).flatten()
```

6.22.5 Visualization Optimization

Layout Computation

Layout algorithms have **varying complexity**:

Layout	Complexity	Best For
Random	$O(n)$	Testing, very large networks
Circular	$O(n)$	Small networks with clear order
Spring/Force	$O(n^2 \times \text{iterations})$	<1,000 nodes
ForceAtlas2	$O(n \log n \times \text{iterations})$	1,000-10,000 nodes

Optimization strategies:

```
# 1. Reduce iterations for approximate layout
pos = dm.compute_layout(G, algorithm='force', iterations=50) # Default is often 500

# 2. Precompute and save layouts
import pickle
pos = dm.compute_layout(G, algorithm='force')
with open('layout.pkl', 'wb') as f:
    pickle.dump(pos, f)

# Load saved layout
with open('layout.pkl', 'rb') as f:
    pos = pickle.load(f)
```

Reduce Visual Elements

For large networks, reduce the number of rendered elements:

```
# 1. Sample edges
import random
all_edges = list(network.get_edges())
sampled_edges = random.sample(all_edges, k=min(1000, len(all_edges)))
```

(continues on next page)

(continued from previous page)

```
# 2. Hide labels
draw_network(network, show_labels=False)

# 3. Use smaller node/edge sizes
draw_network(network, node_size=1, edge_width=0.1)
```

Use Appropriate Visualization Type

Match visualization to network size:

```
num_nodes = len(network.get_nodes())

if num_nodes < 100:
    # Full network with labels
    draw_multilayer_default([network], display=True, show_labels=True)

elif num_nodes < 1000:
    # Network without labels, force layout
    hairball_plot(network.core_network, layout_algorithm='force')

elif num_nodes < 10000:
    # Diagonal projection
    draw_multilayer_default([network], layout_style='diagonal')

else:
    # Matrix visualization
    adj = network.get_supra_adjacency_matrix(sparse=True)
    plt.spy(adj, markersize=0.1)
    plt.show()
```

6.22.6 I/O Optimization

Efficient File Formats

Choose appropriate file formats:

Format	Load Speed	File Size	Best For
gpickle	Fast	Small	Temporary storage, Python only
GraphML	Medium	Large (XML)	Interchange, attributes
Edge list	Fast	Small	Simple networks, no attributes
GML	Slow	Medium	Legacy compatibility

Recommendation:

```
# For temporary storage (fastest)
network.save_network("cache.gpickle", output_type="gpickle")
network = multinet.multi_layer_network().load_network("cache.gpickle", input_type=
↪ "gpickle")
```

(continues on next page)

(continued from previous page)

```
# For interchange (portable)
network.save_network("data.graphml", output_type="graphml")
```

Streaming Large Files

For very large networks, process incrementally:

```
# Read edge list in chunks
network = multinet.multi_layer_network()

chunk_size = 10000
with open('large_edgelist.txt', 'r') as f:
    chunk = []
    for line in f:
        chunk.append(line.strip().split())
        if len(chunk) >= chunk_size:
            network.add_edges(chunk, input_type='list')
            chunk = []

# Add remaining edges
if chunk:
    network.add_edges(chunk, input_type='list')
```

6.22.7 Benchmarking and Profiling

Time Your Code

Use timing utilities to identify bottlenecks:

```
import time

start = time.time()
communities = community_louvain.best_partition(network.core_network)
elapsed = time.time() - start
print(f"Community detection took {elapsed:.2f} seconds")
```

Memory Profiling

Monitor memory usage:

```
import tracemalloc

tracemalloc.start()

# Your code here
adj = network.get_supra_adjacency_matrix(sparse=True)

current, peak = tracemalloc.get_traced_memory()
print(f"Current memory: {current / 1024**2:.2f} MB")
print(f"Peak memory: {peak / 1024**2:.2f} MB")
tracemalloc.stop()
```

Line Profiling

For detailed profiling:

```
pip install line_profiler

# Add @profile decorator to functions
# Run with:
kernprof -l -v script.py
```

6.22.8 Caching Strategies

Function-Level Caching

Cache expensive computations:

```
from functools import lru_cache

@lru_cache(maxsize=128)
def expensive_centrality(network_hash, layer):
    """Cached centrality computation."""
    # Compute centrality
    return centrality_values
```

Network-Level Caching

Save intermediate results:

```
import os
import pickle

cache_file = 'communities_cache.pkl'

if os.path.exists(cache_file):
    with open(cache_file, 'rb') as f:
        communities = pickle.load(f)
else:
    communities = community_louvain.best_partition(network.core_network)
    with open(cache_file, 'wb') as f:
        pickle.dump(communities, f)
```

6.22.9 Best Practices Summary

Network Size Guidelines

< 1,000 nodes:

- Any algorithm works
- Full visualization with labels
- Path-based centrality is fine
- Dense matrices acceptable

1,000 - 10,000 nodes:

- Use Louvain for communities

- Degree/PageRank for centrality
- Sparse matrices essential
- Diagonal projection visualization
- No labels in visualization

10,000 - 100,000 nodes:

- Label propagation for communities
- Degree centrality only
- Batch operations critical
- Matrix visualization
- Sample for exploration

> 100,000 nodes:

- Degree-based measures only
- Label propagation
- Incremental processing
- Sampling required
- Consider distributed tools for some tasks

Memory Checklist

- [OK] Use sparse matrices (`sparse=True`)
- [OK] Batch add edges (not one at a time)
- [OK] Avoid repeated matrix construction
- [OK] Use generators instead of lists where possible
- [OK] Clear unused variables (`del variable`)
- [OK] Monitor memory with `tracemalloc`

Speed Checklist

- [OK] Choose appropriate algorithms for network size
- [OK] Avoid $O(n^2)$ operations on large networks
- [OK] Use NumPy vectorization
- [OK] Cache expensive computations
- [OK] Precompute layouts
- [OK] Sample for exploration
- [OK] Profile to find bottlenecks

6.22.10 Benchmarking Examples

Run the built-in benchmarks:

```
cd benchmarks
python config_benchmark.py
```

Create your own benchmarks:

```
from py3plex.core import multinet
import time

# Create networks of different sizes
sizes = [100, 500, 1000, 5000]
times = []

for size in sizes:
    network = multinet.multi_layer_network()

    # Generate random edges
    import random
    edges = []
    for i in range(size * 5): # ~5 edges per node
        edges.append([
            f'n{random.randint(0, size)}', 'L1',
            f'n{random.randint(0, size)}', 'L1', 1
        ])

    start = time.time()
    network.add_edges(edges, input_type='list')
    elapsed = time.time() - start

    times.append((size, elapsed))
print(f"Size {size}: {elapsed:.3f}s")
```

6.22.11 Next Steps

- *Algorithm Selection Guide* - Algorithm selection guide
- *Quick Start Guide* - Usage examples
- *Multilayer Network Centrality Measures* - Centrality tutorial
- Benchmark scripts in `benchmarks/` directory

6.22.12 Getting Help

For performance issues:

1. Check this guide first
2. Run built-in benchmarks
3. Profile your code
4. Open an issue with profiling results

6.23 Performance and Scalability Best Practices

This guide provides recommendations for optimizing Py3plex performance and handling large multilayer networks.

Table of Contents

- *Network Scale Guidelines*
- *Sparse Matrix Backend*
 - *Why Sparse Matrices?*
 - *Automatic Sparse Matrix Usage*
 - *Force Sparse Operations*
- *Network Sampling*
 - *When to Sample*
 - *Random Node Sampling*
 - *Stratified Sampling (Preserve Layer Distribution)*
 - *Hub-Based Sampling (Keep Important Nodes)*
- *Algorithm Optimization*
 - *Choose Efficient Algorithms*
 - *Limit Algorithm Iterations*
- *Parallel Processing*
 - *Multi-Core Processing*
 - *GPU Acceleration (Advanced)*
- *Visualization Optimization*
 - *Reduce Visual Complexity*
 - *Save to File Instead of Display*
 - *Use Lower Resolution*
- *Memory Management*
 - *Monitor Memory Usage*
 - *Free Memory When Done*
 - *Use Generators Instead of Lists*
- *Batch Processing*
 - *Process Networks in Batches*
- *Benchmark Results*
 - *Performance Benchmarks (2025)*
 - *Scaling Recommendations*
- *Alternative Tools for Scale*
 - *When Py3plex Isn't Enough*

- *Quick Performance Checklist*
 - *Before Running on Large Network*
 - *Optimization Order*
- *Next Steps*

6.23.1 Network Scale Guidelines

Py3plex is optimized for research-scale networks. This table shows expected performance characteristics:

Table 1: Network Scale Performance

Network Size	Performance	Visualization	Recommendations
Small (<100 nodes)	Excellent	Fast, detailed	Use dense visualization mode
Medium (100-1k nodes)	Good	Fast, balanced	Default settings work well
Large (1k-10k nodes)	Good	Slower, minimal	Use sparse matrices, sampling
Very Large (>10k nodes)	Variable	Very slow	Sampling required, use NetworkX/igraph

6.23.2 Sparse Matrix Backend

Why Sparse Matrices?

Most real-world networks are **sparse** (few edges compared to possible edges). Sparse matrices:

- **Reduce memory usage** by 10-100x for typical networks
- **Speed up** matrix operations (multiplication, inversion)
- **Enable** analysis of larger networks

Example:

```
import numpy as np
from scipy.sparse import csr_matrix

# Dense representation (10k x 10k network)
# Memory: 10,000^2 x 8 bytes = 800 MB

# Sparse representation (with 1% density)
# Memory: ~8 MB (100x reduction!)
```

Automatic Sparse Matrix Usage

Py3plex **automatically uses sparse matrices** for:

- Supra-adjacency matrix operations
- Large network storage (>1000 nodes)
- Matrix-based algorithms (PageRank, spectral methods)

Verify sparse usage:


```

from py3plex.core import multinet

network = multinet.multi_layer_network()
network.load_network("large_network.csv", input_type="multiedgelist")

# Get sparse adjacency matrix
adj_sparse = network.get_sparse_adjacency_matrix()

print(f"Matrix size: {adj_sparse.shape}")
print(f"Non-zero entries: {adj_sparse.nnz}")
print(f"Sparsity: {1 - adj_sparse.nnz / (adj_sparse.shape[0]**2):.2%}")

```

Force Sparse Operations

For custom algorithms, explicitly use sparse operations:

```

from scipy.sparse import csr_matrix, lil_matrix
import numpy as np

# Create sparse adjacency matrix
adj = lil_matrix((n_nodes, n_nodes))

for u, v in network.core_network.edges():
    i, j = node_to_idx[u], node_to_idx[v]
    adj[i, j] = 1
    adj[j, i] = 1 # Undirected

# Convert to efficient format for operations
adj_csr = adj.tocsr()

# Sparse matrix operations are MUCH faster
result = adj_csr @ adj_csr # Matrix multiplication
eigvals = scipy.sparse.linalg.eigs(adj_csr, k=10) # Top eigenvalues

```

6.23.3 Network Sampling

When to Sample

Sampling is necessary when:

- Network is too large to visualize (>5k nodes)
- Algorithms take too long (>10 minutes)
- Memory usage is excessive (>8GB RAM)
- You need quick exploratory analysis

Random Node Sampling

```

import random
from py3plex.core import multinet

# Load full network
network = multinet.multi_layer_network()

```

(continues on next page)

(continued from previous page)

```

network.load_network("large_network.csv", input_type="multiedgelist")

# Sample 1000 random nodes
all_nodes = list(network.get_nodes())
sample_nodes = random.sample(all_nodes, min(1000, len(all_nodes)))

# Create subnetwork
subnetwork = network.get_subnetwork(sample_nodes)

print(f"Original: {len(all_nodes)} nodes")
print(f"Sample: {len(sample_nodes)} nodes")
print(f"Sampling ratio: {len(sample_nodes)/len(all_nodes):.1%}")

```

Stratified Sampling (Preserve Layer Distribution)

```

# Sample proportionally from each layer
layers = network.get_layer_names()
sample_nodes_per_layer = {}

for layer in layers:
    layer_nodes = [n for n in network.get_nodes() if n[1] == layer]
    sample_size = len(layer_nodes) // 10 # 10% sample
    sample_nodes_per_layer[layer] = random.sample(layer_nodes, sample_size)

# Combine samples
all_samples = [node for nodes in sample_nodes_per_layer.values() for node in nodes]
subnetwork = network.get_subnetwork(all_samples)

```

Hub-Based Sampling (Keep Important Nodes)

```

import networkx as nx

# Sample high-degree nodes (hubs)
degrees = dict(network.core_network.degree())

# Sort by degree and take top 1000
sorted_nodes = sorted(degrees.items(), key=lambda x: x[1], reverse=True)
hub_nodes = [node for node, deg in sorted_nodes[:1000]]

subnetwork = network.get_subnetwork(hub_nodes)
print(f"Sampled top 1000 hubs")

```

6.23.4 Algorithm Optimization

Choose Efficient Algorithms

Some algorithms scale better than others:

Table 2: Algorithm Complexity

Algorithm	Complexity	Recommendations
Degree centrality	$O(n + m)$	Fast, use freely
Betweenness centrality	$O(nm)$	Slow for large networks, sample first
PageRank	$O(\text{iterations} \times m)$	Fast if sparse, limit iterations
Community detection (Louvain)	$O(m \log n)$	Fast, recommended
Shortest paths (all pairs)	$O(n^2m)$	Very slow, use sampling or approximate
Force-directed layout	$O(n^2)$	Slow for >5k nodes, use alternatives

Example - Fast centrality:

```
import networkx as nx

G = network.core_network

# FAST: Degree centrality
degree_cent = nx.degree_centrality(G) #  $O(n+m)$  - instant

# SLOW: Betweenness centrality
# For large networks, sample first or use approximate algorithm
if G.number_of_nodes() < 1000:
    between_cent = nx.betweenness_centrality(G)
else:
    # Use approximate algorithm
    between_cent = nx.betweenness_centrality(G, k=100) # Sample 100 nodes
```

Limit Algorithm Iterations

```
import networkx as nx

# PageRank with iteration limit
pagerank = nx.pagerank(
    network.core_network,
    max_iter=50,          # Limit iterations
    tol=1e-4              # Tolerance for convergence
)

# Community detection with resolution limit
from py3plex.algorithms.community_detection import community_louvain
communities = community_louvain.best_partition(
    network.core_network,
    resolution=1.0        # Adjust resolution parameter
)
```

6.23.5 Parallel Processing**Multi-Core Processing**

Use joblib for parallel node/edge operations:

```

from joblib import Parallel, delayed
import networkx as nx

def compute_node centrality(node, graph):
    """Compute centrality for a single node."""
    # Custom centrality computation
    neighbors = list(graph.neighbors(node))
    return node, len(neighbors)

# Parallel processing
nodes = list(network.core_network.nodes())
results = Parallel(n_jobs=-1)( # Use all cores
    delayed(compute_node centrality)(node, network.core_network)
    for node in nodes
)

centralities = dict(results)

```

GPU Acceleration (Advanced)

For very large networks, use GPU acceleration:

```

# Install CuPy for GPU NumPy operations
pip install cupy-cuda11x # Replace 11x with CUDA version

```

```

# Requires NVIDIA GPU with CUDA
try:
    import cupy as cp

    # Convert to GPU array
    adj_matrix = network.get_sparse_adjacency_matrix().toarray()
    gpu_adj = cp.array(adj_matrix)

    # GPU-accelerated matrix operations
    gpu_result = cp.dot(gpu_adj, gpu_adj)

    # Transfer back to CPU
    result = cp.asnumpy(gpu_result)

except ImportError:
    print("CuPy not available, using CPU")

```

6.23.6 Visualization Optimization

Reduce Visual Complexity

```

from py3plex.visualization.multilayer import draw_multilayer_default

# For large networks (>1000 nodes)
draw_multilayer_default(
    network.get_layers(),
    node_size=3, # Tiny nodes

```

(continues on next page)

(continued from previous page)

```

labels=False,          # No labels
edge_size=0.3,         # Thin edges
alphalevel=0.2,        # Very transparent
remove_isolated_nodes=True # Remove disconnected
)

```

Save to File Instead of Display

```

import matplotlib.pyplot as plt

# Don't show interactively (faster)
fig, ax = plt.subplots(1, 1, figsize=(10, 8))
draw_multilayer_default(
    network.get_layers(),
    display=False, # Don't show
    axis=ax
)

# Save directly to file
plt.savefig('network.png', dpi=150, bbox_inches='tight')
plt.close() # Free memory

```

Use Lower Resolution

```

# For quick exploration, use low DPI
plt.figure(figsize=(8, 6), dpi=72) # Low resolution

# For publications, use high DPI
plt.figure(figsize=(10, 8), dpi=300) # High resolution

```

6.23.7 Memory Management

Monitor Memory Usage

```

import psutil
import os

def get_memory_usage():
    """Get current memory usage in MB."""
    process = psutil.Process(os.getpid())
    return process.memory_info().rss / 1024 / 1024

print(f"Memory before loading: {get_memory_usage():.1f} MB")

network = multinet.multi_layer_network()
network.load_network("large_network.csv", input_type="multiedgelist")

print(f"Memory after loading: {get_memory_usage():.1f} MB")

```

Free Memory When Done

```
# Delete network when no longer needed
del network

# Force garbage collection
import gc
gc.collect()
```

Use Generators Instead of Lists

```
# BAD: Loads all nodes into memory
all_nodes = list(network.get_nodes())
for node in all_nodes:
    process(node)

# GOOD: Processes nodes one at a time
for node in network.get_nodes():
    process(node)
```

6.23.8 Batch Processing

Process Networks in Batches

For multiple networks:

```
import os
import gc

network_files = ["net1.csv", "net2.csv", "net3.csv", ...]

results = []
for i, file in enumerate(network_files):
    print(f"Processing {i+1}/{len(network_files)}: {file}")

    # Load network
    network = multinet.multi_layer_network()
    network.load_network(file, input_type="multiedgelist")

    # Compute statistics
    result = analyze_network(network)
    results.append(result)

    # Free memory
    del network
    gc.collect()

    # Save intermediate results every 10 networks
    if (i + 1) % 10 == 0:
        save_results(results, f"results_batch_{i+1}.json")
```

6.23.9 Benchmark Results

Performance Benchmarks (2025)

Tested on: Intel i7-10700K, 32GB RAM, Python 3.10

Table 3: Operation Performance

Operation	100 nodes	1k nodes	10k nodes	100k nodes
Load from CSV	<1s	<1s	2s	20s
Basic statistics	<1s	<1s	<1s	3s
Degree centrality	<1s	<1s	1s	10s
PageRank	<1s	<1s	2s	25s
Louvain communities	<1s	1s	5s	60s
Visualization (sparse)	<1s	2s	15s	N/A*

* Visualization not recommended for >10k nodes without sampling

Scaling Recommendations

Based on network size:

<1k nodes:

- Use any algorithms
- Full visualization
- No sampling needed

1k-10k nodes:

- Use sparse matrices
- Minimal visualization
- Sample for some algorithms

10k-100k nodes:

- Sparse matrices required
- Sample for visualization
- Use approximate algorithms
- Consider igraph for speed

>100k nodes:

- Use specialized tools (igraph, graph-tool, NetworKit)
- Sample heavily for Py3plex operations
- Focus on specific analyses

6.23.10 Alternative Tools for Scale

When Py3plex Isn't Enough

For networks >100k nodes, consider:

igraph (C-based, very fast):

```
import igraph as ig

# 10-100x faster for large networks
g = ig.Graph.Read_GraphML("large_network.graphml")
communities = g.community_multilevel()

# Export back to Py3plex if needed
# (via GraphML or edge list)
```

graph-tool (C++, fastest):

```
import graph_tool.all as gt

# Fastest for >1M edges
g = gt.load_graph("large_network.graphml")
communities = gt.community_structure.minimize_blockmodel_dl(g)
```

NetworkKit (C++, parallel):

```
import networkkit as nk

# Excellent for parallel algorithms
G = nk.readGraph("large_network.edgelist", nk.Format.EdgeList)
communities = nk.community.detectCommunities(G)
```

6.23.11 Quick Performance Checklist

Before Running on Large Network

```
[ ] Enable sparse matrices (automatic for most ops)
[ ] Sample network if >10k nodes
[ ] Choose efficient algorithms (degree > betweenness)
[ ] Limit visualization detail
[ ] Monitor memory usage
[ ] Use batch processing for multiple networks
[ ] Consider alternative tools if >100k nodes
```

Optimization Order

1. **Use sparse matrices** (biggest impact, usually automatic)
2. **Sample network** (if >10k nodes)
3. **Choose efficient algorithms** (avoid $O(n^3)$ operations)
4. **Parallelize** (if multi-core available)
5. **GPU acceleration** (only if CUDA GPU available)

6.23.12 Next Steps

- *Dependency Management and Troubleshooting* - Install optional performance packages
- *Visualization Guide* - Optimize visualizations
- *NetworkX Interoperability Guide* - Use NetworkX algorithms

- *CSV Data Loading Examples* - Efficient data loading

For performance issues, open an issue on [GitHub Issues](#).

6.24 Statistical Comparison Framework

6.24.1 Overview

The `stats_comparison` module provides a comprehensive framework for performing statistical comparisons between multilayer networks. This enables quantification of structural or topological differences across network ensembles or experimental conditions.

6.24.2 Features

- **Pairwise and Multi-group Comparisons:** Compare 2 or more multilayer networks
- **Multiple Statistical Tests:**
 - Parametric: t-test, ANOVA
 - Non-parametric: Mann-Whitney U, Kruskal-Wallis
 - Permutation-based: Empirical null hypothesis testing
- **Effect Size Estimation:** Cohen's d, eta-squared
- **Multiple Comparison Correction:** Bonferroni, Holm-Bonferroni, FDR (Benjamini-Hochberg)
- **Bootstrap Confidence Intervals:** Estimate metric uncertainty
- **Comprehensive Metrics:** Density, degree, clustering, node activity, coupling strength, entropy

6.24.3 Installation

The module is part of py3plex. Ensure you have the required dependencies:

```
pip install numpy scipy pandas networkx
```

6.24.4 Quick Start

```
from py3plex.core import multinet
from py3plex.algorithms.statistics.stats_comparison import compare_multilayer_networks

# Create networks
net1 = multinet.multi_layer_network(directed=False)
net1.add_edges([
    ['A', 'L1', 'B', 'L1', 1],
    ['B', 'L1', 'C', 'L1', 1],
    ['C', 'L1', 'A', 'L1', 1]
], input_type='list')

net2 = multinet.multi_layer_network(directed=False)
net2.add_edges([
    ['A', 'L1', 'B', 'L1', 1],
    ['B', 'L1', 'C', 'L1', 1]
], input_type='list')
```

(continues on next page)

(continued from previous page)

```
# Compare networks
results = compare_multilayer_networks(
    [net1, net2],
    metrics=['density', 'average_degree'],
    test='permutation',
    n_permutations=1000,
    correction='fdr_bh',
    alpha=0.05
)

print(results)
```

Expected Output (pandas DataFrame):

	metric	layer	statistic	p_value	adjusted_p_value	effect_size	significant	mean_
↪ group_0	mean_group_1							
0	density	L1	0.250	0.132	0.132	0.500	False	↪
↪	1.000		0.667					
1	average_degree	L1	0.667	0.215	0.215	0.816	False	↪
↪	2.000		1.333					

6.24.5 API Reference

compare_multilayer_networks()

Main function for comparing multilayer networks.

Parameters:

- **networks** (List[MultilayerNetwork]): List of networks to compare (minimum 2)
- **metrics** (List[str], optional): Metrics to compute. Options:
 - 'density': Layer density
 - 'average_degree': Mean node degree
 - 'clustering': Clustering coefficient
 - 'node_activity': Node activity across layers
 - 'coupling_strength': Inter-layer coupling
 - 'entropy': Entropy of multiplexity
 - Default: ['average_degree', 'density', 'clustering']
- **test** (str, optional): Statistical test type. Options:
 - 'permutation': Permutation test (default)
 - 't-test': Student's t-test (2 groups)
 - 'mann-whitney': Mann-Whitney U test (2 groups)
 - 'anova': One-way ANOVA (2+ groups)
 - 'kruskal': Kruskal-Wallis H test (2+ groups)
- **n_permutations** (int, optional): Number of permutations (default: 1000)
- **alpha** (float, optional): Significance level (default: 0.05)

- **correction** (str, optional): Multiple comparison correction. Options:
 - 'bonferroni': Bonferroni correction
 - 'holm': Holm-Bonferroni correction
 - 'fdr_bh': Benjamini-Hochberg FDR (default)
 - None: No correction
- **node_mapping** (Dict, optional): Node correspondence mapping across networks
- **layer_scope** (str, optional): Layer scope ('intralayer', 'interlayer', or 'both')
- **custom_metrics** (Dict[str, Callable], optional): Custom metric functions

Returns:

pandas.DataFrame with columns:

- **metric**: Metric name
- **layer**: Layer identifier or 'global'
- **statistic**: Test statistic value
- **p_value**: Raw p-value
- **adjusted_p_value**: Corrected p-value
- **effect_size**: Effect size (Cohen's d or similar)
- **significant**: Boolean indicating significance
- **mean_group_0, mean_group_1, ...**: Mean values per group

bootstrap_confidence_interval()

Compute bootstrap confidence intervals for network metrics.

Parameters:

- **networks** (List[MultilayerNetwork]): List of networks
- **metric_func** (Callable): Function mapping network to scalar metric
- **n_bootstrap** (int, optional): Number of bootstrap samples (default: 1000)
- **confidence_level** (float, optional): Confidence level (default: 0.95)

Returns:

Dictionary mapping group indices to (lower, upper) CI bounds.

6.24.6 Examples

Basic Two-Network Comparison

```
from py3plex.core import multinet
from py3plex.algorithms.statistics.stats_comparison import compare_multilayer_networks

# Create networks with different densities
net1 = multinet.multi_layer_network(directed=False)
net1.add_edges([
    ['A', 'L1', 'B', 'L1', 1],
    ['B', 'L1', 'C', 'L1', 1],
```

(continues on next page)

(continued from previous page)

```

    ['C', 'L1', 'A', 'L1', 1], # Triangle
    ['A', 'L2', 'B', 'L2', 1]
], input_type='list')

net2 = multinet.multi_layer_network(directed=False)
net2.add_edges([
    ['A', 'L1', 'B', 'L1', 1],
    ['B', 'L1', 'C', 'L1', 1], # Line
    ['A', 'L2', 'B', 'L2', 1]
], input_type='list')

# Compare using permutation test
results = compare_multilayer_networks(
    [net1, net2],
    metrics=['density', 'clustering'],
    test='permutation',
    n_permutations=10000
)

# Show significant differences
significant = results[results['significant']]
print(significant)

```

Multi-Group Comparison

```

# Compare three or more networks
results = compare_multilayer_networks(
    [net1, net2, net3, net4],
    metrics=['density', 'average_degree'],
    test='kruskal', # Non-parametric for multiple groups
    correction='holm'
)

# Filter by metric
density_results = results[results['metric'] == 'density']
print(density_results)

```

Bootstrap Confidence Intervals

```

import numpy as np
from py3plex.algorithms.statistics.stats_comparison import bootstrap_confidence_interval

def avg_clustering(network):
    """Compute average clustering coefficient."""
    from py3plex.algorithms.statistics import multilayer_statistics as mls
    clustering = mls.multilayer_clustering_coefficient(network)
    return np.mean(list(clustering.values()))

# Get 95% CI for clustering
ci = bootstrap_confidence_interval(
    [net1, net2],

```

(continues on next page)

(continued from previous page)

```

    avg_clustering,
    n_bootstrap=1000,
    confidence_level=0.95
)

for group, (lower, upper) in ci.items():
    print(f"{group}: [{lower:.4f}, {upper:.4f}]")

```

6.24.7 Best Practices

Choosing Statistical Tests

1. **Sample Size < 30:** Use permutation or non-parametric tests
2. **Normal Distribution:** Parametric tests (t-test, ANOVA) have more power
3. **Two Groups:** t-test (parametric) or Mann-Whitney (non-parametric)
4. **Multiple Groups:** ANOVA (parametric) or Kruskal-Wallis (non-parametric)
5. **Unknown Distribution:** Permutation test (most general)

Multiple Comparison Correction

When testing multiple metrics or layers:

1. **Conservative:** Use Bonferroni (controls family-wise error rate)
2. **Less Conservative:** Use Holm-Bonferroni (sequential Bonferroni)
3. **Recommended:** Use FDR (Benjamini-Hochberg) - balances power and error control

Effect Sizes

Always report effect sizes in addition to p-values:

- **Cohen's d:** $|d| < 0.2$ (small), 0.2-0.5 (medium), > 0.8 (large)
- **Eta-squared:** $\eta^2 < 0.01$ (small), 0.01-0.06 (medium), > 0.14 (large)

6.24.8 Use Cases

Biological Networks

Compare disease vs. healthy protein interaction networks across multiple tissues:

```

results = compare_multilayer_networks(
    disease_networks + healthy_networks,
    metrics=['density', 'clustering', 'node_activity'],
    test='mann-whitney',
    correction='fdr_bh'
)

```

Social Networks

Analyze structural differences in social networks before/after interventions:

```
results = compare_multilayer_networks(  
    [pre_intervention_net, post_intervention_net],  
    metrics=['average_degree', 'coupling_strength'],  
    test='permutation',  
    n_permutations=10000  
)
```

Temporal Networks

Compare network snapshots across time periods:

```
results = compare_multilayer_networks(  
    [t1_network, t2_network, t3_network],  
    metrics=['density', 'entropy'],  
    test='kruskal'  
)
```

6.24.9 See Also

- Example notebook: `examples/statistical_comparison_example.ipynb`
- Example script: `examples/compare_multilayer_networks_example.py`
- Module documentation: `py3plex.algorithms.statistics.stats_comparison`
- Related: *Supra Matrix Function Centralities*

6.24.10 References

1. Kivelä, M., et al. (2014). “Multilayer networks.” *Journal of Complex Networks*, 2(3), 203-271.
2. Good, P. (2013). “Permutation Tests: A Practical Guide to Resampling Methods for Testing Hypotheses.”
3. Efron, B., & Tibshirani, R. J. (1994). “An Introduction to the Bootstrap.”
4. Benjamini, Y., & Hochberg, Y. (1995). “Controlling the false discovery rate: a practical and powerful approach to multiple testing.”

6.25 Advanced Multilayer Metrics

6.25.1 Overview

The `multilayer_statistics` module provides advanced entropy-based, information-theoretic, and influence metrics for multilayer network analysis. These metrics enable deep characterization of structural complexity, layer interdependencies, and centrality patterns across network layers.

6.25.2 New Metrics (v0.96+)

The following advanced metrics have been added to py3plex for comprehensive multilayer network analysis:

1. **Entropy-based Layer Complexity Measures**
2. **Cross-layer Mutual Information**
3. **Layer Influence Centrality**
4. **Multilayer Betweenness Surface**

5. Inter-layer Degree Correlation Matrix

6.25.3 Installation

These metrics are included in py3plex. Ensure you have the required dependencies:

```
pip install numpy scipy networkx matplotlib seaborn
```

6.25.4 Quick Start

```
from py3plex.core import multinet
from py3plex.algorithms.statistics import multilayer_statistics as mls

# Create a multilayer network
network = multinet.multi_layer_network(directed=False)
network.add_edges([
    ['A', 'L1', 'B', 'L1', 1],
    ['B', 'L1', 'C', 'L1', 1],
    ['A', 'L2', 'C', 'L2', 1],
    ['A', 'L1', 'A', 'L2', 1], # Inter-layer edge
], input_type='list')

# Calculate entropy-based metrics
entropy = mls.layer_connectivity_entropy(network, 'L1')
mi = mls.cross_layer_mutual_information(network, 'L1', 'L2')
influence = mls.layer_influence_centrality(network, 'L1')

print(f"Connectivity entropy: {entropy:.3f}")
print(f"Mutual information: {mi:.3f}")
print(f"Layer influence: {influence:.3f}")
```

Expected Output:

```
Connectivity entropy: 1.500
Mutual information: 0.500
Layer influence: 1.000
```

6.25.5 Entropy-based Complexity Measures

These metrics quantify structural complexity using information-theoretic entropy.

Layer Connectivity Entropy

Measures heterogeneity of node connectivity within a layer using Shannon entropy of the degree distribution.

Formula: $H_c = - \sum (k/k) \log_2(k/k)$

Interpretation: - $H_c = 0$: uniform degree distribution (all nodes have same degree) - Higher values: more diverse connectivity patterns - Useful for identifying layers with hub-and-spoke vs. distributed structures

```
# Calculate for each layer
for layer in ['L1', 'L2', 'L3']:
    entropy = mls.layer_connectivity_entropy(network, layer)
    print(f"Layer {layer} connectivity entropy: {entropy:.3f}")
```

Example Output:

```
Layer L1 connectivity entropy: 2.000
Layer L2 connectivity entropy: 1.585
Layer L3 connectivity entropy: 2.252
```

Inter-layer Dependence Entropy

Measures diversity in how nodes couple two layers through inter-layer edges.

Formula: $H_{\text{dep}} = -p \log_2(p)$, where p is the proportion of inter-layer edges for node n

Interpretation: - $H_{\text{dep}} = 0$: uniform coupling (all nodes equally involved in inter-layer connections) - Higher values: more heterogeneous coupling patterns - Identifies layers with specialized bridge nodes

```
# Calculate inter-layer dependence
entropy = mls.inter_layer_dependence_entropy(network, 'L1', 'L2')
print(f"Inter-layer dependence entropy: {entropy:.3f}")
```

Example Output:

```
Inter-layer dependence entropy: 1.500
```

Cross-layer Redundancy Entropy

Measures diversity in structural overlap across all layer pairs using edge overlap.

Formula: $H_r = -r \log_2(r)$, where r is normalized edge overlap

Interpretation: - $H_r = 0$: uniform redundancy across all layer pairs - Higher values: varied overlap patterns (some layers very similar, others distinct) - Useful for identifying functionally redundant vs. complementary layers

```
# Calculate global redundancy entropy
entropy = mls.cross_layer_redundancy_entropy(network)
print(f"Cross-layer redundancy entropy: {entropy:.3f}")
```

Example Output:

```
Cross-layer redundancy entropy: 1.000
```

6.25.6 Cross-layer Mutual Information

Quantifies statistical dependence between degree distributions in two layers.

Formula: $I(L; L) = H(L) + H(L) - H(L, L)$

Properties: - $I = 0$: layers are statistically independent - $I > 0$: layers are dependent (higher values = stronger dependence) - $I(L; L) \leq \min(H(L), H(L))$ - Symmetric: $I(L; L) = I(L; L)$

Use Cases: - Identify functionally related layers - Detect redundant vs. complementary layers - Guide layer aggregation or dimensionality reduction - Predict cross-layer influence patterns

```
# Calculate mutual information between layer pairs
mi_12 = mls.cross_layer_mutual_information(network, 'L1', 'L2', bins=10)
mi_13 = mls.cross_layer_mutual_information(network, 'L1', 'L3', bins=10)
mi_23 = mls.cross_layer_mutual_information(network, 'L2', 'L3', bins=10)
```

(continues on next page)

(continued from previous page)

```
print(f"I(L1; L2) = {mi_12:.3f} bits")
print(f"I(L1; L3) = {mi_13:.3f} bits")
print(f"I(L2; L3) = {mi_23:.3f} bits")
```

Example Output:

```
I(L1; L2) = 0.250 bits
I(L1; L3) = 0.000 bits
I(L2; L3) = 0.150 bits
```

Parameters: - network: py3plex multi_layer_network object - layer_i: First layer identifier - layer_j: Second layer identifier - bins: Number of bins for discretizing degree distributions (default: 10)

6.25.7 Layer Influence Centrality

Quantifies how much a layer influences others through inter-layer connections or information flow.

Two Methods:

1. Coupling-based Influence

Formula: $I = C^L / (L-1)$

- Based on inter-layer coupling strength
- Structural measure
- Fast to compute

2. Flow-based Influence

Formula: $I = \text{flow probability from layer to other layers}$

- Based on random walk simulations
- Dynamic measure
- Captures indirect influence

Interpretation: - Higher values indicate layers that strongly influence others - Useful for identifying critical layers in multilayer infrastructure - Guides targeted interventions or layer removal strategies

```
# Coupling-based influence
influence_coupling = mls.layer_influence_centrality(
    network, 'L1', method='coupling'
)

# Flow-based influence
influence_flow = mls.layer_influence_centrality(
    network, 'L1', method='flow', sample_size=100
)

print(f"Layer L1 coupling influence: {influence_coupling:.3f}")
print(f"Layer L1 flow influence: {influence_flow:.3f}")
```

Example Output:

```
Layer L1 coupling influence: 1.250
Layer L1 flow influence: 0.420
```

Parameters: - network: py3plex multi_layer_network object - layer: Layer identifier - method: 'coupling' or 'flow' (default: 'coupling') - sample_size: Number of random walk steps for flow method (default: 100)

6.25.8 Multilayer Betweenness Surface

Visualizes betweenness centrality across all node-layer pairs as a 2D matrix (surface).

Output: 2D array of shape (num_nodes, num_layers)

Applications: - Identify bridge nodes that connect different layers - Visualize centrality patterns as heatmaps or 3D surfaces - Compare node importance across layers - Detect layer-specific vs. global central nodes

```
# Calculate betweenness surface
surface, (nodes, layers) = mls.multilayer_betweenness_surface(
    network, normalized=True
)

# Visualize as heatmap
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(10, 6))
sns.heatmap(surface, annot=True, fmt='.3f',
            xticklabels=layers, yticklabels=nodes,
            cmap='YlOrRd', cbar_kws={'label': 'Betweenness Centrality'})
plt.xlabel('Layers')
plt.ylabel('Nodes')
plt.title('Multilayer Betweenness Surface')
plt.tight_layout()
plt.savefig('betweenness_surface.png', dpi=300)
```

Example Output:

Surface shape: (5, 3) nodes × layers

Node	L1	L2	L3

Alice	0.250	0.333	0.115
Bob	0.280	0.000	0.154
Carol	0.180	0.038	0.295
David	0.128	0.038	0.000
Eve	0.000	0.000	0.000

Visualization Tips: - Use heatmaps for quick overview - Use 3D surface plots for presentations - Cluster rows (nodes) to identify groups with similar patterns - Compare surfaces across different time points or conditions

6.25.9 Inter-layer Degree Correlation Matrix

Computes Pearson correlations of node degrees between all pairs of layers.

Output: Symmetric correlation matrix (num_layers × num_layers)

Properties: - Diagonal elements are 1.0 (self-correlation) - Off-diagonal elements in [-1, 1] - Positive values: nodes with high degree in one layer tend to have high degree in another - Negative values: inverse relationship (hubs in one layer are peripheral in another)

Applications: - Identify correlated vs. independent layers - Detect layer redundancy for network compression - Guide layer aggregation strategies - Understand cross-layer degree assortativity

```
# Calculate correlation matrix
corr_matrix, layers = mls.interlayer_degree_correlation_matrix(network)

# Visualize
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(8, 6))
sns.heatmap(corr_matrix, annot=True, fmt='.3f',
            xticklabels=layers, yticklabels=layers,
            cmap='coolwarm', center=0, vmin=-1, vmax=1,
            cbar_kws={'label': 'Pearson Correlation'})
plt.title('Inter-layer Degree Correlation Matrix')
plt.tight_layout()
plt.savefig('degree_correlation_matrix.png', dpi=300)
```

Example Output:

Correlation Matrix:			
	L1	L2	L3
L1	1.000	0.750	-0.250
L2	0.750	1.000	0.000
L3	-0.250	0.000	1.000

Interpretation: - L1 and L2 are strongly positively correlated ($r = 0.75$) - L1 and L3 are negatively correlated ($r = -0.25$) - L2 and L3 are independent ($r = 0.00$)

6.25.10 Complete Example

This example demonstrates all advanced metrics on a social-professional network:

```
from py3plex.core import multinet
from py3plex.algorithms.statistics import multilayer_statistics as mls
import numpy as np

# Create 3-layer social-professional network
network = multinet.multi_layer_network(directed=False)

# Facebook layer (social)
network.add_edges([
    ['Alice', 'facebook', 'Bob', 'facebook', 1],
    ['Alice', 'facebook', 'Carol', 'facebook', 1],
    ['Bob', 'facebook', 'Carol', 'facebook', 1],
], input_type='list')

# LinkedIn layer (professional)
network.add_edges([
    ['Alice', 'linkedin', 'Bob', 'linkedin', 1],
    ['Carol', 'linkedin', 'David', 'linkedin', 1],
], input_type='list')
```

(continues on next page)

(continued from previous page)

```

# Twitter layer (public)
network.add_edges([
    ['Alice', 'twitter', 'Carol', 'twitter', 1],
    ['Bob', 'twitter', 'David', 'twitter', 1],
], input_type='list')

# Inter-layer connections
network.add_edges([
    ['Alice', 'facebook', 'Alice', 'linkedin', 1],
    ['Alice', 'linkedin', 'Alice', 'twitter', 1],
    ['Bob', 'facebook', 'Bob', 'twitter', 1],
    ['Carol', 'facebook', 'Carol', 'twitter', 1],
], input_type='list')

# 1. Entropy-based complexity measures
print("=== ENTROPY-BASED MEASURES ===")
for layer in ['facebook', 'linkedin', 'twitter']:
    entropy = mls.layer_connectivity_entropy(network, layer)
    print(f"{layer}: {entropy:.3f} bits")

# 2. Cross-layer mutual information
print("\n=== MUTUAL INFORMATION ===")
mi = mls.cross_layer_mutual_information(network, 'facebook', 'linkedin')
print(f"I(facebook; linkedin) = {mi:.3f} bits")

# 3. Layer influence centrality
print("\n=== LAYER INFLUENCE ===")
for layer in ['facebook', 'linkedin', 'twitter']:
    influence = mls.layer_influence_centrality(network, layer, method='coupling')
    print(f"{layer}: {influence:.3f}")

# 4. Betweenness surface
print("\n=== BETWEENNESS SURFACE ===")
surface, (nodes, layers) = mls.multilayer_betweenness_surface(network)
print(f"Shape: {surface.shape}")
print(f"Mean betweenness: {np.mean(surface):.3f}")

# 5. Degree correlation matrix
print("\n=== DEGREE CORRELATION MATRIX ===")
corr_matrix, layers = mls.interlayer_degree_correlation_matrix(network)
print("Correlation matrix:")
print(corr_matrix)

```

Expected Output:

```

=== ENTROPY-BASED MEASURES ===
facebook: 1.585 bits
linkedin: 1.500 bits
twitter: 1.500 bits

=== MUTUAL INFORMATION ===
I(facebook; linkedin) = 0.000 bits

```

(continues on next page)

(continued from previous page)

```

=== LAYER INFLUENCE ===
facebook: 1.000
linkedin: 1.000
twitter: 1.000

=== BETWEENNESS SURFACE ===
Shape: (4, 3)
Mean betweenness: 0.158

=== DEGREE CORRELATION MATRIX ===
Correlation matrix:
[[ 1.    0.5  -0.5 ]
 [ 0.5   1.    0. ]
 [-0.5   0.    1. ]]

```

6.25.11 API Reference

layer_connectivity_entropy(network, layer)

Calculate Shannon entropy of degree distribution within a layer.

Parameters: - network (multi_layer_network): The multilayer network - layer (str): Layer identifier

Returns: - float: Entropy in bits (0)

inter_layer_dependence_entropy(network, layer_i, layer_j)

Calculate entropy of inter-layer coupling patterns.

Parameters: - network (multi_layer_network): The multilayer network - layer_i (str): First layer identifier - layer_j (str): Second layer identifier

Returns: - float: Entropy in bits (0)

cross_layer_redundancy_entropy(network)

Calculate entropy of edge overlap across all layer pairs.

Parameters: - network (multi_layer_network): The multilayer network

Returns: - float: Entropy in bits (0)

cross_layer_mutual_information(network, layer_i, layer_j, bins=10)

Calculate mutual information between degree distributions.

Parameters: - network (multi_layer_network): The multilayer network - layer_i (str): First layer identifier - layer_j (str): Second layer identifier - bins (int): Number of bins for discretization (default: 10)

Returns: - float: Mutual information in bits (0)

layer_influence_centrality(network, layer, method='coupling', sample_size=100)

Calculate layer influence through coupling or flow.

Parameters: - network (multi_layer_network): The multilayer network - layer (str): Layer identifier - method (str): 'coupling' or 'flow' (default: 'coupling') - sample_size (int): Number of random walk steps for flow method (default: 100)

Returns: - float: Influence centrality value (0)

multilayer_betweenness_surface(network, normalized=True, weight=None)

Calculate betweenness centrality matrix for all node-layer pairs.

Parameters: - **network** (multi_layer_network): The multilayer network - **normalized** (bool): Normalize betweenness values (default: True) - **weight** (str): Edge weight attribute (default: None)

Returns: - tuple: (surface, (nodes, layers))

- **surface** (np.ndarray): 2D array of shape (num_nodes, num_layers)
- **nodes** (list): Node labels
- **layers** (list): Layer labels

interlayer_degree_correlation_matrix(network)

Calculate Pearson correlations of node degrees between all layer pairs.

Parameters: - **network** (multi_layer_network): The multilayer network

Returns: - tuple: (corr_matrix, layers)

- **corr_matrix** (np.ndarray): Symmetric correlation matrix
- **layers** (list): Layer labels

6.25.12 Best Practices

Choosing Metrics: - Use entropy measures for structural complexity analysis - Use mutual information for layer dependency analysis - Use influence centrality for identifying critical layers - Use betweenness surface for node-level analysis across layers - Use correlation matrix for layer similarity analysis

Computational Considerations: - Entropy and correlation metrics are fast ($O(E)$ or $O(N \times L)$) - Mutual information requires discretization (adjust `bins` parameter) - Flow-based influence requires random walk simulation (adjust `sample_size`) - Betweenness surface can be slow for large networks (consider sampling)

Interpretation Guidelines: - Compare metric values across networks, not absolute values - Use entropy as a relative measure of heterogeneity - Mutual information identifies functional dependencies - Layer influence guides intervention strategies - Betweenness surface reveals cross-layer bridge nodes

6.25.13 References

1. Shannon, C. E. (1948). "A Mathematical Theory of Communication." *Bell System Technical Journal*, 27(3), 379-423.
2. Cover, T. M., & Thomas, J. A. (2006). *Elements of Information Theory*. John Wiley & Sons.
3. De Domenico, M., et al. (2013). "Mathematical formulation of multilayer networks." *Physical Review X*, 3(4), 041022.
4. De Domenico, M., et al. (2015). "Structural reducibility of multilayer networks." *Nature Communications*, 6, 6864.
5. Bianconi, G. (2018). *Multilayer Networks: Structure and Function*. Oxford University Press.
6. Kivelä, M., et al. (2014). "Multilayer networks." *Journal of Complex Networks*, 2(3), 203-271.
7. Cozzo, E., et al. (2013). "Mathematical formulation of multilayer networks." *Physical Review E*, 88(5), 050801.

6.25.14 Examples

See the following example scripts in the repository:

- `examples/network_analysis/example_advanced_multilayer_metrics.py` - Comprehensive demonstration of all new metrics
- `examples/network_analysis/example_multilayer_statistics.py` - Basic multilayer statistics
- `examples/network_analysis/example_new_multiplex_metrics.py` - Multiplex-specific metrics

Repository: <https://github.com/SkBlaz/py3plex/tree/master/examples>

6.25.15 Related Documentation

- *Multiplex Network Analysis* - Basic multiplex network analysis
- *Statistical Comparison Framework* - Statistical comparison framework
- *Supra Matrix Function Centralities* - Centrality measures

6.26 Command-Line Interface (CLI) Tutorial

Py3plex includes a comprehensive command-line interface that provides access to all main algorithms and functionality directly from the terminal. This is useful for quick analysis, scripting, and automation workflows.

6.26.1 Installation

The CLI tool is automatically available after installing py3plex:

```
pip install git+https://github.com/SkBlaz/py3plex.git
```

After installation, verify the CLI is available:

```
py3plex --help
```

Alternative: Using Docker

If you prefer Docker, the CLI is also available via the Docker container:

```
# Build the Docker image
docker build -t py3plex:latest .

# Run CLI commands
docker run --rm py3plex:latest --help
docker run --rm -v $(pwd)/data:/data py3plex:latest create --nodes 100 --output /data/
network.edgelist
```

See *Docker Usage Guide* for complete Docker documentation.

6.26.2 Quick Start

New to py3plex? Start with these essential commands:

```
# Run an interactive demo with example graph
py3plex quickstart

# Verify installation
```

(continues on next page)

(continued from previous page)

```
py3plex selftest

# Check version
py3plex --version

# Get help
py3plex --help
```

The `quickstart` command creates a small demo multilayer network, computes statistics, generates a visualization, and shows you next steps. It's the fastest way to see py3plex in action!

6.26.3 Getting Help

The CLI includes built-in documentation for all commands:

```
# Main help with all commands
py3plex --help

# Help for specific command
py3plex create --help
py3plex community --help
py3plex centrality --help
```

6.26.4 Available Commands

The py3plex CLI provides these main commands:

- `quickstart` - Interactive demo with example graph (recommended for new users)
- `selftest` - Verify installation and core functionality
- `check` - Lint and validate graph data files
- `create` - Create new multilayer networks
- `load` - Load and inspect networks
- `community` - Detect communities
- `centrality` - Compute centrality measures
- `stats` - Compute multilayer network statistics
- `visualize` - Create network visualizations
- `aggregate` - Aggregate multilayer networks into single layer
- `convert` - Convert between network formats
- `help` - Show detailed help information

Quick Reference

Essential flags:

- `--version` - Display py3plex version
- `--help` - Display help for any command

Common workflows:


```
# Quick demo
py3plex quickstart

# Validate → Load → Analyze
py3plex check data.csv
py3plex load data.csv --stats

# Create → Analyze → Visualize
py3plex create --nodes 50 --layers 2 --output net.graphml
py3plex load net.graphml --stats
py3plex visualize net.graphml --output viz.png

# Community detection workflow
py3plex community net.graphml --algorithm louvain --output communities.json
```

6.26.5 Validating Data Files

Before loading network data, you can validate file format and data integrity using the `check` command. This helps catch common issues early and provides actionable suggestions for fixing problems.

Basic Validation

Validate a graph data file (auto-detects format):

```
py3plex check network.csv
```

Output for valid file:

```
Checking file: network.csv
✓ No issues found!
```

Output for file with issues:

```
Checking file: network.csv

Found 5 issue(s):

[INFO] No layer columns found - this appears to be a single-layer network
[ERROR] Line 3: Empty destination node
  → Suggestion: Provide a valid destination node ID
[WARNING] Line 4: Negative weight: -1.5
  → Suggestion: Negative weights may not be supported by all algorithms
[WARNING] Line 5: Self-loop detected: Dave -> Dave
  → Suggestion: Self-loops may not be supported by all algorithms
[WARNING] Line 6: Duplicate edge: Alice -> Bob
  → Suggestion: Remove duplicate edges or consolidate with edge weights

Linting results:
  Errors: 1
  Warnings: 3
  Info: 1

Validation failed with errors
```

Supported File Formats

The `check` command automatically detects and validates these formats:

- **CSV** - Comma-separated values with headers (e.g., `network.csv`)
- **Edgelist** - Space-separated node pairs (e.g., `network.edgelist`)
- **Multiedgelist** - Space-separated with layer information (e.g., `network.txt`)

Strict Mode

Use `--strict` to treat warnings as errors (useful for CI/CD pipelines):

```
py3plex check network.csv --strict
```

With strict mode enabled, the command exits with error code 1 if any warnings are found, making it suitable for automated validation in scripts.

Validation Checks

The linter performs these checks:

Errors (cause validation failure):

- Missing or empty node IDs
- Invalid weight values (non-numeric)
- Missing required columns (for CSV files)
- Malformed file structure

Warnings (reported but don't fail validation):

- Negative weights
- Self-loops
- Duplicate edges

Info (informational messages):

- Single-layer network detected (when no layer columns found)
- Format detection results

Example Workflows

Validate before loading:

```
# Step 1: Validate the file
py3plex check my_network.csv

# Step 2: If valid, load and analyze
py3plex load my_network.csv --stats
```

CI/CD integration:

```
# In your CI script - fail if any issues found
py3plex check network.csv --strict || exit 1
```

(continues on next page)

(continued from previous page)

```
# Proceed with analysis if validation passed
py3plex load network.csv --stats
```

Batch validation:

```
# Validate multiple files
for file in data/*.csv; do
    echo "Checking $file..."
    py3plex check "$file"
done
```

6.26.6 Creating Networks

Create Random Networks

Create a simple multilayer network with random edges:

```
py3plex create --nodes 100 --layers 3 --output network.graphml --seed 42
```

This creates a network with 100 nodes per layer and 3 layers, saving it to `network.graphml`.

Network Generation Models

Erdős-Rényi Random Graphs:

```
py3plex create --nodes 50 --layers 2 --type er --probability 0.15 \
    --output er_network.graphml --seed 42
```

Barabási-Albert Preferential Attachment:

```
py3plex create --nodes 100 --layers 3 --type ba --probability 0.05 \
    --output ba_network.graphml --seed 42
```

Watts-Strogatz Small-World:

```
py3plex create --nodes 50 --layers 2 --type ws --probability 0.2 \
    --output ws_network.graphml --seed 42
```

Output Formats

The CLI supports multiple output formats (automatically detected from file extension):

- `.graphml` - GraphML format (XML-based, recommended)
- `.gexf` - GEXF format (XML-based, for Gephi)
- `.gpickle` - Python pickle format (binary, fastest)

6.26.7 Loading and Inspecting Networks

Basic Network Information

Display basic network statistics:

```
py3plex load network.graphml --info
```

Output:

```
Network Information:
Nodes: 300
Edges: 450
Layers: 3 (layer1, layer2, layer3)
Directed: False
```

Detailed Statistics

Get comprehensive statistics including layer densities and clustering:

```
py3plex load network.graphml --stats
```

Output:

```
Network Information:
Nodes: 300
Edges: 450
Layers: 3 (layer1, layer2, layer3)
Directed: False

Basic Statistics:
Layer Densities:
  layer1: 0.0303
  layer2: 0.0298
  layer3: 0.0305
Avg Clustering: 0.0421
Avg Degree: 3.00
Max Degree: 12
```

Save to JSON

Export network information to JSON for further processing:

```
py3plex load network.graphml --info --output network_info.json
```

6.26.8 Community Detection

Louvain Algorithm

Detect communities using the Louvain modularity optimization algorithm:

```
py3plex community network.graphml --algorithm louvain --output communities.json
```

Output:

```
Loading network from network.graphml...
Detecting communities using louvain...
Found 8 communities
Community sizes: min=15, max=52, avg=37.5
Communities saved to communities.json
```

With Custom Resolution

Adjust the resolution parameter to find more or fewer communities:

```
# Higher resolution = more communities
py3plex community network.graphml --algorithm louvain --resolution 1.5

# Lower resolution = fewer communities
py3plex community network.graphml --algorithm louvain --resolution 0.5
```

Label Propagation

Use label propagation for fast community detection:

```
py3plex community network.graphml --algorithm label_prop --output communities.json
```

Infomap (Optional)

If Infomap is installed, use it for overlapping community detection:

```
py3plex community network.graphml --algorithm infomap --output communities.json
```

6.26.9 Centrality Measures

Degree Centrality

Compute degree centrality and show top nodes:

```
py3plex centrality network.graphml --measure degree --top 10
```

Output:

```
Loading network from network.graphml...
Computing degree centrality...

Top 10 nodes by degree centrality:
node42---layer1: 12.000000
node18---layer2: 11.000000
node77---layer1: 10.000000
...
```

Save Results

Export centrality scores to JSON:

```
py3plex centrality network.graphml --measure degree --output centrality.json
```

All Centrality Measures

Betweenness Centrality:

```
py3plex centrality network.graphml --measure betweenness --top 10
```

Closeness Centrality:

```
py3plex centrality network.graphml --measure closeness --top 10
```

Eigenvector Centrality:

```
py3plex centrality network.graphml --measure eigenvector --top 10
```

PageRank:

```
py3plex centrality network.graphml --measure pagerank --top 10
```

6.26.10 Multilayer Network Statistics

Comprehensive Statistics

Compute all available multilayer statistics:

```
py3plex stats network.graphml --measure all --output stats.json
```

Output includes:

- Layer densities for each layer
- Clustering coefficient
- Node activity samples
- Versatility centrality (top nodes)
- Edge overlap between layers

Specific Statistics

Layer Density:

```
py3plex stats network.graphml --measure layer_density
```

Clustering Coefficient:

```
py3plex stats network.graphml --measure clustering
```

Node Activity:

```
py3plex stats network.graphml --measure node_activity
```

Versatility Centrality:

```
py3plex stats network.graphml --measure versatility
```

Edge Overlap:

```
py3plex stats network.graphml --measure edge_overlap
```

6.26.11 Network Visualization

Multilayer Layout

Create a visualization using the specialized multilayer layout:

```
py3plex visualize network.graphml --layout multilayer --output network.png
```

Standard Layouts

Use NetworkX layouts for different visual styles:

Spring Layout (Force-Directed):

```
py3plex visualize network.graphml --layout spring --output network_spring.png
```

Circular Layout:

```
py3plex visualize network.graphml --layout circular --output network_circular.png
```

Kamada-Kawai Layout:

```
py3plex visualize network.graphml --layout kamada_kawai --output network_kk.png
```

Custom Figure Size

Adjust the output image dimensions:

```
py3plex visualize network.graphml --layout spring --width 15 --height 10 \
--output large_viz.png
```

6.26.12 Network Aggregation

Aggregate multiple layers into a single layer using different methods:

Sum Aggregation

Sum edge weights across layers:

```
py3plex aggregate network.graphml --method sum --output aggregated.graphml
```

Mean Aggregation

Average edge weights across layers:

```
py3plex aggregate network.graphml --method mean --output aggregated_mean.graphml
```

Other Aggregation Methods

- max - Take maximum edge weight across layers
- min - Take minimum edge weight across layers

6.26.13 Format Conversion

Convert Between Formats

Convert networks between different file formats:

To GEXF (for Gephi):

```
py3plex convert network.graphml --output network.gexf
```

To JSON:

```
py3plex convert network.graphml --output network.json
```

To gpickle (Python binary):

```
py3plex convert network.graphml --output network.gpickle
```

6.26.14 Complete Workflow Examples

Example 1: Create, Analyze, and Visualize

```
# Step 1: Create a Barabási-Albert network
py3plex create --nodes 100 --layers 3 --type ba --probability 0.05 \
  --output ba_network.graphml --seed 42

# Step 2: Detect communities
py3plex community ba_network.graphml --algorithm louvain \
  --output communities.json

# Step 3: Compute centrality
py3plex centrality ba_network.graphml --measure pagerank --top 20 \
  --output central_nodes.json

# Step 4: Visualize
py3plex visualize ba_network.graphml --layout spring --width 15 --height 10 \
  --output ba_visualization.png
```

Example 2: Load, Analyze Statistics, and Export

```
# Step 1: Load and inspect
py3plex load existing_network.graphml --stats

# Step 2: Compute comprehensive statistics
py3plex stats existing_network.graphml --measure all --output stats.json

# Step 3: Convert to GEXF for Gephi
py3plex convert existing_network.graphml --output for_gephi.gexf
```

Example 3: Multiple Analysis Pipeline

```
# Create network
py3plex create --nodes 50 --layers 2 --type er --probability 0.15 \
  --output network.graphml --seed 42

# Run all centrality measures
py3plex centrality network.graphml --measure degree --output degree.json
py3plex centrality network.graphml --measure betweenness --output betweenness.json
py3plex centrality network.graphml --measure pagerank --output pagerank.json
```

(continues on next page)

(continued from previous page)

```
# Detect communities with different algorithms
py3plex community network.graphml --algorithm louvain --output louvain.json
py3plex community network.graphml --algorithm label_prop --output label_prop.json

# Create multiple visualizations
py3plex visualize network.graphml --layout multilayer --output viz_ml.png
py3plex visualize network.graphml --layout spring --output viz_spring.png
py3plex visualize network.graphml --layout circular --output viz_circular.png
```

Example 4: Validate External Data Before Analysis

```
# Step 1: Validate the data file
py3plex check external_data.csv

# Step 2: If validation passes, load the network
py3plex load external_data.csv --info

# Step 3: Run analysis
py3plex stats external_data.csv --measure all --output stats.json

# Step 4: Detect communities
py3plex community external_data.csv --algorithm louvain --output communities.json

# Step 5: Create visualization
py3plex visualize external_data.csv --layout spring --output network_viz.png
```

6.26.15 Tips and Best Practices

1. **Validate Data Files First:** Always run `py3plex check` on external data files before analysis to catch format issues early.
2. **Use Seeds for Reproducibility:** Always use `--seed` when creating networks for reproducible results.
3. **JSON Output for Scripting:** Use `--output` to save results as JSON for post-processing with other tools.
4. **Check Help for Each Command:** Each command has detailed help with examples:

```
py3plex <command> --help
```

4. **Format Auto-Detection:** The CLI automatically detects file formats from extensions, so use appropriate extensions (`.graphml`, `.gexf`, `.json`, `.pickle`).
5. **Large Networks:** For large networks (>1000 nodes), prefer:
 - spring or circular layouts for visualization
 - louvain for community detection (faster than infomap)
 - degree centrality for quick analysis
6. **Batch Processing:** Combine CLI commands with shell scripts for batch processing multiple networks.

6.26.16 Common Issues and Solutions

Issue: “Infomap not available”

Solution: Infomap requires separate installation. Use `louvain` or `label_prop` instead.

Issue: “Eigenvector centrality failed”

Solution: Some networks don’t converge for eigenvector centrality. The CLI automatically falls back to degree centrality.

Issue: Visualization too slow

Solution: Use simpler layouts (`circular`) or reduce figure size (`--width 8 --height 6`).

Issue: Memory errors with large networks

Solution: Use aggregation to reduce network size before visualization, or compute statistics without visualization.

6.26.17 See Also

- *Quickstart Guide* - Python API quickstart
- *10-Minute Tutorial: Getting Started with py3plex* - 10-minute Python tutorial
- *Community Detection Tutorial* - Community detection algorithms in detail
- *Multilayer Network Centrality Measures* - Centrality measures for multilayer networks
- *Py3plex Documentation* - Main documentation index

6.27 Docker Usage Guide

This guide provides comprehensive instructions for using Py3plex via Docker.

Table of Contents

- *Quick Start*
- *Building the Image*
 - *Using Docker*
 - *Using Docker Compose*
- *Running Commands*
 - *Basic Commands*
 - *Using Docker Compose*
- *Working with Files*
 - *Creating a Data Directory*
 - *Mounting Volumes*
 - *Complete Workflow Example*
- *Advanced Usage*
 - *Interactive Shell*
 - *Using Python API*

- *Custom Image Builds*
- *Docker Compose with Multiple Services*
- *Helper Scripts*
 - *py3plex-docker.sh*
 - *test-docker-setup.sh*
- *Docker Configuration Files*
 - *.dockerignore*
- *Troubleshooting*
 - *Testing the Docker Setup*
 - *Permission Issues*
 - *Container Not Found*
 - *Network Issues During Build*
 - *Out of Disk Space*
 - *Debugging Build Issues*
- *Best Practices*
- *Practical Examples*
 - *Basic Network Analysis*
 - *Community Detection*
 - *Centrality Analysis*
 - *Batch Processing*
 - *Custom Analysis Script*
 - *Complete Analysis Pipeline*
 - *Comparative Network Analysis*
- *CI/CD Integration*
 - *GitHub Actions*
 - *GitLab CI*
- *Additional Resources*
- *Support*

6.27.1 Quick Start

The fastest way to get started with Py3plex using Docker:

```
# Clone the repository
git clone https://github.com/SkBlaz/py3plex.git
cd py3plex

# Build the Docker image
```

(continues on next page)

(continued from previous page)

```
docker build -t py3plex:latest .

# Run a self-test to verify installation
docker run --rm py3plex:latest selftest

# Display help
docker run --rm py3plex:latest help
```

6.27.2 Building the Image

Using Docker

```
# Build from the repository root
docker build -t py3plex:latest .

# Build with a specific tag
docker build -t py3plex:0.95a .

# Build without cache (if needed)
docker build --no-cache -t py3plex:latest .
```

Using Docker Compose

```
# Build using docker-compose
docker-compose build

# Force rebuild
docker-compose build --no-cache
```

6.27.3 Running Commands

Basic Commands

The Docker container is set up with py3plex as the entrypoint, so you can run any py3plex CLI command directly:

```
# Show version
docker run --rm py3plex:latest --version

# Run self-test
docker run --rm py3plex:latest selftest

# Show help
docker run --rm py3plex:latest help

# Show help for a specific command
docker run --rm py3plex:latest create --help
```

Using Docker Compose

With docker-compose, commands are slightly more verbose but easier to manage:

```
# Show version
docker-compose run --rm py3plex --version

# Run self-test
docker-compose run --rm py3plex selftest -v

# Show help
docker-compose run --rm py3plex help
```

6.27.4 Working with Files

To work with network files, you need to mount a local directory to the container's /data directory.

Creating a Data Directory

```
# Create a local data directory
mkdir -p data
```

Mounting Volumes

With docker run:

```
# Mount current directory's data folder
docker run --rm -v $(pwd)/data:/data py3plex:latest create --nodes 100 --layers 3 --
↳output /data/network.edgelist

# On Windows (PowerShell)
docker run --rm -v ${PWD}/data:/data py3plex:latest create --nodes 100 --layers 3 --
↳output /data/network.edgelist

# On Windows (CMD)
docker run --rm -v %cd%/data:/data py3plex:latest create --nodes 100 --layers 3 --output_
↳/data/network.edgelist
```

With docker-compose:

The docker-compose.yml file already configures volume mounting from ./data to /data, so you can simply:

```
docker-compose run --rm py3plex create --nodes 100 --layers 3 --output /data/network.
↳edgelist
```

Complete Workflow Example

```
# Create data directory
mkdir -p data

# 1. Create a multilayer network
docker run --rm -v $(pwd)/data:/data py3plex:latest \
  create --nodes 100 --layers 3 --type random --probability 0.1 --output /data/network.
↳edgelist
```

(continues on next page)

(continued from previous page)

```

# 2. Load and display network information
docker run --rm -v $(pwd)/data:/data py3plex:latest \
  load /data/network.edgelist --info

# 3. Compute statistics
docker run --rm -v $(pwd)/data:/data py3plex:latest \
  stats /data/network.edgelist --measure all --output /data/stats.json

# 4. Detect communities
docker run --rm -v $(pwd)/data:/data py3plex:latest \
  community /data/network.edgelist --algorithm louvain --output /data/communities.json

# 5. Visualize the network
docker run --rm -v $(pwd)/data:/data py3plex:latest \
  visualize /data/network.edgelist --output /data/network.png --layout multilayer

# 6. View the results
ls -lh data/

```

6.27.5 Advanced Usage

Interactive Shell

To explore the container interactively:

```

# Start an interactive shell in the container
docker run --rm -it -v $(pwd)/data:/data --entrypoint /bin/bash py3plex:latest

# Inside the container, you can run:
# py3plex --version
# py3plex selftest
# python -c "import py3plex; print(py3plex.__version__)"

```

Using Python API

You can also use py3plex as a Python library inside the container:

```

# Create a Python script
cat > data/analyze.py << 'EOF'
from py3plex.core import multinet

# Create a network
network = multinet.multi_layer_network()
nodes = [{"source": f"node{i}", "type": "layer1"} for i in range(10)]
network.add_nodes(nodes, input_type="dict")

print(f"Created network with {network.core_network.number_of_nodes()} nodes")
EOF

# Run the script in the container
docker run --rm -v $(pwd)/data:/data --entrypoint python py3plex:latest /data/analyze.py

```

Custom Image Builds

If you need additional packages:

```
# Create a custom Dockerfile
FROM py3plex:latest

# Install additional packages
RUN pip install --no-cache-dir pandas jupyter

# Or system packages
USER root
RUN apt-get update && apt-get install -y vim && rm -rf /var/lib/apt/lists/*
USER py3plex
```

Docker Compose with Multiple Services

You can extend the `docker-compose.yml` to include related services:

```
version: '3.8'

services:
  py3plex:
    build: .
    image: py3plex:latest
    volumes:
      - ./data:/data

  jupyter:
    image: py3plex:latest
    ports:
      - "8888:8888"
    volumes:
      - ./data:/data
      - ./notebooks:/notebooks
    command: jupyter notebook --ip=0.0.0.0 --port=8888 --no-browser --allow-root
    working_dir: /notebooks
```

6.27.6 Helper Scripts

The repository includes convenient helper scripts to simplify Docker usage.

py3plex-docker.sh

A wrapper script that simplifies running py3plex commands via Docker:

```
# The script automatically handles:
# - Docker installation checks
# - Image building (if needed)
# - Data directory creation
# - Volume mounting

# Usage examples:
./py3plex-docker.sh --version
```

(continues on next page)

(continued from previous page)

```
./py3plex-docker.sh selftest
./py3plex-docker.sh create --nodes 100 --layers 3 --output /data/network.edgelist
./py3plex-docker.sh load /data/network.edgelist --info
```

Features:

- Automatically checks if Docker is installed
- Prompts to build the image if it doesn't exist
- Creates the data/ directory if needed
- Mounts the local data/ directory to /data in the container
- Passes all arguments directly to py3plex

Script location: `py3plex-docker.sh` in the repository root

test-docker-setup.sh

A comprehensive test script to validate your Docker setup:

```
# Run the validation script
./test-docker-setup.sh
```

What it tests:

1. Docker installation verification
2. Dockerfile existence check
3. `.dockerignore` existence check
4. `docker-compose.yml` existence check
5. Docker image build process
6. Container version command
7. Container help command
8. Container selftest
9. Volume mounting and file creation

The script provides colored output ([OK] PASS / [X] FAIL) and a summary report.

Script location: `test-docker-setup.sh` in the repository root

6.27.7 Docker Configuration Files

.dockerignore

The `.dockerignore` file controls which files are excluded from the Docker build context:

Key exclusions:

- **Git and version control:** `.git/`, `.github/`, `.gitignore`
- **Python artifacts:** `__pycache__/`, `*.pyc`, `dist/`, `build/`
- **Virtual environments:** `venv/`, `env/`
- **Testing and coverage:** `.pytest_cache/`, `htmlcov/`, `coverage.*`

- **Documentation builds:** docs/_build/, docfiles/
- **Examples and datasets:** examples/, datasets/, multilayer_datasets/
- **Development files:** tests/, run_tests.py, validate_*.py

Why this matters:

- **Faster builds:** Smaller build context means faster Docker builds
- **Smaller images:** Excludes unnecessary files from the final image
- **Security:** Prevents accidental inclusion of sensitive files
- **Efficiency:** Only includes files needed to run py3plex

The .dockerignore file is located in the repository root.

6.27.8 Troubleshooting

Testing the Docker Setup

To verify your Docker setup is working correctly, use the included test script:

```
# Run the test script
./test-docker-setup.sh
```

This script will:

- Check Docker installation
- Verify all Docker-related files exist
- Build the Docker image
- Run basic py3plex commands (--version, help, selftest)
- Test volume mounting and file creation
- Clean up test artifacts

Permission Issues

If you encounter permission issues with mounted volumes:

```
# On Linux, you might need to run with your user ID
docker run --rm -v $(pwd)/data:/data --user $(id -u):$(id -g) py3plex:latest create --
→output /data/network.edgelist
```

Container Not Found

If the image isn't found:

```
# List available images
docker images | grep py3plex

# Rebuild if necessary
docker build -t py3plex:latest .
```

Network Issues During Build

If you encounter network timeouts during build:

```
# Increase timeout
docker build --build-arg PIP_DEFAULT_TIMEOUT=300 -t py3plex:latest .

# Or use a different PyPI mirror
docker build --build-arg PIP_INDEX_URL=https://pypi.tuna.tsinghua.edu.cn/simple -t
py3plex:latest .
```

Out of Disk Space

Clean up unused Docker resources:

```
# Remove unused images
docker image prune -a

# Remove all stopped containers, unused networks, and dangling images
docker system prune -a
```

Debugging Build Issues

Build with verbose output:

```
docker build --progress=plain --no-cache -t py3plex:latest .
```

6.27.9 Best Practices

1. **Use Volume Mounts:** Always mount volumes for input/output files
2. **Use `--rm` Flag:** Remove containers after execution with `--rm` to save space
3. **Tag Images:** Use specific tags (e.g., `py3plex:0.95a`) for reproducibility
4. **Keep Data Separate:** Store network files in the mounted data directory
5. **Use Docker Compose:** For repeated operations, docker-compose simplifies commands
6. **Regular Updates:** Rebuild the image periodically to get latest py3plex updates

6.27.10 Practical Examples

Basic Network Analysis

Create and analyze a multilayer network:

```
# Create data directory
mkdir -p data

# Create a network
docker run --rm -v $(pwd)/data:/data py3plex:latest \
  create --nodes 100 --layers 3 --type random --probability 0.05 \
  --output /data/network.edgelist

# Analyze the network
docker run --rm -v $(pwd)/data:/data py3plex:latest \
```

(continues on next page)

(continued from previous page)

```
load /data/network.edgelist --info --stats

# Visualize
docker run --rm -v $(pwd)/data:/data py3plex:latest \
  visualize /data/network.edgelist --output /data/network.png
```

Community Detection

```
# Detect communities using Louvain
docker run --rm -v $(pwd)/data:/data py3plex:latest \
  community /data/network.edgelist --algorithm louvain \
  --output /data/communities.json

# View community statistics
cat data/communities.json | python -m json.tool | head -20
```

Centrality Analysis

```
# Compute degree centrality
docker run --rm -v $(pwd)/data:/data py3plex:latest \
  centrality /data/network.edgelist --measure degree \
  --top 10 --output /data/centrality.json

# Compute betweenness centrality
docker run --rm -v $(pwd)/data:/data py3plex:latest \
  centrality /data/network.edgelist --measure betweenness \
  --top 10 --output /data/betweenness.json
```

Batch Processing

Process multiple networks (save as batch-process.sh):

```
#!/bin/bash
# Process multiple networks in batch
set -e

DATA_DIR=$(pwd)/data
mkdir -p $DATA_DIR

# Create several networks
for i in {1..5}; do
  echo "Creating network $i..."
  docker run --rm -v $DATA_DIR:/data py3plex:latest \
    create --nodes $((50 * i)) --layers 3 \
    --type random --probability 0.1 \
    --output /data/network_$i.edgelist
done

# Analyze each network
for i in {1..5}; do
  echo "Analyzing network $i..."
```

(continues on next page)

(continued from previous page)

```
docker run --rm -v $DATA_DIR:/data py3plex:latest \
  stats /data/network_${i}.edgelist --measure all \
  --output /data/stats_${i}.json
done

echo "Done! Results in $DATA_DIR"
```

Custom Analysis Script

Create a Python script (analysis.py) and run it in the container:

Python script:

```
# analysis.py
from py3plex.core import multinet
from py3plex.algorithms.statistics import multilayer_statistics as mls

# Load network
network = multinet.multi_layer_network()
network.load_network("/data/network.edgelist", input_type="multiedgelist")

# Compute statistics
layers = ["layer1", "layer2", "layer3"]
for layer in layers:
    density = mls.layer_density(network, layer)
    print(f"{layer} density: {density:.4f}")

print("Analysis complete!")
```

Run it:

```
# Copy script to data directory
cp analysis.py data/

# Run in container
docker run --rm -v $(pwd)/data:/data \
  --entrypoint python py3plex:latest /data/analysis.py
```

Complete Analysis Pipeline

A comprehensive analysis pipeline:

```
#!/bin/bash
# complete-pipeline.sh
set -e

DATA_DIR=$(pwd)/data
mkdir -p $DATA_DIR

echo "Step 1: Creating network..."
docker run --rm -v $DATA_DIR:/data py3plex:latest \
  create --nodes 200 --layers 3 --type ba --probability 0.05 \
  --output /data/network.edgelist
```

(continues on next page)

(continued from previous page)

```

echo "Step 2: Computing statistics..."
docker run --rm -v $DATA_DIR:/data py3plex:latest \
  stats /data/network.edgelist --measure all \
  --output /data/stats.json

echo "Step 3: Detecting communities..."
docker run --rm -v $DATA_DIR:/data py3plex:latest \
  community /data/network.edgelist --algorithm louvain \
  --output /data/communities.json

echo "Step 4: Computing centrality..."
docker run --rm -v $DATA_DIR:/data py3plex:latest \
  centrality /data/network.edgelist --measure pagerank \
  --top 20 --output /data/centrality.json

echo "Step 5: Visualizing..."
docker run --rm -v $DATA_DIR:/data py3plex:latest \
  visualize /data/network.edgelist --output /data/network.png \
  --layout multilayer --width 16 --height 12

echo "Pipeline complete! Check $DATA_DIR for results."

```

Comparative Network Analysis

Compare different network types:

```

#!/bin/bash
# compare-networks.sh
set -e

DATA_DIR=$(pwd)/data
mkdir -p $DATA_DIR

TYPES=("random" "er" "ba" "ws")

for type in "${TYPES[@]}; do
  echo "Creating $type network..."
  docker run --rm -v $DATA_DIR:/data py3plex:latest \
    create --nodes 100 --layers 3 --type $type --probability 0.1 \
    --output /data/network_${type}.edgelist

  echo "Analyzing $type network..."
  docker run --rm -v $DATA_DIR:/data py3plex:latest \
    stats /data/network_${type}.edgelist --measure all \
    --output /data/stats_${type}.json
done

echo "Comparison complete!"

```

6.27.11 CI/CD Integration

GitHub Actions

Example workflow for network analysis:

```
# .github/workflows/network-analysis.yml
name: Network Analysis with Docker

on: [push, pull_request]

jobs:
  analyze:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3

      - name: Build Docker image
        run: docker build -t py3plex:latest .

      - name: Create test network
        run: |
          mkdir -p data
          docker run --rm -v $PWD/data:/data py3plex:latest \
            create --nodes 50 --layers 2 --output /data/test.edgelist

      - name: Run analysis
        run: |
          docker run --rm -v $PWD/data:/data py3plex:latest \
            stats /data/test.edgelist --measure all --output /data/stats.json

      - name: Upload results
        uses: actions/upload-artifact@v3
        with:
          name: analysis-results
          path: data/
```

GitLab CI

Example GitLab CI configuration:

```
# .gitlab-ci.yml
image: docker:latest

services:
  - docker:dind

variables:
  DOCKER_DRIVER: overlay2

stages:
  - build
  - analyze
```

(continues on next page)

(continued from previous page)

```

build:
  stage: build
  script:
    - docker build -t py3plex:latest .
    - docker save py3plex:latest > py3plex-image.tar
  artifacts:
    paths:
      - py3plex-image.tar

analyze:
  stage: analyze
  script:
    - docker load < py3plex-image.tar
    - mkdir -p data
    - docker run --rm -v $PWD/data:/data py3plex:latest create --nodes 100 --layers 3 --
↪output /data/network.edgelist
    - docker run --rm -v $PWD/data:/data py3plex:latest stats /data/network.edgelist --
↪measure all --output /data/stats.json
  artifacts:
    paths:
      - data/

```

6.27.12 Additional Resources

- [Py3plex Documentation](#)
- *CLI Tutorial*
- [Docker Documentation](#)
- [Docker Compose Documentation](#)
- [DOCKER.md](#) - Main Docker guide in the repository

6.27.13 Support

For issues related to:

- **Py3plex functionality:** Open an issue at <https://github.com/SkBlaz/py3plex/issues>
- **Docker setup:** Check this guide first, then open an issue if problems persist
- **General questions:** See the main README.md and documentation

6.28 CSV Data Loading Examples

This guide provides practical examples for loading multilayer network data from CSV files, which is one of the most common formats for network data.

Table of Contents

- *Standard CSV Schema*
 - *Multilayer Edge List Format*

- *Loading Multilayer CSV*
- *Simple Edge List Format*
- *Loading Simple CSV*
- *Creating CSV from Pandas DataFrame*
 - *Building Network from Existing Data*
- *Alternative Formats*
 - *Space-Delimited Multiedgelist*
- *Common Issues and Solutions*
 - *Issue: “Layer name missing” error*
 - *Issue: “Could not load network” error*
 - *Issue: Encoding errors with special characters*
- *Complete Example Workflow*
 - *End-to-End Pipeline: CSV → Analysis → Visualization*
- *Validation Before Loading*
 - *Pre-validate CSV Format*
- *Tips and Best Practices*
- *Next Steps*

6.28.1 Standard CSV Schema

Multilayer Edge List Format

The most common format for multilayer networks is a CSV with these columns:

Required columns: source, target, layer **Optional columns:** weight (defaults to 1.0)

Example CSV file (save as `network.csv`):

```
source,target,layer,weight
A,B,collaboration,1.0
A,C,dependency,0.8
B,C,collaboration,1.0
A,B,dependency,0.5
C,D,collaboration,0.9
```

Loading Multilayer CSV

```
from py3plex.core import multinet

# Create network object
network = multinet.multi_layer_network()

# Load from CSV
network.load_network(
```

(continues on next page)

(continued from previous page)

```

    "network.csv",
    input_type="multiedgelist",
    directed=False
)

# Display basic statistics
network.basic_stats()

```

Expected Output:

```

Number of nodes: 4
Number of edges: 5
Number of unique nodes (as node-layer tuples): 7
Number of unique node IDs (across all layers): 4
Nodes per layer:
  Layer 'collaboration': 4 nodes
  Layer 'dependency': 3 nodes

```

Simple Edge List Format

For single-layer networks, use a simpler format:

Required columns: source, target **Optional columns:** weight (defaults to 1.0)

Example CSV file (save as `simple_network.csv`):

```

source,target,weight
A,B,1.0
B,C,0.8
C,D,1.5
D,A,0.9

```

Loading Simple CSV

```

from py3plex.core import multinet

network = multinet.multi_layer_network()

# Load simple edge list
network.load_network(
    "simple_network.csv",
    input_type="edgelist",
    directed=False
)

network.basic_stats()

```

6.28.2 Creating CSV from Pandas DataFrame**Building Network from Existing Data**

If you have data in a Pandas DataFrame, convert it to CSV format:

```
import pandas as pd
from py3plex.core import multinet

# Create sample data
data = {
    'source': ['Alice', 'Alice', 'Bob', 'Bob', 'Charlie'],
    'target': ['Bob', 'Charlie', 'Charlie', 'David', 'David'],
    'layer': ['friendship', 'work', 'friendship', 'work', 'friendship'],
    'weight': [1.0, 0.8, 1.0, 0.6, 0.9]
}
df = pd.DataFrame(data)

# Save to CSV
df.to_csv('social_network.csv', index=False)

# Load into py3plex
network = multinet.multi_layer_network()
network.load_network('social_network.csv', input_type="multiedgelist")

print(f"Loaded {network.core_network.number_of_nodes()} nodes")
print(f"Loaded {network.core_network.number_of_edges()} edges")
```

6.28.3 Alternative Formats

Space-Delimited Multiedgelist

Py3plex also supports space-delimited files (no header):

Format: source layer1 target layer2 [weight]

Example file (save as network.txt):

```
A layer1 B layer1 1.0
B layer1 C layer1 1.0
A layer2 B layer2 0.8
B layer2 D layer2 0.6
```

```
# Load space-delimited format
network = multinet.multi_layer_network()
network.load_network(
    "network.txt",
    input_type="multiedgelist",
    directed=False
)
```

6.28.4 Common Issues and Solutions

Issue: “Layer name missing” error

Problem: CSV is missing the ‘layer’ column

Solution 1: Add layer column to CSV

```
import pandas as pd

# Read CSV without layer
df = pd.read_csv('network.csv')

# Add default layer
df['layer'] = 'layer1'

# Save updated CSV
df.to_csv('network_with_layer.csv', index=False)
```

Solution 2: Use ‘edgelist’ format for single-layer networks

```
# If your data has only source/target columns
network.load_network(
    "network.csv",
    input_type="edgelist", # Use edgelist instead of multiedgelist
    directed=False
)
```

Issue: “Could not load network” error

Problem: CSV format doesn’t match expected schema

Solution: Verify CSV structure

```
import pandas as pd

# Check CSV columns
df = pd.read_csv('network.csv')
print("Columns:", df.columns.tolist())
print("\nFirst few rows:")
print(df.head())

# Verify required columns exist
required = ['source', 'target', 'layer']
missing = [col for col in required if col not in df.columns]

if missing:
    print(f"\nMissing columns: {missing}")
else:
    print("\n[OK] All required columns present")
```

Issue: Encoding errors with special characters

Problem: CSV contains non-ASCII characters (é, ñ, , etc.)

Solution: Specify encoding when creating CSV

```
import pandas as pd

# Save with UTF-8 encoding
df.to_csv('network.csv', index=False, encoding='utf-8')
```

(continues on next page)

(continued from previous page)

```
# If loading fails, try reading with explicit encoding
df = pd.read_csv('network.csv', encoding='utf-8')
```

6.28.5 Complete Example Workflow

End-to-End Pipeline: CSV → Analysis → Visualization

```
import pandas as pd
from py3plex.core import multinet
from py3plex.visualization.multilayer import draw_multilayer_default
from py3plex.algorithms.community_detection import community_louvain
import matplotlib.pyplot as plt

# Step 1: Create sample CSV data
data = {
    'source': ['A', 'A', 'B', 'B', 'C', 'C', 'D'],
    'target': ['B', 'C', 'C', 'D', 'D', 'E', 'E'],
    'layer': ['social', 'work', 'social', 'work', 'social', 'work', 'social'],
    'weight': [1.0, 0.8, 1.0, 0.7, 0.9, 0.6, 0.8]
}
df = pd.DataFrame(data)
df.to_csv('example_network.csv', index=False)

# Step 2: Load network
network = multinet.multi_layer_network()
network.load_network('example_network.csv', input_type="multiedgelist")

# Step 3: Basic analysis
print("=== Network Statistics ===")
network.basic_stats()

# Step 4: Community detection
communities = community_louvain.best_partition(network.core_network)
print(f"\nDetected {len(set(communities.values()))} communities")

# Step 5: Centrality analysis
from py3plex.algorithms.statistics import basic_statistics
hubs = basic_statistics.identify_n_hubs(network.core_network, top_n=3)
print("\nTop 3 hub nodes:")
for node, degree in hubs.items():
    print(f" {node}: degree {degree}")

# Step 6: Visualization
fig, ax = plt.subplots(1, 1, figsize=(10, 8))
draw_multilayer_default(
    network.get_layers(),
    display=False,
    axis=ax,
    labels=True,
    background_shape="circle"
)
plt.title("Multilayer Network from CSV")
```

(continues on next page)

(continued from previous page)

```
plt.savefig('network_visualization.png', dpi=300, bbox_inches='tight')
print("\n[OK] Visualization saved to network_visualization.png")

# Step 7: Export to NetworkX for further analysis
nx_graph = network.to_nx_network()
print(f"\n[OK] Exported to NetworkX: {nx_graph.number_of_nodes()} nodes, "
      f"{nx_graph.number_of_edges()} edges")
```

Expected Output:

```
=== Network Statistics ===
Number of nodes: 5
Number of edges: 7
Number of unique nodes (as node-layer tuples): 9
Number of unique node IDs (across all layers): 5
Nodes per layer:
  Layer 'social': 5 nodes
  Layer 'work': 5 nodes

Detected 2 communities

Top 3 hub nodes:
  ('C', 'social'): degree 3
  ('D', 'social'): degree 3
  ('B', 'social'): degree 2

[OK] Visualization saved to network_visualization.png
[OK] Exported to NetworkX: 9 nodes, 7 edges
```

6.28.6 Validation Before Loading

Pre-validate CSV Format

Use the validation module to check CSV format before loading:

```
from py3plex.validation import validate_network_data, ParsingError

try:
    # Validate before loading
    validate_network_data('network.csv', 'multiedgelist')
    print("[OK] Validation passed")

    # Safe to load
    network = multinet.multi_layer_network()
    network.load_network('network.csv', input_type='multiedgelist')

except ParsingError as e:
    print(f"[X] Validation failed:\n{e}")
    # Fix CSV and try again
```

This performs checks for:

- File exists and is readable

- Required columns are present
- CSV format is valid
- Data types are appropriate

6.28.7 Tips and Best Practices

1. **Always use column names** in CSV (first row should be header)
2. **Use UTF-8 encoding** for files with special characters
3. **Validate data** before loading large networks
4. **Use meaningful layer names** (e.g., ‘friendship’, ‘work’ instead of ‘layer1’, ‘layer2’)
5. **Normalize weights** if needed (e.g., scale to [0, 1] range)
6. **Handle missing values** appropriately:

```
import pandas as pd

# Load CSV and handle missing values
df = pd.read_csv('network.csv')

# Fill missing weights with default
df['weight'] = df['weight'].fillna(1.0)

# Remove rows with missing source/target
df = df.dropna(subset=['source', 'target', 'layer'])

# Save cleaned data
df.to_csv('network_cleaned.csv', index=False)
```

6.28.8 Next Steps

After loading CSV data:

- quickstart - Learn basic operations
- basic_usage_analysis - Perform network analysis
- visualization - Create publication-quality visualizations
- *Community Detection Tutorial* - Detect communities and clusters

For more examples, see the [examples/ directory](#) in the GitHub repository.

6.29 Multilayer Network Centrality Measures

This tutorial demonstrates how to compute **various centrality measures** for **multilayer** and **multiplex** networks using py3plex.

6.29.1 Overview

The `py3plex.algorithms.multilayer_algorithms.centrality` module provides **comprehensive implementations** of centrality measures specifically designed for **multilayer networks**. These measures follow **standard definitions** from multilayer network analysis literature.

6.29.2 Supported Centrality Measures

1. Degree/Strength-based Measures

- **Layer-specific degree/strength**: Centrality computed **within individual layers**
- **Supra degree/strength**: Centrality on the **full supra-adjacency matrix**
- **Overlapping degree/strength**: **Node-level aggregation** across all layers
- **Participation coefficient**: Measures how **evenly** a node's connections are distributed across layers

2. Eigenvector-type Measures

- **Multiplex eigenvector centrality**: **Principal eigenvector** of the supra-adjacency matrix
- **Eigenvector versatility**: **Node-level aggregation** of eigenvector centrality
- **Katz-Bonacich centrality**: **Weighted sum of walks** on the supra-graph
- **PageRank centrality**: **Random walk** centrality on the supra-graph

3. Path-based Measures

- **Multilayer closeness centrality**: Based on **shortest paths** in the supra-graph
- **Multilayer betweenness centrality**: Fraction of **shortest paths** passing through each node-layer

4. Advanced Measures

- **HITS centrality**: **Hubs and authorities** (equals eigenvector for undirected networks)
- **Current-flow closeness/betweenness**: Based on **electrical current flow** through the network
- **Subgraph centrality**: Counts **closed walks** via matrix exponential
- **Total communicability**: Sum of **matrix exponential** entries
- **Multiplex k-core**: **Core decomposition** of the multilayer network

5. Extended Measures

- **Information centrality**: **Stephenson-Zelen style** using modified Laplacian inverse
- **Communicability betweenness**: **Estrada-style** measure using matrix exponential
- **Accessibility centrality**: **Entropy-based** measure of h-step reach diversity
- **Percolation centrality**: **Bond percolation** Monte Carlo simulation
- **Spreading (Epidemic) centrality**: **SIR/IC model** based outbreak size
- **Collective influence**: **CI_** identifying influential spreaders at distance
- **Load centrality**: **Shortest-path load** through nodes
- **Flow betweenness**: **Maximum flow** based measure
- **Harmonic closeness**: **Handles disconnections** better than standard closeness
- **Edge betweenness**: **Edge-level** shortest path measure
- **Bridging centrality**: **Combines betweenness** with bridging coefficient
- **Local efficiency**: **Neighbor subgraph** efficiency measure
- **Lp-aggregated centrality**: **Framework** for combining per-layer measures

6. Co-Ranking Algorithms

- **MultiRank**: Co-ranking of nodes and layers through iterative updates
- **Multiplex PageRank variants** (Halu et al. 2013):
 - **Neutral**: Baseline with no cross-layer influence
 - **Additive**: Cross-layer influence via sum
 - **Multiplicative**: Cross-layer influence via product
 - **Combined**: Both additive and multiplicative coupling

6.29.3 Basic Usage

```
from py3plex.core import multinet
from py3plex.algorithms.multilayer_algorithms.centralities import MultilayerCentrality

# Create a multilayer network
network = multinet.multi_layer_network(directed=False)
network.add_edges([
    ['A', 'L1', 'B', 'L1', 1],
    ['B', 'L1', 'C', 'L1', 1],
    ['A', 'L2', 'C', 'L2', 1]
], input_type='list')

# Initialize centrality calculator
calc = MultilayerCentrality(network)

# Compute various centrality measures
layer_degrees = calc.layer_degree_centralities(weighted=False)
overlapping_degrees = calc.overlapping_degree_centralities(weighted=False)
participation = calc.participation_coefficient(weighted=False)
eigenvector = calc.multiplex_eigenvector_centrality()
pagerank = calc.pagerank_centrality()
hits = calc.hits_centrality()
subgraph = calc.subgraph_centrality()
k_core = calc.multiplex_k_core()
```

Example Output:

```
print("Layer degrees:", layer_degrees)
# Layer degrees: {'L1': {'A': 1, 'B': 2, 'C': 1}, 'L2': {'A': 1, 'C': 1}}

print("Overlapping degrees:", overlapping_degrees)
# Overlapping degrees: {'A': 2, 'B': 2, 'C': 2}

print("Participation coefficient:", participation)
# Participation coefficient: {'A': 1.0, 'B': 0.0, 'C': 1.0}
# (Node B only appears in one layer, so participation is 0)
```


6.29.4 Computing All Centralities

For convenience, you can compute **all available centrality measures** at once:

```
from py3plex.algorithms.multilayer_algorithms.centrality import compute_all_centralities

# Compute basic centralities
basic_centralities = compute_all_centralities(network)

# Include path-based measures (computationally expensive)
with_paths = compute_all_centralities(network, include_path_based=True)

# Include all advanced measures (most comprehensive but computationally intensive)
all_centralities = compute_all_centralities(network,
                                           include_path_based=True,
                                           include_advanced=True)
```

6.29.5 Aggregation Methods

The module provides **several methods** to aggregate node-layer centralities to node-level centralities:

```
# Sum aggregation (default)
node_centralities = calc.aggregate_to_node_level(layer_centralities, method='sum')

# Mean aggregation
node_centralities = calc.aggregate_to_node_level(layer_centralities, method='mean')

# Maximum aggregation
node_centralities = calc.aggregate_to_node_level(layer_centralities, method='max')

# Weighted sum with custom layer weights
layer_weights = {'L1': 2.0, 'L2': 1.0}
node_centralities = calc.aggregate_to_node_level(layer_centralities,
                                                method='weighted_sum',
                                                weights=layer_weights)
```

6.29.6 Mathematical Definitions

Degree-based Measures

Layer-specific degree (unweighted) * Undirected: $k_i^{[\alpha]} = \sum_j \mathbf{1}(A_{ij}^{[\alpha]} > 0)$ * Directed: $k_i^{[\alpha],\text{out}} = \sum_j \mathbf{1}(A_{ij}^{[\alpha]} > 0)$,
 $k_i^{[\alpha],\text{in}} = \sum_j \mathbf{1}(A_{ji}^{[\alpha]} > 0)$

Layer-specific strength (weighted) * $s_i^{[\alpha]} = \sum_j A_{ij}^{[\alpha]}$ (out-strength) * $s_i^{[\alpha],\text{in}} = \sum_j A_{ji}^{[\alpha]}$ (in-strength)

Supra degree/strength * $k_{i\alpha} = \sum_{j,\beta} \mathbf{1}(M_{(i,\alpha),(j,\beta)} > 0)$ * $s_{i\alpha} = \sum_{j,\beta} M_{(i,\alpha),(j,\beta)}$

Overlapping degree/strength * $k_i^{\text{over}} = \sum_{\alpha} k_i^{[\alpha]}$ * $s_i^{\text{over}} = \sum_{\alpha} s_i^{[\alpha]}$

Participation coefficient * $P_i = 1 - \sum_{\alpha=1}^m \left(\frac{k_i^{[\alpha]}}{k_i^{\text{over}}} \right)^2$

Eigenvector-type Measures

Multiplex eigenvector centrality * $x = \frac{1}{\lambda_{\max}} Mx$ where $x_{i\alpha}$ is the centrality of node i in layer α

Katz-Bonacich centrality * $z = \sum_{t=0}^{\infty} \alpha^t M^t b = (I - \alpha M)^{-1} b$

PageRank centrality * Standard PageRank algorithm applied to the supra-adjacency matrix

Advanced Measures

HITS centrality * For undirected networks: equivalent to eigenvector centrality * For directed networks: separate hub and authority scores

Subgraph centrality * $SC_i = (e^A)_{ii}$ where A is the adjacency matrix

Total communicability * $TC_i = \sum_j (e^A)_{ij}$ (row sum of matrix exponential)

Multiplex k-core * Core number: maximum k such that node has at least k neighbors in the multilayer network

6.29.7 Performance Considerations

- **Path-based measures** (closeness, betweenness) are **computationally expensive** as they require shortest path computations on the entire supra-graph
- **Advanced measures** (current-flow, communicability, k-core) involve **matrix operations** that can be costly for large networks
- **Matrix exponential** computations (subgraph centrality, communicability) are **particularly expensive**
- For **large networks**, consider using only degree-based and eigenvector-based measures
- Use `include_path_based=False` and `include_advanced=False` (defaults) in `compute_all_centralities()` for **better performance**

6.29.8 MultiRank and Multiplex PageRank Variants

The `py3plex.algorithms.multilayer_algorithms.multirank` module provides **co-ranking algorithms** that simultaneously rank nodes and layers.

MultiRank Usage

MultiRank iteratively updates **node scores** and **layer scores** based on their mutual influence:

```
from py3plex.algorithms.multilayer_algorithms.multirank import multirank
import numpy as np

# Create layer adjacency matrices
L1 = np.array([[0, 1, 1], [1, 0, 1], [1, 1, 0]], dtype=float)
L2 = np.array([[0, 1, 0], [1, 0, 1], [0, 1, 0]], dtype=float)

# Compute co-rankings
node_scores, layer_scores = multirank([L1, L2], alpha=0.85)

print("Node scores:", node_scores)
# Node scores: [0.33, 0.34, 0.33] (normalized)

print("Layer scores:", layer_scores)
# Layer scores: [0.6, 0.4] (L1 has more edges, higher score)
```

Key Parameters:

- *alpha*: Damping parameter (typically 0.85), controls random walk probability
- *tol*: Convergence tolerance (default 1e-6)
- *max_iter*: Maximum iterations (default 1000)
- *interlayer_coupling*: Optional L×L coupling matrix (default: identity)

Multiplex PageRank Variants Usage

The Halu et al. (2013) framework provides **four variants** of cross-layer influence:

```
from py3plex.algorithms.multilayer_algorithms.multirank import multiplex_pagerank
import numpy as np

# Create layer adjacency matrices
L1 = np.array([[0, 1, 1], [1, 0, 1], [1, 1, 0]], dtype=float)
L2 = np.array([[0, 1, 0], [1, 0, 1], [0, 1, 0]], dtype=float)

# Neutral variant (baseline, no cross-layer influence)
result_neutral = multiplex_pagerank([L1, L2], variant='neutral')

# Additive variant (cross-layer sum)
result_additive = multiplex_pagerank([L1, L2], variant='additive', c=0.5)

# Multiplicative variant (cross-layer product)
result_multiplicative = multiplex_pagerank([L1, L2],
                                           variant='multiplicative',
                                           c=0.5)

# Combined variant (additive + multiplicative)
result_combined = multiplex_pagerank([L1, L2],
                                     variant='combined',
                                     c1=0.5,
                                     c2=0.3)

# Access results
node_scores = result_additive['node_scores']      # Aggregated node scores
replica_scores = result_additive['replica_scores'] # Per-layer scores (N x L)
```

Coupling Functions:

- **Neutral:** $F^{ell_i}=1$ (no coupling)
- **Additive:** $F^{ell_i}=1+csum_{ell'neqell}r_i^{ell'}$
- **Multiplicative:** $F^{ell_i}=prod_{ell'neqell}(r_i^{ell'}+epsilon)^c$
- **Combined:** $F^{ell_i}=(1+c1sum_{ell'neqell}r_i^{ell'})timesprod_{ell'neqell}(r_i^{ell'}+epsilon)^{c2}$

Key Parameters:

- *variant*: One of ‘neutral’, ‘additive’, ‘multiplicative’, ‘combined’
- *c*: Coupling strength for additive/multiplicative (default 1.0)
- *c1, c2*: Coupling strengths for combined variant
- *epsilon*: Numerical stability constant (default 1e-12)

- *alpha*: Damping parameter (default 0.85)

6.29.9 Extended Centrality Measures

The extended centrality measures provide specialized metrics for different network analysis scenarios:

```
# Information centrality (Stephenson-Zelen style)
info_cent = calc.information_centrality()

# Communicability betweenness (considers all walks)
comm_between = calc.communicability_betweenness_centrality()

# Accessibility (entropy-based h-step reach)
accessibility = calc.accessibility_centrality(h=2)

# Harmonic closeness (better for disconnected graphs)
harmonic = calc.harmonic_closeness_centrality()

# Local efficiency (fault tolerance)
local_eff = calc.local_efficiency_centrality()

# Bridging centrality (identifies bridges)
bridging = calc.bridging_centrality()
```

Monte Carlo Simulations:

```
# Percolation centrality (bond percolation)
percolation = calc.percolation_centrality(
    edge_activation_prob=0.5,
    trials=100
)

# Spreading/epidemic centrality (SIR model)
spreading = calc.spreading_centrality(
    beta=0.2, # Infection rate
    mu=0.1,  # Recovery rate
    trials=50,
    steps=100
)

# Flow betweenness (max-flow based)
flow_between = calc.flow_betweenness_centrality(samples=100)
```

Influence and Load Measures:

```
# Collective influence (for influential spreaders)
collective = calc.collective_influence(radius=2)

# Load centrality (shortest-path load)
load = calc.load_centrality()

# Edge betweenness (edge-level measure)
edge_between = calc.edge_betweenness_centrality()
```

Aggregation Framework:

```

# Lp-aggregated centrality (combine per-layer measures)
# First get per-layer centralities
layer_degrees = calc.layer_degree_centrality(weighted=False)

# L2 norm aggregation
l2_agg = calc.lp_aggregated_centrality(layer_degrees, p=2)

# L-infinity norm (maximum)
linf_agg = calc.lp_aggregated_centrality(
    layer_degrees,
    p=float('inf')
)

# Weighted aggregation
weights = {'L1': 0.6, 'L2': 0.4}
weighted_agg = calc.lp_aggregated_centrality(
    layer_degrees,
    p=2,
    weights=weights
)

```

Computing All Extended Measures:

```

# Compute all centrality measures including extended ones
results = compute_all_centralities(
    network,
    include_extended=True
)

# Access specific measures
info_centrality = results['information']
spreading_cent = results['spreading']
harmonic_close = results['harmonic_closeness']

```

6.29.10 Example Applications

1. **Social network analysis:** Identify **influential users** across different communication platforms
2. **Transportation networks:** Find **critical stations/stops** across different transport modes
3. **Biological networks:** Analyze **protein interactions** across different cellular conditions
4. **Infrastructure networks:** Study **robustness** of multilayer utility networks
5. **Co-ranking applications:** Simultaneously rank **nodes and layers** for importance (MultiRank)
6. **Cross-platform influence:** Model **influence propagation** across interconnected platforms (Multiplex PageRank)
7. **Epidemic modeling:** Use **spreading centrality** to identify superspreaders in contact networks
8. **Network resilience:** Apply **percolation centrality** to assess infrastructure robustness
9. **Information flow:** Use **accessibility** and **local efficiency** to analyze communication networks

6.29.11 References

The implementations follow **standard definitions** from multilayer network analysis literature, particularly:

- Kivela, M., Arenas, A., Barthelemy, M., Gleeson, J. P., Moreno, Y., & Porter, M. A. (2014). **Multilayer networks**. Journal of complex networks, 2(3), 203-271.
- Boccaletti, S., Bianconi, G., Criado, R., Del Genio, C. I., Gómez-Gardenes, J., Romance, M., ... & Zanin, M. (2014). **The structure and dynamics of multilayer networks**. Physics Reports, 544(1), 1-122.
- Halu, A., Mondragon, R. J., Panzarasa, P., & Bianconi, G. (2013). **Multiplex PageRank**. PloS one, 8(10), e78293.
- De Domenico, M., Solé-Ribalta, A., Cozzo, E., Kivela, M., Moreno, Y., Porter, M. A., ... & Arenas, A. (2013). **Mathematical formulation of multilayer networks**. Physical Review X, 3(4), 041022.
- Stephenson, K., & Zelen, M. (1989). **Rethinking centrality: Methods and examples**. Social networks, 11(1), 1-37.
- Estrada, E., & Hatano, N. (2008). **Communicability in complex networks**. Physical Review E, 77(3), 036111.
- Estrada, E., Higham, D. J., & Hatano, N. (2009). **Communicability betweenness in complex networks**. Physica A, 388(5), 764-774.
- Morone, F., & Makse, H. A. (2015). **Influence maximization in complex networks through optimal percolation**. Nature, 524(7563), 65-68.
- Latora, V., & Marchiori, M. (2001). **Efficient behavior of small-world networks**. Physical review letters, 87(19), 198701.

6.30 Meta Flow Report - Comprehensive Network Analysis

The **Meta Flow Report** provides a unified interface for conducting multiple multilayer network analyses at once. Instead of running centrality measures, community detection, and statistics separately, you can now perform comprehensive analysis with a single function call.

6.30.1 Overview

The meta flow report enables you to:

- Compute **multiple centrality measures** simultaneously (degree, eigenvector, betweenness, etc.)
- Run **multiple community detection algorithms** at once (Louvain, Leiden)
- Calculate **various network statistics** together (density, clustering, etc.)
- Generate **human-readable reports** summarizing all results
- **Export results** for further analysis

This is particularly useful for:

- **Exploratory analysis:** Get a comprehensive view of your network quickly
- **Comparative studies:** Analyze multiple networks with consistent metrics
- **Reporting:** Generate summaries for papers, presentations, or documentation
- **Teaching:** Demonstrate multiple network analysis concepts at once

6.30.2 Quick Start

Basic Usage

The simplest way to use the meta flow report is with the `run_meta_analysis()` convenience function:

```
from py3plex.core import multinet
from py3plex.algorithms.meta_flow_report import run_meta_analysis

# Create or load your network
network = multinet.multi_layer_network(directed=False)
network.add_edges([
    ['Alice', 'facebook', 'Bob', 'facebook', 1],
    ['Bob', 'facebook', 'Carol', 'facebook', 1],
    ['Alice', 'twitter', 'Carol', 'twitter', 1],
    ['Alice', 'facebook', 'Alice', 'twitter', 1],
    ['Bob', 'facebook', 'Bob', 'twitter', 1],
    ['Carol', 'facebook', 'Carol', 'twitter', 1],
], input_type='list')

# Run comprehensive analysis with automatic summary
results = run_meta_analysis(network)
```

Output:

```
Computing centrality measures...
Running community detection...
Computing network statistics...

=====
META FLOW REPORT - Multilayer Network Analysis Summary
=====

CENTRALITY MEASURES
=====

Overlapping Degree:
  Alice                2.000000
  Bob                  2.000000
  Carol                2.000000

[... more centrality measures ...]

=====
COMMUNITY DETECTION
=====

Louvain Communities:
  Number of communities: 2
  Community sizes: {0: 4, 1: 2}

[... more results ...]
```

Selective Analysis

You can choose which analyses to run:

```
from py3plex.algorithms.meta_flow_report import run_meta_analysis

# Run only centralities and statistics (skip communities)
results = run_meta_analysis(
    network,
    include_centralities=True,
    include_communities=False,
    include_statistics=True,
    print_summary=True
)
```

Fine-Grained Control

For more control, use the MetaFlowReport class directly:

```
from py3plex.algorithms.meta_flow_report import MetaFlowReport

# Create report generator
report = MetaFlowReport(network)

# Run analyses separately
centralities = report.compute_centralities(
    include_path_based=False,
    include_advanced=False
)

communities = report.detect_communities(
    methods=['louvain'],
    gamma=1.0,
    omega=1.0
)

statistics = report.compute_statistics(
    include_advanced=False
)

# Or run all at once with custom options
results = report.run_all_analyses(
    include_centralities=True,
    include_communities=True,
    include_statistics=True,
    include_path_based=False,
    include_advanced=False
)

# Print summary
report.print_summary(results, top_n=10)
```


6.30.3 Analysis Components

Centrality Measures

The report computes the following centrality measures:

Degree-based:

- Layer-specific degree/strength
- Supra degree/strength
- Overlapping degree/strength
- Participation coefficient

Eigenvector-based:

- Multiplex eigenvector centrality
- Eigenvector versatility
- Katz-Bonacich centrality
- PageRank centrality

Path-based (optional, computationally expensive):

- Multilayer closeness centrality
- Multilayer betweenness centrality

Advanced (optional, computationally expensive):

- HITS centrality
- Current-flow closeness/betweenness
- Subgraph centrality
- Total communicability
- Multiplex k-core

```
# Include only basic measures (fast)
results = report.compute_centralities(
    include_path_based=False,
    include_advanced=False
)

# Include all measures (slow for large networks)
results = report.compute_centralities(
    include_path_based=True,
    include_advanced=True
)
```

Community Detection

The report supports multiple community detection algorithms:

- **Louvain:** Fast modularity optimization
- **Leiden:** Improved Louvain algorithm

```
# Run specific methods
communities = report.detect_communities(
    methods=['louvain', 'leiden'],
    gamma=1.0, # Resolution parameter
    omega=1.0 # Inter-layer coupling
)

# Results include community assignments and modularity scores
print(communities['louvain'])
# {'Alice', 'facebook': 0, ('Bob', 'facebook'): 0, ...}

print(communities['leiden']['modularity'])
# 0.523
```

Network Statistics

The report computes various network statistics:

Basic statistics:

- Layer densities
- Node activities
- Inter-layer coupling strength
- Edge overlap between layers

Advanced statistics (optional):

- Versatility centrality
- Multilayer clustering coefficient

```
stats = report.compute_statistics(
    include_advanced=False
)

# Access individual statistics
print(stats['layer_densities'])
# {'facebook': 0.667, 'twitter': 0.333}

print(stats['node_activities'])
# {'Alice': 1.0, 'Bob': 0.5, ...}
```

6.30.4 Working with Results

Extracting Top Nodes

Get the most important nodes by any measure:

```
# Run analysis first
report.run_all_analyses()

# Get top nodes by specific measures
top_by_degree = report.get_top_nodes(
    measure='overlapping_degree',
```

(continues on next page)

(continued from previous page)

```

    n=5,
    category='centralities'
)

print(top_by_degree)
# [('Alice', 4.0), ('Bob', 3.5), ...]

top_by_eigenvector = report.get_top_nodes(
    measure='multiplex_eigenvector',
    n=5,
    category='centralities'
)

```

Exporting Results

Export all results as a dictionary for further processing:

```

# Run analysis
report.run_all_analyses()

# Export results
results_dict = report.export_to_dict()

# Access specific results
centralities = results_dict['centralities']
communities = results_dict['communities']
statistics = results_dict['statistics']

# Save to file (example)
import json
with open('network_analysis.json', 'w') as f:
    json.dump(results_dict, f, indent=2, default=str)

```

Custom Summaries

Print custom summaries with different options:

```

# Show top 10 nodes instead of default 5
report.print_summary(results, top_n=10)

# Or create your own summary
if 'centralities' in results:
    overlapping = results['centralities']['overlapping_degree']
    top_nodes = sorted(overlapping.items(), key=lambda x: x[1], reverse=True)[:3]

    print("Most Central Nodes:")
    for node, centrality in top_nodes:
        print(f" {node}: {centrality:.4f}")

```

6.30.5 Performance Considerations

The meta flow report is designed to be efficient, but some measures are computationally expensive:

Fast (suitable for large networks):

- Degree-based centralities
- Eigenvector-based centralities
- Basic statistics

Slow (expensive for large networks):

- Path-based centralities (closeness, betweenness)
- Advanced centralities (current-flow, communicability)
- Advanced statistics

Recommendations:

```
# For large networks (>10,000 nodes): use basic measures only
results = run_meta_analysis(
    network,
    include_path_based=False,
    include_advanced=False
)

# For medium networks (1,000-10,000 nodes): include path-based
results = run_meta_analysis(
    network,
    include_path_based=True,
    include_advanced=False
)

# For small networks (<1,000 nodes): include all measures
results = run_meta_analysis(
    network,
    include_path_based=True,
    include_advanced=True
)
```

6.30.6 Complete Example

Here's a complete example analyzing a social network:

```
from py3plex.core import multinet
from py3plex.algorithms.meta_flow_report import MetaFlowReport

# Create a 3-layer social network
network = multinet.multi_layer_network(directed=False)

# Add connections in different platforms
network.add_edges([
    # Facebook layer
    ['Alice', 'facebook', 'Bob', 'facebook', 1],
    ['Bob', 'facebook', 'Carol', 'facebook', 1],
```

(continues on next page)

(continued from previous page)

```

['Carol', 'facebook', 'David', 'facebook', 1],

# Twitter layer
['Alice', 'twitter', 'Carol', 'twitter', 1],
['Bob', 'twitter', 'David', 'twitter', 1],

# LinkedIn layer
['Alice', 'linkedin', 'Bob', 'linkedin', 1],
['Carol', 'linkedin', 'David', 'linkedin', 1],

# Inter-layer connections
['Alice', 'facebook', 'Alice', 'twitter', 1],
['Alice', 'twitter', 'Alice', 'linkedin', 1],
['Bob', 'facebook', 'Bob', 'twitter', 1],
['Bob', 'twitter', 'Bob', 'linkedin', 1],
['Carol', 'facebook', 'Carol', 'twitter', 1],
['Carol', 'twitter', 'Carol', 'linkedin', 1],
['David', 'facebook', 'David', 'twitter', 1],
['David', 'twitter', 'David', 'linkedin', 1],
], input_type='list')

# Run comprehensive analysis
report = MetaFlowReport(network)
results = report.run_all_analyses(
    include_centralities=True,
    include_communities=True,
    include_statistics=True,
    include_path_based=False,
    include_advanced=False,
    gamma=1.0,
    omega=1.0
)

# Print summary
print("\n" + "="*80)
print("NETWORK ANALYSIS SUMMARY")
print("="*80)
report.print_summary(results, top_n=3)

# Extract key insights
print("\n" + "="*80)
print("KEY INSIGHTS")
print("="*80)

# Most influential users
top_users = report.get_top_nodes('overlapping_degree', n=3, category='centralities')
print("\nMost Active Users:")
for user, degree in top_users:
    print(f" {user}: {degree} connections")

# Community structure
if 'communities' in results and 'louvain' in results['communities']:

```

(continues on next page)

(continued from previous page)

```
comms = results['communities']['louvain']
num_communities = len(set(comms.values()))
print(f"\nCommunity Structure:")
print(f"    {num_communities} communities detected")

# Layer densities
if 'statistics' in results and 'layer_densities' in results['statistics']:
    densities = results['statistics']['layer_densities']
    print(f"\nLayer Activity:")
    for layer, density in sorted(densities.items(), key=lambda x: x[1], reverse=True):
        print(f"    {layer}: {density:.2%} of possible connections")
```

6.30.7 API Reference

run_meta_analysis(network, ...)

Convenience function for quick comprehensive analysis.

Parameters:

- **network**: py3plex multi_layer_network object
- **include_centralities** (bool): Compute centrality measures (default: True)
- **include_communities** (bool): Run community detection (default: True)
- **include_statistics** (bool): Compute statistics (default: True)
- **include_path_based** (bool): Include expensive path-based measures (default: False)
- **include_advanced** (bool): Include expensive advanced measures (default: False)
- **print_summary** (bool): Print summary of results (default: True)

Returns: Dictionary with analysis results

MetaFlowReport Class

Main class for comprehensive network analysis.

Methods:

- **compute_centralities**(include_path_based, include_advanced)
- **detect_communities**(methods, gamma, omega)
- **compute_statistics**(include_advanced)
- **run_all_analyses**(...)
- **print_summary**(results, top_n)
- **export_to_dict**()
- **get_top_nodes**(measure, n, category)

6.30.8 See Also

- *Multilayer Network Centrality Measures* - Detailed centrality measures documentation
- *Community Detection Tutorial* - Community detection algorithms
- `examples/network_analysis/example_meta_flow_report.py` - Complete example

6.30.9 References

This module integrates multiple analysis approaches from:

- Kivelä, M., et al. (2014). “Multilayer networks”. Journal of complex networks, 2(3), 203-271.
- Mucha, P. J., et al. (2010). “Community structure in time-dependent, multiscale, and multiplex networks”. Science, 328(5980), 876-878.
- De Domenico, M., et al. (2013). “Mathematical formulation of multilayer networks”. Physical Review X, 3(4), 041022.

6.31 Multilayer Modularity and Community Detection

This tutorial demonstrates how to use py3plex’s multilayer modularity maximization and community detection capabilities based on Mucha et al. (2010).

6.31.1 Overview

Multilayer modularity extends Newman’s modularity to networks with multiple layers (e.g., temporal or multiplex networks). The key idea is that each node-layer pair can be assigned to a community, allowing the same physical node to belong to different communities in different layers.

6.31.2 Mathematical Background

Multilayer Modularity Quality Function

The multilayer modularity Q is defined as:

$$Q_{\text{multilayer}} = \frac{1}{2\mu} \sum_{i,j} \sum_{\alpha,\beta} \left[(A_{ij}^{[\alpha]} - \gamma^{[\alpha]} P_{ij}^{[\alpha]}) \delta_{\alpha\beta} + \delta_{ij} \omega_{\alpha\beta} \right] \delta(g_{i,\alpha}, g_{j,\beta})$$

where: * $A_{ij}^{[\alpha]}$ is the adjacency matrix of layer α * $P_{ij}^{[\alpha]}$ is the expected weight under a null model (e.g., $k_i^\alpha k_j^\alpha / 2m_\alpha$) * $\gamma^{[\alpha]}$ is the resolution parameter for layer α * $\omega_{\alpha\beta}$ is the inter-layer coupling strength * $\delta(g_{i,\alpha}, g_{j,\beta}) = 1$ if node i in layer α and node j in layer β are in the same community

Supra-Adjacency Matrix Representation

A multilayer network can be represented as a supra-adjacency matrix of size $NL \times NL$ (for N nodes and L layers), where: * Diagonal blocks contain the intra-layer adjacency matrices $A^{[\alpha]}$ * Off-diagonal blocks encode inter-layer connections (e.g., ωI for identity coupling)

6.31.3 Basic Usage

Calculating Multilayer Modularity

```
from py3plex.core import multinet
from py3plex.algorithms.community_detection.multilayer_modularity import multilayer_
    ↪ modularity

# Create a multilayer network
network = multinet.multi_layer_network(directed=False)
network.add_edges([
    ['A', 'L1', 'B', 'L1', 1],
    ['B', 'L1', 'C', 'L1', 1],
```

(continues on next page)

(continued from previous page)

```

    ['C', 'L1', 'A', 'L1', 1],
    ['A', 'L2', 'B', 'L2', 1],
    ['B', 'L2', 'C', 'L2', 1]
], input_type='list')

# Assign communities (node-layer pairs to community IDs)
communities = {
    ('A', 'L1'): 0, ('B', 'L1'): 0, ('C', 'L1'): 1,
    ('A', 'L2'): 0, ('B', 'L2'): 0, ('C', 'L2'): 1
}

# Calculate modularity with uniform parameters
Q = multilayer_modularity(network, communities, gamma=1.0, omega=1.0)
print(f"Modularity: {Q:.3f}")

```

Layer-Specific Resolution Parameters

You can specify different resolution parameters for each layer:

```

# Different resolution per layer
gamma_dict = {
    'L1': 0.5, # Lower resolution (larger communities)
    'L2': 2.0 # Higher resolution (smaller communities)
}

Q = multilayer_modularity(network, communities, gamma=gamma_dict, omega=1.0)
print(f"Modularity with layer-specific gamma: {Q:.3f}")

```

Custom Inter-Layer Coupling

You can specify layer-pair specific coupling strengths:

```

import numpy as np

# Custom coupling matrix (3x3 for 3 layers)
omega_matrix = np.array([
    [0.0, 1.0, 0.5], # Layer 0 couplings
    [1.0, 0.0, 1.0], # Layer 1 couplings
    [0.5, 1.0, 0.0] # Layer 2 couplings
])

Q = multilayer_modularity(network, communities, gamma=1.0, omega=omega_matrix)

```

6.31.4 Community Detection

Generalized Louvain Algorithm

The generalized Louvain algorithm greedily maximizes multilayer modularity:

```

from py3plex.algorithms.community_detection.multilayer_modularity import louvain_
↪multilayer

```

(continues on next page)

(continued from previous page)

```

# Detect communities using Louvain
communities = louvain_multilayer(
    network,
    gamma=1.0,
    omega=1.0,
    max_iter=100,
    random_state=42
)

# Print results
for (node, layer), community_id in sorted(communities.items()):
    print(f"Node {node} in layer {layer}: Community {community_id}")

# Calculate resulting modularity
Q = multilayer_modularity(network, communities, gamma=1.0, omega=1.0)
print(f"Final modularity: {Q:.3f}")

```

Effect of Coupling Strength

The coupling parameter ω controls how much communities align across layers:

```

# Weak coupling (independent layers)
communities_weak = louvain_multilayer(network, gamma=1.0, omega=0.1)

# Strong coupling (same communities across layers)
communities_strong = louvain_multilayer(network, gamma=1.0, omega=10.0)

# Compare results
print("Weak coupling communities:", communities_weak)
print("Strong coupling communities:", communities_strong)

```

6.31.5 Supra-Modularity Matrix

For spectral methods, you can build the supra-modularity matrix:

```

from py3plex.algorithms.community_detection.multilayer_modularity import build_supra_
    modularity_matrix

# Build modularity matrix
B, node_layer_list = build_supra_modularity_matrix(
    network,
    gamma=1.0,
    omega=1.0
)

print(f"Supra-modularity matrix shape: {B.shape}")
print(f"Node-layer pairs: {node_layer_list}")

# Use for spectral analysis
import numpy as np
eigenvalues, eigenvectors = np.linalg.eigh(B)
print(f"Leading eigenvalues: {eigenvalues[-5:]}")

```

6.31.6 Synthetic Benchmark Networks

Multilayer LFR Benchmark

Generate multilayer networks with ground-truth communities:

```
from py3plex.algorithms.community_detection.multilayer_benchmark import generate_
    ↳ multilayer_lfr

# Generate with identical communities across layers
network, ground_truth = generate_multilayer_lfr(
    n=100,
    layers=['L1', 'L2', 'L3'],
    tau1=2.0,          # Degree distribution exponent
    tau2=1.5,          # Community size distribution exponent
    mu=0.1,            # Mixing parameter (10% external edges)
    avg_degree=10,
    min_community=20,
    community_persistence=1.0, # Same communities in all layers
    seed=42
)

# Check ground truth
print(f"Generated {len(set(ground_truth.values()))} communities")

# Detect communities
detected = louvain_multilayer(network, gamma=1.0, omega=1.0)

# Compare with ground truth
# (would need to implement community comparison metrics)
```

Evolving Communities

Generate networks where communities evolve across layers:

```
# Generate with evolving communities
network, ground_truth = generate_multilayer_lfr(
    n=100,
    layers=['T0', 'T1', 'T2', 'T3'], # Temporal layers
    mu=0.1,
    avg_degree=10,
    community_persistence=0.7, # 70% nodes keep community
    seed=42
)

# Analyze community persistence
for layer in ['T0', 'T1', 'T2', 'T3']:
    layer_nodes = [(n, l) for (n, l) in ground_truth.keys() if l == layer]
    print(f"{layer}: {len(layer_nodes)} nodes")
```

Overlapping Communities

Generate networks with nodes belonging to multiple communities:

```
# Generate with overlapping communities
network, ground_truth = generate_multilayer_lfr(
    n=100,
    layers=['L1', 'L2'],
    mu=0.1,
    overlapping_nodes=20,      # 20 nodes overlap
    overlapping_membership=2,  # Each belongs to 2 communities
    seed=42
)

# Check overlapping nodes
overlapping_count = sum(1 for coms in ground_truth.values() if len(coms) > 1)
print(f"Nodes with multiple communities: {overlapping_count}")
```

Partial Node Overlap

Generate networks where not all nodes appear in all layers:

```
# Generate with partial node presence
network, ground_truth = generate_multilayer_lfr(
    n=100,
    layers=['Social', 'Work', 'Family'],
    mu=0.1,
    node_overlap=0.7, # 70% nodes in all layers
    seed=42
)

# Check node presence per layer
for layer in ['Social', 'Work', 'Family']:
    layer_nodes = set(n for (n, l) in ground_truth.keys() if l == layer)
    print(f"{layer}: {len(layer_nodes)} nodes")
```

6.31.7 Coupled Erdős-Rényi Models

Generate random multilayer networks with controllable coupling:

```
from py3plex.algorithms.community_detection.multilayer_benchmark import generate_coupled_
er_multilayer

# Full multiplex (all nodes coupled)
network = generate_coupled_er_multilayer(
    n=100,
    layers=['L1', 'L2', 'L3'],
    p=0.1,      # Edge probability per layer
    omega=1.0,  # Coupling strength
    coupling_probability=1.0, # All nodes coupled
    seed=42
)

# Interdependent networks (partial coupling)
```

(continues on next page)

(continued from previous page)

```

network = generate_coupled_er_multilayer(
    n=100,
    layers=['L1', 'L2'],
    p=0.1,
    omega=0.5,
    coupling_probability=0.5, # Only 50% nodes coupled
    seed=42
)

```

Layer-Specific Edge Probabilities

```

# Different density per layer
network = generate_coupled_er_multilayer(
    n=100,
    layers=['Sparse', 'Dense'],
    p=[0.05, 0.3], # Different probabilities
    omega=1.0,
    seed=42
)

```

6.31.8 Stochastic Block Model

Generate multilayer networks with explicit block structure:

```

from py3plex.algorithms.community_detection.multilayer_benchmark import generate_sbm_
    ↪ multilayer

# Define communities
communities = [
    {0, 1, 2, 3, 4}, # Community 0
    {5, 6, 7, 8, 9}, # Community 1
    {10, 11, 12, 13, 14} # Community 2
]

# Generate with high intra-community, low inter-community edges
network, ground_truth = generate_sbm_multilayer(
    n=15,
    layers=['L1', 'L2'],
    communities=communities,
    p_in=0.7, # 70% intra-community edges
    p_out=0.05, # 5% inter-community edges
    community_persistence=0.8,
    seed=42
)

# Verify structure
print(f"Ground truth communities: {set(ground_truth.values())}")

```

6.31.9 Advanced Examples

Parameter Tuning

Finding optimal resolution and coupling parameters:

```
import numpy as np

# Test range of parameters
gammas = [0.5, 1.0, 1.5, 2.0]
omegas = [0.1, 0.5, 1.0, 2.0, 5.0]

results = []
for gamma in gammas:
    for omega in omegas:
        communities = louvain_multilayer(network, gamma=gamma, omega=omega)
        Q = multilayer_modularity(network, communities, gamma=gamma, omega=omega)
        n_communities = len(set(communities.values()))
        results.append((gamma, omega, Q, n_communities))
        print(f"={gamma}, =omega}: Q={Q:.3f}, n_com={n_communities}")

# Find best parameters
best = max(results, key=lambda x: x[2])
print(f"\nBest: =best[0]}, =best[1]}, Q={best[2]:.3f}")
```

Temporal Community Detection

Analyzing community evolution over time:

```
# Generate temporal network
network, ground_truth = generate_multilayer_lfr(
    n=50,
    layers=[f'T{i}' for i in range(10)], # 10 time steps
    mu=0.1,
    community_persistence=0.9, # Slowly evolving
    seed=42
)

# Detect communities
communities = louvain_multilayer(network, gamma=1.0, omega=1.0)

# Track community evolution
node = 0 # Track first node
timeline = []
for t in range(10):
    com = communities.get((node, f'T{t}'), -1)
    timeline.append(com)

print(f"Node {node} community timeline: {timeline}")
```

Comparing Detection Methods

```
# Compare different coupling strengths
weak_coupling = louvain_multilayer(network, gamma=1.0, omega=0.1)
strong_coupling = louvain_multilayer(network, gamma=1.0, omega=10.0)

Q_weak = multilayer_modularity(network, weak_coupling, gamma=1.0, omega=0.1)
Q_strong = multilayer_modularity(network, strong_coupling, gamma=1.0, omega=10.0)

print(f"Weak coupling: Q={Q_weak:.3f}, n_com={len(set(weak_coupling.values()))}")
print(f"Strong coupling: Q={Q_strong:.3f}, n_com={len(set(strong_coupling.values()))}")
```

6.31.10 Performance Considerations

Large Networks

For large networks, the Louvain algorithm can be slow. Consider:

```
# Limit iterations
communities = louvain_multilayer(
    network,
    gamma=1.0,
    omega=1.0,
    max_iter=50 # Reduce from default 100
)

# Use lower coupling for faster convergence
communities = louvain_multilayer(
    network,
    gamma=1.0,
    omega=0.5, # Lower coupling = faster
    max_iter=100
)
```

Memory Usage

The supra-adjacency matrix can be large. Use sparse representations when possible:

```
# The network internally uses sparse matrices
supra_matrix = network.get_supra_adjacency_matrix()
print(f"Supra-matrix is sparse: {type(supra_matrix)}")
```

6.31.11 References

1. Mucha et al., “Community Structure in Time-Dependent, Multiscale, and Multiplex Networks”, Science 328:876-878 (2010) * Original multilayer modularity formulation
2. Kivelä et al., “Multilayer networks”, J. Complex Netw. 2:203 (2014) * Comprehensive review of multilayer network analysis
3. De Domenico et al., “Mathematical Formulation of Multilayer Networks”, Phys. Rev. X 3, 041022 (2013) * Mathematical foundations and tensor representation
4. Granell et al., “Benchmark model to assess community structure in evolving networks”, Phys. Rev. E 92, 012805 (2015) * Temporal network benchmarks

5. Pamfil et al., “Relating modularity maximization and stochastic block models”, SIAM J. Data Science 1(4):667 (2019) * Theoretical grounding for parameter selection

6.31.12 See Also

- [Multilayer Centrality Tutorial](#)
- [Supra-Adjacency Matrices](#)
- [Community Detection in py3plex](#)

6.32 Community Detection Tutorial

This tutorial demonstrates **community detection algorithms** for **multilayer networks** using py3plex.

6.32.1 Overview

Community detection identifies **groups of nodes** that are more **densely connected** to each other than to the rest of the network. For **multilayer networks**, communities can **span multiple layers**, accounting for both **intra-layer** and **inter-layer** structure.

6.32.2 Supported Algorithms

py3plex provides **several community detection algorithms**:

- **Louvain** - **Fast modularity optimization** (recommended for most use cases)
- **Infomap** - **Flow-based** community detection (requires external binary)
- **Label Propagation** - **Semi-supervised** approach with known seed communities
- **Multilayer Modularity** - **True multilayer** community detection (Mucha et al. 2010)

6.32.3 Louvain Algorithm

Basic Usage

Fastest algorithm for large networks:

```
from py3plex.core import multinet
from py3plex.algorithms.community_detection import community_louvain

# Create or load network
network = multinet.multi_layer_network()
network.load_network("data.graphml", input_type="graphml")

# Detect communities using Louvain
communities = community_louvain.best_partition(network.core_network)

# Print results
for node, community_id in communities.items():
    print(f"Node {node} -> Community {community_id}")
```

Parameters

```
# With custom resolution parameter
communities = community_louvain.best_partition(
    network.core_network,
    resolution=1.0 # Higher = more communities, Lower = fewer communities
)
```

Resolution parameter:

- resolution=1.0 - **Standard modularity**
- resolution>1.0 - **More, smaller communities**
- resolution<1.0 - **Fewer, larger communities**

Advantages

- **Very fast:** $O(n \log n)$
- **Scales to millions** of nodes
- **BSD license** (commercial-friendly)
- **Well-established** and widely used

Disadvantages

- **Non-deterministic** (random initialization)
- **Cannot find overlapping** communities
- **Resolution limit** issues

6.32.4 Infomap Algorithm

Basic Usage

Flow-based approach for detecting communities:

```
from py3plex.algorithms.community_detection import community_wrapper

# Detect communities using Infomap
communities = community_wrapper.infomap_communities(
    network.core_network,
    binary_path="/path/to/infomap" # Path to Infomap binary
)
```

With Hierarchical Structure

```
# Get hierarchical community structure
hierarchical_communities = community_wrapper.infomap_communities(
    network.core_network,
    binary_path="/path/to/infomap",
    hierarchical=True
)
```


Advantages

- Can detect overlapping communities
- Flow-based (natural for many applications)
- Hierarchical structure
- Information-theoretic foundation

Disadvantages

- Requires external binary
- AGPLv3 license (viral copyleft - problematic for commercial use)
- Slower than Louvain

6.32.5 Label Propagation

Semi-Supervised Detection

Use when you have some known community memberships:

```
from py3plex.algorithms.community_detection import label_propagation

# Provide seed labels for some nodes
seed_labels = {
    'node1': 0,
    'node2': 0,
    'node3': 1,
    'node4': 1
}

# Propagate labels to unlabeled nodes
communities = label_propagation.propagate(
    network.core_network,
    seed_labels=seed_labels,
    max_iter=100
)
```

Fully Unsupervised

```
# Without seed labels (random initialization)
communities = label_propagation.propagate(
    network.core_network,
    max_iter=100
)
```

Advantages

- Very fast: $O(m)$ linear in edges
- Can incorporate prior knowledge
- MIT license
- Simple and interpretable

Disadvantages

- Non-deterministic
- Sensitive to initialization
- May not converge
- Lower quality than Louvain/Infomap

6.32.6 Multilayer Modularity

True Multilayer Detection

Accounts for multilayer structure following Mucha et al. (2010):

```
from py3plex.algorithms.community_detection import multilayer_modularity as mlm

# Get supra-adjacency matrix
supra_adj = network.get_supra_adjacency_matrix(sparse=True)

# Detect communities with multilayer modularity
communities = mlm.multilayer_louvain(
    supra_adj,
    gamma=1.0,      # Resolution parameter
    omega=1.0       # Inter-layer coupling strength
)
```

Parameter Tuning

```
# Emphasize layer-specific structure
communities = mlm.multilayer_louvain(
    supra_adj,
    gamma=1.0,
    omega=0.1 # Low coupling = layer-specific communities
)

# Emphasize cross-layer structure
communities = mlm.multilayer_louvain(
    supra_adj,
    gamma=1.0,
    omega=10.0 # High coupling = cross-layer communities
)
```

Mathematical Formulation

Multilayer modularity is defined as:

$$Q^{ML} = \frac{1}{2\mu} \sum_{ij\alpha\beta} \left[(A_{ij}^{[\alpha]} - \gamma_{\alpha} P_{ij}^{[\alpha]}) \delta_{\alpha\beta} + \omega_{\alpha\beta} \delta_{ij} \right] \delta(g_i^{[\alpha]}, g_j^{[\beta]})$$

Where:

- $A_{ij}^{[\alpha]}$ is the adjacency matrix of layer α
- $P_{ij}^{[\alpha]}$ is the null model (e.g., configuration model)
- γ_{α} is the resolution parameter for layer α

- $\omega_{\alpha\beta}$ is the coupling strength between layers
- $\delta(g_i^{[\alpha]}, g_j^{[\beta]})$ is 1 if nodes are in the same community, 0 otherwise

Advantages

- Accounts for multilayer structure
- Implements state-of-the-art algorithm
- Configurable inter-layer coupling
- Published in Science (Mucha et al. 2010)

Disadvantages

- More computationally expensive
- Requires parameter tuning
- May not scale to very large networks (>100k nodes)

6.32.7 Evaluating Community Quality

Modularity Score

```
import networkx as nx

# Compute modularity
modularity = nx.community.modularity(network.core_network, communities)
print(f"Modularity: {modularity:.3f}")
```

Interpretation:

- $Q > 0.3$: Good community structure
- $Q > 0.5$: Strong community structure
- $Q < 0.3$: Weak or no community structure

Coverage and Performance

```
# Coverage: fraction of edges within communities
coverage = nx.community.coverage(network.core_network, communities)

# Performance: fraction of correctly classified node pairs
performance = nx.community.performance(network.core_network, communities)

print(f"Coverage: {coverage:.3f}")
print(f"Performance: {performance:.3f}")
```

6.32.8 Visualizing Communities

Color by Community

```
from py3plex.visualization.multilayer import hairball_plot
import matplotlib.pyplot as plt
```

(continues on next page)

(continued from previous page)

```
# Map communities to colors
node_colors = [communities.get(node, 0) for node in network.core_network.nodes()]

# Visualize with community colors
hairball_plot(
    network.core_network,
    node_color=node_colors,
    layout_algorithm='force',
    cmap='tab10'
)
plt.show()
```

Community Size Distribution

```
from collections import Counter
import matplotlib.pyplot as plt

# Count community sizes
community_sizes = Counter(communities.values())
sizes = list(community_sizes.values())

# Plot distribution
plt.hist(sizes, bins=20)
plt.xlabel('Community Size')
plt.ylabel('Frequency')
plt.title('Community Size Distribution')
plt.show()
```

6.32.9 Comparing Algorithms

Run Multiple Algorithms

```
# Run different algorithms
louvain_comms = community_louvain.best_partition(network.core_network)
label_prop_comms = label_propagation.propagate(network.core_network)

# Compare number of communities
print(f"Louvain: {len(set(louvain_comms.values()))} communities")
print(f"Label Prop: {len(set(label_prop_comms.values()))} communities")

# Compare modularity
louvain_mod = nx.community.modularity(network.core_network,
                                     [set(n for n, c in louvain_comms.items() if c == i)
                                      for i in set(louvain_comms.values())])
label_mod = nx.community.modularity(network.core_network,
                                    [set(n for n, c in label_prop_comms.items() if c ==
↪ i)
                                     for i in set(label_prop_comms.values())])

print(f"Louvain modularity: {louvain_mod:.3f}")
print(f"Label Prop modularity: {label_mod:.3f}")
```

Normalized Mutual Information

Compare similarity between community structures:

```
from sklearn.metrics import normalized_mutual_info_score

# Convert to lists
louvain_list = [louvain_comms[node] for node in network.core_network.nodes()]
label_list = [label_prop_comms[node] for node in network.core_network.nodes()]

# Compute NMI
nmi = normalized_mutual_info_score(louvain_list, label_list)
print(f"NMI between Louvain and Label Prop: {nmi:.3f}")
```

Interpretation:

- NMI = 1.0: Identical community structures
- NMI = 0.0: Completely different structures
- NMI > 0.5: Similar structures

6.32.10 Best Practices

Algorithm Selection

Network Size	Speed Priority	Quality Priority	Recommendation
Small (<1K)	Any	Any	Try all algorithms
Medium (1K-10K)	Louvain	Louvain/Infomap	Louvain (good balance)
Large (10K-100K)	Louvain/Label Prop	Louvain	Louvain
Very Large (>100K)	Label Prop	Louvain	Label Prop or sample

Parameter Guidelines

Louvain resolution:

- Start with resolution=1.0
- Increase if communities are too large
- Decrease if communities are too fragmented

Multilayer coupling (omega):

- omega=1.0 - Default, balanced
- omega<1.0 - Emphasize layer-specific structure
- omega>1.0 - Emphasize cross-layer structure

Validation

Always validate community detection results:

1. **Visual inspection** - Plot and examine communities
2. **Modularity** - Check modularity score (>0.3 is good)
3. **Size distribution** - Check for giant communities or singletons
4. **Domain knowledge** - Do communities make sense for your application?

6.32.11 References

Louvain:

Blondel, V. D., et al. (2008). Fast unfolding of communities in large networks. *Journal of Statistical Mechanics*.

Infomap:

Rosvall, M., & Bergstrom, C. T. (2008). Maps of random walks on complex networks reveal community structure. *PNAS*, 105(4), 1118-1123.

Multilayer Modularity:

Mucha, P. J., et al. (2010). Community structure in time-dependent, multiscale, and multiplex networks. *Science*, 328(5980), 876-878.

See *Citation and References* for complete citations with DOIs.

6.32.12 Next Steps

- *Multilayer Network Centrality Measures* - Centrality measures tutorial
- *Multilayer Modularity and Community Detection* - Multilayer modularity details
- *Algorithm Selection Guide* - Algorithm selection guide
- Examples in `examples/communities/example_community_detection.py`

6.33 Network Decomposition Tutorial

This tutorial demonstrates network decomposition techniques for heterogeneous and multilayer networks using py3plex.

6.33.1 Overview

Network decomposition transforms complex heterogeneous networks into structured feature representations that can be used for:

- Node classification
- Link prediction
- Network comparison
- Feature extraction for machine learning

py3plex's HINMINE module provides methods for decomposing heterogeneous information networks (HINs) based on:

- **Meta-paths** - Typed paths connecting node pairs
- **Cycles** - Closed walks of specific types
- **Structural patterns** - Subgraph patterns and motifs

6.33.2 What is Network Decomposition?

Heterogeneous Information Networks

A **heterogeneous information network (HIN)** contains multiple node types and edge types.

Example - Academic Network:

- **Node types:** Authors, Papers, Venues

- **Edge types:** Author-Paper (writes), Paper-Venue (published_in), Paper-Paper (cites)

Meta-paths

A **meta-path** is a sequence of node and edge types defining a composite relation.

Examples:

- Author → Paper → Author (co-authorship)
- Author → Paper → Venue → Paper → Author (same venue)
- Paper → Paper → Paper (citation path of length 2)

6.33.3 HINMINE Decomposition

Basic Usage

```
from py3plex.core.HINMINE import decomposition
from py3plex.core import multinet

# Load heterogeneous network
network = multinet.multi_layer_network()
network.load_network("academic_network.graphml", input_type="graphml")

# Perform decomposition
decomposer = decomposition.HINMINE()
features = decomposer.run_decomposition(
    network,
    target_node_type='author',
    max_path_length=3
)

# Features is a matrix: (num_nodes, num_meta_paths)
print(f"Feature matrix shape: {features.shape}")
```

Parameters

```
features = decomposer.run_decomposition(
    network,
    target_node_type='author',      # Node type to extract features for
    max_path_length=3,             # Maximum meta-path length
    include_cycles=True,           # Include closed walks
    normalize=True                  # Normalize features
)
```

6.33.4 Meta-Path Extraction

Enumerate Meta-Paths

```
# Get all meta-paths up to length 3
meta_paths = decomposer.enumerate_meta_paths(
    network,
    max_length=3,
    node_types=['author', 'paper', 'venue']
```

(continues on next page)

(continued from previous page)

```
)  
  
print(f"Found {len(meta_paths)} meta-paths")  
for mp in meta_paths[:5]:  
    print(mp)
```

Example output:

```
author-paper-author  
author-paper-venue-paper-author  
paper-paper  
paper-author-paper  
venue-paper-venue
```

Compute Meta-Path Instances

Count instances of specific meta-paths:

```
# Define a meta-path  
meta_path = ['author', 'paper', 'author']  
  
# Count instances for each node  
instances = decomposer.compute_meta_path_instances(  
    network,  
    meta_path,  
    source_nodes=['author1', 'author2']  
)  
  
for node, count in instances.items():  
    print(f"{node}: {count} instances")
```

6.33.5 Cycle Enumeration

Extract Cycles

Find closed walks (cycles) in the network:

```
# Enumerate all cycles up to length 4  
cycles = decomposer.enumerate_cycles(  
    network,  
    max_length=4,  
    starting_node_type='author'  
)  
  
print(f"Found {len(cycles)} cycle types")
```

Common Cycles

Examples of meaningful cycles in academic networks:

- **Triangle:** Author → Paper → Venue → Paper → Author → Paper
- **Square:** Paper → Cites → Paper → Cites → Paper → Cites → Paper
- **Collaboration cycle:** Author → Paper → Author → Paper → Author

6.33.6 Feature Matrix Construction

Build Classification Features

```
# Build feature matrix for author classification
features = decomposer.build_feature_matrix(
    network,
    target_nodes=['author1', 'author2', 'author3'],
    meta_paths=[
        ['author', 'paper', 'author'],      # Co-authorship
        ['author', 'paper', 'venue'],       # Publishing venues
        ['author', 'paper', 'paper'],       # Citation patterns
    ],
    normalize=True
)

# Features shape: (3, 3) for 3 nodes, 3 meta-paths
print(features)
```

Normalization Options

```
# L1 normalization (sum to 1)
features_l1 = decomposer.normalize_features(features, method='l1')

# L2 normalization (unit length)
features_l2 = decomposer.normalize_features(features, method='l2')

# Min-max scaling (0-1 range)
features_minmax = decomposer.normalize_features(features, method='minmax')

# Z-score standardization (mean 0, std 1)
features_zscore = decomposer.normalize_features(features, method='zscore')
```

6.33.7 Node Classification Example

Complete Workflow

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, f1_score

# 1. Load network with node labels
network = multinet.multi_layer_network()
network.load_network("labeled_network.graphml", input_type="graphml")

# 2. Extract features via decomposition
decomposer = decomposition.HINMINE()
features = decomposer.run_decomposition(
    network,
    target_node_type='author',
    max_path_length=3
)
```

(continues on next page)

(continued from previous page)

```

# 3. Get labels
labels = network.labels # Assuming labels are stored in network

# 4. Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    features, labels, test_size=0.3, random_state=42
)

# 5. Train classifier
clf = RandomForestClassifier(n_estimators=100, random_state=42)
clf.fit(X_train, y_train)

# 6. Evaluate
y_pred = clf.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred, average='weighted')

print(f"Accuracy: {accuracy:.3f}")
print(f"F1-score: {f1:.3f}")

```

Feature Importance

Analyze which meta-paths are most informative:

```

import matplotlib.pyplot as plt

# Get feature importances from trained model
importances = clf.feature_importances_
meta_path_names = decomposer.get_meta_path_names()

# Plot
plt.figure(figsize=(10, 6))
plt.barh(range(len(importances)), importances)
plt.yticks(range(len(importances)), meta_path_names)
plt.xlabel('Feature Importance')
plt.title('Meta-Path Importance for Classification')
plt.tight_layout()
plt.show()

```

6.33.8 Link Prediction Example

Predict Missing Edges

```

from sklearn.linear_model import LogisticRegression

# 1. Sample node pairs (positive and negative examples)
positive_pairs = [] # Actual edges
negative_pairs = [] # Non-edges (sampled)

for u, v in network.core_network.edges():
    positive_pairs.append((u, v))

```

(continues on next page)

(continued from previous page)

```

# Sample negative examples
import random
nodes = list(network.get_nodes())
while len(negative_pairs) < len(positive_pairs):
    u, v = random.sample(nodes, 2)
    if not network.core_network.has_edge(u, v):
        negative_pairs.append((u, v))

# 2. Extract features for node pairs
def pair_features(u, v, features, node_to_idx):
    idx_u = node_to_idx[u]
    idx_v = node_to_idx[v]
    # Concatenate, multiply, or subtract features
    return features[idx_u] * features[idx_v]

# Build dataset
X_pos = [pair_features(u, v, features, node_to_idx) for u, v in positive_pairs]
X_neg = [pair_features(u, v, features, node_to_idx) for u, v in negative_pairs]

X = X_pos + X_neg
y = [1] * len(X_pos) + [0] * len(X_neg)

# 3. Train-test split and classify
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3)

clf = LogisticRegression()
clf.fit(X_train, y_train)

# 4. Evaluate
accuracy = clf.score(X_test, y_test)
print(f"Link prediction accuracy: {accuracy:.3f}")

```

6.33.9 Advanced Decomposition

Custom Meta-Paths

Define domain-specific meta-paths:

```

# Academic domain
academic_meta_paths = [
    ['author', 'paper', 'author'],          # Co-authorship
    ['author', 'paper', 'venue', 'paper', 'author'], # Same venue
    ['author', 'paper', 'paper', 'author'],    # Citation-based
    ['venue', 'paper', 'author', 'paper', 'venue'], # Venue similarity
]

features = decomposer.build_feature_matrix(
    network,
    target_nodes=author_nodes,
    meta_paths=academic_meta_paths
)

```

Weighted Meta-Paths

Assign importance weights to different meta-paths:

```
# Define meta-paths with weights
meta_path_weights = {
    'author-paper-author': 2.0,          # High importance
    'author-paper-venue': 1.0,          # Normal importance
    'author-paper-paper-author': 0.5     # Lower importance
}

# Apply weights to features
weighted_features = decomposer.apply_weights(features, meta_path_weights)
```

Temporal Decomposition

For temporal networks with timestamps:

```
# Decompose network at different time slices
time_slices = [2018, 2019, 2020, 2021]

temporal_features = []
for year in time_slices:
    # Filter network to specific time period
    subnetwork = network.filter_by_time(year)

    # Decompose
    features = decomposer.run_decomposition(subnetwork, 'author')
    temporal_features.append(features)

# Stack temporal features
import numpy as np
full_features = np.hstack(temporal_features)
```

6.33.10 Performance Optimization

Caching Results

HINMINE automatically caches decomposition results:

```
# First run: computes and caches
features1 = decomposer.run_decomposition(network, 'author')

# Second run: loads from cache (much faster)
features2 = decomposer.run_decomposition(network, 'author')
```

Cache is stored in `.{md5_hash}` files based on network content.

Parallel Processing

For large networks, process meta-paths in parallel:

```
from joblib import Parallel, delayed

# Compute meta-path instances in parallel
```

(continues on next page)

(continued from previous page)

```

results = Parallel(n_jobs=-1)(
    delayed(decomposer.compute_meta_path_instances)(
        network, mp, target_nodes
    )
    for mp in meta_paths
)

```

Sparse Representations

Use sparse matrices for large feature spaces:

```

from scipy.sparse import csr_matrix

# Convert to sparse format
sparse_features = csr_matrix(features)

# Use with scikit-learn
from sklearn.svm import LinearSVC
clf = LinearSVC()
clf.fit(sparse_features, labels)

```

6.33.11 Best Practices

Meta-Path Selection

1. **Start simple** - Begin with short meta-paths (length 2-3)
2. **Domain knowledge** - Use meaningful paths for your domain
3. **Feature importance** - Analyze which paths are most informative
4. **Avoid redundancy** - Remove highly correlated meta-paths

Normalization

- **L1 normalization** - Good for interpretability (sum to 1)
- **L2 normalization** - Good for distance-based methods
- **Min-max scaling** - Good for neural networks
- **Z-score** - Good for algorithms sensitive to scale

Validation

Always validate decomposition quality:

1. **Feature coverage** - Check that features capture relevant patterns
2. **Downstream performance** - Evaluate on actual task (classification, prediction)
3. **Interpretability** - Verify features are meaningful
4. **Computational cost** - Balance accuracy vs. efficiency

6.33.12 References

HINMINE:

Kralj, J., Robnik-Šikonja, M., & Lavrač, N. (2018). HINMINE: heterogeneous information network mining with information retrieval heuristics. *Journal of Intelligent Information Systems*, 50(1), 29-61.

Meta-paths:

Sun, Y., Han, J., Yan, X., Yu, P. S., & Wu, T. (2011). PathSim: Meta path-based top-k similarity search in heterogeneous information networks. *VLDB*, 4(11), 992-1003.

HIN Classification:

Shi, C., Li, Y., Zhang, J., Sun, Y., & Philip, S. Y. (2016). A survey of heterogeneous information network analysis. *IEEE TKDE*, 29(1), 17-37.

6.33.13 Next Steps

- *Community Detection Tutorial* - Community detection tutorial
- *Multilayer Network Centrality Measures* - Centrality measures
- *Algorithm Selection Guide* - Algorithm selection
- Examples in `examples/advanced/example_decomposition_and_classification.py`

6.34 Incidence Gadget Encoding Tutorial

This tutorial demonstrates how to transform multiplex networks into homogeneous hypergraphs using incidence gadget encoding with prime-based layer signatures in py3plex.

6.34.1 Overview

Incidence gadget encoding provides a lossless transformation of multiplex networks into standard graph representations. This is useful for:

- Converting multiplex networks to formats compatible with classical graph algorithms
- Graph isomorphism testing with layer-aware structure preservation
- Network compression and serialization
- Cross-tool interoperability (exporting to tools without native multiplex support)

The encoding uses **prime numbers** to uniquely identify layers through cycle structures attached to edge-nodes.

6.34.2 What is Incidence Gadget Encoding?

Multiplex Networks

A **multiplex network** contains nodes that appear in multiple layers with different types of connections.

Example - Social Network:

- **Layers:** friendship, colleague, family
- **Nodes:** People appearing in multiple layers
- **Edges:** Different relationships between same people in different layers

Incidence Gadget Transformation

The transformation creates:

- **Vertex-nodes** (v_*): One for each unique node in the multiplex
- **Edge-nodes** (e_*): One for each edge, connected to its endpoint vertex-nodes
- **Signature cycles** (C_p): Prime-length cycles attached to edge-nodes to encode layers

Layer Encoding with Primes

Each layer is assigned a unique prime number:

- Layer 1 $\rightarrow$ prime 2 $\rightarrow$ cycle of length 2
- Layer 2 $\rightarrow$ prime 3 $\rightarrow$ cycle of length 3
- Layer 3 $\rightarrow$ prime 5 $\rightarrow$ cycle of length 5
- Layer 4 $\rightarrow$ prime 7 $\rightarrow$ cycle of length 7
- ...

The prime cycle length uniquely identifies which layer each edge belongs to.

6.34.3 Basic Usage

Encoding a Multiplex Network

```
from py3plex.core import multinet

# Create a multiplex network
network = multinet.multi_layer_network(directed=False)

# Add nodes to different layers
network.add_nodes([
    {'source': 'Alice', 'type': 'friendship'},
    {'source': 'Bob', 'type': 'friendship'},
    {'source': 'Alice', 'type': 'colleague'},
    {'source': 'Bob', 'type': 'colleague'}
], input_type='dict')

# Add edges in each layer
network.add_edges([
    {'source': 'Alice', 'target': 'Bob', 'source_type': 'friendship', 'target_type':
    ↪ 'friendship'},
    {'source': 'Alice', 'target': 'Bob', 'source_type': 'colleague', 'target_type':
    ↪ 'colleague'}
], input_type='dict')

# Encode to homogeneous hypergraph
H, node_mapping, edge_info = network.to_homogeneous_hypergraph()

print(f"Encoded graph has {len(H.nodes())} nodes and {len(H.edges())} edges")
print(f"Node mapping: {node_mapping}")
print(f"Edge information: {edge_info}")
```

Decoding Back to Multiplex

```
# Decode the homogeneous hypergraph back to multiplex structure
recovered = network.from_homogeneous_hypergraph(H)

# recovered is a dict: {layer_name: [(u, v), ...]}
for layer, edges in recovered.items():
    print(f"Layer {layer}: {edges}")
```

6.34.4 Understanding the Output

Node Mapping

The `node_mapping` dictionary maps original node IDs to vertex-nodes in H:

```
node_mapping = {
    'Alice': 'v_Alice',
    'Bob': 'v_Bob',
    'Charlie': 'v_Charlie'
}
```

Edge Information

The `edge_info` dictionary maps edge-nodes to their layer and endpoints:

```
edge_info = {
    'e_0': ('friendship', ('Alice', 'Bob')),
    'e_1': ('colleague', ('Alice', 'Bob')),
    'e_2': ('family', ('Alice', 'Charlie'))
}
```

Homogeneous Graph Structure

The encoded graph H contains three types of nodes:

- **Vertex-nodes** (`v_*`): Original nodes from the multiplex
- **Edge-nodes** (`e_*`): Represent edges in the multiplex
- **Signature nodes** (`_s`): Form prime-length cycles for layer encoding

6.34.5 Complete Example

Social Network Encoding

```
from py3plex.core import multinet
import networkx as nx

# Create a multiplex social network
network = multinet.multi_layer_network(directed=False)

# Define people and relationships
people = ['Alice', 'Bob', 'Charlie', 'Diana']

# Friendship layer
```

(continues on next page)

(continued from previous page)

```

network.add_nodes([
    {'source': p, 'type': 'friendship'} for p in people
], input_type='dict')
network.add_edges([
    {'source': 'Alice', 'target': 'Bob', 'source_type': 'friendship', 'target_type':
    ↪ 'friendship'},
    {'source': 'Bob', 'target': 'Charlie', 'source_type': 'friendship', 'target_type':
    ↪ 'friendship'}
], input_type='dict')

# Colleague layer
network.add_nodes([
    {'source': p, 'type': 'colleague'} for p in ['Alice', 'Bob', 'Diana']
], input_type='dict')
network.add_edges([
    {'source': 'Alice', 'target': 'Diana', 'source_type': 'colleague', 'target_type':
    ↪ 'colleague'}
], input_type='dict')

# Encode to homogeneous hypergraph
H, node_mapping, edge_info = network.to_homogeneous_hypergraph()

# Analyze the encoded graph
print(f"Original multiplex: {len(list(network.get_nodes()))} nodes, {len(list(network.
    ↪ get_edges()))} edges")
print(f"Encoded graph H: {len(H.nodes())} nodes, {len(H.edges())} edges")
print(f"Connected components: {nx.number_connected_components(H)}")

# Decode back
recovered = network.from_homogeneous_hypergraph(H)
print(f"\nRecovered {len(recovered)} layers")
for layer, edges in recovered.items():
    print(f" {layer}: {len(edges)} edges")

```

Analyzing Cycle Structure

```

from sympy import primerange

# The first few primes used for encoding
primes = list(primerange(2, 20))
print(f"Layer encoding primes: {primes[:5]}")
# Output: [2, 3, 5, 7, 11]

# Find cycles in the encoded graph
all_cycles = nx.cycle_basis(H)
print(f"Number of cycles: {len(all_cycles)}")
print(f"Cycle lengths: {sorted([len(c) for c in all_cycles])}")

# Each edge-node has a cycle through it of prime length
for edge_node, (layer, endpoints) in edge_info.items():
    cycles_with_edge = [c for c in all_cycles if edge_node in c]

```

(continues on next page)

(continued from previous page)

```

if cycles_with_edge:
    cycle_len = len(cycles_with_edge[0])
    print(f"{edge_node} (layer {layer}): cycle length = {cycle_len}")

```

6.34.6 Advanced Usage

Working with Large Networks

For networks with many layers and edges:

```

# Create a larger multiplex network
network = multinet.multi_layer_network(directed=False)

# Multiple layers
for layer in ['Layer1', 'Layer2', 'Layer3']:
    # Add nodes
    network.add_nodes([
        {'source': str(i), 'type': layer} for i in range(10)
    ], input_type='dict')

    # Create ring topology
    for i in range(10):
        network.add_edges([
            {'source': str(i), 'target': str((i+1) % 10),
             'source_type': layer, 'target_type': layer}
        ], input_type='dict')

# Encode
H, node_mapping, edge_info = network.to_homogeneous_hypergraph()

# Analyze node types in encoded graph
vertex_nodes = [n for n in H.nodes() if str(n).startswith('v_')]
edge_nodes = [n for n in H.nodes() if str(n).startswith('e_')]
signature_nodes = [n for n in H.nodes()
                    if not str(n).startswith('v_') and not str(n).startswith('e_')]

print(f"Vertex-nodes: {len(vertex_nodes)}")
print(f"Edge-nodes: {len(edge_nodes)}")
print(f"Signature-nodes: {len(signature_nodes)}")

```

Exporting Encoded Graphs

The encoded graph H is a standard NetworkX Graph and can be exported:

```

import networkx as nx

# Export to various formats
nx.write_graphml(H, "encoded_network.graphml")
nx.write_gexf(H, "encoded_network.gexf")
nx.write_edgelist(H, "encoded_network.edgelist")

# Save node mapping and edge info separately

```

(continues on next page)

(continued from previous page)

```
import json

with open("node_mapping.json", "w") as f:
    json.dump(node_mapping, f)

with open("edge_info.json", "w") as f:
    # Convert tuples to lists for JSON serialization
    edge_info_serializable = {
        k: [v[0], list(v[1])] for k, v in edge_info.items()
    }
    json.dump(edge_info_serializable, f)
```

6.34.7 Technical Details

Complexity

- **Encoding time:** $O(E \times p_{\max})$ where E is number of edges, $p_{\max}$ is largest prime
- **Decoding time:** $O(V + E + C)$ where V is nodes, E is edges, C is cycle detection cost
- **Space:** $O(E \times p_{\max})$ due to signature cycles

Limitations

- Maximum 305 layers (number of primes less than 2000)
- Cycle detection cost increases with number of layers
- Output graph size grows linearly with edge count and layer diversity
- Edge attributes are not preserved (only topological structure)

Mathematical Foundation

The encoding is based on:

- **Incidence representation:** Edges become nodes (edge-nodes)
- **Prime signatures:** Unique prime p for each layer
- **Cycle gadgets:** Cycle C_p of length p attached to each edge in layer
- **Isomorphism preservation:** Graph structure uniquely identifies layer membership

References

- Based on incidence gadget constructions from graph theory
- Prime-based encoding ensures unique layer identification through cycle length
- Related to graph product operations and hypergraph representations

6.34.8 See Also

- *Network Decomposition Tutorial* - Alternative network transformation techniques
- *Multilayer Network Centrality Measures* - Centrality measures for multiplex networks
- *Community Detection Tutorial* - Community detection in multilayer networks

6.34.9 Example Scripts

See `examples/advanced/example_incidence_gadget_encoding.py` for complete demonstrations:

- Basic encoding and decoding workflow
- Social network multiplex example
- Cycle structure analysis
- Network properties comparison

All examples are runnable and include detailed output explanations.

6.35 Config-Driven Workflows

Table of Contents

- *Overview*
- *Configuration Format*
 - *YAML Example*
 - *JSON Example*
- *Dataset Specification*
 - *Loading from File*
 - *Generating Networks*
- *Available Operations*
 - *Statistics*
 - *Community Detection*
 - *Centrality*
 - *Visualization*
 - *Aggregation*
 - *Conversion*
- *Running Workflows*
 - *Command Line*
 - *Python API*
- *Examples*
- *Best Practices*
 - *Version Control*
 - *Reproducibility*
 - *Organization*
- *Troubleshooting*
 - *YAML Not Available*

– Validation Errors

- Further Reading

6.35.1 Overview

Config-driven workflows in py3plex allow you to define and execute complex network analysis pipelines using YAML or JSON configuration files. This approach is ideal for:

- **Reproducible research** - Share exact experiment configurations
- **Pipeline automation** - Integrate with CI/CD systems
- **Batch processing** - Run multiple experiments with different parameters
- **Version control** - Track experimental setups alongside code

6.35.2 Configuration Format

A workflow configuration file consists of three main sections:

1. **Metadata:** Workflow name and description
2. **Datasets:** Networks to load or generate
3. **Operations:** Analysis steps to perform
4. **Output:** Where to save results

YAML Example

```
name: "Basic Network Analysis"
description: "Analyze a multilayer network"

datasets:
- name: "my_network"
  type: "generate"
  generator: "random"
  parameters:
    nodes: 100
    layers: 3
    probability: 0.1
    seed: 42

operations:
- type: "stats"
  dataset: "my_network"
  parameters: {}

- type: "community"
  dataset: "my_network"
  parameters:
    algorithm: "louvain"

- type: "visualize"
  dataset: "my_network"
```

(continues on next page)

(continued from previous page)

```
parameters:
  output: "network.png"
  layout: "spring"

output:
  directory: "results"
  summary: "summary.json"
```

JSON Example

```
{
  "name": "Network Comparison",
  "description": "Compare two networks",
  "datasets": [
    {
      "name": "network1",
      "type": "file",
      "path": "network1.graphml"
    },
    {
      "name": "network2",
      "type": "file",
      "path": "network2.graphml"
    }
  ],
  "operations": [
    {
      "type": "stats",
      "dataset": "network1",
      "parameters": {}
    },
    {
      "type": "stats",
      "dataset": "network2",
      "parameters": {}
    }
  ],
  "output": {
    "directory": "comparison",
    "summary": "results.json"
  }
}
```

6.35.3 Dataset Specification

Loading from File

Load existing network files:

```
datasets:
- name: "my_network"
```

(continues on next page)

(continued from previous page)

```

type: "file"
path: "path/to/network.graphml"

```

Supported file formats:

- GraphML (.graphml)
- GPickle (.gpickle)
- Multiedgelist (.edgelist, .txt)

Generating Networks

Generate synthetic networks:

```

datasets:
- name: "random_network"
  type: "generate"
  generator: "random"
  parameters:
    nodes: 50          # Number of nodes
    layers: 2          # Number of layers
    probability: 0.15  # Edge probability
    seed: 42           # Random seed (optional)

```

6.35.4 Available Operations

Statistics

Compute network statistics:

```

- type: "stats"
  dataset: "my_network"
  parameters: {}

```

Computes:

- Node and edge counts
- Layer densities
- Clustering coefficients

Community Detection

Detect communities:

```

- type: "community"
  dataset: "my_network"
  parameters:
    algorithm: "louvain" # Community detection algorithm

```

Supported algorithms:

- louvain - Louvain method

Centrality

Compute node centrality:

```
- type: "centrality"  
  dataset: "my_network"  
  parameters:  
    measure: "degree" # Centrality measure
```

Supported measures:

- degree - Degree centrality
- betweenness - Betweenness centrality
- closeness - Closeness centrality

Visualization

Visualize networks:

```
- type: "visualize"  
  dataset: "my_network"  
  parameters:  
    output: "network.png" # Output file  
    layout: "spring" # Layout algorithm
```

Supported layouts:

- spring - Force-directed spring layout
- circular - Circular layout

Aggregation

Aggregate multilayer networks:

```
- type: "aggregate"  
  dataset: "my_network"  
  parameters:  
    method: "sum" # Aggregation method
```

Supported methods:

- sum - Sum edge weights
- mean - Average edge weights
- max - Maximum edge weight
- min - Minimum edge weight

Conversion

Convert network formats:

```
- type: "convert"  
  dataset: "my_network"  
  parameters:  
    output: "network.json" # Output file
```


6.35.5 Running Workflows

Command Line

Run a workflow from the command line:

```
# Run workflow
py3plex run-config my_workflow.yaml

# Validate configuration without running
py3plex run-config my_workflow.yaml --validate-only
```

Python API

Run workflows programmatically:

```
from py3plex.workflows import run_workflow

# Run workflow from config file
run_workflow("my_workflow.yaml")

# Or use the full API
from py3plex.workflows import WorkflowConfig, WorkflowRunner

config = WorkflowConfig.from_file("my_workflow.yaml")

# Validate
errors = config.validate()
if errors:
    print("Validation errors:", errors)
else:
    # Execute
    runner = WorkflowRunner(config)
    runner.run()
```

6.35.6 Examples

See the `examples/config_driven/` directory for complete examples:

- `example_config.yaml` - Basic workflow with generated network
- `comparison_config.json` - Multi-dataset comparison
- `example_config_workflow.py` - Python example script

To run the examples:

```
cd examples/config_driven
python example_config_workflow.py

# Or using CLI
py3plex run-config example_config.yaml
py3plex run-config comparison_config.json
```

6.35.7 Best Practices

Version Control

- Store config files in version control alongside your code
- Use descriptive names for workflows
- Document parameters in comments

Reproducibility

- Always set random seeds for generated networks
- Document dataset sources and versions
- Include workflow descriptions

Organization

- Use separate configs for different experiments
- Group related operations together
- Create reusable config templates

6.35.8 Troubleshooting

YAML Not Available

If you get an error about YAML support:

```
pip install pyyaml
```

Then use YAML config files. JSON is always supported without extra dependencies.

Validation Errors

Use `--validate-only` to check configuration:

```
py3plex run-config my_workflow.yaml --validate-only
```

Common issues:

- Missing required fields (name, type, etc.)
- Invalid operation types
- Referenced datasets that don't exist
- Invalid file paths

6.35.9 Further Reading

- *CLI Guide* - Command-line interface
- *API Documentation* - Python API reference
- *Examples* - More examples

6.36 Supra Matrix Function Centralities

These centrality measures operate directly on the supra-adjacency matrix of a multilayer network using matrix functions. They provide global measures of node-layer pair importance based on walks, paths, and communication patterns across the entire network structure.

6.36.1 Overview

Matrix function centralities extend classical centrality concepts to multilayer networks by:

- Operating on the full supra-adjacency matrix that captures both intra-layer and inter-layer connections
- Using matrix functions (exponential, matrix inversion) to account for all paths simultaneously
- Providing normalized scores that can be compared across different network structures
- Supporting both sparse and dense computation for efficiency

These measures are particularly useful for:

- Identifying influential node-layer pairs in multiplex networks
- Analyzing information flow across network layers
- Comparing node importance across different network structures
- Understanding long-range dependencies in multilayer systems

6.36.2 Available Methods

Communicability Centrality

Communicability centrality measures the weighted sum of all walks between nodes, with exponentially decaying weights for longer walks. It is based on the matrix exponential of the supra-adjacency matrix.

Mathematical Definition:

$$c_i = \sum_j (e^{A_{\text{supra}}})_{ij}$$

where A_{supra} is the supra-adjacency matrix.

Best for:

- Measuring node importance based on all possible paths
- Accounting for redundancy in communication channels
- Networks where multiple paths contribute to influence
- Global network analysis

Complexity: $O(n^2)$ for sparse matrices using Krylov subspace methods, where n is the total number of node-layer pairs

Reference: Estrada, E., & Hatano, N. (2008). Communicability in complex networks. *Physical Review E*, 77(3), 036111.

`py3plex.algorithms.multilayer_algorithms.supra_matrix_function_centrality.communicability_centrality(supra_matrix, normalize, use_sparse, max_iter, tol, float_precision)`

Compute communicability centrality for each node-layer pair.

Communicability centrality measures the weighted sum of all walks between nodes, with exponentially decaying weights for longer walks. It is computed as the row sum of the matrix exponential:

$$c_i = \sum_j (\exp(A))_{ij}$$

This implementation uses `scipy.sparse.linalg.expm_multiply` for efficient sparse matrix exponential computation.

Parameters

- **supra_matrix** – Sparse supra-adjacency matrix (n x n).
- **normalize** – If True, normalize output to sum to 1.
- **use_sparse** – If True, use sparse matrix operations. Falls back to dense for matrices smaller than 10000 elements.
- **max_iter** – Maximum number of iterations for sparse approximation (currently unused).
- **tol** – Tolerance for convergence (currently unused).

Returns

Communicability centrality scores for each node-layer pair (n,).

Return type

`np.ndarray`

Raises

Py3plexMatrixError – If matrix is invalid (non-square, empty, etc.).

Example

```
>>> from py3plex.core import random_generators
>>> net = random_generators.random_multiplex_ER(50, 3, 0.1)
>>> A = net.get_supra_adjacency_matrix()
>>> comm = communicability_centrality(A)
>>> print(f"Communicability centrality computed for {len(comm)} node-layer pairs")
```

Katz Centrality

Katz centrality measures node influence by accounting for all paths with exponentially decaying weights, controlled by an attenuation parameter α . It includes an exogenous influence term to ensure all nodes have non-zero centrality.

Mathematical Definition:

$$x = (I - \alpha A_{\text{supra}})^{-1} \mathbf{1}$$

where $\alpha < 1/\lambda_{\max}(A_{\text{supra}})$ is the attenuation parameter and $\mathbf{1}$ is a vector of ones.

Best for:

- Controlling the influence of distant connections via α
- Networks with clear source-sink patterns
- Comparing with PageRank and eigenvector centrality
- Tunable sensitivity to network topology

Complexity: $O(n^2)$ for sparse linear system solving, where n is the total number of node-layer pairs

Reference: Katz, L. (1953). A new status index derived from sociometric analysis. *Psychometrika*, 18(1), 39-43.

`py3plex.algorithms.multilayer_algorithms.supra_matrix_function_centrality.katz_centrality`(*supra_matrix*:

*sp-
ma-
trix,*
*al-
pha:*
float
=
None,
beta:
float
=
1.0,
tol:
float
=
1e-06)
→
ndarray

Compute Katz centrality for each node-layer pair.

Katz centrality measures node influence by accounting for all paths with exponentially decaying weights. It is computed as:

$$x = (I - \alpha * A)^{-1} * \text{beta} * 1$$

where $\alpha < 1/\lambda_{\max}(A)$ to ensure convergence. If α is not provided, it defaults to $0.85 / \lambda_{\max}(A)$.

Parameters

- **supra_matrix** – Sparse supra-adjacency matrix (n x n).
- **alpha** – Attenuation parameter. Must be less than $1/\text{spectral_radius}(A)$. If None, defaults to $0.85 / \lambda_{\max}(A)$.
- **beta** – Weight of exogenous influence (typically 1.0).
- **tol** – Tolerance for eigenvalue computation.

Returns

Katz centrality scores for each node-layer pair (n,).
Normalized to sum to 1.

Return type

np.ndarray

Raises

Py3plexMatrixError – If matrix is invalid or α is out of valid range.

Example

```
>>> from py3plex.core import random_generators
>>> net = random_generators.random_multiplex_ER(50, 3, 0.1)
>>> A = net.get_supra_adjacency_matrix()
>>> katz = katz_centrality(A)
>>> print(f"Katz centrality computed for {len(katz)} node-layer pairs")
>>> # With custom alpha
>>> katz_custom = katz_centrality(A, alpha=0.05)
```

6.36.3 Usage Examples

Basic Usage

Compute both centrality measures on a multiplex network:

```
from py3plex.core import random_generators
from py3plex.algorithms.multilayer_algorithms.supra_matrix_function_centrality import (
    communicability_centrality,
    katz_centrality,
)

# Create a random multiplex network
net = random_generators.random_multiplex_ER(50, 3, 0.1)

# Get the supra-adjacency matrix
A = net.get_supra_adjacency_matrix()

# Compute communicability centrality
comm_scores = communicability_centrality(A, normalize=True)
print(f"Communicability centrality computed for {len(comm_scores)} node-layer pairs")
print(f"Top 5 node-layer pairs: {sorted(enumerate(comm_scores), key=lambda x: x[1],
↪reverse=True)[:5]}")
```

(continues on next page)

(continued from previous page)

```
# Compute Katz centrality with automatic alpha
katz_scores = katz_centrality(A, alpha=None)
print(f"Katz centrality computed for {len(katz_scores)} node-layer pairs")
print(f"Normalization check: sum = {katz_scores.sum():.6f}")
```

Advanced Usage

Control computation parameters and compare results:

```
from py3plex.core import random_generators
from py3plex.algorithms.multilayer_algorithms.supra_matrix_function_centrality import (
    communicability_centrality,
    katz_centrality,
)
import numpy as np

# Create a network
net = random_generators.random_multiplex_ER(100, 4, 0.08)
A = net.get_supra_adjacency_matrix()

# Communicability with sparse computation
comm_sparse = communicability_centrality(
    A,
    normalize=True,
    use_sparse=True, # Force sparse computation
    max_iter=100,
    tol=1e-6
)

# Katz with custom alpha and beta
katz_custom = katz_centrality(
    A,
    alpha=0.05,      # Small attenuation for long-range effects
    beta=1.0,        # Uniform exogenous influence
    tol=1e-6
)

# Compare correlation between methods
correlation = np.corrcoef(comm_sparse, katz_custom)[0, 1]
print(f"Correlation between communicability and Katz: {correlation:.3f}")

# Find nodes with high communicability but low Katz
diff = comm_sparse - katz_custom
divergent_nodes = np.argsort(np.abs(diff))[-10:]
print(f"Node-layer pairs with largest differences: {divergent_nodes}")
```

Loading from Network Files

Apply to real network data:

```
from py3plex.core import multinet
from py3plex.algorithms.multilayer_algorithms.supra_matrix_function_centralty import (
    communicability_centralty,
    katz_centralty,
)

# Load a multiplex network from edge list
net = multinet.multi_layer_network().load_network(
    "data/example_multiplex.edgelist",
    input_type="multiplex_edges"
)

# Get supra-adjacency matrix
A = net.get_supra_adjacency_matrix()

# Compute centralities
comm_scores = communicability_centralty(A, normalize=True)
katz_scores = katz_centralty(A, alpha=None)

# Get node-layer mapping to interpret results
node_layer_pairs = list(net.get_nodes())
top_k = 10

print(f"Top {top_k} nodes by communicability centralty:")
for idx in np.argsort(comm_scores)[-top_k:][::-1]:
    node, layer = node_layer_pairs[idx]
    print(f" Node {node} in layer {layer}: {comm_scores[idx]:.6f}")
```

6.36.4 Performance Considerations

Matrix Size and Sparsity

- **Small networks (< 100 node-layer pairs):** Dense computation is faster
- **Medium networks (100-1000 pairs):** Sparse computation recommended
- **Large networks (> 1000 pairs):** Only sparse computation is feasible

The implementation automatically chooses the appropriate method based on matrix size when `use_sparse=True` is set for communicability centralty.

Memory Usage

- **Communicability:** Requires $O(n)$ memory for Krylov subspace methods (sparse) or $O(n^2)$ for dense matrices
- **Katz:** Requires $O(n^2)$ memory for sparse linear system factorization

Computation Time

Typical computation times on a standard workstation:

- **50 node-layer pairs:** < 0.1 seconds
- **150 node-layer pairs (50 nodes × 3 layers):** 0.5-1 second
- **500 node-layer pairs:** 5-10 seconds
- **1500 node-layer pairs (500 nodes × 3 layers):** 30-60 seconds

For very large networks, consider:

- Using higher tolerance values (e.g., `tol=1e-4`)
- Focusing on specific layers rather than the full supra-matrix
- Using other centrality measures like degree or PageRank

6.36.5 Parameter Guidelines

Communicability Centrality

- `normalize=True`: Always recommended for comparing across networks
- `use_sparse=True`: Recommended for networks > 100 node-layer pairs
- `max_iter=100`: Default is sufficient for most networks
- `tol=1e-6`: Default provides good accuracy; increase for speed

Katz Centrality

- `alpha=None`: Auto-compute as $0.85/_\text{max}$ (recommended)
- `alpha=0.01-0.1`: Manual values for custom attenuation
- `beta=1.0`: Default uniform exogenous influence (rarely needs changing)
- `tol=1e-6`: Eigenvalue computation tolerance

6.36.6 Common Issues and Solutions

Matrix Not Square

Error: `Py3plexMatrixError: Matrix must be square`

Solution: Ensure you're passing the supra-adjacency matrix from `get_supra_adjacency_matrix()`, not an edge list or other format.

Alpha Too Large

Error: `Py3plexMatrixError: Alpha must be less than 1/lambda_max`

Solution: Use `alpha=None` for automatic computation, or manually compute `lambda_max` and choose a smaller alpha.

Memory Error

Error: `MemoryError` during computation

Solution:

- Use `use_sparse=True` for communicability centrality
- Ensure your network is not too dense
- Consider analyzing individual layers separately
- Increase available system memory

6.36.7 See Also

- *Multilayer Network Centrality Measures* - Overview of all centrality measures
- *Algorithm Selection Guide* - Choosing the right algorithm
- *Core Concepts and Architecture* - Understanding multilayer networks

6.36.8 References

- Estrada, E., & Hatano, N. (2008). Communicability in complex networks. *Physical Review E*, 77(3), 036111.
- Katz, L. (1953). A new status index derived from sociometric analysis. *Psychometrika*, 18(1), 39-43.
- De Domenico, M., Solé-Ribalta, A., Cozzo, E., Kivelä, M., Moreno, Y., Porter, M. A., ... & Arenas, A. (2013). Mathematical formulation of multilayer networks. *Physical Review X*, 3(4), 041022.

6.37 Random Walk Algorithms

Py3plex provides comprehensive random walk primitives for graph-based algorithms, including Node2Vec, DeepWalk, and diffusion processes. These implementations support weighted, directed, and multilayer networks with deterministic reproducibility.

6.37.1 Overview

Random walks are fundamental building blocks for many graph algorithms:

- **Node2Vec**: Learn node embeddings using biased random walks
- **DeepWalk**: Generate node embeddings through uniform random walks
- **Personalized PageRank**: Compute node importance via random walks with restart
- **Diffusion processes**: Model information or disease propagation
- **Community detection**: Identify network structure through walk patterns

Key Features

- [OK] Basic random walks with proper edge weight handling
- [OK] Second-order (biased) random walks following Node2Vec logic
- [OK] Multiple simultaneous walks with deterministic seeding
- [OK] Support for directed, weighted, and multigraphs
- [OK] Multilayer network-aware walks with layer constraints
- [OK] Efficient sparse adjacency matrix handling
- [OK] Comprehensive test suite validating correctness properties

6.37.2 Basic Random Walk

The basic random walk samples the next node proportionally to normalized edge weights.

Simple Example

```
from py3plex.algorithms.general.walkers import basic_random_walk
import networkx as nx

# Create a simple graph
G = nx.karate_club_graph()

# Perform a random walk
walk = basic_random_walk(
    G,
    start_node=0,
    walk_length=10,
    seed=42 # for reproducibility
)

print(f"Walk: {walk}")
print(f"Length: {len(walk)}") # 11 nodes (includes start)
```

Weighted Graphs

Edge weights are automatically used when `weighted=True` (default):

```
from py3plex.algorithms.general.walkers import basic_random_walk
import networkx as nx

# Create weighted graph
G = nx.Graph()
G.add_weighted_edges_from([
    (0, 1, 10.0), # high weight -> more likely
    (0, 2, 1.0),  # low weight -> less likely
    (1, 2, 5.0),
])

# Weighted walk (respects edge weights)
walk = basic_random_walk(G, 0, walk_length=20, weighted=True, seed=42)

# Unweighted walk (uniform transitions)
walk_uniform = basic_random_walk(G, 0, walk_length=20, weighted=False, seed=42)
```

Directed Graphs

Directed edges are handled correctly:

```
import networkx as nx
from py3plex.algorithms.general.walkers import basic_random_walk

# Create directed graph
G = nx.DiGraph()
G.add_edges_from([(0, 1), (1, 2), (2, 0), (1, 3)])

# Walk respects edge direction
walk = basic_random_walk(G, 0, walk_length=10, seed=42)
```

(continues on next page)

(continued from previous page)

```
# Verify all transitions follow directed edges
for i in range(len(walk) - 1):
    assert G.has_edge(walk[i], walk[i+1])
```

6.37.3 Node2Vec Biased Random Walk

Second-order random walks with return parameter p and in-out parameter q following the Node2Vec algorithm (Grover & Leskovec, 2016).

Parameters

When transitioning from node $t \rightarrow v \rightarrow x$, the probability of choosing x is biased:

- If $x == t$ (return to previous): weight / p
- If x is neighbor of t (stay close): $\text{weight} / 1$
- If x is not neighbor of t (explore): weight / q

Parameter Effects:

- $p > 1$: Less likely to return to previous node (explore forward)
- $p < 1$: More likely to return (backtrack)
- $q > 1$: Less likely to explore distant nodes (stay local, BFS-like)
- $q < 1$: More likely to explore distant nodes (venture far, DFS-like)
- $p = q = 1$: Equivalent to basic random walk

Basic Usage

```
from py3plex.algorithms.general.walkers import node2vec_walk
import networkx as nx

G = nx.karate_club_graph()

# Balanced walk (p=1, q=1)
walk_balanced = node2vec_walk(G, 0, walk_length=20, p=1.0, q=1.0, seed=42)

# BFS-like walk (stay local)
walk_bfs = node2vec_walk(G, 0, walk_length=20, p=1.0, q=2.0, seed=42)

# DFS-like walk (explore outward)
walk_dfs = node2vec_walk(G, 0, walk_length=20, p=1.0, q=0.5, seed=42)
```

Exploring vs Backtracking

```
from py3plex.algorithms.general.walkers import node2vec_walk
import networkx as nx

# Triangle graph
G = nx.Graph()
G.add_edges_from([(0, 1), (1, 2), (2, 0)])
```

(continues on next page)

(continued from previous page)

```

# Low p, high q: tends to backtrack
walk_backtrack = node2vec_walk(G, 0, walk_length=20, p=0.1, q=10.0, seed=42)

# High p, low q: tends to explore outward
walk_explore = node2vec_walk(G, 0, walk_length=20, p=10.0, q=0.1, seed=42)

# Count backtracking patterns
backtracks = sum(1 for i in range(2, len(walk_backtrack))
                  if walk_backtrack[i] == walk_backtrack[i-2])
print(f"Backtracks: {backtracks}")

```

6.37.4 Multiple Walk Generation

Generate multiple walks efficiently with deterministic seeding.

Generate Walks from All Nodes

```

from py3plex.algorithms.general.walkers import generate_walks
import networkx as nx

G = nx.karate_club_graph()

# Generate 10 walks from each node
walks = generate_walks(
    G,
    num_walks=10,
    walk_length=10,
    seed=42
)

print(f"Total walks: {len(walks)}") # 340 (34 nodes × 10 walks)
print(f"First walk: {walks[0]}")

```

Generate from Specific Nodes

```

from py3plex.algorithms.general.walkers import generate_walks
import networkx as nx

G = nx.karate_club_graph()

# Generate walks only from nodes 0, 1, 2
walks = generate_walks(
    G,
    num_walks=5,
    walk_length=15,
    start_nodes=[0, 1, 2],
    seed=42
)

print(f"Total walks: {len(walks)}") # 15 (3 nodes × 5 walks)

```

Node2Vec-Style Walks

```
from py3plex.algorithms.general.walkers import generate_walks
import networkx as nx

G = nx.karate_club_graph()

# Generate biased walks with p=0.5, q=2.0
walks = generate_walks(
    G,
    num_walks=10,
    walk_length=20,
    p=0.5,
    q=2.0,
    seed=42
)
```

Edge Sequences

Return walks as edge sequences instead of node sequences:

```
from py3plex.algorithms.general.walkers import generate_walks
import networkx as nx

G = nx.karate_club_graph()

# Generate edge sequences
edge_walks = generate_walks(
    G,
    num_walks=5,
    walk_length=10,
    return_edges=True,
    seed=42
)

# First walk is a list of edges
print(edge_walks[0]) # [(0, 3), (3, 2), (2, 1), ...]
```

6.37.5 Multilayer Network Walks

Perform random walks on multilayer networks with layer constraints.

Layer-Constrained Walks

```
from py3plex.algorithms.general.walkers import layer_specific_random_walk
from py3plex.core import multinet

# Create multilayer network
network = multinet.multi_layer_network()
network.add_layer("social")
network.add_layer("biological")

network.add_nodes([
```

(continues on next page)

(continued from previous page)

```

    ("A", "social"), ("B", "social"), ("C", "social"),
    ("A", "biological"), ("B", "biological")
])

network.add_edges([
    (("A", "social"), ("B", "social")),
    (("B", "social"), ("C", "social")),
    (("A", "biological"), ("B", "biological")),
])

# Walk constrained to social layer
walk = layer_specific_random_walk(
    network.core_network,
    start_node="A---social",
    walk_length=10,
    layer="social",
    cross_layer_prob=0.0, # never cross layers
    seed=42
)

# All nodes in walk are in social layer
print(walk)

```

Cross-Layer Transitions

Allow occasional transitions between layers:

```

from py3plex.algorithms.general.walkers import layer_specific_random_walk

# Same network as above

# Walk with 10% probability of crossing layers
walk = layer_specific_random_walk(
    network.core_network,
    start_node="A---social",
    walk_length=20,
    layer="social",
    cross_layer_prob=0.1, # 10% chance to cross
    seed=42
)

# May contain nodes from both layers
print(walk)

```

6.37.6 Reproducibility

All walk functions support deterministic seeding for reproducibility.

Fixed Seed

```
from py3plex.algorithms.general.walkers import basic_random_walk
import networkx as nx

G = nx.karate_club_graph()

# Same seed produces identical walks
walk1 = basic_random_walk(G, 0, 50, seed=42)
walk2 = basic_random_walk(G, 0, 50, seed=42)

assert walk1 == walk2 # Identical
```

Different Seeds

```
# Different seeds produce different walks
walk1 = basic_random_walk(G, 0, 50, seed=42)
walk2 = basic_random_walk(G, 0, 50, seed=123)

assert walk1 != walk2 # Different
```

Batch Reproducibility

```
from py3plex.algorithms.general.walkers import generate_walks

G = nx.karate_club_graph()

# Same seed produces identical walk sets
walks1 = generate_walks(G, num_walks=100, walk_length=10, seed=42)
walks2 = generate_walks(G, num_walks=100, walk_length=10, seed=42)

assert walks1 == walks2
```

6.37.7 Performance Tips

Large Graphs

For large sparse graphs (>100k nodes), use basic walks for better performance:

```
from py3plex.algorithms.general.walkers import generate_walks
import networkx as nx

# Large sparse graph
G = nx.erdos_renyi_graph(100000, 0.0001, seed=42)

# Use basic walk (p=1, q=1) for speed
walks = generate_walks(
    G,
    num_walks=10,
```

(continues on next page)

(continued from previous page)

```

walk_length=100,
p=1.0, # No bias = faster
q=1.0,
seed=42
)

```

Parallel Processing

For generating many walks, consider parallel processing:

```

from py3plex.algorithms.general.walkers import generate_walks
import networkx as nx
from multiprocessing import Pool

G = nx.karate_club_graph()

def generate_batch(seed):
    return generate_walks(G, num_walks=10, walk_length=20, seed=seed)

# Generate walks in parallel
with Pool(4) as pool:
    batch_walks = pool.map(generate_batch, range(10))

# Flatten results
all_walks = [walk for batch in batch_walks for walk in batch]

```

6.37.8 Correctness Validation

The implementation has been validated against the following properties:

Uniformity Test

On unweighted regular graphs, transition probabilities are uniform:

```

import networkx as nx
from py3plex.algorithms.general.walkers import basic_random_walk
from collections import Counter

# Complete graph (regular)
G = nx.complete_graph(10)

# Count visits to neighbors from node 0
visits = Counter()
for i in range(10000):
    walk = basic_random_walk(G, 0, 1, weighted=False, seed=i)
    if len(walk) > 1:
        visits[walk[1]] += 1

# All neighbors should be visited roughly equally
print(visits)
# Expected: ~1111 visits to each of 9 neighbors

```

Conservation Test

Sum of transition probabilities equals 1.0 (within machine epsilon):

```
import networkx as nx
import numpy as np

G = nx.karate_club_graph()

# For each node, verify probability conservation
for node in G.nodes():
    neighbors = list(G.neighbors(node))
    if len(neighbors) > 0:
        weights = np.array([G[node][n].get('weight', 1.0) for n in neighbors])
        probs = weights / weights.sum()
        assert abs(probs.sum() - 1.0) < 1e-10 # Machine epsilon
```

Bias Consistency Test

Node2Vec parameters p and q produce expected biases:

```
import networkx as nx
from py3plex.algorithms.general.walkers import node2vec_walk

# Triangle graph
G = nx.Graph()
G.add_edges_from([(0, 1), (1, 2), (2, 0)])

# Low p should encourage backtracking
backtracks_low_p = 0
for i in range(1000):
    walk = node2vec_walk(G, 0, 10, p=0.1, q=1.0, seed=i)
    for j in range(2, len(walk)):
        if walk[j] == walk[j-2]:
            backtracks_low_p += 1

# High p should discourage backtracking
backtracks_high_p = 0
for i in range(1000):
    walk = node2vec_walk(G, 0, 10, p=10.0, q=1.0, seed=i)
    for j in range(2, len(walk)):
        if walk[j] == walk[j-2]:
            backtracks_high_p += 1

assert backtracks_low_p > backtracks_high_p * 2 # Significantly more
```

Edge Weight Test

Visit frequency matches edge weight ratios:

```
import networkx as nx
from py3plex.algorithms.general.walkers import basic_random_walk
from collections import Counter
```

(continues on next page)

(continued from previous page)

```

G = nx.Graph()
G.add_weighted_edges_from([
    (0, 1, 1.0),
    (0, 2, 2.0),
    (0, 3, 3.0),
])

visits = Counter()
for i in range(10000):
    walk = basic_random_walk(G, 0, 1, weighted=True, seed=i)
    if len(walk) > 1:
        visits[walk[1]] += 1

# Expected ratio: 1:2:3
total = sum(visits.values())
ratios = [visits[1]/total, visits[2]/total, visits[3]/total]
expected = [1/6, 2/6, 3/6]

# Within 5% tolerance
for obs, exp in zip(ratios, expected):
    assert abs(obs - exp) < 0.05

```

6.37.9 References

Node2Vec:

Grover, A., & Leskovec, J. (2016). node2vec: Scalable feature learning for networks. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 855-864. <https://doi.org/10.1145/2939672.2939754>

DeepWalk:

Perozzi, B., Al-Rfou, R., & Skiena, S. (2014). DeepWalk: Online learning of social representations. *Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 701-710. <https://doi.org/10.1145/2623330.2623732>

Random Walks on Graphs:

Lovász, L. (1993). Random walks on graphs: A survey. *Combinatorics, Paul Erdős is Eighty*, 2, 1-46.

6.37.10 API Reference

See `py3plex.algorithms.general.walkers` for complete API documentation.

6.37.11 Examples

Complete examples can be found in:

- `examples/advanced/example_random_walks.py` - Basic usage examples
- `tests/test_random_walks.py` - Comprehensive test suite

Repository: <https://github.com/SkBlaz/py3plex>

6.38 Supra-Adjacency Matrices

Multiplex networks can be represented as supra-adjacency matrices for tensor-based operations.

```
from py3plex.core import multinet, random_generators

# Generate network
network = random_generators.random_multilayer_ER(
    num_nodes=500, num_layers=8, probability=0.05, directed=False)

# Get supra-adjacency matrix
supra_matrix = network.get_supra_adjacency_matrix()

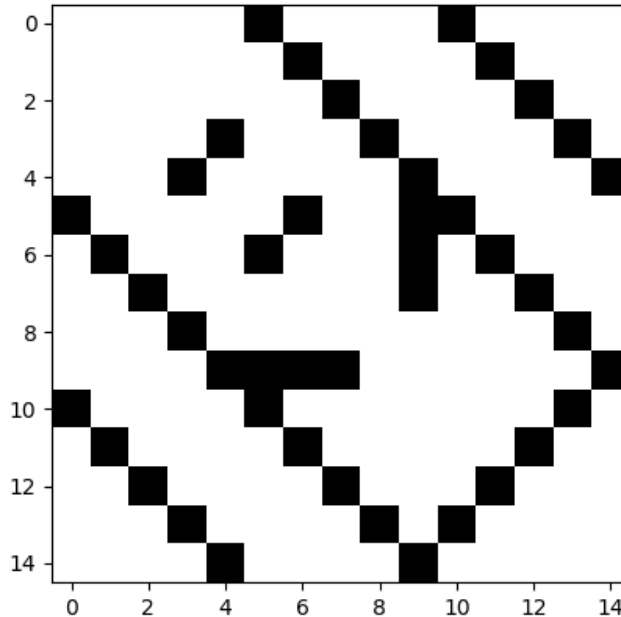
# Visualize matrix
network.visualize_matrix({"display": True})
```

6.38.1 Examples

See:

- `example_supra_adjacency.py` - Supra-adjacency operations
- `example_tensorial_manipulation.py` - Tensor operations

Repository: <https://github.com/SkBlaz/Py3Plex/tree/master/examples>



6.38.2 Tensor-Like Indexing

You can access nodes and edges using tensor-like indexing:

```
from py3plex.core import multinet
from py3plex.core import random_generators

# Initiate an instance of a random graph
ER_multilayer = random_generators.random_multilayer_ER(500, 8, 0.05, directed=False)

# Some simple visualization
visualization_params = {"display": True}
ER_multilayer.visualize_matrix(visualization_params)

# Get some nodes and edges
some_nodes = [node for node in ER_multilayer.get_nodes()][0:5]
some_edges = [edge for edge in ER_multilayer.get_edges()][0:5]

# Random node is accessed as follows
print(ER_multilayer[some_nodes[0]])

# Random edge is accessed as
print(ER_multilayer[some_edges[0][0]][some_edges[0][1]])
```

6.39 SIR Epidemic Simulator on Multiplex Graphs

This module provides efficient, reproducible SIR (Susceptible-Infected-Recovered) epidemic simulators for multiplex networks (same node set, multiple edge layers).

6.39.1 Features

- **Discrete-time simulation:** Step-based updates with configurable time step
- **Continuous-time simulation:** Gillespie algorithm for exact event-driven dynamics
- **Multiplex support:** Multiple interaction layers with per-layer transmission rates
- **Flexible parameterization:** Per-layer transmission rates, per-node recovery rates, layer weights
- **Exogenous importations:** Optional external infection sources
- **Rich outputs:** Incidence curves, event logs, per-layer attribution
- **Sparse linear algebra:** Efficient handling of large networks
- **Reproducible:** Seeded random number generation

6.39.2 Model Definition

States

Each node has a single health state shared across all layers:

- **S (0):** Susceptible
- **I (1):** Infected
- **R (2):** Recovered

Transmission

- Infections occur along layer-specific edges with rate/probability []
- Edge weights can represent contact multiplicity or strength
- Hazards from all layers combine:
 - **Continuous-time (Gillespie)**: Additive combination
 - **Discrete-time**: Independent-trial complement ($1 - \exp(-_ dt)$)

Recovery

- Infected nodes recover at rate (scalar or per-node)
- Recovery is independent of network structure

Optional Features

- **Layer weights**: Scale transmission rates by layer importance
- **Time-varying parameters**: $(, t)$ and (i, t) as callables
- **Exogenous infections**: Poisson import rate (t)

6.39.3 Installation

The SIR multiplex module is part of py3plex. Ensure you have the required dependencies:

```
pip install numpy scipy
```

Or install py3plex with all dependencies:

```
pip install git+https://github.com/SkBlaz/py3plex.git
```

6.39.4 Quick Start

Discrete-Time Simulation

```
import numpy as np
import scipy.sparse
from py3plex.algorithms.sir_multiplex import simulate_sir_multiplex_discrete

# Create a simple ring network
N = 20
edges = [(i, (i+1) % N) for i in range(N)]
row, col = zip(*edges)
A = scipy.sparse.csr_matrix((np.ones(len(edges)), (row, col)), shape=(N, N))

# Run simulation
result = simulate_sir_multiplex_discrete(
    A_layers=[A],
    beta=0.5,          # Transmission rate
    gamma=0.2,         # Recovery rate
    dt=0.5,            # Time step
    steps=100,         # Number of steps
    rng_seed=42        # For reproducibility
```

(continues on next page)

(continued from previous page)

```
)

# Access results
print(f"Peak prevalence: {result.I.max()} at time {result.times[result.I.argmax()]}")
print(f"Final attack rate: {result.R[-1] / N:.2%}")
```

Continuous-Time (Gillespie) Simulation

```
from py3plex.algorithms.sir_multiplex import simulate_sir_multiplex_gillespie

result = simulate_sir_multiplex_gillespie(
    A_layers=[A],
    beta=0.5,
    gamma=0.2,
    t_max=100.0,      # Maximum time
    rng_seed=42,
    return_event_log=True # Get detailed event log
)

# Examine events
for t, event_type, node_id, layer in result.events[:10]:
    print(f"t={t:.2f}: {event_type} at node {node_id}")
```

Multiplex Network Example

```
# Create two-layer network: physical contact + online interaction
N = 50

# Layer 1: Physical proximity (localized)
edges_physical = [(i, (i+1) % N) for i in range(N)]
row, col = zip(*edges_physical)
A_physical = scipy.sparse.csr_matrix((np.ones(len(edges_physical)), (row, col)),
    ↪ shape=(N, N))

# Layer 2: Online network (more random)
np.random.seed(42)
edges_online = [(i, j) for i in range(N) for j in range(N)
    if i != j and np.random.random() < 0.1]
row, col = zip(*edges_online) if edges_online else ([], [])
data = np.ones(len(edges_online)) if edges_online else []
A_online = scipy.sparse.csr_matrix((data, (row, col)), shape=(N, N))

# Simulate with different transmission rates per layer
result = simulate_sir_multiplex_discrete(
    A_layers=[A_physical, A_online],
    beta=np.array([0.3, 0.1]), # Higher transmission in physical layer
    gamma=0.15,
    layer_weights=np.array([1.0, 0.5]), # Weight online layer less
    dt=0.5,
    steps=200,
    rng_seed=123,
```

(continues on next page)

(continued from previous page)

```
    return_layer_incidence=True # Track which layer caused infections
)

# Analyze layer contributions
from py3plex.algorithms.sir_multiplex import summarize
summary = summarize(result)
print(f"Layer contributions: {summary['layer_contributions']}")
```

6.39.5 API Reference

simulate_sir_multiplex_discrete

Discrete-time SIR simulation with synchronous updates.

Parameters:

- **A_layers** (list[csr_matrix]): List of L sparse adjacency matrices (N×N)
- **beta** (float | ndarray): Transmission rate (scalar or length L array)
- **gamma** (float | ndarray): Recovery rate (scalar or length N array)
- **layer_weights** (ndarray, optional): Layer importance weights (length L)
- **dt** (float): Time step size (default: 1.0)
- **steps** (int): Number of simulation steps (default: 100)
- **initial_state** (ndarray, optional): Initial state array with values {0,1,2}
- **initial_infected** (ndarray, optional): Boolean mask for initially infected nodes
- **import_rate** (float | callable): Exogenous infection rate (default: 0.0)
- **rng_seed** (int): Random seed (default: 0)
- **return_event_log** (bool): Return event log (default: False)
- **return_layer_incidence** (bool): Return per-layer infections (default: False)

Returns: EpidemicResult object

simulate_sir_multiplex_gillespie

Continuous-time SIR simulation using Gillespie algorithm.

Parameters: Same as discrete, except:

- **t_max** (float): Maximum simulation time instead of dt and steps
- **return_event_log** (bool): Default is True

Returns: EpidemicResult object

EpidemicResult

Dataclass containing simulation results:

- **times** (ndarray): Time points
- **S, I, R** (ndarray): Counts of each state over time
- **states** (ndarray, optional): Full state history (discrete only)

- `incidence_by_layer` (ndarray, optional): Infections per layer over time
- `events` (list, optional): Event log with tuples (time, type, node, layer)
- `meta` (dict): Simulation metadata

basic_reproduction_number

Compute approximate R_0 using spectral radius.

```
R0 = basic_reproduction_number(A_layers, beta, gamma, layer_weights=None)
```

For homogeneous networks with uniform recovery rate :

$$R_0 \approx \rho \left(\sum_{\alpha} \frac{\beta_{\alpha} w_{\alpha}}{\gamma} A^{(\alpha)} \right)$$

summarize

Extract summary statistics from simulation results.

```
summary = summarize(result)
```

Returns dictionary with:

- `peak_prevalence`: Maximum infected count
- `peak_time`: Time of peak
- `attack_rate`: Final proportion recovered
- `time_to_extinction`: When infections reached zero
- `total_infections`: Total recovered at end
- `layer_contributions`: Proportion of infections per layer
- `duration`: Total simulation time

6.39.6 Update Rules

Discrete-Time

For each time step $t \rightarrow t + dt$:

1. **Infection probability** for susceptible node i :

$$\lambda_{i,\alpha}(t) = \beta_{\alpha} w_{\alpha} \sum_j A_{ji}^{(\alpha)} \mathbb{1}_{\{X_j = I\}}$$

$$p_i(t) = 1 - \prod_{\alpha} \exp(-\lambda_{i,\alpha}(t) dt)$$

2. **Recovery probability** for infected node i :

$$p_{\text{rec}}(i) = 1 - \exp(-\gamma_i dt)$$

3. **Synchronous update**: Compute all probabilities from state at t , then apply changes

Continuous-Time (Gillespie)

1. Compute total event rate:

$$\Lambda = \sum_{i \in S} \sum_{\alpha} \lambda_{i,\alpha} + \sum_{i \in I} \gamma_i + \eta |S|$$

2. Sample time to next event: $\Delta t \sim \text{Exp}(\Lambda)$
3. Select event type proportionally to rates
4. Update state and incrementally adjust affected rates
5. Record event and repeat until $t > t_{\max}$ or no infected remain

6.39.7 Units and Conventions

- **Time:** Arbitrary units (days, hours, etc.). Ensure λ , γ , η , $\text{dt}/t_{\max}$ are in same units.
- **Rates:** Events per time unit. $\lambda = 0.5$ per day means average 0.5 transmissions per infected-susceptible contact per day.
- **Probabilities:** For discrete-time, computed as $1 - \exp(-\text{rate} \times \text{dt})$
- **Edge weights:** Scale transmission. Weight 2.0 doubles effective contact rate.

6.39.8 Performance Tips

- Use sparse matrices (CSR format) for all adjacency matrices
- For large networks ($N > 10^4$), use smaller time steps or $t_{\max}$
- Disable event logs for very long simulations to save memory
- Vectorized operations are used internally for efficiency

6.39.9 Examples

See `examples/advanced/example_sir_multiplex.py` for additional examples including:

- Comparison of discrete vs continuous time
- Effect of network structure on epidemic spread
- Multi-layer network analysis
- Parameter sensitivity studies

6.39.10 References

- Gillespie, D. T. (1977). Exact stochastic simulation of coupled chemical reactions. *Journal of Physical Chemistry*, 81(25), 2340-2361.
- Keeling, M. J., & Rohani, P. (2008). *Modeling infectious diseases in humans and animals*. Princeton University Press.
- De Domenico, M., et al. (2013). Mathematical formulation of multilayer networks. *Physical Review X*, 3(4), 041022.

6.39.11 Citation

If you use this simulator in your research, please cite py3plex:

```
@software{py3plex,
  author = {Škrlj, Blaž},
  title = {Py3plex: A library for analysis and visualization of heterogeneous networks},
  url = {https://github.com/SkBlaz/py3plex},
  year = {2023}
}
```

6.40 Py3plex GUI

A production-ready web-based GUI for **py3plex** multilayer network analysis, running locally via Docker Compose.

6.40.1 Visual Overview

The Py3plex GUI provides an intuitive web interface for multilayer network analysis. Below are screenshots of the main interface pages:

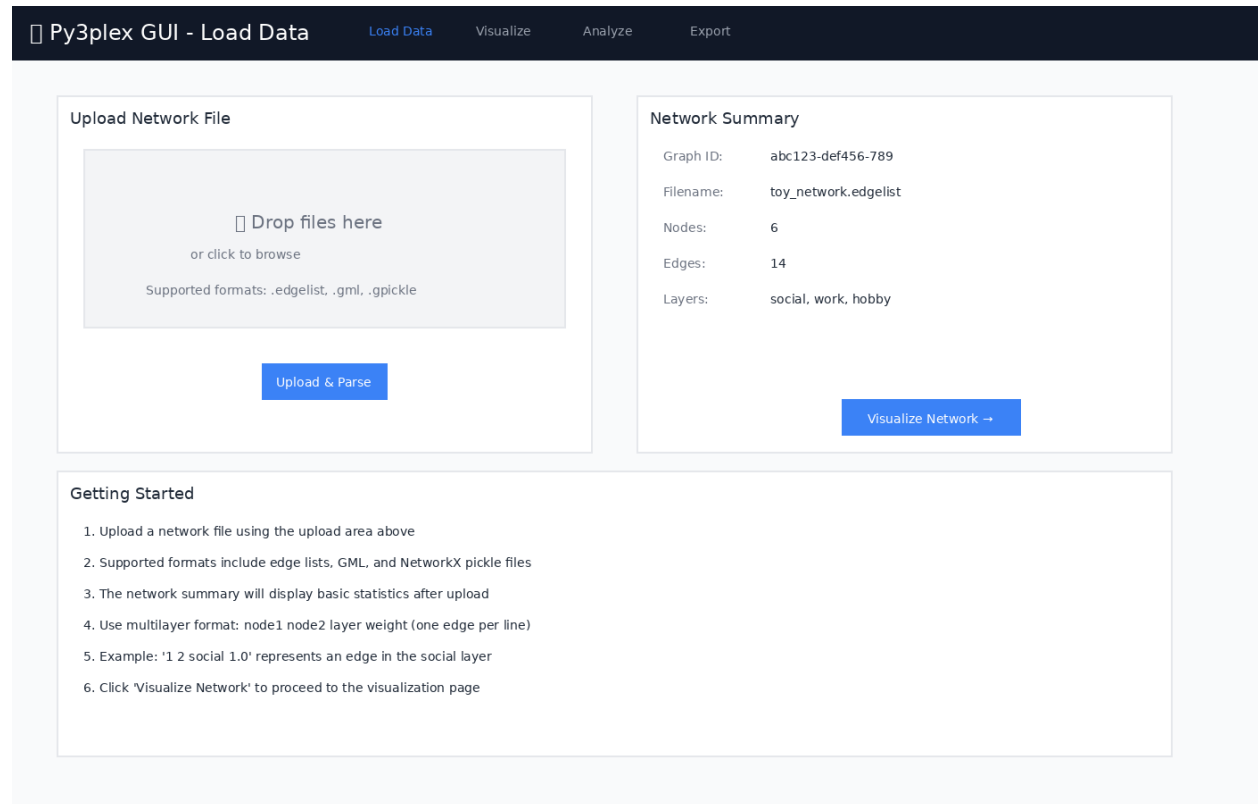


Figure 1: Load Data page for uploading network files in various formats

6.40.2 Quick Start

```
# Navigate to the gui directory
cd gui

# Copy environment configuration
cp .env.example .env

# Start all services (builds automatically)
make up

# Open in browser
# Application: http://localhost:8080
# Data Job Monitor (Flower): http://localhost:5555
```

First time? The build takes ~3-5 minutes. Subsequent starts are instant.

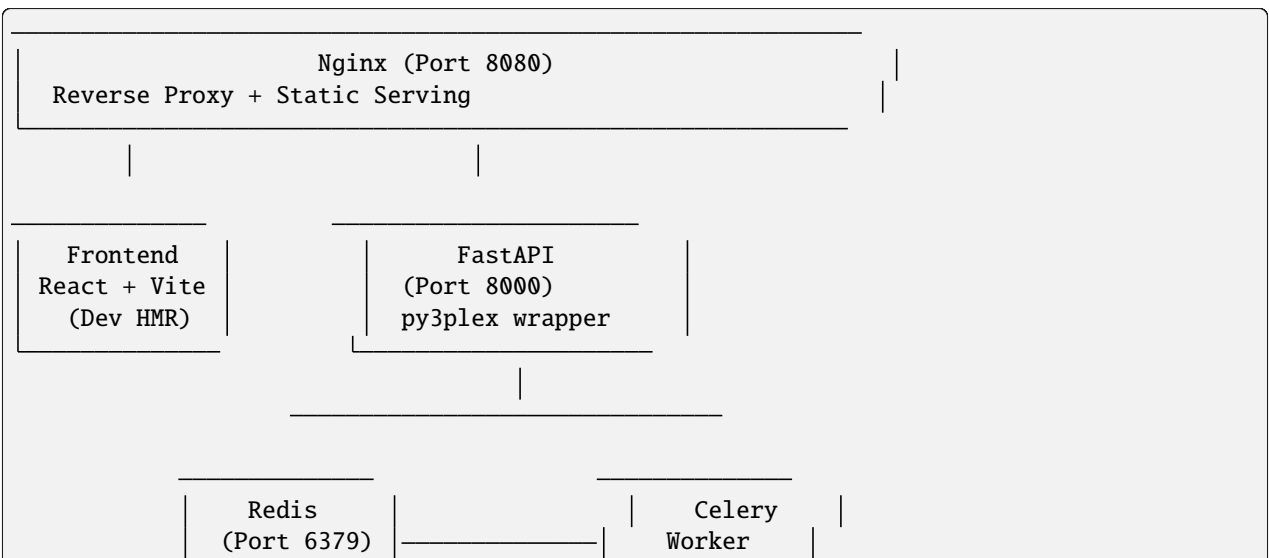
That's it! The application will be running with:

- React + TypeScript frontend with hot reload
- FastAPI backend with py3plex integration
- Celery workers for async jobs
- Redis broker
- Nginx reverse proxy

6.40.3 Prerequisites

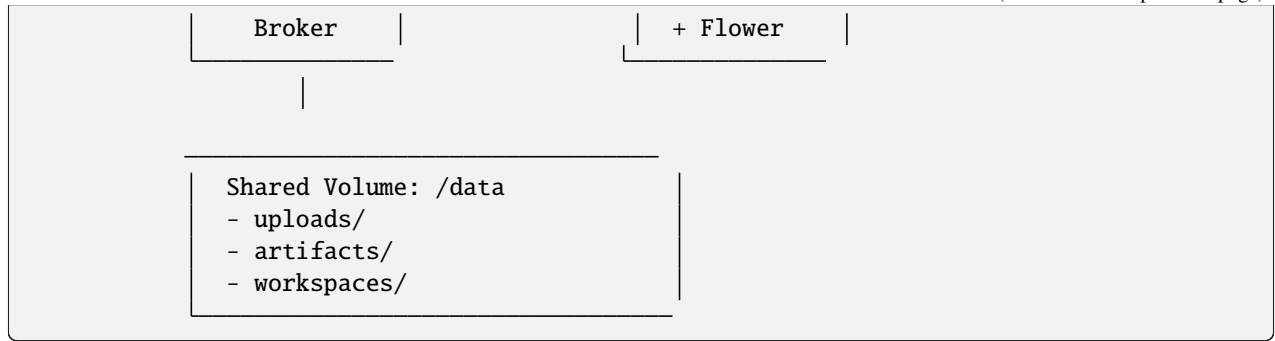
- Docker (≥ 20.10)
- Docker Compose (≥ 2.0)
- 4GB RAM minimum
- Ports available: 8080 (GUI), 5555 (Flower), 8000 (API), 6379 (Redis)

6.40.4 Architecture



(continues on next page)

(continued from previous page)

**Key Components:**

- **Frontend:** React 18 + Vite + TypeScript + Tailwind CSS
- **API:** FastAPI (Python 3.12) with py3plex integration
- **Worker:** Celery for long-running analysis jobs
- **Broker:** Redis for job queue
- **Proxy:** Nginx for unified access
- **Monitoring:** Flower dashboard for job inspection

6.40.5 Features**Data Loading**

- Upload network files (.edgelist, .txt, .gml, .gpickle)
- Automatic format detection
- Multilayer network support
- Real-time file preview

Visualization

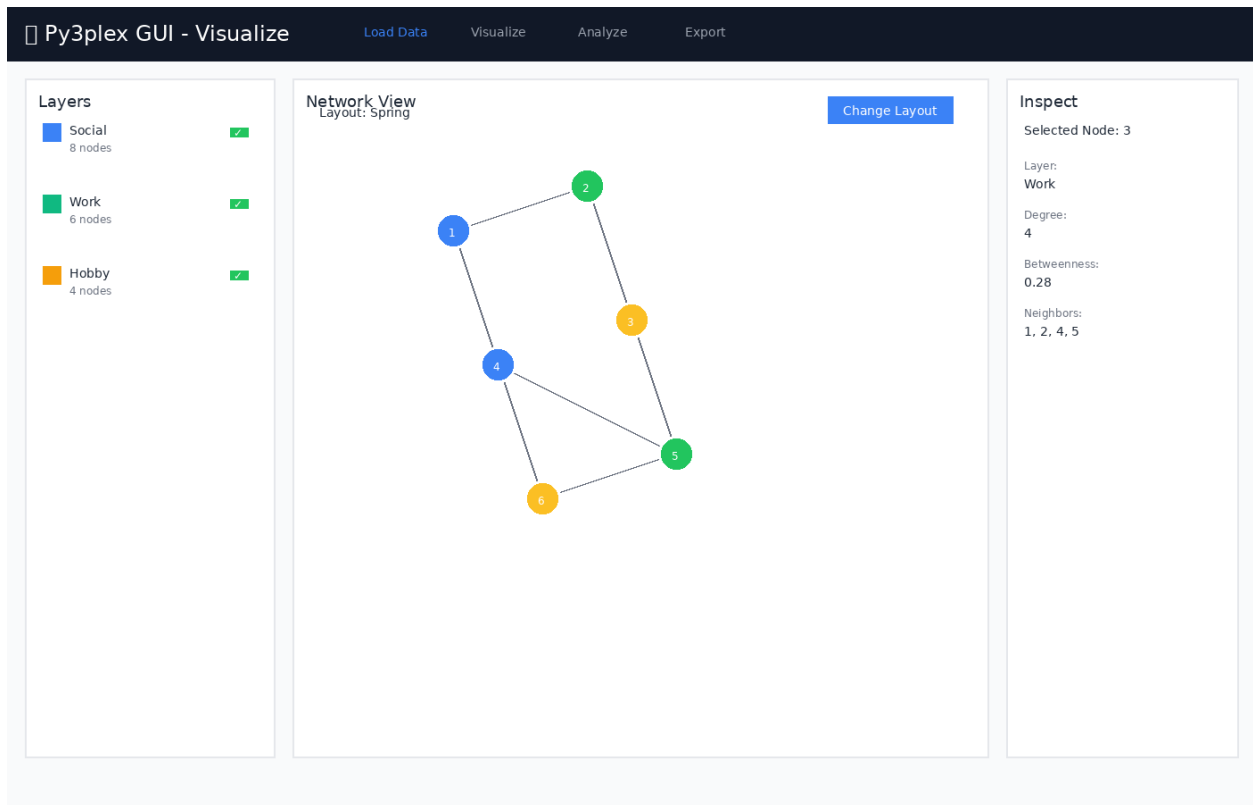


Figure 2: Network visualization with layer controls and node inspection panel

Features:

- Layer-centric view
- Interactive node/edge inspection
- Configurable layouts
- Position caching

Analysis (Async via Celery)

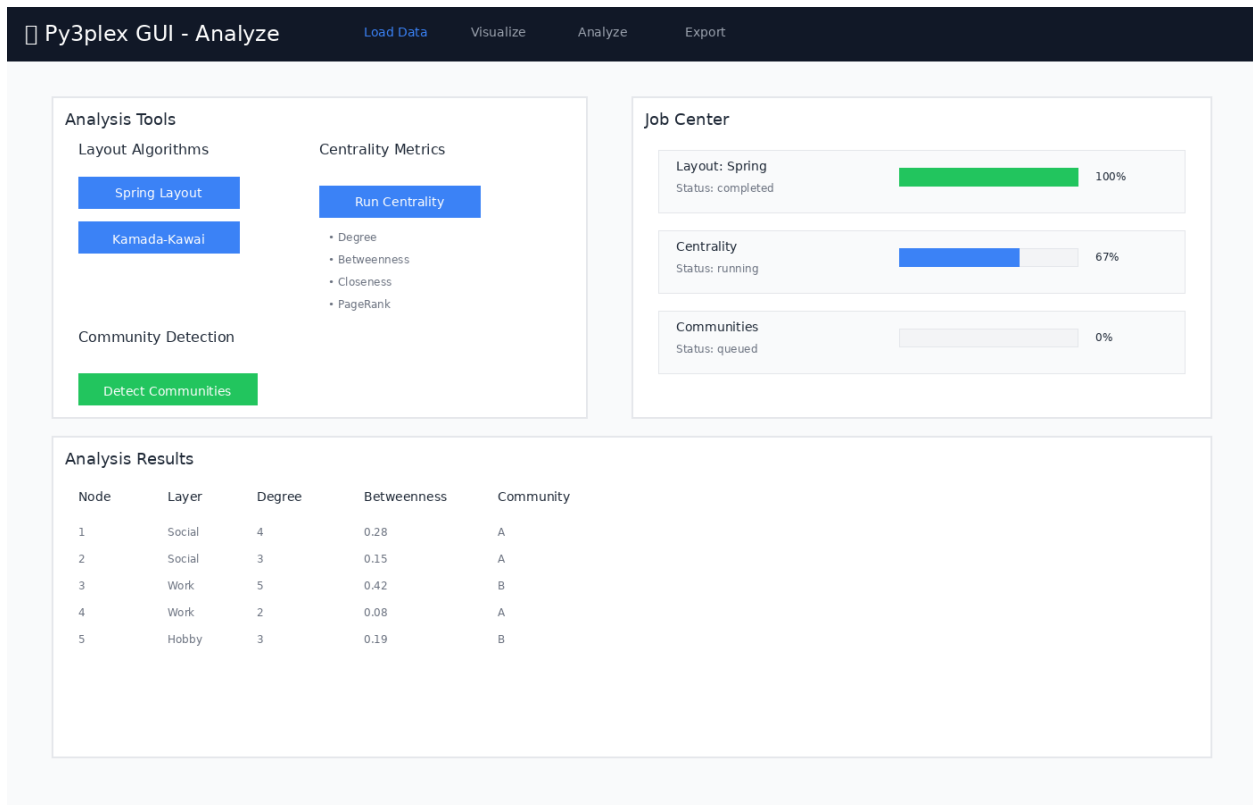


Figure 3: Analysis page with job controls and real-time progress tracking

Available Analysis Tools:

- **Layouts:** Spring, Kamada-Kawai, Circular, Random
- **Centrality:** Degree, Betweenness, Closeness, Eigenvector, PageRank
- **Community Detection:** Louvain, Label Propagation, Greedy Modularity
- Real-time progress tracking
- Result caching

Export

Py3plex GUI - Export
Load Data
Visualize
Analyze
Export

Export Options

Network Positions (JSON)
Export node positions from current layout

Export

Centrality Results (CSV)
Export computed centrality metrics

Export

Community Assignments (CSV)
Export community detection results

Export

Workspace Bundle (ZIP)
Save complete analysis session

Export

Workspace Management

Save Current Workspace

my-analysis-workspace

Save Workspace Bundle

Recent Workspaces

network-analysis-2024-11-23.zip	Load
multilayer-communities-v2.zip	Load
social-network-export.zip	Load

Export Information

- JSON exports contain node positions compatible with web visualizations
- CSV exports can be opened in Excel, R, or Python for further analysis
- Workspace bundles include the original network, all computed results, and UI state
- Workspace bundles can be shared with collaborators or loaded later
- All exports are saved to the data/artifacts/ directory

Figure 4: Export page for saving analysis results and workspace bundles

Export Options:

- CSV summaries (centrality, communities)
- JSON position data
- PNG snapshots (planned)
- Workspace bundles (data + state + results)

Workspace Bundles

Save and restore complete analysis sessions:

```
# Bundle includes:
# - Original network file
# - Computed layouts
# - Centrality results
# - Community assignments
# - UI view state
```


Job Monitoring

The GUI includes Flower dashboard for real-time monitoring of background jobs:

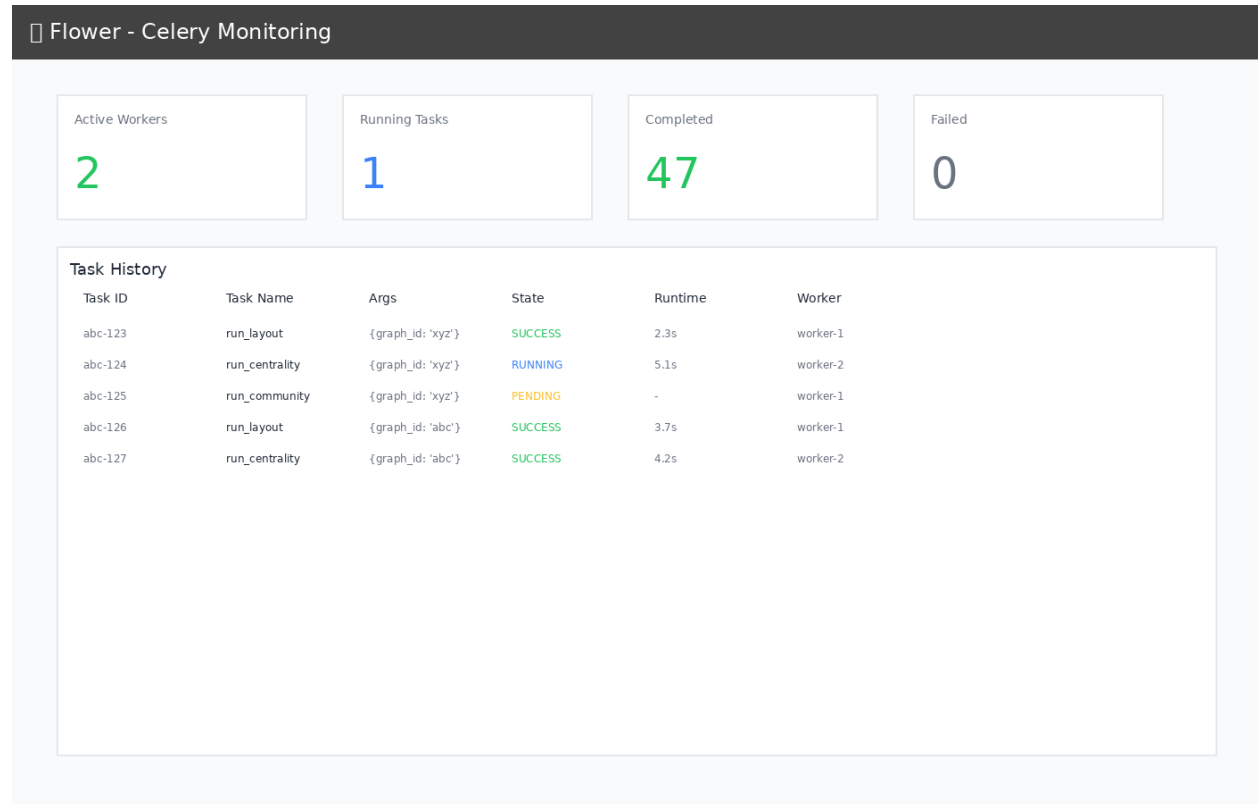


Figure 5: Flower dashboard showing task history and worker status

Monitoring Features:

- Track task execution in real-time
- View worker status and performance
- Inspect task history and results
- Monitor task queue and completion rates
- Access at <http://localhost:5555> when running

6.40.6 Development Guide

Project Structure

```
gui/
├── docker-compose.yml      # Main orchestration
├── compose.gpu.yml         # GPU override (optional)
├── Makefile                # Convenience commands
└── .env.example            # Configuration template
```

(continues on next page)

(continued from previous page)

```

├── nginx/
│   └── nginx.conf          # Reverse proxy config
├── api/
│   ├── Dockerfile.api     # API container
│   ├── pyproject.toml     # Python dependencies
│   └── app/
│       ├── main.py        # FastAPI app
│       ├── schemas.py     # Pydantic models
│       ├── deps.py        # DI & config
│       ├── routes/        # API endpoints
│       ├── services/      # Business logic (py3plex wrappers)
│       ├── workers/       # Celery tasks
│       └── utils/         # Helpers
├── worker/
│   └── Dockerfile         # Worker container
├── frontend/
│   ├── Dockerfile.frontend # Frontend container
│   ├── package.json       # Node dependencies
│   ├── vite.config.ts     # Vite config
│   └── src/
│       ├── App.tsx        # Root component
│       ├── lib/api.ts     # API client
│       ├── pages/         # Page components
│       └── components/    # Reusable components
└── data/                  # Shared volume (gitignored)
    ├── uploads/
    ├── artifacts/
    └── workspaces/

```

Makefile Commands

```

make setup      # Copy .env.example to .env
make up         # Start all services (build if needed)
make down       # Stop and remove containers + volumes
make restart    # Restart all services
make build      # Rebuild Docker images
make logs       # Tail logs from all services

# Development helpers
make bash-api   # Shell into API container
make bash-worker # Shell into worker container
make bash-frontend # Shell into frontend container

# Testing
make test-api   # Run API tests
make e2e        # Run end-to-end tests (WIP)

# Cleanup
make clean      # Remove containers, volumes, and data

```

Local Development Against py3plex

The py3plex repository root is bind-mounted read-only into containers at /workspace:

```
# In Dockerfile.api
ARG PY3PLEX_PATH=/workspace
RUN pip install -e ${PY3PLEX_PATH}
```

Benefits:

- Changes to py3plex core reflect immediately (no rebuild)
- Editable install for development
- Isolated GUI code under gui/

Note: The mount is read-only to prevent accidental writes from containers.

6.40.7 Configuration

Environment Variables (.env)

```
# API
API_WORKERS=2           # Uvicorn workers
MAX_UPLOAD_MB=512       # Max file size

# Celery
CELERY_CONCURRENCY=2    # Worker threads
REDIS_URL=redis://redis:6379/0

# Frontend
VITE_API_URL=http://localhost:8080/api
```

GPU Support (Optional)

Enable NVIDIA GPU acceleration:

```
docker compose -f docker-compose.yml -f compose.gpu.yml up
```

Requirements:

- NVIDIA Docker runtime installed
- CUDA-compatible GPU

6.40.8 Data Formats

Accepted Input Formats

1. Edge List (.edgelist, .txt)

```
# Multilayer format: node1 node2 layer weight
1 2 social 1.0
1 3 work 1.0
2 3 hobby 0.5

# Simple format: node1 node2
A B
```

(continues on next page)

(continued from previous page)

```
B C
C D
```

2. GML (.gml)

- Graph Modeling Language
- Preserves attributes and metadata

3. NetworkX Pickle (.gpickle)

- Native NetworkX serialization
- Fastest for large graphs

Example Dataset

Try the included toy network:

```
# From host
curl -F "file=@gui/toy_network.edgelist" http://localhost:8080/api/upload

# Or use the Web UI at http://localhost:8080
```

6.40.9 Testing

Continuous Integration

GUI tests run automatically on GitHub Actions for any changes to the `gui/` directory:

```
# Workflow: .github/workflows/gui-tests.yml
- API unit tests (pytest)
- Integration tests (Docker Compose)
- Frontend build verification
```

Status:

Tests include:

- Health endpoint validation
- File upload with toy network
- Layout job execution and completion
- Centrality computation
- Service health checks

API Tests

```
# Run inside API container
make bash-api
pytest ci/api-tests/ -v

# Or directly
docker compose exec api pytest ci/api-tests/ -v
```

Integration Tests

The CI workflow runs full integration tests:

```
# Upload and analyze a network
cd gui
curl -F "file=@toy_network.edgelist" http://localhost:8080/api/upload
# Run layout, centrality, and community detection jobs
```

Frontend Tests (WIP)

```
# Playwright smoke tests
make e2e
```

Manual Testing Workflow

1. **Upload** `toy_network.edgelist` via Web UI
2. **Visualize** the network (6 nodes, 14 edges, 3 layers)
3. **Analyze**:
 - Run Spring Layout
 - Compute Centrality (degree + betweenness)
 - Detect Communities (Louvain)
4. **Monitor** jobs at <http://localhost:5555> (Flower)
5. **Export** workspace bundle

6.40.10 Production Deployment

Static Build (Recommended)

For production, serve pre-built frontend assets:

```
# Modify Dockerfile.frontend to use multi-stage build
FROM node:20-alpine AS builder
WORKDIR /app
COPY frontend/ .
RUN npm install && npm run build

FROM nginx:alpine
COPY --from=builder /app/dist /usr/share/nginx/html
COPY nginx/nginx.conf /etc/nginx/nginx.conf
```

Update `docker-compose.yml`:

```
frontend:
  build:
    context: .
    dockerfile: Dockerfile.frontend.prod
  # Remove dev server volume mounts
```

Security Considerations

- Set CORS `allow_origins` to specific domains (not *)
- Add authentication/authorization middleware
- Use HTTPS (add TLS termination in Nginx)
- Validate file uploads strictly
- Set resource limits per user/job
- Enable Celery task rate limiting

Current Status: Development mode - suitable for local use only. Do not expose to public internet without proper security hardening.

6.40.11 Troubleshooting

Issue: Port already in use

```
# Find process using port 8080
lsof -ti:8080 | xargs kill -9

# Or change port in docker-compose.yml
ports:
  - "8081:80" # Use 8081 instead
```

Issue: Permission denied on /data

```
# Fix volume permissions
sudo chown -R $(whoami):$(whoami) gui/data/
```

Issue: py3plex import error in containers

```
# Verify mount
docker compose exec api ls -la /workspace

# Reinstall if needed
docker compose exec api pip install -e /workspace
```

Issue: Frontend can't reach API

Check Nginx proxy config:

```
docker compose exec nginx cat /etc/nginx/nginx.conf

# Test API directly
curl http://localhost:8000/api/health
```

6.40.12 API Documentation

Interactive docs available at:

- **Swagger UI:** <http://localhost:8080/api/docs>
- **ReDoc:** <http://localhost:8080/api/redoc>

Key Endpoints

POST	/api/upload	# Upload network file
GET	/api/graphs/{id}/summary	# Graph statistics
POST	/api/graphs/{id}/layout	# Compute layout (async)
POST	/api/graphs/{id}/analysis/centrality	# Compute centrality (async)
POST	/api/graphs/{id}/analysis/community	# Detect communities (async)
GET	/api/jobs/{id}	# Poll job status
POST	/api/workspaces/save	# Save workspace bundle

Example: Run Analysis

```
# 1. Upload
GRAPH_ID=$(curl -F "file=@toy_network.edgelist" \
  http://localhost:8080/api/upload | jq -r .graph_id)

# 2. Start layout job
JOB_ID=$(curl -X POST \
  -H "Content-Type: application/json" \
  -d '{"algorithm": "spring", "seed": 42, "dimensions": 2}' \
  http://localhost:8080/api/graphs/$GRAPH_ID/layout | jq -r .job_id)

# 3. Poll job
curl http://localhost:8080/api/jobs/$JOB_ID

# 4. Get positions
curl http://localhost:8080/api/graphs/$GRAPH_ID/positions
```

6.40.13 Contributing

This GUI is part of the [py3plex](#) project.

Guidelines:

- Keep all GUI code under `gui/`
- Do not modify `py3plex` core
- Follow existing code style (Black, Ruff for Python; ESLint for TypeScript)
- Add tests for new features
- Update documentation with new features

6.40.14 License

This GUI follows the `py3plex` repository license:

- **GUI code** (under `gui/`): BSD-3-Clause (same as `py3plex`)
- **Dependencies**: See individual package licenses

6.40.15 Acknowledgments

Built with:

- [py3plex](#) - Multilayer network analysis
- [FastAPI](#) - Modern Python web framework

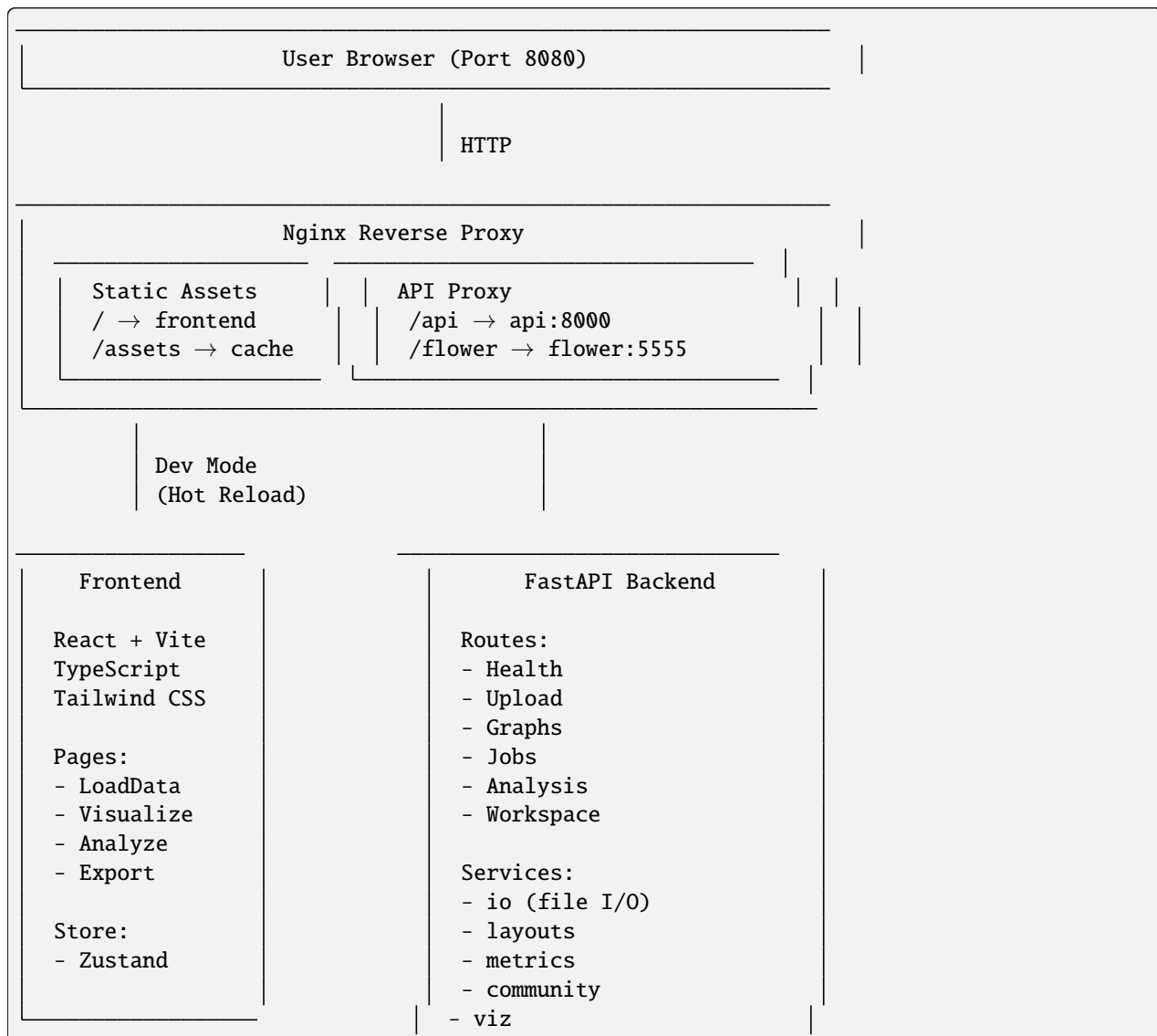
- [Celery](#) - Distributed task queue
- [React](#) - UI library
- [Vite](#) - Frontend build tool
- [Tailwind CSS](#) - Utility-first CSS

6.40.16 Support

- **Issues:** <https://github.com/SkBlaz/py3plex/issues>
- **Docs:** <https://skblaz.github.io/py3plex/>
- **py3plex Paper:** [Applied Network Science 2019](#)

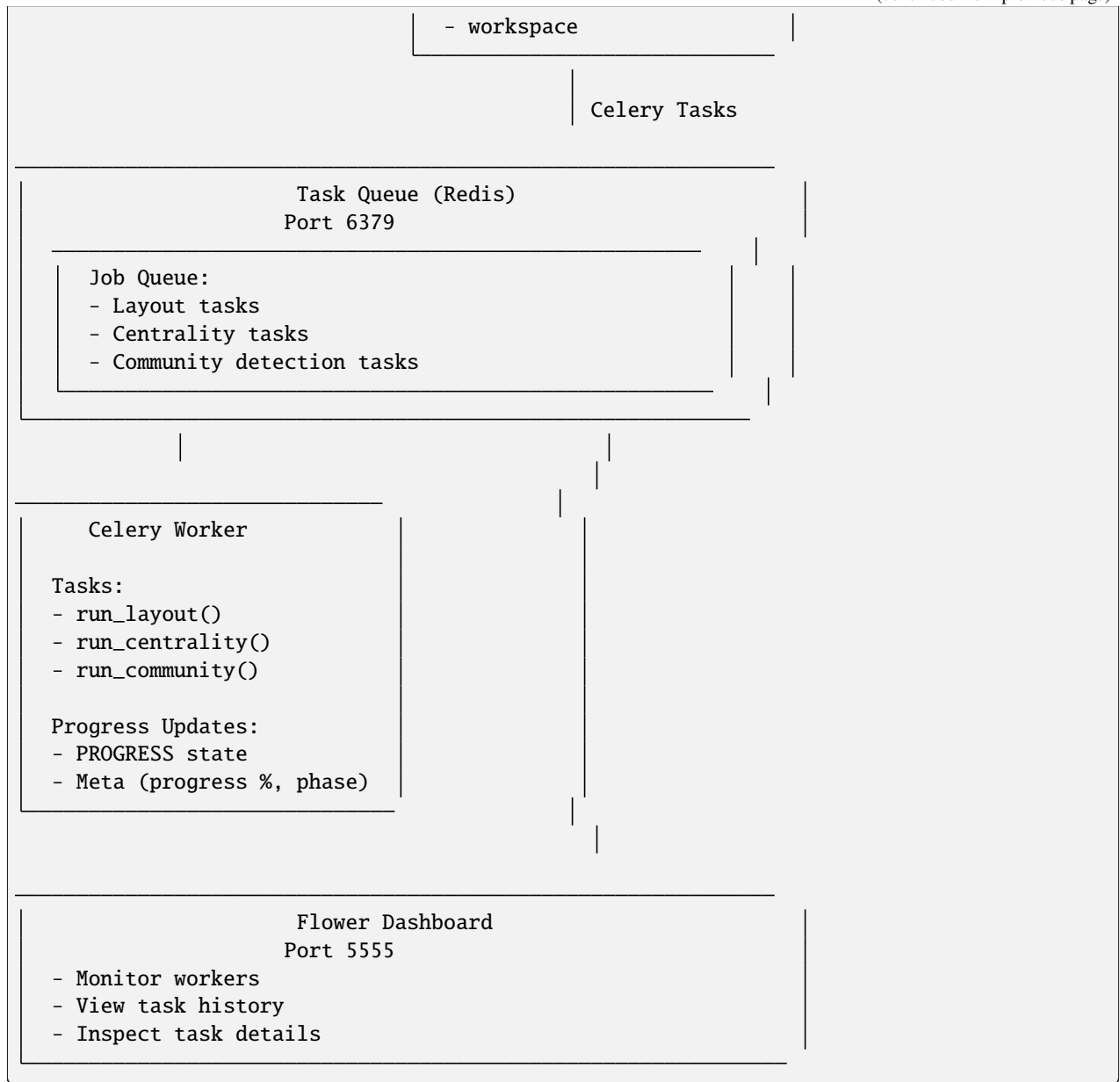
6.41 Py3plex GUI Architecture

6.41.1 System Overview



(continues on next page)

(continued from previous page)



6.41.2 Data Flow

Upload & Parse Flow

```

User → Frontend → API /upload → io.save_upload()
      ↓
      io.load_graph_from_file()
      ↓
      NetworkX graph loaded
      ↓
      Stored in GRAPH_REGISTRY
      ↓
      Return graph_id to frontend
  
```

Analysis Job Flow

```

User → Frontend → API /analysis/centrality
      ↓
      Create Celery task
      ↓
      Return job_id
      ↓
      Task queued in Redis
      ↓
      Worker picks up task
      ↓
      task.update_state(progress=30)
      ↓
      services.metrics.compute_centrality()
      ↓
      Save results to /data/artifacts/
      ↓
      task.update_state(progress=100, result={...})
      ↓
User ← Frontend ← API /jobs/{job_id} ← Redis result backend

```

6.41.3 Directory Structure

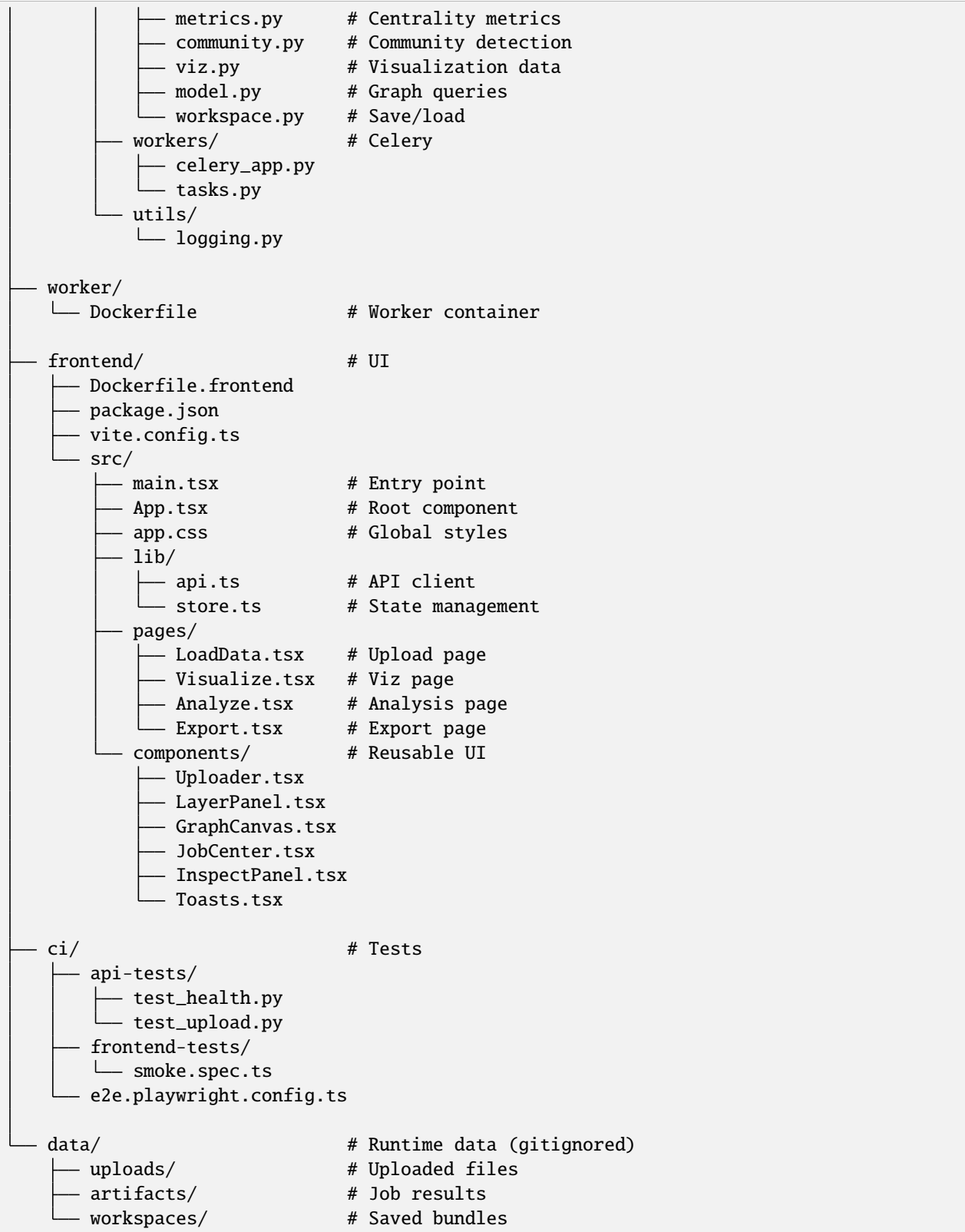
```

gui/
├── docker-compose.yml      # Orchestration
├── compose.gpu.yml         # GPU override
├── Makefile                # Dev commands
├── .env.example            # Config template
├── README                  # User guide
├── TESTING                 # Test guide
├── ARCHITECTURE            # Architecture notes
├── nginx/
│   └── nginx.conf          # Reverse proxy config
├── api/                    # Backend
│   ├── Dockerfile.api
│   ├── pyproject.toml
│   └── app/
│       ├── main.py         # FastAPI app
│       ├── deps.py         # Dependencies
│       ├── schemas.py      # Pydantic models
│       ├── routes/         # Endpoints
│       │   ├── health.py
│       │   ├── upload.py
│       │   ├── graphs.py
│       │   ├── jobs.py
│       │   ├── analysis.py
│       │   └── workspace.py
│       ├── services/       # Business logic
│       │   ├── io.py       # File I/O
│       │   └── layouts.py   # Layout algorithms

```

(continues on next page)

(continued from previous page)



6.41.4 Component Responsibilities

Visual Component Reference

The architecture components are realized in the following user interface pages:

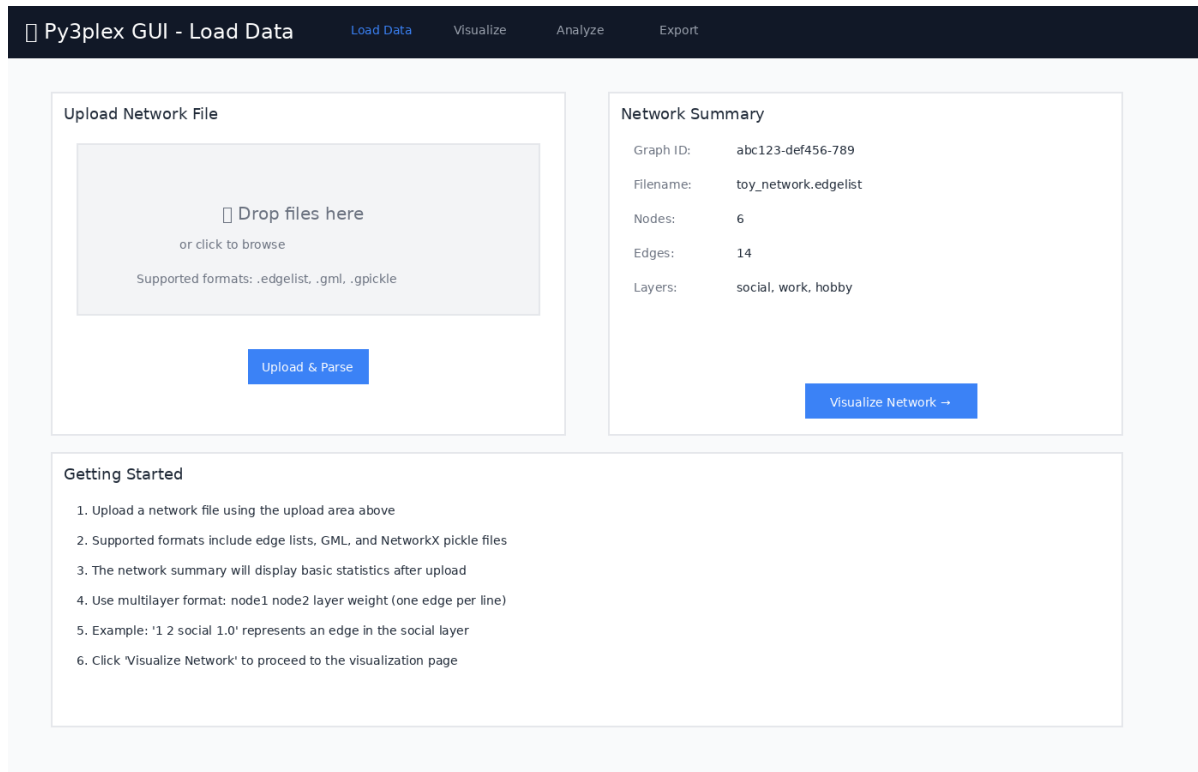


Figure 1: Frontend - Load Data page utilizing the IO service

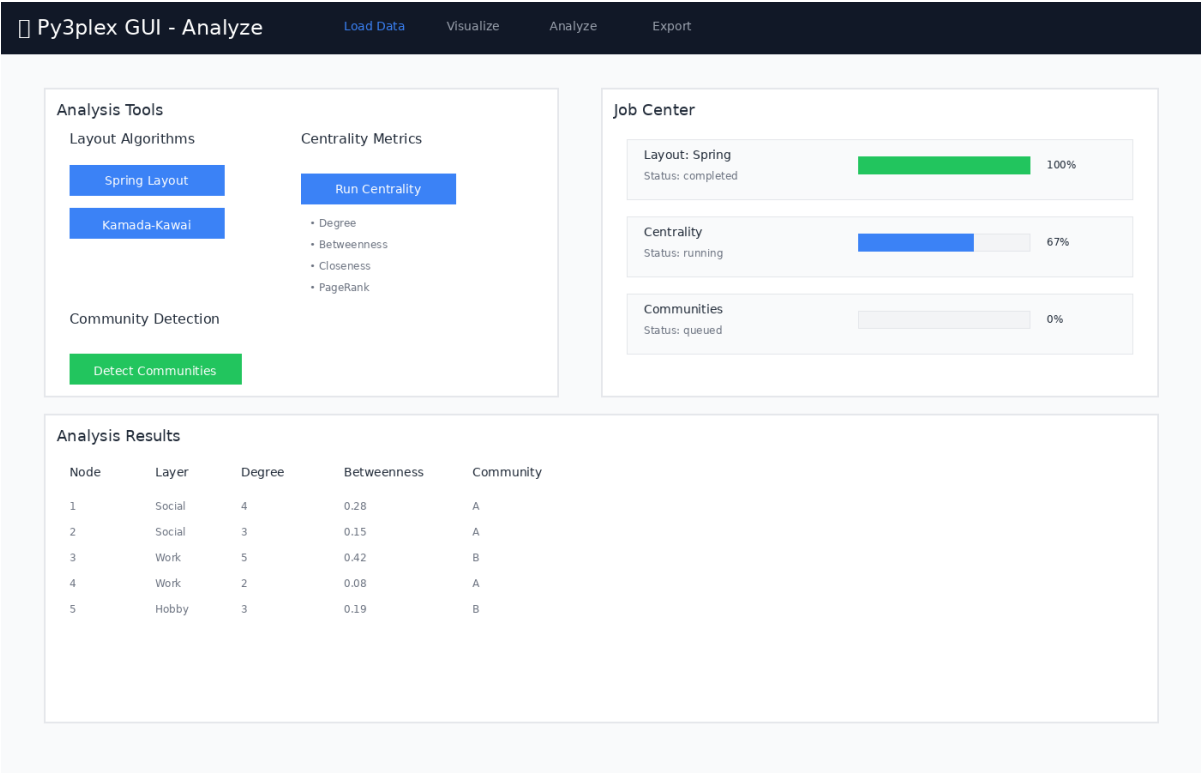


Figure 2: Job orchestration - Analysis page with Celery task execution

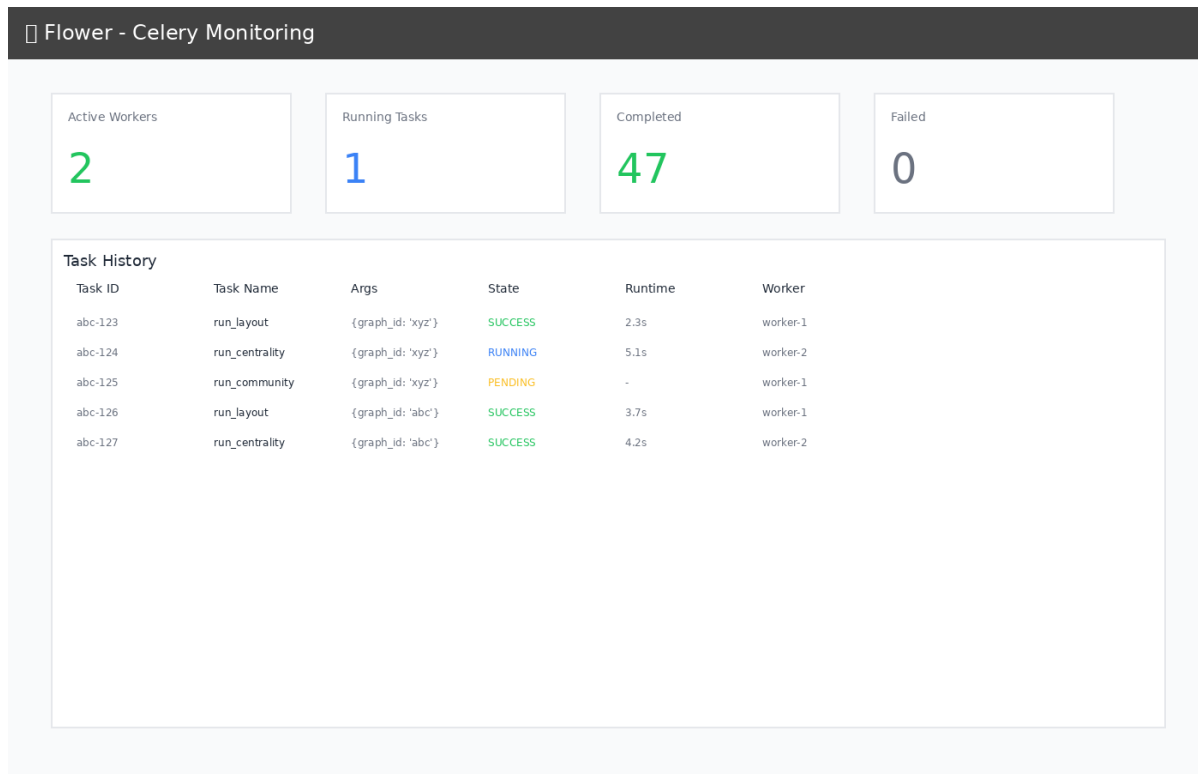


Figure 3: Worker monitoring - Flower dashboard showing task queue and execution

Frontend

Responsibilities:

- User interaction
- File upload UI
- Real-time job polling
- Graph visualization (placeholder)
- State management (Zustand)

Technologies:

- React 18 (UI framework)
- Vite (build tool, dev server)
- TypeScript (type safety)
- Tailwind CSS (styling)
- Axios (HTTP client)

API (FastAPI)

Responsibilities:

- REST API endpoints
- Request validation (Pydantic)
- File upload handling
- Job orchestration
- py3plex integration

Key Services:

- `io`: File loading, format detection
- `layouts`: Layout computation (NetworkX)
- `metrics`: Centrality calculations
- `community`: Community detection
- `workspace`: Save/load bundles

Worker (Celery)

Responsibilities:

- Async job execution
- Progress reporting
- Result persistence
- Resource management

Tasks:

- `run_layout`: Force-directed layouts
- `run_centrality`: Node/edge metrics
- `run_community`: Community detection

Redis

Responsibilities:

- Job queue (broker)
- Result backend
- Session storage (future)

Nginx

Responsibilities:

- Reverse proxy
- Static file serving
- Gzip compression
- Caching headers
- WebSocket proxy (HMR)

Flower

Responsibilities:

- Worker monitoring
- Task history
- Performance metrics

6.41.5 Data Models

Graph Registry (In-Memory)

```
GRAPH_REGISTRY = {
    "<graph_id>": {
        "graph": nx.Graph(),          # NetworkX graph
        "filepath": "/data/uploads/...", # Original file
        "positions": [NodePosition()], # Layout positions
        "metadata": {}                # Extra info
    }
}
```

Job State (Redis)

```
{
    "job_id": "uuid",
    "status": "running", # queued/running/completed/failed
    "progress": 50,      # 0-100
    "phase": "computing", # human-readable
    "result": {...}      # Output data
}
```

Workspace Bundle (Zip)

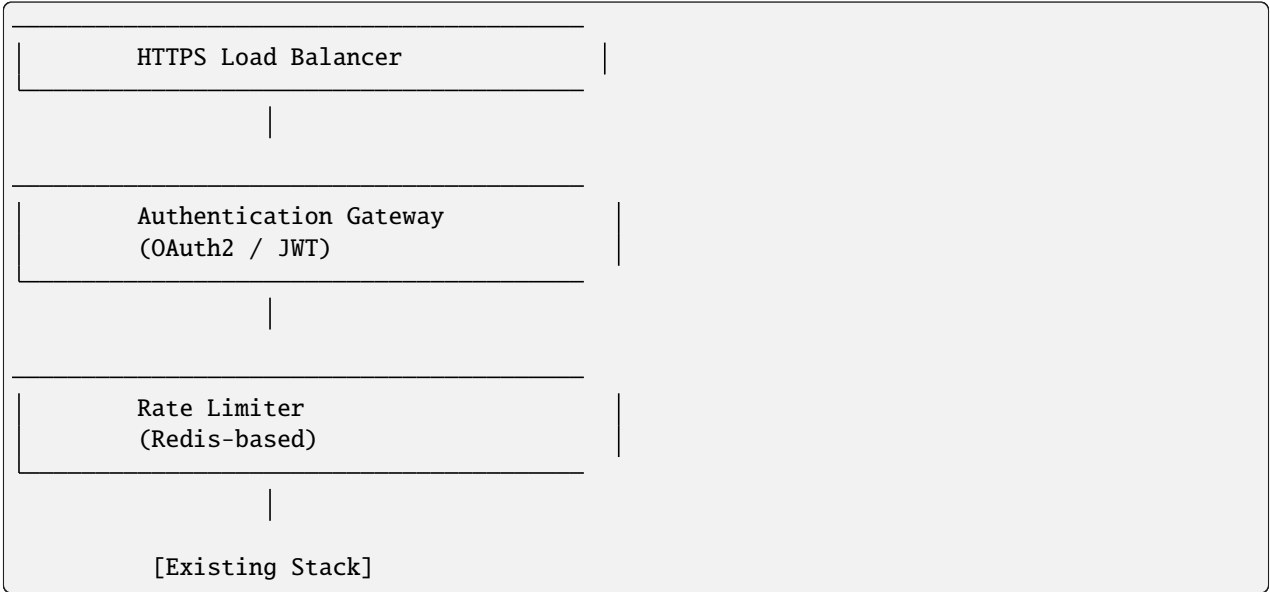
```
workspace_{uuid}.zip
├── metadata.json      # Graph ID, view state
├── network.edgelist   # Original file
├── positions.json     # Layout positions
├── artifacts/
│   ├── centrality.json
│   └── community.json
```

6.41.6 Security Architecture

Current (Development Mode)

- ✓ Read-only py3plex mount
- ✓ Isolated data directories
- CORS allows all origins
- No authentication
- No HTTPS
- No rate limiting

Production Hardening (TODO)



6.41.7 Deployment Variants

Local Development (Current)

```
make up # All containers on localhost
```

GPU-Enabled

```
docker compose -f docker-compose.yml -f compose.gpu.yml up
```

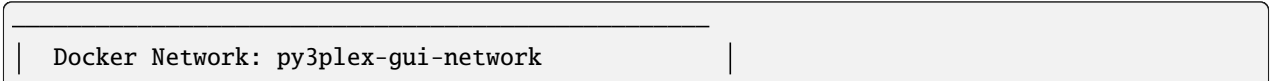
Production (Future)

```
# docker-compose.prod.yml
services:
  frontend:
    image: frontend:prod
    # Pre-built static files

  api:
    replicas: 3
    # Load balanced

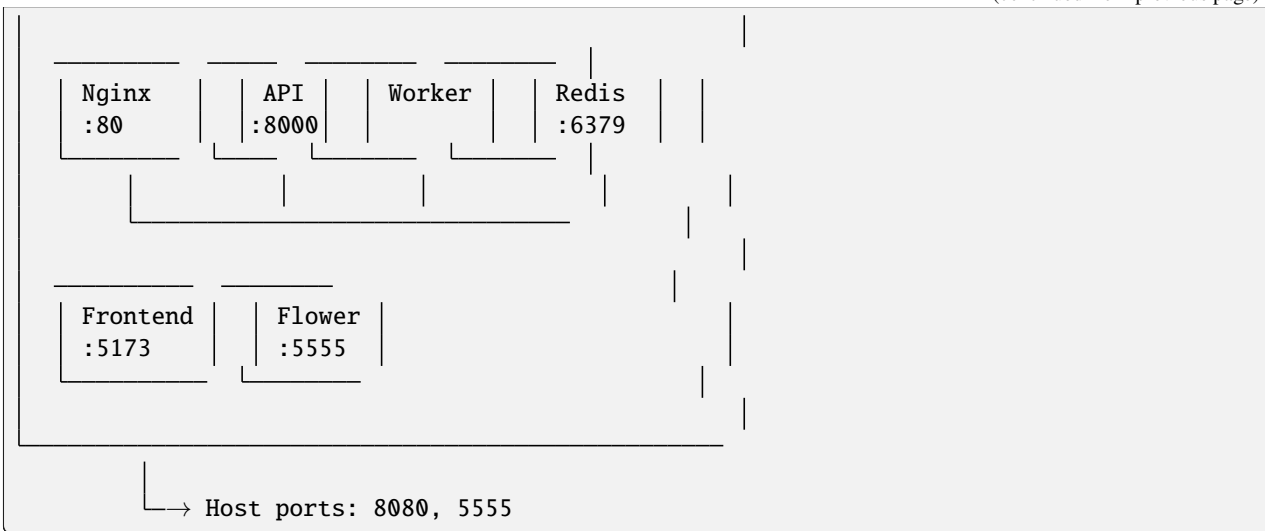
  worker:
    replicas: 5
    # Auto-scaling
```

6.41.8 Network Topology



(continues on next page)

(continued from previous page)



6.41.9 Volume Mounts

Host	→ Container
../	→ /workspace (ro)
../data/	→ /data
../api/app/	→ /app (dev mode)
../frontend/src/	→ /app/src (dev mode)

6.41.10 Environment Variables

```

# API
API_WORKERS=2           # Uvicorn workers
MAX_UPLOAD_MB=512       # Max file size
DATA_DIR=/data          # Data root

# Celery
CELERY_CONCURRENCY=2    # Worker threads
REDIS_URL=redis://redis:6379/0

# Frontend
VITE_API_URL=http://localhost:8080/api

```

6.41.11 Performance Characteristics

Small Graphs (< 100 nodes)

- Upload: < 1s
- Layout: 2-5s
- Centrality: 1-3s
- Community: 1-3s

Medium Graphs (100-1000 nodes)

- Upload: 1-3s
- Layout: 5-15s
- Centrality: 3-10s
- Community: 3-10s

Large Graphs (> 1000 nodes)

- Consider sampling for preview
- Progressive rendering recommended
- May need GPU acceleration
- Memory: ~1GB per 10k nodes

6.41.12 Future Enhancements

Phase 2

- WebGL visualization
- Real-time collaboration
- Database backend (PostgreSQL)
- Authentication service
- CI/CD pipeline

Phase 3

- GraphQL API
- Plugin system
- Custom algorithms
- Cloud deployment
- Multi-tenancy

Version: 0.1.0

Last Updated: 2025-11-09

6.42 GUI Testing Guide

Complete testing guide for the Py3plex GUI, including automated CI tests and manual validation.

6.42.1 Visual Interface Preview

The Py3plex GUI consists of several key pages that work together for network analysis:

Py3plex GUI - Load Data

Load DataVisualizeAnalyzeExport

Upload Network File

Drop files here

or click to browse

Supported formats: .edgelist, .gml, .gpickle

Upload & Parse

Network Summary

Graph ID:abc123-def456-789

Filename:toy_network.edgelist

Nodes:6

Edges:14

Layers:social, work, hobby

Visualize Network →

Getting Started

1. Upload a network file using the upload area above

2. Supported formats include edge lists, GML, and NetworkX pickle files

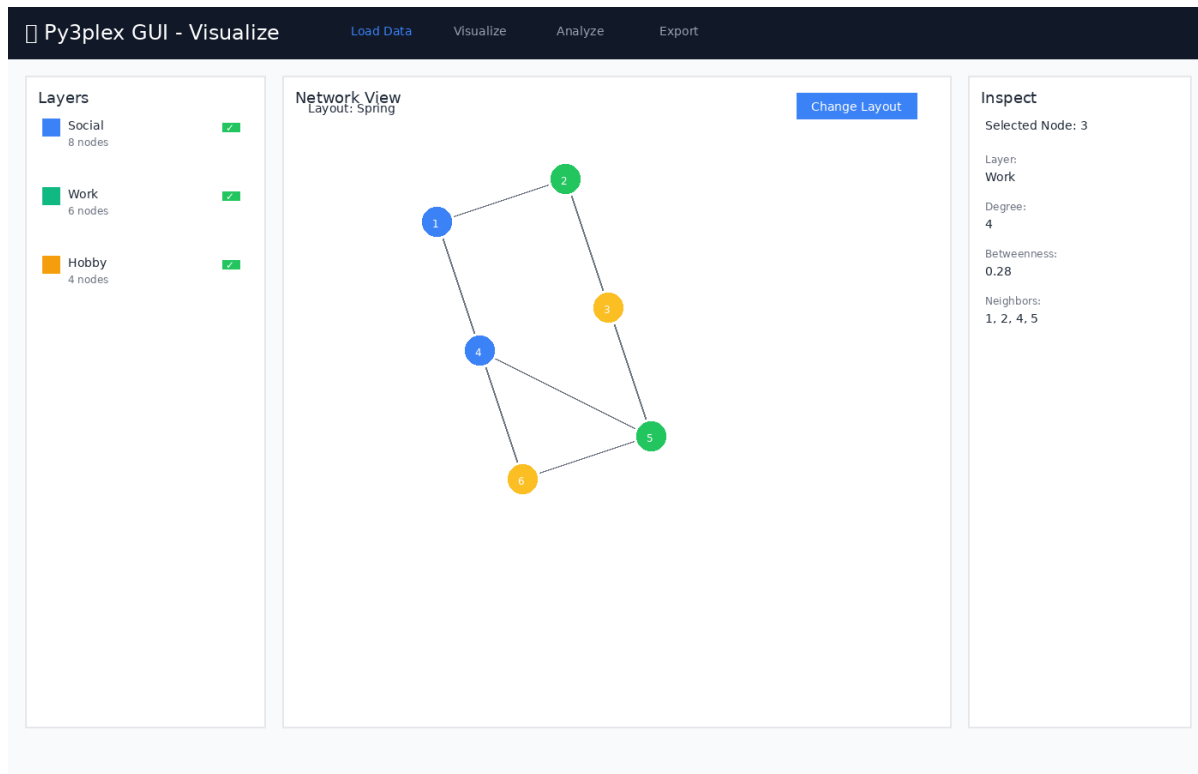
3. The network summary will display basic statistics after upload

4. Use multilayer format: node1 node2 layer weight (one edge per line)

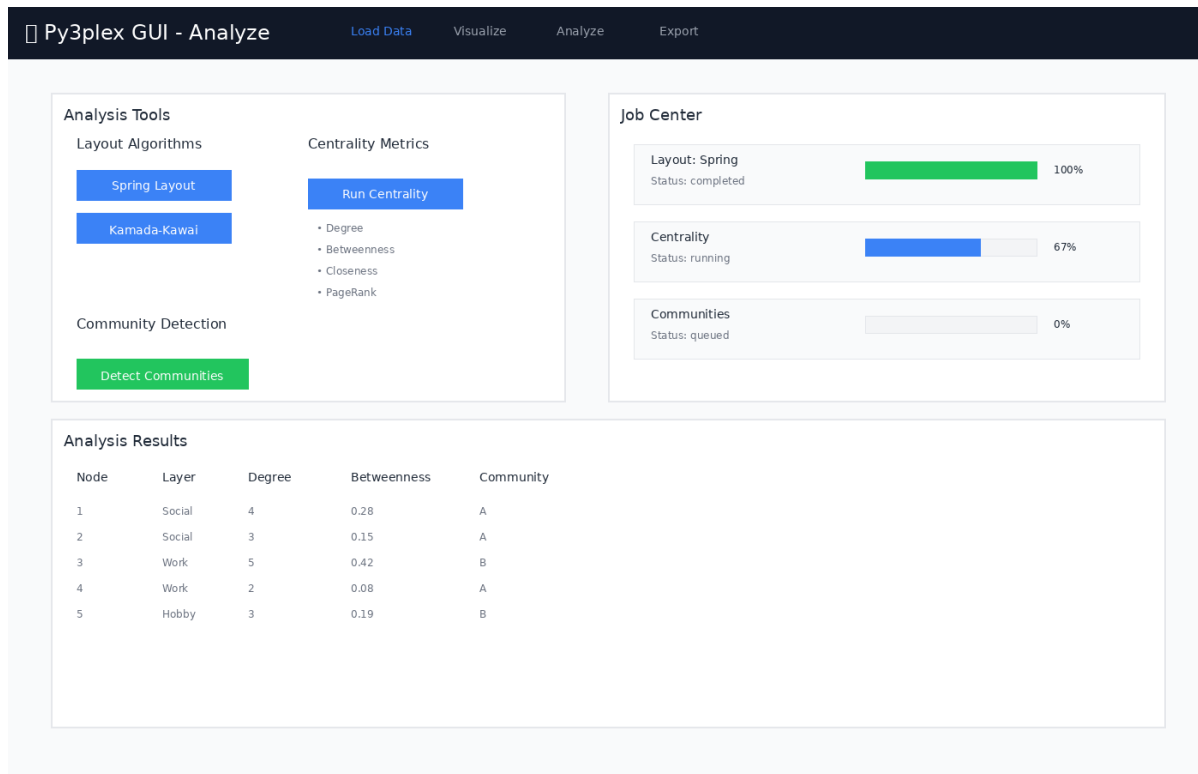
5. Example: '1 2 social 1.0' represents an edge in the social layer

6. Click 'Visualize Network' to proceed to the visualization page

Load Data Page - Upload and parse network files



Visualize Page - Interactive network view with layer controls



Analyze Page - Run jobs and monitor progress

6.42.2 Automated Testing (CI)

GitHub Actions Workflow

All GUI tests run automatically on GitHub Actions:

Workflow: `.github/workflows/gui-tests.yml`

Triggers:

- Push to main/master/develop branches (if gui/ files changed)
- Pull requests targeting main/master/develop (if gui/ files changed)
- Manual workflow dispatch

Test Jobs:

1. API Tests (~5 min)

- Build API, worker, redis containers
- Run pytest suite: `ci/api-tests/`
- Validate health endpoint
- Test file upload functionality

2. Integration Tests (~10 min)

- Build all services (including nginx, frontend)
- Test full upload → analyze → export flow
- Validate layout job execution
- Test centrality computation
- Verify service health

3. Frontend Build (~5 min)

- Type check TypeScript
- Build production bundle
- Verify dist output

View Results: Check the Actions tab on GitHub or the CI badges in the repository README

Running CI Tests Locally

You can run the same tests locally:

```
cd gui

# API tests
docker compose build api worker redis
docker compose up -d api worker redis
docker compose exec api pytest ci/api-tests/ -v
docker compose down -v

# Integration tests
docker compose up -d --build
# Wait for services
curl -f http://localhost:8080/api/health
# Run manual integration tests (see below)
docker compose down -v

# Frontend build
cd frontend
npm ci
npm run build
```

6.42.3 Manual Testing Guide

Manual testing guide for the Py3plex GUI. Follow these steps to validate the implementation.

Prerequisites

- Docker and Docker Compose installed
- At least 4GB RAM available
- Ports 8080, 5555, 8000, 6379 available

Setup

```
cd gui
cp .env.example .env
make up
```

Expected: All containers start successfully. Check with `docker compose ps`.

Test 1: Health Check

API Health

```
curl http://localhost:8080/api/health
```

Expected Response:

```
{"status": "ok", "version": "0.1.0"}
```

Frontend Access

Open browser to `http://localhost:8080`

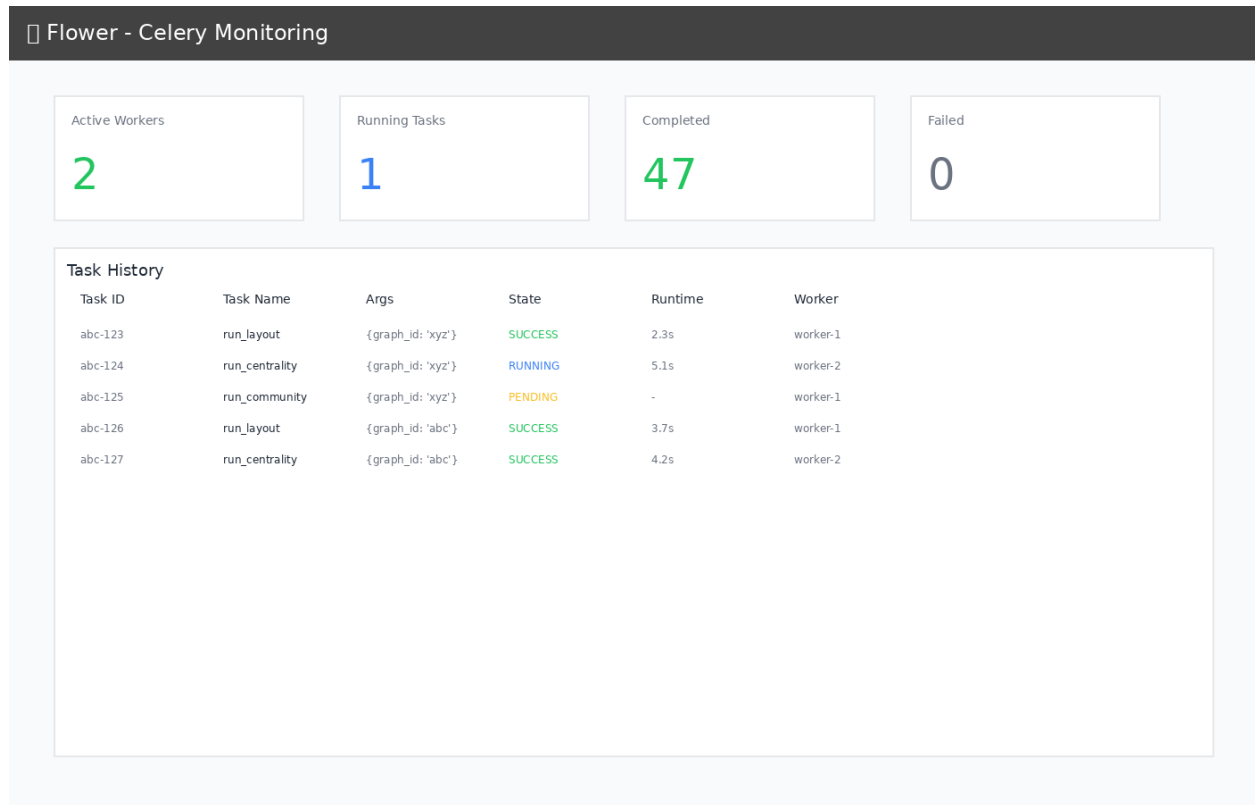
Expected: React app loads with navigation bar showing:

- Load Data
- Visualize
- Analyze
- Export

Flower Dashboard

Open browser to `http://localhost:5555`

Expected: Celery Flower dashboard loads showing workers.



Flower monitoring dashboard showing task history and worker status

Test 2: Upload Network

1. Navigate to **Load Data** page
2. Click “Click to upload” or drag and drop `toy_network.edgelist`
3. Click “Upload & Parse”

Expected:

- Upload progress indicator appears
- Network Summary displays:
 - Graph ID (UUID)
 - Filename: `toy_network.edgelist`
 - Nodes: 6
 - Edges: 14
 - Layers: hobby, social, work

Test 3: Visualize Network

1. Click “Visualize Network →” or navigate to **Visualize** page

Expected:

- Page shows three panels: Layers, Network View, Inspect
- Network View shows placeholder with “Graph with X nodes”
- No errors in console

Test 4: Run Layout Job

1. Navigate to **Analyze** page
2. Click “Run Spring Layout” button

Expected:

- Job appears in Job Center below
- Status shows “queued” → “running” → “completed”
- Progress updates (10% → 30% → 80% → 100%)
- Job shows green checkmark when complete

Verify in Flower

1. Open `http://localhost:5555`
2. Check “Tasks” tab

Expected: Layout task appears with status SUCCESS

Test 5: Run Centrality Analysis

1. On **Analyze** page, click “Run Centrality” button

Expected:

- New job appears in Job Center
- Type: “Centrality”
- Status progresses to “completed”
- No errors

Test 6: Run Community Detection

1. On **Analyze** page, click “Detect Communities” button

Expected:

- New job appears in Job Center
- Type: “Community Detection”
- Status progresses to “completed”
- Result includes number of communities found

Test 7: Export Workspace

1. Navigate to **Export** page
2. Enter workspace name: “test-workspace”
3. Click “Save Workspace Bundle”

Expected:

- Success message: “Workspace saved as test-workspace_{uuid}.zip”
- Workspace ID displayed

Verify File Created

```
ls -lh gui/data/workspaces/
```

Expected: Zip file exists with recent timestamp

Test 8: API Direct Testing

Upload via API

```
curl -F "file=@toy_network.edgelist" \
http://localhost:8080/api/upload
```

Expected: JSON response with graph_id

Get Graph Summary

```
GRAPH_ID=<from_previous_step>
curl http://localhost:8080/api/graphs/$GRAPH_ID/summary
```

Expected: JSON with nodes, edges, layers

Start Layout Job

```
curl -X POST \
-H "Content-Type: application/json" \
-d '{"algorithm":"spring","seed":42,"dimensions":2}' \
http://localhost:8080/api/graphs/$GRAPH_ID/layout
```

Expected: JSON response with job_id

Check Job Status

```
JOB_ID=<from_previous_step>
curl http://localhost:8080/api/jobs/$JOB_ID
```

Expected: JSON with status, progress, result

Test 9: Logs Inspection

```
# View all logs
make logs

# Or individual services
docker compose logs api
docker compose logs worker
docker compose logs frontend
```

Expected: No error messages, only INFO level logs

Test 10: Container Health

```
docker compose ps
```

Expected: All services show “healthy” or “running”

Test 11: Data Persistence

1. Upload a network
2. Run a layout job
3. Stop containers: `make down`
4. Restart: `make up`
5. Upload same network again

Expected:

- Uploads work after restart
- Previous artifacts still in `gui/data/`

Test 12: Multiple Jobs

1. Navigate to **Analyze** page
2. Quickly click:
 - Run Spring Layout
 - Run Centrality
 - Detect Communities

Expected:

- All three jobs appear in Job Center
- Jobs process in parallel (check Flower)
- All complete successfully

6.42.4 Common Issues

Port Already in Use

```
# Find and kill process
lsof -ti:8080 | xargs kill -9

# Or change port in docker-compose.yml
ports:
  - "8081:80"
```

Container Won't Start

```
# Check logs
docker compose logs <service>

# Rebuild
make down && make build && make up
```

Permission Errors

```
# Fix data directory permissions
sudo chown -R $(whoami):$(whoami) gui/data/
```

Frontend Can't Reach API

Check nginx config:

```
docker compose exec nginx cat /etc/nginx/nginx.conf
```

Test API directly:

```
curl http://localhost:8000/api/health
```

6.42.5 Cleanup

```
# Stop and remove everything
make down

# Full cleanup including data
make clean
```

6.42.6 Test Checklist

- [] Health endpoints respond
- [] Frontend loads
- [] Flower dashboard accessible
- [] File upload works
- [] Graph summary displays
- [] Layout job completes
- [] Centrality job completes
- [] Community detection works

- ☐ Workspace save succeeds
- ☐ Job progress updates in real-time
- ☐ Multiple concurrent jobs work
- ☐ Logs show no errors
- ☐ Data persists across restarts

6.42.7 Performance Benchmarks

For reference, on a typical development machine:

- **Startup time:** 30-60 seconds (first build: 3-5 minutes)
- **Upload (toy network):** < 1 second
- **Layout job:** 2-5 seconds
- **Centrality job:** 1-3 seconds
- **Community detection:** 1-3 seconds

Larger networks (1000+ nodes) may take proportionally longer.

6.42.8 Security Testing (Development Mode)

WARNING: Do NOT expose to public internet without:

- ☐ Changing CORS settings
- ☐ Adding authentication
- ☐ Enabling HTTPS
- ☐ Setting rate limits
- ☐ Validating file uploads strictly

6.42.9 Next Steps

If all tests pass:

1. Try with real network datasets
2. Test with large graphs (1000+ nodes)
3. Load test with concurrent users
4. Profile memory usage
5. Test edge cases (malformed files, very large files)

—

Last Updated: 2025-11-09

Version: 0.1.0

6.43 Random Network Generation

Generate **synthetic multilayer networks** for testing and benchmarking.

```

from py3plex.core import random_generators

# Generate Erdős-Rényi multilayer network
network = random_generators.random_multilayer_ER(
    num_nodes=200,
    num_layers=6,
    probability=0.09,
    directed=True)

network.visualize_network(show=True, no_labels=True)

```

6.43.1 Examples

See: `example_random_generator.py`

Repository: <https://github.com/SkBlaz/Py3Plex/tree/master/examples>

6.44 Label Propagation

Semi-supervised learning on networks using label propagation.

```

from py3plex.core import multinet
from py3plex.algorithms.network_classification import validate_label_propagation

network = multinet.multi_layer_network().load_network(
    "../datasets/cora.mat", directed=False, input_type="sparse")

# Run label propagation with different normalization schemes
results = validate_label_propagation(
    network.core_network,
    network.labels,
    dataset_name="cora",
    repetitions=5,
    normalization_scheme="freq")

```

6.44.1 Examples

See: `example_label_propagation.py`

Repository: <https://github.com/SkBlaz/Py3Plex/tree/master/examples>

6.45 Node Embeddings

Generate node embeddings using Node2Vec and visualize them.

Note

Node2Vec binary is no longer bundled with py3plex.

Options:

- Use pure Python alternatives: `pip install node2vec` or `pip install pecanpy`

- Download C++ binary from: <https://github.com/snap-stanford/snap>
- Use py3plex's built-in random walk implementation

```
from py3plex.core import multinet
from py3plex.wrappers import train_node2vec_embedding
from py3plex.visualization.embedding_visualization import embedding_visualization

# Load network
network = multinet.multi_layer_network().load_network(
    "../datasets/imdb_gml.gml", directed=True, input_type="gml")

# Save as edgelist for Node2Vec
network.save_network("../datasets/test.edgelist")

try:
    # Generate embedding (assumes node2vec binary is in PATH)
    train_node2vec_embedding.call_node2vec_binary(
        "../datasets/test.edgelist",
        "../datasets/test_embedding.emb",
        binary="node2vec",
        weighted=False)

    # Load and visualize
    network.load_embedding("../datasets/test_embedding.emb")
    embedding_visualization.visualize_embedding(network)
except FileNotFoundError:
    print("Node2Vec binary not found. Install with: pip install node2vec")
```

6.45.1 Examples

See:

- `example_n2v_embedding.py` - Node2Vec embeddings
- `example_embedding_visualization.py` - Embedding visualization
- `example_embedding_construction.py` - Custom embeddings

Repository: <https://github.com/SkBlaz/Py3Plex/tree/master/examples>

6.46 API Documentation

This section contains the complete API documentation for py3plex, automatically generated from docstrings.

For auto-generated module documentation, run:

```
cd docfiles
sphinx-apidoc -o AUTOGEN_results -f ../py3plex
make html
```


6.46.1 Core Modules

```
py3plex.core.multinet.ensure(*args, **kwargs)
```

```
py3plex.core.multinet.invariant(*args, **kwargs)
```

```
class py3plex.core.multinet.multi_layer_network(verbose: bool = True, network_type: str =
                                                'multilayer', directed: bool = True, dummy_layer: str
                                                = 'null', label_delimiter: str = '---', coupling_weight:
                                                int | float = 1)
```

Bases: object

Main class for multilayer network analysis and manipulation.

This class provides a comprehensive toolkit for creating, analyzing, and visualizing multilayer networks where nodes can exist in multiple layers and edges can connect nodes within or across layers.

Supported Network Types:

- **multilayer**: General multilayer networks with arbitrary layer structure
- **multiplex**: Special case where all layers share the same nodes, with automatic coupling edges between corresponding nodes across layers

Key Features:

- Dict-based API for adding nodes and edges (see `add_nodes()` and `add_edges()`)
- NetworkX interoperability via `to_networkx()` and `from_networkx()`
- Multiple I/O formats (edgelist, GML, GraphML, gpickle, etc.)
- Visualization methods for multilayer layouts
- Community detection and centrality analysis
- Random walk and embedding generation

Hypergraph Support:

This class does NOT natively support true hypergraphs (edges connecting more than two nodes). For hypergraph-like structures, consider: - Using bipartite projections (nodes and hyperedges as separate node types) - The incidence gadget encoding via `to_homogeneous_hypergraph()` - External hypergraph libraries with conversion utilities

Notes

- Nodes in multilayer networks are represented as (node_id, layer) tuples
- Use `add_nodes()` and `add_edges()` with dict format for easiest interaction
- See examples/ directory for usage patterns and best practices

Examples

```
>>> # Create a basic multilayer network
>>> net = multi_layer_network(network_type='multilayer', directed=False)
>>>
>>> # Add nodes to different layers
>>> net.add_nodes([
...     {'source': 'A', 'type': 'social'},
...     {'source': 'B', 'type': 'social'},
```

(continues on next page)

(continued from previous page)

```

...     {'source': 'A', 'type': 'email'} # Same node, different layer
... ])
>>>
>>> # Add edges (intra-layer and inter-layer)
>>> net.add_edges([
...     {'source': 'A', 'target': 'B',
...      'source_type': 'social', 'target_type': 'social'},
...     {'source': 'A', 'target': 'A',
...      'source_type': 'social', 'target_type': 'email'}
... ])
>>>
>>> print(net) # Shows network statistics
<multi_layer_network: type=multilayer, directed=False, nodes=3, edges=2, layers=2>

```

add_dummy_layers()

Internal function, for conversion between objects

add_edges(*edge_dict_list: List[Dict] | List[List] | Tuple, input_type: str = 'dict')* → None

Add edges to the multilayer network.

This method supports multiple input formats for specifying edges between nodes in different layers. The most common format is dict-based.

Parameters

- **edge_dict_list** – Edge data in one of the supported formats (see below)
- **input_type** – Format of edge data ('dict', 'list', or 'px_edge')

Supported Formats:

Dict format (recommended): `python {`

```

    'source': 'node1', # Source node ID 'target': 'node2', # Target node ID 'source_type':
    'layer1', # Source layer name 'target_type': 'layer2', # Target layer name (can be same
    as source) 'weight': 1.0, # Optional: edge weight 'type': 'interaction' # Optional: edge
    type/label

```

List format: `[node1, layer1, node2, layer2]`

px_edge format: `((node1, layer1), (node2, layer2), {'weight': 1.0})`

Examples

```

>>> # Add single intra-layer edge
>>> net = multi_layer_network()
>>> net.add_edges([
...     {'source': 'A',
...      'target': 'B',
...      'source_type': 'protein',
...      'target_type': 'protein'}
... ])

```

```

>>> # Add inter-layer edge with weight
>>> net.add_edges([
...     {'source': 'gene1',

```

(continues on next page)

(continued from previous page)

```
...     'target': 'protein1',
...     'source_type': 'genes',
...     'target_type': 'proteins',
...     'weight': 0.95,
...     'type': 'expression'
... })
```

```
>>> # Add multiple edges at once
>>> edges = [
...     {'source': 'A', 'target': 'B', 'source_type': 'layer1', 'target_type':
...     ↪ 'layer1'},
...     {'source': 'B', 'target': 'C', 'source_type': 'layer1', 'target_type':
...     ↪ 'layer1'}
... ]
>>> net.add_edges(edges)
```

Raises**Exception** – If input_type is not one of 'dict', 'list', or 'px_edge'**Notes**

- For intra-layer edges, use the same layer for source_type and target_type
- For inter-layer edges, use different layers
- Edge weights default to 1.0 if not specified

add_nodes(node_dict_list: List[Dict] | Dict, input_type: str = 'dict') → None

Add nodes to the multilayer network.

Nodes in a multilayer network are identified by both their ID and the layer they belong to. This method adds nodes using a dict-based format.

Parameters

- **node_dict_list** – Node data as a dict or list of dicts (see format below)
- **input_type** – Format of node data (currently only 'dict' is supported)

Dict Format:

```
python {
    'source': 'node_id', # Node identifier (can be string or number)
    'type': 'layer_name', # Layer this node belongs to
    'weight': 1.0, # Optional: node weight/importance
    'label': 'display' # Optional: display label
    # ... any other node attributes
}
```

Examples

```
>>> # Add single node
>>> net = multi_layer_network()
>>> net.add_nodes([{'source': 'A', 'type': 'layer1'}])
```

```
>>> # Add multiple nodes to the same layer
>>> nodes = [
```

(continues on next page)

(continued from previous page)

```
...     {'source': 'A', 'type': 'protein'},
...     {'source': 'B', 'type': 'protein'},
...     {'source': 'C', 'type': 'protein'}
... ]
>>> net.add_nodes(nodes)
```

```
>>> # Add nodes with attributes
>>> net.add_nodes([
...     {'source': 'gene1',
...      'type': 'genes',
...      'weight': 0.8,
...      'label': 'BRCA1',
...      'chromosome': '17'}
... ])
```

```
>>> # Add nodes to multiple layers
>>> multi_layer_nodes = [
...     {'source': 'entity1', 'type': 'layer1'},
...     {'source': 'entity1', 'type': 'layer2'}, # Same entity, different layer
...     {'source': 'entity2', 'type': 'layer1'}
... ]
>>> net.add_nodes(multi_layer_nodes)
```

Notes

- The same node ID can exist in multiple layers
- Each (node_id, layer) combination is treated as a unique node
- Additional attributes beyond ‘source’ and ‘type’ are preserved
- Nodes must be added before edges referencing them

aggregate_edges(metric='count', normalize_by='degree')

Edge aggregation method

Count weights across layers and return a weighted network

Parameters

- **param1** – aggregation operator (count is default)
- **param2** – normalization of the values

Returns

A simplified network.

basic_stats(target_network=None)

A method for obtaining a network’s statistics.

Displays: - Basic network info (nodes, edges) - Total unique nodes (counting each (node, layer) as unique)
- Unique node IDs (across all layers) - Per-layer node counts

compute_ollivier_ricci(mode: str = 'core', layers: List[Any] | None = None, alpha: float = 0.5, weight_attr: str = 'weight', curvature_attr: str = 'ricciCurvature', verbose: str = 'ERROR', backend_kwargs: Dict[str, Any] | None = None, inplace: bool = True, interlayer_weight: float = 1.0) → Dict[str, Any]

Compute Ollivier-Ricci curvature on the multilayer network.

This method provides flexible computation of Ollivier-Ricci curvature at different levels of the multilayer network:

- **core mode:** Compute curvature on the aggregated (flattened) network
- **layers mode:** Compute curvature separately for each layer
- **supra mode:** Compute curvature on the full supra-graph including both intra-layer and inter-layer edges

Parameters

- **mode** – Scope of computation. Options: “core”, “layers”, “supra”.
- **layers** – List of layer identifiers to process (only for mode=“layers”). If None, all layers are processed.
- **alpha** – Ollivier-Ricci parameter in [0, 1] controlling the mass distribution. Default: 0.5.
- **weight_attr** – Name of edge attribute containing weights. Default: “weight”.
- **curvature_attr** – Name of edge attribute to store curvature values. Default: “ricciCurvature”.
- **verbose** – Verbosity level. Options: “INFO”, “DEBUG”, “ERROR”. Default: “ERROR”.
- **backend_kwargs** – Additional keyword arguments for OllivierRicci constructor.
- **inplace** – If True, update internal graphs. If False, return new graphs without modifying the network. Default: True.
- **interlayer_weight** – Weight for inter-layer coupling edges (only for mode=“supra”). Default: 1.0.

Returns

Dictionary mapping scope identifiers to NetworkX graphs with computed curvatures: - mode=“core”: {“core”: graph_with_curvature} - mode=“layers”: {layer_id: graph_with_curvature, ...} - mode=“supra”: {“supra”: supra_graph_with_curvature}

Raises

- **RicciBackendNotAvailable** – If GraphRicciCurvature is not installed.
- **ValueError** – If mode is invalid or layers contains invalid identifiers.

Examples

```
>>> from py3plex.core import multinet
>>> net = multinet.multi_layer_network()
>>> net.add_edges([
...     ['A', 'layer1', 'B', 'layer1', 1],
...     ['B', 'layer1', 'C', 'layer1', 1],
... ], input_type="list")
>>>
>>> # Compute on aggregated network
>>> result = net.compute_ollivier_ricci(mode="core")
>>>
>>> # Compute per layer
>>> result = net.compute_ollivier_ricci(mode="layers")
```

(continues on next page)

(continued from previous page)

```
>>>
>>> # Compute on supra-graph
>>> result = net.compute_ollivier_ricci(mode="supra", inplace=False)
```

compute_ollivier_ricci_flow(*mode*: str = 'core', *layers*: List[Any] | None = None, *alpha*: float = 0.5, *iterations*: int = 10, *method*: str = 'OTD', *weight_attr*: str = 'weight', *curvature_attr*: str = 'ricciCurvature', *verbose*: str = 'ERROR', *backend_kwargs*: Dict[str, Any] | None = None, *inplace*: bool = True, *interlayer_weight*: float = 1.0) → Dict[str, Any]

Compute Ollivier-Ricci flow on the multilayer network.

Ricci flow iteratively adjusts edge weights based on their Ricci curvature, effectively revealing and enhancing community structure. After Ricci flow, edges with negative curvature (community boundaries) have reduced weights, while edges with positive curvature have increased weights.

Parameters

- **mode** – Scope of computation. Options: “core”, “layers”, “supra”.
- **layers** – List of layer identifiers to process (only for mode=“layers”). If None, all layers are processed.
- **alpha** – Ollivier-Ricci parameter in [0, 1]. Default: 0.5.
- **iterations** – Number of Ricci flow iterations. Default: 10.
- **method** – Ricci flow method. Options: “OTD” (Optimal Transport Distance, recommended), “ATD” (Average Transport Distance). Default: “OTD”.
- **weight_attr** – Name of edge attribute containing weights. After Ricci flow, these weights are updated to reflect the flow metric.
- **curvature_attr** – Name of edge attribute for curvature values. Default: “ricciCurvature”.
- **verbose** – Verbosity level. Options: “INFO”, “DEBUG”, “ERROR”. Default: “ERROR”.
- **backend_kwargs** – Additional keyword arguments for OllivierRicci constructor.
- **inplace** – If True, update internal graphs. If False, return new graphs. Default: True.
- **interlayer_weight** – Weight for inter-layer coupling edges (only for mode=“supra”). Default: 1.0.

Returns

Dictionary mapping scope identifiers to NetworkX graphs with Ricci flow applied: - mode=“core”: {“core”: graph_with_flow} - mode=“layers”: {layer_id: graph_with_flow, ...} - mode=“supra”: {“supra”: supra_graph_with_flow}

Raises

- **RicciBackendNotAvailable** – If GraphRicciCurvature is not installed.
- **ValueError** – If mode is invalid or layers contains invalid identifiers.

Examples

```
>>> from py3plex.core import multinet
>>> net = multinet.multi_layer_network()
>>> net.add_edges([
```

(continues on next page)

(continued from previous page)

```

...     ['A', 'layer1', 'B', 'layer1', 1],
...     ['B', 'layer1', 'C', 'layer1', 1],
... ], input_type="list")
>>>
>>> # Apply Ricci flow to aggregated network
>>> result = net.compute_ollivier_ricci_flow(mode="core", iterations=20)
>>>
>>> # Apply to each layer
>>> result = net.compute_ollivier_ricci_flow(mode="layers", iterations=10)

```

edges_from_temporal_table(*edge_df*)

Convert a temporal edge DataFrame to edge tuple list.

Extracts edges from a pandas DataFrame with temporal/activity information and converts them to a list of edge tuples suitable for network construction.

Parameters

edge_df – pandas DataFrame with columns: - node_first: Source node identifier - node_second: Target node identifier - layer_name: Layer identifier

Returns**List of edge tuples in format:**

(node_first, node_second, layer_first, layer_second, weight) where weight is always 1

Return type

list

Notes

- All values are converted to strings
- All edges are assigned weight=1
- Both source and target are assumed to be in the same layer

Examples

```

>>> import pandas as pd
>>> net = multi_layer_network()
>>> df = pd.DataFrame({
...     'node_first': ['A', 'B'],
...     'node_second': ['B', 'C'],
...     'layer_name': ['L1', 'L1']
... })
>>> result = net.edges_from_temporal_table(df)
>>> len(result) >= 2
True

```

See also

fill_tmp_with_edges: Add these edges to layer graphs

fill_tmp_with_edges(*edge_df*)

Fill temporary layer graphs with edges from a DataFrame.

Populates the emptied layer graphs (created by `remove_layer_edges`) with edges from a temporal/activity DataFrame. Useful for temporal network analysis where edge sets change over time.

Parameters

edge_df – pandas DataFrame with columns: - `node_first`: Source node identifier - `node_second`: Target node identifier - `layer_name`: Layer identifier

Notes

- Requires `remove_layer_edges()` to be called first
- Edges are grouped by layer
- Modifies `self.tmp_layers` in place
- Each edge is stored as `((node_first, layer), (node_second, layer))`

Raises

AttributeError – If `self.tmp_layers` doesn't exist (call `remove_layer_edges` first)

Examples

These examples require proper network setup and are for illustration only.

```
>>> import pandas as pd
>>> net = multi_layer_network()
>>> net.split_to_layers()
>>> net.remove_layer_edges()
>>> df = pd.DataFrame({
...     'node_first': ['A', 'B'],
...     'node_second': ['B', 'C'],
...     'layer_name': ['L1', 'L1']
... })
>>> net.fill_tmp_with_edges(df)
```

See also

`remove_layer_edges`: Creates empty layer graphs `edges_from_temporal_table`: Convert DataFrame to edge list

from_homogeneous_hypergraph(*H*)

Decode a homogeneous graph created by `to_homogeneous_hypergraph`.

This method reconstructs a multiplex network from its incidence gadget encoding. It identifies edge-nodes by their degree and cycle structure, then reconstructs the original layers based on cycle lengths (prime numbers).

Parameters

H (*networkx.Graph*) – Homogeneous graph created by `to_homogeneous_hypergraph`).

Returns

- *dict* – Dictionary mapping layer names to lists of edges: `{layer: [(u, v), ...]}`
- *Examples* – Example requires proper network setup - for illustration only.


```

>>> network = multi_layer_network()
>>> network.add_layer("A")
>>> network.add_nodes([("1", "A"), ("2", "A")])
>>> network.add_edges([(("1", "A"), ("2", "A"))])
>>> H, node_map, edge_info = network.to_homogeneous_hypergraph()
>>> recovered = network.from_homogeneous_hypergraph(H)
>>> print(recovered)
{'layer_with_prime_2': [('1', '2')]}

```

Notes

The decoded layer names indicate the prime number used for encoding: - “layer_with_prime_2” corresponds to the first layer - “layer_with_prime_3” corresponds to the second layer, etc.

classmethod `from_networkx`(*G*: *Graph*, *network_type*: *str* = 'multilayer', *directed*: *bool* | *None* = *None*)
→ *multi_layer_network*

Create a `multi_layer_network` from a `NetworkX` graph.

This class method converts a `NetworkX` graph into a `py3plex multi_layer_network`. For multilayer networks, nodes should be tuples of (node_id, layer).

Parameters

- **G** – `NetworkX` graph to convert
- **network_type** – Type of network ('multilayer' or 'multiplex')
- **directed** – Whether to treat the network as directed. If `None`, inferred from `G`.

Returns

A new `multi_layer_network` instance

Return type

multi_layer_network

Examples

```

>>> import networkx as nx
>>> G = nx.Graph()
>>> G.add_nodes_from([('A', 'layer1'), ('B', 'layer1')])
>>> G.add_edge(('A', 'layer1'), ('B', 'layer1'))
>>> net = multi_layer_network.from_networkx(G)
>>> print(net)
<multi_layer_network: type=multilayer, directed=False, nodes=2, edges=1,
↳ layers=1>

```

Notes

- For proper multilayer behavior, ensure nodes are (node_id, layer) tuples
- Edge attributes are preserved during conversion
- The input graph is copied, not referenced

get_decomposition(*heuristic*='all', *cycle*=*None*, *parallel*=*False*, *alpha*=1, *beta*=0)

Core method for obtaining a network’s decomposition in terms of relations

get_decomposition_cycles(*cycle=None*)

A supporting method for obtaining decomposition triplets

get_degrees()

A simple wrapper which computes node degrees.

get_edges(*data: bool = False, multiplex_edges: bool = False*) → Any

A method for obtaining a network's edges

Parameters

- **data** – If True, return edge data along with edge tuples
- **multiplex_edges** – If True, include coupling edges in multiplex networks

Yields

Edge tuples, optionally with data

Raises

Exception – If network type is not specified

get_label_matrix()

Return network labels

get_layers(*style='diagonal', compute_layouts='force', layout_parameters=None, verbose=True*)

A method for obtaining layerwise distributions

get_neighbors(*node_id: str, layer_id: str | None = None*) → Any

Get neighbors of a node in a specific layer.

Parameters

- **node_id** – Node identifier
- **layer_id** – Layer identifier (optional)

Returns

Iterator of neighbor nodes

get_nodes(*data: bool = False*) → Any

A method for obtaining a network's nodes

Parameters

data – If True, return node data along with node identifiers

Yields

Node identifiers, optionally with data

get_nx_object()

Return only core network with proper annotations

get_supra_adjacency_matrix(*mtype='sparse'*)

Get sparse representation of the supra matrix.

Parameters

mtype – 'sparse' or 'dense' - matrix representation type

Returns

Supra-adjacency matrix in requested format

Warning

For large multilayer networks, dense matrices can consume significant memory $(N \cdot L)^2 \cdot 8$ bytes for float64.

get_tensor(*sparsity_type='bsr'*)

TODO

get_unique_entity_counts()

Count unique entities in the network.

Returns

(**total_unique_nodes**, **unique_node_ids**, **nodes_per_layer**)

- **total_unique_nodes**: count of unique (node, layer) tuples
- **unique_node_ids**: count of unique node IDs (across all layers)
- **nodes_per_layer**: dict mapping layer to count of nodes in that layer

Return type

tuple

invert(*override_core=False*)

invert the nodes to edges. Get the “edge graph”. Each node is here an edge.

load_embedding(*embedding_file*)

Embedding loading method

load_layer_name_mapping(*mapping_name, header=False*)

Layer-node mapping loader method

Parameters

param1 – The name of the mapping file.

Returns

self.layer_name_map is filled, returns nothing.

load_network(*input_file: str | None = None, directed: bool = False, input_type: str = 'gml', label_delimiter: str = '---'*) → *multi_layer_network*

Main network loader

This method loads and prepares a given network.

Parameters

- **input_file** – Path to the network file to load
- **directed** – Whether the network is directed
- **input_type** – Format of the input file ('gml', 'graphml', 'edgelist', 'gpickle', etc.)
- **label_delimiter** – Delimiter used to separate layer names in node labels

Returns

Self for method chaining. Populates self.core_network, self.labels, and self.activity

load_network_activity(*activity_file*)

Network activity loader

Args:

param1: The name of the generic activity file -> 65432 61888 1377688175 RE

, n1 n2 timestamp and layer name. Note that layer node mappings MUST be loaded in order to map nodes to activity properly.

Returns:

self.activity is filled.

load_temporal_edge_information(*input_file=None, input_type='edge_activity', directed=False, layer_mapping=None*)

A method for loading temporal edge information

merge_with(*target_px_object*)

Merge two px objects.

monitor(*message*)

A simple monitor method for logging

monoplex_nx_wrapper(*method, kwargs=None*)

A generic networkx function wrapper.

Parameters

- **method** (*str*) – Name of the NetworkX function to call (e.g., ‘degree centrality’, ‘betweenness centrality’)
- **kwargs** (*dict, optional*) – Keyword arguments to pass to the NetworkX function. For example, for betweenness centrality you can pass: - weight: Edge attribute to use as weight - normalized: Whether to normalize betweenness values - distance: Edge attribute to use as distance (for closeness centrality)

Returns

The result of the NetworkX function call.

Raises

AttributeError – If the specified method does not exist in NetworkX.

Example

```
# Unweighted betweenness centrality centralities = network.monoplex_nx_wrapper("betweenness centrality")
```

```
# Weighted betweenness centrality centralities = network.monoplex_nx_wrapper("betweenness centrality",  
kwargs={"weight": "weight"})
```

```
# With multiple parameters centralities = network.monoplex_nx_wrapper("betweenness centrality",  
kwargs={"weight": "weight", "normalized": True})
```

read_ground_truth_communities(*cfile*)

Parse ground truth community file and make mappings to the original nodes. This works based on node ID mappings, exact node,layer tuples are to be added. :param param1: ground truth communities.

Returns

self.ground_truth_communities

remove_edges(*edge_dict_list: List[Dict] | List[List], input_type: str = 'list'*) → None

A method for removing edges..

Parameters

- **edge_dict_list** – Edge data in dict or list format

- **input_type** – Format of edge data ('dict' or 'list')

Raises

Exception – If input_type is not valid

remove_layer_edges()

Remove all edges from separate layer graphs while keeping nodes.

This method creates empty copies of each layer graph with all nodes intact but no edges. Useful for reconstructing networks with different edge sets or for temporal network analysis.

Notes

- Requires split_to_layers() to be called first
- Stores empty layer graphs in self.tmp_layers
- Original graphs in self.separate_layers remain unchanged
- All nodes and their attributes are preserved

Raises

RuntimeError – If split_to_layers() hasn't been called yet

See also

split_to_layers: Must be called before this method fill_tmp_with_edges: Add edges back to emptied layers

remove_nodes(node_dict_list, input_type='dict')

Remove nodes from the network

save_network(output_file=None, output_type='edgelist')

Save the network to a file in various formats.

This method exports the multilayer network to different file formats for persistence, sharing, or use with other tools.

Parameters

- **output_file** – Path where the network should be saved
- **output_type** – Format for saving ('edgelist', 'multiedgelist', 'multiedgelist_encoded', or 'gpickle')

Supported Formats:

- 'edgelist': Simple edge list format (standard NetworkX)
- 'multiedgelist': Multilayer edge list with layer information
- 'multiedgelist_encoded': Multilayer edge list with integer encoding
- 'gpickle': Python pickle format (preserves all attributes)

Examples

```
>>> net = multi_layer_network()
>>> net.add_nodes([{'source': 'A', 'type': 'layer1'}])
>>> net.add_edges([{'source': 'A', 'target': 'B',
...                  'source_type': 'layer1', 'target_type': 'layer1'}])
>>> net.save_network('network.txt', output_type='multiedgelist')
```

```
>>> # For faster I/O with all metadata preserved
>>> net.save_network('network.gpickle', output_type='gpickle')
```

Notes

- ‘gpickle’ format preserves all node/edge attributes
- ‘multiedgelist_encoded’ creates node_map and layer_map attributes
- Edge weights and types are preserved in supported formats

serialize_to_edgelist(*edgelist_file*='./tmp/tmpedgelist.txt', *tmp_folder*='tmp', *out_folder*='out', *multiplex*=False)

Serialize the multilayer network to an edgelist file.

Converts the network to a numeric edgelist format suitable for external tools and algorithms that require integer node/layer identifiers.

Parameters

- **edgelist_file** – Path to output edgelist file (default: “./tmp/tmpedgelist.txt”)
- **tmp_folder** – Temporary folder for intermediate files (default: “tmp”)
- **out_folder** – Output folder for results (default: “out”)
- **multiplex** – If True, use multiplex format (node layer node layer weight) If False, use simple edgelist format (node1 node2 weight)

Returns

Inverse node mapping (numeric_id -> original_node_tuple)

Use this to decode results from external algorithms

Return type

dict

File Formats:

- Multiplex format: node1_id layer1_id node2_id layer2_id weight
- Simple format: node1_id node2_id weight

Notes

- Creates tmp_folder and out_folder if they don’t exist
- Nodes are mapped to sequential integers starting from 0
- Layers are mapped to sequential integers starting from 0 (multiplex mode)
- All edges have weight 1 unless explicitly specified

Examples

Example requires file output - for illustration only.

```
>>> net = multi_layer_network()
>>> # ... build network ...
>>> node_mapping = net.serialize_to_edgelist(
...     edgelist_file='network.txt',
...     multiplex=True
... )
>>> # Use node_mapping to decode results
>>> original_node = node_mapping[0] # Get original node for id 0
```

See also

load_network: Load networks from file save_network: Alternative serialization method

sparse_to_px(directed=None)

Convert sparse matrix to py3plex format

Parameters

directed – Whether the network is directed (uses self.directed if None)

split_to_layers(style='diagonal', compute_layouts='force', layout_parameters=None, verbose=True, multiplex=False, convert_to_simple=False)

A method for obtaining layerwise distributions

subnetwork(input_list=None, subset_by='node_layer_names')

Construct a subgraph based on a set of nodes.

summary()

Generate a summary of network statistics.

Computes and returns key metrics about the multilayer network structure.

Returns

Network statistics including:

- 'Number of layers': Count of unique layers
- 'Nodes': Total number of nodes
- 'Edges': Total number of edges
- 'Mean degree': Average node degree
- 'CC': Number of connected components

Return type

dict

Examples

```
>>> net = multi_layer_network()
>>> net.add_nodes([{'source': 'A', 'type': 'layer1'}])
>>> net.add_edges([{'source': 'A', 'target': 'B',
...                 'source_type': 'layer1', 'target_type': 'layer1'}])
```

(continues on next page)

(continued from previous page)

```
>>> stats = net.summary()
>>> print(f"Network has {stats['Nodes']} nodes and {stats['Edges']} edges")
Network has 2 nodes and 1 edges
```

Notes

- Connected components are computed on the undirected version
- Mean degree is averaged across all nodes in all layers

test_scale_free()

Test the scale-free-ness of the network

to_homogeneous_hypergraph()

Transform a multiplex network into a homogeneous graph using incidence gadget encoding.

This method encodes the multiplex structure where each layer is represented by a unique prime number signature. Each edge becomes an edge-node connected to its endpoints and a cycle of length prime-1 that encodes the layer.

Returns

- *tuple* (*H*, *node_mapping*, *edge_info*) –

H

[networkx.Graph] Homogeneous unlabeled graph encoding the multiplex structure.

node_mapping

[dict] Maps each original node to its vertex-node in H.

edge_info

[dict] Mapping from each edge-node in H to its (layer, endpoints) tuple.

- *Examples* – Example requires sympy dependency - for illustration only.
- `>>> network = multi_layer_network(directed=False) # doctest (+SKIP)`
- `>>> network.add_nodes([{'source': '1', 'type': 'A'}, {'source': '2', 'type': 'A'}], input_type='dict') # doctest: +SKIP`
- `>>> network.add_edges([{'source': '1', 'target': '2', 'source_type': 'A', 'target_type': 'A'}], input_type='dict') # doctest: +SKIP`
- `>>> H, node_map, edge_info = network.to_homogeneous_hypergraph() # doctest (+SKIP)`
- `>>> print(f"Homogeneous graph has {len(H.nodes())} nodes") # doctest (+SKIP)`

Notes

This transformation uses prime-based signatures to encode layers: - Each layer is assigned a unique prime number (2, 3, 5, 7, ...) - Each edge in layer with prime *p* is connected to a cycle of length *p* - The cycle structure uniquely identifies the layer

to_json()

A method for exporting the graph to a json file

Args: self

to_networkx() → Graph

Convert the multilayer network to a NetworkX graph.

Returns a copy of the core network as a NetworkX graph. The returned graph preserves all node and edge attributes, including layer information for multilayer networks (where nodes are typically (node_id, layer) tuples).

Returns

A NetworkX graph (MultiGraph or MultiDiGraph depending on network type)

Return type

nx.Graph

Examples

```
>>> net = multi_layer_network(directed=False)
>>> net.add_nodes([{'source': 'A', 'type': 'layer1'}])
>>> nx_graph = net.to_networkx()
>>> print(type(nx_graph))
<class 'networkx.classes.multigraph.MultiGraph'>
```

Notes

- For multilayer networks, nodes are tuples: (node_id, layer)
- All edge attributes (weight, type, etc.) are preserved
- The returned graph is a copy, not a reference

to_sparse_matrix(*replace_core=False, return_only=False*)

Conver the matrix to scipy-sparse version. This is useful for classification.

visualize_matrix(*kwargs=None*)

Plot the matrix – this plots the supra-adjacency matrix

visualize_network(*style='diagonal', parameters_layers=None, parameters_multiedges=None, show=False, compute_layouts='force', layouts_parameters=None, verbose=True, orientation='upper', resolution=0.01, axis=None, fig=None, no_labels=False, linewidth=1.7, alphachannel=0.3, linepoints='.', legend=False*)

Visualize the multilayer network.

Supports multiple visualization styles: - ‘diagonal’: Layer-centric diagonal layout with inter-layer edges - ‘hairball’: Aggregate hairball plot of all layers - ‘flow’ or ‘alluvial’: Layered flow visualization with horizontal bands - ‘sankey’: Sankey diagram showing inter-layer flow strength

Parameters

- **style** – Visualization style (‘diagonal’, ‘hairball’, ‘flow’, ‘alluvial’, or ‘sankey’)
- **parameters_layers** – Custom parameters for layer drawing
- **parameters_multiedges** – Custom parameters for edge drawing
- **show** – Show plot immediately
- **compute_layouts** – Layout algorithm (currently unused)
- **layouts_parameters** – Layout parameters (currently unused)
- **verbose** – Enable verbose output

- **orientation** – Edge orientation for diagonal style
- **resolution** – Resolution for edge curves
- **axis** – Optional matplotlib axis to draw on
- **fig** – Optional matplotlib figure (currently unused)
- **no_labels** – Hide network labels
- **linewidth** – Width of edge lines
- **alphachannel** – Alpha channel for edge transparency
- **linepoints** – Line style for edges
- **legend** – Show legend (for hairball style)

Returns

Matplotlib axis object

Raises

Exception – If style is not recognized

Performance Notes:

For large networks (>500 nodes), visualization performance may degrade: - Layout computation can be slow ($O(n^2)$ for force-directed layouts) - Rendering many edges is memory and CPU intensive - Consider filtering or sampling for exploratory visualization - Use simpler layouts or increase layout iteration limits

Approximate rendering times on typical hardware: - 100 nodes: <1 second - 500 nodes: 5-10 seconds - 1000 nodes: 30-60 seconds - 5000+ nodes: Several minutes, may run out of memory

visualize_ricci_core(*alpha: float = 0.5, iterations: int = 10, layout_type: str = 'mds', dim: int = 2, **kwargs*)

Visualize the aggregated core network using Ricci-flow-based layout.

This method is a high-level wrapper for Ricci-flow-based visualization of the core (aggregated) network. It automatically computes Ricci flow if not already done and creates an informative layout that emphasizes geometric structure and communities.

Parameters

- **alpha** – Ollivier-Ricci parameter for flow computation. Default: 0.5.
- **iterations** – Number of Ricci flow iterations. Default: 10.
- **layout_type** – Layout algorithm (“mds”, “spring”, “spectral”). Default: “mds”.
- **dim** – Dimensionality of layout (2 or 3). Default: 2.
- ****kwargs** – Additional arguments passed to `visualize_multilayer_ricci_core`.

Returns

Tuple of (figure, axes, positions_dict).

Raises

RicciBackendNotAvailable – If GraphRicciCurvature is not installed.

Examples

Example requires GraphRicciCurvature library - for illustration only.

```
>>> from py3plex.core import multinet
>>> net = multinet.multi_layer_network()
>>> net.add_edges([
...     ['A', 'layer1', 'B', 'layer1', 1],
...     ['B', 'layer1', 'C', 'layer1', 1],
... ], input_type="list")
>>> fig, ax, pos = net.visualize_ricci_core()
>>> import matplotlib.pyplot as plt
>>> plt.show()
```

See also

`visualize_ricci_layers`: Per-layer visualization with Ricci flow `visualize_ricci_supra`: Supra-graph visualization with Ricci flow

visualize_ricci_layers(*layers*: List[Any] | None = None, *alpha*: float = 0.5, *iterations*: int = 10, *layout_type*: str = 'mds', *share_layout*: bool = True, ***kwargs*)

Visualize individual layers using Ricci-flow-based layouts.

This method creates visualizations of individual layers with layouts derived from Ricci flow. Layers can share a common coordinate system (for easier comparison) or have independent layouts.

Parameters

- **layers** – List of layer identifiers to visualize. If None, uses all layers.
- **alpha** – Ollivier-Ricci parameter. Default: 0.5.
- **iterations** – Number of Ricci flow iterations. Default: 10.
- **layout_type** – Layout algorithm. Default: “mds”.
- **share_layout** – If True, use shared coordinates across layers. Default: True.
- ****kwargs** – Additional arguments passed to `visualize_multilayer_ricci_layers`.

Returns

Tuple of (figure, layer_positions_dict).

Raises

RicciBackendNotAvailable – If GraphRicciCurvature is not installed.

Examples

```
>>> fig, pos_dict = net.visualize_ricci_layers(
...     arrangement="grid", share_layout=True
... )
>>> import matplotlib.pyplot as plt
>>> plt.show()
```

See also

visualize_ricci_core: Core network visualization with Ricci flow visualize_ricci_supra: Supra-graph visualization with Ricci flow

visualize_ricci_supra(*alpha: float = 0.5, iterations: int = 10, layout_type: str = 'mds', dim: int = 2, **kwargs*)

Visualize the full supra-graph using Ricci-flow-based layout.

This method visualizes the complete multilayer structure including both intra-layer edges (within layers) and inter-layer edges (coupling between layers) using a layout derived from Ricci flow.

Parameters

- **alpha** – Ollivier-Ricci parameter. Default: 0.5.
- **iterations** – Number of Ricci flow iterations. Default: 10.
- **layout_type** – Layout algorithm. Default: “mds”.
- **dim** – Dimensionality (2 or 3). Default: 2.
- ****kwargs** – Additional arguments passed to visualize_multilayer_ricci_supra.

Returns

Tuple of (figure, axes, positions_dict).

Raises

RicciBackendNotAvailable – If GraphRicciCurvature is not installed.

Examples

```
>>> fig, ax, pos = net.visualize_ricci_supra(dim=3)
>>> import matplotlib.pyplot as plt
>>> plt.show()
```

See also

visualize_ricci_core: Core network visualization with Ricci flow visualize_ricci_layers: Per-layer visualization with Ricci flow

py3plex.core.multinet.**require**(*args, **kwargs)

py3plex.core.parsers.**ensure**(*args, **kwargs)

py3plex.core.parsers.**load_edge_activity_file**(fname: str, layer_mapping: str | None = None) → DataFrame

py3plex.core.parsers.**load_edge_activity_raw**(activity_file: str, layer_mappings: dict) → DataFrame

Basic parser for loading generic activity files. Here, temporal edges are given as tuples -> this can be easily transformed for example into a pandas dataframe!

Parameters

- **activity_file** – Path to activity file
- **layer_mappings** – Dictionary mapping layer names to IDs

Returns

DataFrame with edge activity data

`py3plex.core.parsers.load_temporal_edge_information(input_network: str, input_type: str, layer_mapping: str | None = None) → DataFrame | None`

`py3plex.core.parsers.parse_detangler_json(file_path: str, directed: bool = False) → Tuple[MultiGraph | MultiDiGraph, None]`

Parser for generic Detangler files :param file_path: Path to Detangler JSON file :param directed: Whether to create directed graph

`py3plex.core.parsers.parse_edgelist_multi_types(input_name: str, directed: bool) → Tuple[MultiGraph | MultiDiGraph, None]`

Parse an edgelist file with multiple edge types.

Reads a text file where each line represents an edge, optionally with weights and edge types. Lines starting with '#' are treated as comments.

File Format:

node1 node2 [weight] [edge_type]

Lines starting with '#' are ignored (comments)

Parameters

- **input_name** – Path to edgelist file
- **directed** – Whether to create a directed graph

Returns

(parsed_graph, None for labels)

Return type

Tuple[Union[nx.MultiGraph, nx.MultiDiGraph], None]

Notes

- All nodes are assigned to a “null” layer
- Default weight is 1 if not specified
- Edge type is optional (4th column)
- Handles both 2-column (node pairs) and 3+ column formats

Examples

```
>>> # File content:
>>> # A B 1.0 friendship
>>> # B C 2.0 collaboration
>>> graph, _ = parse_edgelist_multi_types('edges.txt', directed=False)
```

`py3plex.core.parsers.parse_embedding(input_name: str) → Tuple[ndarray, ndarray]`

Loader for generic embedding as outputted by GenSim

Parameters

input_name – Path to embedding file

Returns

Tuple of (embedding matrix, embedding indices)

`py3plex.core.parsers.parse_gml(file_name: str, directed: bool) → Tuple[MultiGraph | MultiDiGraph, None]`

Parse a gml network.

Parameters

- **file_name** – Path to GML file
- **directed** – Whether to create directed graph

Returns

Tuple of (multigraph, possible labels)

Contracts:

- Precondition: file_name must be a non-empty string
- Postcondition: result graph is not None
- Postcondition: result is a MultiGraph or MultiDiGraph

`py3plex.core.parsers.parse_gpickle(file_name: str, directed: bool = False, layer_separator: str | None = None) → Tuple[MultiGraph | MultiDiGraph, None]`

A parser for generic Gpickle as stored by Py3plex.

Parameters

- **file_name** – Path to gpickle file
- **directed** – Whether to create directed graph
- **layer_separator** – Optional separator for layer parsing

Contracts:

- Precondition: file_name must be a non-empty string
- Postcondition: result graph is not None
- Postcondition: result is a MultiGraph or MultiDiGraph

`py3plex.core.parsers.parse_gpickle_biomine(file_name: str, directed: bool) → Tuple[MultiGraph | MultiDiGraph, None]`

Gpickle parser for biomine graphs :param file_name: Path to gpickle containing BioMine data :param directed: Whether to create directed graph

`py3plex.core.parsers.parse_matrix(file_name: str, directed: bool) → Tuple[Any, Any]`

Parser for matrices.

Parameters

- **file_name** – Path to .mat file
- **directed** – Whether the graph is directed

Returns

Tuple of (network, group) from the .mat file

Contracts:

- Precondition: file_name must be a non-empty string
- Postcondition: network must not be None

`py3plex.core.parsers.parse_matrix_to_nx(file_name: str, directed: bool) → Graph | DiGraph`

Parser for matrices to NetworkX graph.

Parameters

- **file_name** – Path to .mat file
- **directed** – Whether to create directed graph

Returns

NetworkX Graph or DiGraph

`py3plex.core.parsers.parse_multi_edgelist(input_name: str, directed: bool) → Tuple[MultiGraph | MultiDiGraph, None]`

A generic multiedgelist parser n l n l w :param input_name: Path to text file containing multiedges :param directed: Whether to create directed graph

`py3plex.core.parsers.parse_multiedge_tuple_list(network: list, directed: bool) → Tuple[MultiGraph | MultiDiGraph, None]`

Parse a list of edge tuples into a multilayer network.

Parameters

- **network** – List of edge tuples (node_first, node_second, layer_first, layer_second, weight)
- **directed** – Whether to create directed graph

`py3plex.core.parsers.parse_multiplex_edges(input_name: str, directed: bool) → Tuple[MultiGraph | MultiDiGraph, None]`

Parse a multiplex edgelist file where each line specifies layer and edge.

File Format:

layer node1 node2 [weight]

Each line: layer_id source_node target_node [optional_weight]

Parameters

- **input_name** – Path to multiplex edgelist file
- **directed** – Whether to create a directed graph

Returns

(parsed_graph, None for labels)

Return type

Tuple[Union[nx.MultiGraph, nx.MultiDiGraph], None]

Notes

- Each edge belongs to a specific layer (first column)
- Nodes are represented as (node_id, layer) tuples
- Default weight is 1 if not specified
- All edges have type='default' attribute
- Automatically couples nodes across layers for multiplex structure

Examples

```
>>> # File content:
>>> # layer1 A B 1.5
>>> # layer2 A B 2.0
>>> # layer1 B C 1.0
>>> graph, _ = parse_multiplex_edges('multiplex.txt', directed=False)
>>> # Creates nodes: (A, layer1), (A, layer2), (B, layer1), etc.
```

`py3plex.core.parsers.parse_multiplex_folder(input_folder: str, directed: bool) → Tuple[MultiGraph | MultiDiGraph, None, DataFrame]`

Parse a folder containing multiplex network files.

Expects a folder with specific file formats for edges, layers, and optional activity.

Expected Files:

- *.edges: Edge information (format: layer_id node1 node2 weight)
- layers.txt: Layer definitions (format: layer_id layer_name)
- activity.txt: Optional temporal activity (format: node1 node2 timestamp layer_name)

Parameters

- **input_folder** – Path to folder containing multiplex network files
- **directed** – Whether to create a directed graph

Returns

- Union[nx.MultiGraph, nx.MultiDiGraph]: Parsed multilayer graph
- None: Placeholder for labels (not used)
- pd.DataFrame: Time series activity data (empty if no activity.txt)

Return type

Tuple containing

Notes

- Uses glob to find files with specific extensions
- Layer mapping is built from layers.txt
- Activity data is optional and returned as pandas DataFrame
- Nodes are represented as (node_id, layer_id) tuples

Examples

```
>>> # Folder structure:
>>> # my_network/
>>> #   network.edges
>>> #   layers.txt
>>> #   activity.txt (optional)
>>> graph, _, activity_df = parse_multiplex_folder('my_network/', directed=False)
```


`py3plex.core.parsers.parse_network(input_name: str | Any, f_type: str = 'gml', directed: bool = False, label_delimiter: str | None = None, network_type: str = 'multilayer') → Tuple[Any, Any, Any]`

A wrapper method for available parsers!

Parameters

- **input_name** – Path to network file or network object
- **f_type** – Type of file format to parse
- **directed** – Whether to create directed graph
- **label_delimiter** – Optional delimiter for labels
- **network_type** – Type of network (multilayer or multiplex)

Returns

Tuple of (parsed_network, labels, time_series)

`py3plex.core.parsers.parse_nx(nx_object: Graph, directed: bool) → Tuple[Graph, None]`

Core parser for networkx objects.

Parameters

- **nx_object** – A networkx graph
- **directed** – Whether the graph is directed

Returns

Tuple of (graph, None)

Contracts:

- Precondition: nx_object must not be None
- Precondition: nx_object must be a NetworkX graph
- Postcondition: result graph is not None

`py3plex.core.parsers.parse_simple_edgelist(input_name: str, directed: bool) → Tuple[Graph | DiGraph, None]`

Simple edgelist n n w :param input_name: Path to text file :param directed: Whether to create directed graph

`py3plex.core.parsers.parse_spin_edgelist(input_name: str, directed: bool) → Tuple[Graph, None]`

Parse SPIN format edgelist file.

SPIN format includes node pairs with edge tags and optional weights.

File Format:

node1 node2 tag [weight]

Each line: source_node target_node edge_tag [optional_weight]

Parameters

- **input_name** – Path to SPIN edgelist file
- **directed** – Whether to create directed graph (currently creates undirected)

Returns

(parsed_graph, None for labels)

Return type

Tuple[nx.Graph, None]

Notes

- Currently always returns `nx.Graph` (undirected) regardless of `directed` parameter
- Edge tag is stored in edge 'type' attribute
- Default weight is 1 if not specified (4th column)

Examples

```
>>> # File content:
>>> # A B protein_interaction 0.95
>>> # B C gene_regulation 0.80
>>> graph, _ = parse_spin_edgelist('spin_edges.txt', directed=False)
```

`py3plex.core.parsers.require(*args, **kwargs)`

`py3plex.core.parsers.save_edgelist(input_network: Graph, output_file: str, attributes: bool = False) → None`

Save network to edgelist format.

For multilayer networks (where nodes are tuples of (node_id, layer)), saves in format: node1 layer1 node2 layer2

For regular networks, saves in format: node1 node2

`py3plex.core.parsers.save_gpickle(input_network: Any, output_file: str) → None`

`py3plex.core.parsers.save_multiedgelist(input_network: Any, output_file: str, attributes: bool = False, encode_with_ints: bool = False) → Tuple[Dict[Any, str], Dict[Any, str]] | None`

Save multiedgelist – as n1, l1, n2, l2, w

Returns

When `encode_with_ints` is `True`, returns tuple of (node_encodings, type_encodings) Otherwise returns `None`

`py3plex.core.converters.compute_layout(network: Graph, compute_layouts: str, layout_parameters: Dict[str, Any] | None, verbose: bool) → Graph`

Compute and normalize layout for a network.

Parameters

- **network** – NetworkX graph to compute layout for
- **compute_layouts** – Layout algorithm to use ('force', 'random', 'custom_coordinates')
- **layout_parameters** – Optional parameters for layout algorithms
- **verbose** – Whether to print verbose output

Returns

Network with 'pos' attribute added to nodes

Contracts:

- Precondition: network must not be `None` and must have at least one node
- Precondition: `compute_layouts` must be a valid algorithm name
- Postcondition: all nodes have 'pos' attribute (layout preserves nodes)

`py3plex.core.converters.ensure(*args, **kwargs)`

`py3plex.core.converters.prepare_for_parsing(multinet)`

Compute layout for a hairball visualization

Parameters

param1 (*obj*) – multilayer object

Returns

(names, prepared network)

Return type

tuple

`py3plex.core.converters.prepare_for_visualization(multinet: Graph, network_type: str = 'multilayer', compute_layouts: str = 'force', layout_parameters: Dict[str, Any] | None = None, verbose: bool = True, multiplex: bool = False) → Tuple[List[Any], List[Graph], Any]`

This functions takes a multilayer object and returns individual layers, their names, as well as multilayer edges spanning over multiple layers.

Parameters

- **multinet** – multilayer network object
- **network_type** – “multilayer” or “multiplex”
- **compute_layouts** – Layout algorithm (‘force’, ‘random’, etc.)
- **layout_parameters** – Optional layout parameters
- **verbose** – Whether to print progress information
- **multiplex** – Whether to treat as multiplex network

Returns

(**layer_names**, **layer_networks_list**, **multiedges**)

- **layer_names**: List of layer names
- **layer_networks_list**: List of NetworkX graph objects for each layer
- **multiedges**: Dictionary of edges spanning multiple layers

Return type

tuple

`py3plex.core.converters.prepare_for_visualization_hairball(multinet, compute_layouts=False)`

Compute layout for a hairball visualization

Parameters

param1 (*obj*) – multilayer object

Returns

(names, prepared network)

Return type

tuple

`py3plex.core.converters.require(*args, **kwargs)`

`py3plex.core.random_generators.ensure(*args, **kwargs)`

`py3plex.core.random_generators.random_multilayer_ER(n: int, l: int, p: float, directed: bool = False) → Any`

Generate random multilayer Erdős-Rényi network.

Parameters

- **n** – Number of nodes (must be positive)
- **l** – Number of layers (must be positive)
- **p** – Edge probability in [0, 1]
- **directed** – If True, generate directed network

Returns

multi_layer_network object

Contracts:

- Precondition: $n > 0$ - must have at least one node
- Precondition: $l > 0$ - must have at least one layer
- Precondition: $0 \leq p \leq 1$ - probability must be valid
- Postcondition: result is not None - must return valid network

`py3plex.core.random_generators.random_multiplex_ER(n: int, l: int, p: float, directed: bool = False) → Any`

Generate random multiplex Erdős-Rényi network.

Parameters

- **n** – Number of nodes (must be positive)
- **l** – Number of layers (must be positive)
- **p** – Edge probability in [0, 1]
- **directed** – If True, generate directed network

Returns

multi_layer_network object

Contracts:

- Precondition: $n > 0$ - must have at least one node
- Precondition: $l > 0$ - must have at least one layer
- Precondition: $0 \leq p \leq 1$ - probability must be valid
- Postcondition: result is not None - must return valid network

`py3plex.core.random_generators.random_multiplex_generator(n: int, m: int, d: float = 0.9) → MultiGraph`

Generate a multiplex network from a random bipartite graph.

Parameters

- **n** – Number of nodes (must be positive)
- **m** – Number of layers (must be positive)
- **d** – Layer dropout to avoid cliques, range [0..1] (default: 0.9)

Returns

Generated multiplex network as a MultiGraph

Contracts:

- Precondition: $n > 0$ - must have at least one node
- Precondition: $m > 0$ - must have at least one layer
- Precondition: $0 \leq d \leq 1$ - dropout must be valid probability
- Postcondition: result is not None
- Postcondition: result is a NetworkX MultiGraph

`py3plex.core.random_generators.require(*args, **kwargs)`

Supporting methods for parsers and converters.

This module provides utility functions for network parsing and conversion, including layer splitting, multiplex edge addition, and GAF parsing.

`py3plex.core.supporting.add_mpx_edges(input_network: Graph) → Graph`

Add multiplex edges between corresponding nodes across layers.

Multiplex edges connect nodes representing the same entity across different layers of a multilayer network.

Parameters

input_network – NetworkX graph with multilayer structure.

Returns

Network with added multiplex edges between corresponding nodes.

Contracts:

- Precondition: `input_network` must not be None
- Precondition: `input_network` must be a NetworkX graph
- Postcondition: result is a NetworkX graph

Example

```
>>> network = nx.Graph()
>>> network.add_node(('A', 'layer1'))
>>> network.add_node(('A', 'layer2'))
>>> network = add_mpx_edges(network)
```

`py3plex.core.supporting.ensure(*args, **kwargs)`

`py3plex.core.supporting.parse_gaf_to_uniprot_GO(gaf_mappings: str, filter_terms: int | None = None)`
 $\rightarrow \text{Dict}[\text{str}, \text{List}[\text{str}]]$

Parse Gene Association File (GAF) to map UniProt IDs to GO terms.

Parameters

- **gaf_mappings** – Path to GAF file.
- **filter_terms** – Optional minimum occurrence threshold for GO terms.

Returns

Dictionary mapping UniProt IDs to lists of associated GO terms.

Example

```
>>> mappings = parse_gaf_to_uniprot_G0("gaf_file.gaf", filter_terms=5)
```

`py3plex.core.supporting.require(*args, **kwargs)`

`py3plex.core.supporting.split_to_layers(input_network: Graph) → Dict[Any, Graph]`

Split a multilayer network into separate layer subgraphs.

Parameters

input_network – NetworkX graph containing nodes from multiple layers.

Returns

Dictionary mapping layer names to their corresponding subgraphs.

Contracts:

- Precondition: `input_network` must not be `None`
- Precondition: `input_network` must be a NetworkX graph
- Postcondition: result is a dictionary
- Postcondition: all values are NetworkX graphs

Example

```
>>> network = nx.Graph()
>>> network.add_node('A', 'layer1')
>>> network.add_node('B', 'layer2')
>>> layers = split_to_layers(network)
```

NetworkX compatibility layer for py3plex. This module provides compatibility functions for different NetworkX versions.

`py3plex.core.nx_compat.ensure(*args, **kwargs)`

`py3plex.core.nx_compat.is_string_like(obj: Any) → bool`

Check if `obj` is string-like (compatible with NetworkX < 3.0).

Parameters

obj – Object to check

Returns

True if string-like

Return type

bool

`py3plex.core.nx_compat.nx_from_scipy_sparse_matrix(A: Any, parallel_edges: bool = False, create_using: Graph | None = None, edge_attribute: str = 'weight') → Graph`

Create a graph from scipy sparse matrix (compatible with NetworkX < 3.0 and >= 3.0).

Parameters

- **A** – scipy sparse matrix
- **parallel_edges** – Whether to create parallel edges (ignored in NetworkX 3.0+)
- **create_using** – Graph type to create

- **edge_attribute** – Edge attribute name for weights

Returns

NetworkX graph

Contracts:

- Precondition: A must not be None
- Precondition: edge_attribute must be a non-empty string
- Postcondition: returns a NetworkX graph

`py3plex.core.nx_compat.nx_info(G: Graph) → str`

Get network information (compatible with NetworkX < 3.0 and >= 3.0).

Parameters

G – NetworkX graph

Returns

Network information

Return type

str

Contracts:

- Precondition: G must not be None and must be a NetworkX graph
- Postcondition: returns a non-empty string

`py3plex.core.nx_compat.nx_read_gpickle(path: str) → Graph`

Read a graph from a pickle file (compatible with NetworkX < 3.0 and >= 3.0).

Parameters

path – File path

Returns

NetworkX graph

Contracts:

- Precondition: path must be a non-empty string
- Postcondition: returns a NetworkX graph

`py3plex.core.nx_compat.nx_to_scipy_sparse_matrix(G: Graph, nodelist: list | None = None, dtype: Any | None = None, weight: str = 'weight', format: str = 'csr') → Any`

Convert graph to scipy sparse matrix (compatible with NetworkX < 3.0 and >= 3.0).

Parameters

- **G** – NetworkX graph
- **nodelist** – List of nodes
- **dtype** – Data type
- **weight** – Edge weight attribute
- **format** – Sparse matrix format

Returns

scipy sparse matrix

Contracts:

- Precondition: G must not be None and must be a NetworkX graph
- Precondition: weight must be a non-empty string
- Precondition: format must be a non-empty string
- Postcondition: returns a non-None sparse matrix

`py3plex.core.nx_compat.nx_write_gpickle(G: Graph, path: str) → None`

Write a graph to a pickle file (compatible with NetworkX < 3.0 and >= 3.0).

Parameters

- **G** – NetworkX graph
- **path** – File path

Contracts:

- Precondition: G must not be None and must be a NetworkX graph
- Precondition: path must be a non-empty string

`py3plex.core.nx_compat.require(*args, **kwargs)`

HINMINE Network Decomposition

`py3plex.core.HINMINE.decomposition.aggregate_sum(input_thing, classes, universal_set)`

`py3plex.core.HINMINE.decomposition.aggregate_weighted_sum(input_thing, classes, universal_set)`

`py3plex.core.HINMINE.decomposition.calculate_importance_chi(classes, universal_set, linked_nodes, n, **kwargs)`

Calculates importance of a single midpoint using chi-squared weighing. :param classes: List of all classes :param universal_set: Set of all indices to consider :param linked_nodes: Set of all nodes linked by the midpoint :param n: Number of elements of universal set :return: List of weights of the midpoint for each label in class

`py3plex.core.HINMINE.decomposition.calculate_importance_delta(classes, universal_set, linked_nodes, n, **kwargs)`

Calculates importance of a single midpoint using delta-idf weighing :param classes: List of all classes :param universal_set: Set of all indices to consider :param linked_nodes: Set of all nodes linked by the midpoint :param n: Number of elements of universal set :return: List of weights of the midpoint for each label in class

`py3plex.core.HINMINE.decomposition.calculate_importance_gr(classes, universal_set, linked_nodes, n, **kwargs)`

Calculates importance of a single midpoint using the GR (gain ratio) :param classes: List of all classes :param universal_set: Set of all indices to consider :param linked_nodes: Set of all nodes linked by the midpoint :param n: Number of elements of universal set :return: List of weights of the midpoint for each label in class

`py3plex.core.HINMINE.decomposition.calculate_importance_idf(classes, universal_set, linked_nodes, n, **kwargs)`

Calculates importance of a single midpoint using idf weighing :param classes: List of all classes :param universal_set: Set of all indices to consider :param linked_nodes: Set of all nodes linked by the midpoint :param n: Number of elements of universal set :return: List of weights of the midpoint for each label in class


```
py3plex.core.HINMINE.decomposition.calculate_importance_ig(classes, universal_set, linked_nodes, n,
                                                         **kwargs)
```

Calculates importance of a single midpoint using IG (information gain) weighing :param classes: List of all classes :param universal_set: Set of all indices to consider :param linked_nodes: Set of all nodes linked by the midpoint :param n: Number of elements of universal set :return: List of weights of the midpoint for each label in class

```
py3plex.core.HINMINE.decomposition.calculate_importance_okapi(classes, universal_set,
                                                             linked_nodes, n, degrees=None,
                                                             avgdegree=None)
```

```
py3plex.core.HINMINE.decomposition.calculate_importance_rf(classes, universal_set, linked_nodes, n,
                                                           **kwargs)
```

Calculates importance of a single midpoint using rf weighing :param classes: List of all classes :param universal_set: Set of all indices to consider :param linked_nodes: Set of all nodes linked by the midpoint :param n: Number of elements of universal set :return: List of weights of the midpoint for each label in class

```
py3plex.core.HINMINE.decomposition.calculate_importance_tf(classes, universal_set, linked_nodes, n,
                                                           **kwargs)
```

Calculates importance of a single midpoint using term frequency weighing. :param classes: List of all classes :param universal_set: Set of all indices to consider :param linked_nodes: Set of all nodes linked by the midpoint :param n: Number of elements of universal set :return: List of weights of the midpoint for each label in class

```
py3plex.core.HINMINE.decomposition.calculate_importance_w2w(classes, universal_set, linked_nodes,
                                                            n, **kwargs)
```

```
py3plex.core.HINMINE.decomposition.calculate_importances(midpoints, classes, universal_set, method,
                                                         degrees=None, avgdegree=None)
```

```
py3plex.core.HINMINE.decomposition.chi_value(actual_pos_num, predicted_pos_num, tp, n)
```

```
py3plex.core.HINMINE.decomposition.get_aggregation_method(method_name)
```

```
py3plex.core.HINMINE.decomposition.get_calculation_method(method_name)
```

```
py3plex.core.HINMINE.decomposition.gr_value(actual_pos_num, predicted_pos_num, tp, n)
```

```
py3plex.core.HINMINE.decomposition.hinmine_decompose(network, heuristic, cycle=None,
                                                       parallel=False)
```

```
py3plex.core.HINMINE.decomposition.hinmine_get_cycles(network, cycle=None)
```

```
py3plex.core.HINMINE.decomposition.ig_value(actual_pos_num, predicted_pos_num, tp, n)
```

```
py3plex.core.HINMINE.decomposition.np_calculate_importance_chi(predicted, label_matrix,
                                                                actual_pos_nums)
```

```
py3plex.core.HINMINE.decomposition.np_calculate_importance_tf(predicted, label_matrix)
```

```
py3plex.core.HINMINE.decomposition.rf_value(predicted_pos_num, tp)
```

```
py3plex.core.HINMINE.IO.load_hinmine_object(infile, label_delimiter='---', weight_tag=False,
                                             targets=True)
```

6.46.2 Configuration and Utilities

Centralized configuration for py3plex.

This module provides default settings for visualization, layout algorithms, and other configurable aspects of the library. Users can override these settings by modifying the values after import.

Example

```
>>> from py3plex import config
>>> config.DEFAULT_NODE_SIZE = 15
>>> config.DEFAULT_EDGE_ALPHA = 0.5
```

`py3plex.config.get_color_palette(name: str = None) → List[str]`

Get a color palette by name.

Parameters

name – Palette name. If None, returns the default palette.

Returns

List of color hex codes.

Raises

ValueError – If palette name is not recognized.

Example

```
>>> from py3plex.config import get_color_palette
>>> colors = get_color_palette("rainbow")
>>> print(colors[0])
'FF6B6B'
```

`py3plex.config.reset_to_defaults() → None`

Reset all configuration values to their defaults.

This is useful for testing or when you want to start fresh.

Example

```
>>> from py3plex import config
>>> config.DEFAULT_NODE_SIZE = 20
>>> config.reset_to_defaults()
>>> print(config.DEFAULT_NODE_SIZE)
10
```

Utility functions for py3plex.

This module provides common utilities used across the library, including random state management for reproducibility and deprecation warnings.

`py3plex.utils.deprecated(reason: str, version: str = None, alternative: str = None) → Callable[[Callable], Callable]`

Decorator to mark functions/methods as deprecated.

This decorator will issue a `DeprecationWarning` when the decorated function is called, providing information about why it's deprecated and what to use instead.

Parameters

- **reason** – Explanation of why the function is deprecated
- **version** – Version in which the function was deprecated (optional)
- **alternative** – Suggested alternative function/method (optional)

Returns

Decorator function

Example

```
>>> @deprecated(
...     reason="This function is obsolete",
...     version="0.95a",
...     alternative="new_function()"
... )
... def old_function():
...     pass
```

py3plex.utils.**ensure**(*args, **kwargs)

py3plex.utils.**get_background_knowledge_dir**() → str

Get the absolute path to the background knowledge directory.

Returns

Absolute path to the background_knowledge directory

Return type

str

Examples

```
>>> from py3plex.utils import get_background_knowledge_dir
>>> dir_path = get_background_knowledge_dir()
```

py3plex.utils.**get_background_knowledge_path**(filename: str) → str

Get the absolute path to a background knowledge file or directory.

Convenience wrapper around get_data_path() specifically for background knowledge files.

Parameters

filename – Name or relative path of the background knowledge file. Use empty string or ‘.’ to get the background_knowledge directory itself.

Returns

Absolute path to the background knowledge file or directory

Return type

str

Examples

```
>>> from py3plex.utils import get_background_knowledge_path
>>> path = get_background_knowledge_path("bk.n3")
>>> dir_path = get_background_knowledge_path(".")
```

`py3plex.utils.get_data_path(relative_path: str) → str`

Get the absolute path to a data file in the repository.

This function searches for data files in multiple locations to support both: - Running examples from a cloned repository - Running scripts/notebooks from any directory with datasets locally available

Search order: 1. Relative to the calling script's directory (for examples in cloned repo) 2. Relative to current working directory (for notebooks/user scripts) 3. Relative to py3plex package location (for editable installs)

Parameters

relative_path – Path relative to repository root (e.g., “datasets/intact02.gpickle”)

Returns

Absolute path to the file

Return type

str

Raises

Py3plexIOError – If the file cannot be found in any search location

Examples

```
>>> from py3plex.utils import get_data_path
>>> path = get_data_path("datasets/intact02.gpickle")
>>> os.path.exists(path)
True
```

Note

When py3plex is installed via pip, datasets are not included in the package. Users should either: - Clone the repository and run examples from there - Download datasets separately and place them relative to their scripts - Use current working directory with datasets folder

`py3plex.utils.get_dataset_path(filename: str) → str`

Get the absolute path to a dataset file.

Convenience wrapper around `get_data_path()` specifically for dataset files.

Parameters

filename – Name or relative path of the dataset file

Returns

Absolute path to the dataset file

Return type

str

Examples

```
>>> from py3plex.utils import get_dataset_path
>>> path = get_dataset_path("intact02.gpickle")
>>> os.path.exists(path)
True
```

`py3plex.utils.get_example_image_path(filename: str) → str`

Get the absolute path to an example image file.

Convenience wrapper around `get_data_path()` specifically for example image files.

Parameters

filename – Name or relative path of the image file

Returns

Absolute path to the example image file

Return type

str

Examples

```
>>> from py3plex.utils import get_example_image_path
>>> path = get_example_image_path("intact_10_BH.png")
```

`py3plex.utils.get_multilayer_dataset_path(relative_path: str) → str`

Get the absolute path to a multilayer dataset file.

Convenience wrapper around `get_data_path()` specifically for multilayer dataset files.

Parameters

relative_path – Relative path within multilayer_datasets directory

Returns

Absolute path to the multilayer dataset file

Return type

str

Examples

```
>>> from py3plex.utils import get_multilayer_dataset_path
>>> path = get_multilayer_dataset_path("MLKing/MLKing2013_multiplex.edges")
```

`py3plex.utils.get_rng(seed: int | Generator | None = None) → Generator`

Get a NumPy random number generator with optional seed.

This provides a unified interface for random state management across the library, ensuring reproducibility when a seed is provided.

Parameters

seed – Random seed for reproducibility. Can be: - None: Use default unseeded generator - int: Seed value for the generator - np.random.Generator: Pass through existing generator

Returns

Initialized random number generator

Return type

np.random.Generator

Examples

```
>>> rng = get_rng(42)
>>> rng.random() # Reproducible random number
0.7739560485559633
```

```
>>> rng1 = get_rng(42)
>>> rng2 = get_rng(42)
>>> rng1.random() == rng2.random()
True
```

```
>>> existing_rng = np.random.default_rng(123)
>>> rng = get_rng(existing_rng)
>>> rng is existing_rng
True
```

Contracts:

- Postcondition: result is a NumPy random Generator

Note

Uses `numpy.random.Generator` (modern API introduced in NumPy 1.17) rather than the legacy `numpy.random.RandomState` API.

Negative seeds are converted to positive values by taking absolute value to ensure compatibility with NumPy's `SeedSequence`.

`py3plex.utils.require(*args, **kwargs)`

`py3plex.utils.validate_multilayer_input(network_data: Any) → None`

Validate multilayer network input data.

Performs sanity checks on multilayer network structures to catch common errors early.

Parameters

network_data – Network data to validate (can be various formats)

Raises

ValueError – If the network data is invalid

Contracts:

- Precondition: `network_data` must not be `None`

Example

```
>>> from py3plex.utils import validate_multilayer_input
>>> validate_multilayer_input(my_network)
```

`py3plex.utils.warn_if_deprecated(feature_name: str, reason: str, alternative: str = None) → None`

Issue a deprecation warning for a feature.

This is useful for deprecating specific usage patterns or parameter combinations rather than entire functions.

Parameters

- **feature_name** – Name of the deprecated feature
- **reason** – Explanation of why it's deprecated
- **alternative** – Suggested alternative (optional)

Example

```
>>> def my_function(old_param=None, new_param=None):
...     if old_param is not None:
...         warn_if_deprecated(
...             "old_param",
...             "This parameter is no longer used",
...             "new_param"
...         )
```

Custom exception types for the py3plex library.

This module defines domain-specific exceptions to provide clear error messages and enable better error handling throughout the library.

exception py3plex.exceptions.**AlgorithmError**

Bases: *Py3plexException*

Exception raised when an algorithm execution fails.

exception py3plex.exceptions.**CentralityComputationError**

Bases: *AlgorithmError*

Exception raised when centrality computation fails.

exception py3plex.exceptions.**CommunityDetectionError**

Bases: *AlgorithmError*

Exception raised when community detection fails.

exception py3plex.exceptions.**ConversionError**

Bases: *Py3plexException*

Exception raised when format conversion fails.

exception py3plex.exceptions.**DecompositionError**

Bases: *AlgorithmError*

Exception raised when network decomposition fails.

exception py3plex.exceptions.**EmbeddingError**

Bases: *AlgorithmError*

Exception raised when embedding generation fails.

exception py3plex.exceptions.**ExternalToolError**

Bases: *Py3plexException*

Exception raised when external tool execution fails.

exception py3plex.exceptions.**IncompatibleNetworkError**

Bases: *Py3plexException*

Exception raised when network format is incompatible with an operation.

exception py3plex.exceptions.**InvalidEdgeError**

Bases: *Py3plexException*

Exception raised when an invalid edge is specified.

exception `py3plex.exceptions.InvalidLayerError`

Bases: *Py3plexException*

Exception raised when an invalid layer is specified.

exception `py3plex.exceptions.InvalidNodeError`

Bases: *Py3plexException*

Exception raised when an invalid node is specified.

exception `py3plex.exceptions.NetworkConstructionError`

Bases: *Py3plexException*

Exception raised when network construction fails.

exception `py3plex.exceptions.ParsingError`

Bases: *Py3plexException*

Exception raised when parsing input data fails.

exception `py3plex.exceptions.Py3plexException`

Bases: *Exception*

Base exception class for all py3plex-specific exceptions.

exception `py3plex.exceptions.Py3plexFormatError`

Bases: *Py3plexException*

Exception raised when input format is invalid or cannot be parsed.

exception `py3plex.exceptions.Py3plexIOError`

Bases: *Py3plexException*

Exception raised when I/O operations fail (file reading, writing, etc.).

exception `py3plex.exceptions.Py3plexLayoutError`

Bases: *Py3plexException*

Exception raised when layout computation or visualization positioning fails.

exception `py3plex.exceptions.Py3plexMatrixError`

Bases: *Py3plexException*

Exception raised when matrix operations fail or matrix is invalid.

exception `py3plex.exceptions.VisualizationError`

Bases: *Py3plexException*

Exception raised when visualization operations fail.

6.46.3 Algorithms

Community Detection


```

py3plex.algorithms.community_detection.community_wrapper.NoRC_communities(network,
                                                                    verbose=True,
                                                                    cluster-
                                                                    ing_scheme='kmeans',
                                                                    output='mapping',
                                                                    prob_threshold=0.001,
                                                                    parallel_step=8,
                                                                    commu-
                                                                    nity_range=None,
                                                                    fine_range=3)

py3plex.algorithms.community_detection.community_wrapper.infomap_communities(graph: Graph,
                                                                    binary: str =
                                                                    './infomap',
                                                                    edgelist_file: str
                                                                    =
                                                                    './tmp/tmpedgelist.txt',
                                                                    multiplex: bool
                                                                    = False,
                                                                    verbose: bool =
                                                                    False,
                                                                    overlapping:
                                                                    bool = False,
                                                                    iterations: int =
                                                                    200, output: str
                                                                    = 'mapping',
                                                                    seed: int =
                                                                    None) →
                                                                    Dict[Any, int] |
                                                                    Dict[Any,
                                                                    List[int]]

```

Detect communities using the Infomap algorithm.

Parameters

- **graph** – Input graph (NetworkX graph or multi_layer_network)
- **binary** – Path to Infomap binary (default: “./infomap”)
- **edgelist_file** – Temporary file for edgelist (default: “./tmp/tmpedgelist.txt”)
- **multiplex** – Whether to use multiplex mode (default: False)
- **verbose** – Whether to show verbose output (default: False)
- **overlapping** – Whether to detect overlapping communities (default: False)
- **iterations** – Number of iterations (default: 200)
- **output** – Output format - “mapping” or “partition” (default: “mapping”)
- **seed** – Random seed for reproducibility (default: None) Note: Requires Infomap binary that supports –seed parameter

Returns

Dict mapping nodes to community IDs (if output=“mapping”) or Dict mapping community IDs to lists of nodes (if output=“partition”)

Raises

- **FileNotFoundError** – If Infomap binary is not found
- **PermissionError** – If Infomap binary is not executable

Examples

```
>>> # Using with seed for reproducibility
>>> partition = infomap_communities(graph, seed=42)
>>>
>>> # Get partition format instead of mapping
>>> communities = infomap_communities(graph, output="partition")
```

```
py3plex.algorithms.community_detection.community_wrapper.louvain_communities(network, out-
                                                                    put='mapping')
```

```
py3plex.algorithms.community_detection.community_wrapper.parse_infomap(outfile)
```

```
py3plex.algorithms.community_detection.community_wrapper.run_infomap(infile: str, multiplex: bool
                                                                    = True, overlapping: bool
                                                                    = False, binary: str =
                                                                    './infomap', verbose: bool
                                                                    = True, iterations: int =
                                                                    1000, seed: int = None)
→ None
```

Multilayer Modularity Maximization (Mucha et al., 2010)

This module implements multilayer modularity quality function and optimization algorithms for community detection in multilayer/multiplex networks.

References

Mucha et al., “Community Structure in Time-Dependent, Multiscale, and Multiplex Networks”, Science 328:876-878 (2010)

```

py3plex.algorithms.community_detection.multilayer_modularity.build_supra_modularity_matrix(network:
Any,
gamma:
float
|
Dict[Any,
float]
=
1.0,
omega:
float
|
ndar-
ray
=
1.0,
weight:
str
=
'weight')
→
Tu-
ple[ndarray,
List[Tuple[Any,
Any]]]

```

Build the supra-modularity matrix B for multilayer network.

The supra-modularity matrix is: $B_{\{i,j\}} = (A_{ij} - \frac{k_i k_j}{2m}) + \frac{\omega}{2m}$

This matrix can be used for spectral community detection methods.

Parameters

- **network** – py3plex multi_layer_network object
- **gamma** – Resolution parameter(s)
- **omega** – Inter-layer coupling strength
- **weight** – Edge weight attribute

Returns

Tuple of (modularity_matrix, node_layer_list) - modularity_matrix: Supra-modularity matrix B (NL × NL) - node_layer_list: List of (node, layer) tuples corresponding to matrix indices

```
py3plex.algorithms.community_detection.multilayer_modularity.louvain_multilayer(network:  
    Any,  
    gamma:  
    float |  
    Dict[Any,  
    float] = 1.0,  
    omega:  
    float |  
    ndarray =  
    1.0, weight:  
    str =  
    'weight',  
    max_iter:  
    int = 100,  
    random_state:  
    int | None =  
    None) →  
    Dict[Tuple[Any,  
    Any], int]
```

Generalized Louvain algorithm for multilayer networks.

This implements the multilayer Louvain method as described in Mucha et al. (2010), which greedily maximizes the multilayer modularity quality function.

Complexity:

- **Time:** $O(n \times L \times d \times k)$ per iteration, where:
 - n = number of nodes per layer
 - L = number of layers
 - d = average degree
 - k = number of communities
- Typical: $O(n \times L)$ iterations until convergence
- Worst case: $O((n \times L)^2)$ for dense networks
- Space: $O((n \times L)^2)$ for supra-adjacency matrix (use sparse for large networks)

Parameters

- **network** – py3plex multi_layer_network object
- **gamma** – Resolution parameter(s)
- **omega** – Inter-layer coupling strength
- **weight** – Edge weight attribute
- **max_iter** – Maximum number of iterations
- **random_state** – Random seed for reproducibility

Returns

Dictionary mapping (node, layer) tuples to community IDs

Examples

```
>>> from py3plex.core import multinet
>>> from py3plex.algorithms.community_detection.multilayer_modularity import _
↳ louvain_multilayer
>>>
>>> network = multinet.multi_layer_network(directed=False)
>>> network.add_edges([
...     ['A', 'L1', 'B', 'L1', 1],
...     ['B', 'L1', 'C', 'L1', 1],
...     ['A', 'L2', 'C', 'L2', 1]
... ], input_type='list')
>>>
>>> communities = louvain_multilayer(network, gamma=1.0, omega=1.0, random_state=42)
>>> print(communities)
```

Note

For reproducible results, always set `random_state` parameter.

`py3plex.algorithms.community_detection.multilayer_modularity.multilayer_modularity(network: Any, communities: Dict[Tuple[Any, Any], int], gamma: float | Dict[Any, float] = 1.0, omega: float | ndarray = 1.0, weight: str = 'weight') → float`

Calculate multilayer modularity quality function (Mucha et al., 2010).

The multilayer modularity is defined as: $Q = (1/2) \sum_{ij} [(A^l_{ij} - P^l_{ij}) + g_l] (g_i, g_j)$

where: - A^l_{ij} is the adjacency matrix of layer - P^l_{ij} is the null model (e.g., Newman-Girvan: $k_i^l k_j^l / 2m_l$) - g_l is the resolution parameter for layer - g_l is the inter-layer coupling strength - $g_l = 1$ if $l =$, else 0 (Kronecker delta) - $g_l = 1$ if $i=j$, else 0 - $(g_i, g_j) = 1$ if node i in layer l and node j in layer l are in same community - W is the total edge weight in the supra-network

Parameters

- **network** – py3plex `multi_layer_network` object
- **communities** – Dictionary mapping (node, layer) tuples to community IDs

- **gamma** – Resolution parameter(s). Can be: - Single float: same resolution for all layers - Dict[layer, float]: layer-specific resolution parameters
- **omega** – Inter-layer coupling strength. Can be: - Single float: uniform coupling between all layer pairs - np.ndarray: layer-pair specific coupling matrix ($L \times L$)
- **weight** – Edge weight attribute (default: “weight”)

Returns

Modularity value Q [-1, 1]

Examples

```
>>> from py3plex.core import multinet
>>> from py3plex.algorithms.community_detection.multilayer_modularity import _
    ↪ multilayer_modularity
>>>
>>> # Create a simple multilayer network
>>> network = multinet.multi_layer_network(directed=False)
>>> network.add_edges([
...     ['A', 'L1', 'B', 'L1', 1],
...     ['B', 'L1', 'C', 'L1', 1],
...     ['A', 'L2', 'C', 'L2', 1]
... ], input_type='list')
>>>
>>> # Assign communities
>>> communities = {
...     ('A', 'L1'): 0, ('B', 'L1'): 0, ('C', 'L1'): 1,
...     ('A', 'L2'): 0, ('C', 'L2'): 0
... }
>>>
>>> # Calculate modularity
>>> Q = multilayer_modularity(network, communities, gamma=1.0, omega=1.0)
>>> print(f"Modularity: {Q:.3f}")
```

This module implements community detection.

class py3plex.algorithms.community_detection.community_louvain.**Status**

Bases: object

To handle several data in one struct.

Could be replaced by named tuple, but don't want to depend on python 2.6

copy()

Perform a deep copy of status

degrees: dict = {}

gdegrees: dict = {}

init(graph, weight, part=None)

Initialize the status of a graph with every node in one community

internals: dict = {}

node2com: dict = {}

```
total_weight = 0
```

```
py3plex.algorithms.community_detection.community_louvain.best_partition(graph: Graph,
                                                                    partition: Dict | None
                                                                    = None, weight: str =
                                                                    'weight', resolution:
                                                                    float = 1.0,
                                                                    randomize: bool =
                                                                    False) → Dict
```

Compute the partition of the graph nodes which maximises the modularity (or try..) using the Louvain heuristics

This is the partition of highest modularity, i.e. the highest partition of the dendrogram generated by the Louvain algorithm.

Parameters

- **graph** (*networkx.Graph*) – the networkx graph which is decomposed
- **partition** (*dict, optional*) – the algorithm will start using this partition of the nodes. It's a dictionary where keys are their nodes and values the communities
- **weight** (*str, optional*) – the key in graph to use as weight. Default to 'weight'
- **resolution** (*double, optional*) – Will change the size of the communities, default to 1. represents the time described in “Laplacian Dynamics and Multiscale Modular Structure in Networks”, R. Lambiotte, J.-C. Delvenne, M. Barahona
- **randomize** (*boolean, optional*) – Will randomize the node evaluation order and the community evaluation order to get different partitions at each call

Returns

partition – The partition, with communities numbered from 0 to number of communities

Return type

dictionnary

Raises

NetworkXError – If the graph is not Eulerian.

See also

generate_dendrogram

Notes

Uses Louvain algorithm

References

large networks. J. Stat. Mech 10008, 1-12(2008).

Examples

```
>>> #Basic usage
>>> G=nx.erdos_renyi_graph(100, 0.01)
>>> part = best_partition(G)
```

```

>>> #other example to display a graph with its community :
>>> #better with karate_graph() as defined in networkx examples
>>> #erdos renyi don't have true community structure
>>> G = nx.erdos_renyi_graph(30, 0.05)
>>> #first compute the best partition
>>> partition = community.best_partition(G)
>>> #drawing
>>> size = float(len(set(partition.values())))
>>> pos = nx.spring_layout(G)
>>> count = 0.
>>> for com in set(partition.values()) :
>>>     count += 1.
>>>     list_nodes = [nodes for nodes in partition.keys()
>>>                    if partition[nodes] == com]
>>>     nx.draw_networkx_nodes(G, pos, list_nodes, node_size = 20,
>>>                            node_color = str(count / size))
>>> nx.draw_networkx_edges(G, pos, alpha=0.5)
>>> plt.show()

```

`py3plex.algorithms.community_detection.community_louvain.generate_dendrogram`(*graph*: Graph, *part_init*: Dict | None = None, *weight*: str = 'weight', *resolution*: float = 1.0, *randomize*: bool = False) → List[Dict]

Find communities in the graph and return the associated dendrogram

A dendrogram is a tree and each level is a partition of the graph nodes. Level 0 is the first partition, which contains the smallest communities, and the best is `len(dendrogram) - 1`. The higher the level is, the bigger are the communities

Parameters

- **graph** (*networkx.Graph*) – the networkx graph which will be decomposed
- **part_init** (*dict, optional*) – the algorithm will start using this partition of the nodes. It's a dictionary where keys are their nodes and values the communities
- **weight** (*str, optional*) – the key in graph to use as weight. Default to 'weight'
- **resolution** (*double, optional*) – Will change the size of the communities, default to 1. represents the time described in “Laplacian Dynamics and Multiscale Modular Structure in Networks”, R. Lambiotte, J.-C. Delvenne, M. Barahona

Returns

dendrogram – a list of partitions, ie dictionaries where keys of the *i+1* are the values of the *i*. and where keys of the first are the nodes of graph

Return type

list of dictionaries

Raises

TypeError – If the graph is not a `networkx.Graph`

See also*best_partition***Notes**

Uses Louvain algorithm

References

networks. J. Stat. Mech 10008, 1-12(2008).

Examples

```
>>> G=nx.erdos_renyi_graph(100, 0.01)
>>> dendo = generate_dendrogram(G)
>>> for level in range(len(dendo) - 1) :
>>>     print("partition at level", level,
>>>           "is", partition_at_level(dendo, level))
:param weight:
:type weight:
```

`py3plex.algorithms.community_detection.community_louvain.induced_graph(partition: Dict, graph: Graph, weight: str = 'weight') → Graph`

Produce the graph where nodes are the communities

there is a link of weight w between communities if the sum of the weights of the links between their elements is w

Parameters

- **partition** (*dict*) – a dictionary where keys are graph nodes and values the part the node belongs to
- **graph** (*networkx.Graph*) – the initial graph
- **weight** (*str, optional*) – the key in graph to use as weight. Default to ‘weight’

Returns

g – a networkx graph where nodes are the parts

Return type

networkx.Graph

Examples

```
>>> n = 5
>>> g = nx.complete_graph(2*n)
>>> part = dict([])
>>> for node in g.nodes() :
>>>     part[node] = node % 2
>>> ind = induced_graph(part, g)
>>> goal = nx.Graph()
>>> goal.add_weighted_edges_from([(0,1,n*n),(0,0,n*(n-1)/2), (1, 1, n*(n-1)/2)]) #_
↪ NOQA
```

(continues on next page)

(continued from previous page)

```
>>> nx.is_isomorphic(int, goal)
True
```

`py3plex.algorithms.community_detection.community_louvain.load_binary(data)`

Load binary graph as used by the cpp implementation of this algorithm

`py3plex.algorithms.community_detection.community_louvain.modularity(partition: Dict, graph: Graph, weight: str = 'weight') → float`

Compute the modularity of a partition of a graph

Parameters

- **partition** (*dict*) – the partition of the nodes, i.e a dictionary where keys are their nodes and values the communities
- **graph** (*networkx.Graph*) – the networkx graph which is decomposed
- **weight** (*str, optional*) – the key in graph to use as weight. Default to ‘weight’

Returns

modularity – The modularity

Return type

float

Raises

- **KeyError** – If the partition is not a partition of all graph nodes
- **ValueError** – If the graph has no link
- **TypeError** – If graph is not a networkx.Graph

References

structure in networks. Physical Review E 69, 26113(2004).

Examples

```
>>> G=nx.erdos_renyi_graph(100, 0.01)
>>> part = best_partition(G)
>>> modularity(part, G)
```

`py3plex.algorithms.community_detection.community_louvain.partition_at_level(dendrogram: List[Dict], level: int) → Dict`

Return the partition of the nodes at the given level

A dendrogram is a tree and each level is a partition of the graph nodes. Level 0 is the first partition, which contains the smallest communities, and the best is `len(dendrogram) - 1`. The higher the level is, the bigger are the communities

Parameters

- **dendrogram** (*list of dict*) – a list of partitions, ie dictionaries where keys of the *i+1* are the values of the *i*.
- **level** (*int*) – the level which belongs to `[0..len(dendrogram)-1]`

Returns

partition – A dictionary where keys are the nodes and the values are the set it belongs to

Return type

dictionnary

Raises

KeyError – If the dendrogram is not well formed or the level is too high

See also

best_partition, generate_dendrogram

Examples

```
>>> G=nx.erdos_renyi_graph(100, 0.01)
>>> dendrogram = generate_dendrogram(G)
>>> for level in range(len(dendrogram) - 1) :
>>>     print("partition at level", level, "is", partition_at_level(dendrogram,
↪level)) # NOQA
```

Community quality measures and metrics.

This module provides various measures for assessing the quality of community partitions in networks, including modularity, size distribution, and other statistical metrics.

`py3plex.algorithms.community_detection.community_measures.modularity`(*G: Graph, communities: Dict[Any, List[Any]], weight: str = 'weight'*) → float

Calculate modularity of a graph partition.

Parameters

- **G** – NetworkX graph
- **communities** – Dictionary mapping community IDs to node lists
- **weight** – Edge weight attribute (default: “weight”)

Returns

Modularity value

`py3plex.algorithms.community_detection.community_measures.number_of_communities`(*network_partition: Dict[Any, List[Any]]*) → int

Count number of communities in a partition.

Parameters

network_partition – Dictionary mapping community IDs to node lists

Returns

Number of communities

`py3plex.algorithms.community_detection.community_measures.size_distribution`(*network_partition: Dict[Any, List[Any]]*) → ndarray

Calculate size distribution of communities.

Parameters

network_partition – Dictionary mapping community IDs to node lists

Returns

Array of community sizes

Multilayer Synthetic Graph Generation for Community Detection Benchmarks

This module implements synthetic multilayer/multiplex graph generators with ground-truth community structure for benchmarking community detection algorithms.

Includes: - Multilayer LFR (Lancichinetti-Fortunato-Radicchi) benchmark - Coupled/Interdependent Erdős-Rényi models - Support for overlapping communities across layers - Support for partial node presence across layers

References

Lancichinetti et al., “Benchmark graphs for testing community detection algorithms”, Phys. Rev. E 78, 046110 (2008)

Granell et al., “Benchmark model to assess community structure in evolving networks”, Phys. Rev. E 92, 012805 (2015)

```
py3plex.algorithms.community_detection.multilayer_benchmark.generate_coupled_er_multilayer(n:
                                                    int,
                                                    lay-
                                                    ers:
                                                    List[str],
                                                    p:
                                                    float
                                                    |
                                                    List[float]
                                                    =
                                                    0.1,
                                                    omega:
                                                    float
                                                    =
                                                    1.0,
                                                    cou-
                                                    pling_probabili
                                                    ty:
                                                    float
                                                    =
                                                    1.0,
                                                    di-
                                                    rected:
                                                    bool
                                                    =
                                                    False,
                                                    seed:
                                                    int
                                                    |
                                                    None
                                                    =
                                                    None)
                                                    →
                                                    Any
```

Generate coupled/interdependent Erdős-Rényi multilayer networks.

Creates random Erdős-Rényi graphs in each layer and couples nodes across layers with specified coupling strength and probability.

Parameters

- **n** – Number of nodes
- **layers** – List of layer names
- **p** – Edge probability per layer. Can be float (same for all layers) or list of floats per layer.
- **omega** – Inter-layer coupling strength (weight of identity links)
- **coupling_probability** – Probability that a node has inter-layer coupling. Range: [0, 1] - 1.0 = all nodes coupled (full multiplex) - <1.0 = partial coupling (interdependent networks)
- **directed** – Whether to generate directed networks
- **seed** – Random seed for reproducibility

Returns

py3plex multi_layer_network object

Examples

```
>>> from py3plex.algorithms.community_detection.multilayer_benchmark import _
    generate_coupled_er_multilayer
>>>
>>> # Full multiplex ER network
>>> network = generate_coupled_er_multilayer(
...     n=100,
...     layers=['L1', 'L2', 'L3'],
...     p=0.1,
...     omega=1.0,
...     coupling_probability=1.0
... )
>>>
>>> # Partially coupled (interdependent)
>>> network = generate_coupled_er_multilayer(
...     n=100,
...     layers=['L1', 'L2'],
...     p=0.1,
...     omega=0.5,
...     coupling_probability=0.5 # Only 50% nodes coupled
... )
```

```
py3plex.algorithms.community_detection.multilayer_benchmark.generate_multilayer_lfr(n: int,  
                                                                                   layers:  
                                                                                   List[str],  
                                                                                   tau1:  
                                                                                   float =  
                                                                                   2.0,  
                                                                                   tau2:  
                                                                                   float =  
                                                                                   1.5,  
                                                                                   mu:  
                                                                                   float |  
                                                                                   List[float]  
                                                                                   = 0.1,  
                                                                                   avg_degree:  
                                                                                   float |  
                                                                                   List[float]  
                                                                                   = 10.0,  
                                                                                   min_community:  
                                                                                   int =  
                                                                                   20,  
                                                                                   max_community:  
                                                                                   int |  
                                                                                   None  
                                                                                   =  
                                                                                   None,  
                                                                                   com-  
                                                                                   mu-  
                                                                                   nity_persistence:  
                                                                                   float =  
                                                                                   1.0,  
                                                                                   node_overlap:  
                                                                                   float =  
                                                                                   1.0,  
                                                                                   over-  
                                                                                   lap-  
                                                                                   ping_nodes:  
                                                                                   int =  
                                                                                   0,  
                                                                                   over-  
                                                                                   lap-  
                                                                                   ping_membership:  
                                                                                   int =  
                                                                                   2, di-  
                                                                                   rected:  
                                                                                   bool =  
                                                                                   False,  
                                                                                   seed:  
                                                                                   int |  
                                                                                   None  
                                                                                   =  
                                                                                   None)  
    → Tu-  
    ple[Any,  
    Dict[Tuple[Any,  
    str],  
    Set[int]]]
```

Generate multilayer LFR benchmark networks with controllable community structure.

This extends the LFR benchmark to multilayer networks, allowing control over:

- Community persistence across layers (how many nodes keep their community)
- Node overlap across layers (which nodes appear in which layers)
- Overlapping communities (nodes belonging to multiple communities)

Parameters

- **n** – Number of nodes
- **layers** – List of layer names
- **tau1** – Power-law exponent for degree distribution (typically 2-3)
- **tau2** – Power-law exponent for community size distribution (typically 1-2)
- **mu** – Mixing parameter (fraction of edges outside community). Can be float (same for all layers) or list of floats per layer. Range: [0, 1], where 0 = perfect communities, 1 = random
- **avg_degree** – Average degree per layer. Can be float (same for all layers) or list of floats per layer.
- **min_community** – Minimum community size
- **max_community** – Maximum community size (default: $n/2$)
- **community_persistence** – Probability that a node keeps its community from one layer to the next. Range: [0, 1] - 1.0 = identical communities across all layers - 0.0 = completely independent communities per layer
- **node_overlap** – Fraction of nodes present in all layers. Range: [0, 1] - 1.0 = all nodes in all layers (full multiplex) - <1.0 = some nodes absent from some layers
- **overlapping_nodes** – Number of nodes that belong to multiple communities within each layer
- **overlapping_membership** – Number of communities each overlapping node belongs to
- **directed** – Whether to generate directed networks
- **seed** – Random seed for reproducibility

Returns

Tuple of (network, ground_truth_communities) - network: py3plex multi_layer_network object
 - ground_truth_communities: Dict mapping (node, layer) to Set of community IDs

Examples

```
>>> from py3plex.algorithms.community_detection.multilayer_benchmark import _
    generate_multilayer_lfr
>>>
>>> # Generate with identical communities across layers
>>> network, communities = generate_multilayer_lfr(
...     n=100,
...     layers=['L1', 'L2', 'L3'],
...     mu=0.1,
...     community_persistence=1.0
... )
>>>
>>> # Generate with evolving communities
>>> network, communities = generate_multilayer_lfr(
```

(continues on next page)

(continued from previous page)

```
...     n=100,  
...     layers=['T0', 'T1', 'T2'],  
...     mu=0.1,  
...     community_persistence=0.7 # 70% nodes keep community  
... )
```

```
py3plex.algorithms.community_detection.multilayer_benchmark.generate_sbm_multilayer(n: int,  
                                                                                   layers:  
                                                                                   List[str],  
                                                                                   com-  
                                                                                   muni-  
                                                                                   ties:  
                                                                                   List[Set[int]],  
                                                                                   p_in:  
                                                                                   float |  
                                                                                   List[float]  
                                                                                   = 0.3,  
                                                                                   p_out:  
                                                                                   float |  
                                                                                   List[float]  
                                                                                   =  
                                                                                   0.05,  
                                                                                   com-  
                                                                                   mu-  
                                                                                   nity_persistence:  
                                                                                   float =  
                                                                                   1.0, di-  
                                                                                   rected:  
                                                                                   bool =  
                                                                                   False,  
                                                                                   seed:  
                                                                                   int |  
                                                                                   None  
                                                                                   =  
                                                                                   None)  
→ Tuple[Any,  
Dict[Tuple[Any,  
str],  
int]]
```

Generate multilayer stochastic block model (SBM) networks.

Creates networks where nodes are divided into communities with different intra- and inter-community connection probabilities.

Parameters

- **n** – Number of nodes
- **layers** – List of layer names
- **communities** – List of node sets defining initial communities
- **p_in** – Intra-community edge probability per layer
- **p_out** – Inter-community edge probability per layer

- **community_persistence** – Probability nodes keep their community across layers
- **directed** – Whether to generate directed networks
- **seed** – Random seed for reproducibility

Returns

Tuple of (network, ground_truth_communities) - network: py3plex multi_layer_network object
 - ground_truth_communities: Dict mapping (node, layer) to community ID

Examples

```
>>> from py3plex.algorithms.community_detection.multilayer_benchmark import _
    generate_sbm_multilayer
>>>
>>> # Define initial communities
>>> communities = [
...     {0, 1, 2, 3, 4}, # Community 0
...     {5, 6, 7, 8, 9} # Community 1
... ]
>>>
>>> network, ground_truth = generate_sbm_multilayer(
...     n=10,
...     layers=['L1', 'L2'],
...     communities=communities,
...     p_in=0.7,
...     p_out=0.1,
...     community_persistence=0.8
... )
```

Statistics

Multilayer Network Statistics

This module implements various statistics for multilayer and multiplex networks, following standard definitions from multilayer network analysis literature.

References

- Kivela et al. (2014), “Multilayer networks”, J. Complex Networks 2(3), 203-271
- De Domenico et al. (2013), “Mathematical formulation of multilayer networks”, PRX 3, 041022
- Mucha et al. (2010), “Community Structure in Time-Dependent, Multiscale, and Multiplex Networks”, Science 328, 876-878

Authors: py3plex contributors Date: 2025

py3plex.algorithms.statistics.multilayer_statistics.**algebraic_connectivity**(network: Any) → float

Calculate algebraic connectivity (λ_2).

Formula: $\lambda_2()$

Second smallest eigenvalue of the supra-Laplacian (Fiedler value).

Indicates global connectivity and diffusion efficiency of the multilayer system.

Properties:

$\lambda_0 = 0$ always (associated with constant eigenvector) $\lambda_1 > 0$ if and only if the multilayer network is connected
Larger λ_1 indicates better connectivity and faster diffusion/synchronization

Parameters

network – py3plex multi_layer_network object

Returns

Second smallest eigenvalue (Fiedler value)

Examples

```
>>> alg_conn = algebraic_connectivity(network)
```

Reference:

Fiedler (1973), Sole-Ribalta et al. (2013)

```
py3plex.algorithms.statistics.multilayer_statistics.community_participation_coefficient(network:  
Any,  
com-  
mu-  
ni-  
ties:  
Dict[Tuple[Any,  
Any],  
int],  
node:  
Any)  
→  
float
```

Calculate participation coefficient for a node across community structure.

Measures how evenly a node's connections are distributed across different communities, across all layers. A node with connections to many communities has high participation.

Formula: $P = 1 - (k_s / k)^2$

where k_s is the number of connections node i has to community s , and k is the total degree of node i across all layers.

Parameters

- **network** – py3plex multi_layer_network object
- **communities** – Dictionary mapping (node, layer) to community ID
- **node** – Node identifier (not node-layer tuple)

Returns

Participation coefficient value between 0 and 1

Examples

```
>>> communities = detect_communities(network)  
>>> pc = community_participation_coefficient(network, communities, 'Alice')  
>>> print(f"Participation: {pc:.3f}")
```

Reference:

Guimerà & Amaral (2005), “Functional cartography of complex metabolic networks”

```
py3plex.algorithms.statistics.multilayer_statistics.community_participation_entropy(network:
Any,
com-
muni-
ties:
Dict[Tuple[Any,
Any],
int],
node:
Any)
→
float
```

Calculate participation entropy for a node across community structure.

Shannon entropy-based measure of how evenly a node distributes its connections across different communities. Higher entropy indicates more diverse community participation.

Formula: $H = - \sum_s (k_s / k) \log(k_s / k)$

where k_s is connections to community s , k is total degree.

Parameters

- **network** – py3plex multi_layer_network object
- **communities** – Dictionary mapping (node, layer) to community ID
- **node** – Node identifier (not node-layer tuple)

Returns

Entropy value (higher = more diverse participation)

Examples

```
>>> entropy = community_participation_entropy(network, communities, 'Alice')
>>> print(f"Participation entropy: {entropy:.3f}")
```

Reference:

Based on Shannon entropy applied to community structure

```
py3plex.algorithms.statistics.multilayer_statistics.compute_modularity_score(network: Any,
communities:
Dict[Tuple[Any,
Any], int],
gamma: float =
1.0, omega:
float = 1.0) →
float
```

Compute explicit multislice modularity score.

Direct computation of the modularity quality function for a given community partition, without running detection algorithms.

Formula: $Q = (1/2) [(A - \sum k_k^2 / (2m)) + \cdot] (c, c)$

Parameters

- **network** – py3plex multi_layer_network object
- **communities** – Dictionary mapping (node, layer) to community ID
- **gamma** – Resolution parameter (default: 1.0)
- **omega** – Inter-layer coupling strength (default: 1.0)

Returns

Modularity score Q (higher is better)

Examples

```
>>> communities = {('A', 'L1'): 0, ('B', 'L1'): 0, ('C', 'L1'): 1}
>>> Q = compute_modularity_score(network, communities)
>>> print(f"Modularity: {Q:.3f}")
```

Reference:

Mucha et al. (2010), Science 328, 876-878

py3plex.algorithms.statistics.multilayer_statistics.**cross_layer_mutual_information**(network: Any, layer_i: str, layer_j: str, bins: int = 10) → float

Calculate cross-layer mutual information ($I(L; L)$).

Formula: $I(L; L) = H(L) + H(L) - H(L, L)$

Measures statistical dependence between degree distributions in two layers; quantifies how much knowing a node's degree in one layer tells us about its degree in another layer.

Variables:

$H(L)$ = entropy of degree distribution in layer i $H(L)$ = entropy of degree distribution in layer j $H(L, L)$ = joint entropy of degree distributions

Properties:

$I = 0$ when layers are independent $I > 0$ indicates statistical dependence (higher = stronger) $I(L; L) \leq \min(H(L), H(L))$

Parameters

- **network** – py3plex multi_layer_network object
- **layer_i** – First layer identifier
- **layer_j** – Second layer identifier
- **bins** – Number of bins for discretizing degree distributions

Returns

Mutual information value in bits

Examples

```
>>> mi = cross_layer_mutual_information(network, 'L1', 'L2', bins=10)
>>> print(f"Mutual information: {mi:.3f} bits")
```

Reference:

Cover & Thomas (2006), “Elements of Information Theory” De Domenico et al. (2015), “Structural reducibility”

`py3plex.algorithms.statistics.multilayer_statistics.cross_layer_redundancy_entropy`(*network: Any*) → float

Calculate cross-layer redundancy entropy ($H_{\text{redundancy}}$).

Formula: $H_r = -r \log_2(r)$, where r is the normalized edge overlap between layers i and j .

Measures diversity in structural redundancy across layer pairs; high entropy indicates varied overlap patterns, low entropy indicates uniform redundancy.

Variables:

$r = \text{edge_overlap}(i,j) / \text{edge_overlap}(.)$ Normalized overlap proportion for each layer pair

Parameters

network – py3plex multi_layer_network object

Returns

Entropy value in bits

Examples

```
>>> entropy = cross_layer_redundancy_entropy(network)
>>> print(f"Cross-layer redundancy entropy: {entropy:.3f}")
```

Reference:

Bianconi (2018), “Multilayer Networks: Structure and Function”

`py3plex.algorithms.statistics.multilayer_statistics.degree_vector`(*network: Any, node: Any, weighted: bool = False*) → Dict[str, float]

Calculate degree vector (k).

Formula: $k = (k^1, k^2, \dots, k)$

Node degree in each layer; can be analyzed via mean, variance, or entropy to capture node versatility.

Variables:

k = degree of node i in layer For undirected: $k = A$

Parameters

- **network** – py3plex multi_layer_network object
- **node** – Node identifier
- **weighted** – If True, return strength instead of degree

Returns

Dictionary mapping layer to degree/strength

Examples

```
>>> degrees = degree_vector(network, 'A')
>>> print(f"Degree in layer L1: {degrees['L1']}")
```

Reference:

Kivela et al. (2014), J. Complex Networks 2(3), 203-271

`py3plex.algorithms.statistics.multilayer_statistics.edge_overlap(network: Any, layer_i: str, layer_j: str) → float`

Calculate edge overlap ($\wedge$).

Formula: $\wedge = |E_i \cap E_j| / |E_i \cup E_j|$

Jaccard similarity of edge sets between two layers; measures structural redundancy.

Variables:

E_i = set of edges in layer i E_j = set of edges in layer j $| \cdot |$ = cardinality (number of elements)

Parameters

- **network** – py3plex multi_layer_network object
- **layer_i** – First layer identifier ()
- **layer_j** – Second layer identifier ()

Returns

Overlap coefficient between 0 and 1 (Jaccard similarity)

Examples

```
>>> overlap = edge_overlap(network, 'L1', 'L2')
```

Reference:

Kivela et al. (2014), J. Complex Networks 2(3), 203-271

`py3plex.algorithms.statistics.multilayer_statistics.entropy_of_multiplexity(network: Any) → float`

Calculate entropy of multiplexity (H).

Formula: $H = - \sum p \log_2(p)$, where $p = E_i / E$

Shannon entropy of layer contributions; measures layer diversity.

Variables:

p = proportion of edges in layer i E_i = number of edges in layer i $\log_2$ gives entropy in bits

Properties:

$H = 0$ when all edges are in one layer (minimum entropy/diversity) $H = \log_2(L)$ when edges are uniformly distributed across L layers (maximum entropy)

Parameters

network – py3plex multi_layer_network object

Returns

Entropy value in bits

Examples

```
>>> entropy = entropy_of_multiplexity(network)
```

Reference:

De Domenico et al. (2013), Shannon (1948)

```
py3plex.algorithms.statistics.multilayer_statistics.inter_layer_assortativity(network: Any,
                                                                              layer_i: str,
                                                                              layer_j: str)
→ float
```

Calculate inter-layer assortativity (r).

Formula: $r^{\wedge} = \text{cov}(k^{\wedge}, k^{\wedge}) / (\) = \text{corr}(k^{\wedge}, k^{\wedge})$

Measures whether nodes with similar degrees tend to connect across different layers.

Variables:

$k^{\wedge}$ = degree vector in layer $k^{\wedge}$ = degree vector in layer , = standard deviations of degrees in layers and
Equivalent to Pearson correlation of degree vectors

Parameters

- **network** – py3plex multi_layer_network object
- **layer_i** – First layer identifier ()
- **layer_j** – Second layer identifier ()

Returns

Assortativity coefficient

Examples

```
>>> assort = inter_layer_assortativity(network, 'L1', 'L2')
```

Reference:

Newman (2002), Nicosia & Latora (2015)

```
py3plex.algorithms.statistics.multilayer_statistics.inter_layer_coupling_strength(network:
                                                                              Any,
                                                                              layer_i:
                                                                              str,
                                                                              layer_j:
                                                                              str) →
float
```

Calculate inter-layer coupling strength ($C^{\wedge}$).

Formula: $C^{\wedge} = (1/N_{_}) w^{\wedge}$

Average weight of inter-layer connections between corresponding nodes in two layers. Quantifies cross-layer connectivity.

Variables:

$N_{_}$ = number of nodes present in both layers and $w^{\wedge}$ = weight of inter-layer edge connecting node i in layer to node i in layer

Parameters

- **network** – py3plex multi_layer_network object
- **layer_i** – First layer identifier ()
- **layer_j** – Second layer identifier ()

Returns

Average coupling strength

Examples

```
>>> coupling = inter_layer_coupling_strength(network, 'L1', 'L2')
```

Reference:

De Domenico et al. (2013), Physical Review X 3(4), 041022

Contracts:

- Precondition: network must not be None
- Precondition: layer_i and layer_j must be non-empty strings
- Postcondition: result is non-negative (weights are non-negative)
- Postcondition: result is not NaN

py3plex.algorithms.statistics.multilayer_statistics.**inter_layer_degree_correlation**(*network:*
Any,
layer_i:
str,
layer_j:
str) →
float

Calculate inter-layer degree correlation ($r^{\wedge}$).

Formula: $r^{\wedge} = (k - k)(k - k) / [((k - k)^2) ((k - k)^2)]$

Pearson correlation of node degrees between two layers; reveals if highly connected nodes in one layer are also central in others.

Variables:

k = degree of node i in layer k = mean degree in layer Sum over nodes present in both layers

Parameters

- **network** – py3plex multi_layer_network object
- **layer_i** – First layer identifier ()
- **layer_j** – Second layer identifier ()

Returns

Pearson correlation coefficient between -1 and 1

Examples

```
>>> corr = inter_layer_degree_correlation(network, 'L1', 'L2')
```

Reference:

Battiston et al. (2014), Nicosia & Latora (2015)


```
py3plex.algorithms.statistics.multilayer_statistics.inter_layer_dependence_entropy(network:
Any,
layer_i:
str,
layer_j:
str) →
float
```

Calculate inter-layer dependence entropy (H_{dep}).

Formula: $H_{dep} = -p \log_2(p)$, where p is the proportion of inter-layer edges for each node n connecting layers i and j .

Measures heterogeneity in how nodes couple the two layers; high entropy indicates diverse coupling patterns, low entropy indicates uniform coupling.

Variables:

p = proportion of inter-layer edges incident to node n Total over all nodes connecting the two layers

Parameters

- **network** – py3plex multi_layer_network object
- **layer_i** – First layer identifier
- **layer_j** – Second layer identifier

Returns

Entropy value in bits

Examples

```
>>> entropy = inter_layer_dependence_entropy(network, 'L1', 'L2')
>>> print(f"Inter-layer dependence entropy: {entropy:.3f}")
```

Reference:

De Domenico et al. (2015), “Ranking in interconnected multilayer networks”

```
py3plex.algorithms.statistics.multilayer_statistics.interdependence(network: Any, sample_size:
int = 100) → float
```

Calculate interdependence ().

Formula: $= \langle d \rangle / \langle d \rangle$

Quantifies how much shortest-path communication depends on inter-layer connections.

Variables:

d = shortest path from node i to node j in the full multilayer network $d = (1/L) \sum d_{ij}$ d is the average shortest path across individual layers $\langle \cdot \rangle$ = average over sampled node pairs

Interpretation:

< 1 : multilayer connectivity reduces path lengths (positive interdependence) $= 1$: inter-layer connections provide little benefit > 1 : multilayer structure increases path lengths (rare)

Parameters

- **network** – py3plex multi_layer_network object
- **sample_size** – Number of node pairs to sample for estimation

Returns

Interdependence ratio

Examples

```
>>> interdep = interdependence(network, sample_size=50)
```

Reference:

Gomez et al. (2013), Buldyrev et al. (2010)

`py3plex.algorithms.statistics.multilayer_statistics.interlayer_degree_correlation_matrix`(*network*: Any)
→
Tuple[ndarray, list]

Calculate inter-layer degree correlation matrix.

Computes Pearson correlation coefficients for node degrees between all pairs of layers, organized as a symmetric correlation matrix.

Formula: $\text{Matrix}[i,j] = r^{ij} = \text{corr}(k^i, k^j)$

where k^i and k^j are degree vectors for layers i and j over common nodes.

Properties:

- Diagonal elements are 1.0 (self-correlation)
- Off-diagonal elements in $[-1, 1]$
- Symmetric matrix
- Positive values indicate positive degree correlation
- Negative values indicate negative degree correlation

Parameters

network – py3plex multi_layer_network object

Returns

- `correlation_matrix`: 2D numpy array of shape (num_layers, num_layers)
- `layer_labels`: List of layer names corresponding to matrix indices

Return type

Tuple of (correlation_matrix, layer_labels)

Examples

```
>>> corr_matrix, layers = interlayer_degree_correlation_matrix(network)
>>> import matplotlib.pyplot as plt
>>> import seaborn as sns
>>> sns.heatmap(corr_matrix, annot=True, xticklabels=layers,
...             yticklabels=layers, cmap='coolwarm', center=0,
...             vmin=-1, vmax=1)
>>> plt.title('Inter-layer Degree Correlation Matrix')
>>> plt.show()
```

Reference:

Nicosia & Latora (2015), “Measuring and modeling correlations in multiplex networks” Battiston et al. (2014), “Structural measures for multiplex networks”

`py3plex.algorithms.statistics.multilayer_statistics.layer_connectivity_entropy`(*network: Any, layer: str*) → float

Calculate entropy of layer connectivity (H_connectivity).

Formula: $H_c = - (k/k) \log_2(k/k)$

Shannon entropy of degree distribution within a layer; measures heterogeneity of node connectivity patterns.

Variables:

k = degree of node i in the layer k = sum of all degrees ($2 * \text{edges}$ for undirected)

Properties:

$H_c = 0$ when all nodes have the same degree (uniform distribution) H_c is maximized when degree distribution is highly uneven

Parameters

- **network** – py3plex multi_layer_network object
- **layer** – Layer identifier

Returns

Entropy value in bits

Examples

```
>>> from py3plex.core import multinet
>>> network = multinet.multi_layer_network(directed=False)
>>> network.add_edges([
...     ['A', 'L1', 'B', 'L1', 1],
...     ['B', 'L1', 'C', 'L1', 1]
... ], input_type='list')
>>> entropy = layer_connectivity_entropy(network, 'L1')
>>> print(f"Connectivity entropy: {entropy:.3f}")
```

Reference:

Solé-Ribalta et al. (2013), “Spectral properties of complex networks” Shannon (1948), “A Mathematical Theory of Communication”

`py3plex.algorithms.statistics.multilayer_statistics.layer_density`(*network: Any, layer: str*) → float

Calculate layer density ().

Formula: $= (2E) / (N(N - 1))$ [undirected]

$= E / (N(N - 1))$ [directed]

Measures the fraction of possible edges present in a specific layer, indicating how densely connected that layer is.

Variables:

E = number of edges in layer N = number of nodes in layer

Parameters

- **network** – py3plex multi_layer_network object
- **layer** – Layer identifier

Returns

Density value between 0 and 1

Examples

```
>>> from py3plex.core import multinet
>>> network = multinet.multi_layer_network(directed=False)
>>> network.add_edges([
...     ['A', 'L1', 'B', 'L1', 1],
...     ['B', 'L1', 'C', 'L1', 1]
... ], input_type='list')
>>> density = layer_density(network, 'L1')
>>> print(f"Layer L1 density: {density:.3f}")
```

Reference:

Kivelä et al. (2014), J. Complex Networks 2(3), 203-271

Contracts:

- Precondition: network must not be None
- Precondition: layer must be a non-empty string
- Postcondition: result is in [0, 1] (fundamental property of density)
- Postcondition: result is not NaN

```
py3plex.algorithms.statistics.multilayer_statistics.layer_influence_centrality(network:
Any, layer:
str, method:
str =
'coupling',
sample_size:
int = 100) →
float
```

Calculate layer influence centrality (I).

Formula (coupling): $I = C^{\wedge} / (L-1)$ Formula (flow): $I = F^{\wedge} / (L-1)$

Quantifies how much a layer influences other layers through inter-layer connections (coupling method) or information flow (flow method).

Variables:

$C^{\wedge}$ = inter-layer coupling strength between layers and $F^{\wedge}$ = flow from layer to layer (random walk transition probability) L = total number of layers

Properties:

Higher values indicate layers that strongly influence others Useful for identifying critical layers in the multilayer structure

Parameters

- **network** – py3plex multi_layer_network object
- **layer** – Layer identifier

- **method** – ‘coupling’ for structural influence, ‘flow’ for dynamic influence
- **sample_size** – Number of random walk steps for flow simulation

Returns

Influence centrality value

Examples

```
>>> influence = layer_influence_centrality(network, 'L1', method='coupling')
>>> print(f"Layer L1 influence: {influence:.3f}")
```

Reference:

Cozzo et al. (2013), “Mathematical formulation of multilayer networks” De Domenico et al. (2014), “Identifying modular flows”

py3plex.algorithms.statistics.multilayer_statistics.**layer_redundancy_coefficient**(*network: Any, layer_i: str, layer_j: str*) → float

Calculate layer redundancy coefficient.

Measures the proportion of edges in one layer that are redundant (also present) in another layer. Values close to 1 indicate high redundancy, while values close to 0 indicate complementary layers.

Formula: $R = |E \cap E| / |E|$

where E and E are edge sets of layers i and j .

Parameters

- **network** – py3plex multi_layer_network object
- **layer_i** – First layer identifier
- **layer_j** – Second layer identifier

Returns

Redundancy coefficient between 0 and 1

Examples

```
>>> redundancy = layer_redundancy_coefficient(network, 'social', 'work')
>>> print(f"Redundancy: {redundancy:.2%}")
```

Reference:

Nicosia & Latora (2015), “Measuring and modeling correlations in multiplex networks”

py3plex.algorithms.statistics.multilayer_statistics.**layer_similarity**(*network: Any, layer_i: str, layer_j: str, method: str = 'cosine'*) → float

Calculate layer similarity (S^A).

Formula: $S^A = \langle A, A \rangle / (\|A\| \|A\|) = AA / ((A)^2) ((A)^2)$

Cosine or Jaccard similarity between adjacency matrices of two layers.

Variables:

A, A = adjacency matrices for layers and $\langle \cdot, \cdot \rangle$ = Frobenius inner product $\| \cdot \|$ = Frobenius norm

Parameters

- **network** – py3plex multi_layer_network object
- **layer_i** – First layer identifier ()
- **layer_j** – Second layer identifier ()
- **method** – ‘cosine’ or ‘jaccard’

Returns

Similarity value between 0 and 1

Examples

```
>>> similarity = layer_similarity(network, 'L1', 'L2', method='cosine')
```

Reference:

De Domenico et al. (2013), Physical Review X 3(4), 041022

```
py3plex.algorithms.statistics.multilayer_statistics.multilayer_betweenness_surface(network:  
Any,  
normalized:  
bool =  
True,  
weight:  
str |  
None =  
None)  
→  
ndarray
```

Calculate multilayer betweenness surface (tensor representation).

Computes betweenness centrality for each node-layer pair and organizes the results as a 2D array (nodes $\times$ layers) that can be visualized as a heatmap or surface plot.

Formula: $\text{Surface}[i,] = B$

where B is the betweenness centrality of node i in layer .

Parameters

- **network** – py3plex multi_layer_network object
- **normalized** – Whether to normalize betweenness values
- **weight** – Edge weight attribute name (None for unweighted)

Returns

2D numpy array of shape (num_nodes, num_layers) containing betweenness values. Also returns tuple of (node_labels, layer_labels) for axis labeling.

Examples

```
>>> surface, (nodes, layers) = multilayer_betweenness_surface(network)
>>> import matplotlib.pyplot as plt
>>> plt.imshow(surface, aspect='auto', cmap='viridis')
>>> plt.xlabel('Layers')
>>> plt.ylabel('Nodes')
>>> plt.xticks(range(len(layers)), layers)
>>> plt.yticks(range(len(nodes)), nodes)
>>> plt.colorbar(label='Betweenness Centrality')
>>> plt.title('Multilayer Betweenness Surface')
>>> plt.show()
```

Reference:

De Domenico et al. (2015), “Structural reducibility of multilayer networks”

py3plex.algorithms.statistics.multilayer_statistics.**multilayer_clustering_coefficient**(*network*: Any, *node*: Any | None = None) → float | Dict[Any, float]

Calculate multilayer clustering coefficient (C).

Formula: $C = T / T$

Extends transitivity to account for triangles that span multiple layers.

Variables:

T = number of closed triplets (triangles) involving node i across all layers T = maximum possible triplets = $k(k - 1)/2$ for undirected networks Average over all nodes: $C = (1/N) \sum C_i$

Parameters

- **network** – py3plex multi_layer_network object
- **node** – If specified, compute for single node; otherwise compute for all

Returns

Clustering coefficient value or dict of values per node

Examples

```
>>> clustering = multilayer_clustering_coefficient(network)
>>> node_clustering = multilayer_clustering_coefficient(network, node='A')
```

Reference:

Battiston et al. (2014), Section III.C

```
py3plex.algorithms.statistics.multilayer_statistics.multilayer_modularity(network: Any,
                                                                           communities:
                                                                           Dict[Tuple[Any,
                                                                           Any], int], gamma:
                                                                           float | Dict[Any,
                                                                           float] = 1.0, omega:
                                                                           float | ndarray =
                                                                           1.0, weight: str =
                                                                           'weight') → float
```

Calculate multilayer modularity (Q).

This is a wrapper for the existing `multilayer_modularity` implementation in `py3plex.algorithms.community_detection.multilayer_modularity`.

Formula: $Q = (1/2) [(A - P) +] (g, g)$

Extension of Newman-Girvan modularity to multiplex networks (Mucha et al., 2010). Measures community quality across layers.

Variables:

w_{ij} = total edge weight in supra-network A = adjacency matrix element for layer $P = \text{kk}/(2m)$ is the null model (configuration model) γ = resolution parameter for layer ω = inter-layer coupling strength δ_{ij} = Kronecker delta (1 if $i=j$, 0 otherwise) $\delta_{ij}^{(l)}$ = Kronecker delta (1 if $i=j$, 0 otherwise) $(g, g) = 1$ if node i in layer l and node j in layer l are in same community

Parameters

- **network** – py3plex multi_layer_network object
- **communities** – Dictionary mapping (node, layer) tuples to community IDs
- **gamma** – Resolution parameter(s)
- **omega** – Inter-layer coupling strength
- **weight** – Edge weight attribute

Returns

Modularity value Q

Examples

```
>>> communities = {('A', 'L1'): 0, ('B', 'L1'): 0, ('C', 'L1'): 1}
>>> Q = multilayer_modularity(network, communities)
```

Reference:

Mucha et al. (2010), Science 328(5980), 876-878

```
py3plex.algorithms.statistics.multilayer_statistics.multilayer_motif_frequency(network:
                                                                           Any,
                                                                           motif_size:
                                                                           int = 3) →
                                                                           Dict[str,
                                                                           float]
```

Calculate multilayer motif frequency (f).

Formula: $f = n / n$

Frequency of recurring subgraph patterns across layers.

Variables:

n = count of motif type m n = total count of all motifs

Note: This is a simplified implementation counting basic patterns (intra-layer vs. inter-layer triangles). Complete multilayer motif enumeration includes many more configurations and is computationally expensive.

Parameters

- **network** – py3plex multi_layer_network object
- **motif_size** – Size of motifs to count (default: 3 for triangles)

Returns

Dictionary of motif type frequencies

Examples

```
>>> motifs = multilayer_motif_frequency(network, motif_size=3)
```

Reference:

Battiston et al. (2014), Section IV

```
py3plex.algorithms.statistics.multilayer_statistics.multiplex_betweenness centrality(network:
Any,
normalized:
bool
=
True,
weight:
str |
None
=
None)
→
Dict[Tuple[Any,
Any],
float]
```

Calculate multiplex betweenness centrality.

Computes betweenness centrality on the supra-graph, accounting for paths that traverse inter-layer couplings. This extends the standard betweenness definition to multiplex networks where paths can cross layers.

Formula: $B = ((i) /)$

where i is the total number of shortest paths from s to t , and (i) is the number of those paths passing through node i in layer $.$

Parameters

- **network** – py3plex multi_layer_network object
- **normalized** – Whether to normalize by the number of node pairs
- **weight** – Edge weight attribute name (None for unweighted)

Returns

Dictionary mapping (node, layer) tuples to betweenness centrality values

Examples

```
>>> betweenness = multiplex_betweenness_centrality(network)
>>> top_nodes = sorted(betweenness.items(), key=lambda x: x[1], reverse=True)[:5]
```

Reference:

De Domenico et al. (2015), “Structural reducibility of multilayer networks”

`py3plex.algorithms.statistics.multilayer_statistics.multiplex_closeness_centrality`(*network*: Any, *normalized*: bool = True, *weight*: str | None = None) → Dict[Tuple[Any, Any], float]

Calculate multiplex closeness centrality.

Computes closeness centrality on the supra-graph, where shortest paths can traverse inter-layer edges. This captures how quickly a node-layer can reach all other node-layers in the multiplex network.

Formula: $C = (N * L - 1) / d(i, j)$

where $d(i, j)$ is the shortest path distance from node i in layer to node j in layer , and $N * L$ is the total number of node-layer pairs.

Parameters

- **network** – py3plex multi_layer_network object
- **normalized** – Whether to normalize by network size
- **weight** – Edge weight attribute name (None for unweighted)

Returns

Dictionary mapping (node, layer) tuples to closeness centrality values

Examples

```
>>> closeness = multiplex_closeness_centrality(network)
>>> central_nodes = {k: v for k, v in closeness.items() if v > 0.5}
```

Reference:

De Domenico et al. (2015), “Structural reducibility of multilayer networks”

`py3plex.algorithms.statistics.multilayer_statistics.multiplex_rich_club_coefficient`(*network*: Any, *k*: int, *normalized*: bool = True) → float

Calculate multiplex rich-club coefficient.

Measures the tendency of high-degree nodes to be more densely connected to each other than expected by chance, accounting for the multiplex structure.

Formula: $\phi(k) = E(>k) / (N(>k) * (N(>k)-1) / 2)$

where $E(>k)$ is the number of edges among nodes with overlapping degree $> k$, and $N(>k)$ is the number of such nodes.

Parameters

- **network** – py3plex multi_layer_network object
- **k** – Degree threshold
- **normalized** – Whether to normalize by random expectation

Returns

Rich-club coefficient value

Examples

```
>>> rich_club = multiplex_rich_club_coefficient(network, k=10)
>>> print(f"Rich-club coefficient: {rich_club:.3f}")
```

Reference:

Alstott et al. (2014), “powerlaw: A Python Package for Analysis of Heavy-Tailed Distributions” Extended to multiplex networks

`py3plex.algorithms.statistics.multilayer_statistics.node_activity`(*network*: Any, *node*: Any) → float

Calculate node activity (a).

Formula: $a = (1/L) \sum_{v \in V} V_v$

Fraction of layers in which node i is active (has at least one connection).

Variables:

L = total number of layers ($v \in V$) = indicator function (1 if node i is active in layer, 0 otherwise) V = set of active nodes in layer

Parameters

- **network** – py3plex multi_layer_network object
- **node** – Node identifier

Returns

Activity value between 0 and 1

Examples

```
>>> activity = node_activity(network, 'A')
```

Reference:

Kivelä et al. (2014), J. Complex Networks 2(3), 203-271

Contracts:

- Precondition: network must not be None
- Precondition: node must not be None
- Postcondition: result is in [0, 1] (fraction of layers)
- Postcondition: result is not NaN

`py3plex.algorithms.statistics.multilayer_statistics.percolation_threshold`(*network*: Any, *removal_strategy*: str = 'random', *trials*: int = 10) → float

Estimate percolation threshold for the multiplex network.

Determines the fraction of nodes that must be removed before the network fragments into disconnected components. Uses sampling to estimate threshold.

Parameters

- **network** – py3plex multi_layer_network object
- **removal_strategy** – ‘random’, ‘degree’, or ‘betweenness’
- **trials** – Number of trials for averaging

Returns

Estimated percolation threshold (fraction of nodes)

Examples

```
>>> threshold = percolation_threshold(network, removal_strategy='degree')
>>> print(f"Percolation threshold: {threshold:.2%}")
```

Reference:

Buldyrev et al. (2010), “Catastrophic cascade of failures in interdependent networks”

`py3plex.algorithms.statistics.multilayer_statistics.resilience`(*network*: Any, *perturbation_type*: str = 'layer_removal', *perturbation_param*: str | float = None) → float

Calculate resilience (R).

Formula: $R = S' / S_0$

Ratio of largest connected component after perturbation to original size.

Variables:

S_0 = size of largest connected component in original network S' = size of largest connected component after perturbation

Perturbation types:

1. Layer removal: Remove all nodes/edges in a specific layer
2. Coupling removal: Remove a fraction of inter-layer edges

Properties:

$R = 1$ indicates full resilience (no impact from perturbation) $R = 0$ indicates complete fragmentation $0 < R < 1$ indicates partial resilience

Parameters

- **network** – py3plex multi_layer_network object
- **perturbation_type** – ‘layer_removal’ or ‘coupling_removal’
- **perturbation_param** – Layer to remove or fraction of inter-layer edges

Returns

Resilience ratio between 0 and 1

Examples

```
>>> r = resilience(network, 'layer_removal', perturbation_param='L1')
>>> r = resilience(network, 'coupling_removal', perturbation_param=0.5)
```

Reference:

Buldyrev et al. (2010), Nature 464, 1025-1028

py3plex.algorithms.statistics.multilayer_statistics.**supra_laplacian_spectrum**(network: Any, k: int = 10) → ndarray

Calculate supra-Laplacian spectrum ().

Formula: = -

Eigenvalue spectrum of the supra-Laplacian matrix; captures diffusion properties. Uses sparse eigenvalue computation when beneficial.

Variables:

= supra-adjacency matrix ($NL \times NL$ block matrix containing all layers and inter-layer couplings) = supra-degree matrix (diagonal matrix with row sums of) = supra-Laplacian matrix = $\{0, 1, \dots, 1\}$ with $0 = 0 \ 1 \dots \ 1$

Parameters

- **network** – py3plex multi_layer_network object
- **k** – Number of smallest eigenvalues to compute

Returns

Array of k smallest eigenvalues

Examples

```
>>> spectrum = supra_laplacian_spectrum(network, k=10)
```

Reference:

De Domenico et al. (2013), Gomez et al. (2013)

Notes

- Uses sparse eigsh() for sparse matrices (more efficient)
- Falls back to dense computation for small matrices ($n < 100$) or when k is large relative to n
- Laplacian is always symmetric for undirected graphs (PSD with smallest eigenvalue = 0)

`py3plex.algorithms.statistics.multilayer_statistics.targeted_layer_removal`(*network*: Any, *layer*: str, *return_resilience*: bool = False) → Any | Tuple[Any, float]

Simulate targeted removal of an entire layer.

Removes all edges in a specified layer and returns the modified network or resilience score.

Parameters

- **network** – py3plex multi_layer_network object
- **layer** – Layer identifier to remove
- **return_resilience** – If True, return resilience score instead of network

Returns

Modified network or resilience score

Examples

```
>>> resilience = targeted_layer_removal(network, 'social', return_resilience=True)
>>> print(f"Resilience after removing social layer: {resilience:.3f}")
```

Reference:

Buldyrev et al. (2010), “Catastrophic cascade of failures”

`py3plex.algorithms.statistics.multilayer_statistics.unique_redundant_edges`(*network*: Any, *layer_i*: str, *layer_j*: str) → Tuple[int, int]

Count unique and redundant edges between two layers.

Returns the number of edges unique to the first layer and the number of edges present in both layers (redundant).

Parameters

- **network** – py3plex multi_layer_network object
- **layer_i** – First layer identifier
- **layer_j** – Second layer identifier

Returns

Tuple of (unique_edges, redundant_edges)

Examples

```
>>> unique, redundant = unique_redundant_edges(network, 'social', 'work')
>>> print(f"Unique: {unique}, Redundant: {redundant}")
```

`py3plex.algorithms.statistics.multilayer_statistics.versatility_centrality`(*network: Any*,
centrality_type: str = 'degree',
alpha: Dict[str, float] | None = None) →
Dict[Any, float]

Calculate versatility centrality (V).

Formula: $V = w C$

Weighted combination of node centrality values across layers; measures overall influence.

Variables:

w = weight for layer (typically $1/L$ for uniform weighting, $w = 1$) C = centrality of node i in layer (can be degree, betweenness, closeness, etc.)

Parameters

- **network** – py3plex multi_layer_network object
- **centrality_type** – Type of centrality ('degree', 'betweenness', 'closeness')
- **alpha** – Layer weights (default: uniform weights)

Returns

Dictionary mapping nodes to versatility centrality values

Examples

```
>>> versatility = versatility_centrality(network, centrality_type='degree')
```

Reference:

De Domenico et al. (2015), Nature Communications 6, 6868

`py3plex.algorithms.statistics.topology.basic_pl_stats`(*degree_sequence: List[int]*) → Tuple[float,
float]

Calculate basic power law statistics for a degree sequence.

Parameters

degree_sequence – Degree sequence of individual nodes

Returns

Tuple of (alpha, sigma) values

`py3plex.algorithms.statistics.topology.plot_power_law`(*degree_sequence: List[int]*, *title: str*, *xlabel: str*, *plabel: str*, *ylabel: str = 'Number of nodes'*, *formula_x: int = 70*, *formula_y: float = 0.05*, *show: bool = True*, *use_normalization: bool = False*) → Any

```
py3plex.algorithms.statistics.correlation_networks.default_correlation_to_network(matrix:  
                                                                                    ndarray,  
                                                                                    in-  
                                                                                    put_type:  
                                                                                    str =  
                                                                                    'matrix',  
                                                                                    prepro-  
                                                                                    cess: str  
                                                                                    = 'stan-  
                                                                                    dard') →  
                                                                                    ndarray
```

Convert correlation matrix to network using optimal thresholding.

Parameters

- **matrix** – Input data matrix
- **input_type** – Type of input (default: “matrix”)
- **preprocess** – Preprocessing method (default: “standard”)

Returns

Binary adjacency matrix

```
py3plex.algorithms.statistics.correlation_networks.pick_threshold(matrix: ndarray) → float
```

Pick optimal threshold for correlation network construction.

Parameters

matrix – Input data matrix

Returns

Optimal threshold value

```
py3plex.algorithms.statistics.basic_statistics.core_network_statistics(G: Graph, labels: Any |  
                                                                                   None = None, name: str  
                                                                                   = 'example') →  
                                                                                   DataFrame
```

Compute core statistics for a network.

Parameters

- **G** – NetworkX graph to analyze
- **labels** – Optional label matrix with shape attribute
- **name** – Name identifier for the network (default: “example”)

Returns

DataFrame containing network statistics

```
py3plex.algorithms.statistics.basic_statistics.ensure(*args, **kwargs)
```

```
py3plex.algorithms.statistics.basic_statistics.identify_n_hubs(G: Graph, top_n: int = 100,  
                                                                 node_type: str | None = None) →  
                                                                 Dict[Any, int]
```

Identify the top N hub nodes in a network based on degree centrality.

Parameters

- **G** – NetworkX graph to analyze
- **top_n** – Number of top hubs to return (default: 100)

- **node_type** – Optional filter for specific node type

Returns

Dictionary mapping node identifiers to their degree values

Contracts:

- Precondition: top_n must be positive
- Postcondition: result has at most top_n entries
- Postcondition: all degree values are non-negative integers

```
py3plex.algorithms.statistics.basic_statistics.require(*args, **kwargs)
```

Multilayer Algorithms**Multilayer/Multiplex Network Centrality Measures**

This module implements various centrality measures for multilayer and multiplex networks, following standard definitions from multilayer network analysis literature.

Weight Handling Notes:

For path-based centralities (betweenness, closeness): - Edge weights from the supra-adjacency matrix are converted to distances (inverse of weight) - NetworkX algorithms use these distances for shortest path computation - betweenness centrality: weight parameter specifies the edge attribute for path computation - closeness centrality: distance parameter specifies the edge attribute for path computation

For disconnected graphs: - closeness centrality uses wf_improved parameter (Wasserman-Faust scaling) by default - When wf_improved=True, scores are normalized by reachable nodes only - When wf_improved=False, unreachable nodes contribute infinite distance

Weight Constraints: - Weights should be positive (> 0) for shortest path algorithms - Zero or negative weights will cause undefined behavior - For unweighted analysis, use weighted=False parameters

Authors: py3plex contributors Date: 2025

```
class py3plex.algorithms.multilayer_algorithms.centrality.MultilayerCentrality(network:
Any)
```

Bases: object

Class for computing centrality measures on multilayer networks.

This class provides implementations of various centrality measures specifically designed for multilayer/multiplex networks, including degree-based, eigenvector-based, and path-based measures.

accessibility centrality(h=2)

Compute accessibility centrality (entropy-based reach within h steps).

Accessibility measures the diversity of nodes reachable within h steps using entropy of the probability distribution.

Access_r = $\exp(H_r)$ where H_r is the entropy of the h-step distribution

Parameters

h – Number of steps (default: 2)

Returns

{(node, layer): accessibility}

Return type
dict

Note

Accessibility is measured as the effective number of h-step destinations, using the entropy of the random walk distribution.

aggregate_to_node_level(*node_layer_centralities*, *method*='sum', *weights*=None)

Aggregate node-layer centralities to node level.

Parameters

- **node_layer_centralities** – dict with {(node, layer): value} entries
- **method** – 'sum', 'mean', 'max', 'weighted_sum'
- **weights** – dict with {layer: weight} for weighted_sum method

Returns

{node: aggregated_value}

Return type
dict

bridging_centrality()

Compute bridging centrality for nodes in the supra-graph.

Bridging centrality combines betweenness with the bridging coefficient, which measures how much a node connects sparse regions of the network.

$\text{Bridging}(u) = B(u) * \text{BCoeff}(u)$ where $\text{BCoeff}(u) = (1 / k_u) * \sum_{v \in N(u)} [1 / k_v]$

Returns

{(node, layer): bridging_centrality}

Return type
dict

Note

Nodes with higher bridging centrality act as important bridges connecting different parts of the network.

collective_influence(*radius*=2)

Compute collective influence (CI_) for multiplex networks.

Collective influence identifies influential spreaders by considering not just immediate neighbors but also nodes at distance .

$\text{CI}_-(u) = (k_u - 1) * \sum_{v \in \text{Ball}_-(u)} (k_v - 1)$

Parameters

radius – Radius for the ball boundary (default: 2)

Returns

{(node, layer): collective_influence}

Return type
dict

Note

Uses overlapping degree across all layers for each physical node.

communicability_betweenness centrality(*normalized=True*)

Compute communicability betweenness centrality (Estrada-style).

This measure quantifies how much a node contributes to the communicability between other pairs of nodes. It uses the matrix exponential to account for all walks between nodes.

Returns

{(node, layer): communicability_betweenness}

Return type

dict

Note

This is computationally expensive as it requires computing the matrix exponential multiple times. For large networks, this may take significant time.

current_flow_betweenness centrality()

Compute current-flow betweenness centrality via supra Laplacian pseudoinverse.

This measure is based on the electrical current flow through each node.

Returns

{(node, layer): current_flow_betweenness}

Return type

dict

current_flow_closeness centrality()

Compute current-flow closeness centrality via supra Laplacian pseudoinverse.

This measure is based on the resistance distance in electrical networks.

Returns

{(node, layer): current_flow_closeness}

Return type

dict

edge_betweenness centrality(*normalized=True*)

Compute edge betweenness centrality on the supra-graph.

Edge betweenness measures the fraction of shortest paths that pass through each edge.

Returns

{(source, target): edge_betweenness}

Return type

dict

Note

This returns edge-level centrality, not node-level.

flow_betweenness centrality(*samples=100*)

Compute flow betweenness based on maximum flow.

Flow betweenness measures how much flow passes through a node when maximizing flow between random source-target pairs.

Parameters

samples – Number of source-target pairs to sample (default: 100)

Returns

{(node, layer): flow_betweenness}

Return type

dict

Note

This uses NetworkX's maximum flow algorithms. For large networks, this can be computationally expensive.

harmonic_closeness centrality()

Compute harmonic closeness centrality on the supra-graph.

Harmonic closeness handles disconnected graphs better than standard closeness by summing the reciprocals of distances instead of taking reciprocal of sum.

$HC(u) = \sum_v (1 / d(u,v))$ for finite distances

Returns

{(node, layer): harmonic_closeness}

Return type

dict

Note

This measure naturally handles disconnected components as unreachable nodes contribute 0 (instead of infinity) to the sum.

hits centrality(*max_iter=1000, tol=1e-06*)

Compute HITS (hubs and authorities) centrality on the supra-graph.

For undirected networks, this equals eigenvector centrality. For directed networks, computes separate hub and authority scores.

Parameters

- **max_iter** – Maximum number of iterations.
- **tol** – Tolerance for convergence.

Returns

If directed network: {'hubs': {(node, layer): score}, 'authorities': {(node, layer): score}}

If undirected network: {(node, layer): score} (equivalent to eigenvector centrality)

Return type

dict

information centrality()

Compute Information Centrality (Stephenson-Zelen style) on the supra-graph.

Information centrality measures the importance of a node based on the information flow through the network. It uses the inverse of a modified Laplacian matrix.

For each physical node u:

$$I(u) = 1 / \text{mean}_{\{a \text{ in } U\}} [G[a,a] - (2/(N*L)) * \sum_b G[a,b] + (1/(N*L)^2) * \sum_{\{b,c\}} G[b,c]]$$

where G is the inverse of $B = L + (1/(N*L)) * 1 * 1^T$

Returns

{(node, layer): information centrality}

Return type

dict

Note

This implementation uses NetworkX's `information centrality` for computation. Falls back to harmonic closeness if information centrality computation fails.

katz_bonacich centrality(alpha=0.1, beta=None)

Compute Katz-Bonacich centrality on the supra-graph.

$$z = \lim_{t \rightarrow \infty} M^t b = (I - M)^{-1} b$$

Parameters

- **alpha** – Attenuation parameter (should be $< 1/(M)$). If None, automatically computes a safe value as $0.85/(M)$ where (M) is the spectral radius.
- **beta** – Exogenous preference vector. If None, uses vector of ones.

Returns

{(node, layer): centrality_value}

Return type

dict

layer_degree centrality(layer: str | None = None, weighted: bool = False, direction: str = 'out') → Dict[str | Tuple[str, str], float]

Compute layer-specific degree (or strength) centrality.

For undirected networks:

$$k^u_{ij} = \sum_j 1(A^u_{ij} > 0) \text{ [unweighted]} \quad s^u_{ij} = \sum_j A^u_{ij} \text{ [weighted]}$$

For directed networks:

$$k^{[out]}_{ij} = \sum_j 1(A^{[out]}_{ij} > 0) \text{ [out-degree]} \quad k^{[in]}_{ij} = \sum_j 1(A^{[in]}_{ji} > 0) \text{ [in-degree]}$$

Parameters

- **layer** – Layer to compute centrality for. If None, compute for all layers.
- **weighted** – If True, compute strength instead of degree.
- **direction** – 'out', 'in', or 'both' for directed networks.

Returns

{(node, layer): centrality_value} if layer is None,
{node: centrality_value} if layer is specified.

Return type

dict

load_centrality()

Compute load centrality (shortest-path load).

Load centrality measures the fraction of shortest paths that pass through each node, counting all paths (not just unique pairs).

Load[k] = sum over all (s,t) pairs of [number of shortest paths through k / total shortest paths]

Returns

{(node, layer): load_centrality}

Return type

dict

Note

This is similar to betweenness but counts all paths rather than normalizing by the number of node pairs.

local_efficiency_centrality()

Compute local efficiency centrality.

Local efficiency measures how efficiently information is exchanged among a node's neighbors when the node is removed. It quantifies the fault tolerance of the network.

$LE(u) = (1 / ((N_u) * (N_u - 1))) * \sum_{ij \in N_u} [1 / d(i,j)]$

Returns

{(node, layer): local_efficiency}

Return type

dict

Note

For nodes with less than 2 neighbors, local efficiency is 0.

lp_aggregated_centrality(layer_centralities, p=2, weights=None)

Compute Lp-aggregated per-layer centrality.

Aggregates per-layer centrality values using Lp norm.

$C_i = (\sum_{l=1..L} w_l * |c[i]|^p)^{1/p}$ for $p < \infty$ **$C_i = \max_l \{ w_l * |c[i]| \}$** for $p = \infty$

Parameters

- **layer_centralities** – dict of {layer: {node: centrality}} or {(node, layer): centrality}
- **p** – Lp norm parameter (default: 2). Use float('inf') for L-infinity norm.
- **weights** – dict of {layer: weight}. If None, uniform weights are used.

Returns

{node: aggregated_centrality}

Return type

dict

Note

This is a framework for aggregating any per-layer centrality measure. Input can be degree, PageRank, eigenvector, etc.

multilayer_betweenness centrality(*normalized=True, endpoints=False*)

Compute betweenness centrality on the supra-graph.

For each node-layer pair (i,), computes the fraction of shortest paths between all pairs of nodes that pass through (i,).

Parameters

- **normalized** – Whether to normalize the betweenness values.
- **endpoints** – Whether to include endpoints in path counts.

Returns

{(node, layer): betweenness centrality}

Return type

dict

Note

This is computationally expensive for large networks as it requires computing shortest paths between all pairs of nodes.

Weight handling: Edge weights from the supra-adjacency matrix are converted to distances (1/weight) for shortest path computation. Weights must be positive (> 0). Zero or negative weights will cause undefined behavior.

multilayer_closeness centrality(*normalized=True, wf_improved=True*)

Compute closeness centrality on the supra-graph.

For each node-layer pair (i,), computes: $C_c(i,) = (n-1) / \sum_{(j,)} d((i,),(j,))$

where $d((i,),(j,))$ is the shortest path distance in the supra-graph.

Parameters

- **normalized** – Whether to normalize by (n-1).
- **wf_improved** – If True, use Wasserman-Faust improved closeness scaling for disconnected graphs. Default is True. This affects the magnitude and ordering of scores in graphs with multiple components (e.g., low interlayer coupling). See NetworkX documentation for details.

Returns

{(node, layer): closeness centrality}

Return type

dict

Note

This implementation uses NetworkX's shortest path algorithms on the supra-graph representation. For large networks, this can be computationally expensive.

The `wf_improved` parameter controls how closeness is computed for nodes that cannot reach all other nodes. When True (default), uses the Wasserman-Faust formula that considers reachable nodes only, making scores comparable across disconnected components.

multiplex_coreness()

Alias for `multiplex_k_core` for compatibility.

Returns

`{(node, layer): core_number}`

Return type

dict

multiplex_eigenvector_centrality(*max_iter: int = 1000, tol: float = 1e-06*) → Dict

Compute multiplex eigenvector centrality (node-layer level).

$x = (1/_{\max}) * M * x$ where $x_{\{i\}}$ is the centrality of node i in layer , and $_{\max}$ is the spectral radius of the supra-adjacency matrix M .

Parameters

- **max_iter** – Maximum number of iterations.
- **tol** – Tolerance for convergence.

Returns

`{(node, layer): centrality_value}`

Return type

dict

multiplex_eigenvector_versatility(*max_iter: int = 1000, tol: float = 1e-06*) → Dict

Compute node-level eigenvector versatility.

$x_i = _x_{\{i\}}$

Parameters

- **max_iter** – Maximum number of iterations.
- **tol** – Tolerance for convergence.

Returns

`{node: versatility_value}`

Return type

dict

multiplex_k_core()

Compute multiplex k-core decomposition.

A node belongs to the k-core if it has at least k neighbors in the multilayer network. This implementation computes the core number for each node-layer pair.

Returns

`{(node, layer): core_number}`

Return type

dict

overlapping_degree_centrality(*weighted: bool = False*) → Dict

Compute overlapping degree/strength centrality (node level).

$k^{\text{over}}_i = \sum k_i$ [unweighted] $s^{\text{over}}_i = \sum s_i$ [weighted]

Parameters

weighted – If True, compute overlapping strength.

Returns

{node: centrality_value}

Return type

dict

pagerank_centrality(*damping=0.85, max_iter=1000, tol=1e-06*)

Compute PageRank centrality on the supra-graph.

Uses the standard PageRank algorithm on the supra-adjacency matrix representing the multilayer network. Properly handles dangling nodes (nodes with no outgoing edges) via teleportation.

This implementation preserves sparsity when possible for memory efficiency.

Parameters

- **damping** – Damping parameter (typically 0.85).
- **max_iter** – Maximum number of iterations.
- **tol** – Tolerance for convergence.

Returns

{(node, layer): centrality_value}

Return type

dict

Mathematical Invariants:

- PageRank values sum to 1.0 (within tol=1e-6)
- All values are non-negative
- Converges for strongly connected components or with teleportation

participation_coefficient(*weighted: bool = False*) → Dict

Compute participation coefficient across layers.

Measures how evenly a node's degree is distributed across layers: $P_i = 1 - \sum (k_i / k^{\text{over}}_i)^2$

Set $P_i = 0$ if $k^{\text{over}}_i = 0$.

Parameters

weighted – If True, use strength instead of degree.

Returns

{node: participation_coefficient}

Return type

dict

percolation_centrality(*edge_activation_prob=0.5, trials=100*)

Compute percolation centrality using bond percolation Monte Carlo simulation.

Percolation centrality measures the importance of a node based on its role in maintaining network connectivity under random edge failures.

Parameters

- **edge_activation_prob** – Probability that an edge is active (default: 0.5)
- **trials** – Number of Monte Carlo trials (default: 100)

Returns

{(node, layer): percolation_centrality}

Return type

dict

Note

Higher values indicate nodes that belong to larger connected components across different percolation realizations.

spreading_centrality(*beta=0.2, mu=0.1, trials=50, steps=100*)

Compute spreading (epidemic) centrality using SIR model.

Spreading centrality measures how influential a node is in spreading information or disease through the network.

Parameters

- **beta** – Infection rate (default: 0.2)
- **mu** – Recovery rate (default: 0.1)
- **trials** – Number of simulation trials per node (default: 50)
- **steps** – Maximum simulation steps (default: 100)

Returns

{(node, layer): spreading_centrality}

Return type

dict

Note

This measures the average outbreak size when seeding from each node. Requires discrete-time SIR simulation on the supra-graph.

subgraph_centrality()

Compute subgraph centrality via matrix exponential of the supra-adjacency matrix.

Subgraph centrality counts closed walks of all lengths starting and ending at each node. $SC_i = (e^A)_{ii}$ where A is the adjacency matrix.

Returns

{(node, layer): subgraph_centrality}

Return type

dict

supra_degree_centrality(*weighted: bool = False*) → Dict

Compute supra degree/strength centrality (node-layer level).

$k_{\{i\}} = \sum_j \mathbb{1}(M_{\{(i),(j)\}} > 0)$ [unweighted] $s_{\{i\}} = \sum_j M_{\{(i),(j)\}}$ [weighted]

Parameters

weighted – If True, compute strength instead of degree.

Returns

{(node, layer): centrality_value}

Return type

dict

total_communicability()

Compute total communicability via matrix exponential.

Total communicability is the row sum of the matrix exponential: $TC_i = \sum_j (e^A)_{ij}$

Returns

{(node, layer): total_communicability}

Return type

dict

```
py3plex.algorithms.multilayer_algorithms.centrality.communicability_centrality(supra_matrix:
                                                                              spmatrix,
                                                                              normalize:
                                                                              bool = True,
                                                                              use_sparse:
                                                                              bool = True,
                                                                              max_iter: int
                                                                              = 100, tol:
                                                                              float =
                                                                              1e-06) →
                                                                              ndarray
```

Compute communicability centrality for each node-layer pair.

Communicability centrality measures the weighted sum of all walks between nodes, with exponentially decaying weights for longer walks. It is computed as the row sum of the matrix exponential:

$$c_i = \sum_j (\exp(A))_{ij}$$

This implementation uses `scipy.sparse.linalg.expm_multiply` for efficient sparse matrix exponential computation.

Parameters

- **supra_matrix** – Sparse supra-adjacency matrix (n x n).
- **normalize** – If True, normalize output to sum to 1.
- **use_sparse** – If True, use sparse matrix operations. Falls back to dense for matrices smaller than 10000 elements.
- **max_iter** – Maximum number of iterations for sparse approximation (currently unused).
- **tol** – Tolerance for convergence (currently unused).

Returns

Communicability centrality scores for each node-layer pair (n,).

Return type

np.ndarray

Raises

Py3plexMatrixError – If matrix is invalid (non-square, empty, etc.).

Example

```
>>> from py3plex.core import random_generators
>>> net = random_generators.random_multiplex_ER(50, 3, 0.1)
>>> A = net.get_supra_adjacency_matrix()
>>> comm = communicability_centrality(A)
>>> print(f"Communicability centrality computed for {len(comm)} node-layer pairs")
```

```
py3plex.algorithms.multilayer_algorithms.centralities.compute_all_centralities(network, include_path_based=False, include_advanced=False, include_extended=False, preset=None, wf_improved=True)
```

Compute all available centrality measures for a multilayer network.

Parameters

- **network** – py3plex multi_layer_network object
- **include_path_based** – Whether to include computationally expensive path-based measures (betweenness, closeness). Default: False
- **include_advanced** – Whether to include advanced measures (HITS, current-flow, communicability, k-core). Default: False
- **include_extended** – Whether to include extended measures (information, accessibility, percolation, spreading, collective influence, load, flow betweenness, harmonic, bridging, local efficiency). Default: False
- **preset** – Convenience parameter to set all inclusion flags at once. Options: - ‘basic’: Only degree and eigenvector-based measures (default behavior) - ‘standard’: Includes path-based measures - ‘advanced’: Includes path-based and advanced measures - ‘all’: Includes all measures (path-based, advanced, and extended) - None: Use individual flags (default)
- **wf_improved** – If True, use Wasserman-Faust improved scaling for closeness centrality in disconnected graphs. Default: True

Returns

Dictionary containing all computed centrality measures with keys:

- Degree-based: “layer_degree”, “layer_strength”, “supra_degree”, “supra_strength”, “overlapping_degree”, “overlapping_strength”, “participation_coefficient”, “participation_coefficient_strength”
- Eigenvector-based: “multiplex_eigenvector”, “eigenvector_versatility”, “katz_bonacich”, “pagerank”
- Path-based (if include_path_based=True): “closeness”, “betweenness”
- Advanced (if include_advanced=True): “hits”, “current_flow_closeness”, “current_flow_betweenness”, “subgraph_centrality”, “total_communicability”, “multiplex_k_core”
- Extended (if include_extended=True): “information”, “communicability_betweenness”, “accessibility”, “harmonic_closeness”, “local_efficiency”, “edge_betweenness”, “bridging”, “percolation”, “spreading”, “collective_influence”, “load”, “flow_betweenness”

Return type
dict

Note

Path-based, advanced, and extended measures are computationally expensive for large networks. Use flags or presets to control which measures are computed.

Examples

```
>>> # Compute only basic measures (fast)
>>> results = compute_all_centralities(network)
```

```
>>> # Use preset for standard analysis
>>> results = compute_all_centralities(network, preset='standard')
```

```
>>> # Compute everything
>>> results = compute_all_centralities(network, preset='all')
```

```
>>> # Fine-grained control
>>> results = compute_all_centralities(
...     network,
...     include_path_based=True,
...     include_extended=True
... )
```

`py3plex.algorithms.multilayer_algorithms.centralities.katz_centrality`(*supra_matrix*: *spmatrix*, *alpha*: *float* = *None*, *beta*: *float* = *1.0*, *tol*: *float* = *1e-06*) → *ndarray*

Compute Katz centrality for each node-layer pair.

Katz centrality measures node influence by accounting for all paths with exponentially decaying weights. It is computed as:

$$x = (I - \alpha * A)^{-1} * \beta * 1$$

where $\alpha < 1/\lambda_{\max}(A)$ to ensure convergence. If α is not provided, it defaults to $0.85 / \lambda_{\max}(A)$.

Parameters

- **supra_matrix** – Sparse supra-adjacency matrix (n x n).
- **alpha** – Attenuation parameter. Must be less than $1/\text{spectral_radius}(A)$. If *None*, defaults to $0.85 / \lambda_{\max}(A)$.
- **beta** – Weight of exogenous influence (typically 1.0).
- **tol** – Tolerance for eigenvalue computation.

Returns

Katz centrality scores for each node-layer pair (n,).
Normalized to sum to 1.

Return type

np.ndarray

Raises**Py3plexMatrixError** – If matrix is invalid or alpha is out of valid range.**Example**

```
>>> from py3plex.core import random_generators
>>> net = random_generators.random_multiplex_ER(50, 3, 0.1)
>>> A = net.get_supra_adjacency_matrix()
>>> katz = katz_centrality(A)
>>> print(f"Katz centrality computed for {len(katz)} node-layer pairs")
>>> # With custom alpha
>>> katz_custom = katz_centrality(A, alpha=0.05)
```

MultiXRank: Random Walk with Restart on Universal Multilayer Networks

This module implements the MultiXRank algorithm described in: Baptista et al. (2022), “Universal multilayer network exploration by random walk with restart”, Communications Physics, 5, 170.

MultiXRank performs random walk with restart (RWR) on a supra-heterogeneous adjacency matrix built from multiple multiplexes connected by bipartite blocks.

References

- Paper: <https://doi.org/10.1038/s42005-022-00937-9>
- arXiv: <https://arxiv.org/abs/2106.07869>
- Package: <https://github.com/anthbapt/multixrank>
- Docs: <https://multixrank-doc.readthedocs.io/>

```
class py3plex.algorithms.multilayer_algorithms.multixrank.MultiXRank(restart_prob: float = 0.4,
                                                                    epsilon: float = 1e-06,
                                                                    max_iter: int = 100000,
                                                                    verbose: bool = True)
```

Bases: object

MultiXRank: Universal multilayer network exploration by random walk with restart.

This class implements the MultiXRank algorithm for node prioritization and ranking in universal multilayer networks. It builds a supra-heterogeneous adjacency matrix from multiple multiplexes and bipartite inter-multiplex connections, then performs random walk with restart.

multiplexes

Dictionary of multiplex supra-adjacency matrices

Type

Dict[str, sp.spmatrix]

bipartite_blocks

Inter-multiplex connection matrices

Type

Dict[Tuple[str, str], sp.spmatrix]

restart_prob

Restart probability (r) for RWR

Type

float

epsilon

Convergence threshold

Type

float

max_iter

Maximum number of iterations

Type

int

node_order

Node ordering for each multiplex

Type

Dict[str, List]

supra_matrix

Built supra-heterogeneous adjacency matrix

Type

sp.spmatrix

transition_matrix

Column-stochastic transition matrix

Type

sp.spmatrix

add_bipartite_block(*multiplex_from*: str, *multiplex_to*: str, *bipartite_matrix*: spmatrix | ndarray, *weight*: float = 1.0)

Add a bipartite block connecting two multiplexes.

Parameters

- **multiplex_from** – Name of source multiplex
- **multiplex_to** – Name of target multiplex
- **bipartite_matrix** – Matrix of connections (rows: from, cols: to)
- **weight** – Optional weight to scale this bipartite block

add_multiplex(*name*: str, *supra_adjacency*: spmatrix | ndarray, *node_order*: List | None = None)

Add a multiplex to the universal multilayer network.

Parameters

- **name** – Unique identifier for this multiplex
- **supra_adjacency** – Supra-adjacency matrix for the multiplex (can be from `multi_layer_network.get_supra_adjacency_matrix()`)
- **node_order** – Optional list of node IDs in the order they appear in the matrix. If None, uses integer indices.

aggregate_scores(*scores: ndarray, aggregation: str = 'sum'*) → Dict[str, Dict]

Aggregate scores per multiplex and optionally per physical node.

Parameters

- **scores** – Probability vector from RWR (length = total supra-matrix dimension)
- **aggregation** – How to aggregate scores ('sum', 'mean', 'max')

Returns

Dictionary mapping multiplex names to dictionaries of node scores

build_supra_heterogeneous_matrix(*block_weights: Dict[str | Tuple[str, str], float] | None = None*)

Build the supra-heterogeneous adjacency matrix S.

This constructs the universal multilayer network matrix by: 1. Placing each multiplex supra-adjacency on the block diagonal 2. Adding bipartite blocks for inter-multiplex connections

Parameters

block_weights – Optional dictionary to weight blocks. Keys can be: - Multiplex names (to weight within-multiplex edges) - Tuple (multiplex_from, multiplex_to) for bipartite blocks

Returns

The supra-heterogeneous adjacency matrix

column_normalize(*handle_dangling: str = 'uniform'*) → spmatrix

Column-normalize the supra-heterogeneous matrix to create a stochastic transition matrix.

This ensures each column sums to 1, making the matrix suitable for RWR.

Parameters

handle_dangling – How to handle dangling nodes (columns with zero sum): - 'uniform': Distribute mass uniformly across all nodes - 'self': Add self-loop (mass stays at the node) - 'ignore': Leave as zero (not recommended for RWR)

Returns

Column-stochastic transition matrix

get_top_ranked(*scores: ndarray, k: int = 10, multiplex: str | None = None, exclude_seeds: bool = True, seed_nodes: List[int] | Dict[str, List] | None = None*) → List[Tuple]

Get top-k ranked nodes from RWR scores.

Parameters

- **scores** – Probability vector from RWR
- **k** – Number of top nodes to return
- **multiplex** – If specified, only return nodes from this multiplex
- **exclude_seeds** – Whether to exclude seed nodes from results
- **seed_nodes** – Seed nodes to exclude (if exclude_seeds=True)

Returns

List of (global_index, score) tuples, sorted by score descending

random_walk_with_restart(*seed_nodes: List[int] | Dict[str, List] | ndarray, seed_weights: ndarray | None = None, multiplex_name: str | None = None*) → ndarray

Perform Random Walk with Restart (RWR) from seed nodes.

Parameters

- **seed_nodes** – Seed node specification. Can be: - List of global indices in the supra-matrix - Dict mapping multiplex names to lists of local node indices - NumPy array of global indices
- **seed_weights** – Optional weights for seed nodes (must match length of seed_nodes). If None, uniform weights are used.
- **multiplex_name** – If seed_nodes is a list/array of local indices, specify which multiplex they belong to

Returns

Steady-state probability vector (length = total nodes across all multiplexes)

```
py3plex.algorithms.multilayer_algorithms.multixrank.multixrank_from_py3plex_networks(networks:
    Dict[str,
    multi-
    net.multi_layer_network
    bi-
    par-
    tite_connections:
    Dict[Tuple[str,
    str],
    sp-
    ma-
    trix |
    ndar-
    ray] |
    None
    =
    None,
    seed_nodes:
    Dict[str,
    List]
    |
    None
    =
    None,
    restart_prob:
    float
    =
    0.4,
    ep-
    silon:
    float
    = 1e-
    06,
    max_iter:
    int =
    100000,
    ver-
    bose:
    bool
    =
    True)
    →
    Tu-
    ple[MultiXRank,
    ndarray]
```

Convenience function to run MultiXRank on py3plex multi_layer_network objects.

Parameters

- **networks** – Dictionary mapping names to multi_layer_network objects
- **bipartite_connections** – Optional dict of inter-network connection matrices
- **seed_nodes** – Dict mapping network names to lists of seed node IDs
- **restart_prob** – Restart probability for RWR

- **epsilon** – Convergence threshold
- **max_iter** – Maximum iterations
- **verbose** – Whether to log progress

Returns

Tuple of (MultiXRank object, scores array)

Example

```
>>> from py3plex.core import multinet
>>> net1 = multinet.multi_layer_network()
>>> net1.load_network('network1.edgelist', ...)
>>> net2 = multinet.multi_layer_network()
>>> net2.load_network('network2.edgelist', ...)
>>>
>>> networks = {'net1': net1, 'net2': net2}
>>> seed_nodes = {'net1': ['node1', 'node2']}
>>>
>>> mxr, scores = multixrank_from_py3plex_networks(
...     networks, seed_nodes=seed_nodes
... )
>>>
>>> # Get aggregated scores per network
>>> aggregated = mxr.aggregate_scores(scores)
```

Authors: Benjamin Renoust (github.com/renoust) Date: 2018/02/13 Description: Loads a Detangler JSON format graph and compute unweighted entanglement analysis with Py3Plex

`py3plex.algorithms.multilayer_algorithms.entanglement.build_occurrence_matrix(network: Any)`
→
Tuple[ndarray,
List[Any]]

Build occurrence matrix from multilayer network.

Parameters

network – Multilayer network object

Returns

Tuple of (c_matrix, layers) where c_matrix is the normalized occurrence matrix and layers is the list of layer names

`py3plex.algorithms.multilayer_algorithms.entanglement.compute_blocks(c_matrix: ndarray) →`
Tuple[List[List[int]],
List[ndarray]]

Compute block decomposition of occurrence matrix.

Parameters

c_matrix – Occurrence matrix

Returns

Tuple of (indices, blocks) where indices are the layer indices in each block and blocks are the submatrices for each block

`py3plex.algorithms.multilayer_algorithms.entanglement.compute_entanglement(block_matrix: ndarray) →`
Tuple[List[float],
List[float]]

Compute entanglement metrics for a block.

Parameters

block_matrix – Block submatrix

Returns

Tuple of ([intensity, homogeneity, normalized_homogeneity], gamma_layers)

`py3plex.algorithms.multilayer_algorithms.entanglement.compute_entanglement_analysis`(*network*:
Any)
→
List[Dict[str,
Any]]

Compute full entanglement analysis for a multilayer network.

Parameters

network – Multilayer network object

Returns

List of block analysis dictionaries with entanglement metrics

Supra Matrix Function Centralities for Multilayer Networks.

This module implements centrality measures based on matrix functions of the supra-adjacency matrix, including: - Communicability Centrality (Estrada & Hatano, 2008) - Katz Centrality (Katz, 1953)

These measures operate directly on the sparse supra-adjacency matrix obtained from `multi_layer_network.get_supra_adjacency_matrix()`.

References

- Estrada, E., & Hatano, N. (2008). Communicability in complex networks. *Physical Review E*, 77(3), 036111.
- Katz, L. (1953). A new status index derived from sociometric analysis. *Psychometrika*, 18(1), 39-43.

Authors: py3plex contributors Date: October 2025 (Phase II)

`py3plex.algorithms.multilayer_algorithms.supra_matrix_function_centrality.communicability_centrality(supra_matrix, normalize, use_sparse, max_iter, tol, flops, nda)`

Compute communicability centrality for each node-layer pair.

Communicability centrality measures the weighted sum of all walks between nodes, with exponentially decaying weights for longer walks. It is computed as the row sum of the matrix exponential:

$$c_i = \sum_j (\exp(A))_{ij}$$

This implementation uses `scipy.sparse.linalg.expm_multiply` for efficient sparse matrix exponential computation.

Parameters

- **supra_matrix** – Sparse supra-adjacency matrix (n x n).
- **normalize** – If True, normalize output to sum to 1.
- **use_sparse** – If True, use sparse matrix operations. Falls back to dense for matrices smaller than 10000 elements.
- **max_iter** – Maximum number of iterations for sparse approximation (currently unused).
- **tol** – Tolerance for convergence (currently unused).

Returns

Communicability centrality scores for each node-layer pair (n,).

Return type

`np.ndarray`

Raises

Py3plexMatrixError – If matrix is invalid (non-square, empty, etc.).

Example

```
>>> from py3plex.core import random_generators
>>> net = random_generators.random_multiplex_ER(50, 3, 0.1)
>>> A = net.get_supra_adjacency_matrix()
>>> comm = communicability centrality(A)
>>> print(f"Communicability centrality computed for {len(comm)} node-layer pairs")
```

py3plex.algorithms.multilayer_algorithms.supra_matrix_function_centrality.**katz_centrality**(*supra_matrix*:
sp-
ma-
trix,
al-
pha:
float
 =
 None,
beta:
float
 =
 1.0,
tol:
float
 =
 1e-06)
 →
 ndarray

Compute Katz centrality for each node-layer pair.

Katz centrality measures node influence by accounting for all paths with exponentially decaying weights. It is computed as:

$$x = (I - \alpha * A)^{-1} * \beta * 1$$

where $\alpha < 1/\lambda_{\max}(A)$ to ensure convergence. If α is not provided, it defaults to $0.85 / \lambda_{\max}(A)$.

Parameters

- **supra_matrix** – Sparse supra-adjacency matrix (n x n).
- **alpha** – Attenuation parameter. Must be less than $1/\text{spectral_radius}(A)$. If None, defaults to $0.85 / \lambda_{\max}(A)$.
- **beta** – Weight of exogenous influence (typically 1.0).
- **tol** – Tolerance for eigenvalue computation.

Returns

Katz centrality scores for each node-layer pair (n,).
 Normalized to sum to 1.

Return type

np.ndarray

Raises

Py3plexMatrixError – If matrix is invalid or alpha is out of valid range.

Example

```
>>> from py3plex.core import random_generators
>>> net = random_generators.random_multiplex_ER(50, 3, 0.1)
>>> A = net.get_supra_adjacency_matrix()
>>> katz = katz_centrality(A)
>>> print(f"Katz centrality computed for {len(katz)} node-layer pairs")
>>> # With custom alpha
>>> katz_custom = katz_centrality(A, alpha=0.05)
```

Multiplex Participation Coefficient (MPC) for multiplex networks.

This module implements the Multiplex Participation Coefficient metric for multiplex networks (networks with identical node sets across all layers).

Authors: py3plex contributors Date: 2025

`py3plex.algorithms.multicentrality.ensure(*args, **kwargs)`

`py3plex.algorithms.multicentrality.multiplex_participation_coefficient`(*multinet*: Any, *normalized*: bool = True, *check_multiplex*: bool = True) → Dict[Any, float]

Compute the Multiplex Participation Coefficient (MPC) for multiplex networks. MPC measures how evenly a node participates across layers.

Parameters

- **multinet** (`py3plex.core.multinet.multi_layer_network`) – Multiplex network object (same node set across layers).
- **normalized** (bool, optional) – Whether to normalize MPC to [0,1]. Default: True.
- **check_multiplex** (bool, optional) – Validate that all layers share the same node set.

Returns

Node → MPC value mapping.

Return type

dict

Notes

The MPC is computed as:

$$\text{MPC}(i) = 1 - \sum (k_i^l / k_i^{\text{total}})^2$$

Where: k_i^l is the degree of node i in layer l - k_i^{total} is the total degree of node i across all layers

When `normalized=True`, the result is multiplied by $L/(L-1)$ to normalize to [0,1] range, where L is the number of layers.

References

- Battiston, F., et al. (2014). “Structural measures for multiplex networks.” *Physical Review E*, 89(3), 032804.
- De Domenico, M., et al. (2015). “Identifying modular flows on multilayer networks reveals highly overlapping organization in interconnected systems.” *Physical Review X*, 5(1), 011027.

- Harooni, M., et al. (2025). “Centrality in Multilayer Networks: Accurate Measurements with MultiNetPy.” The Journal of Supercomputing, 81(1), 92. DOI: 10.1007/s11227-025-07197-8

Contracts:

- Precondition: multinet must not be None
- Precondition: normalized and check_multiplex must be booleans
- Postcondition: returns a dictionary with numeric values

```
py3plex.algorithms.multicentrality.require(*args, **kwargs)
```

General Algorithms

Random walk primitives for graph-based algorithms.

This module provides foundation for higher-level algorithms like Node2Vec, DeepWalk, and diffusion processes. Implements both basic and second-order (biased) random walks with proper edge weight handling and multilayer support.

Key Features:

- Basic random walks with weighted edge sampling
- Second-order (Node2Vec-style) biased random walks with p/q parameters
- Multiple simultaneous walks with deterministic reproducibility
- Support for directed, weighted, and multilayer networks
- Efficient sparse adjacency handling

References

- Grover, A., & Leskovec, J. (2016). node2vec: Scalable feature learning for networks. KDD ‘16. <https://doi.org/10.1145/2939672.2939754>
- Perozzi, B., Al-Rfou, R., & Skiena, S. (2014). DeepWalk: Online learning of social representations. KDD ‘14. <https://doi.org/10.1145/2623330.2623732>

```
py3plex.algorithms.general.walkers.basic_random_walk(G: Graph, start_node: int | str, walk_length:
                                                    int, weighted: bool = True, seed: int | None =
                                                    None) → List[int | str]
```

Perform a basic random walk on a graph with proper edge weight handling.

The next step is sampled proportionally to the normalized edge weights of the current node. For unweighted graphs, transitions are uniform.

Parameters

- **G** – NetworkX graph (directed or undirected, weighted or unweighted)
- **start_node** – Node to start the walk from
- **walk_length** – Number of steps in the walk
- **weighted** – Whether to use edge weights (default: True)
- **seed** – Random seed for reproducibility (default: None)

Returns

List of nodes representing the walk path (includes start_node)

Raises

ValueError – If start_node not in graph or walk_length < 1

Examples

```
>>> G = nx.Graph()
>>> G.add_weighted_edges_from([(0, 1, 1.0), (1, 2, 2.0), (1, 3, 1.0)])
>>> walk = basic_random_walk(G, 0, walk_length=3, seed=42)
>>> len(walk)
4
>>> walk[0]
0
```

Note

- Handles disconnected nodes by terminating walk early
- Edge weights must be positive for weighted walks
- Sum of transition probabilities from any node equals 1.0

`py3plex.algorithms.general.walkers.general_random_walk(G, start_node, iterations=1000, teleportation_prob=0)`

Legacy random walk with teleportation (for backward compatibility).

Deprecated since version 0.95a: Use `basic_random_walk()` or `node2vec_walk()` instead. This function will be removed in version 1.0.

Parameters

- **G** – NetworkX graph
- **start_node** – Starting node
- **iterations** – Number of steps
- **teleportation_prob** – Probability of teleporting to random visited node

Returns

List of visited nodes (excluding start_node)

`py3plex.algorithms.general.walkers.generate_walks(G: Graph, num_walks: int, walk_length: int, start_nodes: List[int | str] | None = None, p: float = 1.0, q: float = 1.0, weighted: bool = True, return_edges: bool = False, seed: int | None = None) → List[List[int | str]] | List[List[Tuple[int | str, int | str]]]`

Generate multiple random walks from specified or all nodes.

This interface supports multiple simultaneous walks with deterministic reproducibility under fixed RNG seed. Can return either node sequences or edge sequences.

Parameters

- **G** – NetworkX graph
- **num_walks** – Number of walks to generate per start node
- **walk_length** – Number of steps in each walk
- **start_nodes** – Nodes to start walks from (if None, uses all nodes)
- **p** – Return parameter for Node2Vec (1.0 = no bias)

- **q** – In-out parameter for Node2Vec (1.0 = no bias)
- **weighted** – Whether to use edge weights
- **return_edges** – Return edge sequences instead of node sequences
- **seed** – Random seed for reproducibility

Returns

- List of nodes (if return_edges=False)
- List of edges as tuples (if return_edges=True)

Return type

List of walks, where each walk is either

Examples

```
>>> G = nx.karate_club_graph()
>>> # Generate 10 walks from each node
>>> walks = generate_walks(G, num_walks=10, walk_length=5, seed=42)
>>> len(walks)
340
```

```
>>> # Generate walks with Node2Vec bias
>>> walks = generate_walks(G, num_walks=5, walk_length=10, p=0.5, q=2.0, seed=42)
```

```
>>> # Get edge sequences
>>> edge_walks = generate_walks(G, num_walks=3, walk_length=5, return_edges=True,
↪ seed=42)
```

Note

- With same seed, generates identical walks across runs
- If $p == q == 1.0$, uses basic random walk (faster)
- Empty walks are included if node has no neighbors

```
py3plex.algorithms.general.walkers.layer_specific_random_walk(G: Graph, start_node: int | str,
                                                             walk_length: int, layer: str | None
                                                             = None, cross_layer_prob: float =
                                                             0.0, weighted: bool = True, seed:
                                                             int | None = None) → List[int | str]
```

Perform random walk with layer constraints for multilayer networks.

In a multilayer network represented in py3plex format (where node names include layer information), this function can constrain walks to specific layers with occasional inter-layer transitions.

Parameters

- **G** – NetworkX graph (multilayer network in py3plex format)
- **start_node** – Node to start walk from (may include layer info)
- **walk_length** – Number of steps in the walk
- **layer** – Target layer to constrain walk to (None = no constraint)

- **cross_layer_prob** – Probability of crossing to different layer (0-1)
- **weighted** – Whether to use edge weights
- **seed** – Random seed for reproducibility

Returns

List of nodes representing the walk path

Examples

```
>>> # Multilayer network with layer-specific walks
>>> from py3plex.core import multinet
>>> network = multinet.multi_layer_network()
>>> network.add_layer("social")
>>> network.add_layer("biological")
>>> # ... add nodes and edges ...
>>> walk = layer_specific_random_walk(
...     network.core_network,
...     "nodeA--social",
...     walk_length=10,
...     layer="social",
...     cross_layer_prob=0.1
... )
```

Note

- If layer is None, behaves like basic_random_walk
- cross_layer_prob controls inter-layer transitions
- Node format should follow py3plex convention: “nodeID—layerID”

`py3plex.algorithms.general.walkers.node2vec_walk(G: Graph, start_node: int | str, walk_length: int, p: float = 1.0, q: float = 1.0, weighted: bool = True, seed: int | None = None) → List[int | str]`

Perform a second-order (biased) random walk following Node2Vec logic.

When transitioning from node $t \rightarrow v \rightarrow x$, the probability of choosing x is biased by parameters p (return) and q (in-out): - If $x == t$ (return to previous): weight / p - If x is neighbor of t (stay close): $\text{weight} / 1$ - If x is not neighbor of t (explore): weight / q

Parameters

- **G** – NetworkX graph (directed or undirected, weighted or unweighted)
- **start_node** – Node to start the walk from
- **walk_length** – Number of steps in the walk
- **p** – Return parameter (higher p = less likely to return to previous node)
- **q** – In-out parameter (higher q = less likely to explore further)
- **weighted** – Whether to use edge weights (default: True)
- **seed** – Random seed for reproducibility (default: None)

Returns

List of nodes representing the walk path (includes start_node)

Raises

ValueError – If $p \leq 0$ or $q \leq 0$ or start_node not in graph

Examples

```
>>> G = nx.Graph()
>>> G.add_edges_from([(0, 1), (1, 2), (1, 3), (0, 2)])
>>> # Low p, high q: tends to backtrack
>>> walk = node2vec_walk(G, 0, walk_length=5, p=0.1, q=10.0, seed=42)
>>> # High p, low q: tends to explore outward
>>> walk2 = node2vec_walk(G, 0, walk_length=5, p=10.0, q=0.1, seed=42)
```

Note

- First step is always a basic random walk (no previous node)
- Properly normalizes probabilities at each step
- Handles disconnected nodes by terminating early

References

Grover & Leskovec (2016), node2vec: Scalable feature learning for networks

Benchmark algorithms for node classification performance evaluation.

This module provides algorithms for benchmarking node classification performance, including oracle-based F1 score evaluation.

`py3plex.algorithms.general.benchmark_classification.evaluate_oracle_F1`(probs: ndarray, Y_real: ndarray) → tuple[float, float]

Evaluate oracle F1 scores for multi-label classification.

This function computes micro and macro F1 scores by selecting the top-k predictions for each sample, where k is determined by the ground truth number of labels.

Parameters

- **probs** – Predicted probability matrix of shape (n_samples, n_labels).
- **Y_real** – Ground truth binary label matrix of shape (n_samples, n_labels).

Returns

- micro: Micro-averaged F1 score
- macro: Macro-averaged F1 score

Return type

A tuple containing

Example

```
>>> probs = np.array([[0.9, 0.1, 0.2], [0.1, 0.8, 0.7]])
>>> Y_real = np.array([[1, 0, 0], [0, 1, 1]])
>>> micro, macro = evaluate_oracle_F1(probs, Y_real)
```

Node Ranking

`py3plex.algorithms.node_ranking.node_ranking.authority_matrix(graph: Graph) → spmatrix`

Get the authority matrix of a graph.

Parameters

graph – NetworkX graph

Returns

Authority matrix

`py3plex.algorithms.node_ranking.node_ranking.hub_matrix(graph: Graph) → spmatrix`

Get the hub matrix of a graph.

Parameters

graph – NetworkX graph

Returns

Hub matrix

`py3plex.algorithms.node_ranking.node_ranking.hubs_and_authorities(graph: Graph) → Tuple[dict, dict]`

Compute hubs and authorities scores using HITS algorithm.

Parameters

graph – NetworkX graph

Returns

Tuple of (hubs dictionary, authorities dictionary)

`py3plex.algorithms.node_ranking.node_ranking.modularity(G: Graph, communities: List[List[Any]], weight: str = 'weight') → float`

Calculate modularity of a graph partition.

Parameters

- **G** – NetworkX graph
- **communities** – List of communities (each community is a list of nodes)
- **weight** – Edge weight attribute name

Returns

Modularity value

`py3plex.algorithms.node_ranking.node_ranking.page_rank_kernel(index_row: int) → Tuple[int, ndarray]`

PageRank kernel for parallel computation.

Note: This function expects global variables `G`, `damping_hyper`, `spread_step_hyper`, `spread_percent_hyper`, and `graph` to be defined. It's designed for use with `multiprocessing.Pool.map()`.

Parameters

index_row – Row index to compute PageRank for

Returns

Tuple of (index, PageRank vector)

```
py3plex.algorithms.node_ranking.node_ranking.sparse_page_rank(matrix: spmatrix, start_nodes:
                                                                List[int] | range | None, epsilon:
                                                                float = 1e-06, max_steps: int =
                                                                100000, damping: float = 0.5,
                                                                spread_step: int = 10,
                                                                spread_percent: float = 0.3,
                                                                try_shrink: bool = False) →
                                                                ndarray
```

Compute sparse PageRank with personalization.

Parameters

- **matrix** – Sparse adjacency matrix (column-stochastic)
- **start_nodes** – List of starting node indices for personalization (can be range or None)
- **epsilon** – Convergence threshold
- **max_steps** – Maximum number of iterations
- **damping** – Damping factor (teleportation probability)
- **spread_step** – Maximum steps for spread calculation
- **spread_percent** – Percentage threshold for spread
- **try_shrink** – Whether to try matrix shrinking optimization

Returns

PageRank vector

```
py3plex.algorithms.node_ranking.node_ranking.stochastic_normalization(matrix: spmatrix) →
                                                                spmatrix
```

Normalize a sparse matrix stochastically.

Parameters

matrix – Sparse matrix to normalize

Returns

Stochastically normalized sparse matrix

```
py3plex.algorithms.node_ranking.node_ranking.stochastic_normalization_hin(matrix: spmatrix)
                                                                → spmatrix
```

Normalize a heterogeneous information network matrix stochastically.

Parameters

matrix – Sparse matrix to normalize

Returns

Stochastically normalized sparse matrix

Network Classification

`py3plex.algorithms.network_classification.label_propagation.label_propagation`(*graph_matrix*:
spmatrix,
class_matrix:
ndarray,
alpha: *float* =
0.001, *epsilon*:
float = 1e-12,
max_steps: *int*
= 100000,
normalization:
str | *List[str]* =
'freq') →
ndarray

Propagate labels through a graph.

Parameters

- **graph_matrix** – Sparse graph adjacency matrix
- **class_matrix** – Initial class label matrix
- **alpha** – Propagation weight parameter
- **epsilon** – Convergence threshold
- **max_steps** – Maximum number of iterations
- **normalization** – Normalization scheme(s) to apply

Returns

Propagated label matrix

`py3plex.algorithms.network_classification.label_propagation.label_propagation_normalization`(*matrix*:
sp-
ma-
trix)
→
spmatrix

Normalize a matrix for label propagation.

Parameters

matrix – Sparse matrix to normalize

Returns

Normalized sparse matrix

`py3plex.algorithms.network_classification.label_propagation.label_propagation_tf`() → None
TensorFlow-based label propagation (TODO: implement).

Placeholder for future TensorFlow implementation.

`py3plex.algorithms.network_classification.label_propagation.normalize_amplify_freq`(*mat*:
ndar-
ray) →
ndarray

Normalize and amplify matrix by frequency.

Parameters

mat – Matrix to normalize

Returns

Normalized and amplified matrix

`py3plex.algorithms.network_classification.label_propagation.normalize_exp(mat: ndarray) → ndarray`

Apply exponential normalization.

Parameters

mat – Matrix to normalize

Returns

Exponentially normalized matrix

`py3plex.algorithms.network_classification.label_propagation.normalize_initial_matrix_freq(mat: ndarray) → ndarray`

Normalize matrix by frequency.

Parameters

mat – Matrix to normalize

Returns

Normalized matrix

`py3plex.algorithms.network_classification.label_propagation.normalize_none(mat: ndarray) → ndarray`

No normalization (identity function).

Parameters

mat – Matrix to return unchanged

Returns

Original matrix


```

py3plex.algorithms.network_classification.label_propagation.validate_label_propagation(core_network:
    sp-
    ma-
    trix,
    la-
    bels:
    ndar-
    ray
    |
    sp-
    ma-
    trix,
    dataset_name:
    str
    =
    'test',
    rep-
    e-
    ti-
    tions:
    int
    =
    5,
    nor-
    mal-
    iza-
    tion_scheme:
    str
    |
    List[str]
    =
    'ba-
    sic',
    al-
    pha_value:
    float
    =
    0.001,
    ran-
    dom_seed:
    int
    =
    123,
    ver-
    bose:
    bool
    =
    False)
    →
    DataFrame

```

Validate label propagation with cross-validation.

Parameters

- **core_network** – Sparse network adjacency matrix

- **labels** – Label matrix
- **dataset_name** – Name of the dataset
- **repetitions** – Number of repetitions
- **normalization_scheme** – Normalization scheme to use
- **alpha_value** – Alpha parameter for propagation
- **random_seed** – Random seed for reproducibility
- **verbose** – Whether to print progress

Returns

DataFrame with validation results

6.46.4 Visualization

`py3plex.visualization.multilayer.draw_multiedges`(*network_list*: *List[Graph] | Dict[Any, Graph]*,
multi_edge_tuple: *List[Any]*, *input_type*: *str* =
'nodes', *linepoints*: *str* = *'-.'*, *alphachannel*: *float* =
0.3, *linecolor*: *str* = *'black'*, *curve_height*: *float* = *1*,
style: *str* = *'curve2_bezier'*, *linewidth*: *float* = *1*,
invert: *bool* = *False*, *linmod*: *str* = *'both'*, *resolution*:
float = *0.001*) → *None*

Draw edges connecting multiple layers.

Parameters

- **network_list** – List of NetworkX graphs (layers) or dict of layer_name -> graph
- **multi_edge_tuple** – Tuple specifying edges to draw
- **input_type** – Type of input (“nodes” or other)
- **linepoints** – Line style
- **alphachannel** – Transparency level
- **linecolor** – Color of the lines
- **curve_height** – Height of curved edges
- **style** – Style of edges (“curve2_bezier”, “line”, etc.)
- **linewidth** – Width of lines
- **invert** – Whether to invert drawing direction
- **linmod** – Line modification mode
- **resolution** – Resolution for curve drawing

`py3plex.visualization.multilayer.draw_multilayer_default`(*network_list*: *List[Graph] | Dict[Any, Graph]*, *display*: *bool = True*, *node_size*: *int = 10*, *alphalevel*: *float = 0.13*, *rectanglex*: *float = 1*, *rectangley*: *float = 1*, *background_shape*: *str = 'circle'*, *background_color*: *str = 'rainbow'*, *networks_color*: *str = 'rainbow'*, *labels*: *bool = False*, *arrowsize*: *float = 0.5*, *label_position*: *int = 1*, *verbose*: *bool = False*, *remove_isolated_nodes*: *bool = False*, *axis*: *Any | None = None*, *edge_size*: *float = 1*, *node_labels*: *bool = False*, *node_font_size*: *int = 5*, *scale_by_size*: *bool = False*) → *None*

Core multilayer drawing method.

Parameters

- **network_list** – List of NetworkX graphs to visualize (or dict of layer_name -> graph)
- **display** – Whether to display the plot directly
- **node_size** – Base size of nodes
- **alphalevel** – Transparency level for background shapes
- **rectanglex** – Width of rectangular backgrounds
- **rectangley** – Height of rectangular backgrounds
- **background_shape** – Background shape type (“circle” or “rectangle”)
- **background_color** – Background color scheme (“default”, “rainbow”, or None)
- **networks_color** – Network color scheme (“rainbow” or “black”)
- **labels** – Layer labels to display
- **arrowsize** – Size of edge arrows
- **label_position** – Position offset for layer labels
- **verbose** – Whether to log network information
- **remove_isolated_nodes** – Whether to remove isolated nodes
- **axis** – Matplotlib axis to draw on (None for current axis)
- **edge_size** – Width of edges
- **node_labels** – Whether to display node labels
- **node_font_size** – Font size for node labels
- **scale_by_size** – Whether to scale node size by degree

Returns

None

```
py3plex.visualization.multilayer.draw_multilayer_flow(graphs: List[Graph], multilinks: Dict[str,
List[Tuple]], labels: List[str] | None = None,
node_activity: Dict[Any, float] | None = None,
ax: Any | None = None, display: bool = True,
layer_gap: float = 3.0, node_size: float = 30,
node_cmap: str = 'viridis', flow_alpha: float
= 0.3, flow_min_width: float = 0.2,
flow_max_width: float = 4.0, aggregate_by:
Tuple[str, ...] = ('u', 'v', 'layer_u', 'layer_v'),
**kwargs) → Any
```

Draw multilayer network as layered flow visualization (alluvial-style).

Shows each layer as a horizontal band with nodes positioned along the x-axis. Intra-layer activity is encoded as node color/size, and inter-layer edges are shown as thick flow ribbons (Bezier curves) where width encodes edge weight.

Parameters

- **graphs** – List of NetworkX graphs, one per layer (from `multi_layer_network.get_layers()`)
- **multilinks** – Dictionary mapping `edge_type` -> list of multi-layer edges
- **labels** – Optional list of layer labels. If None, uses layer indices
- **node_activity** – Optional dict mapping `node_id` -> activity value. If None, computes intra-layer degree
- **ax** – Matplotlib axes to draw on. If None, creates new figure
- **display** – If True, calls `plt.show()` at the end
- **layer_gap** – Vertical distance between layer bands
- **node_size** – Base marker size for nodes
- **node_cmap** – Matplotlib colormap name for node activity coloring
- **flow_alpha** – Base transparency for flow ribbons
- **flow_min_width** – Minimum line width for flows
- **flow_max_width** – Maximum line width for flows
- **aggregate_by** – Tuple of keys for aggregating flows (currently not used, for future extension)
- ****kwargs** – Reserved for future extensions

Returns

Matplotlib axes object

Examples

```
>>> network = multi_layer_network()
>>> network.load_network("data.txt", input_type="multiedgelist")
>>> labels, graphs, multilinks = network.get_layers()
>>> draw_multilayer_flow(graphs, multilinks, labels=labels)
```

```
py3plex.visualization.multilayer.generate_random_multiedges(network_list: List[Graph],
                                                           random_edges: int, style: str = 'line',
                                                           linepoints: str = '-.', upper_first: int =
                                                           2, lower_first: int = 0, lower_second:
                                                           int = 2, inverse_tag: bool = False,
                                                           pheight: float = 1) → None
```

Generate and draw random multi-layer edges.

Parameters

- **network_list** – List of NetworkX graphs (layers)
- **random_edges** – Number of random edges to generate
- **style** – Style of edges to draw
- **linepoints** – Line style
- **upper_first** – Upper bound for first layer
- **lower_first** – Lower bound for first layer
- **lower_second** – Lower bound for second layer
- **inverse_tag** – Whether to invert drawing
- **pheight** – Height parameter for curves

```
py3plex.visualization.multilayer.generate_random_networks(number_of_networks: int) →
                                                         List[Graph]
```

Generate random networks for testing.

Parameters

- **number_of_networks** – Number of random networks to generate

Returns

List of NetworkX graphs with random layouts

```
py3plex.visualization.multilayer.hairball_plot(g: Graph | Any, color_list: List[str] | List[int] | None =
                                               None, display: bool = False, node_size: float = 1,
                                               text_color: str = 'black', node_sizes: List[float] | None
                                               = None, layout_parameters: dict | None = None,
                                               legend: Any | None = None, scale_by_size: bool =
                                               True, layout_algorithm: str = 'force', edge_width: float
                                               = 0.01, alpha_channel: float = 0.5, labels: List[str] |
                                               None = None, draw: bool = True, label_font_size: int
                                               = 2) → Any | None
```

A method for drawing force-directed plots
 Args: network (networkx): A network to be visualized
 color_list (list): A list of colors for nodes
 node_size (float): Size of nodes
 layout_parameters (dict): A dictionary of label parameters
 legend (bool): Display legend?
 scale_by_size (bool): Rescale nodes?
 layout_algorithm (string): What type of layout algorithm is to be used?
 edge_width (float): Width of edges
 alpha_channel (float): Transparency level.
 labels (bool): Display labels?
 label_font_size (int): Sizes of labels
 :returns: None

```
py3plex.visualization.multilayer.interactive_diagonal_plot(network_list: List[Graph] | Dict[Any,
                                                           Graph], layer_labels: List[str] | None
                                                           = None, layout_algorithm: str = 'force',
                                                           layer_gap: float = 4.0, node_size_base:
                                                           int = 8, layer_colors: List[str] | None =
                                                           None, show_interlayer_edges: bool =
                                                           True, interlayer_edges: List[Tuple[Any,
                                                           Any]] | None = None) → bool | Any
```

Create an interactive 2.5D diagonal multilayer plot using Plotly.

This function creates an interactive version of the diagonal multilayer visualization, mimicking the traditional 2D diagonal layout but in an interactive 3D environment. Each layer is positioned diagonally with clear visual separation, similar to the static diagonal visualization.

Parameters

- **network_list** – List of NetworkX graphs (layers) or dict of layer_name -> graph
- **layer_labels** – Optional labels for each layer
- **layout_algorithm** – Layout algorithm for nodes (“force”, “circular”, “random”)
- **layer_gap** – Distance between layers in diagonal direction (default: 2.5)
- **node_size_base** – Base size for nodes (default: 8)
- **layer_colors** – Optional list of colors for each layer (HTML color names or hex)
- **show_interlayer_edges** – Whether to show inter-layer edges
- **interlayer_edges** – List of tuples (node1, node2) for inter-layer connections

Returns

False if plotly not available, otherwise plotly figure object

Examples

```
>>> from py3plex.core import multinet
>>> net = multinet.multi_layer_network()
>>> net.load_network("network.txt", input_type="multiedgelist")
>>> labels, graphs, multilinks = net.get_layers("diagonal")
>>> fig = interactive_diagonal_plot(graphs, layer_labels=labels)
```

`py3plex.visualization.multilayer.interactive_hairball_plot`(*G*: Graph, *nsizes*: List[float],
final_color_mapping: dict, *pos*: dict,
colscale: str = 'Rainbow') → bool |
Any

Create an interactive 3D hairball plot using Plotly.

Parameters

- **G** – NetworkX graph to visualize
- **nsizes** – Node sizes
- **final_color_mapping** – Mapping of nodes to colors
- **pos** – Node positions
- **colscale** – Color scale to use

Returns

False if plotly not available, otherwise plotly figure object

`py3plex.visualization.multilayer.onclick`(*event*: Any) → None

Handle mouse click events on plots.

Parameters

event – Matplotlib event object

```
py3plex.visualization.multilayer.plot_edge_colored_projection(multilayer_network, layout: str =
    'spring', node_size: int = 50,
    layer_colors: Dict[Any, str] | None
    = None,
    aggregate_multilayer_edges: bool
    = True, figsize: Tuple[float, float] =
    (12, 9), edge_alpha: float = 0.7,
    **kwargs)
```

Create an aggregated projection where edge colors indicate layer membership.

This visualization projects all layers onto a single 2D graph, using edge colors to distinguish which layer each edge belongs to. Useful for seeing the overall structure while maintaining layer information.

Parameters

- **multilayer_network** – A MultiLayerNetwork instance
- **layout** – Layout algorithm to use (“spring”, “circular”, “random”, “kamada_kawai”)
- **node_size** – Size of nodes
- **layer_colors** – Optional dict mapping layer names to colors; if None, auto-generated
- **aggregate_multilayer_edges** – If True, show edges from all layers with distinct colors
- **figsize** – Figure size as (width, height) tuple
- **edge_alpha** – Transparency level for edges (0-1)
- ****kwargs** – Additional arguments for customization

Returns

The created figure

Return type

matplotlib.figure.Figure

```
py3plex.visualization.multilayer.plot_ego_multilayer(multilayer_network, ego, layers: List[Any] |
    None = None, max_depth: int = 1, layout: str =
    'spring', figsize: Tuple[float, float] | None =
    None, max_cols: int = 3, node_size: int = 500,
    ego_node_size: int = 1200, **kwargs)
```

Create an ego-centric multilayer visualization.

This visualization focuses on a single node (ego) and shows its neighborhood across different layers, highlighting the ego node’s position in each layer.

Parameters

- **multilayer_network** – A MultiLayerNetwork instance
- **ego** – The ego node to focus on
- **layers** – Optional list of specific layers to visualize; if None, uses all layers
- **max_depth** – Maximum depth of neighborhood to include (number of hops)
- **layout** – Layout algorithm for each ego graph
- **figsize** – Optional figure size; if None, auto-calculated
- **max_cols** – Maximum columns in subplot grid
- **node_size** – Size of regular nodes

- **ego_node_size** – Size of the ego node (highlighted)
- ****kwargs** – Additional drawing parameters

Returns

The created figure

Return type

matplotlib.figure.Figure

```
py3plex.visualization.multilayer.plot_radial_layers(multilayer_network, base_radius: float = 1.0,
                                                    radius_step: float = 1.0, node_size: int = 500,
                                                    draw_inter_layer_edges: bool = True, figsize:
                                                    Tuple[float, float] = (12, 12), edge_alpha: float
                                                    = 0.5, draw_layer_bands: bool = True,
                                                    band_alpha: float = 0.25, **kwargs)
```

Create a radial/concentric visualization with layers as rings.

This visualization arranges layers as concentric circles, with nodes positioned on rings based on their layer. Inter-layer edges appear as radial connections.

Parameters

- **multilayer_network** – A MultiLayerNetwork instance
- **base_radius** – Radius of the innermost layer
- **radius_step** – Distance between consecutive layer rings
- **node_size** – Size of nodes (default: 500 for better visibility)
- **draw_inter_layer_edges** – If True, draw edges between layers
- **figsize** – Figure size as (width, height) tuple
- **edge_alpha** – Transparency for edges
- **draw_layer_bands** – If True, draw semi-transparent circular bands around layers
- **band_alpha** – Transparency for layer bands (default: 0.25)
- ****kwargs** – Additional drawing parameters

Returns

The created figure

Return type

matplotlib.figure.Figure

```
py3plex.visualization.multilayer.plot_small_multiples(multilayer_network, layout: str = 'spring',
                                                       max_cols: int = 3, node_size: int = 50,
                                                       shared_layout: bool = True,
                                                       show_layer_titles: bool = True, figsize:
                                                       Tuple[float, float] | None = None, **kwargs)
```

Create a small multiples visualization with one subplot per layer.

This visualization shows each layer as a separate subplot in a grid layout, making it easy to compare the structure of different layers side-by-side.

Parameters

- **multilayer_network** – A MultiLayerNetwork instance
- **layout** – Layout algorithm to use (“spring”, “circular”, “random”, “kamada_kawai”)

- **max_cols** – Maximum number of columns in the subplot grid
- **node_size** – Size of nodes in each subplot
- **shared_layout** – If True, compute one layout and reuse for all layers; if False, compute independent layouts per layer
- **show_layer_titles** – If True, show layer names as subplot titles
- **figsize** – Optional figure size as (width, height) tuple
- ****kwargs** – Additional arguments passed to `nx.draw_networkx`

Returns

The created figure

Return type

`matplotlib.figure.Figure`

```
py3plex.visualization.multilayer.plot_supra_adjacency_heatmap(multilayer_network,
                                                             include_inter_layer: bool = False,
                                                             inter_layer_weight: float = 1.0,
                                                             node_order: List[Any] | None =
                                                             None, cmap: str = 'viridis', figsize:
                                                             Tuple[float, float] = (10, 10),
                                                             **kwargs)
```

Create a supra-adjacency matrix heatmap visualization.

This visualization shows the multilayer network as a block matrix where each block represents the adjacency matrix of one layer. Optionally includes inter-layer connections.

Parameters

- **multilayer_network** – A `MultiLayerNetwork` instance
- **include_inter_layer** – If True, include inter-layer edges/couplings
- **inter_layer_weight** – Default weight for inter-layer connections
- **node_order** – Optional list specifying node ordering; if None, uses sorted order
- **cmap** – Colormap name for the heatmap
- **figsize** – Figure size as (width, height) tuple
- ****kwargs** – Additional arguments for `imshow`

Returns

The created figure

Return type

`matplotlib.figure.Figure`

```
py3plex.visualization.multilayer.supra_adjacency_matrix_plot(matrix: ndarray, display: bool =
                                                             False) → None
```

Plot a supra-adjacency matrix.

Parameters

- **matrix** – Supra-adjacency matrix to plot
- **display** – Whether to display the plot immediately

```
py3plex.visualization.multilayer.visualize_multilayer_network(multilayer_network,  
                                                             visualization_type: str = 'diagonal',  
                                                             **kwargs)
```

High-level function to visualize multilayer networks with multiple visualization modes.

This function provides a unified interface for various multilayer network visualization techniques, making it easy to switch between different visual representations.

Parameters

- **multilayer_network** – A MultiLayerNetwork instance from `py3plex.core.multinet`
- **visualization_type** – Type of visualization to use. Options: - “diagonal”: Default layer-centric diagonal layout (existing behavior) - “small_multiples”: One subplot per layer with shared or independent layouts - “edge_colored_projection”: Aggregate projection with edge colors by layer - “supra_adjacency_heatmap”: Matrix representation of multilayer structure - “radial_layers”: Concentric circles for layers with radial inter-layer edges - “ego_multilayer”: Ego-centric view focused on a specific node
- ****kwargs** – Additional keyword arguments specific to each visualization type. See individual plot functions for details.

Returns

The created figure object

Return type

`matplotlib.figure.Figure`

Raises

ValueError – If `visualization_type` is not recognized

Examples

```
>>> from py3plex.core import multinet  
>>> net = multinet.multi_layer_network()  
>>> net.load_network("network.txt", input_type="multiedgelist")  
>>>  
>>> # Use default diagonal visualization  
>>> fig = visualize_multilayer_network(net)  
>>>  
>>> # Use small multiples view  
>>> fig = visualize_multilayer_network(net, visualization_type="small_multiples")  
>>>  
>>> # Use edge-colored projection  
>>> fig = visualize_multilayer_network(net, visualization_type="edge_colored_  
↪projection")
```

```
py3plex.visualization.drawing_machinery.draw(G, pos=None, ax=None, **kwds)
```

Draw the graph *G* with Matplotlib.

Draw the graph as a simple representation with no node labels or edge labels and using the full Matplotlib figure area and no axis labels by default. See `draw_networkx()` for more full-featured drawing that allows title, axis labels etc.

Parameters

- **G** (*graph*) – A networkx graph

- **pos** (*dictionary, optional*) – A dictionary with nodes as keys and positions as values. If not specified a spring layout positioning will be computed. See `networkx.drawing.layout` for functions that compute node positions.
- **ax** (*Matplotlib Axes object, optional*) – Draw the graph in specified Matplotlib axes.
- **kwds** (*optional keywords*) – See `networkx.draw_networkx()` for a description of optional keywords.

Examples

```
>>> G = nx.dodecahedral_graph()
>>> nx.draw(G)
>>> nx.draw(G, pos=nx.spring_layout(G)) # use spring layout
```

See also

`draw_networkx`, `draw_networkx_nodes`, `draw_networkx_edges`, `draw_networkx_labels`, `draw_networkx_edge_labels`

Notes

This function has the same name as `pylab.draw` and `pyplot.draw` so beware when using

```
>>> from networkx import *
```

since you might overwrite the `pylab.draw` function.

With `pyplot` use

```
>>> import matplotlib.pyplot as plt
>>> import networkx as nx
>>> G = nx.dodecahedral_graph()
>>> nx.draw(G) # networkx draw()
>>> plt.draw() # pyplot draw()
```

Also see the NetworkX drawing examples at https://networkx.github.io/documentation/latest/auto_examples/index.html

`py3plex.visualization.drawing_machinery.draw_circular(G, **kwargs)`

Draw the graph `G` with a circular layout.

Parameters

- **G** (*graph*) – A `networkx` graph
- **kwargs** (*optional keywords*) – See `networkx.draw_networkx()` for a description of optional keywords, with the exception of the `pos` parameter which is not used by this function.

`py3plex.visualization.drawing_machinery.draw_kamada_kawai(G, **kwargs)`

Draw the graph `G` with a Kamada-Kawai force-directed layout.

Parameters

- **G** (*graph*) – A `networkx` graph

- **kwargs** (*optional keywords*) – See `networkx.draw_networkx()` for a description of optional keywords, with the exception of the `pos` parameter which is not used by this function.

`py3plex.visualization.drawing_machinery.draw_networkx(G, pos=None, arrows=True, with_labels=True, **kws)`

Draw the graph `G` using Matplotlib.

Draw the graph with Matplotlib with options for node positions, labeling, titles, and many other drawing features. See `draw()` for simple drawing without labels or axes.

Parameters

- **G** (*graph*) – A `networkx` graph
- **pos** (*dictionary, optional*) – A dictionary with nodes as keys and positions as values. If not specified a spring layout positioning will be computed. See `networkx.drawing.layout` for functions that compute node positions.
- **arrows** (*bool, optional (default=True)*) – For directed graphs, if `True` draw arrowheads. Note: Arrows will be the same color as edges.
- **arrowstyle** (*str, optional (default='->')*) – For directed graphs, choose the style of the arrowheads. See `:py:class: matplotlib.patches.ArrowStyle` for more options.
- **arrowsize** (*int, optional (default=10)*) – For directed graphs, choose the size of the arrow head's length and width. See `:py:class: matplotlib.patches.FancyArrowPatch` for attribute `mutation_scale` for more info.
- **with_labels** (*bool, optional (default=True)*) – Set to `True` to draw labels on the nodes.
- **ax** (*Matplotlib Axes object, optional*) – Draw the graph in the specified Matplotlib axes.
- **odelist** (*list, optional (default G.nodes())*) – Draw only specified nodes
- **edgelist** (*list, optional (default=G.edges())*) – Draw only specified edges
- **node_size** (*scalar or array, optional (default=300)*) – Size of nodes. If an array is specified it must be the same length as `odelist`.
- **node_color** (*color string, or array of floats, (default='r')*) – Node color. Can be a single color format string, or a sequence of colors with the same length as `odelist`. If numeric values are specified they will be mapped to colors using the `cmap` and `vmin,vmax` parameters. See `matplotlib.scatter` for more details.
- **node_shape** (*string, optional (default='o')*) – The shape of the node. Specification is as `matplotlib.scatter` marker, one of `'so^>v<dph8'`.
- **alpha** (*float, optional (default=1.0)*) – The node and edge transparency
- **cmap** (*Matplotlib colormap, optional (default=None)*) – Colormap for mapping intensities of nodes
- **vmin** (*float, optional (default=None)*) – Minimum and maximum for node colormap scaling
- **vmax** (*float, optional (default=None)*) – Minimum and maximum for node colormap scaling
- **linewidths** (*[None | scalar | sequence]*) – Line width of symbol border (default =1.0)
- **width** (*float, optional (default=1.0)*) – Line width of edges

- **edge_color** (*color string, or array of floats (default='r')*) – Edge color. Can be a single color format string, or a sequence of colors with the same length as edge-list. If numeric values are specified they will be mapped to colors using the `edge_cmap` and `edge_vmin`, `edge_vmax` parameters.
- **edge_cmap** (*Matplotlib colormap, optional (default=None)*) – Colormap for mapping intensities of edges
- **edge_vmin** (*floats, optional (default=None)*) – Minimum and maximum for edge colormap scaling
- **edge_vmax** (*floats, optional (default=None)*) – Minimum and maximum for edge colormap scaling
- **style** (*string, optional (default='solid')*) – Edge line style (solid|dashed|dotted,dashdot)
- **labels** (*dictionary, optional (default=None)*) – Node labels in a dictionary keyed by node of text labels
- **font_size** (*int, optional (default=12)*) – Font size for text labels
- **font_color** (*string, optional (default='k' black)*) – Font color string
- **font_weight** (*string, optional (default='normal')*) – Font weight
- **font_family** (*string, optional (default='sans-serif')*) – Font family
- **label** (*string, optional*) – Label for graph legend

Notes

For directed graphs, arrows are drawn at the head end. Arrows can be turned off with keyword `arrows=False`.

Examples

```
>>> G = nx.dodecahedral_graph()
>>> nx.draw(G)
>>> nx.draw(G, pos=nx.spring_layout(G)) # use spring layout
```

```
>>> import matplotlib.pyplot as plt
>>> limits = plt.axis('off') # turn of axis
```

Also see the NetworkX drawing examples at https://networkx.github.io/documentation/latest/auto_examples/index.html

See also

<code>draw,</code>	<code>draw_networkx_nodes,</code>	<code>draw_networkx_edges,</code>	<code>draw_networkx_labels,</code>
<code>draw_networkx_edge_labels</code>			

```
py3plex.visualization.drawing_machinery.draw_networkx_edge_labels(G, pos, edge_labels=None,
                                                                    label_pos=0.5, font_size=10,
                                                                    font_color='k',
                                                                    font_family='sans-serif',
                                                                    font_weight='normal',
                                                                    alpha=1.0, bbox=None,
                                                                    ax=None, rotate=True,
                                                                    **kwds)
```

Draw edge labels.

Parameters

- **G** (*graph*) – A networkx graph
- **pos** (*dictionary*) – A dictionary with nodes as keys and positions as values. Positions should be sequences of length 2.
- **ax** (*Matplotlib Axes object, optional*) – Draw the graph in the specified Matplotlib axes.
- **alpha** (*float*) – The text transparency (default=1.0)
- **edge_labels** (*dictionary*) – Edge labels in a dictionary keyed by edge two-tuple of text labels (default=None). Only labels for the keys in the dictionary are drawn.
- **label_pos** (*float*) – Position of edge label along edge (0=head, 0.5=center, 1=tail)
- **font_size** (*int*) – Font size for text labels (default=12)
- **font_color** (*string*) – Font color string (default='k' black)
- **font_weight** (*string*) – Font weight (default='normal')
- **font_family** (*string*) – Font family (default='sans-serif')
- **bbox** (*Matplotlib bbox*) – Specify text box shape and colors.
- **clip_on** (*bool*) – Turn on clipping at axis boundaries (default=True)

Returns

dict of labels keyed on the edges

Return type

dict

Examples

```
>>> G = nx.dodecahedral_graph()
>>> edge_labels = nx.draw_networkx_edge_labels(G, pos=nx.spring_layout(G))
```

Also see the NetworkX drawing examples at https://networkx.github.io/documentation/latest/auto_examples/index.html

See also

draw, draw_networkx, draw_networkx_nodes, draw_networkx_edges, draw_networkx_labels

```
py3plex.visualization.drawing_machinery.draw_networkx_edges(G, pos, edgelist=None, width=1.0,
    edge_color='k', style='solid',
    alpha=1.0, arrowstyle='->',
    arrowsize=10, edge_cmap=None,
    edge_vmin=None, edge_vmax=None,
    ax=None, arrows=True, label=None,
    node_size=300, nodelist=None,
    node_shape='o', **kwds)
```

Draw the edges of the graph G.

This draws only the edges of the graph G.

Parameters

- **G** (*graph*) – A networkx graph
- **pos** (*dictionary*) – A dictionary with nodes as keys and positions as values. Positions should be sequences of length 2.
- **edgelist** (*collection of edge tuples*) – Draw only specified edges (default=G.edges())
- **width** (*float, or array of floats*) – Line width of edges (default=1.0)
- **edge_color** (*color string, or array of floats*) – Edge color. Can be a single color format string (default='r'), or a sequence of colors with the same length as edgelist. If numeric values are specified they will be mapped to colors using the edge_cmap and edge_vmin, edge_vmax parameters.
- **style** (*string*) – Edge line style (default='solid') (solid|dashed|dotted,dashdot)
- **alpha** (*float*) – The edge transparency (default=1.0)
- **cmap** (*edge*) – Colormap for mapping intensities of edges (default=None)
- **edge_vmin** (*floats*) – Minimum and maximum for edge colormap scaling (default=None)
- **edge_vmax** (*floats*) – Minimum and maximum for edge colormap scaling (default=None)
- **ax** (*Matplotlib Axes object, optional*) – Draw the graph in the specified Matplotlib axes.
- **arrows** (*bool, optional (default=True)*) – For directed graphs, if True draw arrow-heads. Note: Arrows will be the same color as edges.
- **arrowstyle** (*str, optional (default='->')*) – For directed graphs, choose the style of the arrow heads. See :py:class: *matplotlib.patches.ArrowStyle* for more options.
- **arrowsize** (*int, optional (default=10)*) – For directed graphs, choose the size of the arrow head head's length and width. See :py:class: *matplotlib.patches.FancyArrowPatch* for attribute *mutation_scale* for more info.
- **label** (*[None| string]*) – Label for legend

Returns

- *matplotlib.collection.LineCollection* – *LineCollection* of the edges
- *list of matplotlib.patches.FancyArrowPatch* – *FancyArrowPatch* instances of the directed edges
- *Depending whether the drawing includes arrows or not.*

Notes

For directed graphs, arrows are drawn at the head end. Arrows can be turned off with keyword `arrows=False`. Be sure to include `node_size` as a keyword argument; arrows are drawn considering the size of nodes.

Examples

```
>>> G = nx.dodecahedral_graph()
>>> edges = nx.draw_networkx_edges(G, pos=nx.spring_layout(G))
```

```
>>> G = nx.DiGraph()
>>> G.add_edges_from([(1, 2), (1, 3), (2, 3)])
>>> arcs = nx.draw_networkx_edges(G, pos=nx.spring_layout(G))
>>> alphas = [0.3, 0.4, 0.5]
>>> for i, arc in enumerate(arcs): # change alpha values of arcs
...     arc.set_alpha(alphas[i])
```

Also see the NetworkX drawing examples at https://networkx.github.io/documentation/latest/auto_examples/index.html

See also

<code>draw,</code>	<code>draw_networkx,</code>	<code>draw_networkx_nodes,</code>	<code>draw_networkx_labels,</code>
<code>draw_networkx_edge_labels</code>			

```
py3plex.visualization.drawing_machinery.draw_networkx_labels(G, pos, labels=None, font_size=1,
                                                             font_color='k',
                                                             font_family='sans-serif',
                                                             font_weight='normal', alpha=1.0,
                                                             bbox=None, ax=None, **kws)
```

Draw node labels on the graph G.

Parameters

- **G** (*graph*) – A networkx graph
- **pos** (*dictionary*) – A dictionary with nodes as keys and positions as values. Positions should be sequences of length 2.
- **labels** (*dictionary, optional (default=None)*) – Node labels in a dictionary keyed by node of text labels Node-keys in labels should appear as keys in *pos*. If needed use: *{n:lab for n,lab in labels.items() if n in pos}*
- **font_size** (*int*) – Font size for text labels (default=12)
- **font_color** (*string*) – Font color string (default='k' black)
- **font_family** (*string*) – Font family (default='sans-serif')
- **font_weight** (*string*) – Font weight (default='normal')
- **alpha** (*float*) – The text transparency (default=1.0)
- **ax** (*Matplotlib Axes object, optional*) – Draw the graph in the specified Matplotlib axes.

Returns

dict of labels keyed on the nodes

Return type

dict

Examples

```
>>> G = nx.dodecahedral_graph()
>>> labels = nx.draw_networkx_labels(G, pos=nx.spring_layout(G))
```


Also see the NetworkX drawing examples at https://networkx.github.io/documentation/latest/auto_examples/index.html

See also

`draw`, `draw_networkx`, `draw_networkx_nodes`, `draw_networkx_edges`,
`draw_networkx_edge_labels`

```
py3plex.visualization.drawing_machinery.draw_networkx_nodes(G, pos, nodelist=None,
                                                            node_size=300, node_color='r',
                                                            node_shape='o', alpha=1.0,
                                                            cmap=None, vmin=None,
                                                            vmax=None, ax=None,
                                                            linewidths=None, edgecolors=None,
                                                            label=None, **kwds)
```

Draw the nodes of the graph G.

This draws only the nodes of the graph G.

Parameters

- **G** (*graph*) – A networkx graph
- **pos** (*dictionary*) – A dictionary with nodes as keys and positions as values. Positions should be sequences of length 2.
- **ax** (*Matplotlib Axes object, optional*) – Draw the graph in the specified Matplotlib axes.
- **nodelist** (*list, optional*) – Draw only specified nodes (default G.nodes())
- **node_size** (*scalar or array*) – Size of nodes (default=300). If an array is specified it must be the same length as nodelist.
- **node_color** (*color string, or array of floats*) – Node color. Can be a single color format string (default='r'), or a sequence of colors with the same length as nodelist. If numeric values are specified they will be mapped to colors using the cmap and vmin,vmax parameters. See matplotlib.scatter for more details.
- **node_shape** (*string*) – The shape of the node. Specification is as matplotlib.scatter marker, one of 'so^>v<dph8' (default='o').
- **alpha** (*float or array of floats*) – The node transparency. This can be a single alpha value (default=1.0), in which case it will be applied to all the nodes of color. Otherwise, if it is an array, the elements of alpha will be applied to the colors in order (cycling through alpha multiple times if necessary).
- **cmap** (*Matplotlib colormap*) – Colormap for mapping intensities of nodes (default=None)
- **vmin** (*floats*) – Minimum and maximum for node colormap scaling (default=None)
- **vmax** (*floats*) – Minimum and maximum for node colormap scaling (default=None)
- **linewidths** (*[None | scalar | sequence]*) – Line width of symbol border (default =1.0)
- **edgecolors** (*[None | scalar | sequence]*) – Colors of node borders (default = node_color)
- **label** (*[None | string]*) – Label for legend

Returns

PathCollection of the nodes.

Return type

matplotlib.collections.PathCollection

Examples

```
>>> G = nx.dodecahedral_graph()
>>> nodes = nx.draw_networkx_nodes(G, pos=nx.spring_layout(G))
```

Also see the NetworkX drawing examples at https://networkx.github.io/documentation/latest/auto_examples/index.html

See also

<i>draw,</i>	<i>draw_networkx,</i>	<i>draw_networkx_edges,</i>	<i>draw_networkx_labels,</i>
<i>draw_networkx_edge_labels</i>			

`py3plex.visualization.drawing_machinery.draw_random(G, **kwargs)`

Draw the graph G with a random layout.

Parameters

- **G** (*graph*) – A networkx graph
- **kwargs** (*optional keywords*) – See `networkx.draw_networkx()` for a description of optional keywords, with the exception of the `pos` parameter which is not used by this function.

`py3plex.visualization.drawing_machinery.draw_shell(G, **kwargs)`

Draw networkx graph with shell layout.

Parameters

- **G** (*graph*) – A networkx graph
- **kwargs** (*optional keywords*) – See `networkx.draw_networkx()` for a description of optional keywords, with the exception of the `pos` parameter which is not used by this function.

`py3plex.visualization.drawing_machinery.draw_spectral(G, **kwargs)`

Draw the graph G with a spectral layout.

Parameters

- **G** (*graph*) – A networkx graph
- **kwargs** (*optional keywords*) – See `networkx.draw_networkx()` for a description of optional keywords, with the exception of the `pos` parameter which is not used by this function.

`py3plex.visualization.drawing_machinery.draw_spring(G, **kwargs)`

Draw the graph G with a spring layout.

Parameters

- **G** (*graph*) – A networkx graph
- **kwargs** (*optional keywords*) – See `networkx.draw_networkx()` for a description of optional keywords, with the exception of the `pos` parameter which is not used by this function.

Color utilities for py3plex visualization.

`py3plex.visualization.colors.RGB_to_hex( RGB: List[int] ) → str`

Convert RGB list to hex color.

Parameters

RGB – RGB values as [R, G, B] list

Returns

Hex color string like “#FFFFFF”

`py3plex.visualization.colors.color_dict( gradient: List[List[int]] ) → Dict[str, List]`

Takes in a list of RGB sub-lists and returns dictionary of colors in RGB and hex form for use in a graphing function defined later on.

Parameters

gradient – List of RGB color values

Returns

Dictionary with ‘hex’, ‘r’, ‘g’, ‘b’ keys

`py3plex.visualization.colors.hex_to_RGB( hex: str ) → List[int]`

Convert hex color to RGB list.

Parameters

hex – Hex color string like “#FFFFFF”

Returns

RGB values as [R, G, B] list

`py3plex.visualization.colors.linear_gradient(start_hex: str, finish_hex: str = '#FFFFFF', n: int = 10)
→ Dict[str, List]`

Returns a gradient list of (n) colors between two hex colors.

Parameters

- **start_hex** – Starting color as six-digit hex string (e.g., “#FFFFFF”)
- **finish_hex** – Ending color as six-digit hex string (default: “#FFFFFF”)
- **n** – Number of colors in gradient (default: 10)

Returns

Dictionary with ‘hex’, ‘r’, ‘g’, ‘b’ keys containing gradient colors

`py3plex.visualization.layout_algorithms.compute_force_directed_layout(g: Graph,
layout_parameters:
Dict[str, Any] | None =
None, verbose: bool =
True, gravity: float = 0.2,
strongGravityMode:
bool = False,
barnesHutTheta: float =
1.2,
edgeWeightInfluence:
float = 1, scalingRatio:
float = 2.0, forceImport:
bool = True, seed: int |
None = None) →
Dict[Any, ndarray]`

Compute force-directed layout for a graph using ForceAtlas2 or NetworkX spring layout.

Parameters

- **g** – NetworkX graph to layout
- **layout_parameters** – Optional parameters to pass to layout algorithm
- **verbose** – Whether to print progress information
- **gravity** – Attraction force towards the center (must be non-negative)
- **strongGravityMode** – Use strong gravity mode
- **barnesHutTheta** – Barnes-Hut approximation parameter
- **edgeWeightInfluence** – Influence of edge weights on layout
- **scalingRatio** – Scaling factor for the layout (must be positive)
- **forceImport** – Whether to use ForceAtlas2 (if available)
- **seed** – Random seed for reproducibility in fallback spring layout

Returns

Dictionary mapping nodes to 2D position arrays

Note

For large networks (>1000 nodes), this may be slow. Consider using faster layouts (circular, random, spectral) or matrix visualization.

Contracts:

- Precondition: graph must not be None and be a NetworkX graph
- Precondition: graph must have at least one node
- Precondition: gravity must be non-negative
- Precondition: scalingRatio must be positive
- Postcondition: result is a dictionary
- Postcondition: result has positions for all nodes

`py3plex.visualization.layout_algorithms.compute_random_layout(g: Graph, seed: int | None = None) → Dict[Any, ndarray]`

Compute a random layout for the graph.

Parameters

- **g** – NetworkX graph
- **seed** – Random seed for reproducibility

Returns

Dictionary mapping nodes to 2D positions

Contracts:

- Precondition: graph must not be None and be a NetworkX graph
- Precondition: graph must have at least one node
- Postcondition: result is a dictionary
- Postcondition: result has positions for all nodes

`py3plex.visualization.layout_algorithms.ensure(*args, **kwargs)`

`py3plex.visualization.layout_algorithms.require(*args, **kwargs)`

`py3plex.visualization.bezier.bezier_calculate_dfy`(*mp_y: float, path_height: float, x0: float, midpoint_x: float, x1: float, y0: float, y1: float, dfx: ndarray, mode: str = 'upper'*) → ndarray

Calculate y-coordinates for bezier curve.

Parameters

- **mp_y** – Midpoint y-coordinate
- **path_height** – Height of the path
- **x0** – Start x-coordinate
- **midpoint_x** – Midpoint x-coordinate
- **x1** – End x-coordinate
- **y0** – Start y-coordinate
- **y1** – End y-coordinate
- **dfx** – Array of x-coordinates
- **mode** – Mode for curve calculation (“upper” or “bottom”)

Returns

Array of y-coordinates

`py3plex.visualization.bezier.draw_bezier`(*total_size: int, p1: Tuple[float, float], p2: Tuple[float, float], mode: str = 'quadratic', inversion: bool = False, path_height: float = 2, linemode: str = 'both', resolution: float = 0.1*) → Tuple[ndarray, ndarray]

Draw bezier curve between two points.

Parameters

- **total_size** – Total size of the drawing area
- **p1** – First point coordinates (x0, x1)
- **p2** – Second point coordinates (y0, y1)
- **mode** – Drawing mode (default: “quadratic”)
- **inversion** – Whether to invert the curve
- **path_height** – Height of the path
- **linemode** – Line drawing mode (“upper”, “bottom”, or “both”)
- **resolution** – Resolution for curve sampling

Returns

Tuple of (x-coordinates, y-coordinates) arrays

`py3plex.visualization.benchmark_visualizations.generic_grouping`(*fname: DataFrame, score_name: str, threshold: float = 1.0, percentages: bool = True*) → DataFrame

`py3plex.visualization.benchmark_visualizations.plot_core_macro(fname: DataFrame) → int`

A very simple visualization of the results..

`py3plex.visualization.benchmark_visualizations.plot_core_macro_box(fname: str) → int`

`py3plex.visualization.benchmark_visualizations.plot_core_macro_gg(fname: DataFrame) → None`

`py3plex.visualization.benchmark_visualizations.plot_core_micro(fname: DataFrame) → int`

A very simple visualization of the results..

`py3plex.visualization.benchmark_visualizations.plot_core_micro_gg(fname: DataFrame) → None`

`py3plex.visualization.benchmark_visualizations.plot_core_micro_grid(fname: str) → None`

`py3plex.visualization.benchmark_visualizations.plot_core_time(fname: DataFrame) → int`

`py3plex.visualization.benchmark_visualizations.plot_core_time_gg(fname: str) → None`

`py3plex.visualization.benchmark_visualizations.plot_core_variability(fname: str) → None`

`py3plex.visualization.benchmark_visualizations.plot_critical_distance(fname: str, num_algo:
int = 14) → None`

`py3plex.visualization.benchmark_visualizations.plot_mean_times(fn: DataFrame) → None`

`py3plex.visualization.benchmark_visualizations.plot_robustness(infile: DataFrame) → None`

`py3plex.visualization.benchmark_visualizations.table_to_latex(fname,
outfolder='./final_results/tables/',
threshold=1)`

6.46.5 Wrappers

class `py3plex.wrappers.benchmark_nodes.TopKRanker`(*estimator*, * (*Keyword-only parameters separator*
(PEP 3102)), *n_jobs*=None, *verbose*=0)

Bases: `OneVsRestClassifier`

predict(*X*: ndarray, *top_k_list*: List[int]) → List[List]

Predict top K labels for each sample.

Parameters

- **X** – Feature matrix
- **top_k_list** – List of K values for each sample

Returns

List of predicted labels for each sample

set_partial_fit_request(*, *classes*: bool | None | str = '\$UNCHANGED\$') → `TopKRanker`

Configure whether metadata should be requested to be passed to the `partial_fit` method.

Note that this method is only relevant when this estimator is used as a sub-estimator within a meta-estimator and metadata routing is enabled with `enable_metadata_routing=True` (see `sklearn.set_config()`). Please check the User Guide on how the routing mechanism works.

The options for each parameter are:

- **True**: metadata is requested, and passed to `partial_fit` if provided. The request is ignored if metadata is not provided.

- `False`: metadata is not requested and the meta-estimator will not pass it to `partial_fit`.
- `None`: metadata is not requested, and the meta-estimator will raise an error if the user provides it.
- `str`: metadata should be passed to the meta-estimator with this given alias instead of the original name.

The default (`sklearn.utils.metadata_routing.UNCHANGED`) retains the existing request. This allows you to change the request for some parameters and not others.

Added in version 1.3.

Parameters

classes (*str, True, False, or None, default=sklearn.utils.metadata_routing.UNCHANGED*) – Metadata routing for classes parameter in `partial_fit`.

Returns

self – The updated object.

Return type

object

set_predict_request(*, *top_k_list: bool | None | str = '\$UNCHANGED\$'*) → *TopKRanker*

Configure whether metadata should be requested to be passed to the `predict` method.

Note that this method is only relevant when this estimator is used as a sub-estimator within a meta-estimator and metadata routing is enabled with `enable_metadata_routing=True` (see `sklearn.set_config()`). Please check the User Guide on how the routing mechanism works.

The options for each parameter are:

- `True`: metadata is requested, and passed to `predict` if provided. The request is ignored if metadata is not provided.
- `False`: metadata is not requested and the meta-estimator will not pass it to `predict`.
- `None`: metadata is not requested, and the meta-estimator will raise an error if the user provides it.
- `str`: metadata should be passed to the meta-estimator with this given alias instead of the original name.

The default (`sklearn.utils.metadata_routing.UNCHANGED`) retains the existing request. This allows you to change the request for some parameters and not others.

Added in version 1.3.

Parameters

top_k_list (*str, True, False, or None, default=sklearn.utils.metadata_routing.UNCHANGED*) – Metadata routing for `top_k_list` parameter in `predict`.

Returns

self – The updated object.

Return type

object

set_score_request(*, *sample_weight: bool | None | str = '\$UNCHANGED\$'*) → *TopKRanker*

Configure whether metadata should be requested to be passed to the `score` method.

Note that this method is only relevant when this estimator is used as a sub-estimator within a meta-estimator and metadata routing is enabled with `enable_metadata_routing=True` (see `sklearn.set_config()`). Please check the User Guide on how the routing mechanism works.

The options for each parameter are:

- **True**: metadata is requested, and passed to `score` if provided. The request is ignored if metadata is not provided.
- **False**: metadata is not requested and the meta-estimator will not pass it to `score`.
- **None**: metadata is not requested, and the meta-estimator will raise an error if the user provides it.
- **str**: metadata should be passed to the meta-estimator with this given alias instead of the original name.

The default (`sklearn.utils.metadata_routing.UNCHANGED`) retains the existing request. This allows you to change the request for some parameters and not others.

Added in version 1.3.

Parameters

sample_weight (*str, True, False, or None, default=sklearn.utils.metadata_routing.UNCHANGED*) – Metadata routing for `sample_weight` parameter in `score`.

Returns

self – The updated object.

Return type

object

`py3plex.wrappers.benchmark_nodes.benchmark_node_classification(path: str, core_network: Any, labels_matrix: Any, percent: Any = 'all') → Dict[Any, Any]`

Benchmark node classification using embeddings.

Parameters

- **path** – Path to embeddings file
- **core_network** – Network adjacency matrix
- **labels_matrix** – Labels for nodes
- **percent** – Training percentage or “all” for multiple percentages

Returns

Dictionary of classification results

`py3plex.wrappers.benchmark_nodes.main()`

`py3plex.wrappers.benchmark_nodes.sparse2graph(x: spmatrix) → Dict[str, List[str]]`

Convert sparse matrix to graph dictionary.

Parameters

x – Sparse matrix representation of graph

Returns

Dictionary mapping node IDs to lists of neighbor IDs

6.46.6 I/O Operations

For the complete auto-generated API documentation, see the `AUTOGEN_results` directory after building the docs. Aggregation and Network Operations ————— Vectorized multiplex aggregation for multilayer networks.

This module provides optimized implementations for aggregating edges across multiple layers using vectorized NumPy and SciPy sparse operations, replacing slower Python loops with efficient matrix operations.

Performance targets: - 3× speedup for 1M edges across 4 layers compared to legacy loop-based methods - Memory-efficient sparse matrix output by default - Float tolerance of 1e-6 for numerical equivalence with legacy methods

Author: py3plex development team License: MIT

`py3plex.multinet.aggregation.aggregate_layers`(*edges: ndarray | list, weight_col: str | int = 'w', reducer: Literal['sum', 'mean', 'max'] = 'sum', to_sparse: bool = True*) → *csr_matrix | ndarray*

Aggregate edge weights across multiple layers using vectorized operations.

This function replaces Python loops with efficient NumPy and SciPy sparse matrix operations for superior performance on large multilayer networks.

Complexity: $O(E)$ where E is the number of edges **Memory:** $O(E)$ for sparse output, $O(N^2)$ for dense output

Parameters

- **edges** – Edge data as ndarray with shape $(E, \geq 3)$ containing (layer, src, dst, [weight]) columns. If no weight column, assumes weight=1.0 for all edges. Can also accept list of lists.
- **weight_col** – Column name or index for weights (default “w”). If edges is ndarray, must be int index (0-based). Column 3 is used if it exists, else weights default to 1.
- **reducer** – Aggregation method - one of: - “sum”: Sum weights for edges appearing in multiple layers (default) - “mean”: Average weights across layers - “max”: Take maximum weight across layers
- **to_sparse** – If True, return `scipy.sparse.csr_matrix` (default, memory-efficient). If False, return dense `numpy.ndarray`.

Returns

Aggregated adjacency matrix in requested format (sparse CSR or dense). Shape is (N, N) where N is the maximum node ID + 1.

Raises

- **ValueError** – If edges array has wrong shape or reducer is invalid.
- **TypeError** – If edges is not ndarray or list-like.

Examples

```
>>> import numpy as np
>>> # Create edge list: (layer, src, dst, weight)
>>> edges = np.array([
...     [0, 0, 1, 1.0],
...     [0, 1, 2, 2.0],
...     [1, 0, 1, 0.5], # Same edge in layer 1
...     [1, 2, 3, 1.5],
... ])
>>> mat = aggregate_layers(edges, reducer="sum")
>>> mat.shape
(4, 4)
>>> mat[0, 1] # Sum of weights from both layers
1.5
```

```
>>> # With mean aggregation
>>> mat_mean = aggregate_layers(edges, reducer="mean", to_sparse=False)
>>> mat_mean[0, 1] # Average of 1.0 and 0.5
0.75
```

Notes

- Node IDs are assumed to be integers starting from 0
- Self-loops are supported
- For directed graphs, (i,j) and (j,i) are different edges
- Sparse output recommended for large networks (N > 1000)
- Deterministic output for fixed input order

Performance:

- Achieves 3× speedup vs loop-based aggregation on 1M edges: - Legacy loop: ~2.5s for 1M edges, 4 layers
- Vectorized: ~0.8s for same dataset (measured on standard hardware)

```
py3plex.multinet.aggregation.ensure(*args, **kwargs)
```

```
py3plex.multinet.aggregation.require(*args, **kwargs)
```

6.46.7 Profiling Utilities

Performance profiling utilities for py3plex.

This module provides decorators and utilities for tracking function execution time, memory usage, and performance metrics. These tools enable performance regression detection and optimization efforts.

Example

```
>>> from py3plex.profiling import profile_performance
>>>
>>> @profile_performance
>>> def slow_function():
...     # ... computation
...     pass
>>>
>>> slow_function() # Logs execution time automatically
```

```
class py3plex.profiling.PerformanceMonitor
```

Bases: object

Global performance monitoring registry.

Tracks execution times and call counts for profiled functions. Can be used to generate performance reports and detect regressions.

enabled

Whether profiling is enabled globally

stats

Dictionary mapping function names to performance statistics

clear()

Clear all collected statistics.

get_report() → str

Generate a performance report.

Returns

String containing formatted performance statistics

record(*func_name: str, elapsed: float, memory_delta: float | None = None*)

Record performance metrics for a function call.

Parameters

- **func_name** – Name of the function
- **elapsed** – Execution time in seconds
- **memory_delta** – Memory usage change in MB (optional)

`py3plex.profiling.benchmark`(*func: Callable, iterations: int = 100, warmup: int = 10, args: tuple = (), kwargs: dict = None*) → Dict[str, float]

Benchmark a function with multiple iterations.

Runs the function multiple times and collects statistics about execution time. Includes a warmup phase to allow JIT compilation and caching.

Parameters

- **func** – Function to benchmark
- **iterations** – Number of iterations to run (default: 100)
- **warmup** – Number of warmup iterations (default: 10)
- **args** – Positional arguments for the function
- **kwargs** – Keyword arguments for the function

Returns

- mean: Average execution time (seconds)
- median: Median execution time (seconds)
- min: Minimum execution time (seconds)
- max: Maximum execution time (seconds)
- std: Standard deviation (seconds)
- total: Total time for all iterations (seconds)

Return type

Dictionary containing benchmark statistics

Example

```
>>> from py3plex.profiling import benchmark
>>>
>>> def my_function(n):
...     return sum(range(n))
>>>
```

(continues on next page)

(continued from previous page)

```
>>> stats = benchmark(my_function, iterations=1000, args=(1000,))
>>> print(f"Average time: {stats['mean']*1000:.3f}ms")
```

`py3plex.profiling.get_monitor()` → *PerformanceMonitor*

Get the global performance monitor instance.

Returns

Global performance monitoring instance

Return type

PerformanceMonitor

Example

```
>>> from py3plex.profiling import get_monitor
>>> monitor = get_monitor()
>>> print(monitor.get_report())
```

`py3plex.profiling.profile_performance(func: Callable | None = None, *, log_args: bool = False, track_memory: bool = False)` → *Callable*

Decorator to track function execution time and optionally memory usage.

This decorator measures the wall-clock time taken by a function and logs it. It can also track memory usage changes if requested. Performance metrics are stored in the global performance monitor for later analysis.

Parameters

- **func** – Function to decorate (when used without arguments)
- **log_args** – If True, log function arguments (default: False)
- **track_memory** – If True, track memory usage (default: False)

Returns

Decorated function that logs execution time

Examples

Basic usage: `>>> @profile_performance ... def my_function(x, y): ... return x + y`

With options: `>>> @profile_performance(log_args=True, track_memory=True) ... def expensive_function(data): ... # ... expensive computation ... return result`

Manual wrapping: `>>> def my_function(): ... pass >>> profiled_func = profile_performance(my_function)`

Note

Memory tracking requires the `tracemalloc` module and adds overhead. Use it only when investigating memory issues.

`py3plex.profiling.timed_section(name: str, log_level: str = 'info')`

Context manager for timing code blocks.

Useful for timing specific sections of code without wrapping entire functions.

Parameters

- **name** – Name of the code section being timed

- **log_level** – Logging level ('debug', 'info', 'warning', 'error')

Example

```
>>> from py3plex.profilng import timed_section
>>>
>>> with timed_section("data loading"):
...     data = load_large_dataset()
>>>
>>> with timed_section("computation", log_level="debug"):
...     result = complex_calculation()
```

Yields

None

6.46.8 Logging Configuration

Logging configuration for py3plex.

This module provides centralized logging configuration for the py3plex library.

`py3plex.logging_config.get_logger(name: str | None = None, level: int = 20) → Logger`

Get or create a logger for py3plex modules.

Parameters

- **name** – Logger name. If None, returns the root py3plex logger.
- **level** – Logging level (default: INFO)

Returns

Configured logger instance

Example

```
>>> from py3plex.logging_config import get_logger
>>> logger = get_logger(__name__)
>>> logger.info("Processing network...")
```

`py3plex.logging_config.setup_logging(level: int = 20, format_string: str | None = None) → Logger`

Configure logging for py3plex with custom settings.

Parameters

- **level** – Logging level (default: INFO)
- **format_string** – Custom format string (optional)

Returns

Root py3plex logger

Example

```
>>> from py3plex.logging_config import setup_logging
>>> logger = setup_logging(level=logging.DEBUG)
```

6.46.9 I/O Schema and Validation

Schema definitions for multilayer graphs using dataclasses.

This module provides dataclass representations of multilayer graph components with built-in validation and serialization support.

```
class py3plex.io.schema.Edge(src: ~typing.Hashable, dst: ~typing.Hashable, src_layer: ~typing.Hashable,  
                             dst_layer: ~typing.Hashable, key: int = 0, attributes: ~typing.Dict[str,  
                             ~typing.Any] = <factory>)
```

Bases: object

Represents an edge in a multilayer network.

src

Source node ID

Type

Hashable

dst

Destination node ID

Type

Hashable

src_layer

Source layer ID

Type

Hashable

dst_layer

Destination layer ID

Type

Hashable

key

Optional edge key for multigraphs (default: 0)

Type

int

attributes

Dictionary of edge attributes (must be JSON-serializable)

Type

Dict[str, Any]

attributes: Dict[str, Any]

dst: Hashable

dst_layer: Hashable

edge_tuple() → Tuple[Hashable, Hashable, Hashable, Hashable, int]

Return edge as a tuple for uniqueness checking.

Returns

Tuple of (src, dst, src_layer, dst_layer, key)

classmethod **from_dict**(*data: Dict[str, Any]*) → Edge

Create edge from dictionary.

Parameters

data – Dictionary containing edge data

Returns

Edge instance

key: int = 0

src: Hashable

src_layer: Hashable

to_dict() → Dict[str, Any]

Convert edge to dictionary.

Returns

Dictionary representation of the edge

class py3plex.io.schema.**Layer**(*id: ~typing.Hashable, attributes: ~typing.Dict[str, ~typing.Any] = <factory>*)

Bases: object

Represents a layer in a multilayer network.

id

Unique identifier for the layer

Type

Hashable

attributes

Dictionary of layer attributes (must be JSON-serializable)

Type

Dict[str, Any]

attributes: Dict[str, Any]

classmethod **from_dict**(*data: Dict[str, Any]*) → Layer

Create layer from dictionary.

Parameters

data – Dictionary containing layer data

Returns

Layer instance

id: Hashable

to_dict() → Dict[str, Any]

Convert layer to dictionary.

Returns

Dictionary representation of the layer

```
class py3plex.io.schema.MultiLayerGraph(nodes: ~typing.Dict[~typing.Hashable,  
~py3plex.io.schema.Node] = <factory>, layers:  
~typing.Dict[~typing.Hashable, ~py3plex.io.schema.Layer] =  
<factory>, edges: ~typing.List[~py3plex.io.schema.Edge] =  
<factory>, directed: bool = True, attributes: ~typing.Dict[str,  
~typing.Any] = <factory>)
```

Bases: object

Represents a complete multilayer graph.

nodes

Dictionary mapping node IDs to Node objects

Type

Dict[Hashable, py3plex.io.schema.Node]

layers

Dictionary mapping layer IDs to Layer objects

Type

Dict[Hashable, py3plex.io.schema.Layer]

edges

List of Edge objects

Type

List[py3plex.io.schema.Edge]

directed

Whether the graph is directed

Type

bool

attributes

Dictionary of graph-level attributes

Type

Dict[str, Any]

add_edge(*edge*: Edge)

Add an edge to the graph.

Parameters

edge – Edge to add

Raises

- **ReferentialIntegrityError** – If edge references non-existent nodes or layers
- **SchemaValidationError** – If edge is duplicate

add_layer(*layer*: Layer)

Add a layer to the graph.

Parameters

layer – Layer to add

Raises

SchemaValidationError – If layer ID already exists

add_node(*node: Node*)

Add a node to the graph.

Parameters

node – Node to add

Raises

SchemaValidationError – If node ID already exists

attributes: Dict[str, Any]

directed: bool = True

edges: List[Edge]

classmethod from_dict(*data: Dict[str, Any]*) → MultiLayerGraph

Create graph from dictionary.

Parameters

data – Dictionary containing graph data

Returns

MultiLayerGraph instance

layers: Dict[Hashable, Layer]

nodes: Dict[Hashable, Node]

to_dict() → Dict[str, Any]

Convert graph to dictionary.

Returns

Dictionary representation of the graph

class py3plex.io.schema.**Node**(*id: ~typing.Hashable, attributes: ~typing.Dict[str, ~typing.Any] = <factory>*)

Bases: object

Represents a node in a multilayer network.

id

Unique identifier for the node

Type

Hashable

attributes

Dictionary of node attributes (must be JSON-serializable)

Type

Dict[str, Any]

attributes: Dict[str, Any]

classmethod from_dict(*data: Dict[str, Any]*) → Node

Create node from dictionary.

Parameters

data – Dictionary containing node data

Returns

Node instance

id: Hashable

to_dict() → Dict[str, Any]

Convert node to dictionary.

Returns

Dictionary representation of the node

Public API for reading and writing multilayer graphs.

This module provides the main entry points for I/O operations with format detection and a registry system for extensibility.

`py3plex.io.api.ensure(*args, **kwargs)`

`py3plex.io.api.read(filepath: str | Path, format: str | None = None, **kwargs) → MultiLayerGraph`

Read a multilayer graph from a file.

Parameters

- **filepath** – Path to the input file
- **format** – Format name (e.g., 'json', 'csv'). If None, auto-detected from extension
- ****kwargs** – Additional arguments passed to the format-specific reader

Returns

MultiLayerGraph instance

Raises

- **FormatUnsupportedError** – If format is not supported or cannot be detected
- **FileNotFoundError** – If file does not exist

Example

```
>>> graph = read('network.json')
>>> graph = read('network.csv', format='csv')
```

`py3plex.io.api.register_reader(format_name: str, reader_func: Callable[[...], MultiLayerGraph]) → None`

Register a reader function for a specific format.

Parameters

- **format_name** – Name of the format (e.g., 'json', 'csv', 'graphml')
- **reader_func** – Function that takes (filepath, **kwargs) and returns MultiLayerGraph

Example

```
>>> def my_reader(filepath, **kwargs):
...     # Custom reading logic
...     return MultiLayerGraph(...)
>>> register_reader('myformat', my_reader)
```

Contracts:

- Precondition: format_name must be a non-empty string
- Precondition: reader_func must be callable

- Postcondition: reader is registered in `_READERS`

`py3plex.io.api.register_writer(format_name: str, writer_func: Callable[[...], None]) → None`

Register a writer function for a specific format.

Parameters

- **format_name** – Name of the format (e.g., 'json', 'csv', 'graphml')
- **writer_func** – Function that takes (graph, filepath, ****kwargs**) and writes to file

Example

```
>>> def my_writer(graph, filepath, **kwargs):
...     # Custom writing logic
...     pass
>>> register_writer('myformat', my_writer)
```

Contracts:

- Precondition: format_name must be a non-empty string
- Precondition: writer_func must be callable
- Postcondition: writer is registered in `_WRITERS`

`py3plex.io.api.require(*args, **kwargs)`

`py3plex.io.api.supported_formats(read: bool = True, write: bool = True) → Dict[str, List[str]]`

Get list of supported formats for read and/or write operations.

Parameters

- **read** – Include formats that support reading
- **write** – Include formats that support writing

Returns

Dictionary with 'read' and/or 'write' keys containing lists of format names

Example

```
>>> formats = supported_formats()
>>> print(formats)
{'read': ['json', 'jsonl', 'csv'], 'write': ['json', 'jsonl', 'csv']}
```

Contracts:

- Postcondition: result is a dictionary
- Postcondition: result contains 'read' key when read=True
- Postcondition: result contains 'write' key when write=True

`py3plex.io.api.write(graph: MultiLayerGraph, filepath: str | Path, format: str | None = None, **kwargs) → None`

Write a multilayer graph to a file.

Parameters

- **graph** – MultiLayerGraph to write
- **filepath** – Path to the output file
- **format** – Format name (e.g., 'json', 'csv'). If None, auto-detected from extension
- ****kwargs** – Additional arguments passed to the format-specific writer

Raises

FormatUnsupportedError – If format is not supported or cannot be detected

Example

```
>>> write(graph, 'network.json')
>>> write(graph, 'network.csv', format='csv', deterministic=True)
```

6.46.10 Hedwig Rule Learning

`py3plex.algorithms.hedwig.build_graph(kwarg: Dict[str, Any]) → Any`

`py3plex.algorithms.hedwig.generate_rules_report(kwarg: ~typing.Dict[str, ~typing.Any],
rules_per_target:
~typing.List[~typing.Tuple[~typing.Any,
~typing.List[~typing.Any]]], human:
~typing.Callable[[~typing.Any, ~typing.Any],
~typing.Any] = <function <lambda>>) → str`

`py3plex.algorithms.hedwig.rule_kernel(target: Any) → Tuple[Any, List[Any]]`

`py3plex.algorithms.hedwig.run(kwarg: Dict[str, Any], cli: bool = True, generator_tag: bool = False,
num_threads: str | int = 'all') → List[Tuple[Any, List[Any]]]`

`py3plex.algorithms.hedwig.run_learner(kwarg: Dict[str, Any], kb: ExperimentKB, validator: Validate,
generator: bool = False, num_threads: str | int = 'all') →
List[Tuple[Any, List[Any]]]`

Example-related classes.

@author: anze.vavpetic@ijs.si

class `py3plex.algorithms.hedwig.core.example.Example(id, label, score, annotations=None,
weights=None)`

Bases: `object`

Represents an example with its score, label, id and annotations.

ClassLabeled = 'class'

Ranked = 'ranked'

Predicate-related classes.

@author: anze.vavpetic@ijs.si

class `py3plex.algorithms.hedwig.core.predicate.BinaryPredicate(label, pairs, kb,
producer_pred=None)`

Bases: `Predicate`

A binary predicate.

```
class py3plex.algorithms.hedwig.core.predicate.Predicate(label, kb, producer_pred)
```

Bases: object

Represents a predicate as a member of a certain rule.

i = -1

```
class py3plex.algorithms.hedwig.core.predicate.UnaryPredicate(label, members, kb,
                                                             producer_pred=None,
                                                             custom_var_name=None,
                                                             negated=False)
```

Bases: *Predicate*

A unary predicate.

The rule class.

@author: anze.vavpetic@ijs.si

```
class py3plex.algorithms.hedwig.core.rule.Rule(kb, predicates=None, target=None)
```

Bases: object

Represents a rule, along with its description, examples and statistics.

clone()

Returns a clone of this rule. The predicates themselves are NOT cloned.

clone_append(predicate_label, producer_pred, bin=False)

Returns a copy of this rule where 'predicate_label' is appended to the rule.

clone_negate(target_pred)

Returns a copy of this rule where 'target_pred' is negated.

clone_swap_with_subclass(target_pred, child_pred_label)

Returns a copy of this rule where 'target_pred' is swapped for 'child_pred_label'.

examples(positive_only=False)

Returns the covered examples.

property positives

precision()

rule_report(show_uris=False, latex=False)

Rule as string with some statistics.

static ruleset_examples_json(rules_per_target, show_uris=False)

static ruleset_report(rules, show_uris=False, latex=False, human=<function Rule.<lambda>>)

similarity(rule)

Calculates the similarity between this rule and 'rule'.

size()

Returns the number of conjuncts.

static to_json(rules_per_target, show_uris=False)

6.46.11 Force Atlas 2 Visualization

```
class py3plex.visualization.fa2.forceatlas2.ForceAtlas2(outboundAttractionDistribution=False,  
                                                    linLogMode=False, adjustSizes=False,  
                                                    edgeWeightInfluence=1.0,  
                                                    jitterTolerance=1.0,  
                                                    barnesHutOptimize=True,  
                                                    barnesHutTheta=1.2,  
                                                    multiThreaded=False, scalingRatio=2.0,  
                                                    strongGravityMode=False, gravity=1.0,  
                                                    verbose=True)
```

Bases: object

```
forceatlas2(G, pos=None, iterations=30)
```

```
forceatlas2_igraph_layout(G, pos=None, iterations=100, weight_attr=None)
```

```
forceatlas2_networkx_layout(G, pos=None, iterations=100)
```

```
init(G, pos=None)
```

```
class py3plex.visualization.fa2.forceatlas2.Timer(name="Timer")
```

Bases: object

```
display()
```

```
start()
```

```
stop()
```

```
class py3plex.visualization.fa2.fa2util.Edge
```

Bases: object

```
class py3plex.visualization.fa2.fa2util.Node
```

Bases: object

```
class py3plex.visualization.fa2.fa2util.Region(nodes)
```

Bases: object

```
applyForce(n, theta, coefficient=0)
```

```
applyForceOnNodes(nodes, theta, coefficient=0)
```

```
buildSubRegions()
```

```
updateMassAndGeometry()
```

```
py3plex.visualization.fa2.fa2util.adjustSpeedAndApplyForces(nodes, speed, speedEfficiency,  
                                                            jitterTolerance)
```

```
py3plex.visualization.fa2.fa2util.apply_attraction(nodes, edges, distributedAttraction, coefficient,  
                                                    edgeWeightInfluence)
```

```
py3plex.visualization.fa2.fa2util.apply_gravity(nodes, gravity, useStrongGravity=False)
```

```
py3plex.visualization.fa2.fa2util.apply_repulsion(nodes, coefficient)
```

```
py3plex.visualization.fa2.fa2util.linAttraction(n1, n2, e, distributedAttraction, coefficient=0)
```

```

py3plex.visualization.fa2.fa2util.linGravity(n, g)
py3plex.visualization.fa2.fa2util.linRepulsion(n1, n2, coefficient=0)
py3plex.visualization.fa2.fa2util.linRepulsion_region(n, r, coefficient=0)
py3plex.visualization.fa2.fa2util.strongGravity(n, g, coefficient=0)

```

6.46.12 Embedding Visualization

```

py3plex.visualization.embedding_visualization.embedding_visualization.visualize_embedding(multinet,
                                                                                          la-
                                                                                          bels=None,
                                                                                          ver-
                                                                                          bose=True)

```

6.46.13 Network Generation and Benchmarking

6.46.14 Additional Statistics and Analysis

```

py3plex.algorithms.statistics.bayesiantests.correlated_ttest(x, rope, runs=1, verbose=False,
                                                            names=('C1', 'C2'))

py3plex.algorithms.statistics.bayesiantests.correlated_ttest_MC(x, rope, runs=1,
                                                                nsamples=50000)

```

See correlated_ttest module for explanations

```
py3plex.algorithms.statistics.bayesiantests.heaviside(X)
```

Compute the Heaviside step function.

The Heaviside function returns 1 for positive values, 0.5 for zero, and 0 for negative values. This is used in signed-rank tests for Bayesian comparisons.

Parameters

X – Input array or scalar

Returns

Array with same shape as X containing:

- 1.0 where $X > 0$
- 0.5 where $X == 0$
- 0.0 where $X < 0$

Return type

np.ndarray

Examples

```

>>> heaviside(np.array([-1, 0, 1, 2]))
array([0. , 0.5, 1. , 1. ])

```

```

py3plex.algorithms.statistics.bayesiantests.hierarchical(diff, rope, rho, upperAlpha=2,
                                                         lowerAlpha=1, lowerBeta=0.01,
                                                         upperBeta=0.1, std_upper_bound=1000,
                                                         verbose=False, names=('C1', 'C2'))

```

Perform hierarchical Bayesian test for comparing algorithms across multiple datasets.

This test accounts for the hierarchical structure of the data (multiple datasets, each with multiple folds) and correlations due to overlapping training sets.

Parameters

- **diff** – Array of differences between classifier scores
- **rope** – Width of the region of practical equivalence (ROPE)
- **rho** – Correlation between folds (typically around $1/n_folds$)
- **upperAlpha** – Upper bound for alpha parameter of Gamma prior (default: 2)
- **lowerAlpha** – Lower bound for alpha parameter of Gamma prior (default: 1)
- **lowerBeta** – Lower bound for beta parameter of Gamma prior (default: 0.01)
- **upperBeta** – Upper bound for beta parameter of Gamma prior (default: 0.1)
- **std_upper_bound** – Upper bound multiplier for standard deviation prior (default: 1000)
Posterior is insensitive to this if large enough (>100)
- **verbose** – Whether to print probability results (default: False)
- **names** – Tuple of classifier names for verbose output (default: (“C1”, “C2”))

Returns

(p_left, p_rope, p_right)

- p_left: Probability that first classifier is worse
- p_rope: Probability that classifiers are practically equivalent
- p_right: Probability that first classifier is better

Return type

Tuple[float, float, float]

Notes

- The Gamma distribution parameters control the prior on degrees of freedom
- The hierarchical structure models between-dataset and within-dataset variance
- Use when comparing algorithms across multiple datasets with cross-validation

References

- Benavoli, A., Corani, G., & Mangili, F. (2016). Should we really use post-hoc tests based on mean-ranks? The Journal of Machine Learning Research.

See also

hierarchical_MC: Monte Carlo sampling version correlated_ttest: Simpler test for single dataset comparisons

```
py3plex.algorithms.statistics.bayesiantests.hierarchical_MC(diff, rope, rho, upperAlpha=2,  
                                                            lowerAlpha=1, lowerBeta=0.01,  
                                                            upperBeta=0.1,  
                                                            std_upper_bound=1000,  
                                                            names=('C1', 'C2'))
```


Monte Carlo sampling for hierarchical Bayesian test.

Generates Monte Carlo samples from the posterior distribution for hierarchical comparison of algorithms across multiple datasets with cross-validation.

Parameters

- **diff** – Array of differences between classifier scores (shape: `n_datasets x n_folds`)
- **rope** – Width of the region of practical equivalence (ROPE)
- **rho** – Correlation between folds due to overlapping training sets
- **upperAlpha** – Upper bound for alpha parameter of Gamma prior (default: 2)
- **lowerAlpha** – Lower bound for alpha parameter of Gamma prior (default: 1)
- **lowerBeta** – Lower bound for beta parameter of Gamma prior (default: 0.01)
- **upperBeta** – Upper bound for beta parameter of Gamma prior (default: 0.1)
- **std_upper_bound** – Upper bound multiplier for standard deviation prior (default: 1000)
- **names** – Tuple of classifier names for identification (default: (“C1”, “C2”))

Returns

Monte Carlo samples with shape (`n_samples`, 3)

Each row contains `[p_left, p_rope, p_right]` for one MC sample

Return type

`np.ndarray`

Notes

- Uses PyStan for Bayesian inference with hierarchical model
- Data is rescaled by mean standard deviation for numerical stability
- Hierarchical structure captures both within-dataset and between-dataset variance
- Requires pystan package to be installed

Implementation:

- Uses Stan’s NUTS sampler with 4 chains
- Each chain runs 100 iterations (including warmup)
- Total posterior samples: ~200 after warmup

See also

`hierarchical`: Main function that processes MC samples `correlated_ttest_MC`: Simpler MC version for single dataset

Raises

ImportError – If pystan is not installed

```
py3plex.algorithms.statistics.bayesiantests.plot_posterior(samples, names=('C1', 'C2'),  
                                                           proba_triplet=None)
```

Parameters

- **x** (*array*) – a vector of differences or a 2d array with pairs of scores.
- **names** (*pair of str*) – the names of the two classifiers

Returns

matplotlib.pyplot.figure

```
py3plex.algorithms.statistics.bayesiantests.plot_simplex(points, names=('C1', 'C2'),  
                                                         proba_triplet=None)
```

```
py3plex.algorithms.statistics.bayesiantests.signrank(x, rope, prior_strength=0.6, prior_place=1,  
                                                     nsamples=50000, verbose=False,  
                                                     names=('C1', 'C2'))
```

Parameters

- **x** (*array*) – a vector of differences or a 2d array with pairs of scores.
- **rope** (*float*) – the width of the rope
- **prior_strength** (*float*) – prior strength (default: 0.6)
- **prior_place** (*LEFT, ROPE or RIGHT*) – the region to which the prior is assigned (default: ROPE)
- **nsamples** (*int*) – the number of Monte Carlo samples
- **verbose** (*bool*) – report the computed probabilities
- **names** (*pair of str*) – the names of the two classifiers

Returns

p_left, p_rope, p_right

```
py3plex.algorithms.statistics.bayesiantests.signrank_MC(x, rope, prior_strength=0.6,  
                                                         prior_place=1, nsamples=50000)
```

Parameters

- **x** (*array*) – a vector of differences or a 2d array with pairs of scores.
- **rope** (*float*) – the width of the rope
- **prior_strength** (*float*) – prior strength (default: 0.6)
- **prior_place** (*LEFT, ROPE or RIGHT*) – the region to which the prior is assigned (default: ROPE)
- **nsamples** (*int*) – the number of Monte Carlo samples

Returns

2-d array with rows corresponding to samples and columns to probabilities [*p_left, p_rope, p_right*]

```
py3plex.algorithms.statistics.bayesiantests.signtest(x, rope, prior_strength=1, prior_place=1,  
                                                     nsamples=50000, verbose=False,  
                                                     names=('C1', 'C2'))
```

Parameters

- **x** (*array*) – a vector of differences or a 2d array with pairs of scores.
- **rope** (*float*) – the width of the rope
- **prior_strength** (*float*) – prior strength (default: 1)
- **prior_place** (*LEFT, ROPE or RIGHT*) – the region to which the prior is assigned (default: ROPE)
- **nsamples** (*int*) – the number of Monte Carlo samples
- **verbose** (*bool*) – report the computed probabilities
- **names** (*pair of str*) – the names of the two classifiers

Returns

p_left, p_rope, p_right

```
py3plex.algorithms.statistics.bayesiantests.signtest_MC(x, rope, prior_strength=1, prior_place=1,
                                                         nsamples=50000)
```

Parameters

- **x** (*array*) – a vector of differences or a 2d array with pairs of scores.
- **rope** (*float*) – the width of the rope
- **prior_strength** (*float*) – prior strength (default: 1)
- **prior_place** (*LEFT, ROPE or RIGHT*) – the region to which the prior is assigned (default: ROPE)
- **nsamples** (*int*) – the number of Monte Carlo samples

Returns

2-d array with rows corresponding to samples and columns to probabilities [*p_left, p_rope, p_right*]

6.46.15 Community Detection Advanced

Node Ranking and Clustering (NoRC) module for community detection.

This module implements algorithms for node ranking and hierarchical clustering in networks, including parallel PageRank computation and hierarchical merging.

```
py3plex.algorithms.community_detection.NoRC.NoRC_communities_main(input_graph, clustering_scheme='hierarchical',
                                                                    max_com_num=100,
                                                                    verbose=False,
                                                                    parallel_step=None,
                                                                    prob_threshold=0.0005,
                                                                    community_range=None,
                                                                    fine_range=3,
                                                                    lag_threshold=10)
```

```
py3plex.algorithms.community_detection.NoRC.page_rank_kernel(index_row)
```

Compute normalized PageRank vector for a given node.

Parameters

index_row – Node index for which to compute PageRank

Returns

Tuple of (node_index, normalized_pagerank_vector)

Community-based node ranking framework.

This module implements a framework for ranking nodes within and across communities using PageRank-based metrics and hierarchical clustering.

```
py3plex.algorithms.community_detection.community_ranking.create_tree(centers: ndarray) →  
                                                                    Dict[int, Dict[str, List]]
```

```
py3plex.algorithms.community_detection.community_ranking.page_rank_kernel(index_row: int) →  
                                                                    Tuple[int, ndarray]
```

```
py3plex.algorithms.community_detection.community_ranking.return_infomap_communities(network:  
                                                                    Any)  
                                                                    →  
                                                                    List[List[int]]
```

6.46.16 Node Ranking and Classification

Node ranking algorithms for multilayer networks.

This module provides various node ranking algorithms including PageRank variants, HITS (Hubs and Authorities), and personalized PageRank (PPR) for network analysis.

Key Functions:

- `sparse_page_rank`: Compute PageRank scores using sparse matrix operations
- `run_PPR`: Run Personalized PageRank in parallel across multiple cores
- `hubs_and_authorities`: Compute HITS scores for nodes
- `stochastic_normalization`: Normalize adjacency matrix to stochastic form

Notes

The PageRank implementations use sparse matrices for memory efficiency and support parallel computation for large-scale networks.

```
py3plex.algorithms.node_ranking.authority_matrix(graph: Graph) → ndarray
```

Get the authority matrix representation of a graph.

Computes the matrix $A = A.T @ A$ where A is the adjacency matrix. The authority matrix is used in HITS algorithm computation.

Parameters

graph – NetworkX graph

Returns

Authority matrix (N x N) where N is number of nodes

Return type

np.ndarray

Notes

- For directed graphs: $A[i,j]$ = number of nodes that point to both i and j
- Used internally by HITS algorithm to compute authority scores

See also

hub_matrix: Complementary hub matrix hubs_and_authorities: Compute actual hub/authority scores

py3plex.algorithms.node_ranking.damping_hyper: float

py3plex.algorithms.node_ranking.hub_matrix(*graph: Graph*) → ndarray

Get the hub matrix representation of a graph.

Computes the matrix $H = A @ A.T$ where A is the adjacency matrix. The hub matrix is used in HITS algorithm computation.

Parameters

graph – NetworkX graph

Returns

Hub matrix ($N \times N$) where N is number of nodes

Return type

np.ndarray

Notes

- For directed graphs: $H[i,j]$ = number of nodes pointed to by both i and j
- Used internally by HITS algorithm to compute hub scores

See also

authority_matrix: Complementary authority matrix hubs_and_authorities: Compute actual hub/authority scores

py3plex.algorithms.node_ranking.hubs_and_authorities(*graph: Graph*) → Tuple[dict, dict]

Compute HITS (Hubs and Authorities) scores for all nodes in a graph.

Implements the Hyperlink-Induced Topic Search (HITS) algorithm to identify hub nodes (nodes that point to many authorities) and authority nodes (nodes pointed to by many hubs) in a network.

Parameters

graph – NetworkX graph (directed or undirected)

Returns

(hub_scores, authority_scores)

- hub_scores: Dictionary mapping node → hub score
- authority_scores: Dictionary mapping node → authority score

Return type

Tuple[dict, dict]

Notes

- Uses scipy-based implementation from NetworkX (nx.hits_scipy)
- Scores are normalized so that the sum of squares equals 1
- For undirected graphs, hub and authority scores are identical

- Converges using power iteration method

Examples

```
>>> import networkx as nx
>>> G = nx.DiGraph([(0, 1), (0, 2), (1, 2)])
>>> hubs, authorities = hubs_and_authorities(G)
>>> # Node 0 has high hub score (points to others)
>>> # Node 2 has high authority score (pointed to by others)
```

See also

hub_matrix: Get the hub matrix representation authority_matrix: Get the authority matrix representation

`py3plex.algorithms.node_ranking.page_rank_kernel(index_row: int) → Tuple[int, ndarray]`

Compute PageRank vector for a single starting node (multiprocessing kernel).

This function is designed to be called in parallel via `multiprocessing.Pool.map()`. It computes the personalized PageRank vector starting from a single node.

Parameters

index_row – Index of the starting node for personalized PageRank

Returns

(node_index, normalized_pagerank_vector)

- node_index: The input index (for tracking results)
- pagerank_vector: L2-normalized PageRank scores for all nodes

Return type

Tuple[int, np.ndarray]

Notes

- Accesses global variables: `__graph_matrix`, `damping_hyper`, `spread_step_hyper`, `spread_percent_hyper` (set by `run_PPR` before parallel execution)
- Returns zero vector if normalization fails
- L2 normalization ensures comparable magnitudes across different starting nodes

See also

`run_PPR`: Main function that sets up parallel execution `sparse_page_rank`: Core PageRank computation

`py3plex.algorithms.node_ranking.run_PPR(network: spmatrix, cores: int | None = None, jobs: List[range] | None = None, damping: float = 0.85, spread_step: int = 10, spread_percent: float = 0.3, targets: List[int] | None = None, parallel: bool = True) → Generator[Tuple[int, ndarray] | List[Tuple[int, ndarray]], None, None]`

Run Personalized PageRank (PPR) in parallel for multiple starting nodes.

Computes personalized PageRank vectors for multiple nodes using parallel processing. This is useful for creating node embeddings or analyzing node importance from different perspectives in the network.

Parameters

- **network** – Sparse adjacency matrix (will be automatically normalized to stochastic form)
- **cores** – Number of CPU cores to use (default: all available cores)
- **jobs** – Custom job batches as list of ranges (default: auto-generated)
- **damping** – Damping factor for PageRank (default: 0.85) Higher values (0.85-0.99) emphasize network structure
- **spread_step** – Steps to check spread pattern for optimization (default: 10)
- **spread_percent** – Max node fraction for shrinkage optimization (default: 0.3)
- **targets** – Specific node indices to compute PPR for (default: all nodes)
- **parallel** – Enable parallel processing (default: True) Set to False for debugging or single-core execution

Yields

Union[Tuple[int, np.ndarray], List[Tuple[int, np.ndarray]]] – - If parallel=True: Lists of (node_index, pagerank_vector) tuples (batched) - If parallel=False: Individual (node_index, pagerank_vector) tuples

Notes

- Automatically normalizes input matrix to column-stochastic form
- Uses multiprocessing.Pool for parallel execution
- Global variables are used to share the graph matrix across processes
- Results are yielded incrementally (generator pattern) to save memory
- Each pagerank_vector is L2-normalized for comparability

Examples

```
>>> import scipy.sparse as sp
>>> # Create a small network
>>> adj = sp.csr_matrix([[0, 1, 1], [1, 0, 1], [1, 1, 0]])
>>>
>>> # Compute PPR for all nodes in parallel
>>> for batch in run_PPR(adj, cores=2, parallel=True):
...     for node_idx, pr_vector in batch:
...         print(f"Node {node_idx}: {pr_vector}")
>>>
>>> # Compute PPR for specific nodes without parallelism
>>> for node_idx, pr_vector in run_PPR(adj, targets=[0, 1], parallel=False):
...     print(f"Node {node_idx}: {pr_vector}")
```

Performance:

- Parallel speedup scales with number of cores (up to ~0.8 * cores efficiency)
- Memory usage: $O(N * N_{\text{targets}})$ for storing results
- For large networks (>100K nodes), consider processing targets in batches

See also

`sparse_page_rank`: Core PageRank computation
`page_rank_kernel`: Worker function for parallel execution
`stochastic_normalization`: Matrix normalization (called internally)

```
py3plex.algorithms.node_ranking.sparse_page_rank(matrix: spmatrix, start_nodes: List[int] | range,  
                                                  epsilon: float = 1e-06, max_steps: int = 100000,  
                                                  damping: float = 0.5, spread_step: int = 10,  
                                                  spread_percent: float = 0.3, try_shrink: bool =  
                                                  True) → ndarray
```

Compute personalized PageRank using sparse matrix operations.

Implements an efficient personalized PageRank algorithm with adaptive sparsification to reduce memory usage and computation time. The algorithm uses a power iteration method with early stopping based on convergence criteria.

Parameters

- **matrix** – Column-stochastic sparse adjacency matrix (use `stochastic_normalization` first)
- **start_nodes** – List or range of starting nodes for personalized PageRank
- **epsilon** – Convergence threshold for L1 norm difference (default: 1e-6)
- **max_steps** – Maximum number of iterations (default: 100000)
- **damping** – Damping factor / teleportation probability (default: 0.5) Higher values (e.g., 0.85) favor network structure over random jumps
- **spread_step** – Number of steps to check for sparsity pattern (default: 10)
- **spread_percent** – Maximum fraction of nodes to consider for shrinkage (default: 0.3)
- **try_shrink** – Enable adaptive shrinkage to reduce computation (default: True)

Returns

PageRank scores for all nodes, with `start_nodes` set to 0

Return type

`np.ndarray`

Notes

- Assumes matrix is column-stochastic (use `stochastic_normalization` first)
- Adaptive shrinkage identifies nodes unreachable from `start_nodes` and excludes them from computation for efficiency
- Convergence is measured by L1 norm of rank vector difference
- Start nodes are zeroed out in the final result to avoid self-importance

Complexity:

- Time: $O(k * E)$ where k is iterations and E is edges
- Space: $O(N)$ for rank vectors, plus matrix storage

Examples

```
>>> import scipy.sparse as sp
>>> # Create and normalize adjacency matrix
>>> adj = sp.csr_matrix([[0, 1, 1], [1, 0, 1], [1, 1, 0]])
>>> adj_norm = stochastic_normalization(adj)
>>> # Compute PageRank from node 0
>>> pr = sparse_page_rank(adj_norm, [0], damping=0.85)
>>> pr # PageRank scores (node 0 will be 0)
array([0. , 0.5, 0.5])
```

Raises

AssertionError – If start_nodes is empty

See also

stochastic_normalization: Required preprocessing step
run_PPR: Parallel wrapper for computing multiple PageRank vectors

py3plex.algorithms.node_ranking.spread_percent_hyper: float

py3plex.algorithms.node_ranking.spread_step_hyper: int

py3plex.algorithms.node_ranking.stochastic_normalization(matrix: spmatrix) → spmatrix

Normalize a sparse matrix to stochastic form (column-stochastic).

Converts an adjacency matrix to a stochastic matrix where each column sums to 1. This normalization is required for PageRank-style random walk algorithms.

Parameters

matrix – Sparse adjacency matrix to normalize

Returns

Column-stochastic sparse matrix where each column sums to 1

Return type

sp.spmatrix

Notes

- Removes self-loops (sets diagonal to 0) before normalization
- Handles zero-degree nodes by leaving corresponding columns as zeros
- Preserves sparsity structure for memory efficiency

Examples

```
>>> import scipy.sparse as sp
>>> adj = sp.csr_matrix([[0, 1, 1], [1, 0, 1], [1, 1, 0]])
>>> stoch_adj = stochastic_normalization(adj)
>>> stoch_adj.sum(axis=1).A1 # Column sums should be 1
array([1., 1., 1.])
```

```
py3plex.algorithms.network_classification.benchmark_classification(matrix: spmatrix, labels:
                                                                    ndarray, alpha_value: float
                                                                    = 0.85, iterations: int = 30,
                                                                    normalization_scheme: str =
                                                                    'freq', dataset_name: str =
                                                                    'example', verbose: bool =
                                                                    False, test_size: float | None
                                                                    = None) → DataFrame
```

```
py3plex.algorithms.network_classification.label_propagation(graph_matrix: spmatrix,
                                                             class_matrix: ndarray, alpha: float =
                                                             0.001, epsilon: float = 1e-12,
                                                             max_steps: int = 100000,
                                                             normalization: str | List[str] = 'freq')
                                                             → ndarray
```

Propagate labels through a graph.

Parameters

- **graph_matrix** – Sparse graph adjacency matrix
- **class_matrix** – Initial class label matrix
- **alpha** – Propagation weight parameter
- **epsilon** – Convergence threshold
- **max_steps** – Maximum number of iterations
- **normalization** – Normalization scheme(s) to apply

Returns

Propagated label matrix

```
py3plex.algorithms.network_classification.label_propagation_normalization(matrix: spmatrix)
                                                                    → spmatrix
```

Normalize a matrix for label propagation.

Parameters

matrix – Sparse matrix to normalize

Returns

Normalized sparse matrix

```
py3plex.algorithms.network_classification.label_propagation_tf() → None
```

```
py3plex.algorithms.network_classification.normalize_amplify_freq(mat: ndarray) → ndarray
```

Normalize and amplify matrix by frequency.

Parameters

mat – Matrix to normalize

Returns

Normalized and amplified matrix

```
py3plex.algorithms.network_classification.normalize_exp(mat: ndarray) → ndarray
```

Apply exponential normalization.

Parameters

mat – Matrix to normalize

Returns

Exponentially normalized matrix

`py3plex.algorithms.network_classification.normalize_initial_matrix_freq(mat: ndarray) → ndarray`

Normalize matrix by frequency.

Parameters

mat – Matrix to normalize

Returns

Normalized matrix

`py3plex.algorithms.network_classification.validate_label_propagation(core_network: spmatrix, labels: ndarray | spmatrix, dataset_name: str = 'test', repetitions: int = 5, normalization_scheme: str | List[str] = 'basic', alpha_value: float = 0.001, random_seed: int = 123, verbose: bool = False) → DataFrame`

Validate label propagation with cross-validation.

Parameters

- **core_network** – Sparse network adjacency matrix
- **labels** – Label matrix
- **dataset_name** – Name of the dataset
- **repetitions** – Number of repetitions
- **normalization_scheme** – Normalization scheme to use
- **alpha_value** – Alpha parameter for propagation
- **random_seed** – Random seed for reproducibility
- **verbose** – Whether to print progress

Returns

DataFrame with validation results

6.46.17 Embeddings and Wrappers

`py3plex.wrappers.train_node2vec_embedding.call_node2vec_binary(input_graph: str, output_graph: str, p: float = 1, q: float = 1, dimension: int = 128, directed: bool = False, weighted: bool = True, binary: str | None = None, timeout: int = 300) → None`

Call the Node2Vec C++ binary with specified parameters.

Parameters

- **input_graph** – Path to input graph file
- **output_graph** – Path to output embedding file

- **p** – Return parameter
- **q** – In-out parameter
- **dimension** – Embedding dimension
- **directed** – Whether graph is directed
- **weighted** – Whether graph is weighted
- **binary** – Path to node2vec binary (defaults to PY3PLEX_NODE2VEC_BINARY env var or “./node2vec”)
- **timeout** – Maximum execution time in seconds

Raises

ExternalToolError – If binary is not found or execution fails

```
py3plex.wrappers.train_node2vec_embedding.learn_embedding(core_network: Any, labels: List[Any] |  
                                                         None = None, ssize: float = 0.5,  
                                                         embedding_outfile: str = 'out.emb', p:  
                                                         float = 0.1, q: float = 0.1, binary_path:  
                                                         str | None = None, parameter_range: str  
                                                         = '[0.25,0.50,1,2,4]', timeout: int =  
                                                         300) → Tuple[str, float]
```

Learn node embeddings for a network.

Parameters

- **core_network** – NetworkX graph
- **labels** – Node labels
- **ssize** – Sample size
- **embedding_outfile** – Output file for embeddings
- **p** – Return parameter
- **q** – In-out parameter
- **binary_path** – Path to node2vec binary (defaults to PY3PLEX_NODE2VEC_BINARY env var or “./node2vec”)
- **parameter_range** – String representation of parameter range list
- **timeout** – Maximum execution time in seconds per call

Returns

Tuple of (method_name, elapsed_time)

```
py3plex.wrappers.train_node2vec_embedding.n2v_embedding(G: Any, targets: Any, verbose: bool =  
                                                         False, sample_size: float = 0.5,  
                                                         outfile_name: str = 'test.emb', p: float |  
                                                         None = None, q: float | None = None,  
                                                         binary_path: str | None = None,  
                                                         parameter_range: List[float] | None =  
                                                         None, embedding_dimension: int = 128,  
                                                         timeout: int = 300) → None
```

Train Node2Vec embeddings with parameter optimization.

Parameters

- **G** – NetworkX graph

- **targets** – Target labels for nodes
- **verbose** – Whether to print verbose output
- **sample_size** – Sample size for training
- **outfile_name** – Output embedding file name
- **p** – Return parameter (None triggers grid search)
- **q** – In-out parameter (None triggers grid search)
- **binary_path** – Path to node2vec binary (defaults to PY3PLEX_NODE2VEC_BINARY env var or “./node2vec”)
- **parameter_range** – Range of parameters to search
- **embedding_dimension** – Dimension of embeddings
- **timeout** – Maximum execution time in seconds per call

6.46.18 Network Motifs and Patterns

6.46.19 HINMINE Data Structures

class py3plex.core.HINMINE.dataStructures.**Class**(*lab_id, name, members*)

Bases: object

class py3plex.core.HINMINE.dataStructures.**HeterogeneousInformationNetwork**(*network, label_delimiter, weight_tag=False, target_tag=True*)

Bases: object

add_label(*node, label_id, label_name=None*)

calculate_decomposition_candidates(*max_decomposition_length=10*)

calculate_schema()

create_label_matrix(*weights=None*)

decompose_from_iterator(*name, weighing, summing, generator=None, degrees=None, parallel=False, pool=None*)

midpoint_generator(*node_sequence, edge_sequence*)

process_network(*label_delimiter*)

split_to_indices(*train_indices=(), validate_indices=(), test_indices=()*)

split_to_parts(*lst, n*)

6.46.20 Command-Line Interface

Command-line interface for py3plex.

This module provides a comprehensive CLI tool for multilayer network analysis with full coverage of main algorithms.

`py3plex.cli.cmd_aggregate(args: Namespace) → int`

Aggregate multilayer network into single layer.

Parameters

args – Parsed command-line arguments

Returns

Exit code (0 for success)

`py3plex.cli.cmd_centrality(args: Namespace) → int`

Compute node centrality measures.

Parameters

args – Parsed command-line arguments

Returns

Exit code (0 for success)

`py3plex.cli.cmd_check(args: Namespace) → int`

Lint and validate a graph data file.

Parameters

args – Parsed command-line arguments

Returns

Exit code (0 for success, non-zero for errors)

`py3plex.cli.cmd_community(args: Namespace) → int`

Detect communities in the network.

Parameters

args – Parsed command-line arguments

Returns

Exit code (0 for success)

`py3plex.cli.cmd_convert(args: Namespace) → int`

Convert network between different formats.

Parameters

args – Parsed command-line arguments

Returns

Exit code (0 for success)

`py3plex.cli.cmd_create(args: Namespace) → int`

Create a new multilayer network.

Parameters

args – Parsed command-line arguments

Returns

Exit code (0 for success)

`py3plex.cli.cmd_help(args: Namespace) → int`

Show detailed help information.

Parameters

args – Parsed command-line arguments

Returns

Exit code (0 for success)

`py3plex.cli.cmd_load(args: Namespace) → int`

Load and inspect a network.

Parameters

args – Parsed command-line arguments

Returns

Exit code (0 for success)

`py3plex.cli.cmd_quickstart(args: Namespace) → int`

Run quickstart demo - creates a tiny demo graph and demonstrates basic operations.

Parameters

args – Parsed command-line arguments

Returns

Exit code (0 for success)

`py3plex.cli.cmd_run_config(args: Namespace) → int`

Run workflow from configuration file.

Parameters

args – Parsed command-line arguments

Returns

Exit code (0 for success)

`py3plex.cli.cmd_selftest(args: Namespace) → int`

Run self-test to verify installation and core functionality.

Parameters

args – Parsed command-line arguments

Returns

Exit code (0 for success)

`py3plex.cli.cmd_stats(args: Namespace) → int`

Compute multilayer network statistics.

Parameters

args – Parsed command-line arguments

Returns

Exit code (0 for success)

`py3plex.cli.cmd_visualize(args: Namespace) → int`

Visualize the multilayer network.

Parameters

args – Parsed command-line arguments

Returns

Exit code (0 for success)

`py3plex.cli.create_parser() → ArgumentParser`

Create and configure the argument parser for py3plex CLI.

Returns

Configured ArgumentParser instance

`py3plex.cli.main(argv: List[str] | None = None) → int`

Main entry point for the CLI.

Parameters

argv – Command-line arguments (defaults to `sys.argv`)

Returns

Exit code (0 for success, non-zero for errors)

6.46.21 Validation Utilities

Input validation utilities for Py3plex.

This module provides pre-validation functions to catch common input errors early and provide clear, actionable error messages to users.

`py3plex.validation.validate_csv_columns(file_path: str, required_columns: List[str], optional_columns: List[str] | None = None) → None`

Validate that a CSV file has required columns.

Parameters

- **file_path** – Path to CSV file
- **required_columns** – List of column names that must be present
- **optional_columns** – List of column names that are optional

Raises

ParsingError – If required columns are missing

Contracts:

- Precondition: `file_path` must be a non-empty string
- Precondition: `required_columns` must be a non-empty list of strings

`py3plex.validation.validate_edgelist_format(file_path: str, delimiter: str = None) → None`

Validate simple edgelist file format (source target weight).

Parameters

- **file_path** – Path to edgelist file
- **delimiter** – Optional delimiter (default: whitespace)

Raises

ParsingError – If file format is invalid

Contracts:

- Precondition: `file_path` must be a non-empty string
- Precondition: `delimiter` must be `None` or a string

`py3plex.validation.validate_file_exists(file_path: str) → None`

Validate that a file exists and is readable.

Parameters

file_path – Path to file to validate

Raises

ParsingError – If file doesn't exist or isn't readable

Contracts:

- Precondition: `file_path` must be a non-empty string

`py3plex.validation.validate_input_type(input_type: str, valid_types: Set[str] | None = None) → None`

Validate that `input_type` is recognized.

Parameters

- **`input_type`** – The input type string to validate
- **`valid_types`** – Optional set of valid types (uses default if `None`)

Raises

ParsingError – If `input_type` is not valid

Contracts:

- Precondition: `input_type` must be a non-empty string
- Precondition: `valid_types` must be `None` or a set

`py3plex.validation.validate_multiedgelist_format(file_path: str, delimiter: str = None) → None`

Validate multiedgelist file format (source target layer weight).

Parameters

- **`file_path`** – Path to multiedgelist file
- **`delimiter`** – Optional delimiter (default: whitespace)

Raises

ParsingError – If file format is invalid

Contracts:

- Precondition: `file_path` must be a non-empty string
- Precondition: `delimiter` must be `None` or a string

`py3plex.validation.validate_network_data(file_path: str, input_type: str) → None`

Validate network data before parsing.

This is the main entry point for validation. It performs appropriate validation based on the input type.

Parameters

- **`file_path`** – Path to network file
- **`input_type`** – Type of input file

Raises

ParsingError – If validation fails

Contracts:

- Precondition: `file_path` must be a string
- Precondition: `input_type` must be a non-empty string

6.46.22 Network Comparison and Testing

6.46.23 Hedwig Learning Algorithms

Main learner class.

@author: anze.vavpetic@ijs.si

```
class py3plex.algorithms.hedwig.learners.learner.Learner(kb, n=None, min_sup=1, sim=1, depth=4,  
                                                         target=None, use_negations=False,  
                                                         optimal_subclass=False)
```

Bases: object

Learner class, supporting various types of induction from the knowledge base.

Default = 'default'

Improvement = 'improvement'

Similarity = 'similarity'

can_specialize(*rule*)

Is the rule good enough to be further refined?

extend(*rules, specializations*)

Extends the ruleset in the given way.

extend_replace_worst(*rules, specializations*)

Extends the list by replacing the worst rules.

extend_with_similarity(*rules, specializations*)

Extends the list based on how similar is 'new_rule' to the rules contained in 'rules'.

get_subclasses(*pred*)

get_superclasses(*pred*)

group_score(*rules*)

Calculates the score of the whole list of rules.

induce()

Induces rules for the given knowledge base.

is_implicit_root(*pred*)

non_redundant(*rule, new_rule*)

Is the rule non-redundant compared to its immediate generalization?

specialize(*rule*)

Returns a list of all specializations of 'rule'.

specialize_add_relation(*rule*)

Specialize with new binary relation.

Main learner class.

@author: anze.vavpetic@ijs.si

```
class py3plex.algorithms.hedwig.learners.bottomup.BottomUpLearner(kb, n=None, min_sup=1,  
                                                                sim=1, depth=4, target=None,  
                                                                use_negations=False)
```

Bases: `object`

Bottom-up learner.

Default = `'default'`

Improvement = `'improvement'`

Similarity = `'similarity'`

bottom_clause()

get_subclasses(*pred*)

get_superclasses(*pred*)

induce()

Induces rules for the given knowledge base.

is_implicit_root(*pred*)

Main learner class.

@author: anze.vavpetic@ijs.si

```
class py3plex.algorithms.hedwig.learners.optimal.OptimalLearner(kb, n=None, min_sup=1, sim=1,  
                                                                depth=4, target=None,  
                                                                use_negations=False,  
                                                                optimal_subclass=True)
```

Bases: `Learner`

Finds the optimal top-k rules.

induce()

Induces rules for the given knowledge base.

6.46.24 Hedwig Statistics and Scoring

Score function definitions.

@author: anze.vavpetic@ijs.si

`py3plex.algorithms.hedwig.stats.scorefunctions.chisq(rule)`

`py3plex.algorithms.hedwig.stats.scorefunctions.enrichment_score(rule)`

`py3plex.algorithms.hedwig.stats.scorefunctions.interesting(rule)`

Checks if a given rule is interesting for the given score function

`py3plex.algorithms.hedwig.stats.scorefunctions.kaplan_meier_AUC(rule)`

`py3plex.algorithms.hedwig.stats.scorefunctions.leverage(rule)`

`py3plex.algorithms.hedwig.stats.scorefunctions.lift(rule)`

`py3plex.algorithms.hedwig.stats.scorefunctions.precision(rule)`

py3plex.algorithms.hedwig.stats.scorefunctions.**t_score**(rule)

py3plex.algorithms.hedwig.stats.scorefunctions.**wracc**(rule)

py3plex.algorithms.hedwig.stats.scorefunctions.**z_score**(rule)

Module for ruleset validation.

@author: anze.vavpetic@ijs.si

```
class py3plex.algorithms.hedwig.stats.validate.Validate(kb, significance_test=<function
    apply_fisher>, adjustment=<function
    fdr>)
```

Bases: object

test(ruleset, alpha=0.05, q=0.01)

Tests the given ruleset and returns the significant rules.

6.46.25 Hedwig Core Components

```
py3plex.algorithms.hedwig.core.converters.convert_mapping_to_rdf(input_mapping_file,
    extract_subnode_info=False,
    split_node_by=':',
    keep_index=1,
    layer_type='uniprotkb', annotation_mapping_file='test.gaf',
    go_identifier='GO:',
    prepend_string=None)
```

py3plex.algorithms.hedwig.core.converters.**obo2n3**(obofile, n3out, gaf_file)

Knowledge-base class.

@author: anze.vavpetic@ijs.si

```
class py3plex.algorithms.hedwig.core.kb.ExperimentKB(triplets, score_fun, instances_as_leaves=True)
```

Bases: object

The knowledge base for one specific experiment.

add_sub_class(sub, obj)

Adds the resource 'sub' as a subclass of 'obj'.

bits_to_indices(bits)

Converts the bitset to a set of indices.

get_domains(predicate)

Returns the bitsets for input and output examples of the binary predicate 'predicate'.

get_empty_domain()

Returns a bitset covering no examples.

get_examples()

Returns all examples for this experiment.

get_full_domain()

Returns a bitset covering all examples.

get_members(*predicate*, *bit=True*)

Returns the examples for this predicate, either as a bitset or a set of ids.

get_reverse_members(*predicate*, *bit=True*)

Returns the examples for this predicate, either as a bitset or a set of ids.

get_root()

Root predicate, which covers all examples.

get_score(*ex_idx*)

Returns the score for example id 'ex_idx'.

get_subclasses(*predicate*, *producer_pred=None*)

Returns a list of subclasses (as predicate objects) for 'predicate'.

indices_to_bits(*indices*)

Converts the indices to a bitset.

is_discrete_target()

n_examples()

Returns the number of examples.

n_members(*predicate*)

super_classes(*pred*)

Returns all super classes of pred (with transitivity).

Global settings file.

@author: anze.vavpetic@ijs.si

class py3plex.algorithms.hedwig.core.settings.Defaults

Bases: object

ADJUST = 'fwer'

ALPHA = 0.05

BEAM_SIZE = 20

COVERED = None

DEPTH = 5

FDR_Q = 0.05

FORMAT = 'n3'

LEARNER = 'heuristic'

LEAVES = False

MODE = 'subgroups'

NEGATIONS = False

NO_CACHE = False

OPTIMAL_SUBCLASS = False

```
OUTPUT = None
SCORE = 'lift'
SUPPORT = 0.1
TARGET = None
URIS = False
VERBOSE = False
```

6.46.26 Time Series and Temporal Analysis

6.46.27 Advanced Visualization

Sankey diagram visualization for multilayer networks.

This module provides inter-layer flow visualization to show connection strength between layers in multilayer networks. The visualization displays flows as arrows with widths proportional to the number of inter-layer connections.

Note: This uses a simplified flow diagram approach rather than matplotlib's Sankey class, as the Sankey class is designed for more complex flow networks and doesn't map directly to multilayer network inter-layer connections.

```
py3plex.visualization.sankey.draw_multilayer_sankey(graphs: List[Graph], multilinks: Dict[str,
List[Tuple]], labels: List[str] | None = None, ax:
Any | None = None, display: bool = True,
**kwargs) → Any
```

Draw inter-layer flow diagram showing connection strength in multilayer networks.

Creates a flow visualization where: - Each layer is represented in the diagram - Flows between layers show the strength (number) of inter-layer connections - Flow width/text indicates the number of inter-layer edges

Parameters

- **graphs** – List of NetworkX graphs, one per layer
- **multilinks** – Dictionary mapping edge_type -> list of multi-layer edges
- **labels** – Optional list of layer labels. If None, uses layer indices
- **ax** – Matplotlib axes to draw on. If None, creates new figure
- **display** – If True, calls plt.show() at the end
- ****kwargs** – Reserved for future extensions

Returns

Matplotlib axes object

Examples

```
>>> network = multi_layer_network()
>>> network.load_network("data.txt", input_type="multiedgelist")
>>> labels, graphs, multilinks = network.get_layers()
>>> draw_multilayer_sankey(graphs, multilinks, labels=labels)
```

Note

This visualization is most effective for networks with 2-5 layers. For networks with many layers, the diagram may become cluttered. The implementation uses a simplified flow visualization approach.

6.46.28 Link Prediction

6.47 Network Construction

The `multi_layer_network` class is the **core data structure** in py3plex.

6.47.1 Basic Usage

```
from py3plex.core import multinet

# Initialize network
network = multinet.multi_layer_network()

# Load network
network.load_network("data.gml", directed=False, input_type="gml")

# Basic statistics
network.basic_stats()
```

6.47.2 Supported Input Formats

py3plex supports **all NetworkX formats** plus **multilayer-specific formats**:

- **edgelist** - Simple edge lists
- **multiedgelist** - Multilayer edge lists (**node1 layer1 node2 layer2 weight**)
- **multiplex_edges** - Multiplex format (**layer node1 node2 weight**)
- **gml** - GML format
- **gpickle** - Compressed NetworkX format
- **graphml** - GraphML format
- **nx** - Direct NetworkX object

6.47.3 Multilayer Conventions

- Nodes and edges can have a **type** flag
- Edges can have **weights**
- Node labels use **delimiter-separated** classes (e.g., `class1---class2`)
- Specify delimiter with: `load_network(..., label_delimiter="---")`

6.47.4 Examples

For **detailed examples**, see:

- `example_IO.py` - **Loading and saving networks**
- `example_manipulation.py` - **Network construction**
- `example_multilayer_functionality.py` - **Core operations**

Repository: <https://github.com/SkBlaz/Py3Plex/tree/master/examples>

6.48 Community Detection

py3plex provides **wrappers for community detection algorithms** optimized for **multilayer networks**.

6.48.1 Supported Algorithms

- **Infomap** - **Information-theoretic**, supports overlapping communities
- **Louvain** - **Modularity optimization**
- **Label Propagation** - **Semi-supervised learning**
- **NoRC (Node Ranking and Clustering)** - **PageRank-based hierarchical clustering**

6.48.2 Basic Usage

Louvain:

```
from py3plex.algorithms.community_detection import community_wrapper as cw
from py3plex.core import multinet

network = multinet.multi_layer_network().load_network(
    "../datasets/cora.mat", directed=False, input_type="sparse")

partition = cw.louvain_communities(network)
```

NoRC (Node Ranking and Clustering):

```
from py3plex.algorithms.community_detection.NoRC import NoRC_communities_main
import networkx as nx

# Create or load a graph
G = nx.karate_club_graph()

# Detect communities using k-means clustering
communities_kmeans = NoRC_communities_main(
    G,
    clustering_scheme="kmeans",
    verbose=True,
    community_range=[2, 3, 5, 7, 10]
)

# Or use hierarchical clustering
communities_hierarchical = NoRC_communities_main(
    G,
```

(continues on next page)

(continued from previous page)

```
clustering_scheme="hierarchical",
verbose=True,
community_range=[2, 3, 5, 7, 10]
)
```

NoRC Parameters:

- `clustering_scheme`: “kmeans” or “hierarchical” (default: “hierarchical”)
- `parallel_step`: Number of parallel workers (default: auto-detect based on CPU count)
- `prob_threshold`: Threshold for PageRank probabilities (default: 0.0005)
- `community_range`: List of community counts to try (default: [1, 3, 5, 7, 11, 20, 40, 50, 100, 200, 300])
- `lag_threshold`: Early stopping after N iterations without improvement (default: 10)
- `fine_range`: Range for fine-grained search around optimal solution (default: 3)

Infomap (multiplex):**Note**

Infomap requires an external binary that is no longer bundled with py3plex.

Options:

- Download from: <https://www.mapequation.org/infomap/>
- Install via: `pip install infomap`
- Use Louvain (above) as a Python-only alternative

```
network = multinet.multi_layer_network(network_type="multiplex").load_network(
    "../datasets/simple_multiplex.edgelist",
    directed=False,
    input_type="multiplex_edges")

# Assumes Infomap binary is in PATH or current directory
partition = cw.infomap_communities(
    network, binary="infomap", multiplex=True, verbose=True)
```

6.48.3 Examples

See **detailed examples**:

- `example_community_detection.py` - **Basic community detection**
- `example_community_multiplex.py` - **Multiplex community detection**
- `example_community_visualization.py` - **Visualizing communities**

Repository: <https://github.com/SkBlaz/Py3Plex/tree/master/examples>

6.49 Contributing to py3plex

We welcome contributions to py3plex. This guide explains how to contribute effectively.

6.49.1 Ways to Contribute

You can contribute in many ways:

- Report bugs - Open issues for bugs you encounter
- Suggest features - Propose new features or improvements
- Write documentation - Improve or expand documentation
- Fix bugs - Submit pull requests fixing issues
- Add features - Implement new algorithms or capabilities
- Write tests - Improve test coverage
- Review PRs - Help review pull requests from others

6.49.2 Getting Started

Fork and Clone

1. Fork the repository on GitHub
2. Clone your fork:

```
git clone https://github.com/YOUR_USERNAME/py3plex.git
cd py3plex
```

3. Add upstream remote:

```
git remote add upstream https://github.com/SkBlaz/py3plex.git
```

Development Setup

Install in development mode with all dependencies:

```
# Create virtual environment
python3 -m venv venv
source venv/bin/activate # On Windows: venv\Scripts\activate

# Install in editable mode with dev dependencies
pip install -e ".[dev]"
```

This installs:

- Core dependencies
- Testing tools (pytest, coverage)
- Linting tools (black, ruff, isort, mypy)
- Documentation tools (sphinx)

6.49.3 Development Workflow

Create a Branch

Always create a new branch for your work:

```
git checkout -b feature/my-new-feature
# or
git checkout -b fix/issue-123
```

Make Changes

1. Write your code following our coding standards (see below)
2. Add or update tests
3. Update documentation
4. Run linters and tests

Run Tests

```
# Run all tests
make test

# Or directly with pytest
python run_tests.py
```

Run Linters

```
# Run all linters
make lint

# Or individual tools
make format # Auto-format code (black, isort)
ruff check .
mypy py3plex
```

Commit Changes

Write clear, descriptive commit messages:

```
git add .
git commit -m "Add feature X to support Y

- Implement algorithm for Z
- Add tests for edge cases
- Update documentation"
```

Push and Create PR

```
git push origin feature/my-new-feature
```

Then create a Pull Request on GitHub with:

- Clear title describing the change

- Description of what changed and why
- Reference to related issues (e.g., “Fixes #123”)
- Screenshots for UI changes

6.49.4 Coding Standards

Code Style

We follow PEP 8 with these specifics:

- Line length: 100 characters (not 80)
- Indentation: 4 spaces (no tabs)
- String quotes: Single quotes preferred (‘text’ not “text”)
- Imports: Grouped and sorted (using isort)

Auto-format your code:

```
make format
```

This runs:

- isort - Sort and organize imports
- black - Format code consistently
- ruff --fix - Auto-fix linting issues

Naming Conventions

- Functions/variables: snake_case
- Classes: PascalCase
- Constants: UPPER_SNAKE_CASE
- Private members: _leading_underscore

```
# Good
def compute centrality(network, node_id):
    MAX_ITERATIONS = 100
    centrality_scores = {}
    return centrality_scores

class MultilayerNetwork:
    def __init__(self):
        self._internal_state = {}
```

Docstrings

Use NumPy-style docstrings for all public functions and classes:

```
def multilayer_degree centrality(network, layer=None, weighted=False):
    """
    Compute degree centrality for multilayer network.

    Parameters
```

(continues on next page)

(continued from previous page)

```

-----
network : multi_layer_network
    The multilayer network to analyze.
layer : str, optional
    Specific layer to analyze. If None, aggregates across all layers.
weighted : bool, default=False
    If True, compute strength (weighted degree) instead of degree.

Returns
-----
dict
    Dictionary mapping node names to centrality scores.

Examples
-----
>>> network = multinet.multi_layer_network()
>>> network.add_edges([[ 'A', 'L1', 'B', 'L1', 1]])
>>> centrality = multilayer_degree_centrality(network)
>>> centrality['A---L1']
1.0

See Also
-----
multilayer_betweenness_centrality : Betweenness centrality

Notes
-----
For directed networks, this computes out-degree by default.

References
-----
.. [1] De Domenico, M., et al. (2013). Mathematical formulation of
    multilayer networks. Physical Review X, 3(4), 041022.
"""
pass

```

Type Hints

Add type hints to improve code clarity:

```

from typing import Dict, List, Optional
from py3plex.core.multinet import multi_layer_network

def compute_statistics(
    network: multi_layer_network,
    layers: Optional[List[str]] = None
) -> Dict[str, float]:
    """Compute network statistics."""
    pass

```

6.49.5 Testing Guidelines

Test Requirements

All new code should include tests:

- Unit tests for individual functions
- Integration tests for workflows
- Edge cases for boundary conditions
- Documentation tests for examples in docstrings

Writing Tests

Use pytest for testing:

```
# tests/test_my_feature.py
import pytest
from py3plex.core import multinet
from py3plex.algorithms.statistics import multilayer_statistics as mls

def test_layer_density():
    """Test layer density calculation."""
    # Setup
    network = multinet.multi_layer_network()
    network.add_edges([
        ['A', 'L1', 'B', 'L1', 1],
        ['B', 'L1', 'C', 'L1', 1],
    ], input_type='list')

    # Execute
    density = mls.layer_density(network, 'L1')

    # Assert
    assert 0 <= density <= 1
    assert density == pytest.approx(0.333, abs=0.01)

def test_invalid_layer():
    """Test error handling for invalid layer."""
    network = multinet.multi_layer_network()

    with pytest.raises(ValueError):
        mls.layer_density(network, 'nonexistent')
```

Test Coverage

Aim for high test coverage:

```
# Run tests with coverage
pytest --cov=py3plex --cov-report=html

# View coverage report
open htmlcov/index.html # macOS
# or
xdg-open htmlcov/index.html # Linux
```

Target: >80% coverage for new code

6.49.6 Documentation

Update Documentation

When adding features, update:

1. **Docstrings** in the code
2. **RST files** in `docfiles/` if adding major features
3. **Examples** in `examples/` directory
4. **README** if changing installation or basic usage

Build Documentation

```
cd docfiles
make html

# View documentation
open _build/html/index.html
```

Documentation Style

- Clear and concise - Use simple language
- Code examples - Include working examples
- Cross-references - Link to related documentation
- Visual aids - Add diagrams where helpful

6.49.7 Pull Request Guidelines

Before Submitting

[OK] Code follows style guide (`make lint` passes)

[OK] All tests pass (`make test` passes)

[OK] New code has tests

[OK] Documentation is updated

[OK] Commit messages are clear

[OK] Branch is up to date with main

PR Description

Include in your PR description:

- What changed
- Why the change is needed
- How you implemented it
- Testing done
- Screenshots for visual changes

- Breaking changes if any

Example PR template:

```
## Description

Implements multilayer PageRank centrality following [citation].

## Motivation

Closes #123 - needed for analyzing directed multilayer networks.

## Changes

- Add `multilayer_pagerank()` function
- Add tests with 90% coverage
- Add example in `example_centrality.py`
- Update documentation in `centrality.rst`

## Testing

- [x] Unit tests pass
- [x] Integration test on real network
- [x] Tested on Python 3.8, 3.10, 3.12
- [x] Linting passes

## Breaking Changes

None.
```

Review Process

1. Maintainers will review your PR
2. Address any feedback or requested changes
3. Once approved, your PR will be merged
4. Your contribution will be in the next release!

6.49.8 Reporting Issues

Bug Reports

When reporting bugs, include:

- Description of the bug
- Steps to reproduce
- Expected behavior
- Actual behavior
- Python version (`python --version`)
- py3plex version
- Operating system

- Minimal code example

Example:

```
## Bug Description

`layer_density()` returns negative values for certain graphs.

## Steps to Reproduce

```python
network = multinet.multi_layer_network()
network.add_edges([['A', 'L1', 'B', 'L1', -1]])
density = mls.layer_density(network, 'L1')
print(density) # -0.5
```

## Expected

Density should be between 0 and 1, or raise ValueError for negative weights.

## Actual

Returns -0.5

## Environment

- Python 3.10.5
- py3plex 0.95a
- Ubuntu 22.04
```

Feature Requests

For feature requests, describe:

- Use case - What problem does it solve?
- Proposed solution - How should it work?
- Alternatives - Other approaches considered
- Additional context - Examples, papers, etc.

6.49.9 Code of Conduct

- Be respectful and inclusive
- Welcome newcomers
- Focus on what is best for the community
- Show empathy towards other community members

Contributors are expected to follow these principles to foster a welcoming and productive community.

6.49.10 License

By contributing, you agree that your contributions will be licensed under the MIT License.

6.49.11 Recognition

Contributors are recognized in:

- GitHub contributors page
- Release notes
- *Acknowledgements* section in the documentation

6.49.12 Getting Help

- Questions: Open a GitHub Discussion
- Chat: Join our community chat (link in README)
- Email: Contact maintainers at blaz.skrlj@ijs.si

6.49.13 Next Steps

- Read *Development Guide* for development workflow details
- See *Architecture and Design* for system architecture
- Browse [existing issues](#)
- Check [good first issues](#)

Thank you for contributing to py3plex!

6.50 Development Guide

This guide covers **development workflows**, **Makefile commands**, **testing**, and **contributing** to py3plex.

6.50.1 Getting Started

Clone the repository and install in **development mode**:

```
git clone https://github.com/SkBlaz/py3plex.git
cd py3plex

# Setup development environment
make setup

# Install package in editable mode with dev dependencies
make dev-install
```

6.50.2 Makefile Commands

The project includes a **production-grade Makefile** for streamlined development:

```
# View all available commands
make help
```

(continues on next page)

(continued from previous page)

```

# Environment setup
make setup          # Create virtual environment
make dev-install    # Install in editable mode

# Code quality
make format         # Auto-format code
make lint           # Run linters

# Testing
make test           # Run tests with coverage
make coverage       # Open coverage report

# Documentation
make docs           # Build Sphinx documentation

# Build & publish
make clean          # Remove build artifacts
make build          # Build distributions
make publish        # Publish to PyPI

# CI
make ci             # Run full CI checks

```

Key Features:

- **Smart tool detection:** Uses `.venv/bin/` tools if available, otherwise falls back to **global tools**
- **Colorized output:** Clear **success/warning/error** messages
- **Cross-platform:** Works on **Linux** and **macOS**

6.50.3 Testing

Quick testing:

```
python run_tests.py
```

Development testing with pytest:

```

# Run all tests
pytest

# Run with coverage
pytest --cov=py3plex --cov-report=html

# Run specific test file
pytest tests/test_core.py

# Using Makefile (recommended)
make test

```

6.50.4 Code Quality

Tools configured in `pyproject.toml`:

- **Black**: Code formatting
- **Ruff**: Fast linting
- **isort**: Import sorting
- **Mypy**: Type checking
- **Pytest**: Testing with coverage

Before committing:

```
make format  # Auto-format code
make lint    # Check code quality
make test    # Run tests
make ci      # Run all checks
```

6.50.5 Contributing

We welcome contributions! Here's how:

1. Fork the repository
2. Create a feature branch
3. Make your changes
4. Run tests and linting: `make ci`
5. Commit with clear messages
6. Push to your fork
7. Open a Pull Request

Code Guidelines:

- Follow existing code style (enforced by Black and Ruff)
- Add tests for new features
- Update documentation as needed
- Keep changes focused and atomic
- Write clear commit messages

6.50.6 Documentation

Build Sphinx documentation:

```
# Using Makefile (recommended)
make docs

# Manual build
cd docfiles
sphinx-build -b html . _build/html
```

The built documentation will be in `docfiles/_build/html/`.

6.50.7 Resources

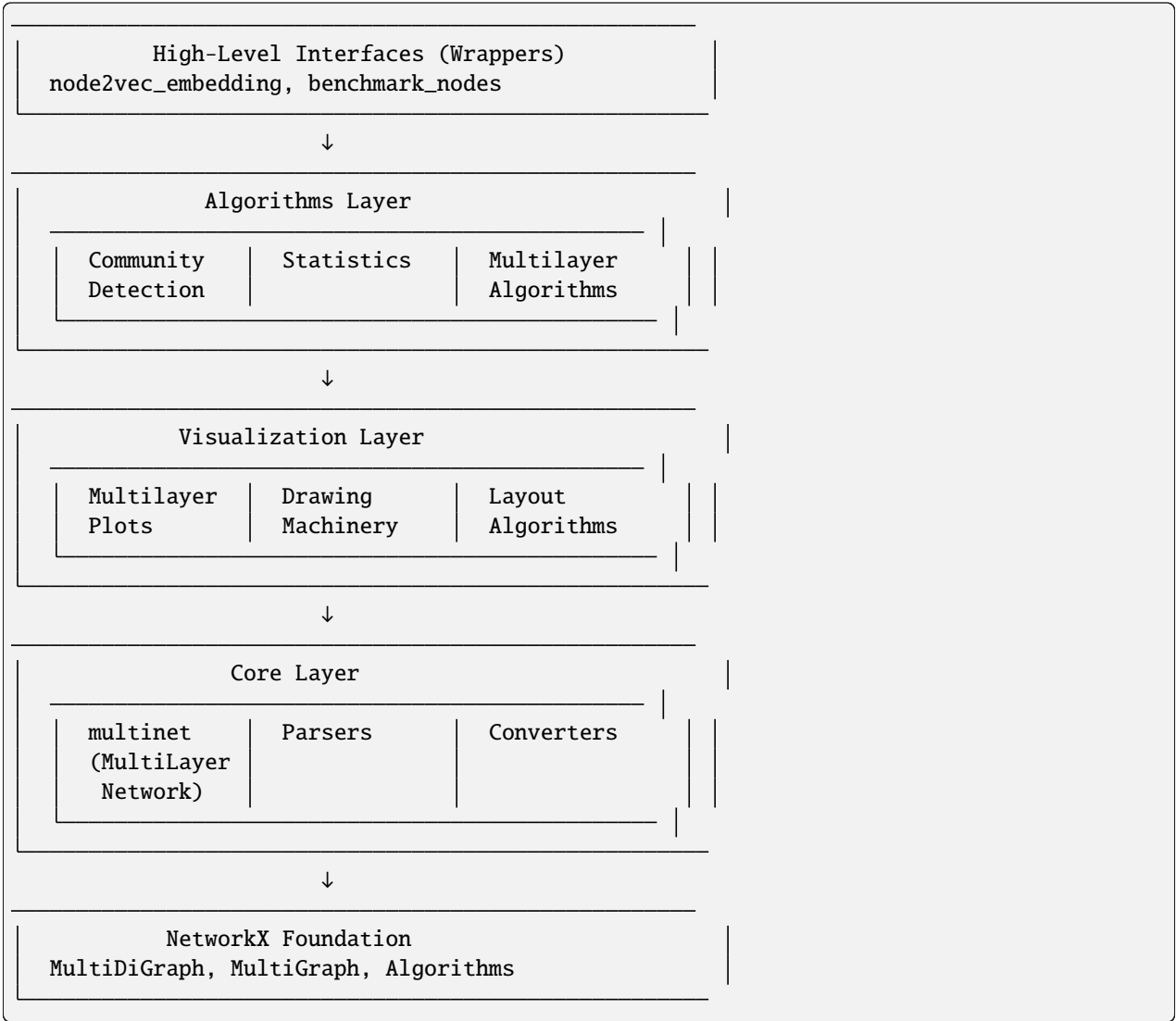
- **Repository:** <https://github.com/SkBlaz/py3plex>
- **Documentation:** <https://skblaz.github.io/py3plex/>
- **Issues:** <https://github.com/SkBlaz/py3plex/issues>
- **Examples:** <https://github.com/SkBlaz/Py3Plex/tree/master/examples>

6.51 Architecture and Design

This document describes the **system architecture**, **design patterns**, and **extension points** of py3plex.

6.51.1 System Overview

py3plex is built as a **modular, layered architecture** with clear separation of concerns:



6.51.2 Architectural Layers

Core Layer

Purpose: Fundamental data structures and I/O operations

Key Components:

- `multinet.py` - The `multi_layer_network` class
- `parsers.py` - **Input/output** for various formats
- `converters.py` - **Format conversion** utilities
- `random_generators.py` - **Random network** generators
- `HINMINE/` - **Heterogeneous network decomposition**

Responsibilities:

- **Network construction** and manipulation
- **File I/O** (GraphML, GML, GEXF, edge lists, etc.)
- **Layer management**
- **Matrix representations** (adjacency, supra-adjacency)
- **NetworkX integration**

Design Pattern: Facade Pattern - `multi_layer_network` provides a unified interface to complex NetworkX operations

Algorithms Layer

Purpose: Network analysis algorithms optimized for multilayer networks

Key Components:

- `community_detection/` - **Community detection** algorithms
- `statistics/` - **Network statistics** and metrics
- `multilayer_algorithms/` - **Multilayer-specific** algorithms
- `node_ranking/` - **Centrality** and ranking measures
- `general/` - **General-purpose** algorithms (random walks, etc.)

Responsibilities:

- **Community detection** (Louvain, Infomap, Label Propagation)
- **Statistical analysis** (17+ multilayer metrics)
- **Centrality computation** (degree, betweenness, PageRank, etc.)
- **Random walks** and embeddings
- **Network decomposition**

Design Pattern: Strategy Pattern - Different algorithms implement common interfaces

Visualization Layer

Purpose: Network plotting and rendering

Key Components:

- `multilayer.py` - High-level multilayer plotting
- `drawing_machinery.py` - Core drawing primitives
- `layout_algorithms.py` - Layout computation
- `colors.py` - Color scheme generators
- `fa2/` - ForceAtlas2 layout

Responsibilities:

- Diagonal projection plots
- Force-directed layouts
- Matrix visualizations
- Color mapping and legends
- Interactive plots (via Plotly)

Design Pattern: Template Method Pattern - Layout algorithms follow a common template

Wrappers Layer

Purpose: High-level interfaces for common workflows

Key Components:

- `node2vec_embedding.py` - Node2Vec embedding generation
- `benchmark_nodes.py` - Node classification benchmarking

Responsibilities:

- Simplified interfaces for complex workflows
- Integration with external tools
- Benchmarking and evaluation

Design Pattern: Facade Pattern - Simplify complex multi-step operations

6.51.3 Core Data Structure

The `multi_layer_network` Class

Central to py3plex, this class manages multilayer network state:

```
class multi_layer_network:
    def __init__(self, directed=True, label_delimiter="---", coupling_weight=1.0):
        self.core_network = nx.MultiDiGraph() if directed else nx.MultiGraph()
        self.layer_name_map = {} # Bidirectional mapping
        self.label_delimiter = label_delimiter
        self.coupling_weight = coupling_weight
        self.embedding = None
        self.labels = None
```

Key Attributes:

- `core_network` - Underlying NetworkX graph
- `layer_name_map` - Maps layer names to integer IDs
- `label_delimiter` - Separator for node-layer encoding (default: “—“)
- `coupling_weight` - Default weight for inter-layer edges
- `embedding` - Cached node embedding matrix
- `labels` - Node classification labels

Encoding Scheme:

Nodes are encoded as "{node_id}{delimiter}{layer_id}"

Example: Node ‘A’ in layer ‘social’ becomes "A---social"

This allows NetworkX to handle multilayer structure transparently.

6.51.4 Design Patterns

Facade Pattern

Used in: `multi_layer_network`, wrappers

Purpose: Provide simplified interface to complex subsystems

```
# Complex underlying operations hidden behind simple interface
network = multinet.multi_layer_network()
network.add_edges(edges, input_type='list') # Handles parsing, encoding, validation
network.basic_stats() # Aggregates multiple NetworkX calls
```

Strategy Pattern

Used in: Algorithms, layout computation

Purpose: Interchangeable algorithms following common interface

```
# Different community detection strategies
def detect_communities(network, method='louvain'):
    strategies = {
        'louvain': community_louvain.best_partition,
        'infomap': community_wrapper.infomap_communities,
        'label_prop': label_propagation.propagate
    }
    return strategies[method](network.core_network)
```

Template Method Pattern

Used in: Visualization, layout algorithms

Purpose: Define algorithm skeleton, allow customization in subclasses

```
class LayoutAlgorithm:
    def compute(self, graph):
        self.initialize(graph)
        self.iterate()
        return self.finalize()
```

(continues on next page)

(continued from previous page)

```
def initialize(self, graph):
    raise NotImplementedError

def iterate(self):
    raise NotImplementedError

def finalize(self):
    raise NotImplementedError
```

Dependency Injection

Used in: Configuration, algorithm parameters

Purpose: Inject dependencies rather than hard-coding

```
# Configuration injected rather than hard-coded
from py3plex.config import DEFAULT_COLORS, LAYOUT_PARAMS

def draw_network(network, colors=None, layout_params=None):
    colors = colors or DEFAULT_COLORS
    layout_params = layout_params or LAYOUT_PARAMS
    # Use injected configuration
```

6.51.5 Data Flow

Typical Workflow

1. **Input:** Load or create network

```
network = multinet.multi_layer_network()
network.load_network("data.graphml", input_type="graphml")
```

2. **Processing:** Apply algorithms

```
communities = community_louvain.best_partition(network.core_network)
centrality = calc.multilayer_degree_centrality(network)
```

3. **Analysis:** Compute statistics

```
density = mls.layer_density(network, 'layer1')
correlation = mls.inter_layer_degree_correlation(network, 'L1', 'L2')
```

4. **Visualization:** Render results

```
draw_multilayer_default([network], display=True)
```

5. **Output:** Export results

```
network.save_network("output.graphml", output_type="graphml")
```

State Management

Immutable Operations: Most algorithms don't modify the network

```
# These don't modify the network
centrality = calc.multilayer_degree_centrality(network)
communities = community_louvain.best_partition(network.core_network)
```

Mutable Operations: Some operations modify network state

```
# These modify the network
network.add_edges(new_edges, input_type='list')
network.aggregate_layers(['L1', 'L2'], 'combined')
```

6.51.6 Extension Points

Custom Algorithms

Add new algorithms by following existing patterns:

```
# py3plex/algorithms/my_module/my_algorithm.py
from py3plex.core.multinet import multi_layer_network

def my_centrality(network: multi_layer_network) -> dict:
    """
    Custom centrality measure.

    Parameters
    -----
    network : multi_layer_network
        Input network

    Returns
    -----
    dict
        Node centrality scores
    """
    G = network.core_network
    centrality = {}

    for node in G.nodes():
        # Implement custom logic
        centrality[node] = compute_score(node, G)

    return centrality
```

Custom Visualizations

Create custom plots using drawing machinery:

```
from py3plex.visualization import drawing_machinery as dm
import matplotlib.pyplot as plt

def my_custom_plot(network):
```

(continues on next page)

(continued from previous page)

```

"""Custom visualization."""
fig, ax = plt.subplots(figsize=(10, 8))

# Compute layout
pos = dm.compute_layout(network.core_network, 'force')

# Draw elements
dm.draw_nodes(ax, network.core_network, pos, node_size=50)
dm.draw_edges(ax, network.core_network, pos, edge_width=1)
dm.draw_labels(ax, pos, labels=network.get_node_labels())

plt.show()

```

Custom Parsers

Add support for new file formats:

```

# py3plex/core/parsers.py
def parse_my_format(input_file, **kwargs):
    """
    Parse custom file format.

    Parameters
    -----
    input_file : str
        Path to input file

    Returns
    -----
    multi_layer_network
        Parsed network
    """

    network = multi_layer_network()

    with open(input_file, 'r') as f:
        for line in f:
            # Parse line and add to network
            pass

    return network

```

6.51.7 Configuration System

Centralized Configuration

py3plex/config.py provides centralized configuration:

```

# Default color palettes (8 options including colorblind-safe)
DEFAULT_COLORS = 'Set1'
COLORBLIND_SAFE = 'colorblind'

# Visualization defaults

```

(continues on next page)

(continued from previous page)

```

DEFAULT_NODE_SIZE = 20
DEFAULT_EDGE_WIDTH = 1.0
DEFAULT_ALPHA = 0.7

# Layout parameters
LAYOUT_PARAMS = {
    'force': {'iterations': 500, 'optimal_distance': 1.0},
    'fa2': {'iterations': 1000, 'gravity': 1.0}
}

# Performance settings
SPARSE_THRESHOLD = 1000 # Use sparse matrices above this node count
MEMORY_WARNING_THRESHOLD = 10000 # Warn for large dense matrices

```

Usage:

```

from py3plex.config import DEFAULT_COLORS, LAYOUT_PARAMS

colors = DEFAULT_COLORS
iterations = LAYOUT_PARAMS['force']['iterations']

```

6.51.8 Testing Architecture

Test Organization

```

tests/
├── test_core_functionality.py      # Core data structure tests
├── test_multilayer_*.py           # Multilayer algorithm tests
├── test_random_walks.py           # Random walk tests
├── test_io_*.py                   # I/O and parsing tests
├── test_config_api.py             # Configuration tests
└── test_utils.py                  # Utility function tests

```

Test Patterns

Unit Tests: Test individual functions in isolation

```

def test_layer_density():
    network = create_test_network()
    density = mls.layer_density(network, 'layer1')
    assert 0 <= density <= 1

```

Integration Tests: Test workflows across modules

```

def test_community_detection_workflow():
    network = load_network("test_data.graphml")
    communities = community_louvain.best_partition(network.core_network)
    assert len(communities) > 0

```

Property-Based Tests: Test invariants

```
def test_centrality_normalization():
    network = create_random_network()
    centrality = calc.multilayer_degree_centrality(network)
    # Centrality values should be normalized
    assert all(0 <= v <= 1 for v in centrality.values())
```

6.51.9 Performance Considerations

Lazy Evaluation

Expensive operations are computed on-demand:

```
class multi_layer_network:
    @property
    def supra_adjacency(self):
        if self._supra_adj_cache is None:
            self._supra_adj_cache = self._compute_supra_adjacency()
        return self._supra_adj_cache
```

Sparse Matrices

Use sparse representations for large networks:

```
def get_supra_adjacency_matrix(self, sparse=True):
    if sparse or len(self.get_nodes()) > SPARSE_THRESHOLD:
        return scipy.sparse.csr_matrix(adj)
    return np.array(adj)
```

Vectorization

Prefer NumPy vectorized operations:

```
# Bad: Python loop
degrees = [sum(1 for _ in G.neighbors(node)) for node in nodes]

# Good: Vectorized
degrees = np.array(list(dict(G.degree()).values()))
```

6.51.10 Logging Infrastructure

Centralized Logging

py3plex/logging_config.py provides structured logging:

```
import logging
from py3plex.logging_config import get_logger

logger = get_logger(__name__)

logger.info("Processing network with %d nodes", num_nodes)
logger.warning("Large network detected, using sparse matrices")
logger.error("Invalid layer: %s", layer_name)
```

Log Levels

- **DEBUG:** Detailed diagnostic information
- **INFO:** General informational messages
- **WARNING:** Warning messages (e.g., performance concerns)
- **ERROR:** Error messages
- **CRITICAL:** Critical errors

6.51.11 Error Handling

Custom Exceptions

py3plex/exceptions.py defines domain-specific exceptions:

```
class NetworkError(Exception):
    """Base exception for network errors."""
    pass

class LayerNotFoundError(NetworkError):
    """Raised when layer doesn't exist."""
    pass

class InvalidFormatError(NetworkError):
    """Raised when file format is invalid."""
    pass
```

Usage:

```
from py3plex.exceptions import LayerNotFoundError

def get_layer(self, layer_name):
    if layer_name not in self.layer_name_map:
        raise LayerNotFoundError(f"Layer '{layer_name}' not found")
    return self.layer_name_map[layer_name]
```

6.51.12 Future Architecture

Planned Improvements

1. **Backend Registry:** Support for igraph, cugraph backends
2. **Streaming API:** Process networks larger than memory
3. **Distributed Computing:** Dask/Ray integration for large-scale analysis
4. **Plugin System:** Easy addition of third-party algorithms
5. **Type System:** Full type hints coverage (currently 65%)

6.51.13 See Also

- *Contributing to py3plex* - Contributing guidelines
- *Development Guide* - Development workflow
- *Network Construction* - Core API documentation

6.52 Citation and References

If you use py3plex in your research, please **cite our work**.

6.52.1 Primary Citation

Recommended citation for py3plex:

```
@Article{Škrlj2019,
  author   = {{\v{S}}krlj, Bla{\v{z}} and Kralj, Jan and Lavra{\v{c}}, Nada},
  title    = {Py3plex toolkit for visualization and analysis of multilayer networks},
  journal  = {Applied Network Science},
  year     = {2019},
  volume   = {4},
  number   = {1},
  pages    = {94},
  doi      = {10.1007/s41109-019-0203-7},
  url      = {https://doi.org/10.1007/s41109-019-0203-7}
}
```

Plain text:

Škrlj, B., Kralj, J., & Lavrač, N. (2019). Py3plex toolkit for visualization and analysis of multilayer networks. *Applied Network Science*, 4(1), 94. <https://doi.org/10.1007/s41109-019-0203-7>

6.52.2 Conference Paper

For the **initial algorithmic work**:

```
@InProceedings{Škrlj2019Conference,
  author    = {{\v{S}}krlj, Bla{\v{z}} and Kralj, Jan and Lavra{\v{c}}, Nada},
  editor    = {Aiello, Luca Maria and Cherifi, Chantal and Cherifi, Hocine
               and Lambiotte, Renaud and Li{\v{c}}o, Pietro and Rocha, Luis M.},
  title     = {Py3plex: A Library for Scalable Multilayer Network Analysis and
               ↪Visualization},
  booktitle = {Complex Networks and Their Applications VII},
  year      = {2019},
  publisher = {Springer International Publishing},
  address   = {Cham},
  pages     = {757--768},
  isbn      = {978-3-030-05411-3},
  doi       = {10.1007/978-3-030-05411-3_60}
}
```

6.52.3 Algorithm-Specific Citations

When using **specific algorithms**, please also cite the **original papers**:

Multilayer Modularity

```
@Article{Mucha2010,
  author = {Mucha, Peter J. and Richardson, Thomas and Macon, Kevin
            and Porter, Mason A. and Onnela, Jukka-Pekka},
  title  = {Community structure in time-dependent, multiscale, and multiplex networks},
```

(continues on next page)

(continued from previous page)

```
journal = {Science},
year    = {2010},
volume  = {328},
number  = {5980},
pages   = {876--878},
doi     = {10.1126/science.1184819}
}
```

Node2Vec

```
@InProceedings{Grover2016,
  author    = {Grover, Aditya and Leskovec, Jure},
  title     = {node2vec: Scalable Feature Learning for Networks},
  booktitle = {Proceedings of the 22nd ACM SIGKDD International Conference
               on Knowledge Discovery and Data Mining},
  year      = {2016},
  pages     = {855--864},
  doi       = {10.1145/2939672.2939754}
}
```

Louvain Algorithm

```
@Article{Blondel2008,
  author = {Blondel, Vincent D. and Guillaume, Jean-Loup and Lambiotte, Renaud
            and Lefebvre, Etienne},
  title  = {Fast unfolding of communities in large networks},
  journal = {Journal of Statistical Mechanics: Theory and Experiment},
  year   = {2008},
  volume = {2008},
  number = {10},
  pages  = {P10008},
  doi    = {10.1088/1742-5468/2008/10/P10008}
}
```

Infomap

```
@Article{Rosvall2008,
  author = {Rosvall, Martin and Bergstrom, Carl T.},
  title  = {Maps of random walks on complex networks reveal community structure},
  journal = {Proceedings of the National Academy of Sciences},
  year   = {2008},
  volume = {105},
  number = {4},
  pages  = {1118--1123},
  doi    = {10.1073/pnas.0706851105}
}
```


MultiXRank

```
@Article{Baptista2022,
  author = {Baptista, Anthony and Gonzalez, Aur{\`e}lien and Baudot, Ana{\`i}s},
  title = {Universal multilayer network exploration by random walk with restart},
  journal = {Communications Physics},
  year = {2022},
  volume = {5},
  number = {1},
  pages = {170},
  doi = {10.1038/s42005-022-00937-9}
}
```

DeepWalk

```
@InProceedings{Perozzi2014,
  author = {Perozzi, Bryan and Al-Rfou, Rami and Skiena, Steven},
  title = {DeepWalk: Online Learning of Social Representations},
  booktitle = {Proceedings of the 20th ACM SIGKDD International Conference
    on Knowledge Discovery and Data Mining},
  year = {2014},
  pages = {701--710},
  doi = {10.1145/2623330.2623732}
}
```

Multilayer Network Theory

Foundational papers on multilayer networks:

```
@Article{Kivela2014,
  author = {Kivel{\`a}, Mikko and Arenas, Alex and Barthelemy, Marc
    and Gleeson, James P. and Moreno, Yamir and Porter, Mason A.},
  title = {Multilayer networks},
  journal = {Journal of Complex Networks},
  year = {2014},
  volume = {2},
  number = {3},
  pages = {203--271},
  doi = {10.1093/comnet/cnu016}
}
```

```
@Article{Boccaletti2014,
  author = {Boccaletti, Stefano and Bianconi, Ginestra and Criado, Regino
    and del Genio, Charo I. and Gomez-Garde{\~n}es, Jes{\`u}s
    and Romance, Miguel and Sendi{\~n}a-Nadal, Irene and Wang, Zhen
    and Zanin, Massimiliano},
  title = {The structure and dynamics of multilayer networks},
  journal = {Physics Reports},
  year = {2014},
  volume = {544},
  number = {1},
  pages = {1--122},
}
```

(continues on next page)

(continued from previous page)

```
doi      = {10.1016/j.physrep.2014.07.001}
}
```

```
@Article{DeDomenico2013,
  author   = {De Domenico, Manlio and Sol{\`e}-Ribalta, Albert and Cozzo, Emanuele
              and Kivel{\`a}, Mikko and Moreno, Yamir and Porter, Mason A.
              and G{\`o}mez, Sergio and Arenas, Alex},
  title    = {Mathematical formulation of multilayer networks},
  journal  = {Physical Review X},
  year     = {2013},
  volume   = {3},
  number   = {4},
  pages    = {041022},
  doi      = {10.1103/PhysRevX.3.041022}
}
```

6.52.4 Complete Citation List

For algorithm citations, see the *Algorithm Selection Guide* which includes all major algorithms with their original references and DOIs.

6.52.5 Acknowledgments

Development and Contributors

py3plex is developed and maintained by:

- **Blaž Škrlj** (lead developer) - Jožef Stefan Institute, Ljubljana, Slovenia
- **Jan Kralj** (contributor) - Jožef Stefan Institute
- **Nada Lavrač** (contributor) - Jožef Stefan Institute

Funding and Support

This work has been **supported by**:

- **Jožef Stefan Institute**, Ljubljana, Slovenia
- **Slovenian Research Agency** (research program P2-0103)

External Libraries

py3plex builds upon **excellent open-source libraries**:

- **NetworkX** - Graph data structures and algorithms
- **NumPy** - Numerical computing
- **SciPy** - Scientific computing
- **Matplotlib** - Visualization
- **scikit-learn** - Machine learning utilities

Community Acknowledgments

We thank all contributors, users, and the broader network science community for their support, feedback, and contributions.

Special thanks to contributors who have submitted pull requests, reported issues, or provided feedback.

6.52.6 License Information

py3plex is released under the **MIT License**.

MIT License

Copyright (c) 2019–2025 Blaž Škrlj, Jan Kralj, Nada Lavrač

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

Note: Some bundled algorithms may have **different licenses**:

- Infomap community detection code: **AGPLv3**
- Louvain community detection: **BSD-3-Clause**

See LICENSE file in the repository for detailed license compatibility information.

6.52.7 Contact

For questions about citing py3plex or collaboration opportunities:

- **Email:** blaz.skrj@ijs.si
- **GitHub:** <https://github.com/SkBlaz/py3plex>
- **Issues:** <https://github.com/SkBlaz/py3plex/issues>

6.52.8 Related Work

Other tools and libraries for multilayer network analysis:

- **muxViz** - Multilayer network visualization (R)
- **pymnet** - Multilayer networks in Python
- **multinet** - R package for multilayer networks

- **graphistry** - GPU-accelerated graph visualization

For a comprehensive comparison and positioning of py3plex, see the original publication.

6.52.9 Usage in Publications

If you've used py3plex in your research and would like to be listed here, please:

1. Open an issue or pull request
2. Provide citation to your paper
3. Brief description of how py3plex was used

We maintain a list of publications using py3plex to showcase applications and build community.

6.52.10 See Also

- *Acknowledgements* - Full acknowledgments
- *Algorithm Selection Guide* - Complete algorithm citations with references
- [py3plex GitHub](#) - Source code and issues
- [Applied Network Science paper](#) - Primary publication

6.53 Acknowledgements

ForceAtlas2 cython implementation is based on the one provided at <https://github.com/bhargavchippada/forceatlas2>, developed by Bhargav Chippada. The code is included by the author's permission.

We also thank Thomas Aynaud for the permission to include the initial version of the Louvain algorithm. Other contributors to this project are Jan Kralj and Nika Erzen.

EXAMPLES

The best way to learn py3plex is through examples.

All examples are available in the [examples/](#) directory.

Key examples:

- `tutorial_10min.py` - Executable version of the 10-minute tutorial
- `example_random_walks.py` - Random walk primitives (Node2Vec, DeepWalk)
- `example_multilayer_visualization.py` - Network visualization
- `example_community_detection.py` - Community detection with Louvain and Infomap
- `example_network_decomposition.py` - Meta-path feature extraction
- `example_multilayer_statistics.py` - Computing multilayer metrics
- `example_n2v_embedding.py` - Node2Vec embeddings
- `example_label_propagation.py` - Semi-supervised learning

CITATION

If you use py3plex in your research, please cite:

```
@Article{Skrlj2019,  
  author={Škrlj, Blaž and Kralj, Jan and Lavrač, Nada},  
  title={Py3plex toolkit for visualization and analysis of multilayer networks},  
  journal={Applied Network Science},  
  year={2019},  
  volume={4},  
  number={1},  
  pages={94},  
  doi={10.1007/s41109-019-0203-7}  
}
```


INDICES AND TABLES

- `genindex`
- `modindex`
- `search`

PYTHON MODULE INDEX

p

py3plex.algorithms.community_detection.community_detection, ??
 py3plex.algorithms.community_detection.community_measures, ??
 py3plex.algorithms.community_detection.community_ranking, ??
 py3plex.algorithms.community_detection.multilayer_benchmark, ??
 py3plex.algorithms.community_detection.multilayer_modularity, ??
 py3plex.algorithms.community_detection.NoRC, ??
 py3plex.algorithms.general.benchmark_classification, ??
 py3plex.algorithms.general.walkers, ??
 py3plex.algorithms.hedwig, ??
 py3plex.algorithms.hedwig.core.converters, ??
 py3plex.algorithms.hedwig.core.example, ??
 py3plex.algorithms.hedwig.core.kb, ??
 py3plex.algorithms.hedwig.core.predicate, ??
 py3plex.algorithms.hedwig.core.rule, ??
 py3plex.algorithms.hedwig.core.settings, ??
 py3plex.algorithms.hedwig.learners.bottomup, ??
 py3plex.algorithms.hedwig.learners.learner, ??
 py3plex.algorithms.hedwig.learners.optimal, ??
 py3plex.algorithms.hedwig.stats.scorefunctions, ??
 py3plex.algorithms.hedwig.stats.validate, ??
 py3plex.algorithms.multicentrality, ??
 py3plex.algorithms.multilayer_algorithms.centralities, ??
 py3plex.algorithms.multilayer_algorithms.entanglement, ??
 py3plex.algorithms.multilayer_algorithms.multixrank, ??
 py3plex.algorithms.multilayer_algorithms.supra_matrix_function_centrality, ??
 py3plex.algorithms.network_classification, ??
 py3plex.algorithms.node_ranking, ??
 py3plex.algorithms.node_ranking.node_ranking, ??
 py3plex.algorithms.statistics.basic_statistics, ??
 py3plex.algorithms.statistics.bayesian_tests, ??
 py3plex.algorithms.statistics.correlation_networks, ??
 py3plex.algorithms.statistics.enrichment, ??
 py3plex.algorithms.statistics.multilayer_statistics, ??
 py3plex.algorithms.statistics.topology, ??
 py3plex.cli, ??
 py3plex.config, ??
 py3plex.core.converters, ??
 py3plex.core.HINMINE.dataStructures, ??
 py3plex.core.HINMINE.decomposition, ??
 py3plex.core.HINMINE.IO, ??
 py3plex.core.multinet, ??
 py3plex.core.nx_compat, ??
 py3plex.core.parsers, ??
 py3plex.core.random_generators, ??
 py3plex.core.supporting, ??
 py3plex.exceptions, ??
 py3plex.io.api, ??
 py3plex.logging_config, ??
 py3plex.multinet.aggregation, ??
 py3plex.profiling, ??
 py3plex.utils, ??
 py3plex.validation, ??
 py3plex.visualization.benchmark_visualizations, ??
 py3plex.visualization.bezier, ??
 py3plex.visualization.colors, ??
 py3plex.visualization.drawing_machinery, ??
 py3plex.visualization.embedding_visualization.embedding_visualization, ??
 py3plex.visualization.fa2_fa2util, ??
 py3plex.visualization.fa2_forceatlas2, ??

py3plex.visualization.layout_algorithms, ??
py3plex.visualization.multilayer, ??
py3plex.visualization.sankey, ??
py3plex.wrappers.benchmark_nodes, ??
py3plex.wrappers.train_node2vec_embedding, ??